目录

第一部分 数据库体系架构 1

第 1 章 DM 逻辑结构概述 1

1.1 数据库和实例 1
1.1.1 数据库 1
1.1.2 实例 1
1.2 DM 逻辑存储结构 1
1.2.1 表空间 2
1.2.2 页 3
1.2.3 簇 4
1.2.4 段 4

第 2 章 DM 物理存储结构 6

2.1 配置文件 6
2.1.1 DM 数据库服务配置 7
2.1.2 复制配置 37
2.2 控制文件 39
2.3 数据文件 39
2.4 重做日志文件 40
2.5 归档日志文件 40
2.6 逻辑日志文件 41
2.7 备份文件 41
2.8 跟踪日志文件 41
2.9 事件日志文件 42
2.10 数据重演文件 42

第 3 章 DM 内存结构 43

3.1 内存池 43
3.1.1 HEAP 43
3.1.2 VPOOL 43
3.2 缓冲区 44
3.2.1 数据缓冲区 44
3.2.2 日志缓冲区 45
3.2.3 字典缓冲区 45
3.2.4 SQL 缓冲区 45
3.3 排序区 46
3.4 哈希区 46
3.5 SSD 缓冲区 46

第 4 章 管理 DM 线程 47

4.1 监听线程 47
4.2 工作线程 47

4.3 IO 线程 47
4.4 调度线程 48
4.5 日志 FLUSH 线程 48
4.6 日志归档线程 48
4.7 日志重做线程 48
4.8 日志 APPLY 线程 49
4.9 定时器线程 49
4.10 逻辑日志归档线程 49
4.11 MAL 系统相关线程 49
4.12 其他线程 50
4.13 线程信息的查看 50
第 5 章 DM7 的升级 51
5.1 选择升级方法 51
5.2 升级前准备工作 51
5.3 使用数据迁移工具 52
5.4 使用数据导入导出工具 52
5.5 升级后期工作 52
第二部分 基础数据库管理 53
第 6 章 DM 系统管理员 53
6.1 DM 系统管理员的类型 53
6.2 数据库管理员的任务 54
6.3 数据库安全员的任务 55
6.4 数据库审计员的任务 55
第 7 章 创建和配置 DM 数据库 56
7.1 创建 DM 数据库 56
7.2 使用数据库配置工具创建数据库 56
7.2.1 启动数据库配置工具 56
7.2.2 使用数据库配置工具创建数据库 57
7.3 使用 dminit 创建数据库 64
7.4 注册数据库服务 68
7.5 查看数据库信息 70
7.6 删除数据库 70
7.7 删除数据库服务 73
7.8 数据库服务配置脚本 76
7.8.1 服务安装脚本文件 76
7.8.2 服务卸载脚本文件 76
第 8 章 启动和关闭数据库 78
8.1 启动数据库 78
8.1.1 Windows 系统 78
8.1.2 Linux 系统 80
8.1.3 检查 LICENSE 80
8.2 数据库状态和模式 80
8.3 关闭数据库 81

8.3.1 Windows 系统 81
8.3.2 Linux 系统 82
第 9 章 管理模式对象的空间 84
9.1 设置存储参数 84
9.1.1 普通表和索引 84
9.1.2 堆表 85
9.1.3 HFS 表 85
9.2 收回多余的空间 85
9.3 用户和表上的空间限制 86
9.3.1 用户的空间限制 86
9.3.2 表对象的空间限制 86
9.4 查看模式对象的空间使用 86
9.4.1 查看用户占用的空间 86
9.4.2 查看表占用的空间 86
9.4.3 查看表使用的页数 86
9.4.4 查看索引占用的空间 87
9.4.5 查看索引使用的页数 87
9.5 数据类型的空间使用 87
第 10 章 管理表 89
10.1 管理表的准则 89
10.1.1 设计表 89
10.1.2 指定表的存储空间上限 90
10.1.3 指定表的存储位置 90
10.2 创建表 90
10.2.1 创建普通表 90
10.2.2 指定表的聚集索引 91
10.2.3 指定表的填充因子 91
10.2.4 查询建表 91
10.2.5 创建临时表 92
10.3 更改表 93
10.4 删除表 93
10.5 清空表 93
10.5.1 使用 DELETE 93
10.5.2 使用 DROP 和 CREATE 94
10.5.3 使用 TRUNCATE 94
10.6 查看表信息 94
10.6.1 查看表定义 94
10.6.2 查看自增列信息 94
10.6.3 查看表的空间使用情况 95
第 11 章 管理索引 96
11.1 管理索引的准则 96
11.1.1 在表中插入数据后创建索引 96
11.1.2 索引正确的表和列 96
11.1.3 为性能而安排索引列 96
11.1.4 限制每个表的索引的数量 97

11.1.5 估计索引大小和设置存储参数 97
11.1.6 为每个索引指定表空间 97
11.2 创建索引 97
11.2.1 明确地创建索引 97
11.2.2 创建聚集索引 98
11.2.3 明确地创建唯一索引 98
11.2.4 自动创建与约束相关的唯一索引 99
11.2.5 创建基于函数的索引 99
11.2.6 创建位图索引 100
11.2.7 创建位图连接索引 100
11.3 重建索引 101
11.4 删除索引 101
11.5 查看索引信息 102
第 12 章 PL/SQL 103
12.1 PL/SQL 概述 103
12.1.1 PL/SQL 优点 103
12.1.2 PL/SQL 语句块 103
12.2 PL/SQL 变量 104
12.3 PL/SQL 数据类型和操作符 105
12.3.1 SQL 数据类型 105
12.3.2 %TYPE 和%ROWTYPE 106
12.3.3 记录类型 107
12.3.4 数组类型 108
12.3.5 操作符 109
12.4 PL/SQL 流控制 110
12.4.1 IF 语句 110
12.4.2 LOOP 语句 111
12.4.3 WHILE 语句 111
12.4.4 FOR 语句 112
12.4.5 EXIT 语句 112
12.4.6 GOTO 语句 112
12.4.7 NULL 语句 113
12.5 PL/SQL 访问数据库 113
12.5.1 数据查询 113
12.5.2 数据操纵 114
12.5.3 数据定义 115
12.6 PL/SQL 与数据库交互 115
12.6.1 存储过程 115
12.6.2 存储函数 116
12.6.3 游标 117
12.6.4 引用游标 119
12.6.5 动态 SQL 120
12.7 自治事务 122
12.7.1 定义自治事务 122
12.7.2 自治事务完整性与死锁检测 123
12.7.3 自治事务嵌套 123
12.8 异常处理 123

12.8.1 异常处理程序 123
12.8.2 预定义的异常 124
12.8.3 用户自定义的异常 125
12.9 PL/SQL 调试工具 125
12.9.1 命令行参数 126
12.9.2 功能描述 126
12.10 小结 132

第 13 章 管理触发器 134

13.1 触发器的使用 134
13.2 表级触发器 136
13.3 事件触发器 136
13.4 时间触发器 137
13.5 触发器总结 138

第 14 章 管理视图、序列和同义词 139

14.1 管理视图 139
14.2 管理序列 139
14.3 管理同义词 140
14.4 查看视图、序列和同义词信息 140

第 15 章 模式对象的常规管理 141

15.1 在单个操作中创建多个模式对象 141
15.2 重命名模式对象 142
15.3 启用和停用触发器 142
15.4 管理完整性约束 143
15.4.1 完整性约束状态 143
15.4.2 定义完整性约束 143
15.4.3 修改或删除现有的完整性约束 143
15.4.4 查看约束信息 144
15.5 管理对象依赖性 144
15.6 管理对象名称解析 144
15.7 显示有关模式对象的信息 145

第三部分 高级数据库管理 146

第 16 章 数据库布局和存储管理 146

16.1 管理表空间 146
16.1.1 创建表空间 146
16.1.2 扩展表空间 146
16.1.3 删除表空间 146
16.1.4 修改表空间名 146
16.1.5 修改表空间状态 147
16.1.6 修改表空间数据缓冲区 147
16.1.7 查询表空间与数据文件对应关系 147
16.1.8 表空间文件失效检查 147
16.1.9 表空间失效文件恢复 148
16.2 管理数据文件 148

16.2.1 添加数据文件 148
16.2.2 扩展数据文件的大小 148
16.2.3 指定数据文件的扩展属性 148
16.2.4 修改数据文件的路径 149
16.3 管理重做日志文件 149
16.3.1 添加重做日志文件 149
16.3.2 扩展重做日志文件 149
16.4 管理回滚空间 149
16.5 管理控制文件 150

第 17 章 管理分区表和分区索引 151

17.1 分区的概念 151
17.2 分区的方法 152
17.3 创建水平分区表 152
17.3.1 创建范围分区表 152
17.3.2 创建 LIST 分区表 154
17.3.3 创建哈希分区表 154
17.3.4 创建组合分区表 155
17.4 在水平分区表建立索引 156
17.5 维护水平分区表 157
17.5.1 增加分区 157
17.5.2 删除分区 157
17.5.3 交换分区 158
17.5.4 合并分区 158
17.5.5 拆分分区 159
17.6 水平分区表的限制 159
17.7 创建垂直分区表 160
17.8 垂直分区表的限制 161

第 18 章 管理列存储表 163

18.1 什么是列存储 163
18.2 什么是 HFS 表 164
18.3 创建 HFS 表 166
18.4 维护 HFS 表 168
18.5 查看有关 HFS 表的信息 168

第 19 章 管理堆表 169

19.1 什么是堆表 169
19.2 创建堆表 169
19.3 堆表的限制 170
19.4 维护堆表 170
19.5 查看有关堆表的信息 171

第 20 章 全文检索 172

20.1 全文检索概述 172
20.2 创建全文索引 174
20.3 更新全文索引 174
20.4 执行全文检索 174

20.5 删除全文索引 175
第 21 章 管理事务 176
21.1 事务简介 176
21.2 事务特性 177
21.2.1 原子性 177
21.2.2 一致性 177
21.2.3 隔离性 177
21.2.4 持久性 177
21.3 提交事务 178
21.3.1 自动提交模式 178
21.3.2 手动提交模式 178
21.3.3 隐式提交 179
21.4 回滚事务 179
21.4.1 自动回滚 179
21.4.2 手动回滚 179
21.4.3 回滚到保存点 179
21.4.4 语句级回滚 180
21.5 事务锁定 180
21.5.1 锁模式 180
21.5.2 锁粒度 181
21.5.3 查看锁 182
21.6 多版本 183
21.6.1 物理记录格式 183
21.6.2 回滚记录格式 183
21.6.3 可见性原则 183
21.6.4 历史数据获取 184
21.6.5 回滚段自动清理 184
21.7 事务隔离级 184
21.7.1 读提交隔离级 185
21.7.2 串行化隔离级 186
21.7.3 读未提交隔离级 186
21.7.4 只读事务 186
21.8 锁等待与死锁检测 186
21.9 闪回 187
第四部分 故障排除和性能优化 188
第 22 章 问题跟踪和解决 188
22.1 问题分析 188
22.2 监控系统性能(V$) 189
22.3 数据库重演(REPLAY) 189
22.4 检查数据物理一致性 190
22.5 调整配置参数 190
22.6 优化数据库布局 191
第 23 章 动态管理/性能视图 192
23.1 理解动态管理视图 192

23.2 使用动态管理视图 192
第 24 章 查询优化 196
24.1 优化目标 196
24.2 查询优化器 196
24.2.1 查询转换 196
24.2.2 估算代价 196
24.2.3 生成计划 197
24.3 数据访问路径 197
24.4 连接 198
24.5 统计信息 199
24.6 执行计划 200
24.7 使用索引 201
24.8 并行查询 201
24.8.1 并行查询概念 201
24.8.2 确定并行任务个数 201
24.8.3 确定并行工作线程数 202
24.8.4 执行查询 203
24.8.5 使用场景 203
24.9 查询计划重用 203
24.10 结果集重用 204
第 25 章 SQL 调优 205
25.1 简介 205
25.2 调优目标 205
25.3 确定高负载的 SQL 205
25.4 自动 SQL 调整 206
25.5 开发有效的 SQL 语句 206
25.6 使用优化器提示 208
25.6.1 索引提示 210
25.6.2 连接方法提示 211
25.6.3 连接顺序提示 214
25.6.4 统计信息提示 214
第五部分 数据高可用性 216
第 26 章 故障恢复 216
26.1 概述 216
26.2 REDO 日志 216
26.3 重做日志归档 217
26.3.1 本地归档 217
26.3.2 实时归档 217
26.3.3 即时归档 217
26.3.4 异步归档 217
26.3.5 MPP 归档 217
26.4 检查点 218
26.5 回滚段与回滚记录 218
26.6 系统故障恢复 218

26.6.1 恢复流程 218
26.6.2 并行恢复 219
26.7 介质故障恢复 219
第 27 章 备份与还原 220
27.1 概述 220
27.2 备份 220
27.2.1 基本概念 220
27.2.2 物理备份 221
27.2.3 B 树备份 224
27.2.4 备份加密与压缩 225
27.3 还原 226
27.3.1 基本概念 226
27.3.2 还原数据库 226
27.3.3 还原表空间 231
27.3.4 还原用户表 232
第 28 章 数据复制 233
28.1 概述 233
28.2 重要概念 233
28.3 体系构架 234
28.4 配置数据复制 235
28.5 监控数据复制 237
28.5.1 复制故障监控 237
28.5.2 复制故障处理 238
28.6 复制用户和系统表 239
第 29 章 大规模并行处理 MPP 242
29.1 概述 242
29.2 基本概念 242
29.3 体系架构 242
29.4 支持的功能 244
29.4.1 支持多种分布 244
29.4.2 函数和过程 244
29.4.3 支持触发器 244
29.4.4 支持分区表 244
29.4.5 自增列和序列 244
29.4.6 支持大字段 244
29.4.7 支持列存储 244
29.4.8 支持事务的 ACID 特性 245
29.5 数据加载 245
29.6 环境配置 245
29.6.1 普通配置 245
29.6.2 基于主备的配置 246
29.6.3 监控器使用说明 251
29.7 并行查询计划 256
29.8 MPP 常见用法 257

第六部分 附录 258

附录 1 数据字典 258

附录 2 动态性能视图 268

附录 3 ORACLE 兼容视图 320

附录 4 执行计划操作符 353

附录 5 数据复制的系统表 356

附录 6 DM 技术支持 359

第一部分 数据库体系架构

第1章 DM 逻辑结构概述

1.1 数据库和实例

在 DM7 之前版本的 DM 数据库中，“数据库”和“实例”这两个术语经常可以互相替换，意义也很相近。在新版本 DM7 数据库中，“数据库”和“实例”这两个概念之间有着很大的差别，甚至可以说它们是两个完全不同的实体。

1.1.1 数据库

在有些情况下，数据库的概念包含的内容会很广泛。如在单独提到 DM 数据库时，可能指的是 DM 数据库产品，也有可能是正在运行的 DM 数据库实例，还可能是 DM 数据库运行中所需的一系列物理文件的集合等。但是，当同时出现 DM 数据库和实例时，DM 数据库指的是磁盘上存放在 DM 数据库中的数据的集合，一般包括：数据文件、日志文件、控制文件以及临时数据文件等。

1.1.2 实例

实例一般是由一组正在运行的 DM 后台进程/线程以及一个大型的共享内存组成。简单来说，实例就是操作 DM 数据库的一种手段，是用来访问数据库的内存结构以及后台进程的集合。

DM 数据库存储在服务器的磁盘上，而 DM 实例则存储于服务器的内存中。通过运行 DM 实例，可以操作 DM 数据库中的内容。在任何时候，一个实例只能与一个数据库进行关联（装载、打开或者挂起数据库）。在大多数情况下，一个数据库也只有一个实例对其进行操作。但是在即将提供的 DM 共享磁盘高性能集群中，多个实例可以同时装载并打开一个数据库（位于一组由多台服务器共享的物理磁盘上）。此时，我们可以同时从多台不同的计算机访问这个数据库。

1.2 DM 逻辑存储结构

DM 数据库为数据库中的所有对象分配逻辑空间，并存放在数据文件中。在 DM 数据库内部，所有的数据文件组合在一起被划分到一个或者多个表空间中，所有的数据库内部对象都存放在这些表空间中。同时，表空间被进一步划分为段、簇和页（也称块）。通过这种细分，可以使得 DM 数据库能够更加高效地控制磁盘空间的利用率。图 1.1 显示了这些数据结构之间的关系。

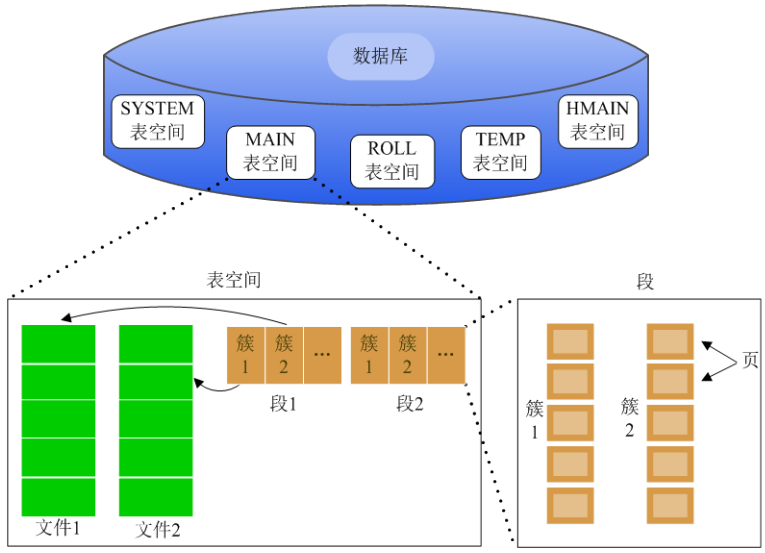


图 1.1 显示表空间、数据文件、段、簇、页的关系

可以看出，在 DM7 中存储的层次结构如下：

1. 数据库由一个或多个表空间组成；
2. 每个表空间由一个或多个数据文件组成；
3. 每个数据文件由一个或多个簇组成；
4. 段是簇的上级逻辑单元，一个段可以跨多个数据文件；
5. 簇由磁盘上连续的页组成，一个簇总是在一个数据文件中；
6. 页是数据库中最小的分配单元，也是数据库中使用的最小的 IO 单元。

1.2.1 表空间

在 DM 数据库中，表空间由一个或者多个数据文件组成。DM 数据库中的所有对象在逻辑上都存放在表空间中，而物理上都存储在所属表空间的数据文件中。

在创建 DM 数据库时，会自动创建 5 个表空间：SYSTEM 表空间、ROLL 表空间、MAIN 表空间、TEMP 表空间和 HMAIN 表空间。

1. SYSTEM 表空间存放了有关 DM 数据库的字典信息，用户不能在 SYSTEM 表空间创建表和索引。
2. ROLL 表空间完全由 DM 数据库自动维护，用户无需干预。该表空间用来存放事务运行过程中执行 DML 操作之前的值，从而为访问该表的其他用户提供表数据的读一致性视图。
3. MAIN 表空间在初始化库的时候，就会自动创建一个大小为 128M 的数据文件 MAIN.DBF。在创建用户时，如果没有指定默认表空间，则系统自动指定 MAIN 表空间为用户默认的表空间。
4. TEMP 表空间完全由 DM 数据库自动维护。当用户的 SQL 语句需要磁盘空间来完成某个操作时，DM 数据库会从 TEMP 表空间分配临时段。如创建索引、无法在内存中完成的排序操作、SQL 语句中间结果集以及用户创建的临时表等都会使用到 TEMP 表空间。
5. HMAIN 表空间属于 HTS 表空间，完全由 DM 数据库自动维护，用户无需干涉。当用户在创建 HFS 表时，未指定 HTS 表空间的情况下，充当默认 HTS 表空间。

每一个用户都有一个默认的表空间。对于 SYS、SYSSSO、SYSAUDITOR 系统用户，默认的用户表空间是 SYSTEM，SYSDBA 的默认表空间为 MAIN，新创建的用户如果没有指定默认表空间，则系统自动指定 MAIN 表空间为用户默认的表空间。如果用户在创建表的时候，

指定了存储表空间 A，并且和当前用户的默认表空间 B 不一致时，表存储在用户指定的表空间 A 中，并且默认情况下，在这张表上面建立的索引也将存储在 A 中，但是用户的默认表空间是不变的，仍为 B。

一般情况下，建议用户自己创建一个表空间来存放业务数据，或者将数据存放在默认的用户表空间 MAIN 中。

用户可以通过执行如下语句来查看表空间相关信息。

SYSTEM、ROLL、MAIN 和 TEMP 表空间查看语句：

```
SELECT * FROM V$TABLESPACE;
```

HMAIN 表空间查看语句：

```
SELECT * FROM V$HUGE_TABLESPACE;
```

结果集中各字段的含义请参考附录部分动态性能视图章节中对 V$TABLESPACE 的介绍，关于表空间该如何使用，请参考第 16 章。

1.2.2 页

数据页（也称数据块）是 DM 数据库中最小的数据存储单元。页的大小对应物理存储空间上特定数量的存储字节，在 DM 数据库中，页大小可以为 4KB、8KB、16KB 或者 32KB，用户在创建数据库时可以指定，默认大小为 8KB，一旦创建好了数据库，则在该库的整个生命周期内，页大小都不能够改变。图 1.2 显示了 DM 数据库页的典型格式。



图 1.2 DM 数据页的组成

页头控制信息包含了关于页类型、页地址等信息。页的中部存放数据，为了更好地利用数据页，在数据页的尾部专门留出一部分空间用于存放行偏移数组，行偏移数组用于标识页上的空间占用情况以便管理数据页自身的空间。

在绝大多数情况下，用户都无需干预 DM 数据库对数据页的管理。但是 DM 数据库还是提供了选项供用户选择，使得在某些情况下能够为用户提供更佳的数据处理性能。

FILLFACTOR 是 DM 数据库提供的一个与性能有关的数据页级存储参数，它指定一个数据页初始化后插入数据时最大可以使用空间的百分比（100），该值在创建表/索引时可以指定。设置 FILLFACTOR 参数的值，是为了指定数据页中的可用空间百分比（FILLFACTOR）和可扩展空间百分比（100-FILLFACTOR）。可用空间用来执行更多的 INSERT 操作，可扩展空间用来为数据页保留一定的空间，以防止在今后的更新操作中增加列或者修改变长列的长度时，引起数据页的频繁分裂。当插入的数据占据的数据页空间百分比低于 FILLFACTOR 时，允许数据插入该页，否则将当前数据页中的数据分为两部分，一部分保留在当前数据页中，另一部分存入一个新页中。

对于 DBA 来说，使用 FILLFACTOR 时应该在空间和性能之间进行权衡。为了充分利用空间，用户可以设置一个很高的 FILLFACTOR 值，如 100，但是这可能会导致在后续更新数据时，频繁引起页分裂，而导致需要大量的 I/O 操作。为了提高更新数据的性能，可以设置一个相对较低（但不是过低）的 FILLFACTOR 值，使得后续执行更新操作时，可以尽量

避免数据页的分裂，提升 I/O 性能，不过这是以牺牲空间利用率来换取性能的提高。

1.2.3 簇

簇是数据页的上级逻辑单元，由同一个数据文件中 16 个或 32 个连续的数据页组成。在 DM 数据库中，簇的大小由用户在创建数据库时指定，默认大小为 16。假定某个数据文件大小为 32MB，页大小为 8KB，则共有 32MB/8KB/16=256 个簇，每个簇的大小为 8K*16=128K。和数据页的大小一样，一旦创建好数据库，此后该数据库的簇的大小就不能够改变。

1． 分配数据簇

当创建一个表/索引的时候，DM 为表/索引的数据段分配至少一个 簇，同时数据库会自动生成对应数量的空闲数据页，供后续操作使用。如果初始分配的 簇 中所有数据页都已经用完，或者新插入/更新数据需要更多的空间，DM 数据库将自动分配新的 簇 。在缺省情况下，DM 数据库在创建表/索引时，初始分配 1 个簇，当初始分配的空间用完时，DM 数据库会自动扩展。

当 DM 数据库的表空间为新的簇分配空闲空间时，首先在表空间按文件从小到大的顺序在各个数据文件中查找可用的空闲簇，找到后进行分配；如果各数据文件都没有空闲簇，则在各数据文件中查找空闲空间足够的，将需要的空间先进行格式化，然后进行分配；如果各文件的空闲空间也不够，则选一个数据文件进行扩充。

2． 释放数据簇

对于用户数据表空间，在用户将一个数据段对应的表/索引对象 DROP 之前，该表对应的数据段会保留至少 1 个簇不被回收到表空间中。在删除表/索引对象中的记录的时候，DM 数据库通过修改数据文件中的位图来释放簇，释放后的簇被视为空闲簇，可以供其他对象使用。当用户删除了表中所有记录时，DM 数据库仍然会为该表保留 1-2 个簇供后续使用。若用户使用 DROP 语句来删除表/索引对象,则此表/索引对应的段以及段中包含的簇全部收回，并供存储于此表空间的其他模式对象使用。

对于临时表空间，DM 数据库会自动释放在执行 SQL 过程中产生的临时段，并将属于此临时段的簇空间还给临时表空间。需要注意的是，临时表空间文件在磁盘所占大小并不会因此而缩减，用户可以通过系统函数 SF_RESET_TEMP_TS 来进行磁盘空间的清理。

对于回滚表空间，DM 数据库将定期检查回滚段，并确定是否需要从回滚段中释放一个或多个簇。

1.2.4 段

段是簇的上级逻辑分区单元，它由一组簇组成。在同一个表空间中，段可以包含来自不同文件的簇，即一个段可以跨越不同的文件 。而一个簇以及该簇所包含的数据页则只能来自一个文件，是连续的 16或者 32 个数据页。由于簇的数量是按需分配的，数据段中的不同簇在磁盘上不一定连续。

1． 数据段

段可以被定义成特定对象的数据结构，如表数据段或索引数据段。表中的数据以表数据段结构存储，索引中的数据以索引数据段结构存储。DM 以簇为单位给每个数据段分配空间，当数据段的簇空间用完时，DM 数据库就给该段重新分配簇，段的分配和释放完全由 DM 数据库自动完成，可以在创建表/索引时设置存储参数来决定数据段的簇如何分配。

当用户使用 CREATE 语句创建表/索引时，DM 创建相应的数据段。表/索引的存储参数用来决定对应数据段的簇如何被分配，这些参数将会影响与对象相关的数据段的存储与访问

效率。对于分区表，每个分区使用单独的数据段来容纳所有数据，对于分区表上的非分区索引，使用一个索引数据段来容纳所有数据，而对于分区索引，每个分区使用一个单独索引数据段来容纳其数据。表的数据段和与其相关的索引段不一定要存储在同一表空间中，用户可以在创建表和索引时，指定不同的表空间存储参数。

2． 临时段

在 DM 数据库中，所有的临时段都创建在临时表空间中，这样可以分流磁盘设备的 I/O，也可以减少由于在 SYSTEM 或其他表空间内频繁创建临时数据段而造成的碎片。

当处理一个查询时，经常需要为 SQL 语句的解析与执行的中间结果准备临时空间。DM 数据库会自动地分配临时段的磁盘空间。例如，DM 在进行排序操作时就可能需要使用临时段，当排序操作可以在内存中执行，或设法利用索引就可以执行时，就不必创建临时段。对于临时表及其索引，DM 数据库也会为它们分配临时段。

临时段的分配和释放完全由系统自动控制，用户不能手工进行干预。

3． 回滚段

DM 数据库在回滚表空间的回滚段中保存了用于恢复数据库操作的信息。对于未提交事务，当执行回滚语句时，回滚记录被用来做回滚变更。在数据库恢复阶段，回滚记录被用来做任何未提交变更的回滚。在多个并发事务运行期间，回滚段还为用户提供读一致性，所有正在读取受影响行的用户将不会看到行中的任何变动，直到他们事务提交后发出新的查询。

DM 数据库提供了全自动回滚管理机制来管理回滚信息和回滚空间，自动回滚管理消除了管理回滚段的复杂性。此外，系统将尽可能保存回滚信息，来满足用户查询回滚信息的需要。事务被提交后，回滚数据不能再回滚或者恢复，但是从数据读一致性的角度出发，长时间运行查询可能需要这些早期的回滚信息来生成早期的数据页镜像，基于此，数据库需要尽可能长时间的保存回滚信息。DM 数据库会收集回滚信息的使用情况，并根据统计结果对回滚信息保存周期进行调整，数据库将回滚信息保存周期设为比系统中活动的最长的查询时间稍长。

第2章 DM 物理存储结构

DM 数据库使用了磁盘上大量的物理存储结构来保存和管理用户数据。典型的物理存储结构包括：用于进行功能设置的配置文件；用于记录文件分布的控制文件；用于保存用户实际数据的数据文件、重做日志文件、归档日志文件、备份文件；用来进行问题跟踪的跟踪日志文件等，如图 2.1 所示。

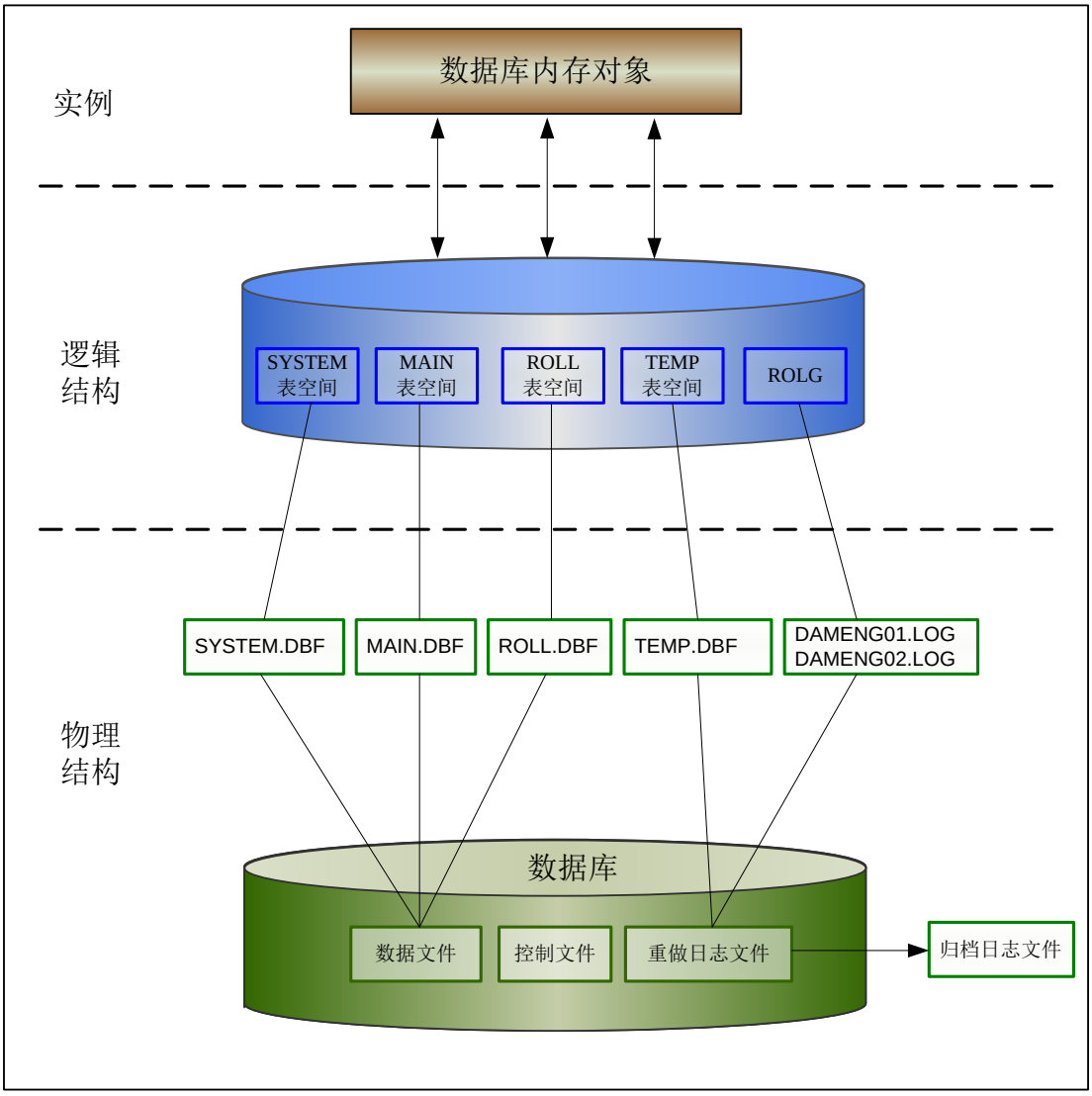


图 2.1 DM 物理存储结构示意图

2.1 配置文件

配置文件是 DM 数据库用来设置功能选项的一些文本文件的集合，配置文件以 ini 为扩展名，它们具有固定的格式，用户可以通过修改其中的某些参数取值来达成如下两个方面的目标：

1. 启用/禁用特定功能项；
2. 针对当前系统运行环境设置更优的参数值以提升系统性能。

2.1.1 DM 数据库服务配置

2.1.1.1 dm.ini

每创建一个 DM 数据库，就会自动生成 dm.ini。dm.ini 是 DM 数据库启动所必须的配置文件，通过配置该文件可以设置 DM 数据库服务器的各种功能和性能选项，主要的配置内容见表 2.1。

表中参数属性的“静态”指修改后重启服务器才能生效；“动态”指修改后即时生效；“手动”表示服务器运行过程中不可修改。动态参数又分为会话级和系统级两个级别，会话级参数在服务器运行过程中被修改时，之前创建的会话不受影响，只有新创建的会话使用新的参数值；系统级参数的修改则会影响所有的会话。

这里所说的动态修改 INI 参数指 DBA 用户在数据库服务器运行期间，通过调用系统过程 SP_SET_PARA_VALUE()、SP_SET_PARA_DOUBLE_VALUE()和 SP_SET_PARA_STRING_VALUE()对参数值进行修改。

表 2.1 dm.ini 配置项

参数名	缺省值	属性	说明
控制文件相关参数（注意：本类参数不建议用户修改）			
CTL_PATH	安装时指定	手动	控制文件路径
CTL_BAK_PATH	安装时指定	手动	控制文件备份路径，初始化库时指定为“SYSTEM_PATH/CTL_BAK”路径，如果没有配置，则在控制文件路径下创建 CTL_BAK 目录，用于存放备份文件。 备份文件命名格式： “DM_年月日时分秒_毫秒.CTL” 备份文件在初始化库和每次修改 DM.CTL 控制文件后生成
CTL_BAK_NUM	10	手动	控制文件备份个数限制，取值 1~100，在此限制之外，会再多保留一个备份文件，在生成新的备份文件时，如果当前已存在的备份文件个数大于指定值，则自动删除创建时间最早的备份文件，小于或等于指定值的情况下，不会有删除操作，缺省值为 10
SYSTEM_PATH	安装时指定	手动	系统库目录
CONFIG_PATH	安装时指定	手动	指定 DMSERVER 所读取的配置文件（DMMAL.INI, DMARCH.INI, DMTIMER.INI 等）的路径。缺省使用 SYSTEM_PATH 路径
TEMP_PATH	安装时指定	手动	临时库文件路径
BAK_PATH	安装时指定	手动	备份路径
BAK_POLICY	0	手动	备份策略。取值 0、1 或 2。为 0 表示同时支持 BAK1 和 BAK2 版本；为 1 时只能使用 BAK1 版本；为 2 表示只支持 BAK2 版本。BAK1 为备份还原的老版本，BAK2 为备份还原的新版本
实例名			
INSTANCE_NAME	DMSERVER	手动	实例名（一般情况下，长度不超过 128 个字符；

			但是在守护进程中，长度不超过 16 个字符）
内存相关参数			
MAX_OS_MEMORY	90	静态	DM 服务器能使用的最大内存占操作系统物理内存与虚拟内存总和的百分比，有效值范围（40~100）。当取值 100 时，服务器不进行内存的检查 注：对于 32 位版本的 DM 服务器，虚拟内存最大为 2G
MEMORY_POOL	80	静态	系统内存池大小，以兆为单位。系统内存池是由DM管理的内存，系统内存的申请都从内存池中申请。有效值范围：32位平台为（5~2000），64位平台为（5~67108864）
MEMORY_BAK_POOL	4	静态	系统备份内存池大小，以兆为单位。系统备份内存池是由 DM 管理的内存。有效值范围（2~10000）
HUGE_MEMORY_THRESHOLD	0	静态	设置超过多大的常规内存分配优先从 HUGE_BUFFER 走，以 KB 为单位。有效值 0~1M。0 表示不从 HUGE_BUFFER 分配
HUGE_MEMORY_PERCENTAGE	50	静态	指示 HUGE_BUFFER 中可以借用作常规内存分配的空间百分比，有效值为 0~100。
HUGE_BUFFER	8	静态	HFS 表使用的缓冲区大小，以兆为单位。有效值范围（8~1048576）
BUFFER	100	静态	系统缓冲区大小，以兆为单位。推荐值：系统缓冲区大小为可用物理内存的 60%～80%。有效值范围（8~1048576）
BUFFER_POOLS	1	静态	BUFFER 系统分区数，每个 BUFFER 分区的大小为 BUFFER/BUFFER_POOLS。有效值范围（1~512）
FAST_POOL_PAGES	0	静态	快速缓冲区页面数。有效值范围（0~99999）。FAST_POOL_PAGES 的值最多不能超过 BUFFER 总页数的一半，如果超过，系统会自动调整为 BUFFER 总页数的一半
FAST_ROLL_PAGES	0	静态	快速缓冲区回滚页面数。有效值范围（0~99999）。 FAST_ROLL_PAGES 不能超过 FAST_POOL_PAGES-64，至少保证快速缓冲区中有超过 64 个非回滚页
KEEP	8	静态	KEEP 缓冲区大小，以兆为单位。有效值范围（8~1048576）
RECYCLE	64	静态	RECYCLE 缓冲区大小，以兆为单位。有效值范围（8~1048576）
RECYCLE_POOLS	1	静态	RECYCLE 缓冲区分区数，每个 RECYCLE 分区的大小为 RECYCLE/RECYCLE_POOLS。有效值范围（1~512）
MULTI_PAGE_GET_NUM	16	静态	缓冲区最多一次读取的页面数。有效值范围（1~128） 注：当数据库加密和 SSD_BUF_SIZE>0 时不支

			持多页读取，此时 dm.ini 中此参数值无效
MAX_BUFFER	100	静态	系统最大缓冲区大小，以兆为单位。采用动态缓冲区时，可以扩展到的最大缓冲区页数（>=BUFFER）。有效值范围（8~1048576）
SORT_BUF_SIZE	2	动态，会话级	排序缓存区最大值，以 M 为单位。有效值范围（1~2048）
HAGR_HASH_SIZE	100000	动态，会话级	HAGR 操作时，建立 HASH 表的桶个数。有效值范围（10000~100000000）
MAL_LEAK_CHECK	0	动态，系统级	是否打开 MAL 内存泄露检查。0：关闭；1：打开
HJ_BUF_GLOBAL_SIZE	500	动态，系统级	HASH 连接操作符的数据总缓存大小（>= HJ_BUF_SIZE），系统级参数，以兆为单位。有效值范围(10~500000)
HJ_BUF_SIZE	50	动态，会话级	单个 HASH 连接操作符的数据总缓存大小，以兆为单位。有效值范围。有效值范围（2~100000）
HJ_BLK_SIZE	1	动态，会话级	HASH 连接操作符每次分配缓存（BLK）大小，以兆为单位，必须小于 HJ_BUF_SIZE。有效值范围（1~50）
HAGR_BUF_GLOBAL_SIZE	500	动态，系统级	HAGR、DIST、集合操作、SPOOL、NTTS 以及 HTAB 操作符的数据总缓存大小（>= HAGR_BUF_SIZE），系统级参数，以兆为单位。有效值范围(10~1000000)
HAGR_BUF_SIZE	50	动态，会话级	单个 HAGR、DIST、集合操作、SPOOL、NTTS 以及 HTAB 操作符的数据总缓存大小，以兆为单位。有效值范围（2~500000）
HAGR_BLK_SIZE	1	动态，会话级	HAGR、DIST、集合操作、SPOOL、NTTS 以及 HTAB 操作符每次分配缓存（BLK）大小，以兆为单位，必须小于 HAGR_BUF_SIZE。有效值范围（1~50）
MTAB_MEM_SIZE	8	静态	MTAB 缓存 BDTA 占用内存空间的大小，以 KB 为单位，有效值范围（1~1048576）
DICT_BUF_SIZE	5	静态	字典缓冲区大小，以兆为单位，有效值范围（1~2048）。单位：MB
HFS_CACHE_SIZE	160	动态，系统级	HFS I/U/D 时 HDTA_BUFFER 缓存池大小，单位为 MB。有效值范围（160~ 2000）
VM_STACK_SIZE	128	静态	系统执行时虚拟机堆栈大小，单位为 K，堆栈的空间是从操作系统中申请的，有效值范围（64~256*1024）
VM_POOL_SIZE	64	静态	系统执行时虚拟机内存池大小，在执行过程中用到的内存大部分是从这里申请的，它的空间是从操作系统中直接申请的，有效值范围（32~1024*1024）
SESS_POOL_SIZE	16	动态，系统级	会话缓冲区大小，以 KB 为单位，有效值范围（16~1024*1024）。若所申请的内存超过实际能申请的大小，则系统将按 16KB 大小重新申请
VM_MEM_HEAP	0	静态	VM 是否使用 HEAP 分配内存。1：是，0：否
VM_SQL_TREE_CA	0	动态，系	SQL 运行树缓冲区大小，以 MB 为单位，设置

CHE_SIZE		统级	为 0 则不缓存。有效值范围(0~1024*512)
RFIL_RECV_BUF_SIZE	16	静态	控制服务器启动时，进行 REDO 操作过程中，REDO 日志文件恢复时 BUFFER 的大小，以 MB 为单位，有效值范围（16~4000）
N_MEM_POOLS	1	静态	内存池的数量，有效值范围（1~128）
COLDATA_POOL_SIZE	0	动态，系统级	COLDATA 池的大小，以 M 为单位
HAGR_DISTINCT_HASH_TABLE_SIZE	10000	动态，会话级	分组 DISTINCT 操作中 HASH 表的大小(桶数)。取值范围为(10000~100000000)
CNNTB_HASH_TABLE_SIZE	100	动态，会话级	指定 CNNTB 操作符中创建 HASH 表的大小。有效值范围（100~100000000）
GLOBAL_RTREE_BUF_SIZE	100	动态，会话级	R 树全局缓冲区大小，以 MB 为单位
SINGLE_RTREE_BUF_SIZE	10	动态，会话级	单个 R 树的缓冲区大小，以 MB 为单位
SSD_BUF_SIZE	0	静态	指定 SSD 缓冲区大小，以兆为单位。取值范围（0~ 4294967294），0 表示关闭
SSD_FILE_PATH	需要时指定	静态	SSD 缓冲区文件所在的文件夹路径，管理员保证其在 SSD 分区上
SSD_REF_BUF_SIZE	80	静态	SSD 缓冲专用 BUF 大小，SSD_BUF_SIZE 不为 0 时有效。以兆为单位，有效值范围（20~4096）
SSD_FLUSH_INTERVAL	10	静态	SSD 缓冲刷盘轮询间隔，以 MS 为单位，取值范围（0~1000）
SSD_FLUSH_STEPS	100	静态	SSD 缓存刷盘向前页数，有效值范围（10~100000）
线程相关参数			
WORKER_THREADS	4	静态	工作线程的数目，有效值范围（1~64）
TASK_THREADS	4	静态	任务线程个数，有效值范围（1~1000）
UTHR_FLAG	0	手动	用户线程标记，1：启用；0：不启用。 启用用户线程时，并行查询失效，并行查询的相关参数不起作用
FAST_RW_LOCK	1	手动	快速读写锁标记，1 表示启用，0 不启用
SPIN_TIME	4000	静态	线程在不能进入临界区时，自旋的次数，有效值范围（0~4000）
WORK_THRD_STACK_SIZE	1024	静态	工作线程堆栈大小，以 KB 为单位。有效值范围（64~4096）
WORKER_CPU_PERCENT	0	手动	工作线程占 CPU 的比重，仅非 WINDOWS 下有效，有效值范围（0~100）
查询相关参数			
USE_PLN_POOL	1	静态	是否重用执行计划。0：禁止执行计划的重用；1：启用执行计划的重用功能 ；2：对不包含显式参数的语句进行常量参数化优化；3：即使包含显式参数的语句，也进行常量参数化优化
N_PLN_POOLS	1	静态	支持将计划缓存进行分片管理。有效值范围（1~1000）

RS_CAN_CACHE	0	静态	结果集缓存配置。0：默认不缓存结果集，但 DBA 可通过 HINT 等方式手动配置结果集缓存；1：默认缓存所有可缓存结果集，DBA 可手动取消某些结果集的缓存
RS_CACHE_TABLES	空串	手动	指定可以缓存结果集的基表的清单，当 RS_CAN_CACHE=1 时，只有查询涉及的所有基表全部在此参数指定范围内，该查询才会缓存结果集
RS_CACHE_MIN_TIME	0	动态，系统级	结果集缓存的语句执行时间下限，只有实际执行时间不少于指定时间值的查询，其结果集才会被缓存，仅在 RS_CAN_CACHE=1 时有效。默认值 0，表示不限制；有效值范围（0~4294967294），以 MS 为单位
RS_BDTA_FLAG	0	静态	是否以 BDTA 形式返回结果集。0：以行为单位返回结果集；2：以 BDTA 形式返回结果集
RS_BDTA_BUF_SIZE	32	静态	配置消息长度，单位为 K。有效值范围（8~32768）
RESULT_SET_LIMIT	10000	动态，会话级	一次请求可以生成的结果集最大个数。有效值范围（1~65000）
RESULT_SET_FOR_QUERY	0	动态，会话级	是否生成非查询结果集。0：生成；1：不生成
SESSION_RESULT_SET_LIMIT	1	动态，系统级	会话上结果集个数上限,有效值范围（1~ 65000）
BUILD_FORWARD_RS	0	静态	游标是否生成结果集。0：不生成；1：生成
MAX_OPT_N_TABLES	6	动态，会话级	优化器在处理连接时，一次能优化的最大表连接个数。有效值范围（3~8）
CNNTB_MAX_LEVEL	20000	动态，会话级	层次查询的最大支持层次。有效值范围（1~100000）
BATCH_PARAM_OPT	0	静态	是否启用批量参数优化，0：不启用；1：启用，默认不启用。当置为 1 时，不返回操作影响的行数
SESS_PLN_NUM	0	动态，会话级	每个会话上最多拥有的临时计划缓存个数，如果大于该个数，则会淘汰掉一部分缓存计划。0 表示不启用，也可以理解为会话上不会临时缓存计划。有效值范围（0~4294967294）。 只有在计划重用参数 USE_PLN_POOL 启用时，该参数才会生效
CLT_CONST_TO_PARAM	0	静态	是否进行语句的常量参数化优化，0：不进行；1：进行
LIKE_OPT_FLAG	1	动态，会话级	LIKE 查询的优化开关。0：不优化；1：对于 LIKE 表达式首尾存在通配符的情况，优化为 POSITION()函数；对于 LIKE 表达式首部存在通配符，并且条件列存在 REVERSE()函数索引时，优化为 REVERSE()函数；2：对于 COL1 LIKE COL2 ‖ '%'的情况，优化为 POSITION()函数；4：对于 COL1 LIKE ‘A’‖‘B%’的

			情况，优化为 COL1 LIKE ‘AB%’ 支持使用上述有效值的组合值，如 5 表示同时进行 1 和 4 的优化
MPP_OP_JUMP	1	动态，会话级	MPP 系统中操作符的跳转开关，是否支持通讯操作符的跳转功能。0：不支持；1：支持
PHF_NTTS_OPT	1	动态，会话级	MPP 系统中是否进行 NTTS 计划的优化，打开时可能减少计划中的 NTTS 操作符。0: 不支持；1：支持
USE_FTTS	0	动态，会话级	执行过程中产生的临时数据的存放格式。0：用临时表空间的数据页存放；1：用临时文件存放
UPD_DEL_OPT	0	动态，会话级	删除更新计划优化方式，取值范围 0、1、2 单节点计划，0：不优化 1，2:都可优化 NTTS MPP 计划，0：不优化； 1：优化删除更新计划，不优化 NTTS；2：优化删除更新计划，同时优化 NTTS
ENABLE_DIST_IN_SUBQUERY_OPT	0	动态，系统级	优化子查询里面的 IN，0：不优化；1：优化
MAX_OPT_N_OR_BEXPS	7	动态，会话级	能参与优化的最大 OR 分支个数，超过时布尔表达式仅用于过滤使用。有效值范围（7~16）
USE_HAGR_FLAG	0	动态，会话级	当带有 DISTINCT 的集函数不能使用 SAGR 操作符时，是否使用 HAGR，0：不使用；1：使用
DTABLE_PULLUP_FLAG	1	动态，会话级	是否在语法分析阶段对派生表进行上拉优化处理，0：不优化；1：优化
VIEW_PULLUP_FLAG	0	动态，会话级	是否对视图进行上拉优化，把视图转换为其原始定义，消除视图。0：不启用；1：启用
VIEW_PULLUP_MAX_TAB	7	动态，会话级	对视图进行上拉优化支持的表的个数。有效值值范围：（1~16）
HAGR_PARALLEL_OPT_FLAG	0	动态，会话级	MPP、并行下对 GROUP BY、分析函数等的优化开关。0：不优化；1：无 DISTINCT 时 HAGR 按照分组列分发；2：有 DISTINCT 时，HAGR 按照分组列分发；4：去除多余的通讯操作符；8：同 4，仅限于外连接操作；16：分析函数按照 PARTITION BY 列分发；32：MPP+LPQ 下，AAGR 的优化处理 支持使用上述有效值的组合值，如 5 表示同时进行 1 和 4 的优化
HAGR_DISTINCT_OPT_FLAG	0	动态，会话级	MPP 下是否对 HAGR+DISTINCT 进行优化，0：不优化；1：优化
REFED_EXISTS_OPT_FLAG	0	动态，会话级	是否把相关 EXISTS 优化为非相关 IN 查询：0：否；1：是
MAX_PHC_BE_NUM	512	动态，会话级	优化阶段存放临时布尔表达式的个数。有效值范围（512~ 20480000）
PARTIAL_JOIN_EVALUATION_FLAG	0	动态，会话级	是否对去除重复值操作的下层连接进行转换优化，0：不优化；1：优化 此参数仅在参数 OPTIMIZER_MODE 为 1 时才有效

USE_FK_REMOVE_TABLES_FLAG	0	动态，会话级	是否利用外键约束消除冗余表。0：不启用；1：启用
MPP_HASH_LR_RATE	10	动态，会话级	MPP 下，对 HASH JOIN 节点，可以根据左右儿子 CARD 代价的比值，调整 HASH_JOIN 的左右儿子的 MOTION 添加，从而影响计划。如果 CARD 比值超过此值，则小数据量的一方全部收集到主 EP 来做。有效值范围（1~4294967294）
LPQ_HASH_LR_RATE	30	动态，会话级	LPQ 下，对 HASH JOIN 节点，可以根据左右儿子 CARD 代价的比值，调整 HASH_JOIN 的左右儿子的 MOTION 添加，从而影响计划。 如果 CARD 比值超过此值，则小数据量的一方全部收集到主 EP 来做。有效值范围（1~4294967294）
SEL_ITEM_HTAB_FLAG	0	动态，会话级	当查询项中有相关子查询时，是否做 HTAB 优化。0：不优化；1：优化。
OR_CVT_HTAB_FLAG	1	动态，会话级	当查询条件 OR 中含有公共因子时，是否允许使用 HTAB 来进行优化。0：不使用；1：使用
CASE_WHEN_CVT_IFUN	1	动态，会话级	是否将CASE WHEN THEN ELSE END语句转换为IFOPERATOR函数。0：不转换；1：转换
OR_NBEXP_CVT_CASE_WHEN_FLAG	0	动态，会话级	是否将OR 转化为CASE WHEN THEN ELSE END语句。0：不转换；1：转换
OUTER_CVT_INNER_PULL_UP_COND_FLAG	0	动态，会话级	当外连接转化为内连接时，是否打开连接条件。0：不打开；1：打开
OPT_OR_FOR_HUGE_TABLE_FLAG	1	动态，会话级	是否使用 HFSEK 优化 HUGE 表中列的 OR 过滤条件。0：不使用；1：使用
ORDER_BY_NULLS_FLAG	0	动态，会话级	ASC 升序排序时，控制 NULL 值返回的位置。取值 0 或 1。1 表示 NULL 值在最后返回，0 表示 NULL 值在最前面返回。在参数等于 1 的情况下，NULL 值的返回与 ORACLE 保持一致。DESC 降序时该参数无效
SUBQ_CVT_SPL_FLAG	1	动态，会话级	控制相关子查询的实现方式，0：不优化；1：使用 SPOOL 方式实现相关子查询；4：将多列 IN 转换为 EXISTS；8：将引用列转换为变量 VAR；16：用临时函数替代查询项中的相关查询表达式 支持使用上述有效值的组合值，如 5 表示同时进行 1 和 4 的优化
ENABLE_RQ_TO_SPL	1	动态，会话级	是否将相关子查询转换为 SPOOL 方式，0：不优化；1：优化
MULTI_IN_CVT_EXISTS	0	动态，会话级	多列 IN 是否转换为等价的 EXISTS 过滤。0：不转换；1：转换
PRJT_REPLACE_NPAR	1	动态，会话级	是否将引用列转换为变量 VAR，并替换查询表达式中的引用列，0：不优化；1：优化
ENABLE_RQ_TO_INV	0	动态，会话级	相关查询表达式是否转换为函数方式实现。0：不转换；1：转换

USE_REFER_TAB_ONLY	0	动态，会话级	处理相关子查询时是仅将相关的表下放，或者连同上方的 SEMI/HASH JOIN 一起下放。0：一起下放；1：仅下放相关表
REFED_SUBQ_CROSS_FLAG	0	动态，会话级	是否将相关子查询与外层表优化为 CROSS JOIN，0：不优化；1：优化
IN_LIST_AS_JOIN_KEY	0	动态，会话级	搜索多表连接方式时，对于索引连接（INDEX JOIN）的探测，NEXP_IN_LST 表达式类型可以作为多表连接的 KEY。 取值范围： 0：表示搜索多表连接方式时，对于索引连接（INDEX JOIN）的探测，NEXP_IN_LST 不可以作为连接 KEY； 1：表示搜索多表连接方式时，对于索引连接（INDEX JOIN）的探测，把 NEXP_IN_LST 当做普通等值 KEY 的处理方式来生成 INDEX JOIN 的连接 KEY。
OUTER_JOIN_INDEX_OPT_FLAG	0	动态，会话级	外连接优化为索引连接的优化开关。1：优化；0：不优化。
OUTER_JOIN_FLATING_FLAG	0	动态，会话级	优化外连接。 0：不启用优化； 1：启用优化，例如，使得类似这样的连接 A LEFT (B CROSS C) ON A.C1=B.C1 优化为 A LEFT (B JOIN (DISTINCT A) ON A.C1=B.C1 CROSS C) ON A.C1=B.C1 ，相当于把 A 和 B 的过滤器平坦化到了下层查询中，使得 B 获取的中间结果较小
TOP_ORDER_OPT_FLAG	0	动态，会话级	优化带有 TOP 和 ORDER BY 子句的查询，使得 SORT 操作符可以省略。优化的效果是尽量使得ORDER BY的排序列所对应的基表可以使用包含排序列的索引，从而可以移除排序 SORT 操作符，减少排序操作。如果排序列不属于同一个基表，或者排序列不是基表列，则肯定是不可以优化。 取值： 0：不启用该优化 1：启用该优化
TOP_DIS_HASH_FLAG	1	动态，会话级	是否通过禁用 HASH JOIN 方式来优化 TOP 查询，0：不优化；1：当 OPTIMIZER_MODE 为 0 时，TOP 下方连接禁用 HASH JOIN；当 OPTIMIZER_MODE 为 1 时，TOP 下方连接倾向于不使用 HASH JOIN
ENABLE_RQ_TO_NONREF_SPL	0	动态，会话级	相关查询表达式转化为非相关查询表达式，目的在于相关查询表达式的执行处理由之前的平坦化方式转化为一行一行处理，类似 ORACLE 的每行处理机制。 0：不启用该优化；1：启用该优化。
OPTIMIZER_MODE	0	动态，会话级	DM 优化器的模式，0：老优化器模式；1：新优化器模式

OPTIMIZER_MAX_PERM	7200	动态，会话级	控制计划探测过程中的最大排列数，如果大于该阈值，则减少相应的排列，减少探测计划的时间，有效值范围（1~4294967293）
ENABLE_INDEX_FILTER	0	动态，会话级	是否进行索引过滤优化，可取值为 0、1、2。 0 不进行优化； 1 使用索引过滤优化，如果过滤条件涉及的列包含在索引中，那么索引进行 SSEK2 后就可以使用此过滤条件，可以减少中间结果集； 2 在取值为 1 的基础上，将 IN 查询列表转换为 HASH RIGHT SEMI JOIN。
OPTIMIZER_DYNAMIC_SAMPLING	0	动态，会话级	当统计信息不可用时是否启用动态统计信息。取值范围：0~12。0：不启用；1~10：启用，采用率 10%~100%；11：启用，由优化器确定采样率（0.1%~99.9%）；12：同 11，但收集的结果会持久化保存
NONREFED_SUBQUERY_AS_CONST	0	动态，会话级	是否将非相关子查询转化为常量处理。0：不进行优化，对非相关子查询使用连接方式处理；1：将非相关子查询转换为常量，作为过滤条件使用
HASH_CMP_OPT_FLAG	0	动态，会话级	是否启用静态哈希表的优化。0：不启用优化；1：分组中的 DISTINCT 开启此优化；2：对 HASH 连接启用该优化；4：对 HAGR 分组计算启用该优化；8：DISTINCT 操作开启此优化。 支持使用上述有效值的组合值，如 5 表示同时进行 1 和 4 的优化
OUTER_OPT_NLO_FLAG	0	动态，会话级	外连接或内连接是否支持右孩子为非基表情况下的 INDEX JOIN。0：不启用；1：启用外连接下的优化；2：启用内连接下的优化；3：同时启用内连接和外连接。 该参数仅在 OPTIMIZER_MODE = 0 时生效
USE_INDEX_SKIP_SCAN	0	动态，会话级	间隔索引扫描的开关，打开后，即使搜索 KEY 不是索引的前导列，依然可能使用该索引进行探测扫描。 0：关闭；1：打开：通过代价计算选择，倾向于选择前导列 DISTINCT 值少，搜索列 DISTINCT 值较多的索引；2：只要能够找到索引使用间隔扫描，就强制使用。
INDEX_SKIP_SCAN_RATE	0.0025	动态，会话级	间隔索引扫描的代价调节开关。当列的 DISTINCT 数/总行数大于该值时，就不再使用间隔索引扫描的方式
SPEED_SEMI_JOIN_PLAN	1	动态，会话级	是否加速半连接的探测过程。 0：不加速；1：加速计划探测；2：加速计划探测和执行
COMPLEX_VIEW_MERGING	0	动态，会话级	对于复杂视图（一般含有 GROUP 或者集函数等）会执行归并操作，使得 GROUP 分组操作在连接之后才执行。0：不启用；1：启用。
OP_SUBQ_CVT_IN_	1	动态，会	当查询条件为=(SUBQUERY)，是否考虑转换为

FLAG		话级	等价的 IN(SUBQUERY)。0：不转换；1：转换
HLSM_FLAG	1	动态，会话级	控制多列非相关 NOT IN 的查询实现方式，1：当数据量较大，HASH BUF 放不下时，采用 MTAB 方式处理；2：当数据量较大，HASH BUF 放不下时，采用 B 树方式处理，使用细粒度扫描；3：当数据量较大，HASH BUF 放不下时，采用 B 树方式处理，使用粗粒度扫描
DEL_HP_OPT_FLAG	0	动态，会话级	控制分区表的操作优化，0：不优化；1：打开分区表 DELETE 优化；2：控制范围分区表创建的优化处理，转换为数据流方式实现
OPTIMIZER_OR_NBEXP	0	动态 会话级	OR 表达式的优化方式。0：不优化；1：生成 UNION_FOR_OR 操作符时，优化为无 KEY 比较方式；2：OR 表达式优先考虑整体处理方式；4：相关子查询的 OR 表达也优考虑整体处理方式；8：OR 布尔表达式的范围合并优化；16：同一列上同时存在常量范围过滤和 IS NULL 过滤时的优化，如 C1 > 5 OR C1 IS NULL。 支持使用上述有效值的组合值，如 7 表示同时进行 1、2、4 的优化
CNNTB_OPT_FLAG	0	动态，会话级	是否使用优化的层次查询执行机制。0：不使用；1：强制使用；2：优化器自动决定是否使用
MULTI_UPD_OPT_FLAG	0	动态，会话级	是否使用优化的多列更新。0：不使用，仍按照语句改写方式实现；1：利用多列 SPL 功能加以实现。
HAGR_HASH_ALGORITHM_FLAG	0	动态，会话级	HAGR 中 HASH 算法选择标记。0：按照移位方式计算；1：按照异或方式计算
DIST_HASH_ALGORITHM_FLAG	0	动态，会话级	DISTINCT 中 HASH 算法选择标记。0：按照移位方式计算；1：按照异或方式计算
UNPIVOT_ORDER_FLAG	0	动态，会话级	是否对 UNPIVOT 的结果按照公共列排序，公共列指表中未出现在 UNPIVOT 的列。0：不排序；1：排序
VIEW_FILTER_MERGING	1	动态，会话级	是否对视图条件进行合并优化以及如何优化，0：不优化；1：尽可能地进行视图条件合并；2：自动判断是否进行视图条件合并
OPT_MEM_CHECK	0	动态，会话级	内存紧张时，优化器是否缩减计划探测空间，仅 OPTIMIZER_MODE=0 时生效。0：不缩减计划探测空间；1：缩减计划探测空间；
ENABLE_JOIN_FACTORIZATION	0	动态，会话级	是否启用连接分解，即 UNION ALL 分支间存在公共部分时是否进行公因子提取。0：不启用；1：启用
检查点相关参数			
CKPT_RLOG_SIZE	100	动态，系统级	产生多大日志文件后做检查点，以兆为单位。有效值范围（0~4294967294）
CKPT_DIRTY_PAGES	10000	动态，系统级	产生多少脏页后产生检查点，以页为单位。有效值范围（0~4294967294）
CKPT_INTERVAL	1800	动态，系统级	指定检查点的时间间隔。以秒为单位，为 0 时表示不自动定时做检查点。有效值范围（0~

			2147483647）
CKPT_FLUSH_RATE	5.00	动态，系统级	检查点刷盘比例。有效值范围（0~100.00）
CKPT_FLUSH_PAGES	1000	动态，系统级	检查点刷盘的最小页数。有效值范围（1000~100000）
CKPT_WAIT_PAGES	128	动态，系统级	检查点一次发起的最大写入页数，等待这些页写入磁盘完成、调整检查点信息后，再发起新的刷盘请求，避免过于集中发起写磁盘请求，操作系统 IO 压力过大，导致 IO 性能下降。有效值范围（1~1024）
FORCE_FLUSH_PAGES	10	动态，系统级	调度线程启动刷脏页流程时，每个 BUFFER POOL 写入磁盘的脏页数。有效值范围（0~1000）
IO 相关参数			
DIRECT_IO	0	静态	非 WINDOWS 下有效，0：使用 OS 文件系统缓存；1：不使用 OS 文件系统缓存，采用系统模拟的异步 IO，异步 IO 的线程数由 IO_THR_GROUP 控制；2：不适用 OS 文件系统缓存，采用系统提供的 NATIVE IO 机制， 要求 LINUX 内核 2.6 以上
IO_THR_GROUPS	2	静态	非 WINDOWS 下有效，表示 IO 线程组个数。有效值范围（0~512）
HIO_THR_GROUPS	2	静态	HUGE 缓冲区 IO 线程组数目。 有效值范围 （1 ~ 512）
FIL_CHECK_INTERVAL	0	动态，系统级	指定检查数据文件是否存在的时间间隔，单位：S。有效值范围（0~4294967294）。0 表示不检查。
数据库相关参数			
MAX_SESSIONS	100	静态	系统允许同时连接的最大数，同时还受到 LICENSE 的限制，取二者中较小的值，有效值范围（1~65000）
MAX_CONCURRENT_TRX	0	静态	表示系统支持同时运行事务数的最大值。有效值范围（0~1500），0 表示不控制 注：这个参数仅在需要超大数量连接时才需要设置
MAX_SESSION_STATEMENT	100	动态，系统级	单个会话上允许同时打开的语句句柄最大数，有效值范围（64~20480）
MAX_CONCURRENT_OLAP_QUERY	0	静态	OLAP 模式下能同时执行的复杂查询个数，超过限制后，复杂查询将在执行阶段进入等待。0 表示不做限制。有效值范围（0~100） 注：此处的复杂查询是指计划中存在行数超过参数 BIG_TABLE_THRESHHOLD*10000 的节点，含有哈希连接、哈希分组等耗内存操作符，且不涉及到系统表
BIG_TABLE_THRESHHOLD	1000	动态，会话级	OLAP 环境下对复杂查询最大并发数（参数 MAX_CONCURRENT_OLAP_QUERY）进行限制时，认定是否为复杂查询中表的行数下限，以万为单位。取值范围(0~4294967294)

MAX_EP_SITES	64	手动	MPP 环境下 EP 站点的最大数量，有效值范围（2~1024）
PORT_NUM	5236	静态	服务器通讯端口号，有效值范围（1024~65534）
FAST_LOGIN	0	静态	是否在登录时记录登录历史信息。0记录，1不记录
DDL_AUTO_COMMIT	1	动态，会话级	指定 DDL 语句是否自动提交，1：自动；0：手动。当 COMPATIBLE_MODE=1 时，DDL_AUTO_COMMIT 的实际值均为 0。
COMPRESS_MODE	0	动态，会话级	建表时是否缺省压缩。0：不进行；1：进行
PK_WITH_CLUSTER	1	动态，会话级	在建表语句中指定主关键字时，是否缺省指定为 CLUSTER，0：不指定；1：指定 注：该参数对列存储表和堆表无效
EXPR_N_LEVEL	200	动态，会话级	表达式最大嵌套层数。有效值范围（30~1000）
N_PARSE_LEVEL	100	动态，会话级	表示对象 PROC、VIEW、PKG、CLASS 的最大解析层次，如果层次过深则报错返回。有效值范围（30~1000）
BDTA_SIZE	1000	静态	BDTA 缓存的记录数。有效值范围（1~10000）
OLAP_FLAG	0	动态，会话级	启用联机分析处理，0：不启用；1：启用；2：不启用，同时倾向于使用索引范围扫描
JOIN_HASH_SIZE	500000	动态，会话级	HASH JOIN 操作时，HASH 表大小，以 CELL 个数为单位。有效值范围（0~250000000）
USE_DHASH_FLAG	0	动态，会话级	是否使用动态 HASH。0：不使用；1：使用
HFILES_OPENED	256	静态	设置可以同时打开的列存储表数据文件个数。有效值范围（60-10000）
FAST_COMMIT	0	动态，系统级	批量提交事务的个数，有效值范围（0~100）
ISO_IGNORE	0	动态，会话级	是否忽略显示设置的事务隔离级别，1 表示忽略，0 不忽略。
TEMP_SIZE	10	静态	默认创建的临时表空间大小，以兆为单位。有效值范围（10~1048576）
TEMP_SPACE_LIMIT	0	动态，系统级	临时表空间大小上限，以 M 为单位。0 表示不限制临时表空间大小。 有效范围（0~ 4294967294）。 注意：TEMP_SPACE_LIMIT 一定要大于等于 TEMP_SIZE
FILE_TRACE	0	静态	日志中是否记录文件操作，0：不记录；1：记录
COMM_TRACE	1	动态，会话级	服务器日志是否记录通信中产生的警告信息。0：不记录；1：记录
CACHE_POOL_SIZE	10	静态	SQL 缓冲池大小，以兆为单位。有效值范围：32 位平台下为（1~2048）；64 位平台下为（1~67108864）。单位：MB
STAT_COLLECT_SI	10000	动态，会	统计信息收集时，样本的最小行数。有效值范

ZE		话级	围（0~10000000）
STAT_ALL	0	动态，会话级	在估算分区表行数时，是否采样所有分区子表。0：不采样所有；1：采样所有
PHC_MODE_ENFORCE	15	动态，会话级	对派生视图进行分组查询，且分组项是派生视图分组项的子集时，是否考虑将两层分组进行合并。0：不考虑合并；1：考虑合并
ENABLE_HASH_JOIN	1	动态，会话级	是否允许使用哈希连接，0：不允许；1：允许。
ENABLE_INDEX_JOIN	1	动态，会话级	是否允许使用索引连接，0：不允许；1：允许。
ENABLE_MERGE_JOIN	1	动态，会话级	是否允许使用归并连接，0：不允许；1：允许
MPP_INDEX_JOIN_OPT_FLAG	0	动态，会话级	MPP 下，是否可以使用索引左外连接，0：不使用；1：使用
MPP_NLI_OPT_FLAG	0	动态，会话级	MPP 下嵌套连接的优化方式，0：嵌套连接操作的左右孩子添加 MPP GATHER，汇总到主 EP 执行， 父节点添加 MPP SCATTER；3：嵌套连接操作的左右孩子都添加 MPP GATHER 和 MPP SCATTER
MAX_PARALLEL_DEGREE	1	动态，会话级	用来设置默认并行任务个数。取值范围：1~128。缺省值 1，表示无并行任务。当 PARALLEL_POLICY 值为 1 时该参数值才有效
PARALLEL_POLICY	0	静态	用来设置并行策略。取值范围：0、1 和 2，缺省为 0。其中，0 表示不支持并行；1 表示自动并行模式；2 表示手动并行模式
PARALLEL_THRD_NUM	10	静态	用来设置并行工作线程个数。有效值范围（1~1024）
PARALLEL_MODE_COMMON_DEGREE	1	动态，会话级	并发模式程度。此值越大，扫描代价将越大，越倾向于使用索引连接而不是哈希连接，需酌情使用。有效值范围（1~1024）
PUSH_SUBQ	0	动态，会话级	是否下放子查询使先做。0：相关子查询不下放，非相关子查询在不存在单表过滤条件时下放；1：不存在单表过滤条件时下放；2：始终考虑下放
OPTIMIZER_AGGR_GROUPBY_ELIM	0	动态，会话级	优化器被允许的连接方式。1：嵌套循环连接；2：索引连接；4：哈希连接；8：归并连接 还可使用以上值的组合值
UPD_TAB_INFO	0	静态	系统启动时是否更新表信息（如行数），1：启用更新；0：不更新。
ENABLE_IN_VALUE_LIST_OPT	0	动态，系统级	是否允许 IN LIST 表达式优化。0：不优化。1：将 IN LIST 表达式在语义分析阶段优化为 CONSTV 处理； 2：在查询计划确定后，把 IN LIST 表达式转化为 CONSTV；4：允许 IN LIST 参与等值传递；8：从 IN VALUE 中构造出范围条件（仅限于 HUGE 表） 支持使用上述有效值的组合值，如 3 表示同时进行 1 和 2 的优化
ENHANCED_BEXP_TRANS_GEN	0	动态，系统级	是否允许非等值布尔表达式的传递闭包。0：不允许；1：允许
ENABLE_DIST_VIE	0	动态，系	是否支持更新含有 DISTINCT 的视图，1：允许；

W_UPDATE		统级	0：不允许，当 COMPATIBLE_MODE=1 时，ENABLE_DIST_VIEW_UPDATE 的实际值均为 1
STAR_TRANSFORMATION_ENABLED	0	动态，会话级	是否允许对星形模型查询改写以使用位图连接索引。0：不启用；1：启用 此参数仅在 OLAP_FLAG 为 1 时才有效
MONITOR_INDEX_FLAG	0	动态，会话级	是否对索引进行监控，0：关闭自动监控，可使用 ALTER INDEX 语句启用索引监控；1：打开自动监控，对用户定义的二级索引进行监控；2：禁止索引监控
FIRST_ROWS	100	动态，系统级	结果集一个消息中返回的最大行数，有效值范围（1~1000）
LIST_TABLE	0	动态，会话级	默认情况下，创建的表是否为堆表，0：否；1：是
ENABLE_SPACELIMIT_CHECK	1	静态	是否启用检查 SPACE LIMIT。1：启动；0：不启用
BUILD_VERTICAL_PK_BTREE	0	手动	HUGE表上创建的主键是否需要创建物理B树。0：不创建；1：创建
BDTA_PACKAGE_COMPRESS	0	动态，会话级	是否启动bdta压缩传递功能，1：压缩；0：不压缩
HFINS_PARALLEL_FLAG	0	动态，系统级	是否启动HUGE表查询插入优化，1：是；0：否
HFINS_MAX_THRD_NUM	100	动态，系统级	HUGE表查询插入优化时并行的最大线程数，有效值范围（4~200）
LINK_CONN_KEEP_TIME	15	静态	DBLINK空闲连接保持时间，以分钟为单位，为0时表示不主动释放空闲DBLINK连接。
DETERMIN_CACHE_SIZE	5	动态，会话级	确定性函数值缓冲区大小，以MB为单位
NTYPE_MAX_OBJ_NUM	1000000	动态，系统级	复合数据类型中包含的对象或者字符串的总个数，以及查询中包含的变量的总个数，有效值范围为（2000, 100000000）
CTAB_SEL_WITH_CONS	0	动态，系统级	查询建表时，是否对原始表上的约束进行拷贝。1：是，0：否。
HLDR_BUF_SIZE	8	动态，系统级	进行 HUGE 表导入时，缓冲区的大小，单位 MB，有效值范围（4~1024）
HUGE_ACID	0	动态，系统级	是否支持 HUGE 表增删改与查询的并发，取值范围：0、1、2，默认值为 0 0：不支持 HUGE 表增删改与查询的并发 1：支持 HUGE 表增删改与查询的并发，操作符 HFLKUP 检查数据区是否上锁时使用二分查找 2：支持 HUGE 表增删改与查询的并发，操作符 HFLKUP 检查数据区是否上锁时使用遍历查找
HBUF_DATA_MODE	0	静态	HUGE 表数据缓冲区中缓存数据的格式，取值范围：0、1，默认值为 0 0：格式与数据文件中的数据块格式一致 1：缓存解密解压后的数据如果数据块时压缩加密的情况下

SEC_INDEX_PARALLEL_INSERT_FLAG	0	动态，系统级	二级索引是否使用并行插入优化标记，0：否；1：是
FILL_COL_DESC_FLAG	1	动态，系统级	服务器返回给客户端的结果集是否含有列描述信息，0：否；1：是
BTR_SPLIT_MODE	0	动态，系统级	B 树叶子节点分裂方式，0：按对半分裂；1：在插入点进行分裂
BLOB_OUTROW_REC_STOR	5	动态，系统级	大数据行外记录式存储配置参数。如果使用记录式存储，可以在一个数据页中存放多个大数据。有效值范围（0~10），0 表示不使用记录式存数；其他设置指定记录式存储的分组数，可提高并发效率
TS_RESERVED_EXTENTS	64	静态	系统为每个表空间提前预留的簇个数，减少系统在执行过程中申请不到空间的情况。有效值范围（2 ~ 1024）。 RAC 环境下，该参数所有站点需要保持一致
TS_SAFE_FREE_EXTENTS	512		系统认为安全的 FREE EXTENTS 空间，有效值范围（128 ~65534）。 RAC 环境下，该参数所有站点需要保持一致
TS_MAX_ID	1024		限制系统所支持的最大表空间 ID，有效值范围（5 ~ 65517）， 与系统实际最大 ID 比较，取大。 如果设置为 90，但是系统最大 ID 已经为 100 了，最后结果是 100。 RAC 环境下，该参数所有站点需要保持一致
TS_FIL_MAX_ID	255		限制每个表空间所支持的最大文件个数，范围（2 ~255）。 RAC 环境下，该参数所有站点需要保持一致
PWR_FLUSH_PAGES	10000	动态，系统级	控制生成特殊 PWR 记录的频率，每写入 PWR_FLUSH_PAGES 页到磁盘，即生成一条特殊的 PWR 记录，标记之前的 PWR 记录是有效的。有效值范围（0~4294967294）
REDO_UNTIL_LSN	空串	手动	系统故障重启时，重做 REDO 日志的最大 LSN 值。 使用这个参数要十分慎重，一般只在数据库文件系统损坏，系统无法正常启动情况下使用，通过指定这个参数，将数据库恢复到稍早时间点，让数据库可以正常启动，以便抢救部分数据。 注意：使用这个参数后，会截断指定 LSN 之后的所有日志。另外，系统启动后，一定不要忘记将参数值重新修改为缺省值。
IGNORE_FILE_SYS_CHECK	0	静态	系统重启时，是否检查 SYSTEM/ROLL/MAIN 表空间文件系统；取值范围 0，1；默认 0，检查文件系统。1，不检查文件系统。在实际使用空间比较大情况下，可以考虑关闭文件系统检查，提高系统启动速度。
STARTUP_CHECKP	0	静态	系统启动时 REDO 后是否强制做完全检查点。

OINT			0：不强制；1：强制
预先装载表相关参数			
LOAD_TABLE	空串	手动	在服务器启动时预先装载的表的完整表名，即“模式名.表名”，多个表之间用逗号分隔，最多可指定10个表
LOAD_HTABLE			
客户端缓存			
CLT_CACHE_TABLES	空串	手动	指定可以在客户端缓存的表。表名必须带模式名前缀，如果表名或模式名中包含特殊字符，需要使用双引号包含。如果指定多个缓存表，须以逗号间隔。服务器最多支持指定100个可缓存表。为避免参数值太长导致INI文件分析困难，允许在INI文件中设置多行CLT_CACHE_TABLES参数
REDO 日志相关参数			
RLOG_BUF_SIZE	512	静态	单个日志缓冲区大小（以日志页个数为单位），取值只能为 2 的次幂值，最小值为 1，最大值为 20480
RLOG_POOL_SIZE	128	静态	最大日志缓冲区大小（以 M 为单位）。有效值范围（16~1024）
RLOG_PARALLEL_ENABLE	0	静态	是否启动并行日志，1：启用；0：不启用
RLOG_APPEND_LOGIC	0	静态	是否启用在日志中记录逻辑操作的功能，取值范围 0、1、2 0：不启用；1：如果有主键列，记录 UPDATE 和 DELETE 操作时只包含主键列信息，若没有主键列则包含所有列信息；2：不论是否有主键列，记录 UPDATE 和 DELETE 操作时都包含所有列的信息
RLOG_APPEND_SYSTAB_LOGIC	0	静态	是否启用在日志中记录系统表逻辑操作的功能，启用 RLOG_APPEND_LOGIC 后有效，取值范围 0、1 0：不启用；1：启用
RLOG_RESERVE_SIZE	40960	动态，系统级	INSERT/DELETE/UPDATE 等操作预留的日志空间大小（以日志页个数为单位）。有效值范围（0~262144）
RLOG_CHECK_SPACE	1	动态，系统级	是否检查日志空间，取值范围 0、1。 1：日志刷盘时，检查日志空间是否足够，如果不够，则生成错误日志并强制退出，以确保数据文件不被破坏。 0：如果系统配置为不预留日志空间，则 RLOG_CHECK_SPACE 要强制设置为 0，即不检查日志空间
RLOG_SAFE_SPACE	128	静态	安全的可用日志空间大小（以 M 为单位）。有效值范围(0 ~ 1024)。当系统的可用日志空间小于这个值时，自动触发检查点释放日志空间
RLOG_SAFE_PERC	25	静态	安全的可用日志空间比例（FREE_SPACE /

ENT			TOTAL_SPACE * 100）。有效值范围(0 ~ 100)。当系统的 可用日志空间 * 100 / 系统日志总空间 小于这个值时，自动触发检查点释放日志空间
LOG_REDO_THREAD_NUM	4	静态	重做日志线程数，有效值范围（1~16）
REDO_PWR_OPT	1	静态	系统故障重启时，是否启动 PWR 优化；取值范围 0，1；默认 1，不再重做已经写入磁盘数据页的 REDO 日志，提高故障重启速度。0，取消优化，严格按照 REDO 日志生成顺序依次重做 REDO 日志。
REDO_IGNORE_DB_VERSION	0	静态	启动重做 REDO 日志时，是否检查版本信息。0：忽略版本检查，直接使用新版本重做 REDO 日志；1：正常检查版本，不兼容的库会报错，需要使用对应版本启动并正常关闭后，再用新版本执行码启动
ELOG_REPORT_LINK_SQL	0	动态，会话级	是否记录 DBLINK 执行的 SQL 到服务器日志文件中。0：不记录；1：记录
事务相关参数			
ISOLATION_LEVEL	1	静态	系统默认隔离级别。1：读提交；3：可串行化
DDL_WAIT_TIME	10	动态，会话级	DDL 操作的锁超时时间，以秒为单位。有效值范围（0~60）
FAST_RELEASE_SLOCK	1	动态，系统级	是否启用快速释放 S 锁，1：启用；0：不启用
SESS_CHECK_INTERVAL	3	动态，会话级	循环检测会话状态的时间间隔，以秒为单位。有效值范围（1~60）
LOCK_TID_MODE	1	动态，系统级	SELECT FOR UPDATE 封锁方式。0：结果集记录小于 100 行，直接封锁 TID，否则升级为表锁； 1：不升级表锁,一律使用 TID 锁
DML_ONLY	0	静态	0 表示操作没有任何限制；1 表示 DDL FORBIDDEN，支持所有 DML 操作（包括 HUGE 表），支持 FAST LOADER，不支持 DDL 操作；2 表示 DML ONLY，仅支持普通行表的 DML 操作，不支持 DDL 操作，不支持 FAST LOADER，不支持 HUGE 表的 DML 操作。仅支持在 MOUNT 系统状态下修改此参数值
NOWAIT_WHEN_UNIQUE_CONFLICT	0	静态	插入数据时，如果和未提交数据有 UNIQUE 约束的冲突，是否等待未提交事务结束，0：等待，直至未提交事务结束；1：不等待，立即返回错误
UNDO_EXTENT_NUM	16	静态	表示系统启动时，为每个工作线程分配的回滚簇个数。有效值范围（1~ 256）
MAX_DE_TIMEOUT	10	动态，会话级	C、JAVA 外部函数的执行超时时间，以秒为单位。有效值范围（1~3600）
TRANSACTIONS	75	静态	指定一个会话中可以并发的自治事务数量。有效值范围（1~1000）
MVCC_RETRY_TIM	5	静态	指定发生 MVCC 冲突时的最大重试次数。有效

ES			值范围（1~4294967294）
ENABLE_FLASHBACK	0	动态，系统级	是否启用闪回查询，0：不启用；1：启用
UNDO_RETENTION	900	动态，系统级	事务提交后回滚页保持时间，单位为秒。有效值范围（0~ 86400）
UNDO_RETENTION_GUARANTEE	1	动态，系统级	是否需要保证已提交事务在 UNDO_RETENTION 设置时间后再 PURGE。1：保证；0：表示在系统压力比较大（系统监测内存使用情况，超过一定阈值）时，将 UNDO_RETENTION 时间减半，尽快启动 PURGE 动作，避免 PURGE 系统消耗大量内存
PURGE_WAIT_TIME	100	动态，系统级	检测到系统清理动作滞后（待清理事务提交时间-当前系统时间>UNDO_RETENTION）情况下，系统等待时间（MS）。有效范围（0～60000），0 表示不等待
PSEG_PAGE_OPT	0	动态，系统级	回滚页 PURGE 时优化，0：不使用；1：使用
PSEG_RECV	1	动态，系统级	系统故障重启时，是否回滚活动事务并 PURGE 已经提交事务。1：是；0：否，跳过回滚活动事务和 PURGE 已经提交事务的步骤。在回滚表空间出现异常、损坏、系统无法正常启动时，可将 PSEG_RECV 设置为 0，让系统启动；但存在一定风险，未提交事务的修改将无法回滚，破坏事务的原子性；另外，已提交未 PURGE 的事务，将导致部分存储空间无法回收。
ENABLE_IGNORE_PURGE_REC	1	动态，会话级	当返回 EC_RN_NREC_PURGED（-7120）错误（回滚记录版本太旧，无法获取用户记录）时的处理策略；1：报错；0：忽略这一条记录，继续执行
ROLL_ON_ERR	0	动态，系统级	服务器执行出错时的回滚策略选择，0：回滚当前语句；1：回滚整个事务
ENABLE_TMP_TAB_ROLLBACK	1	动态，系统级	临时表操作是否生成回滚记录，0：不生成；1：生成 注：置为 0 时，临时表的 DML 操作无法回滚
安全相关参数			
PWD_POLICY	2	动态，系统级	设置系统默认口令策略。0： 无策略；1： 禁止与用户名相同；2： 口令长度不小于 9；4： 至少包含一个大写字母（A-Z）；8 ： 至少包含一个数字（0－9）；16：至少包含一个标点符号（英文输入法状态下，除“和空格外的所有符号；若为其他数字，则表示配置值的和，如 3＝1+2，表示同时启用第 1 项和第 2 项策略。当 COMPATIBLE_MODE=1 时，PWD_POLICY 的实际值均为 0
ENABLE_ENCRYPT	0	静态	通讯加密所采用的方式。0：不加密；1：SSL 加密，此时如果没有配置好 SSL 环境，则通讯仍旧不加密；2：SSL 认证，不加密，此时如果

			服务器 SSL 环境没有配置则服务器无法正常启动，如果客户端 SSL 环境没有配置则无法连接服务器
ENABLE_UDP	0	静态	是否支持 UDP。0：否；1：单发单收；2：多发多收。缺省使用 TCP
UDP_MAX_IDLE	15	静态	UDP 最大等待时间，单位秒，超过则认为连接断开。取值范围(5~60)
UDP_BTU_COUNT	8	静态	UDP 最大不等待连续发送消息块数，当不连续差值大于它时则等待发送。取值范围(4~32)
ENABLE_AUDIT	0	动态，系统级	审计开关，0：关闭；1：打开普通审计；2 打开普通审计和实时审计
AUDIT_FILE_FULL_MODE	1	静态	审计文件满后的处理方式，1：删除文件；2：不删除文件，也不添加审计记录
AUDIT_MAX_FILE_SIZE	100	动态，系统级	审计文件的最大大小，以兆为单位。有效值范围（1~4096）
ENABLE_OBJ_REUSE	0	静态	是否支持客体重用，0：不支持；1：支持。（该参数设置仅安全版有效）
ENABLE_REMOTE_OSAUTH	0	静态	是否支持远程操作系统认证，0：不支持；1：支持。 该参数设置仅安全版有效
MSG_COMPRESS_TYPE	1	静态	与客户端的通信消息是否压缩，0：不压缩；1：压缩；2：系统自动决定每条消息是否压缩
ENABLE_STRICT_CHECK	0	静态	是否检查存储过程中 EXECUTE IMMEDIATE 语句的权限，1：表示检查；0：不检查
MAC_LABEL_OPTION	1	动态，系统级	用于控制SP_MAC_LABEL_FROM_CHAR过程的使用范围。 0：只有SSO可以调用 1：所有用户都可以调用 2：所有用户可以调用，但是非SSO用户不会主动创建新的LABEL
LDAP_HOST	空串	手动	LDAP服务器IP
COMM_ENCRYPT_NAME	空串	静态	消息加密算法名。如果为空则不进行通信加密；如果给的加密算法名错误，则用使用加密算法 DES_CFB。DM支持的加密算法名可以通过查询动态视图V$CIPHERS获取
COMM_VALIDATE	0	动态，系统级	是否对消息进行校验。0：不检验；1：检验
兼容性相关参数			
BACKSLASH_ESCAPE	0	动态，会话级	语法分析对字符串中的反斜杠是否需要进行转义处理，0：不进行转义处理；1：进行转义处理
STR_LIKE_IGNORE_MATCH_END_SPACE	1	动态，会话级	LIKE 运算中是否忽略匹配串的结尾 0。0：不忽略；1：忽略
CLOB_LIKE_MAX_LEN	32	静态	LIKE 语句中 CLOB 类型的最大长度，单位 KB，有效值范围（8~102400）
EXCLUDE_DB_NAME	空串	静态	服务器可以忽略的数据库名列表，各数据库名以逗号“,”隔开，数据库名不需要加引号。

MS_PARSE_PERMIT	0	静态	是否支持 MS SQLSERVER 的语法。 0：不支持；1：支持； 2：在 MS_PARSE_PERMIT =1 的基础上，兼容 MS SQLSERVER 的查询项中支持“标识符=列名”或“@变量名=列名”用法。 当 COMPATIBLE_MODE=3 时，MS_PARSE_PERMIT 的实际值为 1
COMPATIBLE_MODE	0	静态	是否兼容其他数据库模式。0：不兼容，1：兼容 SQL92 标准，2：兼容 ORACLE，3：兼容 MS SQL SERVER，4：兼容 MYSQL，5：兼容 DM6，6：兼容 TERADATA
CASE_COMPATIBLE_MODE	1	动态，系统级	涉及不同数据类型的 CASE 运算，是否需要兼容 ORACLE 的处理策略。0：不兼容；1：兼容
EXCLUDE_RESERVED_WORDS	空串	静态	语法解析时，需要去除的保留字列表，保留字之间以逗号分隔
COUNT_64BIT	1	动态，会话级	COUNT 集函数的值是否设置为 BIGINT。0：否；1：是
CALC_AS_DECIMAL	0	静态	0：默认值，表示整数类型的除法、整数与字符或 BINARY 串的所有四则运算，结果都处理成整数 1：表示整数类型的除法全部转换为 DEC(0，0)处理 2：表示将整数与字符或 BINARY 串的所有四则运算都转换为 DEC(0，0)处理 备注：该参数只有在 USE_PLN_POOL 为 0 或 1 时有效。当 USE_PLN_POOL 为 2 或 3 时，按照 CALC_AS_DECIMAL=2 处理
CMP_AS_DECIMAL	0	静态	0：默认值，表示不对字符串与整型的比较结果类型做任何修改 1：表示字符串与整型转换为 BIGINT 比较数据溢出时，则转换为 DEC 进行比较 2：表示字符串与整型的比较强制转换为 DEC 进行比较
CAST_VARCHAR_MODE	1	动态，系统级	在字符串向整型转换进行比较时，是否将溢出值转为特殊伪值。0：否；1：是
PL_SQLCODE_COMPATIBLE	0	静态	默认值为 0；如果设置为 1，则 PL 的异常处理中，SQLCODE 的错误码值需要尽量与 ORACLE 一致
LEGACY_SEQUENCE	0	动态，系统级	序列兼容参数，0：与 ORACLE 兼容；1：与旧版本 DM 兼容，不推荐使用
DROP_CASCADE_VIEW	0	动态，会话级	删除表或者视图的时候级联删除视图，0：只删除表或者视图；1：删除表或者视图时删除关联的视图，当 COMPATIBLE_MODE=1 时，DROP_CASCADE_VIEW 的实际值为 1
用户请求跟踪相关参数			
SQL_TRACE_MASK	1	动态，系统级	LOG 记录的语句类型掩码，是一个格式化的字符串，表示一个 32 位整数上哪一位将被置为 1，

			置为 1 的位则表示该类型的语句要记录，格式为：位号:位号:位号。列如：3:5:7 表示第 3，第 5，第 7 位上的值被置为 1。每一位的含义见下面说明： 1 全部记录(全部记录并不包含原始语句) 2 全部 DML 类型语句 3 全部 DDL 类型语句 4 UPDATE 类型语句(更新) 5 DELETE 类型语句(删除) 6 INSERT 类型语句(插入) 7 SELECT 类型语句(查询) 8 COMMIT 类型语句(提交) 9 ROLLBACK 类型语句(回滚) 10 CALL 类型语句(过程调用) 11 BACKUP 类型语句(备分) 12 RESTORE 类型语句(恢复) 13 表对像操作(TABLE DDL) 14 视图对像操作(VIEW DDL) 15 过程或函数对像操作(PROCEDURE DDL) 16 触发器对像操作(TRIGGER DDL) 17 序列对像操作(SEQUEN DDL) 18 模式对像操作(SCHEMA DDL) 19 库对像操作(DATABAE DDL) 20 用户对像操作(USER DDL) 21 索引对像操作(INDEX DDL) 22 绑定参数 23 存在错误的语句（语法错误，语义分析错误等） 24 是否需要记录执行语句 25 是否需要打印计划和语句和执行的时间 26 是否需要记录执行语句的时间 27 原始语句（服务器从客户端收到的未加分析的语句）
SVR_LOG_FILE_NUM	0	动态，系统级	总共记录多少个日志文件，当日志文件达到这个设定值以后，再生成新的文件时，会删除最早的那个日志文件，日志文件的命令格式为 LOG_COMMIT_时间.LOG。当这个参数配置成 0 时，则按传统的日志文件记录，也就是 LOG_COMMIT01.LOG 和 LOG_COMMIT02.LOG 相互切换着记录。有效值范围（0~4294967294）
SVR_LOG	0	动态，系统级	是否打开 SQL 日志功能，0：表示关闭；1：表示打开。
SVR_LOG_NAME	空串	动态，系统级	使用 SQLLOG.INI 中预设的模式的名称
SVR_LOG_SWITCH_COUNT	100000	动态，系统级	一个日志文件中的 SQL 记录条数达到多少条之后系统会自动将日志切换到另一个文件中。有

			效值范围（1000~ 10000000）
SVR_LOG_ASYNC_FLUSH	0	动态，系统级	是否打开异步 SQL 日志功能，0：表示关闭；1：表示打开
SVR_LOG_MIN_EXEC_TIME	0	动态，系统级	详细模式下，记录的最小语句执行时间，单位为毫秒。执行时间小于该值的语句不记录在日志文件中。有效值范围（0~ 4294967294）
SVR_LOG_FILE_PATH	..\LOG	动态，系统级	日志文件所在的文件夹路径
SVR_LOG_BUF_TOTAL_SIZE	10240	动态，系统级	SQL 日志 BUFFER 占用空间的上限，单位为 KB，取值范围 1024~1024000
SVR_LOG_BUF_SIZE	1024	动态，系统级	一块 SQL 日志 BUFFER 的空间大小，单位为 KB，取值范围 50~409600
SVR_LOG_BUF_KEEP_CNT	6	动态，系统级	系统保留的 SQL 日志缓存的个数，有效值范围（1~ 100）
系统跟踪相关参数			
GDB_THREAD_INFO	1	静态	系统强制 HALT 时，是否打印线程堆栈信息到日志文件中。0：不打印；1：打印
TRACE_PATH	DM.INI 文件所在目录	手动	存放系统 TRACE 文件的路径
MONITOR 监控相关参数			
ENABLE_MONITOR	2	动态，系统级	启用动态监控功能标记，0：不启用；1：低级别监控；2：高级别监控；3：在高级别监控的基础上增加搜集各操作符的执行时间 仅支持在 MOUNT 系统状态下修改此参数值
MEMORY_MONITOR	0	静态	是否进行内存泄露监控。0：不进行；1：进行
ENABLE_FREQROOTS	0	静态	是否启用常用B树地址收集。0：不启用；1：启用
MAX_FREQ_ROOTS	200000	静态	收集常用B树地址的最大数量，仅在ENABLE_FREQROOTS为1时有效，有效值范围（0~10000000）
MIN_FREQ_CNT	100000	静态	最常使用 B 树地址的阈值，仅在 ENABLE_FREQROOTS 为 1 时有效，有效值范围（0~10000000）
数据守护相关参数			
DW_UDP_PORT	0	手动	守护进程的 UDP 端口号，需要和 DMWATCH.INI 中的同名配置项保持一致。有效值范围（1024~ 65534），为 0 时表示没有配置 UDP 通信方式的守护进程，不允许和 DW_PORT 同时配置
INST_UDP_PORT	0	手动	实例接收守护进程消息的 UDP 端口号，需要和 DMWATCH.INI 中的同名配置项保持一致。有效值范围（1024~ 65534），为 0 时表示没有配置 UDP 通信方式的守护进程，不允许和 DW_PORT 同时配置
DW_ERROR_TIME	60	动态，系统级	服务器认定守护进程未启动的时间，有效值范围（0~1800），单位为 S，默认 60S，距离服务

			器上次收到守护进程消息的时间间隔在设定的时间范围内，则认为守护进程处于活动状态，不允许手工执行 SQL 语句，超过设定时间仍没有收到守护进程消息，认为守护进程未启动，允许手工方式执行 SQL 语句
DW_PORT	0	手动	服务器和守护进程之间的 TCP 通信端口，服务器监听此端口，接收守护进程的 TCP 连接请求。有效值范围（1024~ 65534），为 0 时表示没有配置 TCP 通信方式的守护进程，不允许和 DW_UDP_PORT/INST_UDP_PORT 同时配置
ALTER_MODE_STATUS	1	动态，系统级	是否允许手工修改服务器的模式和状态，1：允许；0：不允许。 数据守护环境下建议配置为 0，实例处于主机或备机模式后，不允许用户直接通过 SQL 语句修改服务器的模式和状态
ENABLE_OFFLINE_TS	1	动态，系统级	是否允许 OFFLINE 表空间，0：不允许；1：允许；2：备库不允许。 数据守护环境下建议配置为 2
SESS_FREE_IN_SUSPEND	60	动态，系统级	远程归档失败会导致系统挂起，为了防止应用的连接一直挂住不切换到新主库，设置该参数，表示归档失败挂起后隔一段时间自动断开所有连接。有效值范围（0-1800），单位为 S，0 表示不断开。
全文索引相关参数			
CTI_HASH_SIZE	100000	动态，会话级	使用全文索引查询时，用于设置关键字匹配的哈希表大小。有效值范围（1000~10000000）
CTI_HASH_BUF_SIZE	50	动态，会话级	使用全文索引查询时，用于设置哈希缓存内存大小，以 M 为单位。有效值范围(1~4000)
USE_FORALL_ATTR	0	动态，会话级	是否使用 FORALL 语句的游标属性，0：不使用；1：使用
ALTER_TABLE_OPT	0	动态，会话级	是否对加列、修改列、删除列操作进行优化，0：全部不优化；1：全部优化；2：打开快速加列，对于删除列和修改列与1等效
DCP 相关参数			
ENABLE_DCP_MODE	0	静态	服务器是否作为 DCP 集群代理运行，0：否；1：是
DCP_PORT_NUM	5237	静态	DCP 管理端口号
DCP_CONN_POOL_SIZE	1000	动态，系统级	DCP 连接池大小，有效值范围（1~100000）
配置文件相关参数			
MAL_INI	0	静态	是否启用 MAL 系统，0：不启用；1：启用
ARCH_INI	0	动态，系统级	是否启用归档，0：不启用；1：启用
REP_INI	0	静态	是否启用复制，0：不启用；1：启用
LLOG_INI	0	静态	是否启用逻辑日志，0：不启用；1：数据复制使用
TIMER_INI	0	静态	是否启用定时器，0：不启用；1：启用

MPP_INI	0	静态	是否启用 MPP 系统，0：不启用；1：启用
其他			
IDLE_MEM_THRESHOLD	50	动态，系统级	可用物理内存的报警阀值，单位为兆，有效值范围（10~6000）
IDLE_DISK_THRESHOLD	1000	动态，系统级	磁盘可用空间的报警阀值，单位为兆，有效值范围（50~50000）
IDLE_SESS_THRESHOLD	5	动态，系统级	有限的会话数阀值，有效值范围（1~10）
ENABLE_PRISVC	0	静态	是否启用服务优先级的功能，0：不启用；1：启用
RAC 相关参数			
RAC_N_CTLS	10000	静态	LBS/GBS 控制页数目。有效值范围（10000~4294967294）
RAC_N_POOLS	1	静态	LBS/GBS 池数目。有效值范围（1~1024）
DCR_PATH	空串	静态	指定 DCR 磁盘的路径。配置 RAC 环境/或者使用 ASM 环境需要配置
HA_INST_CHECK_FLAG	1	手动	是否启用 DMINST.SYS 文件检测多个实例同时启动同一份数据库数据，0：不启用；1：启用
HA_INST_CHECK_TIME	3	手动	DMINST.SYS 文件检测校验次数，有效值范围（3~180）
HA_INST_CHECK_IP	需要时指定	手动	HA 实例启动检测 IP，是指 HA 系统中另外一台机器的 IP 地址，考虑到多网卡情况，最多允许配置 5 个 IP。如果需要配置多个 IP，则编辑多个配置项即可，比如： HA_INST_CHECK_IP = 192.168.0.8 HA_INST_CHECK_IP = 192.168.0.9
HA_INST_CHECK_PORT	需要时指定	手动	HA 实例监听端口，数据库实例启动后，监听此端口的连接请求，并使用此端口号连接 HA_INST_CHECK_IP 判断另外一个 HA 实例是否已经启动。 只允许配置一个监听端口，所有 HA_INST_CHECK_IP 使用同一个端口进行检测，有效值范围(1024 ~ 65533)

2.1.1.2 dmmal.ini

dmmal.ini 是 MAL 系统的配置文件。dmal.ini 的配置项见表 2.2。需要用到 MAL 环境的实例，所有站点 dmmal.ini 需要保证严格一致。

表 2.2 dmmal.ini 的配置项

项目	项目意义	字段	字段意义
MAL_CHECK_INTERVAL	检测线程检测间隔，默认 30S，范围（0-1800），如果配置为 0，则表示不进行链路检测	无	无

MAL_CONN_FAIL_INTERVAL	检测线程认定链路断开的时间，默认 10S，范围（2-1800）	无	无
MAL_LEAK_CHECK	是否打开 MAL 内存泄露检查（0:关闭，1：打开），默认 0	无	无
MAL_LOGIN_TIMEOUT	MPP/DBLINK 等实例间登录时的超时检测间隔（3-1800），以秒为单位，默认 15s	无	无
MAL_UTHR_FLAG	是否使用用户态线程标记。1，使用；0，不使用。默认为 0	无	无
MAL_BUF_SIZE	单个 MAL 缓存大小限制，以兆为单位。当此 MAL 的缓存邮件超过此大小，则会将邮件存储到文件中。有效值范围（0~500000），默认为 100	无	无
MAL_SYS_BUF_SIZE	MAL 系统总内存大小限制，单位：M。有效值范围（0~500000），默认为 0，表示 MAL 系统无总内存限制	无	无
MAL_VPOOL_SIZE	MAL 系统使用的内存初始化大小，以兆为单位。有效值范围（1~500000），默认为 10，此值一般要设置的比 MAL_BUF_SIZE 大一些	无	无
MAL_COMPRESS_LEVEL	Mal 消息压缩等级，取值范围 0-10。0 代表不进行消息压缩，为默认值 1 – 9 表示采用 lz 算法，从 1 到 9 表示压缩速度依次递减，压缩率依次递增。 10 表示采用 QUICKLZ 算法，压缩速度高于 LZ 算法，压缩率相对低	无	无
[MAL_INST1]	实例配置	MAL_INST_NAME	实例名。MAL 系统中的各数据库实例不允许同名。
		MAL_HOST	实例主库
		MAL_PORT	监听端口
		MAL_INST_PORT	数据库实例服务器的端口，对于每个实例，一定要和对应 DM.INI 中的 PORT_NUM 保持一样。（在 MPP 下以及读写分离的即时归档主备系统下一定要配置）

		MAL_INST_HOST	连接数据库服务器使用的 IP 地址。(在 MPP 下以及读写分离的即时归档主备系统下一定要配置)
		MAL_DW_PORT	服务器对应的守护进程使用的 TCP 端口，用于守护进程之间，以及守护进程和监视器之间的 TCP 通信（控制文件方式的数据守护环境下需要配置)。

注意：dmmal.ini 和 dm.ini 中都可配置 MAL_LEAK_CHECK，启动时以 dmmal.ini 中设置为准，如果 dmmal.ini 中没有配置，则以 dm.ini 中配置为准，两个都没配置时，则默认 0。此参数在 dm.ini 中可动态更改。

2.1.1.3 dmarch.ini

dmarch.ini 用于本地归档和远程归档。dmarch.ini 的配置项见表 2.3。

表 2.3 dmarch.ini 的配置项

项目	项目意义	字段	字段意义
		ARCH_WAIT_APPLY	是否需要等待备库做完日志，1 表示需要，0 表示不需要，默认为 1
[ARCHIVE_LOCAL1]	本地归档配置	ARCH_TYPE	归档类型
		ARCH_DEST	归档路径
		ARCH_FILE_SIZE	归档文件大小，单位 MB，取值范围(64~2048)
		ARCH_SPACE_LIMIT	归档文件限制，单位 MB，取值范围（1024~4294967294)，0 表示无空间限制
ARCHIVE_REALTIME	实时归档配置	ARCH_TYPE	归档类型
		ARCH_DEST	归档目标
ARCHIVE_ASYNC	异步归档	ARCH_TYPE	归档类型
		ARCH_DEST	归档目标
		ARCH_TIMER_NAME	定时器名称
ARCHIVE_TIMELY	即时归档	ARCH_TYPE	归档类型
		ARCH_DEST	归档目标
MARCH	MPP 归档配置	ARCH_TYPE	归档类型
		ARCH_DEST	归档目标

相关说明

1. 归档类型 ARCH_TYPE 有以下几种：
   - 本地归档 LOCAL（一台主库最多配 8 个）
   - 远程实时归档 REALTIME（一台主库只能配 1 个）
   - 远程异步归档 ASYNC（一台主库最多配 8 个）
   - 即时归档 TIMELY（一个主库最多配 8 个）
   - MPP 归档 MARCH（一个主库最多配 8 个）
2. 配置名[ARCHIVE_*]表示归档名，在配置文件中必须唯一。其中归档类型为

REALTIME，SYNC 只能有一个，其归档名必须为 ARCHIVE_REALTIME 和 ARCHIVE_SYNC。其他类型的归档可以多个；

3. 不能存在相同实例名的不同归档；
4. 不能存在 DEST 相同的不同归档实例；
5. ARCH_TIMER_NAME 为定制的定时器名称，定时器配置见 dmtimer.ini；
6. ARCH_FILE_SIZE 必须是整型，仅对本地归档有效，表示归档文件大小，单位是 M，值域为 64M ~ 2048M；
7. ARCH_SPACE_LIMIT 必须是整型，仅对本地归档有效，表示归档文件的磁盘空间限制，单位是 MB，值域为 1024~4294967294 或 0，0 表示无空间限制。如果归档文件总大小超过这个值，则在生成新归档文件前会删除最老的一个归档文件。

2.1.1.4 dm_svc.conf

DM 安装时生成一个配置文件 dm_svc.conf，在 Windows 操作平台下此文件位于%SystemRoot%\system32 目录，在 Linux 平台下此文件位于/etc 目录。

dm_svc.conf 文件中包含的配置项如表 2.4 所示。

表 2.4 dm_svc.conf 配置项介绍

配置项	缺省值	简述
服务名	无	连接服务名，参数值格式为 IP[:PORT],IP[:PORT],.....
TIME_ZONE	+8:00	指明客户端的默认时区，设置范围为：-12:59~+14:00
LANGUAGE	操作系统语言	当前数据库服务器使用的语言，会影响帮助信息错误和提示信息。支持的选项为：CN（表示中文）和 EN（表示英文）。可以不指定，若不指定，系统会读取操作系统信息获得语言信息，建议有需要才指定。
CHAR_CODE	操作系统编码格式	客户端使用的编码格式，会影响帮助信息和错误提示信息，要与客户端使用的编码格式一致。支持的选项为：PG_UTF8（表示 UTF8 编码）；PG_GBK/PG_GB18030（两者都表示 GBK 编码）；PG_BIG5（表示 BIG5 编码）；PG_ISO_8859_9（表示 ISO88599 编码）；PG_EUC_JP（表示 EUC_JP 编码）；PG_EUC_KR（表示 EUC_KR 编码）；PG_KOI8R（表示 KOI8R 编码）；PG_ISO_8859_1（表示 ISO_8859_1 编码）。 可以不指定，若不指定，系统会读取操作系统信息获得编码信息，建议有需要才指定。
COMPRESS_MSG	0	是否启用消息压缩。0：不启用；1：启用
LOGIN_ENCRYPT	1	是否进行通信加密。0：不加密；1：加密
DIRECT	Y	是否使用快速装载。Y：使用；N：不使用
DEC2DOUB	0	指明在 DPI、DMODBC、DCI、DMPHP 和 DM PRO*C 中，是否将 DEC 类型转换为 DOUBLE 类型。0：不转换；1：转换
KEYWORDS	无	标识用户关键字，所有在列表中的字符串，如果以单词的形式出现在 SQL 语句中，则这个单词会被加上双引号。该参数主要用来解决用户需要使用 DM7 中的保留字作为对象名使用的状况。
ENABLE_RS_CACHE	0	是否进行客户端结果集缓存。0：不进行；1：进行
RS_CACHE_SIZE	10	设置结果集缓冲区大小，以 M 为单位。有效值为 1~65535，如果设置太大，可能导致空间分配失败，进而使缓存失效
RS_REFRESH_FREQ	10	结果集缓存检查更新的频率，以秒为单位，有效值为 0~10000，如果设置为 0，则不需检查更新

LOGIN_MODE	0	是否仅登录到主库或备库。0：主库不存在的情况下可连接备库；1：只连接主库；2：只连接备库
SWITCH_TIME	3	在服务器之间切换的次数，有效值范围 1~9223372036854775807
SWITCH_INTERVAL	200	在服务器之间切换的时间间隔，单位为毫秒，有效值范围 1~9223372036854775807
RW_SEPARATE	0	是否启用读写分离。0：不启用；1：启用
RW_PERCENT	25	读写分离分发比例，有效值范围 0~100
dexp 配置项		
DUMMY	2	写文件时，发现文件已存在的处理方式。0：报错；1：直接覆盖文件；2：询问用户，有交互信息
dpc_new 配置项		
DPC_TRACE	无	dpc_new 的 TRACE 文件路径，不配置时不写 TRACE 信息
ENABLE_SSL	0	是否启用 SSL。1/Y/y：启用 SSL；其他值：不启用
SSL_CONFIG	无	在启用 SSL 的前提下，设置 SSL 值，格式如下：SSL_CONFIG=((USER=(用户名 1) SSL_PATH=(SSL 路径 1) SSL_PWD=(SSL KEY1)) (USER=(用户名 2) SSL_PATH=(SSL 路径 2) SSL_PWD=(SSL KEY2)))

dm_svc.conf 配置文件的内容分为全局配置区和服务配置区。全局配置区在前，可配置表 2.4 中所有的配置项，服务配置区在后，以“[服务名]”开头，可配置除了服务名外的所有配置项。服务配置区中的配置优先级高于全局配置区。

下面是一个 dm_svc.conf 的例子：

```
# 以#开头的行表示是注释
# 全局配置区
O2000=(192.168.0.1:5000,192.168.0.2:5236)
O3000=(192.168.0.1:5236,192.168.0.3:4350)
TIME_ZONE=(+8:00)
LOGIN_ENCRYPT=(0)
DIRECT=(Y)

# 服务配置区
[O2000]
TIME_ZONE=(+9:00)
LOGIN_PRIMARY=(2)
SWITCH_TIME=(3)
SWITCH_INTERVAL=(10)
```

2.1.1.5 sqllog.ini

sqllog.ini 用于 sql 日志的配置。

SQL_TRACE_MASK	1	动态，系统级	LOG 记录的语句类型掩码，是一个格式化的字符串，表示一个 32 位整数上哪一位将被置为 1，置为 1 的位则表示该类型的语句要记录，格式为：位号:位号:位号。列如：3:5:7 表示第 3，第 5，第 7 位上的值被置为 1。每一位的含义见下面说明（2~17 前提是:SQL 标记位 24 也要置)：：

			1 全部记录(全部记录并不包含原始语句) 2 全部 DML 类型语句 3 全部 DDL 类型语句 4 UPDATE 类型语句(更新) 5 DELETE 类型语句(删除) 6 INSERT 类型语句(插入) 7 SELECT 类型语句(查询) 8 COMMIT 类型语句(提交) 9 ROLLBACK 类型语句(回滚) 10 CALL 类型语句(过程调用) 11 BACKUP 类型语句(备分) 12 RESTORE 类型语句(恢复) 13 创建对象操作(CREATE DDL) 14 修改对象操作(ALTER DDL) 15 删除对象操作(DROP DDL) 16 授权操作(GRANT DDL) 17 回收操作(REVOKE DDL) 22 绑定参数 23 存在错误的语句（语法错误，语义分析错误等） 24 是否需要记录执行语句 25 是否需要打印计划和语句和执行的时间 26 是否需要记录执行语句的时间 27 原始语句（服务器从客户端收到的未加分析的语句） 28 是否记录参数信息，包括参数的序号、数据类型和值 29 是否记录事务相关事件
FILE_NUM	0	动态，系统级	总共记录多少个日志文件，当日志文件达到这个设定值以后，再生成新的文件时，会删除最早的那个日志文件，日志文件的命令格式为 LOG_COMMIT_时间.LOG。当这个参数配置成 0 时，则按传统的日志文件记录，也就是 LOG_COMMIT01.LOG 和 LOG_COMMIT02.LOG 相互切换着记录。有效值范围（0~1024）
SWITCH_MODE	0	手动	表示 SQL 日志文件切换的模式: 0：不切换 1：按文件中记录数量切换 2：按文件大小切换 3：按时间间隔切换
SWITCH_LIMIT	100000	动态，系统级	不同切换模式 SWITCH_MODE 下，意义不同： ◆ 按数量切换时，一个日志文件中的 SQL 记录条数达到多少条之后系统会自动将日志切换到另一个文件中。一个日志文件中的 SQL 记录条数达到多少条之后系统会自动将日志切换到

			另一个文件中。有效值范围（1000~ 10000000） ◆ 按文件大小切换时，一个日志文件达到该大小后，系统自动将日志切换到另一个文件中，单位为 M。有效值范围（1~ 2000） ◆ 按时间间隔切换时，每个指定的时间间隔，按文件新建时间进行文件切换，单位为分钟。有效值范围（1~ 30000）
ASYNC_FLUSH	0	动态，系统级	是否打开异步 SQL 日志功能。0：表示关闭；1：表示打开
MIN_EXEC_TIME	0	动态，系统级	详细模式下，记录的最小语句执行时间，单位为毫秒。执行时间小于该值的语句不记录在日志文件中。有效值范围（0~ 4294967294）
FILE_PATH	..\LOG	动态，系统级	日志文件所在的文件夹路径
BUF_TOTAL_SIZE	10240	动态，系统级	SQL 日志 BUFFER 占用空间的上限，单位为 KB，取值范围（1024~1024000）
BUF_SIZE	1024	动态，系统级	一块 SQL 日志 BUFFER 的空间大小，单位为 KB，取值范围（50~409600）
BUF_KEEP_CNT	6	动态，系统级	系统保留的 SQL 日志缓存的个数，有效值范围（1~ 100）
PART_STOR	0	手动	SQL 日志分区存储，表示 SQL 日志进行分区存储的划分条件。 1 表示 USER：USER 根据不同用户分布存储
ITEMS	0	手动	配置 SQL 日志记录中的那些列要被记录。 该参数是一个格式化的字符串，表示一个记录中的那些项目要被记录，格式为：列号:列号:列号。列如：3:5:7 表示第 3，第 5，第 7 列要被记录。0 表示记录所有的列 1 TIME 执行的时间 2 SEQNO 服务器的站点号 3 SESS 操作的 SESS 地址 4 USER 执行的用户 5 TRXID 事务 ID 6 STMT 语句地址 7 APPNAME 客户端工具 8 IP 客户端 IP 9 STMT_TYPE 语句类型 10 INFO记录内容 11 RESULT 运行结果，包括运行用时和影响行数（可能没有）
USER_MODE	0	手动	SQL 日志按用户过滤时的过滤模式，取值 0：关闭用户过滤 1：白名单模式，只记录列出的用户操作的 SQL 日志

			2：黑名单模式，列出的用户不记录 SQL 日志
USERS	空串	手动	打开 SVR_LOG_USER_MODE 时指定的用户列表。格式为：用户名：用户名：用户名

2.1.2 复制配置

2.1.2.1 dmrep.ini

dmrep.ini 用于配置复制实例，具体内容请见“数据复制”章节。dmrep.ini 的配置项见表 2.6。

表 2.6 dmrep.ini 的配置项

项目	项目意义	字段	字段意义
[REP_RPS_INST_NAME]	该复制结点所属的复制服务器的实例名		
[REP_MASTER_INFO]	作为主服务器的相关信息，其中一条记录就是一个复制关系	REP_ID	复制关系的 ID；同时也是逻辑日志的 ID，由复制服务器生成
[REP_SLAVE_INFO]	作为从服务器的相关信息，其中一条记录就是一个复制关系	REP_ID	复制关系的 ID
		MASTER_INSTNAME	复制关系主服务器的实例名
		ADD_TICK	复制关系创建的时间，用于检验复制关系的正确性
[REP_SLAVE_TAB_MAP]	作为从服务器时，记录的复制映射的信息，其中一条记录就是一个复制映射	REP_ID	映射所属的复制的 ID
		SRC_TAB_ID	复制源对象表的 ID
		DST_TAB_ID	复制目标对象表的 ID
		READONLY_MODE	1：只读模式，0：非只读模式
[REP_SLAVE_SRC_COL_INFO]	作为从服务器时，记录的复制映射中使用的复制源对象的列信息	REP_ID	所属映射所在的复制关系的 ID
		SRC_TAB_ID	所属映射的复制源对象的表 ID
		COL_ID	列 ID
		SQL_PL_TYPE	列类型
		LEN	长度
		PREC	精度

2.1.2.2 dmllog.ini

dmllog.ini 用于配置逻辑日志，具体内容请见“数据复制”章节。dmllog.ini 的配置项见表 2.7。

表 2.7 dmllog.ini 的配置项

项目	项目意义	字段	字段意义
[LLOG_INFO]	逻辑日志的整体信息	ID	逻辑日志 ID，若参与复制，则与复制关系 ID 相同；-1 全局逻辑日志
		LLOG_PATH	逻辑日志本地归档路径
		LOCAL_ARCH_SPACE_LIMIT(G)	本地归档文件的空间限制，以 G 为单位，-1 表示无限制
		REMOTE_ARCH_FLAG	逻辑日志远程归档是否有效标识，0 表示远程归档无效；1 表示远程归档有效
		REMOTE_ARCH_INSTNAME	远程归档的归档实例名，若参与复制，即为从服务器的实例名
		REMOTE_ARCH_TYPE	远程归档的类型，同步或异步
		REMOTE_ARCH_TIMER	远程归档使用的定时器，同步时无意义
[LLOG_TAB_MAP]	逻辑日志记录的对象的信息	LLOG_ID	逻辑日志的 ID
		SCH_ID	逻辑日志记录的模式 ID
		TAB_ID	逻辑日志记录的表 ID

2.1.2.3 dmtimer.ini

dmtimer.ini 用于配置定时器，记录异步复制的定时器信息，具体使用情况请见“数据复制”章节。dmtimer.ini 的配置项见表 2.8。

表 2.8 dmtimer.ini 的配置项

项目	项目意义	字段	字段意义
[TIMER_NAME1]	定时器信息	TIMER_NAME	定时器名称
		TYPE	定时器调度类型： 1：执行一次 2：按日执行 3：按周执行 4：按月执行的第几天 5：按月执行的第一周 6：按月执行的第二周 7：按月执行的第三周 8：按月执行的第四周 9：按月执行的最后一周
		FREQ_MONTH_WEEK_INTERVAL	间隔月或周数
		FREQ_SUB_INTERVAL	间隔天数
		FREQ_MINUTE_INTERVAL	间隔分钟数
		START_TIME	开始时间
		END_TIME	结束时间
		DURING_START_DATE	开始时间点
		DURING_END_DATE	结束时间点
		NO_END_DATE_FLAG	是否结束标记
		DESCRIBE	定时器描述
		IS_VALID	有效标记

2.2 控制文件

每个 DM 数据库都有一个名为 dm.ctl 的控制文件。控制文件是一个二进制文件，它记录了数据库必要的初始信息，其中主要包含以下内容：

1. 数据库名称；
2. 数据库服务器模式；
3. OGUID 唯一标识；
4. 数据库服务器版本；
5. 数据文件版本；
6. 表空间信息，包括表空间名，表空间物理文件路径等，记录了所有数据库中使用的表空间，数组的方式保存起来；
7. 控制文件校验码，校验码由数据库服务器在每次修改控制文件后计算生成，保证控制文件合法性，防止文件损坏及手工修改。

2.3 数据文件

数据文件以 dbf 为扩展名，它是数据库中最重要的文件类型，一个 DM 数据文件对应磁盘上的一个物理文件，数据文件是真实数据存储的地方，每个数据库至少有一个与之相关的数据文件。在实际应用中，通常有多个数据文件。

当 DM 的数据文件空间用完时，它可以自动扩展。可以在创建数据文件时通过 MAXSIZE 参数限制其扩展量，当然，也可以不限制。但是，数据文件的大小最终会受物理磁盘大小的限制。在实际使用中，一般不建议使用单个巨大的数据文件，为一个表空间创建多个较小的数据文件是更好的选择。

数据文件在物理上按照页、簇和段的方式进行管理，详细结构请参考第一章的内容。

数据文件按数据组织形式，可以分为如下几种：

1. B 树数据

行存储数据，也是应用最广泛的存储形式，其数据是按 B 树索引组织的。普通表、分区表、B 树索引的物理存储格式都是 B 树。

一个 B 树包含两个段，一个内节点段，存放内节点数据；一个叶子段，存放叶子节点数据。其 B 树的逻辑关系由段内页面上的记录，通过文件指针来完成。

当表上没有指定聚簇索引时，系统会自动产生一个唯一标识 rowid 作为 B 树的 key 来唯一标识一行。

2. 堆表数据

堆表的数据是以挂链形式存储的，一般情况下，支持最多 128 个链表，一个链表在物理上就是一个段，堆表采用的是物理 rowid，在插入过程中，rowid 在事先已确定，并保证其唯一性，所以可以并发插入，插入效率很高，且由于 rowid 是即时生成，无需保存在物理磁盘上，也节省了空间。

3. 列存储数据

数据按列方式组织存储，每个列包含 2 个段，一个段存放列数据，一个段存放列的控制信息，读取列数据时，只需要顺序扫描这两个段。在某些特殊应用场景下，其效率要远远高于行存储。

4. 位图索引

位图索引与 B 树索引不同，每个索引条目不是指向一行数据，而是指向多行数据。每个索引项保存的是一定范围内所有行与当前索引键值映射关系的位图。

数据文件中还有两类特殊的数据文件：ROLL 和 TEMP 文件。

1). ROLL 文件

ROLL 表空间的 dbf 文件，称为 ROLL 文件。ROLL 文件用于保存系统的回滚记录，提供事务回滚时的信息。回滚文件整个是一个段。每个事务的回滚页在回滚段中各自挂链，页内则顺序存放回滚记录。

2). TEMP 文件

TEMP.DBF 临时数据文件，临时文件可以在 dm.ini 中通过 TEMP_SIZE 配置大小。

当数据库查询的临时结果集过大，缓存已经不够用时，临时结果集就可以保存在 TEMP.DBF 文件中，供后续运算使用。系统中用户创建的临时表也存储在临时文件中。

2.4 重做日志文件

重做日志，又叫 REDO 日志，指在 DM 数据库中添加、删除、修改对象，或者改变数据，DM 都会按照特定的格式，将这些操作执行的结果写入到当前的重做日志文件中。重做日志文件以 log 为扩展名。每个 DM 数据库实例必须至少有 2 个重做日志文件，默认两个日志文件为 DAMENG01.log、DAMENG02.log，这两个文件循环使用。

重做日志文件主要用于数据库的备份与恢复。理想情况下，数据库系统不会用到重做日志文件中的信息。然而现实世界总是充满了各种意外，比如电源故障、系统故障、介质故障，或者数据库实例进程被强制终止等，数据库缓冲区中的数据页会来不及写入数据文件。这样，在重启 DM 实例时，通过重做日志文件中的信息，就可以将数据库的状态恢复到发生意外时的状态。

重做日志文件对于数据库是至关重要的。它们用于存储数据库的事务日志，以便系统在出现系统故障和介质故障时能够进行故障恢复。在 DM 数据库运行过程中，任何修改数据库的操作都会产生重做日志，例如，当一条元组插入到一个表中的时候，插入的结果写入了重做日志，当删除一条元组时，删除该元组的事实也被写了进去，这样，当系统出现故障时，通过分析日志可以知道在故障发生前系统做了哪些动作，并可以重做这些动作使系统恢复到故障之前的状态。

2.5 归档日志文件

日志文件分为联机日志文件和归档日志文件。DM 数据库可以在归档模式和非归档模式下运行。只有当数据库处于归档模式下，才将联机日志文件中的内容保存到硬盘中，形成归档日志文件。

重做日志文件就是联机日志文件。联机日志文件指的是系统当前正在使用的日志文件，创建数据库时，联机日志文件通常被扩展至一定长度，其内容则被初始化为空，当系统运行时，该文件逐渐被产生的日志所填充。对日志文件的写入是顺序连续的。然而系统磁盘空间总是有限，系统必须能够循环利用日志文件的空间，为了做到这一点，当所有日志文件空间被占满时，系统需要清空一部分日志以便重用日志文件的空间，为了保证被清空的日志所“保护”的数据在磁盘上是安全的，这里需要引入一个关键的数据库概念—检查点。当产生检查点时，系统将系统缓冲区中的日志和脏数据页都写入磁盘，以保证当前日志所“保护”的数据页都已安全写入磁盘，这样日志文件即可被安全重用。

归档日志文件，就是在归档模式下，重做联机日志被连续拷贝到归档日志后，所生成了归档日志文件。归档日志文件以归档时间命名，扩展名也是 log。但只有在归档模式下运行时，DM 数据库在重做联机日志文件时才能生成归档日志文件。

采用归档模式会对系统的性能产生影响，然而系统在归档模式下运行会更安全，当出现

故障时其丢失数据的可能性更小，这是因为一旦出现介质故障，如磁盘损坏时，利用归档日志，系统可被恢复至故障发生的前一刻，也可以还原到指定的时间点，而如果没有归档日志文件，则只能利用备份进行恢复。

归档日志还是数据守护功能的核心，数据守护中的备库就是通过重做日志来完成与主库的数据同步的。

2.6 逻辑日志文件

如果在 DM 数据库上配置了复制功能，复制源就会产生逻辑日志文件。逻辑日志文件是一个流式的文件，它有自己的格式，且不在第一章所述的页，簇和段的管理之下。

逻辑日志文件内部存储按照复制记录的格式，一条记录紧接着一条记录，存储着复制源端的各种逻辑操作。用于发送给复制目的端。详细内容请看“数据复制”章节。

2.7 备份文件

备份文件以 bak 为扩展名，当系统正常运行时，备份文件不会起任何作用，它也不是数据库必须有的联机文件类型之一。然而，从来没有哪个数据库系统能够保证永远正确无误地运行，当数据库不幸出现故障时，备份文件就显得尤为重要了。

当客户利用管理工具或直接发出备份的 SQL 命令时，DM Server 会自动进行备份，并产生一个或多个备份文件，备份文件自身包含了备份的名称、对应的数据库、备份类型和备份时间等信息。同时，系统还会自动记录备份信息及该备份文件所处的位置，但这种记录是松散的，用户可根据需要将其拷贝至任何地方，并不会影响系统的运行。

2.8 跟踪日志文件

用户在 dm.ini 中配置 SVR_LOG 和 SVR_LOG_SWITCH_COUNT 参数后就会打开跟踪日志。跟踪日志文件是一个纯文本文件，以“dm_commit_日期_时间”命名，默认生成在 DM 安装目录的 log 子目录下面，管理员可通过 ini 参数 SVR_LOG_FILE_PATH 设置其生成路径。

跟踪日志内容包含系统各会话执行的 SQL 语句、参数信息、错误信息等。跟踪日志主要用于分析错误和分析性能问题，基于跟踪日志可以对系统运行状态有一个分析，比如，可以挑出系统现在执行速度较慢的 SQL 语句，进而对其进行优化。

系统中 SQL 日志的缓存是分块循环使用，管理员可根据系统执行的语句情况及压力情况设置恰当的日志缓存块大小及预留的缓冲块个数。当预留块不足以记录系统产生的任务时，系统会分配新的用后即弃的缓存块，但是总的空间大小由 ini 参数 SVR_LOG_BUF_TOTAL_SIZE 控制，管理员可根据实际情况进行设置。

打开跟踪日志会对系统的性能会有较大影响，一般用于查错和调优的时候才会打开，默认情况下系统是关闭跟踪日志的。若需要跟踪日志但对日志的实时性没有严格的要求，又希望系统有较高的效率，可以设置参数 SQL_TRACE_MASK 和 SVR_LOG_MIN_EXEC_TIME 只记录关注的相关记录，减少日志总量；设置参数 SVR_LOG_ASYNC_FLUSH 打开 SQL 日志异步刷盘提高系统性能。

2.9 事件日志文件

DM 数据库系统在运行过程中，会在 log 子目录下产生一个“dm_实例名_日期”命名的事件日志文件。事件日志文件对 DM 数据库运行时的关键事件进行记录，如系统启动、关闭、内存申请失败、IO 错误等一些致命错误。事件日志文件主要用于系统出现严重错误时进行查看并定位问题。事件日志文件随着 DM 数据库服务的运行一直存在。

事件日志文件打印的是中间步骤的信息，所以出现部分缺失属于正常现象。

2.10 数据重演文件

调用系统存储过程 SP_START_CAPTURE 和 SP_STOP_CAPTURE，可以获得数据重演文件。重演文件用于数据重演，存储了从抓取开始到抓取结束时，DM 数据库与客户端的通信消息。使用数据重演文件，可以多次重复抓取这段时间内的数据库操作，为系统调试和性能调优提供了另一种分析手段。

第3章 DM 内存结构

数据库管理系统是一种对内存申请和释放操作频率很高的软件，如果每次对内存的使用都使用操作系统函数来申请和释放，效率会比较低，加入自己的内存管理是 DBMS 系统所必须的。通常内存管理系统会带来以下好处：

1. 申请、释放内存效率更高；
2. 能够有效地了解内存的使用情况；
3. 易于发现内存泄露和内存写越界的问题。

DM 数据库管理系统的内存结构主要包括内存池、缓冲区、排序区、哈希区等。根据系统中子模块的不同功能，对内存进行了上述划分，并采用了不同的管理模式。

3.1 内存池

DM Server 的内存池指的是共享内存池，根据内存使用情况的不同，对共享内存池的使用采用了两种工作方式：HEAP 和 VPOOL。

共享内存池用于解决 DM Server 对于小片内存的申请与释放问题。系统在运行过程中，经常会申请与释放小片内存，而向操作系统申请和释放内存时需要发出系统调用，此时可能会引起线程切换，降低系统运行效率。采用共享内存池则可一次向操作系统申请一片较大内存，即为内存池，当系统在运行过程中需要申请内存时，则在共享内存池内进行申请，当用完该内存时，再释放掉，即归还给共享内存池。当系统采用较好的策略管理共享内存池时，小片内存的申请与释放不会对系统影响太大。这种方式还有一个优点，可以比较容易地检测系统是否存在内存泄漏。

DM 系统管理员可以通过 DM Server 的配置文件（dm.ini）来对共享内存池的大小进行设置，共享池的参数为 MEMORY_POOL，该配置默认为 80M。而 HEAP 与 VPOOL 使用的是共享内存池中的内存，所以一般情况下，HEAP 与 VPOOL 两种方式对内存申请的大小不会超过 MEMORY_POOL 值。

3.1.1 HEAP

HEAP 工作方式采用了堆的思想。每次申请内存时，都是从堆顶上申请，如果不够，则继续向共享内存池中申请内存页，然后加入到 HEAP 中，继续供系统的申请使用，这样 HEAP 的长度可以无限增长下去；释放 HEAP 时，可以释放堆顶上的内存页，也可以释放整个 HEAP。使用内存堆来管理小片内存的申请有个特点，每次申请小片内存以后，不能单独对这片内存进行释放，也就是不用关心这片内存何时释放，它在堆释放时统一释放，能有效防止内存泄露的发生。

3.1.2 VPOOL

VPOOL 的工作方式主要是采用了“伙伴系统”的思想进行管理。申请的 VPOOL 内存分为私有和公有两种。私有 VPOOL 只提供给某个单独功能模块使用，公有 VPOOL 则提供给那些需要共享同一资源而申请的模块，所以对于公有 VPOOL 需要进行保护，而私有 VPOOL 则不需要。

VPOOL 和 HEAP 的区别在于，VPOOL 申请的出去的每片内存都可以单独的进行释放。

3.2 缓冲区

3.2.1 数据缓冲区

数据缓冲区是 DM Server 在将数据页写入磁盘之前以及从磁盘上读取数据页之后，数据页所存储的地方。这是 DM Server 至关重要的内存区域之一，将其设定得太小，会导致缓冲页命中率低，磁盘 IO 频繁；将其设定得太大，又会导致操作系统内存本身不够用。

系统启动时，首先根据配置的数据缓冲区大小向操作系统申请一片连续内存并将其按数据页大小进行格式化，并置入“自由”链中。数据缓冲区存在三条链来管理被缓冲的数据页，一条是“自由”链，用于存放目前尚未使用的内存数据页，一条是“LRU”链，用于存放已被使用的内存数据页(包括未修改和已修改)，还有一条即为“脏”链，用于存放已被修改过的内存数据页。

LRU 链对系统当前使用的页按其最近是否被使用的顺序进行了排序。这样当数据缓冲区中的自由链被用完时，从 LRU 链中淘汰部分最近未使用的数据页，能够较大程度地保证被淘汰的数据页在最近不会被用到，减少 IO。

在系统运行过程中，通常存在一部分“非常热”（反复被访问）的数据页，将它们一直留在缓冲区中，对系统性能会有好处。对于这部分数据页，数据缓冲区开辟了一个特定的区域用于存放它们，以保证这些页不参与一般的淘汰机制，可以一直留在数据缓冲区中。

3.2.1.1 类别

DM Server 中有四种类型的数据缓冲区，分别是 NORMAL、KEEP、FAST 和 RECYCLE。其中，用户可以在创建表空间或修改表空间时，指定表空间属于 NORMAL 或 KEEP 缓冲区。RECYCLE 缓冲区供临时表空间使用，FAST 缓冲区根据用户指定的 FAST_POOL_PAGES 和 FAST_ROLL_PAGES 大小由系统自动进行管理，用户不能指定使用 RECYCLE 和 FAST 缓冲区的表或表空间。

NORMAL 缓冲区主要是提供给系统处理的一些数据页，没有特定指定缓冲区的情况下，默认缓冲区为 NORMAL；KEEP 的特性是对缓冲区中的数据页很少或几乎不怎么淘汰出去，主要针对用户的应用是否需要经常处在内存当中，如果是这种情况，可以指定缓冲区为 KEEP。

DM Server 提供了可以更改这些缓冲区大小的参数，用户可以根据自己应用需求情况，指定 dm.ini 文件中 BUFFER(80MB)、KEEP(8MB)、RECYCLE(64MB)、FAST_POOL_PAGES(0)和 FAST_ROLL_PAGES(0)值（括号中为默认值），这些值分别对应是 NORMAL 缓冲区大小、KEEP 缓冲区大小、RECYCLE 缓冲区大小、FAST 缓冲区页面数和 FAST 缓冲区回滚页面数。

3.2.1.2 读多页

在需要进行大量 I/O 的应用当中，DM 之前版本的策略是每次只读取一页。如果知道用户需要读取表的大量数据，当读取到第一页时，可以猜测用户可能需要读取这页的下一页，在这种情况下，一次性读取多页就可以减少 I/O 次数，从而提高了数据的查询、修改效率。

DM Server 提供了可以读取多页的参数，用户可以指定这些参数来调整数据库运行效率的最佳状态。在 DM 配置文件 dm.ini 中，可以指定参数 MULTI_PAGE_GET_NUM 大小（默认值为 16 页），来控制每次读取的页数。

如果用户没有设置较适合的参数 MULTI_PAGE_GET_NUM 值大小，有时可能会给用户带来更差的效果。如果 MULTI_PAGE_GET_NUM 太大，每次读取的页可能大多都不是以后所用到的数据页，这样不仅会增加 I/O 的读取，而且每次都会做一些无用的 I/O，所以系统管理员需要衡量好自己应用需求，给出最佳方案。

3.2.2 日志缓冲区

日志缓冲区是用于存放重做日志的内存缓冲区。为了避免由于直接的磁盘 IO 而使系统性能受到影响，系统在运行过程中产生的日志并不会立即被写入磁盘，而是和数据页一样，先将其放置到日志缓冲区中。那么为何不在数据缓冲区中缓存重做日志而要单独设立日志缓冲区呢？主要是基于以下原因：

1. 重做日志的格式同数据页完全不一样，无法进行统一管理；
2. 重做日志具备连续写的特点；
3. 在逻辑上，写重做日志比数据页 IO 优先级更高。

DM Server 提供了参数 RLOG_BUF_SIZE 对日志缓冲区大小进行控制，日志缓冲区所占用的内存是从共享内存池中申请的，单位为页数量，且大小必须为 2 的 N 次方，否则采用系统默认大小 256 页。

3.2.3 字典缓冲区

字典缓冲区主要存储一些数据字典信息，如模式信息、表信息、列信息、触发器信息等。每次对数据库的操作都会涉及到数据字典信息，访问数据字典信息的效率直接影响到相应的操作效率，如进行查询语句，就需要相应的表信息、列信息等，这些字典信息如果都在缓冲区里，则直接从缓冲区中获取即可，否则需要 I/O 才能读取到这些信息。

DM7 采用的是将部分数据字典信息加载到缓冲区中，并采用 LRU 算法进行字典信息的控制。缓冲区大小设置问题，如果太大，会浪费宝贵的内存空间，如果太小，可能会频繁的进行淘汰，该缓冲区配置参数为 DICT_BUF_SIZE，默认的配置大小为 5M。

DM7 采用缓冲部分字典对象，那会影响效率吗？数据字典信息访问存在热点现像，并不是所有的字典信息都会被频繁的访问，所以按需加载字典信息并不会影响到实际的运行效率。

但是如果在实际应用中涉及对分区数较多的水平分区表访问，例如上千个分区，那么就需要适当调大 DICT_BUF_SIZE 参数值。

3.2.4 SQL 缓冲区

SQL 缓冲区提供在执行 SQL 语句过程中所需要的内存，包括计划、SQL 语句和结果集缓存。

很多应用当中都存在反复执行相同 SQL 语句的情况，此时可以使用缓冲区保存这些语句和它们的执行计划，这就是计划重用。这样带来的好处是加快了 SQL 语句执行效率，但同时给内存也增加了压力。

DM Server 在配置文件 dm.ini 提供了参数来支持是否需要计划重用，参数为 USE_PLN_POOL，当指定为 1 时，则启动计划重用；否则禁止计划重用。DM 同时还提供了参数 CACHE_POOL_SIZE（单位为 MB），来改变 SQL 缓冲区大小，系统管理员可以设置该值以满足应用需求，默认值为 10M。

结果集缓存包括 SQL 查询结果集缓存和 PL/SQL 函数结果集缓存，在 INI 参数文件中同时设置参数 RS_CAN_CACHE=1 且 USE_PLN_POOL 非零（1 或者 2）时 DM 服务器才会缓

存结果集。DM 还提供了一些手动设置结果集缓存的方法，具体可参看 24.10 节。

客户端结果集也可以缓存，但需要在配置文件 dm_svc.conf 中设置参数：

```
ENABLE_RS_CACHE = (1)   //表示启用缓存;
RS_CACHE_SIZE = (100)   //表示缓存区的大小为 100M, 可配置为 1-65535
RS_REFRESH_FREQ = (30) //表示每 30 秒检查缓存的有效性，如果失效，自动重查； 0 表示不检查。
```

同时在服务器端使用 INI 参数文件中的 CLT_CACHE_TABLES 参数设置哪些表的结果集需要缓存。另外，FIRST_ROWS 参数表示当查询的结果达到该行数时，就返回结果，不再继续查询，除非用户向服务器发一个 FETCH 命令。这个参数也用于客户端缓存的配置，仅当结果集的行数不超过 FIRST_ROWS 时，该结果集才可能被客户端缓存。

3.3 排序区

排序缓冲区提供数据排序所需要的内存空间。当用户执行 SQL 语句时，常常需要进行排序，所使用的内存就是排序缓冲区提供的。在每次排序过程中，都首先申请内存，排序结束后再释放内存。

DM Server 提供了参数来指定排序缓冲区的大小，参数 SORT_BUF_SIZE 在 DM 配置文件 dm.ini 中，系统管理员可以设置其大小以满足需求，由于该值是由系统内部排序算法和排序数据结构决定，建议使用默认值 2M。

3.4 哈希区

DM7 提供了为哈希连接而设定的缓冲区，不过该缓冲区是个虚拟缓冲区。之所以说是虚拟缓冲，是因为系统没有真正创建特定属于哈希缓冲区的内存，而是在进行哈希连接时，对排序的数据量进行了计算。如果计算出的数据量大小超过了哈希缓冲区的大小，则使用 DM7 创新的外存哈希方式；如果没有超过哈希缓冲区的大小，实际上使用的还是 VPOOL 内存池来进行哈希操作。

DM Server 在 dm.ini 中提供了参数 HJ_BUF_SIZE 来进行控制，由于该值的大小可能会限制哈希连接的效率，所以建议保持默认值，或设置为更大的值。

除了提供 HJ_BUF_SIZE 参数外，DM Server 还提供了创建哈希表个数的初始化参数，其中，HAGR_HASH_SIZE 表示处理聚集函数时创建哈希表的个数，建议保持默认值 100000。

3.5 SSD 缓冲区

固态硬盘采用闪存作为存储介质，因没有机械磁头的寻道时间，在读写效率上比机械磁盘具有优势。在内存、SSD 磁盘、机械磁盘之间，符合存储分级的条件。为提高系统执行效率，DM Server 将 SSD 文件作为内存缓存与普通磁盘之间的缓冲层，称为“SSD 缓存”。DM Server 在的 dm.ini 中提供参数 SSD_BUF_SIZE 和 SSD_FILE_PATH 来配置 SSD 缓冲，SSD_BUF_SIZE 指定缓冲区的大小，单位是 M，DM Server 根据该参数创建相应大小的文件作为缓冲区使用；SSD_FILE_PATH 指定该文件所在的文件夹路径，管理员需要保证设置的路径是位于固态磁盘上。

默认 SSD 缓冲区是关闭的，即 SSD_BUF_SIZE 为 0。若要配置 SSD 缓冲区，将其设置为大于 0 的数并指定 SSD_FILE_PATH 即可，根据存储分级的概念，建议将其大小 SSD_BUF_SIZE 配置为 BUFFER_SIZE 的 2 倍左右。

第4章 管理 DM 线程

DM 服务器使用“对称服务器构架”的单进程、多线程结构。这种对称服务器构架在有效地利用了系统资源的同时又提供了较高的可伸缩性能,这里所指的线程即为操作系统的线程。服务器在运行时由各种内存数据结构和一系列的线程组成，线程分为多种类型，不同类型的线程完成不同的任务。线程通过一定的同步机制对数据结构进行并发访问和处理，以完成客户提交的各种任务。DM 数据库服务器是共享的服务器,允许多个用户连接到同一个服务器上,服务器进程称为共享服务器进程。

DM 进程中主要包括监听线程、IO 线程、工作线程、调度线程、日志线程等，以下分别对它们进行介绍。

4.1 监听线程

监听线程主要的任务是在服务器端口上进行循环监听，一旦有来自客户的连接请求，监听线程被唤醒并生成一个会话申请任务，加入工作线程的任务队列，等待工作线程进行处理。它在系统启动完成后才启动，并且在系统关闭时首先被关闭。为了保证在处理大量客户连接时系统具有较短的响应时间，监听线程比普通线程优先级更高。

4.2 工作线程

工作线程是 DM 服务器的核心线程，它从任务队列中取出任务，并根据任务的类型进行相应的处理，负责所有实际的数据相关操作。

DM7 的初始工作线程个数由配置文件指定，随着会话连接的增加，工作线程也会同步增加，以保持每个会话都有专门的工作线程处理请求。为了保证用户所有请求及时响应，一个会话上的任务全部由同一个工作线程完成，这样减少了线程切换的代价，提高了系统效率。当会话连接超过预设的阀值时，工作线程数目不再增加，转而由会话轮询线程接收所有用户请求，加入任务队列，等待工作线程一旦空闲，从任务队列依次摘取请求任务处理。

4.3 IO 线程

在数据库活动中，IO 操作历来都是最为耗时的操作之一。当事务需要的数据页不在缓冲区中时，如果在工作线程中直接对那些数据页进行读写，将会使系统性能变得非常糟糕，而把 IO 操作从工作线程中分离出来则是明智的做法。IO 线程的职责就是处理这些 IO 操作。

通常情况下，DM Server 需要进行 IO 操作的时机主要有以下三种：

1. 需要处理的数据页不在缓冲区中，此时需要将相关数据页读入缓冲区；
2. 缓冲区满或系统关闭时，此时需要将部分脏数据页写入磁盘；
3. 检查点到来时，需要将所有脏数据页写入磁盘。

IO 线程在启动后，通常都处于睡眠状态，当系统需要进行 IO 时，只需要发出一个 IO 请求，此时 IO 线程被唤醒以处理该请求，在完成该 IO 操作后继续进入睡眠状态。

IO 线程的个数是可配置的，可以通过设置 dm.ini 文件中的 IO_THR_GROUPS 参数来设置，默认情况下，IO 线程的个数是 2 个。同时，IO 线程处理 IO 的策略根据操作系统平台的不同会有很大差别，一般情况下，IO 线程使用异步的 IO 将数据页写入磁盘，此时，系统将所有的 IO 请求直接递交给操作系统，操作系统在完成这些请求后才通知 IO 线程，这种

异步 IO 的方式使得 IO 线程需要直接处理的任务很简单，即完成 IO 后的一些收尾处理并发出 IO 完成通知，如果操作系统不支持异步 IO，此时 IO 线程就需要完成实际的 IO 操作。

4.4 调度线程

调度线程用于接管系统中所有需要定时调度的任务。调度线程每秒钟轮询一次，负责的任务有以下一些：

1. 检查系统级的时间触发器，如果满足触发条件则生成任务加到工作线程的任务队列由工作线程执行；
2. 清理 SQL 缓存、计划缓存中失效的项，或者超出缓存限制后淘汰不常用的缓存项；
3. 检查数据重演捕获持续时间是否到期，到期则自动停止捕获；
4. 执行动态缓冲区检查。根据需要动态扩展或动态收缩系统缓冲池；
5. 自动执行检查点。为了保证日志的及时刷盘，减少系统故障时恢复时间，根据 INI 参数设置的自动检查点执行间隔定期执行检查点操作；
6. 会话超时检测。当客户连接设置了连接超时时，定期检测是否超时，如果超时则自动断开连接；
7. 必要时执行数据更新页刷盘；
8. 唤醒等待的工作线程。

4.5 日志 FLUSH 线程

任何数据库的修改，都会产生重做 REDO 日志，为了保证数据故障恢复的一致性，REDO 日志的刷盘必须在数据页刷盘之前进行。事务运行时，会把生成的 REDO 日志保留在日志缓冲区中，当事务提交或者执行检查点时，会通知 FLUSH 线程进行日志刷盘。由于日志具备顺序写入的特点，比数据页分散 IO 写入效率更高。日志 FLUSH 线程和 IO 线程分开，能获得更快的响应速度，保证整体的性能。DM7 的日志 FLUSH 线程进行了优化，在刷盘之前，对不同缓冲区内的日志进行合并，减少了 IO 次数，进一步提高了性能。

如果系统配置了实时归档，在 FLUSH 线程日志刷盘前，会直接将日志通过网络发送到实时备库。如果配置了本地归档，则生成归档任务，通过日志归档线程完成。

4.6 日志归档线程

日志归档线程包含异步归档线程，负责远程异步归档任务。如果配置了非实时归档，由日志 FLUSH 线程产生的任务会分别加入日志归档线程，日志归档线程负责从任务队列中取出任务，按照归档类型做相应归档处理。

将日志 FLUSH 线程和日志归档线程分开的目的是为了减少不必要的效率损失，除了远程实时归档外，本地归档、远程异步归档都可以脱离 FLUSH 线程来做，如果放在 FLUSH 线程中一起做会严重影响系统性能。

4.7 日志重做线程

为了提高故障恢复效率，DM 在故障恢复时采用了并行机制重做 REDO 日志，日志重做线程就用于日志的并行恢复。通过 INI 参数 REDO_THREAD_NUM 可配置重做线程数，默认是 4 个线程。

日志重做线程仅在故障恢复时生成，线程在故障恢复完成后即退出。

4.8 日志 APPLY 线程

在配置了数据守护的系统中，创建了一个日志 APPLY 线程。当服务器作为备库时，每次接收到主库的物理 REDO 日志生成一个 APPLY 任务加入到任务队列，APPLY 线程从任务队列中取出一个任务在备库上将日志重做，并生成自己的日志，保持和主库数据的同步或一致，作为主库的一个镜像。备库数据对用户只读，可承担报表、查询等任务，均衡主库的负载。

4.9 定时器线程

在数据库的各种活动中，用户常常需要数据库完成在某个时间点开始进行某种操作，如备份；或者是在某个时间段内反复进行某种操作等。定时器线程就是为这种需求而设计的。

通常情况下，DM Server 需要进行定时操作的事件主要有以下几种：

1. 逻辑日志异步归档；
2. 异步归档日志发送（只有在 PRIMARY 模式下，且是 OPEN 状态下）；
3. 作业调度。

定时器线程启动之后，每秒检测一次定时器链表，查看当前的定时器是否满足触发条件，如果满足，则把执行权交给设置好的任务，如逻辑日志异步归档等。

默认情况下，达梦服务器启动的时候，定时器线程是不启动的。用户可以设置 dm.ini 中的 TIMER_INI 参数为 1 来设置定时器线程在系统启动时启动。

4.10 逻辑日志归档线程

逻辑日志归档用于 DM7 的数据复制中，目的是为了加快异地访问的响应速度，包含本地逻辑日志归档线程和远程逻辑日志归档线程。当配置了数据复制，系统才会创建这两个线程。

1. 本地逻辑日志归档线程

本地归档线程从本地归档任务列表中取出一个归档任务，生成到逻辑日志，并将逻辑日志写入到逻辑日志文件中。如果当前逻辑日志的远程归档类型是同步异地归档并且当前的刷盘机制是强制刷盘，那么就生成一个异地归档任务加入到临时列表中。

2. 远程逻辑日志归档线程

远程归档线程从远程归档任务列表中取出一个归档任务，并根据任务的类型进行相应的处理。任务的类型包括同步发送和异步发送。

4.11 MAL 系统相关线程

MAL 系统是 DM 内部高速通信系统，基于 TCP/IP 协议实现。服务器的很多重要功能都是通过 MAL 系统实现通信的，例如数据守护、数据复制、MPP、远程日志归档等。MAL 系统内部包含一系列线程，有 MAL 监听线程、MAL 发送工作线程、MAL 接收工作线程等。

4.12 其他线程

事实上，DM 数据库系统中还不止以上这些线程，在一些特定的功能中会有不同的线程，例如回滚段清理 PURGE 线程、审计写文件线程、重演捕获写文件线程等，这里不一一列出。

4.13 线程信息的查看

为了增加用户对 DM 数据库内部信息的了解，以及方便数据库管理员对数据库的维护，DM 提供了很多动态性能视图，通过它们用户可以直观地了解当前系统中有哪些线程在工作，以及线程的相关信息。相关动态视图见表 4.1。

表 4.1 DM 线程相关的动态视图

名称	说明
V$LATCHES	记录当前正在等待的线程信息
V$THREADS	记录当前系统中活动线程的信息
V$WTHRD_HISTORY	记录自系统启动以来，所有活动过线程的相关历史信息。
V$PROCESS	记录服务器进程信息

以上视图的详细定义参考本手册中附录 2“动态性能视图”。

第5章 DM7 的升级

相对于 DM6，DM7 不仅对原有功能做了大幅度的改进，也提供了不少新特色。DM7 的系统表同时进行了重新设计和组织，因此原有的 DM6 数据文件 DM7 不能直接识别和加载。旧版本的数据，必须升级为 DM7 的数据。这一章将会介绍如何将 DM6 的数据升级为 DM7 的数据。

5.1 选择升级方法

DM7 提供 2 种方法进行数据升级：1.利用数据迁移工具（DTS）；2.利用数据导入导出工具（dm6 的 dmloader，dm7 的 dmfldr）。数据迁移工具 DTS，由于支持众多模式对象的迁移（例如表、视图、索引、序列等），所以适用于将 DM6 的数据库升级至 DM7。数据导入导出工具由于处理单个表时效率较高，所以适用于单独将某些大表升级至 DM7。

因此对 DM 数据库进行升级前，需要根据待升级数据选择合适的升级方法。如果进行全库升级，那就选择数据迁移工具；如果涉及数据量很大的单个表，那就选择数据导入导出工具；如果既要导入全库，其中又包含数据量非常大的表，那么可以 2 种方法结合，在利用 DTS 迁移全库时，对于大表选择只迁移表结构，不迁移数据，然后再利用 DM 数据导入导出工具完成大数据表的升级。

5.2 升级前准备工作

数据升级前需要从整体考虑升级工作，制定升级规划和操作步骤。通常来说，升级工作按照如下流程进行：

1. 在目标数据库上重建原库的用户；
2. 迁移元数据结构，包括表、视图、序列；
3. 迁移数据；
4. 重建索引；
5. 迁移其它数据库对象，如存储过程、存储函数、触发器、同义词等等。注意为了保证迁移数据的正确性，触发器迁移成功后，需要禁用触发器；
6. 重建权限体系，包括建立角色，授予权限；
7. 启用所有触发器。

至此，数据升级工作完成。

依据数据升级工作流程，在正式进行数据升级前，我们需要准备创建用户、角色以及授予权限的 SQL 脚本，创建存储过程、存储函数、触发器、同义词等数据对象的 SQL 脚本。由于 dm7 的存储模块同 dm6 有些差别，所以存储过程、存储函数、触发器的 SQL 脚本可能需要进行部分修改。

对于元数据结构和数据的升级，需要根据对象选择合适的方法。因此，我们推荐升级工作进行前确定数据量大的表使用数据导入导出工具完成迁移。

小数据量表上的索引可以利用数据迁移工具 DTS 进行重建。对于大数据量表上的索引，建议在数据导入完成后通过 sql 脚本重建索引。为此，我们还需要准备索引重建的 SQL 脚本。

注意：数据迁移工具 DTS 可以得到上述除用户、角色、授权外的 SQL 脚本。方法是选择 DM 迁移到 SQL 脚本文件，配置迁移策略为不迁移数据。

5.3 使用数据迁移工具

由于数据迁移工具 DTS 采用向导式界面引导用户进行数据迁移，所以用 DTS 进行数据升级比较简单，这里简单描述下整体过程及注意事项。

在目标数据库上重建原库用户后，我们即可利用 DTS 迁移元数据及数据了。注意工作流程应当遵循我们设定的升级规划逐步完成。对于计划利用数据导入导出工具完成数据迁移的大表，配置迁移策略为不迁移数据。如果需要数据迁移完成后重建索引，那么配置迁移策略为不迁移索引。

如果发生迁移错误，我们需要详细查看错误日志信息，找到出错 sql 脚本。总体来说，最常见的错误原因为依赖对象不存在，或者 sql 语法错误。对于依赖对象不存在的错误，可以在依赖对象定义后再次进行迁移。对于 sql 语法错误，可以修改 sql 脚本后在 disql 中单独执行。

5.4 使用数据导入导出工具

使用数据导入导出工具是为了提高大数据量表升级效率而采用的方法，步骤如下：

1. 在 DM7 上创建目标表结构。可利用 DTS 完成，或者通过 sql 脚本完成；
2. 利用 dm6 的 dmloader，将大表数据导出形成文本文件；
3. 利用 dm7 的 dmfldr，导入文本文件；
4. 在 dm7 上重建该表索引。

5.5 升级后期工作

元数据和数据迁移完成后，首先检查数据的正确性。数据正确后，还有以下 4 个工作有待完成：

1. 重建索引；
2. 迁移存储过程、存储函数、触发器、同义词等数据库对象；
3. 重建权限体系，包括建立角色，授予权限；
4. 启用触发器。

这些工作均通过执行 sql 脚本完成。

第二部分 基础数据库管理

第6章 DM 系统管理员

为了保证数据库系统的安全性，DM 数据库采用“三权分立”或“四权分立”的安全机制，“三权分立”时系统内置三种系统管理员，包括数据库管理员、数据库安全员和数据库审计员，四权分立”时新增了一类用户，称为数据库对象操作员。它们各司其职，互相制约，有效地避免了将所有权限集中于一人的风险，保证了系统的安全性。

6.1 DM 系统管理员的类型

在现实生活中，任何一个系统如果将所有的权利都赋予给某一个人，而不加以监督和控制，势必会产生权利滥用的风险。从数据库安全角度出发，一个大型的数据库系统有必要将数据库系统的权限分配给不同的角色来管理，并且各自偏重于不同的工作职责，使之能够互相限制和监督，从而有效保证系统的整体安全。

DM 数据库实现了 B1 级安全特性。“三权分立”的安全机制，将系统管理员分数据库管理员、数据库安全员和数据库审计员三类。在安装过程中，DM 数据库会预设数据库管理员账号 SYSDBA、数据库安全员账号 SYSSSO 和数据库审计员账号 SYSAUDITOR，其缺省口令都与用户名一致。“四权分立”的安全机制，将系统管理员分数据库管理员、数据库对象操作员、数据库安全员和数据库审计员四类，在“三权分立”的基础上，新增数据库对象操作员账户 SYSDBO，其缺省口令为 SYSDBO。用户需要在安装过程中或者安装完毕后立即修改缺省口令，避免因口令泄漏造成的安全问题。

1. 数据库管理员（DBA）

“三权分立”的安全机制，每个 DM 数据库至少需要一个数据库管理员来管理，负责评估数据库运行所需的软、硬件环境、安装和升级 DM 数据库、配置 DM 数据库参数、创建主要的数据库存储结构（表空间）和对象（如表、视图、索引、角色、用户等）、监控和优化数据库性能、数据导入导出以及数据库的备份和恢复等。

“三权分立”时数据库管理员既可进行系统管理和维护工作，也可对数据内容进行增删查改动作。根据国产数据库军事使用要求，数据库管理员只能进行系统管理和维护工作，不能对数据内容进行增删查改，数据库应用人员则可操作数据内容，而不能管理和维护系统。

“四权分立”的安全机制，在原有“三权分立”基础上调整自主访问控制权限，只具有“三权分立”中 DBA 角色预设的一部分与数据库管理相关的明确的数据库权限，如数据库创建、备份、还原和校验等，具体见附录 7。

2. 数据库安全员（SSO）

对于很多对安全性要求不高的系统来说，C2 级安全特性已经能够工作得很好，此时不需要考虑通过数据库安全员来进一步加强系统的安全机制。但是在很多大型的系统中，安全性还是至关重要的，有必要由安全员来制定安全策略，强化系统安全机制，此时数据库安全员的主要任务就是制定安全策略，定义新的数据库安全员，设置系统的安全等级、范围和组，并为主、客体定义安全标记，从而全面提升系统安全性。

3. 数据库审计员（AUDITOR）

数据库审计员可以设置要审计的对象和操作、定义新的数据库审计员、查看和分析审计记录。通过设置审计，几乎可以跟踪任何人在系统内执行的任何操作，为事后追查提供便利。

4. 数据库对象操作员（DBO）

数据库对象操作员是“四权分立”新增加的一类用户，可以创建数据库对象，并对自己拥有的数据库对象（表、视图、存储过程、序列、包、外部链接）具有所有的对象权限并可以授出与回收，但其无法管理与维护数据库对象。

6.2 数据库管理员的任务

每个数据库至少需要一个 DBA 来管理，DBA 可能是一个团队，也可能是一个人。在不同的公司，数据库管理员的职责可能也会有比较大的区别，总体而言，数据库管理员的职责主要包括以下任务。

1. 评估数据库服务器所需的软、硬件运行环境

通常情况下，DBA 不会直接配置除数据库之外的软、硬件环境，不过在某些情况下还是需要。即便不用负责配置其他软、硬件，还是应该负责为软、硬件的选型提供指南和规范，以便最终配置能够提供满足业务需要的最优级别的性能、可靠性和可扩展性，同时保证总体费用在预算范围内。

作为 DBA，需要结合实际应用负载、总体费用、性能预期目标等来评估可供选择的软、硬件运行环境。其中实际应用负载和总体费用基本上都是确定的，DBA 的目标就是保证在负载和费用基本不变的情况下，实现系统总体性能的最优化。DBA 主要考虑的因素包括：

1) 操作系统/中间件等通用软件的稳定性、性能、安全性；
2) 处理器的个数和性能；
3) 内存容量和性能；
4) 网络带宽；
5) 存储容量和读写性能；
6) HBA 卡传输性能。

2. 安装和升级 DM 服务器

作为数据库管理员，应负责安装 DM 数据库服务器软件。具体的安装配置步骤请参考《DM_INSTALL》。需要注意的是，和所有软件一样，应先在测试系统上完成安装，这样可以确保不会影响到生产系统。

随着 DM 数据库的发展，会在 DM 数据库发布后提供服务包和安全更新。正确地安装服务包和安全更新是至关重要的任务，它影响到系统的稳定性、性能和安全。DBA 需要清楚地了解服务包和安全更新对现有系统的影响，然后考虑是否进行升级。是在生产系统上应用新的升级包之前，必须在测试环境下进行确认。

3. 数据库结构设计

在安装 DM 数据库之后，DBA 就可以开始进行数据库结构设计。数据库的结构设计直接影响系统的综合性能，是 DBA 最为重要的工作任务之一。

DBA 可以依次参考第 7 章来创建和配置 DM 数据库，参考第 8 章来启动和关闭数据库，参考第 16 章来实施计划好的数据库存储结构部署工作，参考第 9-15 章、第 18-21 章为每个对象规划数据库对象和存储特性的相关性的设计。通过在创建对象之前规划每个对象和它的物理存储之间的关系，可以直接影响数据库的性能。

4. 监控和优化数据库的性能

监控和优化数据库的性能是数据库管理员的核心职责之一。对系统进行监控有助于在问题发生时识别它们，性能优化则有助于消除那些问题并防止它们发生。监视长时间运行的查

询有助于发现可以改进的领域，如增加索引、重写查询、改进应用程序逻辑等，还可以发现由于缺少内存/内存分配不合理而导致的性能降低，以便调整 DM 数据库的内存使用参数，或为服务器增加更多的内存。DBA 需要提供对于各种问题的解决方案，努力优化 DM 数据库的各个方面，必要时，可以直接拨打 DM 数据库产品售后服务热线来寻求帮助。

5. 计划和实施备份与故障恢复

即便是最好、最稳定的系统也会存在意外，如断电、硬件故障等。DBA 必须对各种可能的故障和计划中的停机提供解决方案。一般来说，系统的可靠性与成本之间存在着类似线性的关系，越是可靠的解决方案，其整体成本必然也会更高，DBA 必须在企业限定的成本内制定可行的可靠性解决方案。

DM 数据库提供了包括备份与还原、故障恢复、数据守护、数据复制等高可靠性解决方案，DBA 可以参考本书第五部分来选择合适的解决方案。

6.3 数据库安全员的任务

有些应用对于安全性有着很高的要求，传统的由 DBA 一人拥有所有权限并且承担所有职责的安全机制可能无法满足企业实际需要，此时数据库安全员和数据库审计员两类角色和用户就显得异常重要，它们对于限制和监控数据库管理员的所有行为都起着至关重要的作用。

数据库安全员的主要职责就是制定并应用安全策略，强化系统安全机制。数据库安全员 SYSSSO 是 DM 数据库初始化的时候就已经创建好的，可以以该用户登录到 DM 数据库来创建新的数据库安全员。SYSSSO 或者新的数据库安全员都可以制定自己的安全策略，在安全策略中定义安全级别、范围和组，然后基于定义的安全级别、范围和组来创建安全标记，并将安全标记分别应用到主体（用户）和客体（表），以便启用强制访问控制功能。数据库安全员不能对用户数据进行增、删、改、查，也不能执行普通的 DDL 操作，他们只负责制定安全机制，将合适的安全标记应用到主体和客体，通过这种方式可以有效的对 DBA 的权限进行限制，DBA 此后就不能直接访问添加有安全标记的数据，除非安全员给 DBA 也设定了与之匹配的安全标记，DBA 的权限受到了有效的约束，详细的安全特性介绍请参考《DM 安全管理》。

6.4 数据库审计员的任务

我们可以想象一下，某个企业内部 DBA 非常熟悉公司内部 ERP 系统的数据库设计，该系统包括了员工工资表，里面记录了所有员工的工资，公司的出纳通过查询系统内部员工工资表来发放工资。传统的 DBA 集所有权利于一身，可以很容易修改工资表，从而导致公司工资账务错乱。为了预防该问题，可以采用前面数据库安全员制定安全策略的方法，避免 DBA 或者其他数据库用户具有访问该表的权限。为了能够及时找到 DBA 或者其他用户的非法操作，在 DM 数据库中还可以在系统建设初期，由数据库审计员（SYSAUDITOR 或者其他由 SYSAUDITOR 创建的审计员）来设置审计策略（包括审计对象和操作），在需要时，数据库审计员可以查看审计记录，及时分析并查找出幕后真凶。

从上面的介绍我们也可以看到，在 DM 数据库中，审计员的主要职责就是创建和删除数据库审计员，设置/取消对数据库对象和操作的审计设置，查看和分析审计记录等。关于审计的详细介绍，请参考《DM 安全管理》。

第7章 创建和配置 DM 数据库

DM 数据库可以在安装 DM 软件时创建，也可以在安装 DM 软件之后，通过数据库配置工具或 dminit 来手工创建数据库，创建数据库时要使用初始化参数。

7.1 创建 DM 数据库

用户创建数据库之前，需要规划数据库，如数据库名、实例名、端口、文件路径、簇大小、页大小、日志文件大小、SYSDBA 和 SYSAUDITOR 等系统用户的密码等，然后可以使用图形化界面或者 dminit 创建数据库。用户可以在安装 DM 数据库软件时创建数据库，也可以在安装之后创建数据库。

在创建数据库之前需要做如下准备工作：

1. 规划数据库表和索引，并估算它们所需的空间大小；
2. 确定字符集。所有字符集数据，包括数据字典中的数据，都被存储在数据库字符集中，用户在创建数据库时可以指定数据库字符集，如不指定则使用默认字符集 GB18030；
3. 配置数据库时区，如中国是+8:00 时区；
4. 设置数据库簇大小、页大小、日志文件大小，在数据库创建时由 EXTENT_SIZE, PAGE_SIZE, LOG_SIZE 初始化参数来指定，并且在数据库创建完成之后不能修改此参数。

创建数据库之前，必须满足以下必要条件：

1. 安装必需的 DM 软件，包括为操作系统设置各种环境变量，并为软件和数据库文件建立目录结构；
2. 必须有足够的内存来启动 DM 数据库实例；
3. 在执行 DM 的计算机上要有足够的磁盘存储空间来容纳规划的数据库。

7.2 使用数据库配置工具创建数据库

数据库配置工具提供了一个图形化界面来引导用户创建和配置数据库。使用数据库配置工具来创建数据库是用户的首选，因为数据库配置工具更趋于自动化，当使用数据库配置工具安装完成时，数据库也做好了使用准备工作。数据库配置工具可以通过 DM 数据库安装之后作为一个独立的工具来启动。数据库配置工具可以执行以下任务：

1. 创建数据库；
2. 改变数据库的配置；
3. 删除数据库；
4. 管理模板；
5. 自动存储管理；
6. 注册数据库服务。

7.2.1 启动数据库配置工具

数据库配置工具可以通过 DM 支持的模板或用户自定义的模板来创建数据库。

通过下列步骤来启动数据库配置工具（以 Windows 操作系统为例）：

1. 使用通过验证的具有管理权限组的成员登录到计算机上，安装 DM 数据库软件，并创建和运行数据库；
2. 在 Windows 操作系统中启动数据库配置工具。选择“开始”→“程序”→“达梦数据库”→“客户端”→“数据库配置工具”，启动数据库配置工具，将出现数据库配置工具界面，按照数据库配置工具操作，如图 7.1 所示。

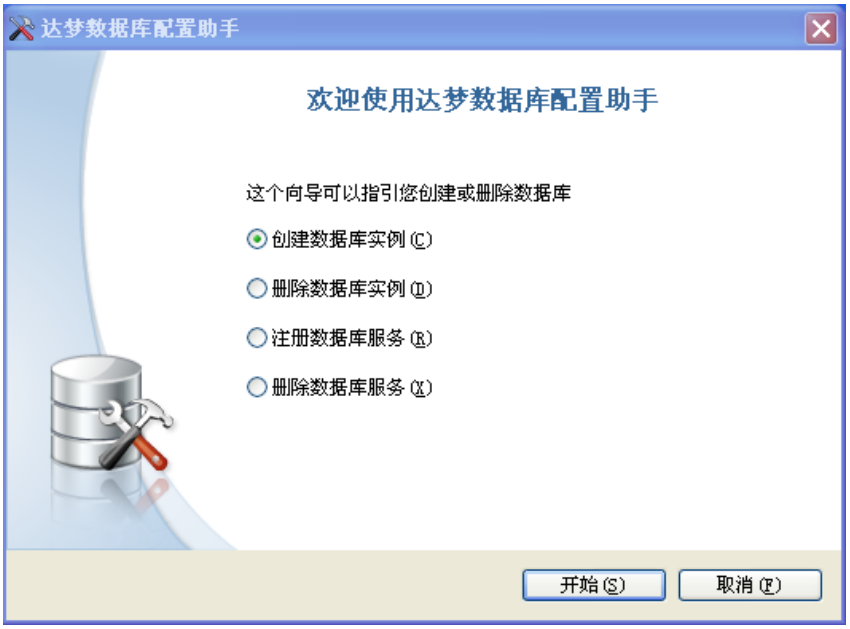


图 7.1 启动数据库配置工具

7.2.2 使用数据库配置工具创建数据库

在数据库配置工具操作窗口中，选择“创建数据库”选项启动能够创建和配置一个数据库的向导，如图 7.2 所示。这个向导引导用户完成以下操作：

1. 数据库模板；
2. 数据库目录；
3. 数据库标识；
4. 数据库文件；
5. 初始化参数；
6. 口令管理；
7. 创建示例库；
8. 创建摘要；
9. 创建数据库。



图 7.2 数据库配置工具操作界面

7.2.2.1 数据库模板

在这个窗口中可以选择需要创建数据库的类型，DM 预定义了一些模板，如一般用途、联机分析处理模板或联机事务处理模板，用户还可以选择是否以配置状态启动数据库，以及数据库初始模式(包括普通、主库、备库模式)。如图 7.3 所示。

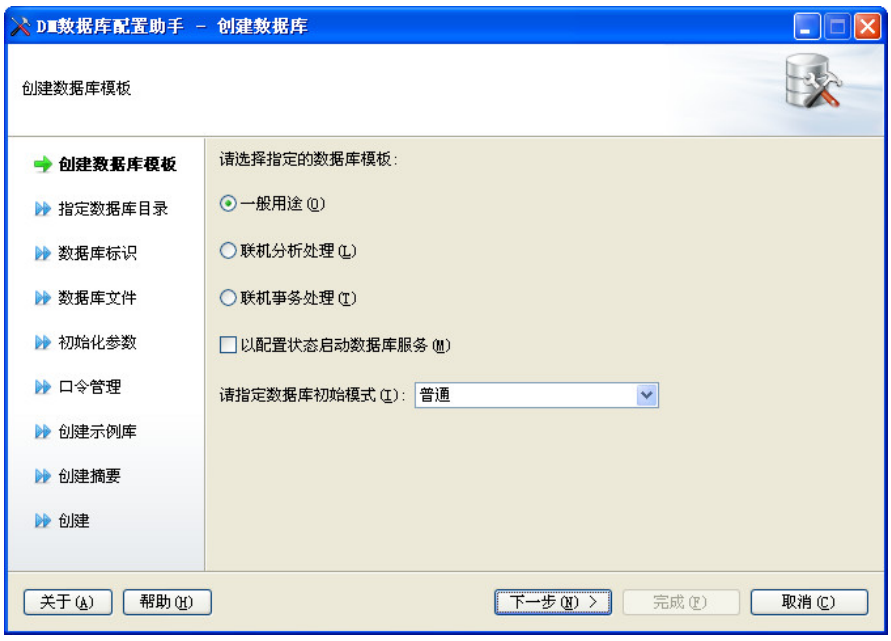


图 7.3 数据库模板选择

7.2.2.2 数据库目录

指定数据库目录。如图 7.4 所示。

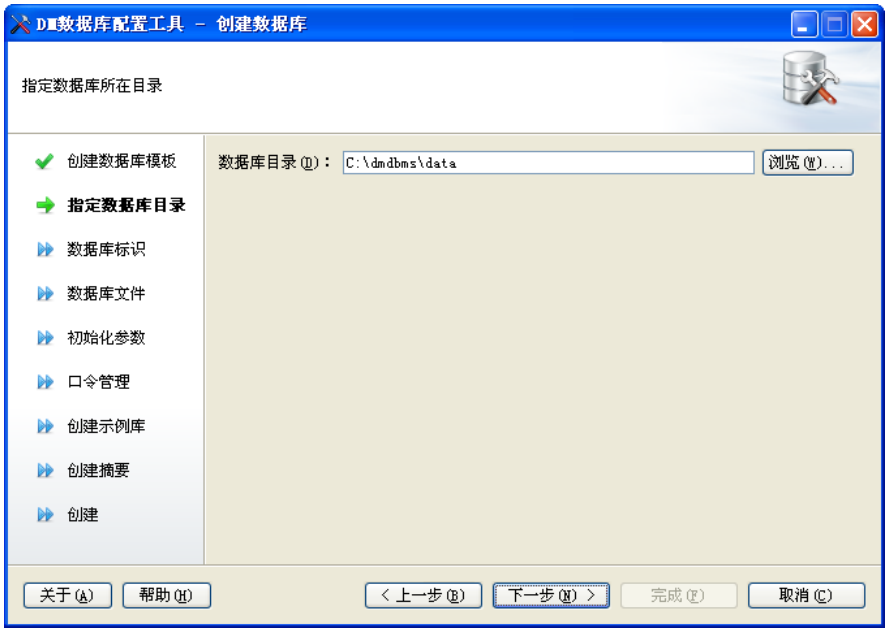


图 7.4 指定数据库目录

7.2.2.3 数据库标识

在“数据库名（D）”文本框中，输入数据库名；在“实例名（I）”文本框中输入数据库实例名；在“端口（P）”文本框中，输入端口号。如图 7.5 所示。



图 7.5 设置数据库标识

7.2.2.4 数据库文件

如图 7.6 所示，此界面包含三个选项卡：“控制文件”、“数据文件”和“日志文件”，可以通过双击路径来更改文件路径。

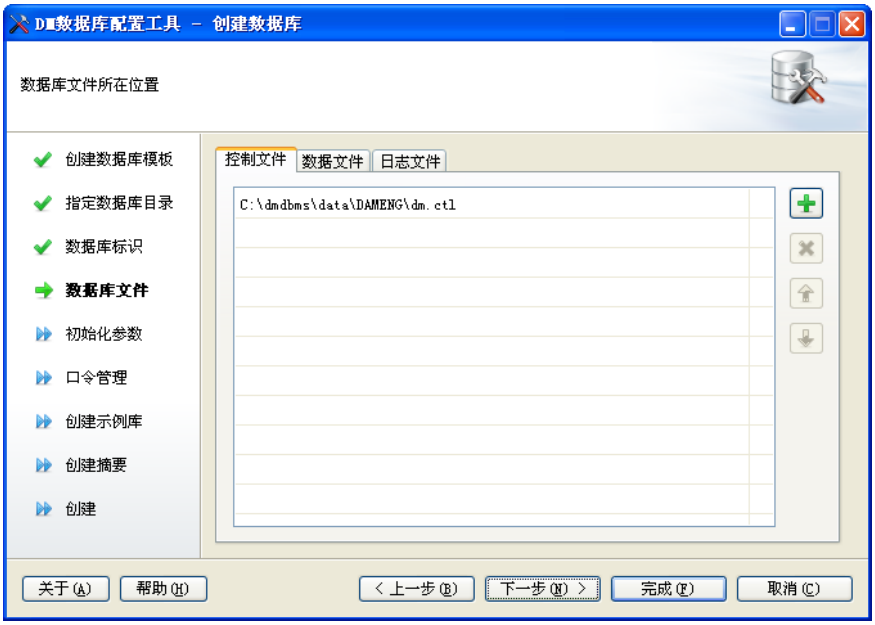


图 7.6 设置数据库文件路径

控制文件

与配置文件类似，控制文件对系统的运行及性能有很大的影响，但不同的是，配置文件中的配置项可以随意更改，而控制文件中的控制信息一般在系统第一次创建完毕后就不能随意更改，所以控制文件不是一个文本文件，而是一个二进制文件。另外控制文件一旦被破坏或丢失仍就无法手工重新创建，基于这一点，对控制文件的保护显得尤为重要。

整个系统中只有一个控制文件，其中存储的控制信息包括主要数据文件路径、日志文件路径、LSN 信息等。由于控制文件对系统至关重要，如果控制文件损坏，系统将无法启动，因此，DM 允许在创建数据库时指定多个控制文件的镜像。这些控制文件的内容是一模一样的，系统每次写控制文件时会顺序对它们进行修改。如果系统在写某一个控制文件时发生硬件故障导致该文件损坏，可以通过其他的控制文件来恢复这个损坏的控制文件，之后重新启动数据库。

数据文件

“数据文件”选项卡用来指定系统表空间路径、用户表空间路径、回滚表空间路径和临时表空间路径。同时还可以指定系统表空间镜像、用户表空间镜像、回滚表空间镜像路径。三个镜像文件是分别和系统表空间、用户表空间、回滚表空间一模一样的文件，当系统表空间、用户表空间、回滚表空间文件损坏时，就可以使用相应的镜像文件来替换。

数据文件是数据库中最重要的文件类型之一，这是数据最终要存储的地方，每个数据库至少有一个与之相关的数据文件，通常情况下，将会有多个。为了理解 DM Server 如何组织这些文件，以及数据在它们内部是如何组织的，必须理解数据页和簇的概念，它们都是 DM Server 用于保存数据库对象的分配单元。

数据页是系统进行磁盘 IO 和缓冲区调度的单元，其大小在数据库创建时就固定下来了，而且一旦固定就不可更改，它们的容量也都是相同的。所有数据页的格式大致相同。

簇是数据文件中一个连续的分配空间，簇由多个数量固定的数据页组成。数据文件对空间的标识都以簇为单位，每个数据文件都维护着两条链，一条为半空簇的链，另一条为自由簇的链，其中半空链用于标识文件中所有被用过一部分的簇，自由链则标识文件中所有未被用过的簇。通常情况下，系统在分配空间时，以簇为单位分配会更有效。

日志文件

重做日志文件对于 DM Server 是至关重要的。它们用于存储数据库的事务日志，以便系统在出现系统故障和介质故障时能够进行故障恢复。在 DM Server 中，任何修改数据库

的操作都会产生重做日志，这样，当系统出现故障时，通过分析日志可以知道在故障发生前系统做了哪些动作，并可以重做这些动作使系统恢复到故障之前的状态。

7.2.2.5 初始化参数

数据文件使用的簇大小，即每次分配新的段空间时连续的页数，只能是 16 页或 32 页，缺省使用 16 页。

数据文件使用的页大小，可以为 4K、8K、16K 或 32K，选择的页大小越大，则 DM 支持的元组长度也越大，但同时空间利用率可能下降，缺省使用 8K。

日志文件使用的大小，默认是 64，范围为 64 和 2048 之间的整数，单位为 M。如图 7.7 所示。

时区设置，默认是+08:00，范围为-12:59 和+14:00 之间，如图 7.7 所示。

页面检查，默认是不启用，选项包括不启用、简单检查、严格检查，如图 7.7 所示。

字符集，默认是 GB18030，选项包括 GB18030、Unicode、EUC-KR，如图 7.7 所示。



图 7.7 初始化参数

7.2.2.6 口令管理

为了数据库管理安全，提供了为数据库的 SYSDBA 和 SYSAUDITOR 系统用户指定新口令功能，如果安装版本为安全版，将会增加 SYSSSO 和 SYSDBO 用户的密码修改。用户可以选择为每个系统用户设置不同口令（留空表示使用默认口令），也可以为所有系统用户设置同一口令。口令必须是合法的字符串，不能少于 9 个或多于 48 个字符。如图 7.8 所示。

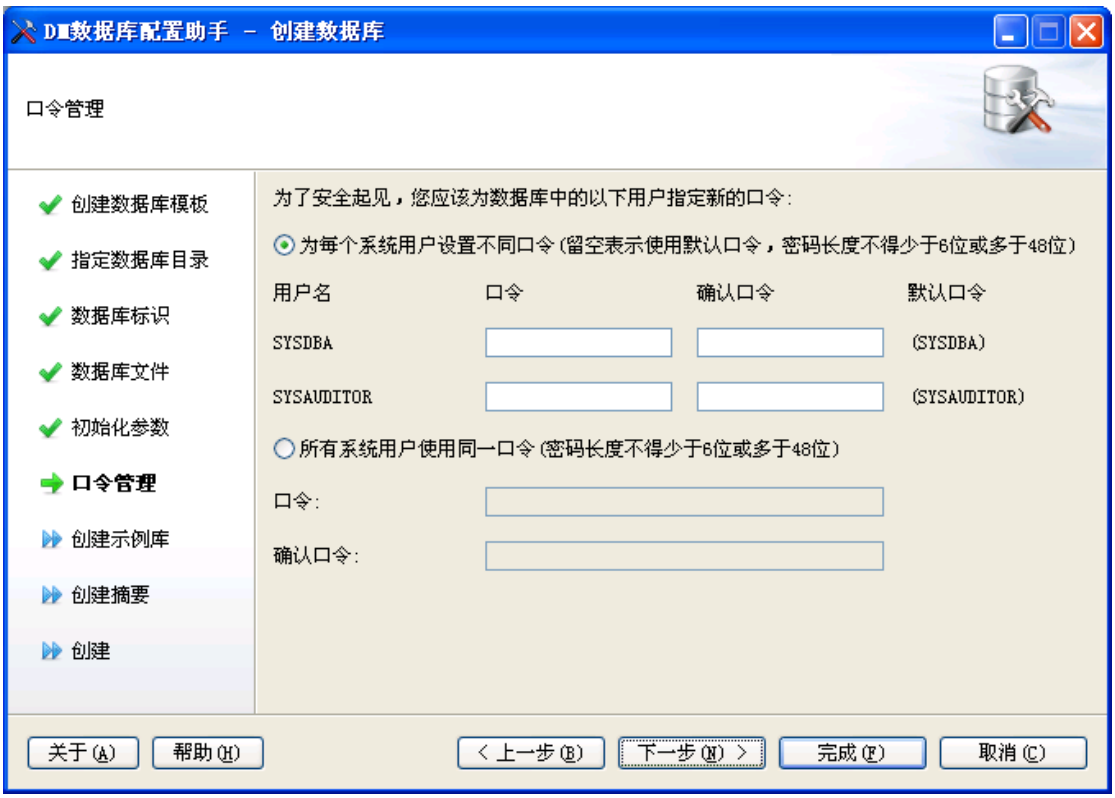


图 7.8 口令管理

7.2.2.7 创建示例库

示例库 BOOKSHOP 模拟武汉代理图书的某销售公司，该公司欲建立在线购物平台来拓展其代理产品的销售渠道。该示例在 DM 各演示程序中使用。安装该示例后，将在数据库中创建 BOOKSHOP 表空间，同时创建 RESOURCES、PERSON、PRODUCTION、PURCHASING、SALES、OTHER 这 6 个模式和相关的表。如图 7.9 所示。

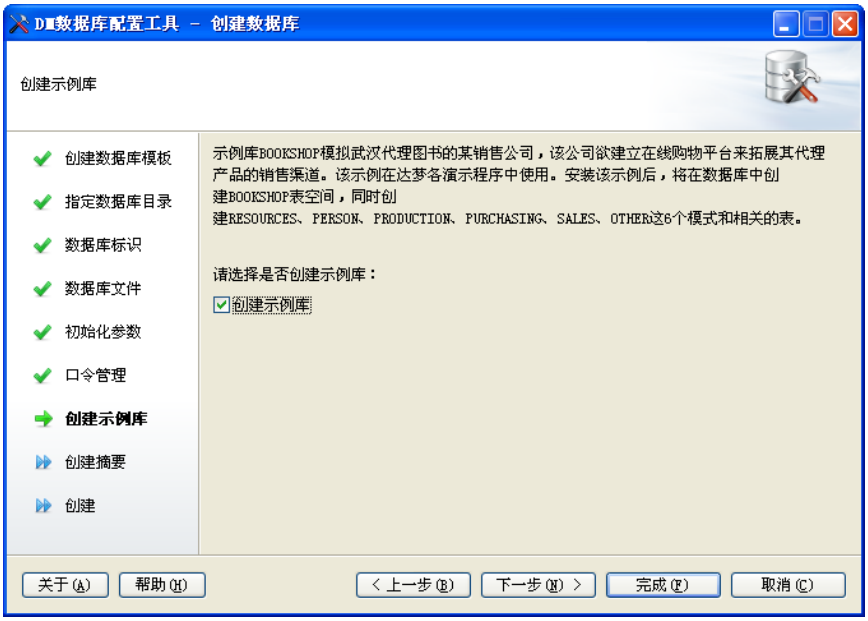


图 7.9 创建示例库

7.2.2.8 创建摘要

列举创建数据库纲要，会列举创建时指定的数据库名、示例名、数据库目录、端口、控制文件路径、数据文件路径、日志文件路径、簇大小、页大小、日志文件大小、标识符大小写是否敏感、是否使用 unicode 等信息，方便用户确认创建信息是否符合自己的需求，及时返回修改。如图 7.10 所示。

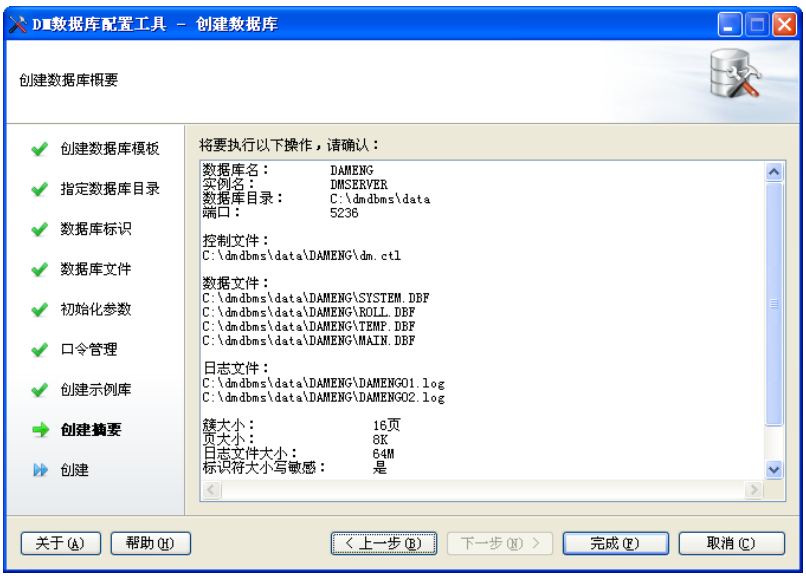


图 7.10 创建摘要

7.2.2.9 创建数据库

核对完创建信息后，开始创建数据库、创建并启动实例、创建示例库。如图 7.11 所示。

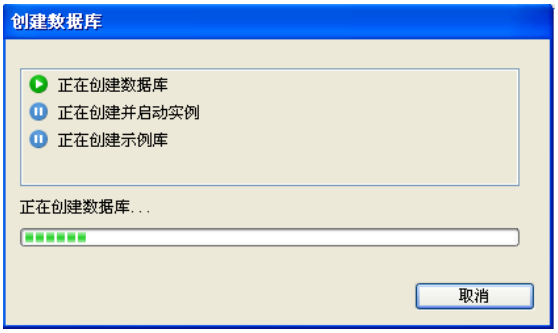


图 7.11 创建数据库

安装完成之后将显示对话框，提示安装完成或错误反馈信息，如图 7.12 所示。



图 7.12 创建数据库完成

如果数据库配置工具运行在 Linux、Solaris、AIX 和 HP-UNIX 系统中，使用非 root 系统用户在创建数据库完成时，将弹出提示框，提示应以 root 系统用户执行以下命令，用来创建数据库的随机启动服务，如图 7.13 所示。



图 7.13 执行命令提示

7.3 使用 dminit 创建数据库

在安装 DM 的过程中，用户可以选择是否创建初始数据库，如果当时没有创建，也可以在完成安装后，利用初始化库工具 dminit 来创建。系统管理员可以利用该工具提供的各种参数，设置数据库存放路径、段页大小、是否对大小写敏感以及是否使用 unicode，创建出满足用户需要的初始数据库。该工具位于安装目录的 bin\目录下。

在 Windows“命令提示符”窗口中输入带参数的 dminit 命令启动 dminit 工具，命令格式为 dminit [para=value][para=value]……，参数说明见表 7.1。

表 7.1 dminit 工具参数说明

参数	含义	取值	备注
INI_FILE	已有 INI 文件的路径，此 INI 文件用于将其所有参数值作为当前新生成 INI 文件的参数值	合法的路径。文件路径长度最大为 257(含结束符)，不包括文件名	可选
PATH	初始数据库存放的路径，默认路径为 dminit.exe 当前所在的工作目录	合法的路径。文件路径长度最大为 257(含结束符)，不包括文件名	可选
CTL_PATH	初始数据库控制文件的路径，默认值 windows 下是 **PATH\DB_NAME**\dm.ctl，linux 下是**/PATH/DM_NAME/**dm.ctl（粗体表示前面设置的参数）	合法的路径。文件路径长度最大为 257(含结束符)，不包括文件名	可选
LOG_PATH	初始数据库日志文件的路径，默认值 windows 下是 **PATH\DB_NAME**\DB_NAME01.log 和 **PATH\DB_NAME**\DB_NAME02.log，linux 下是 **PATH/DB_NAME**/DB_NAME01.log 和 **PATH/DB_NAME**/DB_NAME02.log（粗体表示前面设置的参数）	合法的路径。文件路径长度最大为 257(含结束符)，不包括文件名。日志文件路径个数不超过 10 个	可选
EXTENT_SIZE	数据文件使用的簇大小，即每次分配新的段空间时连续的页数	只能是 16 页或 32 页之一，缺省使用 16 页	可选
PAGE_SIZE	数据文件使用的页大小，可以为 4K、8K、16K 或 32K 之一，选择的页大小越大，则 DM 支持的元组长度也越大，但同时空间利用率可能下降，缺省使用 8K	只能是 4K、8K、16K 或 32K 之一	可选
LOG_SIZE	日志文件使用的簇大小，以 M 为单位，默认每个日志文件大小为 256M	64 和 2048 之间的整数	可选
CASE_SENSITIVE	标识符大小写敏感，默认值为 Y。当大小写敏感时，小写的标识符应用双引号括起，否则被转换为大写；当大小写不敏感时，系统不自动转换标识符的大小写，在标识符比较时也不区分大小写	只能是 Y, y, N, n,1,0 之一	可选
CHARSET/UNICODE_FLAG	字符集选项。0 代表 GB18030,1 代表 UTF-8,2 代表韩文字符集 EUC-KR	取值 0、1 或 2 之一。默认值为 0	可选
AUTO_OVERWRITE	2 表示清理建库目录下所有文件； 1 表示覆盖所有同名文件； 0 表示不覆盖所有同名文件； 默认值为 0。	只能是 0,1,2 之一	可选
LENGTH_IN_CHAR	VARCHAR 类型对象的长度是否以字符为单位。 1：是，设置为以字符为单位时，定义长度并非真正按照字符长度调整，而是	取值 0 或 1。默认值为 0	可选

	将存储长度值按照理论字符长度进行放大。所以会出现实际可插入字符数超过定义长度的情况，这种情况也是允许的。同时，存储的字节长度 8188 上限仍然不变，也就是说，即使定义列长度为 8188 字符，其实际能插入的字符串占用总字节长度仍然不能超过 8188；0：否，所有 VARCHAR 类型对象的长度以字节为单位		
USE_NEW_HASH	字符类型在计算 HASH 值时所采用的 HASH 算法类别。0：原始 HASH 算法；1：改进的 HASH 算法。默认值为 1。	取值 0 或 1	可选
SYSDBA_PWD	初始化时设置 SYSDBA 的密码，默认为 SYSDBA	合法的字符串，长度在 6 到 48 个字符之间	可选
SYSAUDITOR_PWD	初始化时设置 SYSAUDITOR 的密码，默认为 SYSAUDITOR	合法的字符串，长度在 6 到 48 个字符之间	可选
DB_NAME	初始化数据库名字，默认是 DAMENG	有效的字符串，不超过 128 个字符	可选
INSTANCE_NAME	初始化数据库实例名字，默认是 DMSERVER	有效的字符串，不超过 128 个字符	可选
PORT_NUM	初始化时设置 dm.ini 中的 PORT_NUM，默认 5236	取值范围：1024~65534	可选
TIME_ZONE	初始化时区，默认是东八区	格式为[正负号]小时[：分钟]（正负号和分钟为可选）。时区设置范围为：-12:59~+14:00	可选
PAGE_CHECK	是否启用页面内容校验，0：不启用；1:简单校验;2:严格校验(使用 CRC16 算法生成校验码)。默认 0	取值范围：0~2	可选
RAC_NODE	高性能集群的节点数目	取值范围 2~16，单站点时不写	可选
EXTERNAL_CIPHER_NAME	设置默认加密算法	有效的字符串，不超过 128 个字符	可选
EXTERNAL_HASH_NAME	设置默认 HASH 算法	有效的字符串，不超过 128 个字符	可选
EXTERNAL_CRYPTO_NAME	设置根密钥加密引擎	有效的字符串，不超过 128 个字符	可选
SVR_KEY	服务器主密钥，用于加密数据库主密钥	合法的字符串，最长为 32 个字节	可选
DB_KEY	数据库主密钥，用于加密库内密钥，如用户加密密钥、列加密密钥	合法的字符串，最长为 32 个字节	可选
ENCRYPT_PWD	全库加密密钥	合法的字符串，最长为 32 个字节	可选
ENCRYPT_NAME	全库加密密钥使用的算法名。算法可以是 DM 内部支持的加密算法，或者是第三方的加密算法。默认使用"AES256_ECB"算法加密	合法的字符串，最长为 128 个字节	可选
RLOG_ENC_FLAG	设置日志文件是否加密	取值 Y/N，y/n，1/0，默认 N	可选

USBKEY_PIN	USBKEY PIN,用于加密服务器根密钥	合法的字符串，最长为 48 个字节	可选
BLANK_PAD_MODE	设置字符串比较时，结尾空格填充模式是否兼容 ORACLE	取值 0 或 1。0 不兼容，1 兼容。默认为 0	可选
SYSTEM_MIRROR_PATH	指定 system.dbf 文件的镜像路径	绝对路径，默认为空	可选
MAIN_MIRROR_PATH	指定 main.dbf 文件的镜像路径	绝对路径，默认为空	可选
ROLL_MIRROR_PATH	指定 roll.dbf 文件的镜像路径	绝对路径，默认为空	可选
MAL_FLAG	初始化时设置 dm.ini 中的 MAL_INI，默认 0	取值 0 或 1	可选
ARCH_FLAG	初始化时设置 dm.ini 中的 ARCH_INI，默认 0	取值 0 或 1	可选
MPP_FLAG	Mpp 系统内的库初始化时设置 dm.ini 中的 mpp_ini，默认 0。	取值 0 或 1	可选
CONTROL	指定初始化配置文件路径。初始化配置文件是一个保存了各数据文件路径设置等信息的文本。使用 control 初始化时，若文件已存在，系统会屏幕打印提示，然后直接覆盖	主要用于将数据文件放在裸设备或 rac 环境下	可选
DCP_MODE	是否是 DCP 代理模式	取值：1 是；0 否。默认值为 0	可选
DCP_PORT_NUM	DCP 代理模式下管理端口	取值范围：1024~65534。当 DCP_MODE=1 时，该参数才有效	可选
SYSSSO_PWD	初始化时设置 SYSSSO 的密码，默认为 SYSSSO，仅在安全版本下可见和可设置	合法的字符串，长度在 6 到 48 个字符之间	可选
SYSDBO_PWD	初始化时设置 SYSDBO 的密码，默认为 SYSDBO，仅在安全版本下可见和可设置	合法的字符串，长度在 6 到 48 个字符之间	可选
PRIV_FLAG	是否是四权分立。默认值为 0（不使用），四权分立的具体权限见附录 7。默认情况下，使用三权分立。 仅在安全版本下可见和可设置	只能是 0 或 1	可选
ELOG_PATH	指定初始化过程中生成的日志文件所在路径	合法的路径。文件路径长度最大为 257(含结束符)，不包括文件名	可选
HELP	显示帮助信息		可选

说明：dminit 一般是要有参数的，如果没有带参数，系统就会引导用户设置。另外，参数、等号和值之间不能有空格。Help 参数的后面不用添加'='号。

命令举例：

```
dminit PATH=c:\dm7data PAGE_SIZE=16
```

如果创建成功，则屏幕显示如图 7.13 所示。此时在 c 盘根目录下出现 dm7Data 文件夹，内容包含初始数据库 DAMENG 的相关文件和初始化文件 dm.ini。将 dm.ini 文件拷贝到 DM 安装目录的 bin 下，DM 服务器就可以启动该初始数据库了。

```
initdb V7.0.1.3-Build(2011.07.28)
db version: 0X70001
mem_installed : 2031052
create dm database success.
```

图 7.14 创建成功后的屏幕显示

7.4 注册数据库服务

注册数据库服务，该功能用于将使用命令行工具生成的数据库，重新注册成系统服务，方便用户管理与控制。

用户注册数据库服务可以通过图形化界面和 shell 脚本两种方式，本节讲述的是通过数据库配置工具注册数据库服务，通过 shell 脚本注册数据库服务详细操作请参见 **7.8 节数据库服务配置脚本**。如图 7.16 所示。



图 7.16 选择操作方式

用户选择 dm.ini 文件来注册相应的数据库，并可修改相应的端口号和实例名，也可以选择是否以配置状态启动数据库，如图 7.17 所示。

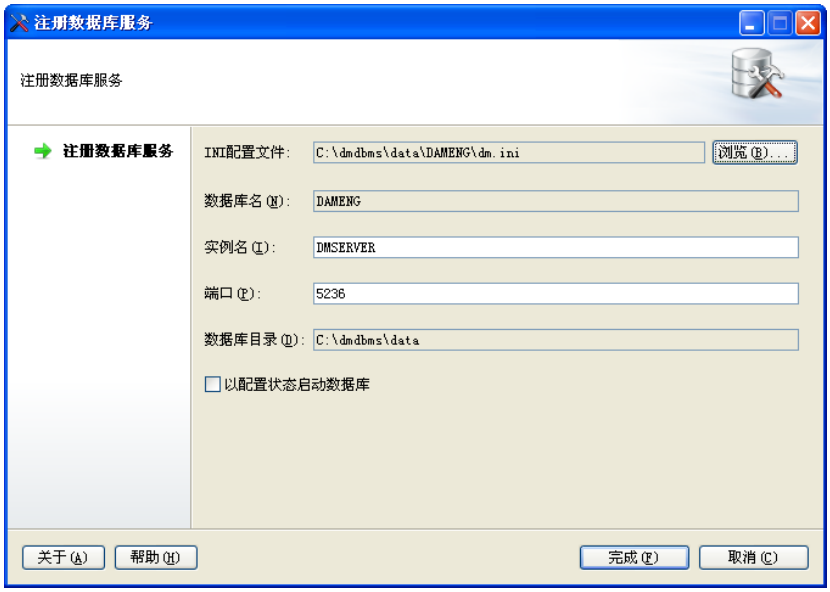


图 7.17 要删除的数据库

正在注册数据库服务，如图 7.18 所示。

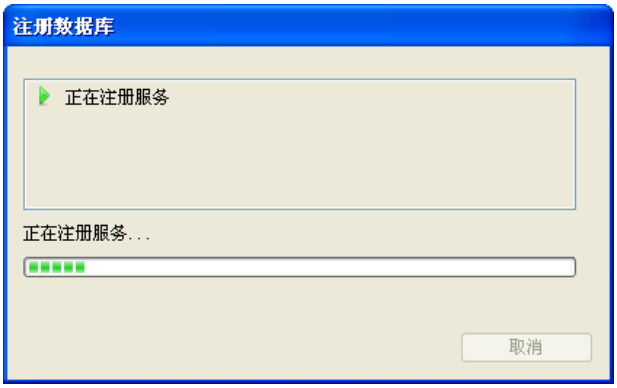


图 7.18 注册数据库服务

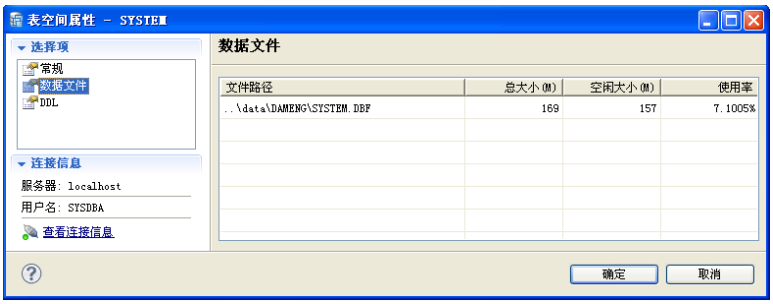
注册完成之后将显示对话框，提示完成或错误反馈信息，如图 7.19 所示。



图 7.19 注册数据库完成

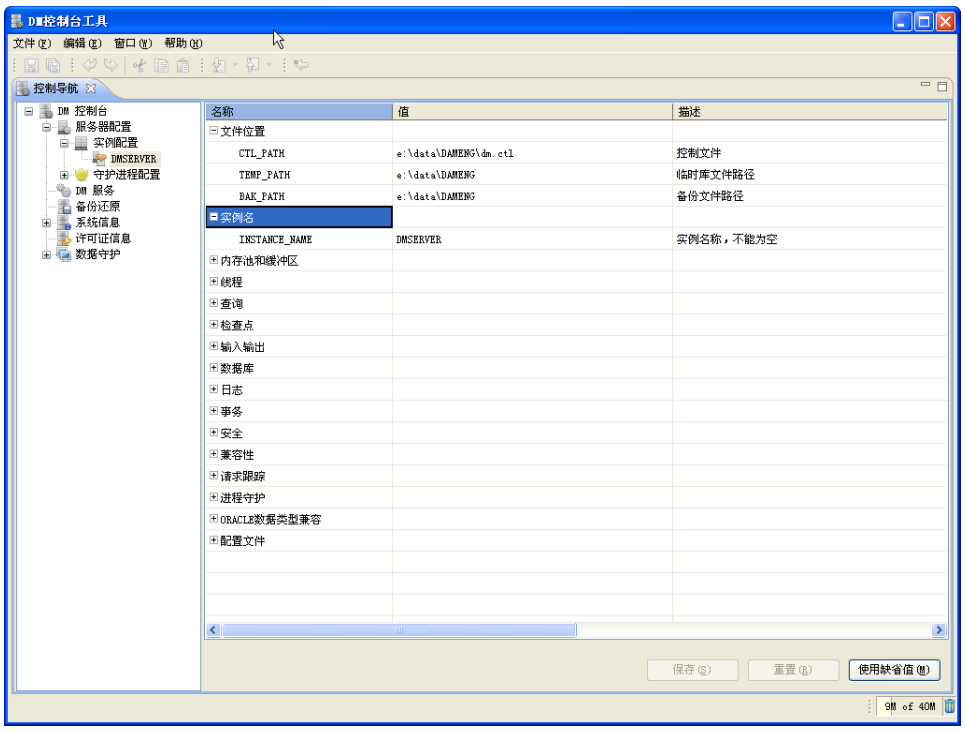
7.5 查看数据库信息

登录数据库管理工具，通过表空间属性，可以查看数据文件路径、总空间大小、空闲空间大小、使用率。如图 7.20 所示。



7.20 查看数据文件

通过 DM 控制台工具，查看实例配置属性，如图 7.21 所示，可以查看到如下信息：文件位置、实例名、内存池和缓冲区、线程、查询、检查点、输入输出、数据库、日志、事务、安全、兼容性、请求跟踪、进程守护、ORACLE 数据类型兼容和配置文件。



7.21 查看数据文件

7.6 删除数据库

删除数据库，包括删除数据库的数据文件、日志文件、控制文件和初始化参数文件。为了保证删除数据库成功，必须保证 dmserver 已关闭。

可以使用数据库配置工具来删除数据库。如图 7.22 所示。



图 7.22 选择操作方式

根据数据库名称，选择要删除的数据库，如图 7.23 所示，也可以通过指定数据库配置文件删除数据库。

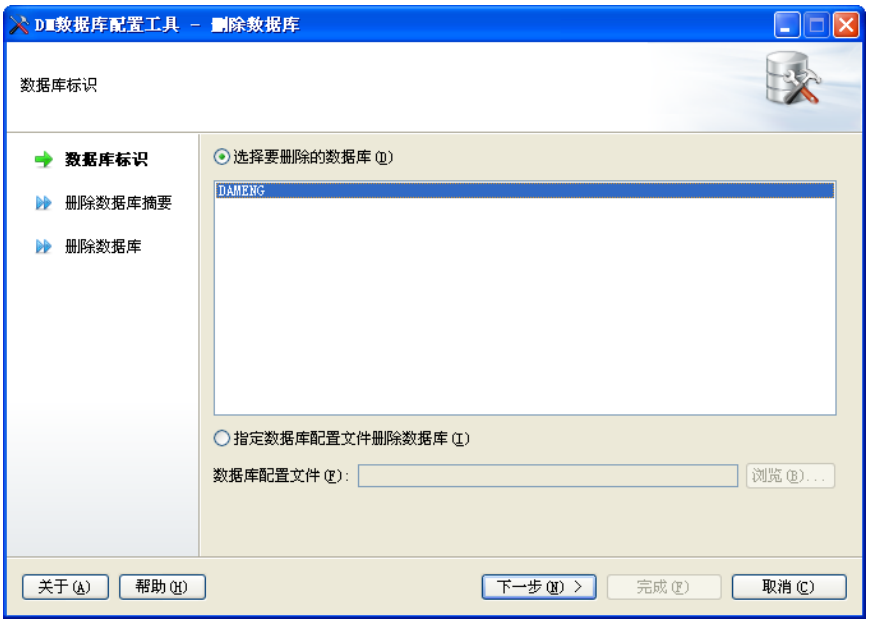


图 7.23 要删除的数据库

确认将删除的数据库名、实例名、数据库目录，如图 7.24 所示。

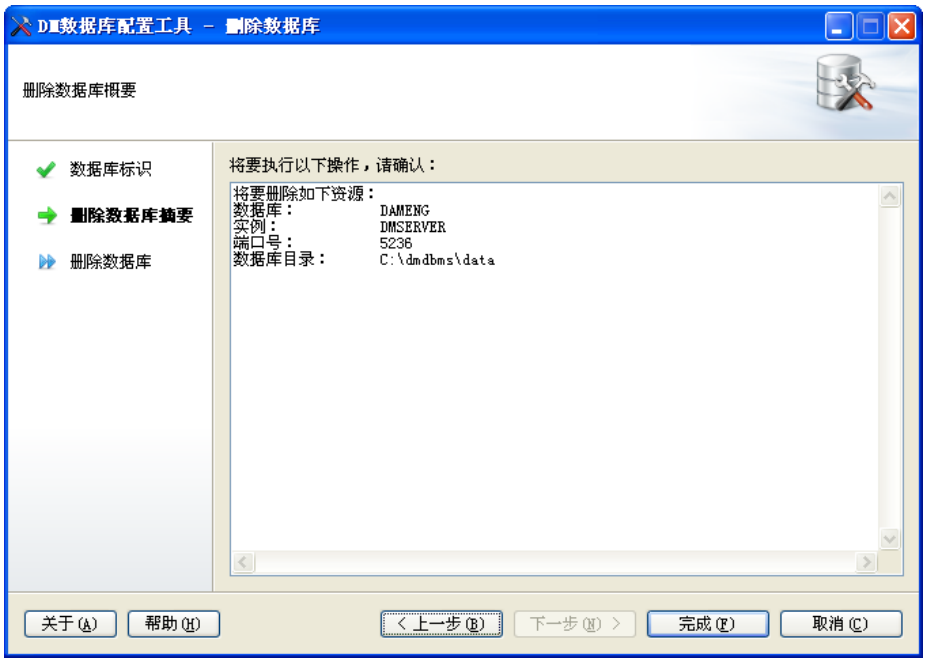


图 7.24 删除数据库概要

首先停止实例，然后删除实例，最后删除数据库，如图 7.25 所示。

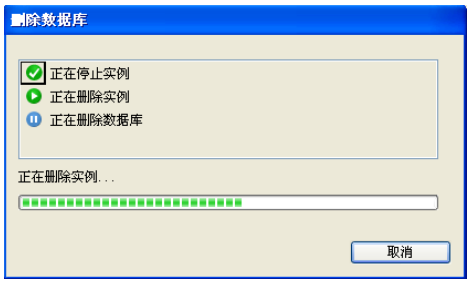


图 7.25 删除数据库

删除完成之后将显示对话框，提示完成或错误反馈信息，如图 7.26 所示。



图 7.26 删除数据库完成

如果数据库配置工具运行在 Linux、Solaris、AIX 和 HP-UNIX 系统中，使用非 root 系统用户在删除数据库完成时，将弹出提示框，提示应以 root 系统用户执行以下命令，用来删除数据库的随机启动服务，如图 7.27 所示。

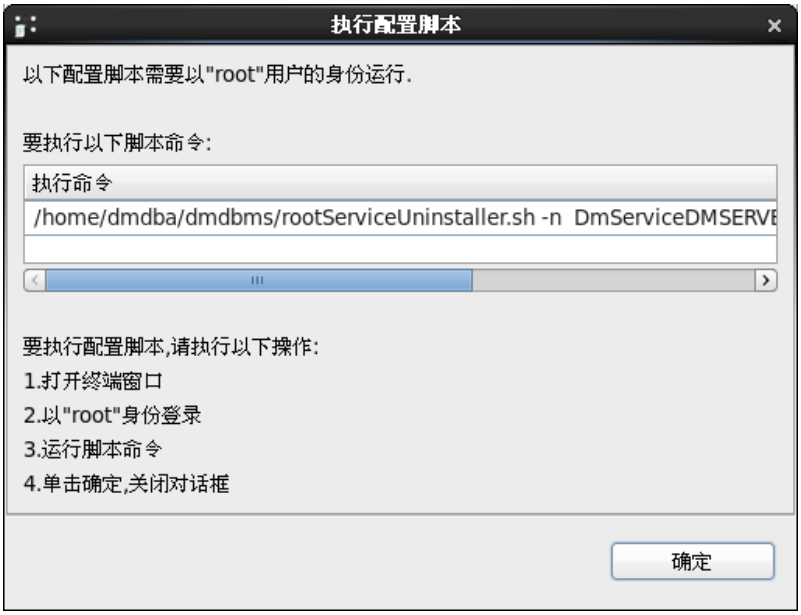


图 7.27 执行命令提示

7.7 删除数据库服务

删除数据库服务，只删除用于启动和停止数据库的服务文件，不会删除数据库的数据文件、日志文件、控制文件和初始化参数文件。

用户删除数据库服务可以通过图形化界面和 shell 脚本两种方式，本节讲述的是通过数据库配置工具删除数据库服务，通过 shell 脚本删除数据库服务详细操作请参见 7.8 节数据库服务配置脚本。如图 7.28 所示。



图 7.28 选择操作方式

根据数据库服务名称，选择要删除的数据库服务，如图 7.29 所示，也可以通过指定数据库配置文件删除数据库服务。

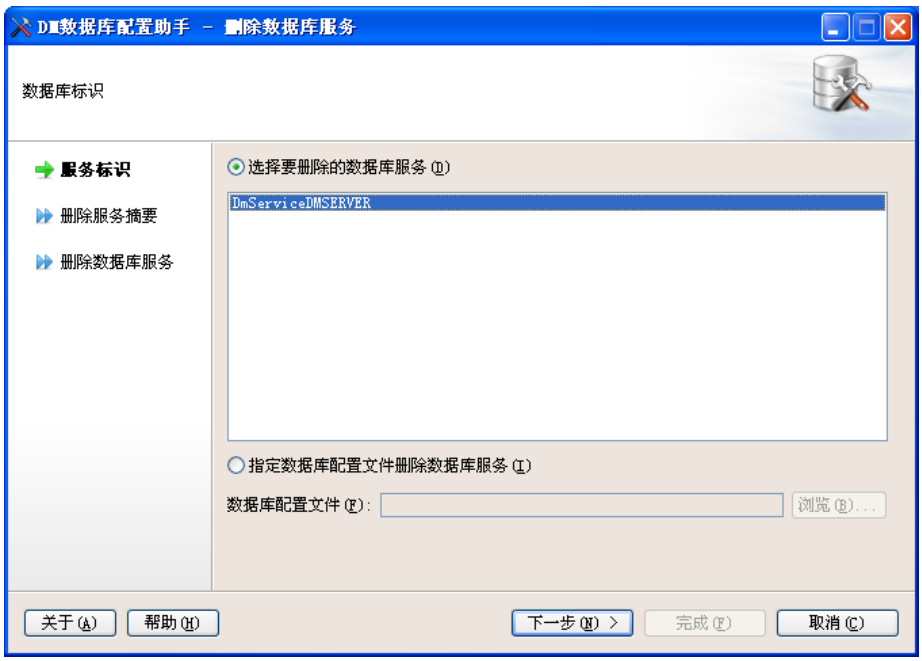


图 7.29 要删除的数据库服务

确认将删除的数据库名、实例名、数据库服务名、数据库目录，如图 7.30 所示。

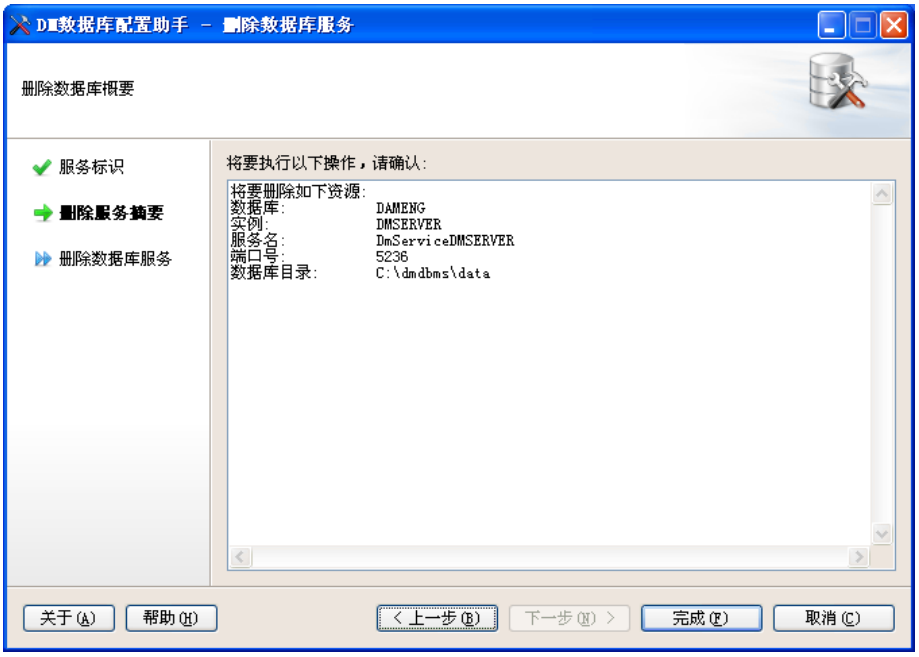


图 7.30 删除数据库概要

首先检查数据库服务，然后删除数据库服务，如图 7.31 所示。

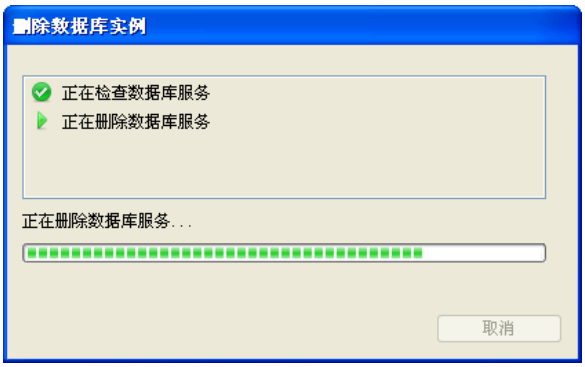


图 7.31 删除数据库服务

删除完成之后将显示对话框，提示完成或错误反馈信息，如图 7.32 所示。



图 7.32 删除数据库服务完成

如果数据库配置工具运行在 Linux、Solaris、AIX 和 HP-UNIX 系统中，使用非 root 系统用户在删除数据库服务完成时，将弹出提示框，提示应以 root 系统用户执行以下命令，用来删除数据库的随机启动服务，如图 7.33 所示。

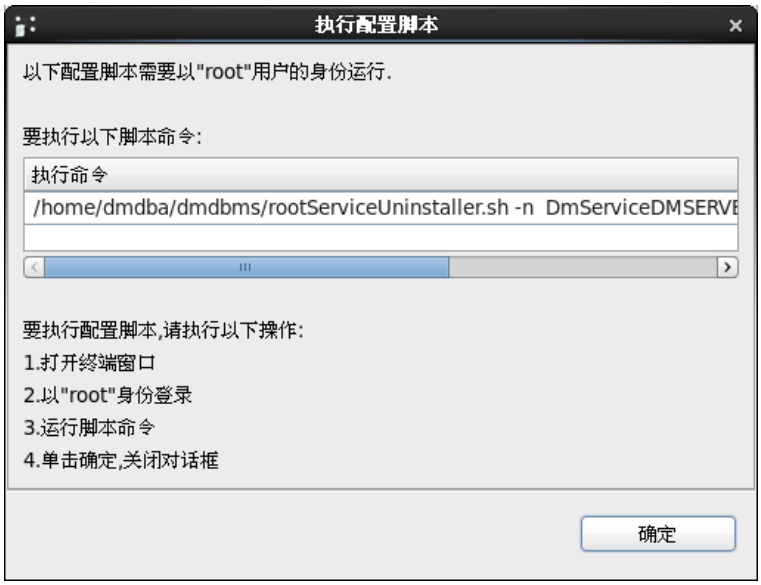


图 7.33 执行命令提示

7.8 数据库服务配置脚本

在 Linux、Solaris、AIX 和 HP-UNIX 系统下，创建和删除系统随机启动服务需要 root 权限，所以用户运行数据库服务配置脚本时需要使用 root 用户运行配置脚本，用来创建或删除达梦数据库的随机启动服务。

数据库服务配置脚本默认位置在数据库安装目录下的 script 目录下的 root 目录中，包含 root_db_service_installer.sh 服务安装脚本文件和 rootServiceUninstaller.sh 服务卸载脚本文件。

7.8.1 服务安装脚本文件

服务安装脚本文件即为 root_db_service_installer.sh。参数如下表：

标志	参数	说明
-F	INI 文件路径	指定数据库服务所需要的 INI 文件路径
-N	数据库实例名	指定数据库的实例名
-M	OPEN 或 MOUNT	指定数据库启动方式，可选
-S	数据库服务名	如果已存在数据库服务脚本文件，可直接指定服务名。即脚本文件名。如果设置此参数则忽略 –F、–N、–M
-H		帮助

使用方法如下：

首先使用 root 系统用户登录或切换至 root 系统用户。

假设数据库的 ini 所在路径为/opt/dmdbms/data/DAMENG/dm.ini，此数据库的实例名为 DMSERVER。

1. 通过 ini 文件注册服务，命令如下：

```
./root_db_service_installer.sh –f /opt/dmdbms/data/DAMENG/dm.ini –n DMSERVER
```

2. 通过服务名创建服务。非 root 系统用户通过数据库配置工具创建数据库完成时，会在安装目录的 bin 目录下生成数据库服务文件，文件名为 DmService+实例名，此文件名即为数据库的服务名。用户可通过此服务名创建数据库随机启动服务。命令如下：

```
./root_db_service_installer.sh –s DmServiceDMSERVER
```

7.8.2 服务卸载脚本文件

服务卸载脚本文件即为 rootServiceUninstaller.sh。参数如下表：

标志	参数	说明
-A		删除所有的数据库随机启动服务
-N	数据库服务名	删除指定的数据库随机启动服务
-H		帮助

使用方法如下：

首先使用 root 系统用户登录或切换至 root 系统用户。

1. 删除所有的数据库服务。命令如下：

```
./rootServiceUninstaller.sh –a
```

2. 删除指定的数据库服务。假设存在数据库随机启动服务 DmServiceDMSERVER，命令如下：

```
./rootServiceUninstaller.sh –n DmServiceDMSERVER
```

第8章 启动和关闭数据库

DM 支持多种操作系统。常用的操作系统基本上可以划分为 Windows 操作系统和 Linux 操作系统。下面分别介绍这两种操作系统下启动和关闭 DM 数据库的操作。

8.1 启动数据库

8.1.1 Windows 系统

1. 菜单方式

安装 DM 数据库后(默认情况下安装成功后 DM 服务会自动启动)，在 Windows 的开始菜单选项中选择如图 8.1 所示的菜单项中的 DM 服务查看器可以启动 DM 数据库。

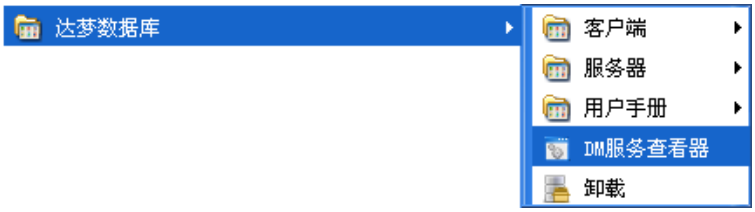


图 8.1 DM 数据库菜单方式启动 1

点击 DM 服务查看器选项后，会弹出如图 8.2 所示的界面：



图 8.2 DM 数据库菜单方式启动 2

在弹出界面中选中所要启动的数据库，点击鼠标右键，在菜单栏中选择启动。

2. Windows 服务方式

安装 DM 数据库并且新建一个 DM 实例后。Windows 的服务中会自动增加一项和该实例名对应的服务。例如新建一个实例名为 DMSERVER 的 DM 数据库，Windows 的服务中会增加一项名称为“DmServiceDMSERVER”的服务。打开 Windows 的管理工具，选择“服务”，打

开 Windows 服务控制台，如图 8.3 所示，选择“DmServiceDMSERVER”，用鼠标在工具栏点击启动按钮或者点击鼠标右键，在菜单栏中选择“启动”，启动 DM 数据库。

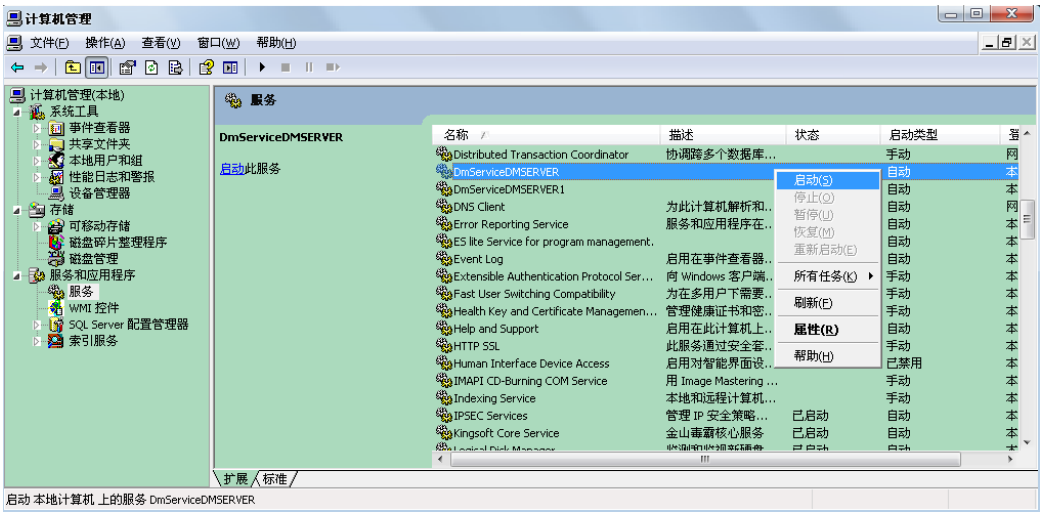


图 8.3 DM 数据库服务方式启动

3. 命令行方式

进入 DM 安装目录下的 bin 目录，直接打开应用程序 dmserver 就可以启动 DM 数据库。或者先打开 Windows 命令提示符工具，在命令工具中执行命令进入 DM 服务器的目录，再执行 dmserver 的命令启动 DM 数据库，如图 8.4 所示。

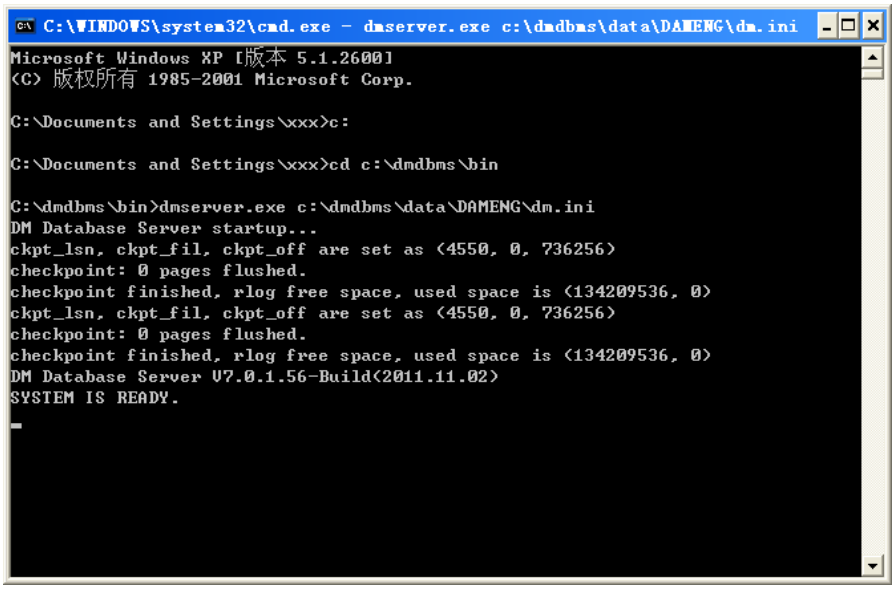


图 8.4 DM 数据库命令行方式启动

命令行方式启动参数如下：

```
dmserver [ini_file_path] [-noconsole] [mount]
```

说明：

1. Dmserver 命令行启动参数可指定 dm.ini 文件的路径，非控制台方式启动及指定数据库是否以 MOUNT 状态启动。关于数据库状态见下一节介绍；
2. Dmserver 启动时可不指定任何参数，默认使用当前目录下的 dm.ini 文件，如果当前目录不存在 dm.ini 文件，则无法启动；
3. Dmserver 启动时可以指定-noconsole 参数。如果以此方式启动，则无法通过在控制台中输入服务器命令。

当不确定启动参数的使用方法时，可以使用 help 参数，将打印出格式、参数说明和使

用示例。使用方法如下：

```
dmserver help
```

当以控制台方式启动 Dmserver 时，用户可以在控制台输入一些命令，服务器将在控制台打印出相关信息或执行相关操作。支持的命令见下表。

表 8.1 Dmserver 控制台支持的命令

命令	操作
EXIT	退出服务器
LOCK	打印锁系统信息
TRX	打印等待事务信息
CKPT	设置检查点
BUF	打印内存池中缓冲区的信息
MEM	打印服务器占用内存大小
SESSION	打印连接个数
DEBUG	打开 DEBUG 模式

8.1.2 Linux 系统

1. 菜单方式

安装 DM 数据库后(默认情况下安装成功后 DM 服务会自动启动)，在 Linux 的开始菜单选项中选择启动服务器菜单项可以启动 DM 数据库。启动方式类似 Windows。

2. Linux 服务方式

安装 DM 数据库后，在/etc/rc.d/init.d 中有名称为 DmService 开头的文件，文件全名为 DmService+实例名（例：如果实例名为 DMSERVER，则相对应的服务文件为 DmServiceDMSERVER ）。以实例名为 DMSERVER 为例，在终端输入./DmServiceDMSERVER start 或者 service DmServiceDMSERVER start 即可启动 DM 数据库。

3. 命令行方式

在终端进入 DM 安装目录下的 bin 目录，执行./dmserver 启动 DM 数据库，参数选项同 Windows。

8.1.3 检查 LICENSE

无论是在何种操作系统下运行，DM 数据库在启动时都会进行 LICENSE 检查。若 LICENSE 过期或 KEY 文件与实际运行环境不配套，DM 服务器会强制退出。

可通过查看 V$LICENSE 了解所安装的 DM 数据库的 LICENSE 信息。

8.2 数据库状态和模式

DM 数据库包含以下几种状态：

1. 配置状态（MOUNT）：不允许访问数据库对象，只能进行控制文件维护、归档配置、数据库模式修改等操作；
2. 打开状态（OPEN）：不能进行控制文件维护、归档配置等操作，可以访问数据库对

象，对外提供正常的数据库服务；

3. 挂起状态（SUSPEND）：与 OPEN 状态的唯一区别就是，限制磁盘写入功能；一旦修改了数据页，触发 REDO 日志、数据页刷盘，当前用户将被挂起。

OPEN 状态与 MOUNT 和 SUSPEND 能相互转换，但是 MOUNT 和 SUSPEND 之间不能相互转换。

DM 数据库包含以下几种模式：

1. 普通模式（NORMAL）：用户可以正常访问数据库，操作没有限制；

2. 主库模式（PRIMARY）：用户可以正常访问数据库，所有对数据库对象的修改强制生成 REDO 日志，在归档有效时，发送 REDO 日志到备库；

3. 备库模式（STANDBY）：接收主库发送过来的 REDO 日志并重做。数据对用户只读。

三种模式只能在 MOUNT 状态下设置，模式之间可以相互转换。

一般情况下，数据库为 NORMAL 模式，如果不指定 MOUNT 状态启动，则自动启动到 OPEN 状态。

在需要对数据库配置时（如配置数据守护、数据复制），服务器需要指定 MOUNT 状态启动。当数据库模式为非 NORMAL 模式（PRIMARY、STANDBY 模式），无论是否指定启动状态，服务器启动时自动启动到 MOUNT 状态。

8.3 关闭数据库

8.3.1 Windows 系统

1. 菜单方式

在 Windows 的开始->程序菜单中选择达梦数据库->DM 服务查看器，在弹出的界面中，选中要关闭的数据库，点击鼠标右键，在菜单栏中选择停止。

2. Windows 服务方式

安装 DM 数据库并且新建一个 DM 实例后。Windows 的服务中会自动增加一项和该实例名对应的服务。例如新建一个实例名为 DMSERVER1 的 DM 数据库，Windows 的服务中会增加一项名称为“DmServiceDMSERVER1”的服务。打开 Windows 的管理工具，选择服务，打开 Windows 服务控制台，如图 8.5 所示，选择“DmServiceDMSERVER1”，用鼠标在工具栏点击停止按钮或者点击鼠标右键，在菜单栏中选择停止，关闭 DM 数据库。

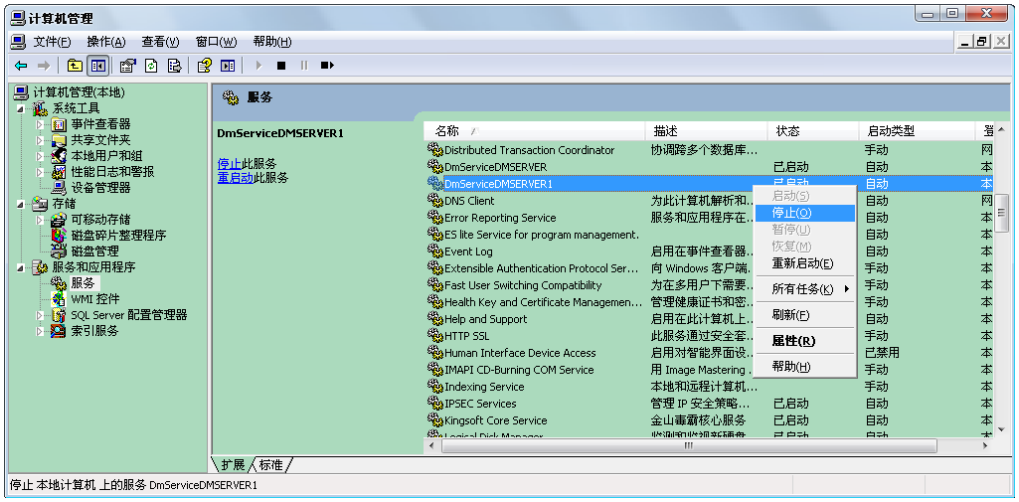


图 8.5 DM 数据库服务方式停止

3. 命令行方式

在启动数据库的命令工具中输入 exit，然后回车，关闭 DM 数据库。如图 8.6 所示。

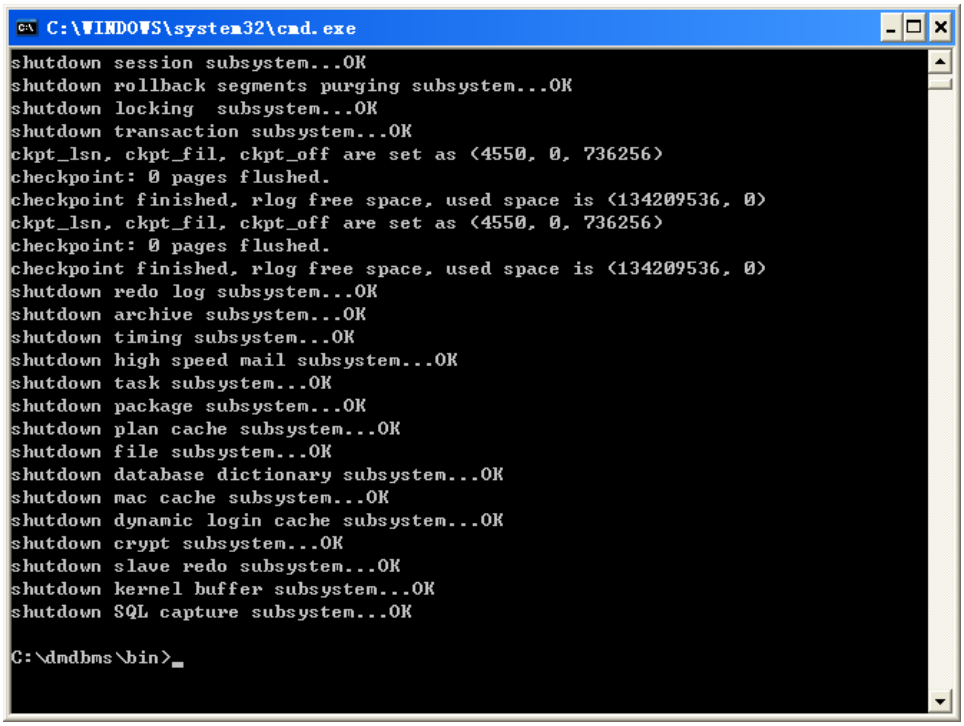


图 8.6 DM 数据库命令行方式停止

8.3.2 Linux 系统

1. 菜单方式

在 Linux 的开始菜单中选择关闭数据库的菜单项可以关闭 DM 数据库。关闭方式类似 Windows。

2. Linux 服务方式

进入/etc/rc.d/init.d，以实例名为 DMSERVER 为例，在命令行工具中输入./DmServiceDMSERVER stop 即可关闭 DM 数据库。

3. 命令行方式

在启动数据库的命令工具中输入 exit，然后回车，退出 DM 数据库。关闭方式类似 Windows。

第9章 管理模式对象的空间

模式对象的空间管理关系到空间的有效使用和数据的合理分布，本章介绍了模式对象相关的存储参数、多余空间的回收机制、模式对象上的空间限制、如何查看已使用的存储空间以及各种数据类型实际使用的空间大小。

9.1 设置存储参数

9.1.1 普通表和索引

对于普通表和索引，DM7 提供了以下的存储参数：

1. 初始簇数目 INITIAL：指建立表时分配的簇个数，必须为整数，最小值为 1，最大值为 256，缺省为 1；
2. 下次分配簇数目 NEXT：指当表空间不够时，从数据文件中分配的簇个数，必须为整数，最小值为 1，最大值为 256，缺省为 1；
3. 最小保留簇数目 MINEXTENTS：当删除表中的记录后，如果表使用的簇数目小于这个值，就不再释放表空间，必须为整数，最小值为 1，最大值为 256，缺省为 1；
4. 填充比例 FILLFACTOR：指定插入数据时数据页的充满程度，取值范围从 0 到 100。默认值为 0，等价于 100，表示全满填充，未充满的空间可供页内的数据更新时使用。插入数据时填充比例的值越低，可由新数据使用的空间就越多；更新数据时填充比例的值越大，更新导致出现的页分裂的几率越大；
5. 表空间名：在指定的表空间上建表或索引，表空间必须已存在，默认为用户缺省的表空间。

如表 PERSON 建立在表空间 TS_PERSON 中，初始簇大小为 5，最小保留簇数目为 5，下次分配簇数目为 2，填充比例为 85。

```
CREATE TABLE PERSON.PERSON
( PERSONID  INT IDENTITY(1,1) CLUSTER PRIMARY KEY,
SEX CHAR(1) NOT NULL,
NAME VARCHAR(50) NOT NULL,
EMAIL VARCHAR(50),
PHONE VARCHAR(25))
STORAGE
     ( INITIAL     5,  MINEXTENTS   5,
NEXT        2,  ON   TS_PERSON,   FILLFACTOR    85);
```

也可以在分区表上指定某个分区的存储参数，如下面的建表语句指定了 PAR2 分区存储在 TS_PAR2 表空间上。

```
CREATE TABLE PARTITION_TABLE
(C1 INT,
C2 INT)
PARTITION BY RANGE(C1)
 (PARTITION PAR1 VALUES LESS THAN(5),
PARTITION PAR2 VALUES LESS THAN(100) STORAGE (ON TS_PAR2));
```

9.1.2 堆表

对于堆表可以指定并发分支 BRANCH 和非并发分支 NOBRANCH 的数目，其范围是（1=<BRANCH <= 64，1<=NOBRANCH<=64），堆表最多支持 128 个链表，具体可参考第 19 章“管理堆表”。

如下例创建的 LIST_TABLE 表有并发分支 2 个，非并发分支 4 个。

```
CREATE TABLE LIST_TABLE(C1 INT) STORAGE(BRANCH (2,4));
```

9.1.3 HFS 表

HFS 表是建立在自己特有的 HTS 表空间上的。建立 HFS 表如果不使用默认的表空间，则必须要先创建一个 HUGE TABLESPACE，默认 HTS 表空间为 HMAIN。

如建立一个名称 HTS_NAME 的 HTS 表空间，表空间路径为为 e:\HTSSPACE。示例如下：

```
CREATE HUGE TABLESPACE HTS_NAME PATH 'e:\HTSSPACE';
```

对于 HFS 表可以指定如下参数：

1. 区大小（一个区的数据行数）。 区大小可以通过设置表的存储属性来指定，区的大小必须是 2 的多少次方，如果不是则向上对齐。取值范围：1024 行~1024*1024 行。默认值为 65536 行。
2. 是否记录区统计信息，即在修改时是否做数据的统计。
3. 所属的表空间。创建 HFS 表，需要通过存储属性指定其所在的表空间，不指定则存储于默认表空间 HMAIN 中。HFS 表指定的表空间只能是 HTS 表空间。
4. 文件大小。创建 HFS 表时还可以指定单个文件的大小，通过表的存储属性来指定，取值范围为 16M~1024*1024M。不指定则默认为 64M。文件大小必须是 2 的多少次方，如果不是则向上对齐。
5. 日志属性。1）LOG NONE：不做镜像；2）LOG LAST：做部分镜像；3）LOG ALL：全部做镜像。

如下面的建表语句：STUDENT 表的区大小为 65536 行，文件大小为 64M，指定所在的表空间为 HTS_NAME，做完整镜像，S_COMMENT 列指定的区大小为不做统计信息，其它列（默认）都做统计信息。

```
CREATE HUGE TABLE STUDENT
(
    S_NO            INT,
    S_CLASS         VARCHAR,
    S_COMMENT           VARCHAR(79) STORAGE(STAT NONE)
)STORAGE(SECTION(65536) , FILESIZE(64), ON HTS_NAME) LOG ALL;
```

9.2 收回多余的空间

DM7 中表和索引对象的所占用的簇要么是全满的状态要么是半满的状态，空闲的簇会被系统自动回收。

9.3 用户和表上的空间限制

9.3.1 用户的空间限制

用户占用的空间是其下所有用户表对象占用空间的总和。可以限制用户使用空间的大小，当用户创建表，创建索引，或者插入更新数据超过了指定的空间限制时，会报空间不足的错误。如创建用户 TEST_USER 时可指定该用户使用的最大磁盘空间为 50M。

```
CREATE USER TEST_USER IDENTIFIED BY TEST_PASSWORD DISKSPACE LIMIT 50;
```

对用户的空间限制也可进行更改，如修改用户 TEST_USER 的磁盘空间限制为无限制。

```
ALTER USER TEST_USER DISKSPACE UNLIMITED;
```

9.3.2 表对象的空间限制

表对象占用的空间是其上所有索引占用空间的总和。可以限制表对象使用空间的大小，当在表对象上创建索引或者插入更新数据超过了指定的空间限制时，会报空间不足的错误。如创建表 TEST 时可指定该表对象可使用的最大磁盘空间为 500M。

```
CREATE TABLE TEST (SNO INT, MYINFO VARCHAR) DISKSPACE LIMIT 500;
```

对表对象空间的限制也可进行更改，如修改表 TEST 的磁盘空间限制为 50M。

```
ALTER TABLE TEST MODIFY DISKSPACE LIMIT 50;
```

9.4 查看模式对象的空间使用

9.4.1 查看用户占用的空间

可以使用系统函数 USER_USED_SPACE 得到用户占用空间的大小，函数参数为用户名，返回值为占用的页的数目。

```
SELECT USER_USED_SPACE('TEST_USER');
```

9.4.2 查看表占用的空间

可以使用系统函数 TABLE_USED_SPACE 得到表对象占用空间的大小，函数参数为模式名和表名，返回值为占用的页的数目。

```
SELECT TABLE_USED_SPACE('SYSDBA', 'TEST');
```

9.4.3 查看表使用的页数

可以使用系统函数 TABLE_USED_PAGES 得到表对象实际使用页的数目，函数参数为模式名和表名，返回值为实际使用页的数目。

```
SELECT TABLE_USED_PAGES('SYSDBA', 'TEST');
```

9.4.4 查看索引占用的空间

可以使用系统函数 INDEX_USED_SPACE 得到索引占用空间的大小，函数参数为索引 ID，返回值为占用的页的数目。

```
SELECT INDEX_USED_SPACE(33555463);
```

9.4.5 查看索引使用的页数

可以使用系统函数 INDEX_USED_PAGES 得到索引实际使用页的数目，函数参数为索引 ID，返回值为实际使用页的数目。

```
SELECT INDEX_USED_PAGES(33555463);
```

9.5 数据类型的空间使用

各种数据类型占用的空间是不同的，下表 9.1 列出了主要数据类型所需要的空间。

表 9.1 主要数据类型所需的空间

数据类型	所需空间
CHAR	SIZE 为 1~8188 字节， 具体情况受到页面大小和记录大小的共同限制
VARCHAR	SIZE 为 1~8188 字节， 具体情况受到页面大小和记录大小的共同限制
TINYINT BIT BYTE	需要 1 个字节
SMALLINT	需要 2 个字节
INT	需要 4 个字节
BIGINT	需要 8 个字节
REAL	需要 4 个字节
FLOAT	需要 8 个字节
DOUBLE DOUBLE PRECISION	需要 8 个字节
DEC DECIMAL NUMERIC	SIZE 为 1~20 个字节
BINARY	SIZE 为 1~8188 字节， 具体情况受到页面大小和记录大小的共同限制
VARBINARY	SIZE 为 1~8188 字节， 具体情况受到页面大小和记录大小的共同限制
DATE	需要 3 个字节
TIME	需要 5 个字节
TIMESTAMP DATETIME	需要 8 个字节
TIME WITH TIME ZONE	需要 7 个字节
TIMESTAMP WITH TIME ZONE	需要 10 个字节

INTERVAL YEAR INTERVAL MONTH INTERVAL YEAR TO MONTH	需要 12 个字节
INTERVAL DAY INTERVAL HOUR INTERVAL MINUTE INTERVAL SECOND INTERVAL DAY TO HOUR INTERVAL DAY TO MINUTE INTERVAL DAY TO SECOND INTERVAL HOUR TO MINUTE INTERVAL HOUR TO SECOND INTERVAL MINUTE TO SECOND	需要 24 个字节
BLOB IMAGE LONGVARBINARY	SIZE 为 1~2G 字节
CLOB TEXT LONGVARCHAR	SIZE 为 1~2G 字节

第10章 管理表

表是数据库中数据存储的基本单元，是对用户数据进行读和操纵的逻辑实体。表由列和行组成，每一行代表一个单独的记录。表中包含一组固定的列，表中的列描述该表所跟踪的实体的属性，每个列都有一个名字及各自的特性。

列的特性由两部分组成:数据类型（dataType）和长度（length）。对于 NUMERIC、DECIMAL 以及那些包含秒的时间间隔类型来说，可以指定列的小数位及精度特性。在 DM 系统中，CHAR、CHARACTER、VARCHAR 数据类型的最大长度由数据库页面大小决定，数据库页面大小在初始化数据库时指定。DM 系统具有 SQL-92 的绝大部分数据类型，以及部分 SQL-99、Oracle 和 SQL Server 的数据类型。

为了确保数据库中数据的一致性和完整性，在创建表时可以定义表的实体完整性、域完整性和参考完整性。实体完整性定义表中的所有行能唯一地标识，一般用主键、唯一索引、UNIQUE 关键字及 IDENTITY 属性来定义；域完整性通常指数据的有效性，限制数据类型、缺省值、规则、约束、是否可以为空等条件，域完整性可以确保不会输入无效的值；参考完整性维护表间数据的有效性、完整性，通常通过建立外键联系另一表的主键来实现。

如果用户在创建表时没有定义表的完整性和一致性约束条件，那么用户可以利用 DM 所提供的基表修改语句来进行补充或修改。DM 系统提供基表修改语句，可对基表的结构进行全面的修改，包括修改基表名、列名、增加列、删除列、修改列类型、增加表级约束、删除表级约束、设置列缺省值、设置触发器状态等一系列修改功能。

本章描述管理表的几个方面，包括以下内容：

1. 管理表的准则；
2. 创建表；
3. 更改表；
4. 删除表；
5. 清空表；
6. 查看表信息。

10.1 管理表的准则

10.1.1 设计表

表是数据库设计过程中的基本构件，基于来自应用开发者的有关应用如何运作和所期望的数据类型，数据库管理员应与应用开发者一起工作，并认真规划每个表，具体需要做到以下几点：

1. 规范化表，估算并校正表结构，使数据冗余达到最小；
2. 为每个列选择合适的数据类型，是否允许为空等，并根据实际情况判断是否需要对列进行加密或压缩处理；
3. 建立合适的完整性约束，管理约束可查看 15 章管理完整性约束的内容；
4. 建立合适的聚集索引。每个表（列存储表，堆表除外）都含一个聚集索引，默认以 ROWID 建立，而建立合适的聚集索引，可以有效加快表的检索效率；
5. 根据实际需要，建立合适类型的表。DM 支持的表类型包括普通表、临时表、水平分区表、垂直分区表、堆表和列存储表。本章只介绍普通表和临时表，其他类型表将在其他章节中重点介绍。

10.1.2 指定表的存储空间上限

在创建表时指定 SPACE LIMIT 子句，可以对表的存储空间指定上限。DM 支持对表的存储空间指定大小，单位是 MB，即表的大小可由管理员指定，便于表的规模管理。当表的所有索引所占用的存储空间超过指定大小时，表将不能再新增数据。

10.1.3 指定表的存储位置

创建表时，在 STORAGE 子句中，可对表指定存储的表空间。如果没有指定，则该表将创建在用户的默认表空间中。

在创建表时，通过指定合适的表空间，有以下优点：

1. 提高数据库系统的性能，因为不同的数据库表可能对应不同的数据文件，可减少对相同文件的竞争；
2. 减少数据库管理的时间，数据库表分布在不同的表空间中，即使一个表空间损坏，也不影响其他表空间上数据库表的正常访问。

10.2 创建表

10.2.1 创建普通表

如果要在所属模式中创建新表，需要有 CREATE TABLE 数据库权限；而要在其他用户的模式中创建新表，则需要有 CREATE ANY TABLE 数据库权限。创建表时，应当为表指定一个表空间，否则，表将在 MAIN 创建。下面给出一个创建一个简单表的例子。

```
CREATE  TABLE  EMPLOYEE (
  EMPNO INT             PRIMARY KEY,
  ENAME VARCHAR(15)     NOT NULL,
  JOB       VARCHAR(10),
  MGR       INT
     CONSTRAINT EMP_FKEY REFERENCES EMPLOYEE(EMPNO),
  HIREDATE  DATE            DEFAULT (CURDATE),
  SALARY        FLOAT,
  DEPTNO    TINYINT         NOT NULL
CONSTRAINT DEPT_FKEY REFERENCES DEPT(DEPTNO))
STORAGE (
INITIAL         50,
NEXT            50,
MINEXTENTS      10,
FILLFACTOR      80,
ON              USERS);
```

在上述 CREATE TABLE 语句中，在 users 表空间上建立了 employee 表，并有几个完整性约束，其中包含定义在不同列上的一个主键和外键。约束将在 15 章中讨论。

创建表之后，可以使用 INSERT 命令插入数据或使用达梦数据导入导出工具装载数据，还可以直接使用 CREATE TABLE AS SELECT 创建一个表。

10.2.2 指定表的聚集索引

表（列存储表和堆表除外）都是使用 B+树(以下简称 B 树)索引结构管理的，每一个普通表都有一个聚集索引，数据通过聚集索引键排序，根据聚集索引键可以快速查询任何记录。

当建表语句未指定聚集索引键，DM 的默认聚集索引键是 ROWID，即记录默认以 ROWID 在页面中排序。ROWID 是 B 树为记录生成的逻辑递增序号，表上不同记录的 ROWID 是不一样的，并且最新插入的记录 ROWID 最大。很多情况下，以 ROWID 建的默认聚集索引并不能提高查询速度，因为实际情况下很少人根据 ROWID 来查找数据。

因此，DM 提供三种方式供用户指定聚集索引键：

1. CLUSTER PRIMARY KEY：指定列为聚集索引键，并同时指定为主键，称为聚簇主键；
2. CLUSTER KEY：指定列为聚集索引键，但是是非唯一的；
3. CLUSTER UNIQUE KEY：指定列为聚集索引键，并且是唯一的。

例如，创建 student 表，指定 stu_no 为聚簇主键。

```
CREATE  TABLE  STUDENT(
  STUNO     INT             CLUSTER PRIMARY KEY,
  STUNAME   VARCHAR(15)     NOT NULL,
  TEANO     INT,
  CLASSID   INT
);
```

指定聚簇索引键后，如果查询条件中含有聚簇索引键，可以定位记录在 B 树上的位置，使查询性能大大提高。然而，插入记录也需要根据聚簇索引键定位插入位置，有可能导致页面的分裂而影响插入性能。

在 dm.ini 配置文件中，可以指定配置项使表中的主键自动转化为聚簇主键，该配置项为 PK_WITH_CLUSTER。默认情况下，PK_WITH_CLUSTER 为 1，即建表时指定的主键自动转化为聚簇主键；若为 0，则主键不会自动变为聚簇主键。

10.2.3 指定表的填充因子

上文提到，每个普通表都含有一个聚集索引，指定表的填充因子，即指定聚集索引的填充因子。索引的填充因子指在新建和重组索引时，页面记录存储空间占页面总大小的百分比。而这部分预留空间是为更新字段时使用的，一个有效的填充因子可以大大减少由更新记录导致的页面拆分。

例如，上述的创建 employee 表语句中的 STORAGE 子句，指定了 FILLFACTOR 为 80，即填充因子为 80%。

当填充因子取值低，则需要更多的页来存储数据，因而读取范围大，会影响性能。而当填充因子取值高，则在更新数据时可能造成大量的页拆分，页拆分需要消耗较多 CPU 和 I/O 资源，同样会影响性能。原则上，在只读表上应该设置填充因子高，而有大量更新的表上应该设置较低的值。默认情况下，DM 新建的表和索引的填充因子是 100，可根据实际情况设置合适的填充因子大小。

10.2.4 查询建表

为了创建一个与已有表相同的新表，或者为了创建一个只包含另一个表的一些行和列的新表，可以使用 CREATE TABLE AS SELECT(CTAS)命令。使用该命令，可以通过使用 WHERE

条件将已有表中的一部分数据装载到一个新表中，或者可以通过 SELECT * FROM 子句将已有表的所有数据装载到创建的表中，如以下程序片段所示：

```
CREATE TABLE NEW_EMP
AS
SELECT * FROM EMPLOYEE;
```

如果用户通过单表的全表查询进行建表操作，则可以通过将 INI 参数 CTAB_SEL_WITH_CONS 置为 1 进行原始表上约束的拷贝，列上能拷贝的约束包括默认值属性、自增属性、非空属性以及加密属性，表上能拷贝的约束包括唯一约束、PK 约束以及 CHECK 约束。

10.2.5 创建临时表

当处理复杂的查询或事务时，由于在数据写入永久表之前需要暂时存储一些行信息或需要保存查询的中间结果，可能需要一些表来临时存储这些数据。DM 允许创建临时表来保存会话甚至事务中的数据。在会话或事务结束时，这些表上的数据将会被自动清除。

临时表中的数据不能像在其它永久表中的数据那样进行备份，当事务结束或会话断开时，数据就会被清空。在临时表创建过程中，不会像永久表和索引那样自动分配数据段，而是仅当第一次执行 DML 语句时，才会为临时表在临时表空间中分配空间。并且，对于不同的会话，临时表上的数据是独享的，不会互相干扰，即会话 A 不能访问会话 B 临时表上的数据。

对复杂查询的传统响应方式之一是使用一个视图，使复杂查询更易于操作。但是，视图在每次访问时都需要执行，因而大大降低了性能。而通过 AS SELECT 子句建立的临时表是将复杂查询的结果通过临时 B 树记录了下来，下次访问不用重新执行查询就可以获得数据，并且会话或事务结束后数据将自动删除，是复杂查询的一个优秀的解决方案，且提高了性能。

DM 临时表支持以下功能：

1. 在临时表中，会话可以像普通永久表一样更新、插入和删除数据；
2. 临时表的 DML 操作产生较少的 REDO 日志；
3. 临时表支持建索引，以提高查询性能；
4. 在一个会话或事务结束后，数据将自动从临时表中删除；
5. 不同用户可以访问相同的临时表，每个用户只能看到自己的数据；
6. 临时表的数据量很少，意味着更高效的查询效率；
7. 临时表的表结构在数据删除后仍然存在，便于以后的使用；
8. 临时表的权限管理跟普通表一致。

临时表 ON COMMIT 关键词指定表中的数据是事务级还是或会话级的，默认情况下是事务级的。

1. ON COMMIT DELETE ROWS：指定临时表是事务级的，每次事务提交或回滚之后，表中所有数据都被删除；
2. ON COMMIT PRESERVE ROWS：指定临时表是会话级的，会话结束时才清空表，并释放临时 B 树。

下面的例子创建一个事务级的临时表：

```
CREATE GLOBAL TEMPORARY TABLE TMP_EMP(
 EMPNO  INT     PRIMARY KEY,
 ENAME  VARCHAR(15)     NOT NULL,
 JOB    VARCHAR(10))
ON COMMIT DELETE ROWS;
```

10.3 更改表

想更改的表如果在所属的模式中，用户必须具有 ALTER TABLE 数据库权限；若在其他模式中，用户必须有 ALTER ANY TABLE 的数据库权限。

通过更改表，用户可以对数据库中的表作如下修改：

1. 添加或删除列，或修改现有的列的定义（列名、数据类型、长度、默认值）。其中，对于添加列，当设置 INI 参数 ALTER_TABLE_OPT 为 1 时，添加列采用查询插入实现，会导致 ROWID 的改变；ALTER_TABLE_OPT 为 2 时，系统开启快速加列功能，对于没有默认值或者默认值为 NULL 的新列，系统内部会标记为附加列，能够达到瞬间加列的效果，此时 ROWID 不会改变，若有默认值且默认值不为 NULL，则仍旧采取查询插入实现。
2. 添加、修改或删除与表相关的完整性约束；
3. 重命名一个表；
4. 启动或停用与表相关的完整性约束；
5. 启动或停用与表相关的触发器；
6. 修改表的 SPACE LIMIT；
7. 增删自增列。

10.4 删除表

当一个表不再使用时，可以将其删除。删除表时，将产生以下结果：

1. 表的结构信息从数据字典中删除，表中的数据不可访问；
2. 表上的所有索引和触发器被一起清除；
3. 所有建立在该表上的同义词、视图和存储过程变为无效；
4. 所有分配给表的簇标记为空闲，可被分配给其他的数据库对象。

一般情况下，普通用户只能删除自己模式下的表。若要删除其他模式下的表，则必须具有 DROP ANY TABLE 数据库权限。

以下语句可删除 employee 表：

```
DROP TABLE employee;
```

如果要删除的表被其他表引用，即其他表的外键引用了表的任何主键或唯一键，则需要在 DROP TABLE 语句中包含 CASCADE 选项，如：

```
DROP TABLE employee CASCADE;
```

10.5 清空表

有些情况下，当表的数据不再使用时，需要删除表的所有行，即清空该表。DM7 支持以下方式来删除表中的所有的行：

1. 使用 DELETE 语句；
2. 使用 DROP 和 CREATE 语句；
3. 使用 TRUNCATE 语句。

10.5.1 使用 DELETE

使用 DELETE 语句能删除表中的行。例如，下面的语句删除 employee 表中的所有行：

```
DELETE FROM employee;
```

但是，使用 DELETE 清空表，当表有很多行时，会消耗很多系统资源。因为，DELETE 操作需要 CPU 时间，并且会产生大量的 REDO 日志和 UNDO 记录。另外，如果表上关联了元组级触发器，每删除一行，就会启动一次触发器。这都需要大量的系统资源。

10.5.2 使用 DROP 和 CREATE

使用 DROP 删除一个表，然后创建一个同名的表，也可以达到清空表的效果。例如，下面的语句先删除 employe 表，之后再重新创建。

```
DROP TABLE EMPLOYEE;
CREATE TABLE EMPLOYEE(…);
```

当删除和重新创建表时，所有与之相关联的索引、完整性约束和触发器也被删除。同样，所有针对被删除表的授权也会被删除。

10.5.3 使用 TRUNCATE

使用 TRUNCATE 语句能删除表中的所有行。例如，下面的语句清空 employee 表。

```
TRUNCATE TABLE EMPLOYEE;
```

TRUNCATE 语句为我们提供了一种快速、有效地删除表所有行的方法。并且 TRUNCATE 是一个 DDL 语句，不会产生任何回滚信息。执行 TRUNCATE 会立即提交，而且不能回滚。

TRUNCATE 语句并不影响与被删除的表相关联的任何结构、约束、触发器或者授权。 另外，DM 数据库 TRUNCATE 表后，原来分配给该表的空间会被释放，供其他数据库对象使用，大大提高空间的利用效率。

一般情况下，普通用户只能 TRUNCATE 自己模式下的表。若要 TRUNCATE 其他模式下的表，则必须具有 DROP ANY TABLE 数据库权限。

如果要清空的表被其他表引用，即其他表的外键引用了表的任何主键或唯一键，并且子表不为空或子表的外键约束未被禁用，则不能 TRUNCATE 该表。

10.6 查看表信息

10.6.1 查看表定义

创建表后，可以通过 SP_TABLEDEF 系统过程查看表的定义。

```
CALL SP_TABLEDEF('SYSDBA', 'EMPLOYEE');
```

DM 通过提供的 TABLEDEF 函数来显示当前表的定义。当表多次进行 ALTER TABLE 后，显示的表定义将是最后一次修改后的建表语句。

10.6.2 查看自增列信息

DM 支持 INT 和 BIGINT 两种数据类型的自增列，并提供以下函数查看表上自增列的当前值、种子和增量等信息：

1. IDENT_CURRENT：获得表上自增列的当前值；
2. IDENT_SEED：获得表上自增列的种子信息；
3. IDENT_INCR：获得表上自增列的增量信息。

```
CREATE TABLE IDENT_TABLE (
```

```
    C1          INT     IDENTITY(100, 100),
    C2          INT
);
SELECT IDENT_CURRENT('SYSDBA.IDENT_TABLE');
SELECT IDENT_SEED('SYSDBA.IDENT_TABLE');
SELECT IDENT_INCR('SYSDBA.IDENT_TABLE');
```

10.6.3 查看表的空间使用情况

DM 使用段、簇和页实现数据的物理组织。DM 支持查看表的空间使用情况，包括：

1. TABLE_USED_SPACE：已分配给表的页面数；
2. TABLE_USED_PAGES：表已使用的页面数。

```
CREATE TABLE SPACE_TABLE (
    C1          INT,
    C2          INT
);
SELECT TABLE_USED_SPACE('SYSDBA','SPACE_TABLE');
SELECT TABLE_USED_PAGES('SYSDBA','SPACE_TABLE');
```

第11章 管理索引

11.1 管理索引的准则

索引是与表相关的可选的结构（聚簇索引除外），它能使对应于表的 SQL 语句执行得更快，因为有索引比没有索引能更快地定位信息。DM7 索引能提供访问表的数据的更快路径，可以不用重写任何查询而使用索引，其结果与不使用索引是一样的，但速度更快。

DM7 提供了几种最常见类型的索引，对不同场景有不同的功能，它们是：

1. 聚集索引：每一个普通表有且只有一个聚集索引；
2. 唯一索引：索引数据根据索引键唯一；
3. 函数索引：包含函数/表达式的预先计算的值；
4. 位图索引：对低基数的列创建位图索引；
5. 位图连接索引：针对两个或者多个表连接的位图索引，主要用于数据仓库中；
6. 全文索引：在表的文本列上而建的索引。具体内容请参考第 20 章。

索引在逻辑上和物理上都与相关的表的数据无关，作为无关的结构，索引需要存储空间。创建或删除一个索引，不会影响基本的表、数据库应用或其他索引。当插入、更改和删除相关的表的行时，DM7 会自动管理索引。如果删除索引，所有的应用仍继续工作，但访问以前被索引了的数据时速度可能会变慢。

11.1.1 在表中插入数据后创建索引

一般情况下，在插入或装载了数据后，为表创建索引会更加有效率。如果在装载数据之前创建了一个或多个索引，那么在插入每行时 DM7 都必须更改和维护每个索引，使得插入效率降低。

11.1.2 索引正确的表和列

使用下面的准则来决定何时创建索引：

1. 如果需要经常地检索大表中的少量的行，就为查询键创建索引；
2. 为了改善多个表的连接的性能，可为连接列创建索引；
3. 主键和唯一键自动具有索引，在外键上很多情况下也创建索引；
4. 小表不需要索引。

选取表中的索引列时可以考虑以下几点：

1. 列中的值相对比较唯一 ；
2. 取值范围大，适合建立索引；
3. CLOB 和 TEXT 只能建立全文索引、BLOB 不能建立任何索引。

11.1.3 为性能而安排索引列

在 CREATE INDEX 语句中列的排序会影响查询的性能。通常，将最常用的列放在最前面。

如果查询中有多个字段组合定位，则不应为每个字段单独创建索引，而应该创建一个组

合索引。当两个或多个字段都是等值查询时，组合索引中各个列的前后关系是无关紧要的。但是如果是非等值查询时，要想有效利用组合索引，则应该按等值字段在前，非等值字段在后的原则创建组合索引，查询时只能利用一个非等值的字段。

11.1.4 限制每个表的索引的数量

一个表可以有任意数量的索引。但是，索引越多，修改表数据的开销就越大。当插入或删除行时，表上的所有索引也要被更改；更改一个列时，包含该列的所有索引也要被更改。因此，在从表中检索数据的速度和更新表的速度之间有一个折衷。例如，如果一个表主要仅用于读，则索引多就有好处；如果一个表经常被更新，则索引不宜多建。

11.1.5 估计索引大小和设置存储参数

创建索引之前先估计索引的大小能更好地促进规划和管理磁盘空间。可以用索引以及回滚段、重做日志文件的组合估计的大小来决定支持所期望的数据库所需的磁盘空间的大小。通过这些估计，就可以购买合适的硬件和做出其他正确的决定。

用单个索引估计的大小能更好地管理索引使用的磁盘空间。创建索引时，可以设置适当的存储参数，并改善使用该索引的应用的 I/O 性能。例如，假设在创建索引之前估计索引的最大大小，之后就可以在创建该索引时设置适当的存储参数，就能很少为表的数据段分配簇。并且，所有的该索引的数据都被保存在相对连续的磁盘空间扇区中，这就减少了使用该索引的磁盘 I/O 操作所需的时间。

11.1.6 为每个索引指定表空间

可以在除临时表空间、日志表空间和回滚段表空间外的其他任何表空间中创建索引，也可以在其索引的表的相同或不同的表空间中创建索引。如果表及其索引使用相同的表空间 能更方便地对数据库进行管理（如表空间或文件备份）或保证应用的可用性，因为所有有关的数据总是在一起联机。然而，将表及其索引放在不同的表空间（在不同磁盘上）产生的性能比放在相同的表空间更好，因为这样做减少了磁盘竞争。但是将表及其索引放在不同的表空间时，如果一个表上某索引所在的表空间脱机了，则涉及这张表的 SQL 语句可能由于执行计划仍旧需要使用被脱机的索引而不能成功执行。

11.2 创建索引

本节描述如何创建索引。要在用户自己的模式中创建索引，至少要满足如下条件之一：

1. 要被索引的表是在自己的模式中 ；
2. 在要被索引的表上有 CREATE INDEX 权限；
3. 具有 CREATE ANY INDEX 数据库权限。

要在其他模式中创建索引，用户必须具有 CREATE ANY INDEX 数据库权限。

11.2.1 明确地创建索引

可以用 CREATE INDEX 语句明确地创建索引。如下语句在 emp 表的 ename 列上创建一个名为 emp_ename 的索引，该索引使用表空间 users。

```
CREATE INDEX emp_ename ON emp(ename)
STORAGE (
INITIAL 50,
NEXT    50,
ON      USERS);
```

注意，上述语句为该索引明确地指定了几个存储设置和一个表空间。如果没有给索引指定存储选项，则 INITIAL 和 NEXT 等存储选项会自动使用表空间的默认存储选项。

11.2.2 创建聚集索引

DM7 中表（列存储表和堆表除外）都是使用 B+树索引结构管理的，每一个普通表都有且仅有一个聚集索引，数据都通过聚集索引键排序，根据聚集索引键可以快速查询任何记录。

当建表语句未指定聚集索引键时，DM7 的默认聚集索引键是 ROWID。若指定索引键，表中数据都会根据指定索引键排序。

建表后，DM7 也可以用创建新聚集索引的方式来重建表数据，并按新的聚集索引排序。例如，可以对 emp 表以 ename 列新建聚集索引。

```
CREATE CLUSTER INDEX clu_emp_name ON emp(ename);
```

新建聚集索引会重建这个表以及其所有索引，包括二级索引、函数索引，是一个代价非常大的操作。因此，最好在建表时就确定聚集索引键，或在表中数据比较少时新建聚集索引，而尽量不要对数据量非常大的表建立聚集索引。

创建聚集索引的约束条件：

1. 每张表中只允许有一个聚集索引，如果之前已经指定过 CLUSTER INDEX 或者指定了 CLUSTER PK，则用户新建立 CLUSTER INDEX 时系统会自动删除原先的聚集索引。但如果新建聚集索引时指定的创建方式（列，顺序）和之前的聚集索引一样，则会报错；
2. 指定 CLUSTER INDEX 操作需要重建表上的所有索引，包括 PK 索引；
3. 删除聚集索引时，缺省以 ROWID 排序，自动重建所有索引；
4. 若聚集索引是默认的 ROWID 索引，不允许删除；
5. 聚集索引不能应用到函数索引中；
6. 垂直分区表在建表后不能更改聚集索引；如果建表时没有指定聚集索引，后续也不能再指定；
7. 不能在列存储表上新建/删除聚集索引；
8. 建聚集索引语句不能含有 partition_clause 子句；
9. 在临时表上增删索引会使当前会话上临时 b 树数据丢失。

11.2.3 明确地创建唯一索引

索引可以是唯一的或非唯一的。唯一索引可以保证表上不会有两行数据在键列上具有相同的值，非唯一索引不会在键列上施加这个限制。

可用 CREATE UNIQUE INDEX 语句来创建唯一索引，如下例子创建一个唯一索引：

```
CREATE UNIQUE INDEX dept_unique_index ON dept (dname)
STORAGE (ON users);
```

用户可以在希望的列上定义 UNIQUE 完整性约束，DM7 通过自动地在唯一键上定义一个唯一索引来保证 UNIQUE 完整性约束。

11.2.4 自动创建与约束相关的唯一索引

DM7 通过在唯一键或主键上创建一个唯一索引来在表上实施 UNIQUE KEY 或 PRIMARY KEY 完整性约束。当启用约束时 DM7 自动创建该索引。如下面的语句会自动在表 emp 的 name 列上创建一个唯一索引。

```
ALTER TABLE EMP ADD CONSTRAINT PK_EMP_NAME PRIMARY KEY (NAME);
```

11.2.5 创建基于函数的索引

基于函数的索引促进了限定函数或表达式的返回值的查询，该函数或表达式的值被预先计算出来并存储在索引中。正确使用函数索引，可以带来以下好处：

1. 创建更强有力的分类，例如可以用 UPPER 和 LOWER 函数执行区分大小写的分类；
2. 预先计算出计算密集的函数的值，并在索引中将其分类。可以在索引中存储要经常访问的计算密集的函数，当需要访问值时，该值已经计算出来了。因此，极大地改善了查询的执行性能；
3. 增加了优化器执行范围扫描而不是全表扫描的情况的数量。

例如，考虑如下 WHERE 子句中的表达式

```
CREATE INDEX IDX ON EXAMPLE_TAB(COLUMN_A + COLUMN_B);
SELECT * FROM EXAMPLE_TAB WHERE COLUMN_A + COLUMN_B < 10;
```

因为该索引是建立在 column_a + column_b 之上的，所以优化器可以为该查询使用范围扫描。优化器根据该索引计算查询代价，如果代价最少，优化器就会选择该函数索引，column_a + column_b 就不会重复计算。

创建函数索引有以下约束条件：

1. 函数索引表达式可以由多列组成，不同的列不能超过 63 个；
2. 函数索引表达式里面不允许出现大字段和时间间隔类型列；
3. 不支持建立分区函数索引；
4. 函数索引表达式的长度理论值不能超过 816 个字符（包括生成后的指令和字符串）；
5. 表达式中必须包含表列；
6. 函数索引不能为 CLUSTER 或 PRIMARY KEY 类型；
7. 表达式不支持集函数和不确定函数，不确定函数为每次执行得到的结果不确定，系统中不确定函数包括：RAND、SOUNDEX、CURDATE、CURTIME、CURRENT_DATE、CURRENT_TIME、CURRENT_TIMESTAMP、GETDATE、NOW、SYSDATE、CUR_DATABASE、DBID、EXTENT、PAGE、SESSID、UID、USER、VSIZE、SET_TABLE_OPTION、SET_INDEX_OPTION、UNLOCK_LOGIN、CHECK_LOGIN、GET_AUDIT、CFALGORITHMSENCRYPT、SF_MAC_LABEL_TO_CHAR、CFALGORITHMSDECRYPT、BFALGORITHMSENCRYPT、SF_MAC_LABEL_FROM_CHAR、BFALGORITHMSDECRYPT、SF_MAC_LABEL_CMP；
8. 快速装载不支持含有函数索引的表；
9. 当表中含有行前触发器并且该触发器会修改函数索引涉及列的值时，不能建立函数索引；
10. 若函数索引中要使用用户自定义的函数，则函数必须是指定了 DETERMINISTIC 属性的确定性函数；
11. 若函数索引中使用的确定性函数发生了变更或删除，用户需手动重建函数索引；
12. 若函数索引中使用的确定性函数内有不确定因素，会导致前后计算结果不同的情况。在查询使用函数索引时，使用数据插入函数索引时的计算结果为 KEY 值；修改时可能会导致在使用函数索引过程中出现根据聚集索引无法在函数索引中找到相

应记录的情况，对此进行报错处理。

11.2.6 创建位图索引

位图索引主要针对含有大量相同值的列而创建。位图索引被广泛引用到数据仓库中，创建方式和普通索引一致，对低基数（不同的值很少）的列创建位图索引，能够有效提高基于该列的查询效率。且执行查询语句的 where 子句中带有 AND 和 OR 谓词时，效率更加明显。

如下例子创建一个位图索引：

```
CREATE  BITMAP INDEX  S1  ON  PURCHASING.VENDOR (VENDORID);
```

位图索引具有以下约束：

1. 支持普通表、堆表和水平分区表创建位图索引；
2. 不支持对大字段创建位图索引；
3. 不支持对计算表达式列创建位图索引；
4. 不支持在 UNIQUE 列和 PRIMARY KEY 上创建位图索引；
5. 不支持对存在 CLUSTER KEY 的表创建位图索引；
6. 仅支持单列或者不超过 64 个组合列上创建位图索引；
7. MPP 环境下不支持位图索引的创建；
8. 不支持快速装载建含有位图索引的表；
9. 不支持全局位图索引；
10. 包含位图索引的表不支持并发的插入、删除和更新操作。

11.2.7 创建位图连接索引

位图连接索引是一种提高通过连接实现海量数据查询效率的有效方式，主要用于数据仓库环境中。区别于上一节所说的建立在单表上的位图索引，位图连接索引是针对两个或者多个表的连接而建立的位图索引，同时保存了连接的位图结果。对于索引列中的每一个值，位图连接索引在索引表中保存了对应行的 ROWID。

如下例子创建一个位图连接索引：

```
create bitmap index SALES_CUSTOMER_NAME_IDX
on SALES.SALESORDER_HEADER(SALES.CUSTOMER.PERSONID)
from   SALES.CUSTOMER, SALES.SALESORDER_HEADER
where  SALES.CUSTOMER.CUSTOMERID = SALES.SALESORDER_HEADER.CUSTOMERID;
```

使用说明

1. 适用于常规索引的基本限制也适用于位图连接索引；
2. 用于连接的列必须是维度表中的主键或存在唯一约束；如果是复合主键，则必须使用复合主键中的所有列；
3. 当多个事务同时使用位图连接索引时，同一时间只允许更新一个表；
4. 连接索引创建时，基表只允许出现一次；
5. 不允许对存在 cluster key 的表创建位图连接索引；
6. 位图连接索引表（命名为 BMJ$_索引名）仅支持 select 操作，其他操作都不支持：如 insert、delete、update、alter、drop 和建索引等；
7. 位图连接索引不支持事实表、维度表的备份还原，不支持位图连接索引表的表级备份还原；
8. 不支持位图连接索引表、位图连接索引以及虚索引的导出导入；
9. 位图连接索引及其相关表不支持快速装载；
10. 位图连接索引名称的长度限制为：事实表名的长度+索引名称长度+6<128；

11. 仅支持普通表、堆表和 HFS 表；
12. WHERE 条件只能是列与列之间的等值连接，并且必须含有所有表；
13. 事实表上聚集索引和位图连接索引不能同时存在；
14. 不支持对含有位图连接索引的表中的数据执行 DML，如需要执行 DML,则先删除该索引；
15. 含有位图连接索引的表不支持下列 DDL 操作：删除、修改表约束，删除、修改列，更改表名。另外，含位图连接索引的堆表不支持添加列操作；
16. 不允许对含有位图连接索引的表并发操作；
17. 创建位图连接索引时，在存储参数中可指定存储位图的字节数，有效值为：1~ 128，服务器自动校正为 4 的倍数，默认值为 48。如 STORAGE(SECTION(4))，表示使用 4 个字节存储位图信息。

11.3 重建索引

当一个表经过大量的增删改操作后，表的数据在物理文件中可能存在大量碎片，从而影响访问速度。另外，当删除表的大量数据后，若不再对表执行插入操作，索引所处的段可能占用了大量并不使用的簇，从而浪费了存储空间。

可以使用重建索引来对索引的数据进行重组，使数据更加紧凑，并释放不需要的空间，从而提高访问效率和空间效率。DM7 提供的重建索引的系统函数为：

```
SP_REBUILD_INDEX(SCHEMA_NAME varchar(256), INDEX_ID int);
```

SCHEAM_NAME 为索引所在的模式名，INDEX_ID 为索引 ID。

使用说明：

1. 水平分区子表，临时表和系统表上建的索引不支持重建
2. 虚索引和聚集索引不支持重建

例如，需要重建索引 emp_name，假设其索引 ID 为 1547892，那么使用以下语句重建索引：

```
SP_REBUILD_INDEX('SYSDBA ', 1547892);
```

11.4 删除索引

用户可能出于以下某项原因需要删除一个索引：

1. 不再需要该索引；
2. 该索引没有为针对其相关的表所发布的查询提供所期望的性能改善。例如，表可能很小，或者尽管表中有许多行但只有很少的索引项；
3. 应用没有用该索引来查询数据。

要想删除索引，则该索引必须包含在用户的模式中或用户必须具有 DROP ANY INDEX 数据库权限。索引删除之后，该索引的段的所有簇都返回给包含它的表空间，并可用于表空间中的其他对象。

如何删除索引，取决于是否是用 CREATE INDEX 语句明确地创建该索引的，是则可以用 DROP INDEX 语句删除该索引。如下面的语句删除 emp_ename 索引。

```
DROP INDEX emp_ename;
```

然而，不能直接删除与已启用的 UNIQUE KEY 键或 PRIMARY KEY 键约束相关的索引。要删除一个与约束相关的索引，必须停用或删除该约束本身。如下面的语句删除主键约束 pk_emp_name，同时删除其对应的索引。

```
ALTER TABLE emp DROP CONSTRAINT pk_emp_name;
```

除了删除普通索引，DM7 还提供删除聚集索引，只要其聚集索引是通过 CREATE

CLUSTER INDEX 明确建立的。例如，下面的语句删除 emp 表的聚集索引 clu_emp_name。

```
DROP INDEX clu_emp_name;
```

删除聚集索引其实是使用 ROWID 作为索引列重建聚集索引，即跟新建聚集索引一样会重建这个表以及其所有索引。

删除表就自动删除了所有与其相关的索引。

11.5 查看索引信息

创建索引后，可以通过 INDEXDEF 系统函数查看索引的定义。

```
INDEXDEF(INDEX_ID int, PREFLAG int);
```

INDEX_ID 为索引 ID，PREFLAG 表示返回信息中是否增加模式名前缀。例如，需要查看索引 emp_name 的定义，假设其索引 ID 为 1547892，那么使用以下语句查看索引定义。

```
SELECT INDEXDEF(1547892, 0);
或 SELECT INDEXDEF(1547892, 1);
```

第12章 PL/SQL

12.1 PL/SQL 概述

PL/SQL 是达梦数据库对标准 SQL 语言的扩展，叫做过程化 SQL 语言。在 PL/SQL 程序中，包括一整套的数据类型、条件结构、循环结构和异常处理结构等，PL/SQL 可以执行 SQL 语句，SQL 语句中也可以使用 PL/SQL 函数。

12.1.1 PL/SQL 优点

与 SQL 相比，PL/SQL 体现了模块化的程序设计思想，具体优点如下：

1. PL/SQL 是一种高性能的基于事务处理的语言，能运行在 DM 环境中，支持所有数据处理命令。通过使用 PL/SQL 程序单元处理 SQL 的数据定义和数据控制元素；
2. PL/SQL 支持所有 SQL 数据类型和所有 SQL 函数，同时支持所有 DM 对象类型；
3. PL/SQL 块可以被命名和存储在 DM 数据库中，同时也能被其他的 PL/SQL 程序或 SQL 命令调用，任何客户/服务器工具都能访问 PL/SQL 程序，具有很好的可重用性；
4. 可以使用 DM 的管理工具 Manager 管理存储在服务器中的 PL/SQL 程序的安全性。可以授权或撤销数据库其他用户访问 PL/SQL 程序的能力；
5. PL/SQL 代码可以使用任何 ASCII 文本编辑器编写，所以对任何达梦数据库能够运行的操作系统都是非常便利的；
6. 对于 SQL，DM 服务器必须处理每一条 SQL 语句，在网络环境下这就意味着每一个独立的调用都必须被 DM 服务器处理，占用大量的服务器时间，容易导致网络拥挤，而 PL/SQL 是以整个语句块被服务器处理，降低了网络拥挤，同时可以大大提高访问数据库的效率。

12.1.2 PL/SQL 语句块

PL/SQL 语言的基本程序单元是语句块，主要的块形式有函数、过程和匿名块。每个语句块由关键字 DECLARE、BEGIN、EXCEPTION 和 END 划分为说明部分、执行部分和异常处理部分。其中执行部分是必须的，说明和异常处理部分可以省略。语句块可以嵌套，它可以出现在任何其他语句可以出现的位置，格式如下：

```
DECLARE
    变量声明部分
BEGIN
    可执行部分
EXCEPTION
    异常处理部分
END
```

其中变量声明部分用来定义变量、类型、游标、子程序、异常等。被声明的元素在本块范围内有效，这一部分在 PL/SQL 块中是可选的。

可执行部分是 PL/SQL 块的主体，它包含了 PL/SQL 块的可执行代码，是必不可少的部

分，PL/SQL 块的主要功能体现在这一部分。

异常处理部分用于处理 PL/SQL 块在执行过程中发生的错误。如果块执行正常，则块正常结束，否则从出现错误的语句开始，转至异常处理部分开始执行，即进行异常处理，这部分在 PL/SQL 块中也是可选的。

最简单的 PL/SQL 块仅包含由 BEGIN 和 END 限定的可执行部分。例如：

```
BEGIN
    PRINT 'HELLO WORLD';
END;
```

PL/SQL 块的代码可以直接在“SQL>”提示符下输入。输入结束后，执行命令“/”即可使该 PL/SQL 块开始执行。

12.2 PL/SQL 变量

变量的定义必须放在 PL/SQL 块的变量声明部分，变量定义有两种格式，分别为：

```
变量名[CONSTANT]  类型 [NOT NULL] [DEFAULT 默认值]
变量名[CONSTANT]  类型 [NOT NULL] [:=初始值]
```

变量名要遵守一定的命名规则。变量名必须以字母开头，包含数字、字母、下划线以及$、#符号，长度不能超过 128 字符，并且不能与达梦数据库的关键字相同，变量名与大小写是无关的。

变量在定义时可以指定默认值或初始值，在 PL/SQL 块的运行过程中也可以对其赋值。赋值语句有两种方式：

（1）直接赋值语句

```
变量名:=表达式 或 SET 变量名=表达式
```

（2）通过 SQL SELECT INTO 或 FETCH INTO 给变量赋值

```
SELECT <表达式>{,<表达式>} [INTO <主变量名>{,<主变量名>}] FROM <表引用>{,<表引用>} …;
```

或

```
FETCH [NEXT|PREV|FIRST|LAST|ABSOLUTE N|RELATIVE N]<游标名> [INTO<主变量名>{,<主变量名>}];
```

注：FETCH 中的 N，起始值为 0。

常量与变量相似，但常量的值在程序内部不能改变，常量的值在定义时赋予，它的声明方式与变量相似，但必须包含关键字 CONSTANT。

如果需要打印输出变量的值，则要调用 PRINT 语句，如果数据类型不一致，则系统会自动将它转换为 VARCHAR 类型输出。除了变量的声明外，变量的赋值、输出等操作都要放在 PL/SQL 块的可执行部分。

下面是一个变量声明和赋值的例子。

```
CREATE TABLE STUDENT(NAME VARCHAR(30) , AGE INT, SNO INT PRIMARY KEY);
INSERT INTO STUDENT VALUES('ZHANGSAN', 18, 10010);
INSERT INTO STUDENT VALUES('LISI', 20, 10011);
CREATE OR REPLACE PROCEDURE PROC AS
C1 CHAR(20) DEFAULT 'ABC';              --默认值
C2 CONSTANT VARCHAR:= 'HELLO WORLD!';   --常量
C3 INT NOT NULL := 1;                   --约束
C4 INT;
NAME1 VARCHAR;
SNO1 INT;
CURSOR_1 CURSOR;
BEGIN
```

```
OPEN CURSOR_1 FOR SELECT SNO FROM STUDENT;
SET C3 = 10;
C4 := C3 * 100;
SELECT NAME INTO NAME1 FROM STUDENT WHERE SNO = 10010;
FETCH ABSOLUTE 1 CURSOR_1 INTO SNO1;
CLOSE CURSOR_1;
PRINT C1; PRINT C2; PRINT C3; PRINT C4;
PRINT NAME1||' '||SNO1;
END;
```

运行结果：

```
ABC
HELLO WORLD!
10
1000
ZHANGSAN 10011
```

12.3 PL/SQL 数据类型和操作符

在 PL/SQL 块中可以定义变量和数据类型，同时 PL/SQL 有一系列操作符，这使得 PL/SQL 块对数据的处理更加灵活。

12.3.1 SQL 数据类型

PL/SQL 支持所有的 SQL 数据类型，其中包括：字符数据类型、多媒体数据类型、精确数值数据类型、近似数值数据类型、一般日期时间数据类型、时间间隔数据类型，如表 12.1 所示：

表 12.1 PL/SQL 支持的数据类型列表

类型名	类型描述
CHAR(N)	固定串长度为 N 的字符串，N<=8188
VARCHAR(N)	最大字符串长度为 N 的可变长度字符串，N<=8188
IMAGE	影像数据类型，可变长度的二进制数据，最大长度为 2G-1
TEXT	文本数据类型，可变长度的字符数据，最大长度为 2G-1
BLOB	二进制大对象，可变长度的二进制数据，最大长度为 2G-1
CLOB	字符串大对象，可变长度的字符数据，最大长度为 2G-1
BFILE	操作系统文件大对象，作为二进制文件处理，只读
BIT	单个二进制位数据
BOOL	在数据库中用 1、0 进行存储，1 表示 TRUE，0 表示 FALSE
BINARY(N)	固定长度为 N 的二进制数据，N<=8188
VARBINARY(N)	最大长度为 N 的可变长度二进制数据，N<=8188
TINYINT	精度为 3，刻度为 0 的有符号精确数字，取值范围-128…127
SMALLINT	精度为 5，刻度为 0 的有符号精确数字，取值范围-32,768…32,767
BYTE	与 TINYINT 相似，精度为 3，刻度为 0
INTEGER/INT	精度为 10，刻度为 0 的有符号精确数字，取值范围-2[31]…2[31]-1
PLS_INTEGER	与 INTEGER 相同
BIGINT	精度为 19，刻度为 0 的有符号精确数字值，取值范围-2[63]…2[63]-1

DECIMAL(P,S)	精度为 P，刻度为 S 的有符号精确数字值，1≤P≤38，S≤P
DEC(P,S)	与 DECIMAL 相同
NUMERIC(P,S)	精度为 P，刻度为 S 的有符号精确数字值，1≤P≤38，S≤P
REAL	二进制精度为 24 的有符号近似数字值，取值范围 0 或者绝对值为：10[-38]…10[38]
FLOAT	二进制精度为 53 的有符号近似数字值，取值范围 0 或者绝对值为：10[-308]…10[308]
DOUBLE	二进制精度为 53 的有符号近似数字值，取值范围 0 或者绝对值为：10[-308]…10[308]
DATE	日期数据类型，年月日字段，格式与 GREGORIAN（罗马）日历一致
TIME(P)	时间数据类型，时分秒字段，精度 P 指定了秒的精度
TIMESTAMP(P)	时间戳数据类型，年月日时分秒字段，精度 P 指定了秒的精度
INTERVAL YEAR(P)	年间隔，即两个日期之间的年数字，P 为时间间隔的首项字段精度(后面简称为：首精度)
INTERVAL MONTH(P)	月间隔，即两个日期之间的月数字，P 为时间间隔的首精度
INTERVAL DAY(P)	日间隔，即为两个日期/时间之间的日数字，P 为时间间隔的首精度
INTERVA L HOUR(P)	时间隔，即为两个日期/时间之间的时数字，P 为时间间隔的首精度
INTERVAL MINUTE(P)	分间隔，即为两个日期/时间之间的分数字，P 为时间间隔的首精度
INTERVAL SECOND(P,Q)	秒间隔，即为两个日期/时间之间的秒数字，P 为时间间隔的首精度，Q 为时间间隔秒精度
INTERVAL YEAR(P) TO MONTH	年月间隔，即两个日期之间的年月数字，P 为时间间隔的首精度
INTERVAL DAY(P) TO HOUR	日时间隔，即为两个日期/时间之间的日时数字，P 为时间间隔的首精度
INTERVAL DAY(P) TO MINUTE	日时分间隔，即为两个日期/时间之间的日时分数字，P 为时间间隔的首精度
INTERVAL DAY(P)TO SECOND(Q)	日时分秒间隔，即为两个日期/时间之间的日时分秒数字，P 为时间间隔的首精度，Q 为时间间隔秒精度
INTERVALL HOUR(P) TO MINUTE	时分间隔，即为两个日期/时间之间的时分数字，P 为时间间隔的首精度
INTERVAL HOUR(P) TO SECOND(Q)	时分秒间隔，即为两个日期/时间之间的时分秒数字，P 为时间间隔的首精度，Q 为时间间隔秒精度
INTERVAL MINUTE(P) TO SECOND(Q)	分秒间隔，即为两个日期/时间之间的分秒间隔，P 为时间间隔的首精度，Q 为时间间隔秒精度

PL/SQL 除了支持表 12.1 中所示的 SQL 数据类型外，为了方便用户使用，DM 扩展提供了%TYPE、%ROWTYPE、记录类型、数组类型、索引表类型和类类型。

12.3.2 %TYPE 和%ROWTYPE

在许多情况下，存储过程变量可以被用来处理存储在数据库表中的数据。在这种情况下，变量应该拥有与表列相同的类型。例如表 T(ID INT, NAME VARCHAR(30))中有个字段 NAME 类型为 VARCHAR(30)。对应的在存储过程中，我们可以声明一个变量： DECLARE V_NAME VARCHAR(30); 但是如果 T 中的 NAME 字段定义发生了变化，比如变为 VARCHAR(100)。那么存储过程中的变量 V_NAME 也要做相应修改为 DECLARE V_NAME VARCHAR(100); 如果您有很多的变量以及存储过程代码，这种处理可能是十分耗时和容易出错的。

为了解决上述问题，DM 提供了%TYPE 类型。%TYPE 可以附加在表列或者另外一个变量上，并返回其类型。例如：DECLARE V_NAME T.NAME % TYPE; 通过使用%TYPE，V_NAME 将拥有 T 表的 NAME 列所拥有的类型；如果表 T 的 NAME 列类型定义发生变化，V_NAME 的类型也随之自动发生变化，而不需要用户手动修改。

下面给出一个%TYPE 类型的使用例子。

```
CREATE TABLE T(ID INT, NAME VARCHAR(30));
INSERT INTO T VALUES(1, 'ZHANGSAN');
CREATE OR REPLACE PROCEDURE PROC AS
DECLARE
V1 T.NAME %TYPE;
C1 CURSOR;
BEGIN
OPEN C1 FOR SELECT NAME FROM T WHERE ID = 1;
FETCH C1 INTO V1;
PRINT V1;
CLOSE C1;
END;
```

与%TYPE 类似，%ROWTYPE 将返回一个基于表定义的运算类型，它将一个记录声明为具有相同类型的数据库行。例如：DECLARE V_TREC T % ROWTYPE; 将定义一个记录，该记录中的字段将与表 T 中的列相对应。V_TREC 变量会拥有下面这样的结构:(ID INT, NAME VARCHAR(30))，与%TYPE 类似，如果表定义改变了，那么%ROWTYPE 定义的变量也会随之改变。

下面是一个%ROWTYPE 类型的使用例子。

```
CREATE OR REPLACE PROCEDURE PROC AS
DECLARE
V1 T % ROWTYPE;
C1 CURSOR;
BEGIN
OPEN C1 FOR SELECT * FROM T WHERE ID = 1;
FETCH C1 INTO V1;
PRINT V1.ID; PRINT V1.NAME;
CLOSE C1;
END;
```

12.3.3 记录类型

记录类型类似于 C 语句的结构。记录类型提供了处理分离，但又作为一个整体单元的相关变量的一种机制。例如： DECLARE V_ID INT; V_NAME VARCHAR(30); 这两个变量在逻辑上是相互关联的，因为它们对应 T 表中的相同字段。如果为这些变量声明一个记录类型，那么它们之间的关系就十分明显了。定义记录类型的语法如下：

```
TYPE 记录类型名 IS RECORD
(记录字段名 1 数据类型, [记录字段名 2  数据类型, ……]);
```

记录类型可以单独对记录中的字段赋值，也可以将一个记录直接赋值给另外一个记录。可以使用点标记引用一个记录中的字段（记录名.字段名），如果要进行记录赋值，那么这两个记录必须是相同类型的，或者记录类型中字段的数据类型是相同的。

在下面的例子中，可以看到记录类型单独字段赋值和记录直接给另外一个记录赋值。

```
DECLARE TYPE T_REC IS RECORD( ID INT, NAME VARCHAR(30));
```

```
V_REC1 T_REC;
V_REC2 T_REC;
BEGIN
V_REC1.ID := 100;
V_REC1.NAME:= '雷锋';
V_REC2 := V_REC1;
PRINT V_REC1.ID;PRINT V_REC1.NAME;
PRINT V_REC2.ID;PRINT V_REC2.NAME;
END;
```

下面的例子则显示了记录类型不同，但记录类型中字段的数据类型相同时的赋值。

```
DECLARE
TYPE T_REC IS RECORD( ID INT, NAME VARCHAR(30));
TYPE T_REC_NEW IS RECORD( ID INT, NAME VARCHAR(30));
V_REC1 T_REC;
V_REC2 T_REC_NEW;
BEGIN
V_REC1.ID := 100; V_REC1.NAME:= '雷锋';
V_REC2 := V_REC1;
PRINT V_REC2.ID;
PRINT V_REC2.NAME;
END;
```

12.3.4 数组类型

数组数据类型包括静态数组类型和动态数组类型。静态数组的长度是预先定义好的，在整个程序中，一旦给定大小后就无法改变。而动态数组则不然，它可以随程序需要而重新指定大小。动态数组的内存空间是从堆（HEAP）上分配（即动态分配）的，通过执行代码为其分配存储空间。当程序执行到这些语句时，才为其分配，程序员自己负责释放内存。需要注意的是，DM 中数组下标的起始值为 1。静态数组的语法格式为：

```
TYPE 数组名 IS  ARRAY 数据类型 [常量表达式, 常量表达式, …];
或
TYPE 数组名 IS  ARRAY (常量表达式) OF 数据类型;
变量名 数组名 [常量表达式, 常量表达式, …];
```

下面给出一个静态数组使用的例子。

```
DECLARE
TYPE ARR IS ARRAY INT[3];        --TYPE 定义数组类型
TYPE ARR2 IS ARRAY INT[2,3];     --TYPE 定义数组类型
A ARR;                          --用自己定义的数组类型申明数组
B ARR2;                          --多维数组
BEGIN
FOR I IN 1..3 LOOP  --TYPE 定义的数组
A[I] := I * 10;
PRINT A[I];
END LOOP;
FOR I IN 1..2 LOOP
      FOR J IN 1..3 LOOP
          B[I][J] := I * 10 + J;
```

```
            PRINT B[I][J];
        END LOOP;
    END LOOP;
    END;
```

动态数组与静态数组的用法类似，区别只在于动态数组没有指定下标，需要动态分配空间。动态数组的语法格式为：

```
TYPE 数组名 IS  ARRAY 数据类型 [,…];  或  TYPE 数组名 IS TABLE OF 数据类型;
变量名 数组名[,…];
```

多维动态数组分配空间的语法为：

```
数组名 := NEW 数据类型[常量表达式,…];
```

或者可以使用如下语法对多维数组的某一维度进行空间分配：

```
数组名 := NEW 数据类型[常量表达式][];
```

下面是一个动态数组使用的例子

```
DECLARE
TYPE ARR IS ARRAY INT[];
A ARR;
BEGIN
A := NEW INT[3];        --动态分配空间
FOR I IN 1..3 LOOP
A[I] := I * 10;
PRINT A[I];
END LOOP;
PRINT ARRAYLEN(A);  --函数 ARRAYLEN 用于求取数组的长度
END;
```

12.3.5 操作符

PL/SQL 支持的操作符包括：算术操作符、关系操作符、比较操作符、逻辑操作符，如表 12.2 所示。

表 12.2 PL/SQL 支持的操作符列表

操作符类型	操作符	对应操作
算术操作符	+	加
	-	减
	/	除
	*	乘
关系操作符	<	小于操作符
	<=	小于或等于操作符
	>	大于操作符
	>=	大于或等于操作符
	=	等于操作符
	!=	不等于操作符
	<>	不等于操作符
	:=	赋值操作符
比较操作符	IS NULL	如果操作数为 NULL 返回 TRUE
	LIKE	比较字符串值
	BETWEEN	验证值是否在范围之内

	IN	验证操作数在设定的一系列值中
逻辑操作符	AND	两个条件都必须满足
	OR	只要满足两个条件中的一个
	NOT	取反

12.4 PL/SQL 流控制

PL/SQL 提供了丰富的流控制语句，用于对程序的执行流程进行控制。通过流控制语句，我们可以编写更复杂的 PL/SQL 块。流控制语句分为两类，即条件判断语句和循环语句。

12.4.1 IF 语句

IF 语句是基于布尔条件的语句序列，以实现条件分支控制结构。最简单的 IF 语句格式为：

```
IF 条件 THEN
    代码
END IF;
```

如果条件成立，则执行 THEN 后面的语句，否则跳过 THEN 到 END IF 之间的语句，执行 END IF 后面的语句。如果需要在条件不成立时执行另外的代码，格式如下：

```
IF 条件 THEN
    代码 1
ELSE
    代码 2
END IF
```

即当条件成立时执行代码 1，条件不成立时执行代码 2。

IF 可以嵌套，可以在 IF 或 IF ..ELSE 语句中使用 IF 或 IF..ELSE 语句。条件表达式中的因子可以是布尔类型的参数、变量、也可以是条件谓词，控制语句中支持的条件谓词有：比较谓词、BETWEEN、IN、LIKE、和 IS NULL。含 BETWEEN 谓词的条件表达式。

下面是一个 IF..THEN..ELSE 语句的例子

```
CREATE OR REPLACE PROCEDURE P_CONDITION(A INT) AS
BEGIN
IF A BETWEEN -5 AND 5 THEN
   PRINT 'TRUE';
ELSE
   PRINT 'FALSE';
END IF;
END;
```

更复杂的 IF 语句要先后判断多个条件，格式如下：

```
IF 条件 1 THEN
    代码 1
ELSEIF 条件 2 THEN
    代码 2
    …
ELSE
    代码 N
END IF
```

在执行上面的 IF 语句时，首先判断条件 1，当条件 1 成立时执行代码 1，否则继续判断条件 2，条件成立则执行代码 2，否则继续判断下面的条件。如果前面的条件都不成立，则执行 ELSE 后面的代码 N。

12.4.2 LOOP 语句

循环控制的基本形式是 LOOP 语句，LOOP 和 END LOOP 之间的语句将无限次的执行。LOOP 语句的基本格式如下：

```
LOOP
    循环体
END LOOP
```

LOOP 和 END LOOP 之间的语句无限次的执行显然是不行的，那么在使用 LOOP 语句时必须使用 EXIT 或 EXIT WHEN 语句结束 LOOP 循环。EXIT 语句使循环结束，一般与 IF 语句相结合使用。EXIT WHEN 指定一个条件，当条件满足时退出循环。

下面是一个 LOOP 循环语句的例子。

```
DECLARE X INT;
BEGIN
   X:=100;
   LOOP
      X:=X+100;
      IF X>1000 THEN
         PRINT X;
         EXIT;
      END IF;
   END LOOP;
END;
```

12.4.3 WHILE 语句

WHILE..LOOP 循环语句在每次循环开始以前，先计算搜索条件，若该条件为 TRUE，语句序列被执行一次，然后控制重新回到循环顶部。若搜索条件为 FALSE，则结束循环。WHILE 语句的格式如下：

```
WHILE 条件 LOOP
    循环体
END LOOP
```

在循环内部，也可以通过 EXIT WHEN、EXIT、CONTINUE WHEN、CONTINUE 来控制循环的执行流程，如下面的例子所示。

```
CREATE OR REPLACE PROCEDURE PROC(A INT) AS
DECLARE I INT:=0;
BEGIN
    WHILE 1 LOOP
        I = I + 1;
        EXIT WHEN I > A;
    END LOOP;
    PRINT I;
END;
```

12.4.4 FOR 语句

LOOP 和 WHILE 循环的循环次数都是不确定的，FOR 循环的循环次数是固定的，循环变量是一个隐式声明的变量，循环变量从起始值开始，每执行一次循环，循环变量自动加 1 或者减 1，直到与终止值相等时，循环结束。如果使用 REVERSE，则按照相反的顺序取集合中的元素，即先取最后一个元素，然后依次取前面的元素，直到第一个元素。FOR 语句的格式如下：

```
FOR 循环变量 IN[REVERSE] 起始值…终止值 LOOP
    循环体
END LOOP
```

FOR 语句中的循环变量不需要定义，也不需要显式地将集合中元素赋值给它，但只能在 FOR 循环内使用。循环变量可与当前语句块内的参数或变量同名，这时该同名的参数或变量在 FOR 语句的范围内将被屏蔽。FOR 语句也可以通过 EXIT WHEN、EXIT、CONTINUE WHEN、CONTINUE 来控制循环的执行流程，如下面例子所示。

```
DECLARE I INT:=0;
BEGIN
     FOR I IN 1..5 LOOP
          PRINT I;           --循环变量 I，语句块定义的变量 I 被屏蔽了
        EXIT WHEN I = 3;
     END LOOP;
     PRINT I;             --语句块定义的变量 I
END;
运行结果：1 2 3 0
```

12.4.5 EXIT 语句

EXIT 语句与循环语句一起使用，用于终止其所在循环语句的执行，将控制转移到该循环语句外的下一个语句继续执行。EXIT 还可以和条件表达式结合起来使用，当条件满足时才会退出。EXIT 格式如下：

```
EXIT [LABEL]
[WHEN <条件表达式>]
```

EXIT 语句必须出现在一个循环语句中，否则编译器将报错。

12.4.6 GOTO 语句

GOTO 语句无条件地跳转到一个标签 LABEL 所指的位置，LABEL 在其作用域中必须是唯一的。GOTO 语句将控制权交给带有标号的语句或语句块。格式如下：

```
GOTO <LABEL>
```

但是 GOTO 语句有一些缺点，因为在一个 PL/SQL 语句块中可能会定义很多变量，如果从某一个位置直接跳转到另一个地方，会引起不同部位的变量共享现象，导致程序不易读懂和维护，所以一般情况下不要使用 GOTO 语句来从嵌套很深的循环中直接跳到外面去。为了保证 GOTO 语句的使用不会引起程序的混乱，对 GOTO 语句的使用有下列限制：

1. GOTO 语句不能跳入一个 IF 语句、循环语句或下层语句块中；
2. GOTO 语句不能从一个异常处理器跳回当前块，但可以跳转到包含当前块的上层语句块。

12.4.7 NULL 语句

NULL 语句的语法为:

```
NULL
```

该语句不做任何事情，只是用于保证语法的正确性，或增加程序的可读性。例如:

```
CREATE OR REPLACE PROCEDURE PROC(SCORE INT) AS
BEGIN
IF SCORE = 100 THEN
   PRINT 'YOUR ARE WONDERFUL! ';
ELSE
NULL;
END IF;
END;
```

12.5 PL/SQL 访问数据库

编写 PL/SQL 块的目的主要是对数据进行访问，因此，在 PL/SQL 块中可以包含数据查询 SELECT 语句，也可以包含 INSERT、DELETE、UPDATE 等数据操纵语句。需要注意的是，在 PL/SQL 中不能直接包含 DDL 语句，如果要利用 PL/SQL 块完成诸如创建表、修改表结构等操作，需要通过 SQL 语句的动态执行。

通过 SQL 语句以及流控制语句，可以编写复杂的 PL/SQL 语句块，对数据库进行复杂的访问。由于 PL/SQL 块一般是在应用程序中调用执行，而不是以交互方式执行，所以在 PL/SQL 块中的 SQL 语句与一般的 SQL 语句有所不同。使用 PL/SQL 来控制对数据库的操作流程，功能更加丰富、灵活。

12.5.1 数据查询

在 PL/SQL 块中通过 SELECT 语句从数据库中检索数据。由于要对数据进行查询以及处理，而不仅仅是显示出来，所以 SELECT 语句需要采用下面的特殊格式先把数据放到一个临时空间中以便后面对它进行引用：

```
SELECT 列 1，列 2，列 3… INTO 变量 1，变量 2，变量 3…
FROM 表…
```

与一般的 SELECT 语句相比，PL/SQL 块中的 SELECT 语句使用了 INTO 语句，利用 INTO 子句，把查询到的数据存放在变量中，以进行相应的处理。例如下面的例子首先从学生表中检索学生的姓名、年龄、学号信息，分别存放在 O_NAME、O_AGE、O_SNO 本地变量中，然后将这些信息合起来输出。

```
CREATE OR REPLACE PROCEDURE PROC AS
    DECLARE
        O_NAME VARCHAR;
        O_AGE INT;
        O_SNO INT;
    BEGIN
SELECT NAME, AGE ,SNO INTO O_NAME, O_AGE ,O_SNO
FROM STUDENT WHERE SNO = 10010;
        PRINT O_NAME || ' ' || O_AGE || ' ' || O_SNO;
```

```
    END;
```

需要注意的是，在 SELECT 语句中，需要查询的列与 INTO 子句中的变量在数目、类型上要一致，否则会报错。

在执行 SELECT INTO 语句时，可能会发生两种例外情况，一是没有查询到满足条件的数据，二是存在多行满足条件的数据，这就是通常所说的异常。达梦预定义了一些异常，例如在第一种情况下发生的是异常 NO_DATA_FOUND，在第二种情况下发生的异常是 TOO_MANY_ROWS，对于这样的异常必须做出相应处理，否则会影响 PL/SQL 块的正确执行。

12.5.2 数据操纵

在 PL/SQL 块中也可以包含 INSERT、DELETE、UPDATE 语句，用于对数据库中的表进行增、删、改操作，在这些语句中，可以使用数字、字符串等形式的常量，也可以使用变量。下面是一个对表进行 INSERT 操作的例子。

```
CREATE OR REPLACE PROCEDURE PROC AS
DECLARE
NAME1 VARCHAR;
AGE1 INT;
SNO1 INT;
BEGIN
    INSERT INTO STUDENT VALUES(NAME1, AGE1, SNO1);
    NAME1 = 'WANGWU';
    AGE1 = 23;
    SNO1 = 10013;
    INSERT INTO STUDENT VALUES(NAME1, AGE1, SNO1);
END;
```

两条 INSERT 语句都是将三个常量作为三个列的值，第一条记录所用的变量还没有初始化，所以插入的记录的三个字段都是 NULL。

在删除表中的数据时，在 DELETE 语句的 WHERE 子句中也可以使用变量，例如下面的例子从 STUDENT 表中删除上面插入的两条记录。

```
CREATE OR REPLACE PROCEDURE PROC AS
DECLARE
NAME1 VARCHAR;
BEGIN
    DELETE FROM STUDENT WHERE NAME IS NULL;
    NAME1 = 'WANGWU';
    DELETE FROM STUDENT WHERE NAME = NAME1;
END;
```

在修改表中数据时，在 UPDATE 语句的 SET 子句和 WHERE 子句中可以使用变量，如下例所示。

```
CREATE OR REPLACE PROCEDURE PROC (NAME1 VARCHAR(30), AGE1 INT) AS
BEGIN
    UPDATE STUDENT SET AGE = AGE1 WHERE NAME = NAME1;
END;
```

调用存储过程 CALL PROC('ZHANGSAN',24)，即可将学生名字为 ZHANGSAN 的年龄改为 24 岁。

12.5.3 数据定义

在 PL/SQL 中也可以动态执行 DDL 语句。

下面是一个在 PL/SQL 中执行 CREATE TABLE 语句的例子。

```
CREATE OR REPLACE PROCEDURE PROC AS
SQL_STMT VARCHAR(100);
BEGIN
   SQL_STMT := 'CREATE TABLE TEST(I INT)';
   EXECUTE IMMEDIATE SQL_STMT;
END;
```

调用存储过程 CALL PROC，即可创建表 TEST。

12.6 PL/SQL 与数据库交互

没有名字的 PL/SQL 程序块被称为匿名块，它仅存放在缓冲区中，只能在当前 DISQL 环境中执行。如果希望 PL/SQL 块能随时被调用执行，并且能被数据库用户共享，就需要创建存储程序。存储程序是有名字的 PL/SQL 块，它在创建时经过了编译与优化，被存放在数据库中，任何用户只要有适合的权限，就可以根据它的名字进行多次调用。存储程序在调用时无需要再进行编译，因此能以很快的速度执行，但如果需要对存储程序进行修改，不能直接修改它的源代码，只能执行 CREATE 命令重新创建。

存储程序可以在 SQL 语句、应用程序以及其它 PL/SQL 块中被调用，在第一次被调用时，存储程序的代码被装载到系统全局区的共享池中，再次调用时即可直接从共享池中取出代码执行。存储程序的形式包括：存储过程，存储函数、触发器、包等，它为用户提供了一种高效率的编程手段，成为现代数据库系统的重要特征。

12.6.1 存储过程

用户在自己的模式中创建存储过程，需要具有 CREATE PROCEDURE 数据库权限，如果要在其它用户的模式中创建存储过程，则需要具有 CREATE ANY PROCEDURE 数据库权限。创建存储过程的语法为：

```
CREATE [OR REPLACE] PROCEDURE
过程名 [WITH  ENCRYPTION] [(参数 1，参数 2，…)]
AS
    声明部分
BEGIN
    可执行部分
EXCEPTION
    异常处理部分
END;
```

其中 OR REPLACE 选项的作用是当同名的存储过程存在时，首先将其删除，再创建新的存储过程，前提条件是当前用户具有删除原存储过程的权限，如果没有删除权限，则创建失败。存储过程可以带有参数，这样在调用存储过程时就需指定相应的实际参数，如果没有参数，过程名后面的圆括号和参数列表就可以省略了。参数定义的格式如下：

```
参数名 [参数传递模式] 数据类型 [:= 默认值]
参数名 [参数传递模式] 数据类型 [DEFAULT 默认值]
```

参数传递模式包括：输入参数（IN）、输出参数（OUT）、输入输出参数（IN OUT）三种形式，其中 IN 是默认的参数传递模式。调用存储过程时，如果没有提供实际参数，则使用参数默认值。

WITH ENCRYPTION 为可选项，如果指定 WITH ENCRYPTION 选项，则对 BEGIN 到 END 之间的语句块进行加密，防止非法用户查看其具体内容。可直接创建加密后的存储过程或函数，语法为：

```
CREATE [OR REPLACE] PROCEDURE|FUNCTION
过程名|函数名 WRAPPED 密文;
```

加密后的存储过程或函数的定义可在 SYS.SYSTEXTS 系统表中查询。

可执行部分是存储过程的核心部分，由 SQL 语句和流控制语句构成。异常处理部分用于处理存储过程在执行过程中可能出现的错误。

下面的例子创建了一个存储过程，功能是查看 STUDENT 表中学生总数。

```
CREATE OR REPLACE PROCEDURE STUDENT_COUNT AS
TOTAL_NUM INT;
BEGIN
    SELECT COUNT(*) INTO TOTAL_NUM FROM STUDENT;
    PRINT TOTAL_NUM;
END;
```

调用存储过程 CALL STUDENT_COUNT，即可获取学生总数。

所有用户都可调用自己创建的存储过程，如果要调用其他用户的存储过程，则需要对该存储过程具有 EXECUTE 权限，即存储过程的所有者将 EXECUTE 权限授予该用户。授予 EXECUTE 权限的语法为：

```
GRANT EXECUTE ON 过程名 TO 用户;
```

删除存储过程的语法为：

```
DROP PROCEDURE 过程名;
```

12.6.2 存储函数

存储函数与存储过程在结构和功能上十分相似，但还是有所差异的。它们的区别在于：

1. 存储过程没有返回值，调用者只能通过访问 OUT 或 IN OUT 参数来获得执行结果，而存储函数有返回值，它把执行结果直接返回给调用者；
2. 存储过程中可以没有返回语句，而存储函数必须通过返回语句结束；
3. 不能在存储过程的返回语句中带表达式，而存储函数必须带表达式；
4. 存储过程不能出现在一个表达式中，而存储函数只能出现在表达式中。

创建存储函数的语法格式为：

```
CREATE OR REPLACE FUNCTION
函数名 [WITH  ENCRYPTION] (参数 1,参数 2,…) RETURN 返回类型
    AS
    声明部分
BEGIN
    可执行部分
    RETURN 表达式;
EXCEPTION
    异常处理部分
END;
```

如果用户要在自己的模式中创建存储函数，只需具有 CREATE FUNCTION 的数据库权限，如果要在其它用户的模式中创建存储函数，则需要具有 CREATE ANY FUNCTION 的数

据库权限。

下面的例子创建一个存储函数，实现斐波那契数列算法。

```
CREATE OR REPLACE FUNCTION FIBONACCI(A INT) RETURN INT AS
    DECLARE S INT; LV INT; RV INT;
BEGIN
   IF A <= 0 THEN
      PRINT '请输入正数';
   ELSIF A = 1 OR A = 2 THEN
      S := 1;
   ELSE
      RV := FIBONACCI (A - 1);
      LV := FIBONACCI (A - 2);
      S := LV + RV;
      PRINT LV || '+' || RV || '=' || S;
   END IF;
   RETURN S;
END;
```

调用函数 SELECT FIBONACCI(4)，即可获取 4 的斐波那契数列。

每个用户都可以直接调用自己创建的存储函数，如果要调用其它用户的存储函数，则需要具有对相应存储函数的 EXECUTE 权限。为此，存储函数的所有者要将 EXECUTE 权限授予适当的用户，授予 EXECUTE 权限的语句格式为：

```
GRANT EXECUTE ON 函数名 TO 用户
```

删除存储函数的语句格式为：

```
DROP FUNCTION 函数名;
```

存储过程和存储函数统称为存储模块，存储模块的调用命令有三种：

1. SELECT 语句调用；
2. CALL 语句调用；
3. 直接在 DISQL 中输入存储过程或函数名即可调用。

上述三种调用方式的不同之处在于：

1. SELECT 调用存储模块必须是有返回值的，也就是说 SELECT 命令只能调用存储函数；
2. 如果通过 CALL 来调用，它不要求调用的是存储过程还是存储函数，如果存储模块有返回值，则 CALL 命令会忽略它的返回值，执行结果是成功或者失败；
3. 如果不通过任何语句直接调用存储模块，默认方式为 CALL。

12.6.3 游标

由于 DM 支持的 SQL 语言是面向集合的语言，一条 SQL 语句可以产生或处理多条记录，而 PL/SQL 语言是面向记录的，一组主变量一次只能存放一条记录，所以仅使用主变量并不能满足 SQL 语句向应用程序输出数据的要求。为解决这一问题，SQL 语言引用了游标来协调这两种不同的工作方式，从而为应用程序逐个处理查询结果提供了强有力的手段。

使用游标必须先定义，定义游标实际上是定义了一个游标工作区，并给该工作区分配了一个指定名字的指针（即游标）。游标工作区用以存放满足查询条件行的集合，因此它是一个说明性语句，是不可执行的。在打开游标时，就可从指定的基表中取出所有满足查询条件的行送入游标工作区并根据需要分组排序，同时将游标置于第一行的前面以备读出该工作区中的数据。当对行集合操作结束后，应关闭游标，释放与游标有关的资源。

DM 支持的 SQL 语言提供了有关游标的语句：定义游标语句、打开游标语句、拨动游标

语句和关闭游标语句。

定义游标的语法格式为：

```
CURSOR <游标名> [FAST | NO FAST] [<cursor 选项>]
或者
<游标名> CURSOR [FAST | NO FAST] <cursor 选项>

<cursor 选项> := <cursor 选项 1>|<cursor 选项 2>|<cursor 选项 3>|<cursor 选项 4>
<cursor 选项 1>:= <IS|FOR>  {<查询表达式>|<连接表>}
<cursor 选项 2>:= <IS|FOR> TABLE <表名>
<cursor 选项 3>:= (<参数声明> {,<参数声明>})IS <查询表达式>
<cursor 选项 4>:= [(<参数声明> {,<参数声明>})] RETURN <plsql 数据类型> IS <查询表达式>
<参数声明> ::= <参数名> [IN] <参数类型> [ DEFAULT|:= <缺省值> ]
<plsql 数据类型> ::= <普通数据类型>
                    | <变量名> %TYPE
                    | <表名> %ROWTYPE
                    | CURSOR
                    | REF <游标名>
<查询表达式>::=请参考《DM7_SQL 语言使用手册》
<连接表>::=请参考《DM7_SQL 语言使用手册》
```

为了处理游标中的数据，首先要打开游标。打开游标的语法格式为：

```
OPEN 游标名
OPEN 游标名 FOR SELECT 语句;
```

游标打开后，就可以取出游标中的数据，并对其进行处理。拨动游标的语法格式为：

```
FETCH [<fetch 选项> [FROM]] <游标名> [ [BULK COLLECT]  INTO <主变量名 >{,<主变量名>} ] [LIMIT rows]
 <fetch 选项>::= NEXT | PRIOR | FIRST | LAST | ABSOLUTE n | RELATIVE n
```

FETCH 命令可以将一行数据中各列的值依次赋给指定的变量，需要注意的是，变量的类型、数目要与游标中的一行的各列相对应。游标在使用完后应及时关闭，以释放它所占用的内存空间，关闭游标的语法格式为：

```
CLOSE   游标名
```

当游标关闭后，不能再从游标中获取数据。如果需要，可以再次打开游标。

下面给出一个典型的使用游标的例子。

```
CREATE OR REPLACE PROCEDURE PROC AS
STR VARCHAR;
CURSOR CSR FOR SELECT NAME FROM STUDENT;  --声明游标并绑定 SQL 语句
CURSOR CSR1;        --先声明，打开游标的时候再绑定 SQL 语句
BEGIN
    OPEN CSR;
    OPEN CSR1 FOR SELECT NAME FROM SYSOBJECTS
WHERE NAME='STUDENT';
    LOOP
    FETCH CSR INTO STR;     --从游标 CSR 中取数据;
    EXIT WHEN CSR%NOTFOUND;
    PRINT STR;
    END LOOP;
    FETCH CSR1 INTO STR;        --从游标 CSR1 中取数据;
    PRINT STR;
    CLOSE CSR;
    CLOSE CSR1;
END;
```

运行结果：

```
ZHANGSAN
LISI
STUDENT
```

在 DM 中，游标的属性主要包括以下四种，如表 12.3 所示。

表 12.3 游标的属性

属性	描述
CURSOR%ISOPEN	判断当前游标是否已经打开，如果打开，该属性值为 TRUE，否则为 FALSE
CURSOR%ROWCOUNT	表示到目前为止，用 FETCH 语句已经取到的行数
CURSOR%FOUND	表示最后一次 FETCH 操作是否从游标中取到一行数据，如果已经取到，其值为 TRUE，如果没有取到，其值为 FALSE
CURSOR%NOTFOUND	和 CURSOR%FOUND 正好相反

以上游标的属性被称为显式游标属性。在这四个游标属性中，%ISOPEN 属性用于测试游标的状态。其它三个属性用来测试 FETCH 命令的执行结果，%FOUND 和%NOTFOUND 属性用来测试最近的一次 FETCH 是否取到数据，%ROWCOUNT 属性表示自游标打开以来，到目前为止，用 FETCH 命令获取的行数。

在大多数情况下，游标的应用步骤是：打开游标；开始循环；从游标中取出一行数据并进行处理；关闭循环；关闭游标，这一类应用称为游标用于循环。但还有一种循环与这种类型不同，这就是游标 FOR 循环。用于 FOR 循环的游标与一般游标的区别在于不需要显式的打开、关闭、取数据，测试数据的存在、定义存放数据的变量等。

下面的例子进行游标 FOR 循环，将学生的年龄加 1。

```
DECLARE
CURSOR STU_CURSOR IS
SELECT * FROM STUDENT FOR UPDATE;
BEGIN
FOR STU_ROW IN STU_CURSOR LOOP
UPDATE STUDENT SET AGE = AGE + 1 WHERE CURRENT OF STU_CURSOR;
END LOOP;
COMMIT;
END;
```

从上面的例子中我们可以看到游标 FOR 循环确实很好的简化了游标的开发，我们不再需要 OPEN、FETCH 和 CLOSE 语句，也不需要用%FOUND 属性检测是否到最后一条记录，这一切 DM 服务器隐式的帮我们完成了。

12.6.4 引用游标

定义引用游标的语法格式：

```
<游标变量名> CURSOR;
或者
<游标变量名> CURSOR := SOURCE_CURSOR;
```

引用游标有以下几个特点：

1. 不只局限于一个查询，可以为一个查询声明或者打开一个游标，然后对其结果集进行处理，之后又可以将这个引用游标为其它的查询打开；
2. 可以对引用游标进行赋值；
3. 可以像用一个变量一样在一个表达式中用这个引用游标；
4. 可以作为一个子程序的参数，也就是它可以像 C 语言中的变量一样充当参数；
5. 可以在 PL/SQL 的不同子程序中传递结果集。

同一个游标变量再次打开游标的时候可以关联另一个查询语句，而原来的游标句柄会被

自动关闭。

如果定义引用游标的时候没有赋值。可以在后面需要的地方赋值。引用游标赋值操作，语法如下：

```
<游标变量名> := SOURCE_CURSOR;
```

给一个引用游标变量赋值时，可以继承所有 SOURCE_CURSOR 的属性。如果 SOURCE_CURSOR 已经打开，则 DEST_CURSOR 也已经打开，也就是说 DEST_CURSOR 只是个变量，它相当于一个指针，指向的位置和 SOURCE_CURSOR 是完全一样的。

下面是一个使用引用游标的例子。

```
CREATE OR REPLACE PROCEDURE PROC AS
STR VARCHAR;
CURSOR C1 IS SELECT NAME FROM STUDENT WHERE SNO = 10010;
CURSOR C2 IS SELECT NAME FROM STUDENT WHERE SNO = 10011;
CURSOR CSR;
BEGIN
CSR := C1;
OPEN C1;      --赋值后再打开
    IF CSR%ISOPEN THEN
        PRINT 'CURSOR IS ALREADY OPENED';
        FETCH CSR INTO STR;
        PRINT STR;
ELSE
        PRINT 'CURSOR IS NOT OPENED';
    END IF;
    OPEN C2;   --打开后再赋值
    CSR := C2;
    IF CSR%ISOPEN THEN
        PRINT 'CURSOR IS ALREADY OPENED';
        FETCH CSR INTO STR;
        PRINT STR;
    ELSE
        PRINT 'CURSOR IS NOT OPENED';
    END IF;
    CLOSE C1;
    CLOSE C2;
END;
```

运行结果：

```
CURSOR IS ALREADY OPENED
ZHANGSAN
CURSOR IS ALREADY OPENED
LISI
```

12.6.5 动态 SQL

SQL 的动态执行是通过语句 EXECUTE IMMEDIATE 实现的，它的作用是执行一个指定的 SQL 语句，这个 SQL 语句可以带参数，也可以不带参数。如果带参数，则 EXECUTE IMMEDIATE 后面需要加上 USING 来指定 SQL 语句中参数的值；如果不带参数，EXECUTE IMMEDIATE 后面直接跟上需要执行的 SQL 语句即可。动态 SQL 的语法格式如下：

```
EXECUTE IMMEDIATE <SQL 动态语句文本> [INTO VARIABLES]
[USING [参数 1][参数 2]…];
```

1. 该语句只能在嵌入式方式中使用；
2. 该语句首先分析动态语句文本，检查是否有错误，如果有错误则不执行它，并在 SQLCODE 中返回错误码；如果没发现错误则执行它；
3. 一般来说，应该使用一个字符串变量来表示一个动态 SQL 语句的文本，下列三个语句不能作动态 SQL 语句：CLOSE、FETCH、OPEN。如下面几个例子所示：

例 不支持 OPEN。调用 PROC 过程时，报错：不支持的语句类型。

```
CREATE OR REPLACE PROCEDURE PROC AS
CURSOR C1;
SQL_STMT1 VARCHAR := 'OPEN C1 FOR SELECT NAME FROM STUDENT;';
BEGIN
   EXECUTE IMMEDIATE SQL_STMT1;
END;
```

例 不支持 CLOSE。调用 PROC 过程时，报错：不支持的语句类型。

```
CREATE OR REPLACE PROCEDURE PROC AS
CURSOR C1;
SQL_STMT1 VARCHAR := 'CLOSE C1;';
BEGIN
OPEN C1 FOR SELECT NAME FROM STUDENT;
EXECUTE IMMEDIATE SQL_STMT1;
END;
```

例 不支持 FETCH。调用 PROC 过程时，报错：不支持的语句类型。

```
CREATE OR REPLACE PROCEDURE PROC AS
CURSOR C1;
STR VARCHAR;
SQL_STMT1 VARCHAR := 'FETCH ABSOLUTE 1 C1 INTO STR;';
BEGIN
OPEN C1 FOR SELECT NAME FROM STUDENT;
EXECUTE IMMEDIATE SQL_STMT1;
PRINT STR;
CLOSE C1;
END;
```

动态 SQL 语句不支持嵌入式异常处理 WHENEVER，即不支持下列类似语句：

```
EXEC SQL WHENEVER SQLERROR GOTO sql_error; ……
sql_error:
EXEC SQL WHENEVER SQLERROR CONTINUE;
```

动态 SQL 语句的文本也可以是一个 PL/SQL 语句块，如下例所示：

```
CREATE OR REPLACE PROCEDURE PROC AS
CURSOR C1;
SQL_STMT1 VARCHAR := 'DECLARE STR VARCHAR; BEGIN SELECT NAME INTO STR FROM
STUDENT WHERE SNO=10010; PRINT STR;END;';
BEGIN
EXECUTE IMMEDIATE SQL_STMT1;
END;
```

如果动态 SQL 语句的文本中有多条 SQL 语句，那么只执行第一条语句，如下例所示：

```
CREATE OR REPLACE PROCEDURE PROC AS
SQL_STMT VARCHAR := 'SELECT NAME FROM STUDENT WHERE SNO=10010;
```

```
SELECT NAME FROM STUDENT WHERE SNO=10011;';
BEGIN
     EXECUTE IMMEDIATE SQL_STMT;
END;
```

运行结果：ZHANGSAN

对于仅执行一次的动态语句，用立即执行语句较合适。如果动态 SQL 语句是查询语句并且结果集只有一条记录，则可以通过 INTO 语句把结果存储赋值给变量，如下例所示：

```
CREATE OR REPLACE PROCEDURE PROC AS
NAME VARCHAR := 'STUDENT';
SQL_STMT VARCHAR := 'SELECT NAME FROM SYSOBJECTS WHERE NAME = ?';
STR VARCHAR;
BEGIN
        EXECUTE IMMEDIATE SQL_STMT INTO STR USING NAME;
       PRINT STR;
END;
```

在动态执行 SQL 语句的时候也可以对指定的 SQL 语句设置参数。指定参数有两种方式，如果某个值设置为参数，可以用“？”来表示，也可以用“:VARIABLE”来表示，如下例所示：

```
CREATE OR REPLACE PROCEDURE PROC AS
    STR VARCHAR := 'HELLO';
    START_POS INT:= 1;
    END_POS INT := 3;
    STR_SQL VARCHAR := 'SELECT SUBSTR(:X, :Y, :Z)';
    BEGIN
        EXECUTE IMMEDIATE STR_SQL USING STR, START_POS, END_POS;
    END;
```

运行结果：HEL

用“？”表示参数时，在传入时可以是任意的值，但参数个数一定要与“？”的个数相同，同时数据类型一定要匹配（可以互相转换也可以），不然会报数据类型不匹配的错误。

12.7 自治事务

DM 支持通过将一个 PL/SQL 语句块定义成自治事务，将该块中的 DML 语句和调用程序的事务环境隔离开。如此一来，该语句块就成为一个由其他事务启动的独立的事务，前一个事务被称为主事务。

一个定义了自治事务的语句块称为自治例程。

在自治事务块中，主事务是挂起的；等待自治事务完成后，会话自动切换回主事务。

12.7.1 定义自治事务

自治事务仅能在 PL/SQL 块中定义。定义自治事务，需要在 PL/SQL 块的声明单元中添加如下的语句：

```
PRAGMA  AUTONOMOUS_TRANSACTION;
```

作为自治事务的 PL/SQL 块可以是下面中的一种：

1. 最顶层的（不是嵌套的）匿名 PL/SQL 块
2. 函数和过程，或者在一个包里定义或者是一个独立的程序
3. 对象类型的方法

4. 数据库触发器
5. 嵌套子过程

自治事务的定义语句可以放在 PL/SQL 块声明单元的任何地方，但推荐放在数据结构声明之前。

12.7.2 自治事务完整性与死锁检测

自治事务的语句块执行完成时，如果事务仍处于活动状态，则系统需要报错：“检测到活动的自治事务处理”，并将未提交事务回滚。所以需要在语句块结束处显式添加 COMMIT 或 ROLLBACK 语句。

另一方面，COMMIT 和 ROLLBACK 语句只是结束了活动的自治事务，但不会终止自治例程。实际上，可以在一个自治块中使用多个 COMMIT、ROLLBACK 语句。

自治事务相对主事务是完全独立的。如果自治事务需要的锁资源已经被主事务拥有，则进行死锁报错。

12.7.3 自治事务嵌套

自治块中可以调用其它被定义自治事务的过程，从而产生嵌套的自治事务。INI 参数 TRANSACTIONS 指定一个会话中可以并发的自治事务数量，其默认值为 75。如果实际生成自治事务数超出该参数限定值时，服务器将报错：“并发事务数超出上限”。该参数理论值上下限分别为 1000 和 1。

12.8 异常处理

用户编写的 PL/SQL 块在执行过程中不可避免地会发生一些错误。这里涉及的错误并不是由于程序的语法引起的，而是因为处理的数据超出了处理的范围而引发的错误。如果给这样的错误起一个名字，这就是异常。当 PL/SQL 块在执行过程中检测到一个错误时，就会抛出相应的异常。在块中应当处理这样的异常，否则会引起应用程序的运行停止。

12.8.1 异常处理程序

异常一般是在 PL/SQL 程序中执行错误时由服务器抛出，也可以在 PL/SQL 块中由程序员在一定的条件下显式抛出，无论是哪种形式的异常，都可以在 PL/SQL 块的异常处理部分编写一段程序进行处理，如果不做任何处理，异常将被传递到调用者，由调用者统一处理。

如果要在 PL/SQL 块中对异常进行处理，就需要在异常处理部分编写处理程序。异常处理程序的格式如下：

```
EXCEPTION
    WHEN 异常 1 [OR 异常 2 … ] THEN
        异常处理程序 1;
    WHEN OTHERS THEN
        异常处理程序 2;
END;
```

异常处理程序以关键字 EXCEPTION 开始，以关键字 END 结束。在这部分可以对多个异常分别进行不同的处理，也可以进行相同的处理。如果没有列出所有异常，可以用关键字

OTHERS 代替其它的异常，在异常处理程序的最后加上一条 WHEN OTHERS 子句，用来处理前面没有列出的所有异常。

如果 PL/SQL 块执行出错，或者遇到显式抛出异常的语句，则程序立即停止执行，转去执行异常处理程序。异常被处理结束后，整个 PL/SQL 块的执行便告结束。所以一旦发生异常，则在 PL/SQL 块的可执行部分中，从发生异常的地方开始，以后的代码将不再执行。

在 PL/SQL 块中有三种类型的异常，即预定义的异常、非预定义的异常和用户自定义的异常。下面分别介绍这几种异常的使用方法。

12.8.2 预定义的异常

为方便用户编程，DM 提供了一些预定义的异常，这些异常与最常见的 DM 错误相对应。如表 12.4 所示：

表 12.4 数据库预定义的异常

异常名称	SQLCODE	说明
TOO_MANY_ROWS	-3034	SELECT INTO 中包含多行数据
NO_DATA_FOUND	-7065	数据未找到
DUP_VAL_ON_INDEX	-6602	违反唯一性约束
INVALID_CURSOR	-4535	无效的游标操作
ZERO_DIVIDE	-6103	除 0 错误

此外，还有一个特殊的异常名 OTHERS，它处理所有没有明确列出的异常。OTHERS 对应的异常处理语句必须放在其他异常处理语句之后。

下面给出一个异常处理的例子。

```
DECLARE
    OBJ_NAME VARCHAR(50);
BEGIN
    SELECT NAME INTO OBJ_NAME FROM STUDENT WHERE SNO = 10010;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        PRINT '没有找到满足条件的数据';
    WHEN TOO_MANY_ROWS THEN
        PRINT '找到的记录数大于 1 条，不满足 SELECT INTO 的条件';
    WHEN OTHERS THEN
        PRINT 'OTHERS EXCEPTION';
END;
```

PL/SQL 的 WHEN 子句中也可以指定多个异常的名字，用 OR 分隔，例如：

```
WHEN NO_DATA_FOUND OR TOO_MANY_ROWS THEN
      PRINT 'SELECT 结果集不符合要求';
```

在向表的主键列上写入一个重复的值时将引发异常 DUP_VAL_ON_INDEX，如下例所示：

```
CREATE OR REPLACE PROCEDURE ADD_STUDENT
(NAME VARCHAR, AGE INT, SNO INT)  AS
    BEGIN
        INSERT INTO STUDENT VALUES(NAME, AGE, SNO);
    EXCEPTION
        WHEN DUP_VAL_ON_INDEX THEN
            PRINT '违反主键的唯一性约束';
            PRINT 'ERROR CODE: ' || "%DM_EXCEPTION_OBJ"."CODE";
            PRINT 'DESCRIBE: ' ||  "%DM_EXCEPTION_OBJ"."DESCR";
```

```
        WHEN OTHERS THEN
            PRINT 'OTHERS EXCEPTION';
    END;
```

从这个例子可以看出，PL/SQL 在异常处理的异常类型中提供了两个成员，“CODE”和“DESCRIBE”，分别表示引起异常时发生错误的错误码及错误码的描述信息。在日常处理中，我们可以创建一个表用来存储在 PL/SQL 的异常处理部分出现的错误情况，或者生成日志信息，以备后续对错误的统计、分析以及处理。

引发异常的一个重要原因是处理数据时发生错误。编写 PL/SQL 的主要目的是处理数据，而 PL/SQL 块在逻辑与数据是分开的，程序员根本无法预料数据的变化，所以在编写程序时需要考虑各种可能出现异常的情况，并对这些异常进行相应的处理。

12.8.3 用户自定义的异常

除了 DM 定义的两种异常外，在 PL/SQL 中还可以用户自定义异常。程序员可以把一些特定的状态定义为异常，在一定的条件下抛出，然后利用 PL/SQL 的异常机制进行处理。用户自定义异常的语法格式为：

```
异常名称 EXCEPTION [FOR 自定义的错误码]
```

或

```
<异常变量名> EXCEPTION;
PRAGMA EXCEPTION_INIT(<异常变量名>, <错误号>);
```

抛出异常的语法格式为：

```
RAISE   异常名
```

异常一般在一定的条件下抛出，因此 RAISE 语句通常跟在某个条件判断的后面，这样就把这个异常与这个条件关联起来了，抛出异常的原因可能是数据出错，也可能是满足了某个自定义的条件，处理自定义异常的方法与处理前两种异常的方法相同。

下面是一个用户自定义异常的例子。

```
CREATE OR REPLACE PROCEDURE PROC AS
DECLARE
    E1 EXCEPTION FOR -20001;
BEGIN
    RAISE E1;
EXCEPTION
    WHEN E1 THEN
        PRINT 'NORMAL EXCEPTION' || ' ' || "%DM_EXCEPTION_OBJ"."CODE";
    WHEN OTHERS THEN
        PRINT 'OTHER EXCEPTION' || ' ' || "%DM_EXCEPTION_OBJ"."CODE";
END;
```

12.9 PL/SQL 调试工具

DM 为用户提供了 PL/SQL 调试工具 DMDBG。DMDBG 可调试在 DISQL 中直接执行的非 DDL 语句或语句块，以便程序开发人员定位 PL/SQL 中存在的错误。用户要使用 DMDBG 调试工具，首先要调用系统过程 SP_INIT_DBG_SYS(1)创建调试所需要的包。

12.9.1 命令行参数

DMDBG 的整个运行过程中可以处于初始状态(S)、待执行(W)、执行(R)、调试(D)、执行结束(O)等不同的状态。

初始状态(S)：工具启动完成后，尚未设置调试语句；

待执行状态(W)：设置调试语句后，等待用户执行；

执行状态(R)：开始执行后，未中断而运行的过程；

调试状态(D)：执行到断点或强制中断后进入交互模式；

执行结束(O)：执行完当前设置的调试语句，并返回结果。

DMDBG 在不同的状态下可以执行不同的操作，具体如表 12.5 所示。备注表示不同的命令分别在哪几种状态下可以使用。

表 12.5 DMDBG 调试命令

命令行参数	含义	备注
LOGIN	登录	S/W/D/O
SQL	设置调试语句	S/W/O
B	设置断点	W/D/O
INFO B	显示断点信息	W/D/O
D	取消断点	W/D/O
R	执行语句	W/O
CTRL+C	中断执行	R
L	显示调试脚本	D
C	继续执行	D
N	单步执行	D
S	执行进入	D
F或FINISH	执行跳出	D
P	打印变量	D
BT	显示堆栈	D
UP	上移栈帧	D
DOWN	下移栈帧	D
KILL	结束当前执行	W/D/O
QUIT	退出调试工具	S/W/D/O

12.9.2 功能描述

1. 登录

提供交互与命令行 2 种登录方式。交互方式与 DISQL 相同，使用者根据提示依次输入服务器、用户名、口令及端口号等登录信息。

命令行登录参数格式如下：

```
DMDBG <用户名>/<口令>[@<服务器名|IP>[:<端口号>]]
```

默认用户名和口令为 SYSDBA/SYSDBA，服务器 LOCALHOST，端口号为 5236，则登录

命令为：

```
DMDBG SYSDBA/SYSDBA@LOCALHOST:5236
```

2. 设置语句

命令 SQL 供用户设置需要执行的 SQL 语句，其命令格式如下：

```
<SQL> ['/']
```

SQL 语句是非 DDL 语句或语句块。如果 SQL 是单条语句，则以分号“;”结尾；如果是语句块，则语句块后必须以单独一行的斜杠符“/”结束。在调试中行数从 SQL 语句实际起始行开始计数。

在实际执行前，用户可以多次调用 SQL 命令设置待调试的语句，而 DMDBG 只是记录最后一次的设置语句。

如果需要调试自定义函数，需要和过程调用方式一样，使用 call 调用。

具体实例如下：

```
第一步，在 DISQL 中创建表和存储过程。
CREATE TABLE STUDENT (ID INT, NAME VARCHAR(20));        --创建游标类型对应的表
INSERT INTO STUDENT VALUES(10010,'ZHANGSAN');
INSERT INTO STUDENT VALUES(10011,'LISI');

CREATE OR REPLACE PROCEDURE P0 AS                    --创建需要调试的存储过程
BEGIN
PRINT 'HELLO';
END;
/

CREATE CLASS MYCYCLE                                 --定义类对象
AS
ID INT;
C1 INT;
FUNCTION MYCYCLE(ID INT , C1 INT) RETURN MYCYCLE;
END;
/

CREATE OR REPLACE CLASS BODY MYCYCLE                --定义类体
AS
FUNCTION MYCYCLE(ID INT, C1 INT) RETURN MYCYCLE
AS
BEGIN
 THIS.ID = ID;
 THIS.C1 = C1;
 RETURN THIS;
END;
END;
/

CREATE OR REPLACE PROCEDURE P AS                     --创建需要调试的存储过程
TYPE T_REC IS RECORD(ID INT, NAME VARCHAR(20));             --定义记录类型
TYPE ARR is ARRAY INT[4];                               --定义数组类型
R1 T_REC;                                               --声明记录类型
C1 CURSOR;                                             --声明游标类型
```

```
B ARR;                                                    --声明数组类型
O MYCYCLE;                                                --声明对象类型
I1 INT;                                                   --声明整型
V1 VARCHAR(20);                                           --声明字符串类型
BEGIN
R1.ID = 10012;
R1.NAME = 'WANGWU';
OPEN C1 FOR SELECT * FROM STUDENT;
LOOP
FETCH C1 INTO I1,V1;
EXIT WHEN C1%NOTFOUND;
END LOOP;
FOR I IN 1..4 LOOP
B[I] := I * 10;
PRINT B[I];
END LOOP;
O = NEW MYCYCLE(1, 2);
CALL P0;
END;
/
```

第二步，在 DMDBG 中调试存储过程 P，设置调试语句如下：

```
DBG>SQL CALL P;
```

3. 设置断点

设置断点命令为 B，可以为指定的脚本设置执行中断位置。调试执行到断点设置位置后，将自动中断当前执行。设置断点命令有 2 种不同格式。

```
B <方法名>
B [<方法名>:]<行号>
```

如果省略方法名，则为当前方法或语句块设置行号断点。包中的方法可以设置断点，但不能指定行号；如果断点设置在包上，则可以指定行号，但可以设置断点部分只能是该包体的初始化代码。

行号从 1 开始计数。对于方法来说，其开始的定义声明部分语句也包含在行号计数中。对于断点的行号设置必须遵守以下原则：

定义声明部分不能设置断点；

如果一条语句跨多行，则断点应设置在语句最后一行；

如果多条语句在同一行，则执行中断在其中第一条语句执行前；

如果设置断点行号不能中断，则自动将其下移到第一个可以中断的行；

可以在调试过程中动态添加断点。如果在同一位置重复设置断点，则重复断点被忽略。

例如：

```
DBG>B 3                          --没有方法名，则在最顶层设置断点
No line 3 in method "@dbg_main".--由于顶层只有一行，无法设置断点，报错返回
DBG>B P:3                          --不能设置断点，则下移到第一条可以中断的行
Breakpoint 1 at SYSDBA.P, line 11@{R1.ID = 10012;}
DBG>B P:12                       --直接在指定行设置断点
Breakpoint 2 at SYSDBA.P, line 12@{R1.NAME = 'WANGWU';}
```

4. 显示断点

显示断点命令为：

```
INFO B
```

该命令显示所有已设置的断点。显示的每个断点包含以下信息：

1) 序号。这是一个自增值，从 1 开始。设置断点时自动为其分配。
2) 方法名。如果为语句块，则方法名为空。
3) 行号。

例如：

```
DBG> INFO B
Num    Schema  Package  Method        What
1      SYSDBA  NULL    P      line 11@{R1.ID = 10012;}
2      SYSDBA  NULL    P      line 12@{R1.NAME = 'WANGWU';}
```

5. 取消断点

取消断点的命令格式为：

```
D <断点序号>
```

断点序号可以通过 INFO B 命令获取，删除后断点的编号不能重用。取消断点命令可以在工具运行的任何时间调用。

例如：

```
DBG>D 1              --删除断点，成功则等待再次输入
DBG>D 3              --删除不存在的断点，则报错
No breakpoint number 3.
DBG>INFO B           --显示删除后的断点信息
DBG> INFO B
Num    Schema  Package Method  What
2      SYSDBA  NULL    P      line 12@{R1.NAME = 'WANGWU';}
```

6. 执行语句

执行语句命令为：

```
R
```

命令 R 执行设置的 SQL 语句。此命令无参数，执行 R 命令前必须已经设置过调试语句，且调试过程中命令 R 不允许重复使用。

如果设置了断点，则执行到断点指定的位置时中断，中断将转入调试状态，并显示当前执行所在行的语句；否则，执行完成并显示结果。

例如：

```
DBG>R
Breakpoint 2, SYSDBA.P line: 12@{R1.NAME = 'WANGWU';}
DBG>R                        --调试过程中不可以再次运行
The program is already running.
DBG>B P:23                   --调试过程中可以设置断点
Breakpoint 3 at SYSDBA.P, line 23@{CALL P0;}
```

7. 显示脚本

可以使用命令 L，在工具中显示指定的方法脚本，但不能显示包的单个方法脚本。此命令可以帮助用户准确设置断点位置。L 命令每次显示 5 行代码，多次执行 L 命令时，从以显示的下一行开始显示。L 命令格式如下：

```
L <方法名>
```

方法名可以包含模式、包等前缀。在调试状态下，可以省略方法名，则显示当前正在运行的方法。例如：

```
DBG>L SYSDBA.P
1      CREATE OR REPLACE PROCEDURE P AS
2      TYPE T_REC IS RECORD(ID INT, NAME VARCHAR(20));
3      TYPE ARR is ARRAY INT[4];
4      R1 T_REC;
5      C1 CURSOR;
```

```
DBG>L              --注意，第二次以后显示脚本不要带方法名，否则从第一行显示
6       B ARR;
7       O MYCYCLE;
8       I1 INT;
9       V1 VARCHAR(20);
10      BEGIN
```

8. 中断执行

中断执行命令为：

```
CTRL+C
```

在正常执行过程中，当监听到用户输入 CTRL+C，则 DMDBG 强制中断当前执行，转入调试状态。该命令对于调试死循环错误非常有效。

9. 继续执行

继续执行命令为：

```
C
```

在调试状态下，可以使用命令 C 继续 SQL 语句的执行，直到运行到断点或执行完成。

例如：

```
DBG>C
Breakpoint 3, SYSDBA.P line: 23@{CALL P0;}
```

10. 执行进入

执行进入命令为：

```
S
```

如果当前中断语句包含方法调用，则执行进入命令 S 可以将方法调用展开，执行到其语句块的第一条语句中断。如果当前行没有方法调用，则命令 S 与单步执行命令 N 等效。

例如：

```
DBG>S                    --调入存储过程 P0
SYSDBA.P0 line: 1      @{CREATE OR REPLACE PROCEDURE P0 AS}
```

需要说明的是，对于确定性函数，会直接进行结果重用，在重复调用时将不能执行进入调试。

11. 单步执行

单步执行命令为：

```
N
```

在调试状态下，单步执行命令 N 可以执行到当前层次下一个可中断的行。如果当前行包含多条语句，则所有该行语句都被执行。

例如：

```
DBG>N                    --单步执行存储过程 P0
SYSDBA.P0 line: 3      @{PRINT 'HELLO';}
```

12. 执行跳出

执行进入命令为：

```
F 或 FINISH
```

命令 F 可以执行当前函数的余下操作，如果剩余行有断点则执行到断点中断，否则返回到上层调用后中断。如果当前已经是最上层，则与单步执行命令 N 等效。

例如：

```
DBG>F                    --跳出存储过程 P0
SYSDBA.P line: 32      @{CALL P0;}
```

13. 打印变量

打印变量命令为：

```
P <变量名>
```

命令 P 可以打印指定的变量值。SQL 类型中，除了大字段数据，其它类型都可以打印。

PLSQL 数据类型往往为复杂数据类型，其显示格式如下：

1) 结构类型

在大括号内显示结构所有成员的值，各成员值用“,”分隔。

2) 数组类型

如果是数组，则可以与结构类型一样，依次显示数组所有成员的值，也可以只显示其中某一下标的值。

3) 对象类型

可以打印对象中所有的静态成员，或者打印某一个静态成员，函数可以不打印。

4) 游标类型

游标也是对象类型，其成员如表 12.6 所示。

表 12.6 游标对象成员

名称	数据类型	说明
ROWCOUNT	BIGINT	查询结果集总行数。如果游标未打开或查询失败时该值无效
FOUND	INT	最近一次 FETCH 操作是否取得数据。如果取得为 1，否则为 0。如果游标未打开或查询失败时该值无效
ISOPEN	INT	游标是否打开。打开为 1，否则为 0

P 命令只能打印当前方法所在栈帧中的变量，如果要打印其它层次方法栈帧的变量，首先需要移动到对应栈帧。打印变量的信息以$开始，从 1 开始计数，每打印一个变量，自动增加 1。

例如：

```
DBG>R                              --开始执行
Breakpoint 2, SYSDBA.P line: 12@{R1.NAME = 'WANGWU';}
DBG> P R1.ID                       --打印记录类型中单个字段
$1 = 10012
DBG> N                             ----单步执行后，R1 所有字段都被赋值
SYSDBA.P line: 13      @{OPEN C1 FOR SELECT * FROM STUDENT;}
DBG>P R1                           --打印记录类型 R1
$2 = {ID=10012, NAME=WANGWU}
DBG>C                               --继续运行，遇断点则中断
Breakpoint 2, SYSDBA.P line: 32@{CALL P0;}
DBG>P C1                           --打印游标的所有成员值
$3 = {ISOPEN=1,FOUND=0,ROWCOUNT=2}
DBG>P B                            --打印数组
$4 = {10, 20, 30, 40}
DBG>P B[3]                         --数组下标以 1 开始
$5 = 30
DBG> P O.C1                        --打印对象类型中的成员
$6 = 2
DBG> P O                           --打印对象类型
$7 = MYCYCLE<0>{ID=1, C1=2}
```

14. 显示堆栈

显示堆栈命令为：

```
BT
```

命令 BT 可以显示当前执行的堆栈状态。首先显示当前方法执行到的行数，然后从下至上显示各层方法名称及调用参数值，输出参数不打印。

例如：

```
DBG>BT
#0     SYSDBA.P()     CALL P0;              --#0 表示最下层
#1     @dbg_main      CALL P             --表示栈帧只有两层
```

15. 下移栈帧

下移栈帧命令为：

```
DOWN
```

命令 DOWN 可以将当前调试栈帧下移一层。下移后，显示当前层执行到的语句行数。如果当前已经是堆栈最下层，则此命令无效。

例如：

```
DBG>DOWN                          --存储过程已到最下层，无法下移
Bottom (innermost) frame selected; you cannot go down.
```

16. 上移栈帧

上移栈帧命令为：

```
UP
```

命令 UP 可以将当前调试栈帧上移一层。上移后，显示当前层执行到的语句行数。如果当前已经是堆栈最上层，则此命令无效。

例如：

```
DBG>UP
#1     @dbg_main      CALL P
DBG>UP                            --已经到最上层，无法上移
Initial frame selected; you cannot go up.
DBG> C                            --执行结束，并显示结果
10
20
30
40
HELLO
```

17. 结束执行

结束执行命令为：

```
KILL
```

在调试过程中，用户可以使用命令 KILL 结束当前正在调试的 SQL 语句，并重新设置预调试的 SQL 语句。

KILL 之后，必须重新设置 SQL 语句，断点信息被清空。R 执行结束返回结果之后：如果没有重新设置 SQL 语句，则再次运行的仍是之前设置的 SQL 语句，断点信息可以重用；如果重新设置了 SQL 语句，则断点信息亦被清空。

18. 退出工具

退出 DMDBG 工具的命令为：

```
QUIT
```

如果不希望继续调试，可以输入命令 QUIT。DMDBG 工具将中断连接，退出执行。

12.10 小结

DM 的存储模块机制是一种技术，而不是一种独立的工具，它是和服务器紧密结合在一起的。可以认为这种技术是执行 DM PL/SQL 语言的一种虚拟机，它可以接受任何有效的存储模块，按照语言本身所规定的语义执行，并将结果返回给客户。

存储模块机制具有如下优点：

1. 提供更高的生产率：在设计应用时，围绕存储过程/函数设计应用，可以避免重复编码，提高生产率；在自顶向下设计应用时，不必关心实现的细节，编程方便;
2. 便于维护：用户的存储模块在数据库集中存放，用户可以随时查询、删除、修改它们，而应用程序可以不作任何修改，或只做少量调整;
3. 提供更好的性能：存储模块在创建时被编译成伪码序列，在运行时不需要重新进行编译和优化处理，具有更快的执行速度，可以同时被多个用户调用，并能够减少操作错误;
4. 可以使用存储过程来帮助实施数据的安全性。可以不授权用户直接访问应用程序中的一些表，而是授权用户执行访问这些表的一个过程。当过程被执行时，它将以过程拥有者的权限来执行。除非通过过程，否则用户就不能访问这些表;
5. 存储模块是一种高效访问数据库的机制，使用存储模块可显著减少应用对 DM 的访问次数，提高系统性能;
6. DM 存储模块的编写主要是采用 PL/SQL 语言，DM PL/SQL 语言对三种控制结构（分支结构、迭代结构和顺序结构）都提供了相应的语句支持。DM PL/SQL 还支持异常处理、动态 SQL、游标变量以及返回结果集等功能。同时，DM 也预先定义了一些常用的存储过程和函数供用户使用。

0 表级触发器的触发事件包括某个基表上的 INSERT、DELETE 和 UPDATE 操作，无论对于哪种操作，都能够为其创建 BEFORE 触发器和 AFTER 触发器。如果触发器的动作代码不取决于受影响的数据，语句级触发器就非常有用。例如，可以在表上创建一个 BEFORE INSERT 语句触发器，以防止在某些特定期限以外的时间对一个表进行插入。

每张基表上可创建的触发器的个数没有限制，但是触发器的个数越多，处理 DML 语句所需的时间就越长，这是显而易见的。创建触发器的用户必须是基表的创建者，或者拥有 DBA 权限。注意，不存在触发器的执行权限，因为用户不能主动“调用”某个触发器，是否激发一个触发器是由系统来决定的。

对于语句级和元组级的触发器来说，都是在 DML 语句运行时激发的。在执行 DML 语句的过程中，基表上所创建的触发器按照下面的次序依次执行：

1. 如果有语句级前触发器的话，先运行该触发器；
2. 对于受语句影响每一行：
   1） 如果有行级前触发器的话，运行该触发器；
   2） 执行该语句本身；
   3） 如果有行级后触发器的话，运行该触发器；
3. 如果有语句级后触发器的话，运行该触发器。

需要注意的是，同类触发器的激发顺序没有明确的定义。如果顺序非常重要的话，应该把所有的操作组合在一个触发器中。触发器功能强大，但需要谨慎使用，过多的触发器或复杂触发器过程脚本会降低数据库的运行效率。

还需要注意的是，在 DM7 的数据守护环境下，备库上定义的触发器是不会被触发的。

第13章　管理触发器

DM 是一个具有主动特征的数据库管理系统，其主动特征包括约束机制和触发器机制。约束机制主要用于对某些列进行有效性和完整性验证；触发器（TRIGGER）定义当某些与数据库有关的事件发生时，数据库应该采取的操作。通过触发器机制，用户可以定义、删除和修改触发器。DM 自动管理和运行这些触发器，从而体现系统的主动性，方便用户使用。

触发器是一种特殊的存储过程，它在创建后就存储在数据库中。触发器的特殊性在于它是建立在某个具体的表或视图之上的，或者是建立在各种事件前后的，而且是自动激发执行的，如果用户在这个表上执行了某个 DML 操作（INSERT、DELETE、UPDATE），触发器就被激发执行。

触发器常用于自动完成一些数据库的维护工作。例如，触发器可以具有以下功能：

1. 可以对表自动进行复杂的安全性、完整性检查；
2. 可以在对表进行 DML 操作之前或者之后进行其它处理；
3. 进行审计，可以对表上的操作进行跟踪；
4. 实现不同节点间数据库的同步更新。

触发器与存储模块类似，都是在服务器上保存并执行的一段 DM PL/SQL 语句。不同的是，存储模块必须被显式地调用执行，而触发器是在相关的事件发生时由服务器自动隐式地激发。触发器是激发它们的语句的一个组成部分，即直到一个语句激发的所有触发器执行完成之后该语句才结束，而其中任何一个触发器执行的失败都将导致该语句的失败，触发器所做的任何工作都属于激发该触发器的语句。

触发器为用户提供了一种自己扩展数据库功能的方法。可以使用触发器来扩充引用完整性，实施附加的安全性或增强可用的审计选项。关于触发器应用的例子有：

1. 利用触发器实现表约束机制（如：PRIMARY KEY、FOREIGN KEY、CHECK 等）无法实现的复杂的引用完整性；
2. 利用触发器实现复杂的事务规则（如：想确保薪水增加量不超过 25%）；
3. 利用触发器维护复杂的缺省值（如：条件缺省）；
4. 利用触发器实现复杂的审计功能；

0 表级触发器的触发事件包括某个基表上的 INSERT、DELETE 和 UPDATE 操作，无论对于哪种操作，都能够为其创建 BEFORE 触发器和 AFTER 触发器。如果触发器的动作代码不取决于受影响的数据，语句级触发器就非常有用。例如，可以在表上创建一个 BEFORE INSERT 语句触发器，以防止在某些特定期限以外的时间对一个表进行插入。

每张基表上可创建的触发器的个数没有限制，但是触发器的个数越多，处理 DML 语句所需的时间就越长，这是显而易见的。创建触发器的用户必须是基表的创建者，或者拥有 DBA 权限。注意，不存在触发器的执行权限，因为用户不能主动“调用”某个触发器，是否激发一个触发器是由系统来决定的。

对于语句级和元组级的触发器来说，都是在 DML 语句运行时激发的。在执行 DML 语句的过程中，基表上所创建的触发器按照下面的次序依次执行：

1. 如果有语句级前触发器的话，先运行该触发器；
2. 对于受语句影响每一行：
   1） 如果有行级前触发器的话，运行该触发器；
   2） 执行该语句本身；
   3） 如果有行级后触发器的话，运行该触发器；
3. 如果有语句级后触发器的话，运行该触发器。

需要注意的是，同类触发器的激发顺序没有明确的定义。如果顺序非常重要的话，应该把所有的操作组合在一个触发器中。触发器功能强大，但需要谨慎使用，过多的触发器或复杂触发器过程脚本会降低数据库的运行效率。

还需要注意的是，在 DM7 的数据守护环境下，备库上定义的触发器是不会被触发的。

5. 利用触发器防止非法的操作。

触发器是应用程序分割技术的一个基本组成部分，它将事务规则从应用程序的代码中移到数据库中，从而可确保加强这些事务规则并提高它们的性能。

DM 提供了四种类型的触发器：

1. 表级触发器：基于表中的数据进行触发；
2. 事件触发器：基于特定系统事件进行触发；
3. 时间触发器：基于时间而进行触发。

13.1 触发器的使用

触发器是依附于某个具体的表或视图的特殊存储过程，它在某个 DML 操作的激发下自动执行。在创建触发器时应该仔细考虑它的相关信息。具体来说，应该考虑以下几个方面的问题：

1. 触发器应该建立在哪个表/视图之上；
2. 触发器应该对什么样的 DML 操作进行响应；
3. 触发器在指定的 DML 操作之前激发还是在之后激发；
4. 对每次 DML 响应一次，还是对受 DML 操作影响的每一行数据都响应一次。

在确定了触发器的实现细节后，现在就可以创建触发器了，创建触发器的语法格式为：

```
CREATE [OR REPLACE] TRIGGER 触发器名[WITH  ENCRYPTION]
    BEFORE|AFTER|INSTEAD OF
    DELETE|INSERT|UPDATE [OF 列名]
    ON 表名
    [FOR EACH ROW [WHEN 条件]]
    BEGIN
        PL/SQL 语句
    END;
```

用户如果要在自己的模式中创建触发器，需要具有 CREATE TRIGGER 数据库权限。如果希望能够在其它用户的模式中创建触发器，需要具有 CREATE ANY TRIGGER 数据库权限。

在创建触发器的语法结构中，用方括号限定的部分是可选的，可以根据需要选用。创建

0 表级触发器的触发事件包括某个基表上的 INSERT、DELETE 和 UPDATE 操作，无论对于哪种操作，都能够为其创建 BEFORE 触发器和 AFTER 触发器。如果触发器的动作代码不取决于受影响的数据，语句级触发器就非常有用。例如，可以在表上创建一个 BEFORE INSERT 语句触发器，以防止在某些特定期限以外的时间对一个表进行插入。

每张基表上可创建的触发器的个数没有限制，但是触发器的个数越多，处理 DML 语句所需的时间就越长，这是显而易见的。创建触发器的用户必须是基表的创建者，或者拥有 DBA 权限。注意，不存在触发器的执行权限，因为用户不能主动"调用"某个触发器，是否激发一个触发器是由系统来决定的。

对于语句级和元组级的触发器来说，都是在 DML 语句运行时激发的。在执行 DML 语句的过程中，基表上所创建的触发器按照下面的次序依次执行：

1. 如果有语句级前触发器的话，先运行该触发器；
2. 对于受语句影响每一行：
   1） 如果有行级前触发器的话，运行该触发器；
   2） 执行该语句本身；
   3） 如果有行级后触发器的话，运行该触发器；
3. 如果有语句级后触发器的话，运行该触发器。

需要注意的是，同类触发器的激发顺序没有明确的定义。如果顺序非常重要的话，应该把所有的操作组合在一个触发器中。触发器功能强大，但需要谨慎使用，过多的触发器或复杂触发器过程脚本会降低数据库的运行效率。

还需要注意的是，在 DM7 的数据守护环境下，备库上定义的触发器是不会被触发的。

触发器的命令是 CREATE TRIGGER，根据指定的名字创建一个触发器。OR REPLACE 子句的作用是如果已经存在同名的触发器，则删除它，并重新创建。

触发器可以是前激发的（BEOFRE），也可以是后激发的（AFTER）。如果是前激发的，则触发器在 DML 语句执行之前激发。如果是后激发的，则触发器在 DML 语句执行之后激发。用 BEFORE 关键字创建的触发器是前激发的，用 AFTER 关键字创建的触发器是后激发的，这两个关键字只能使用其一。INSTEAD OF 子句仅用于视图上的触发器，表示用触发器体内定义的操作代替原操作。

触发器可以被任何 DML 命令激发，包括 INSERT、DELETE、UPDATE。如果希望其中的一种、两种或者三种命令能够激发该触发器，则可以指定它们之间的任意组合，两种不同的命令之间用 OR 分开。如果指定了 UPDATE 命令，还可以进一步指定当表中的哪个列受到 UPDATE 命令的影响时激发该触发器。

FOR EACH ROW 子句的作用是指定创建的触发器为元组级触发器。如果没有这样的子句，则创建触发器为语句级触发器。INSTEAD OF 触发器固定为元组级触发器。

由关键字 BEGIN 和 END 限定的部分是触发器的代码，也就是触发器被激发时所执行的代码。代码的编写方法与普通的 PL/SQL 块的编写方法相同。

在触发器中可以定义变量，但必须以 DECLARE 开头。触发器也可以进行异常处理，如果发生异常，就执行相应的异常处理程序。

例如，下面创建的触发器是为了监视用户对表 emp 中的数据所进行的删除操作。如果有这样的访问，则打印相应的信息。

```
CREATE OR REPLACE TRIGGER DEL_TRG
BEFORE DELETE
ON emp
BEGIN
    PRINT '您正在对表 emp 进行删除操作';
END;
```

如果在表 EMP 上进行 DELETE 操作，则激发这个触发器，例如：

```
SQL> DELETE FROM EMP;
```

您正在对表 EMP 进行删除操作

从触发器的执行情况可以看出，无论用户通过 DELETE 命令删除 0 行、1 行或者多行数据，这个触发器只对每次 DELETE 操作激发一次，所以这是一个典型的语句级触发器。

0 表级触发器的触发事件包括某个基表上的 INSERT、DELETE 和 UPDATE 操作，无论对于哪种操作，都能够为其创建 BEFORE 触发器和 AFTER 触发器。如果触发器的动作代码不取决于受影响的数据，语句级触发器就非常有用。例如，可以在表上创建一个 BEFORE INSERT 语句触发器，以防止在某些特定期限以外的时间对一个表进行插入。

每张基表上可创建的触发器的个数没有限制，但是触发器的个数越多，处理 DML 语句所需的时间就越长，这是显而易见的。创建触发器的用户必须是基表的创建者，或者拥有 DBA 权限。注意，不存在触发器的执行权限，因为用户不能主动“调用”某个触发器，是否激发一个触发器是由系统来决定的。

对于语句级和元组级的触发器来说，都是在 DML 语句运行时激发的。在执行 DML 语句的过程中，基表上所创建的触发器按照下面的次序依次执行：

1. 如果有语句级前触发器的话，先运行该触发器；
2. 对于受语句影响每一行：
   1） 如果有行级前触发器的话，运行该触发器；
   2） 执行该语句本身；
   3） 如果有行级后触发器的话，运行该触发器；
3. 如果有语句级后触发器的话，运行该触发器。

需要注意的是，同类触发器的激发顺序没有明确的定义。如果顺序非常重要的话，应该把所有的操作组合在一个触发器中。触发器功能强大，但需要谨慎使用，过多的触发器或复杂触发器过程脚本会降低数据库的运行效率。

还需要注意的是，在 DM7 的数据守护环境下，备库上定义的触发器是不会被触发的。

如果一个触发器不再使用，那么可以删除它。删除触发器的语法为：

```
DROP TRIGGER 触发器名;
```

例如，要删除刚才创建的触发器 DEL_TRG，使用语句为：

```
SQL> DROP TRIGGER DEL_TRG;
```

触发器的创建者和数据库管理员可以使触发器失效。触发器失效后将暂时不起作用，直到再次使它有效，使触发器失效的命令格式为：

```
ALTER TRIGGER 触发器名    DISABLE;
```

触发器失效后只是暂时不起作用，它仍然存在于数据库中，使用命令可以使它再次起作用，使触发器再次有效的命令格式为：

```
ALTER TRIGGER 触发器 ENABLE;
```

例如，下面的两条命令先使触发器 DEL_TRG 失效，然后使其再次有效：

```
SQL> ALTER TRIGGER DEL_TRG DISABLE;
SQL> ALTER TRIGGER DEL_TRG ENABLE;
```

13.2 表级触发器

表级触发器都是基于表中数据的触发器，它通过针对相应表对象的插入/删除/修改等 DML 语句触发。

■ 触发动作

激发表级触发器的触发动作是三种数据操作命令，即 INSERT、DELETE 和 UPDATE 操作。在触发器定义语句中用关键字 INSERT、DELETE 和 UPDATE 指明构成一个触发器事件的数据操作的类型，其中 UPDATE 触发器会依赖于所修改的列，在定义中可通过 UPDATE OF <触发列清单>的形式来指定所修改的列，<触发列清单>指定的字段数不能超过 128 个。

■ 触发级别

根据触发器的级别可分为元组级 (也称行级)和语句级。

元组级触发器，对触发命令所影响的每一条记录都激发一次。假如一个 DELETE 命令从表中删除了 1000 行记录，那么这个表上的元组级 DELETE 触发器将被执行 1000 次。元组级触发器常用于数据审计、完整性检查等应用中。元组级触发器是在触发器定义语句中通过 FOR EACH ROW 子句创建的。对于元组级触发器，可以用一个 WHEN 子句来限制针对当前记

0 表级触发器的触发事件包括某个基表上的 INSERT、DELETE 和 UPDATE 操作，无论对于哪种操作，都能够为其创建 BEFORE 触发器和 AFTER 触发器。如果触发器的动作代码不取决于受影响的数据，语句级触发器就非常有用。例如，可以在表上创建一个 BEFORE INSERT 语句触发器，以防止在某些特定期限以外的时间对一个表进行插入。

每张基表上可创建的触发器的个数没有限制，但是触发器的个数越多，处理 DML 语句所需的时间就越长，这是显而易见的。创建触发器的用户必须是基表的创建者，或者拥有 DBA 权限。注意，不存在触发器的执行权限，因为用户不能主动“调用”某个触发器，是否激发一个触发器是由系统来决定的。

对于语句级和元组级的触发器来说，都是在 DML 语句运行时激发的。在执行 DML 语句的过程中，基表上所创建的触发器按照下面的次序依次执行：

1. 如果有语句级前触发器的话，先运行该触发器；
2. 对于受语句影响每一行：
   1） 如果有行级前触发器的话，运行该触发器；
   2） 执行该语句本身；
   3） 如果有行级后触发器的话，运行该触发器；
3. 如果有语句级后触发器的话，运行该触发器。

需要注意的是，同类触发器的激发顺序没有明确的定义。如果顺序非常重要的话，应该把所有的操作组合在一个触发器中。触发器功能强大，但需要谨慎使用，过多的触发器或复杂触发器过程脚本会降低数据库的运行效率。

还需要注意的是，在 DM7 的数据守护环境下，备库上定义的触发器是不会被触发的。

录是否执行该触发器。WHEN 子句包含一条布尔表达式，当它的值为 TRUE 时，执行触发器；否则，跳过该触发器。

语句级触发器，对每个触发命令执行一次。例如，对于一条将 500 行记录插入表 TABLE_1 中的 INSERT 语句，这个表上的语句级 INSERT 触发器只执行一次。语句级触发器一般用于对表上执行的操作类型引入附加的安全措施。语句级触发器是在触发器定义语句中通过 FOR EACH STATEMENT 子句创建的，该子句可缺省。

■ 触发时机

触发时机通过两种方式指定。一是通过指定 BEFORE 或 AFTER 关键字，选择在触发动作之前或之后运行触发器；二是通过指定 INSTEAD OF 关键字，选择在动作触发的时候，替换原始操作，INSTEAD OF 允许建立在视图上，并且只支持行级触发。

在元组级触发器中可以引用当前修改的记录在修改前后的值，修改前的值称为旧值，修改后的值称为新值。对于插入操作不存在旧值，而对于删除操作则不存在新值。

对于新、旧值的访问请求常常决定一个触发器是 BEFORE 类型还是 AFTER 类型。如果需要通过触发器对插入的行设置列值，那么为了能设置新值，需要使用一个 BEFORE 触发器，因为在 AFTER 触发器中不允许用户设置已插入的值。在审计应用中则经常使用 AFTER 触发器，因为元组修改成功后才有必要运行触发器，而成功地完成修改意味着成功地通过了该表的引用完整性约束。

13.3 事件触发器

前面表级触发器都是基于表中数据的触发器，它通过针对相应表对象的插入/删除/修改等 DML 语句触发。DM 还支持事件触发器，包括库级和模式级触发器，这类触发器并不依赖于某个表，而是基于特定系统事件触发的，通过指定 DATABASE 或某个 SCHEMA 来表示事件触发器的作用区域。创建事件触发器的用户需要拥有 CREATE_TRIGGER 或 CREATE_ANY_TRIGGER 的权限。

可以触发的事件包含以下两类：

1. DDL 事件，包括 CREATE、ALTER、DROP、GRANT、REVOKE 以及 TRUNCATE；
2. 系统事件，包括 LOGIN/LOGON、LOGOUT/LOGOFF、AUDIT、NOAUDIT、BACKUP DATABASE、RESTORE DATABASE、TIMER、STARTUP、SHUTDOWN 以及 SERERR

0 表级触发器的触发事件包括某个基表上的 INSERT、DELETE 和 UPDATE 操作，无论对于哪种操作，都能够为其创建 BEFORE 触发器和 AFTER 触发器。如果触发器的动作代码不取决于受影响的数据，语句级触发器就非常有用。例如，可以在表上创建一个 BEFORE INSERT 语句触发器，以防止在某些特定期限以外的时间对一个表进行插入。

每张基表上可创建的触发器的个数没有限制，但是触发器的个数越多，处理 DML 语句所需的时间就越长，这是显而易见的。创建触发器的用户必须是基表的创建者，或者拥有 DBA 权限。注意，不存在触发器的执行权限，因为用户不能主动“调用”某个触发器，是否激发一个触发器是由系统来决定的。

对于语句级和元组级的触发器来说，都是在 DML 语句运行时激发的。在执行 DML 语句的过程中，基表上所创建的触发器按照下面的次序依次执行：

1. 如果有语句级前触发器的话，先运行该触发器；
2. 对于受语句影响每一行：
   1） 如果有行级前触发器的话，运行该触发器；
   2） 执行该语句本身；
   3） 如果有行级后触发器的话，运行该触发器；
3. 如果有语句级后触发器的话，运行该触发器。

需要注意的是，同类触发器的激发顺序没有明确的定义。如果顺序非常重要的话，应该把所有的操作组合在一个触发器中。触发器功能强大，但需要谨慎使用，过多的触发器或复杂触发器过程脚本会降低数据库的运行效率。

还需要注意的是，在 DM7 的数据守护环境下，备库上定义的触发器是不会被触发的。

（即执行错误事件）。

所有 DDL 事件触发器都可以设置 BEFORE 或 AFTER 的触发时机，但系统事件中 LOGOUT，SHUTDOWN 仅能设置为 BEFORE，而其它则只能设置为 AFTER。模式级触发器不能是 LOGIN/LOGON、LOGOUT/LOGOFF、SERERR、BACKUP DATABASE、RESTORE DATABASE、STARTUP 和 SHUTDOWN 事件触发器。

与数据触发器不同，事件触发器不能影响对应触发事件的执行。它的主要作用是帮助管理员监控系统运行发生的各类事件，进行一定程度的审计和监视工作。

13.4 时间触发器

从 DM7 开始，触发器模块中新增了一种特殊的事件触发器类型，就是时间触发器，时间触发器的特点是用户可以定义在任何时间点、时间区域、每隔多长时间等等的方式来激发触发器，而不是通过数据库中的某些操作包括 DML、DDL 操作等来激发，它的最小时间精度为分钟。

时间触发器与其它触发器的不同只是在触发事件上，在 PL/SQL 语句块（BEGIN 和 END 之间的语句）的定义是完全相同的。

时间触发器的创建语句如下：

```
CREATE [OR REPLACE] TRIGGER 触发器名 WITH ENCRYPTION
    AFTER TIMER ON DATABASE
    {时间定义语句}
    BEGIN
        PL/SQL 语句
    END;
```

时间定义语句的语法规则较多，具体可参考《DM_SQL》中时间触发器相关章节。

下面的例子在每个月的第 28 天，从早上 9 点开始到晚上 18 点之间，每隔一分钟就打印一个字符串“Hello World”。

```
CREATE OR REPLACE TRIGGER timer2
    AFTER TIMER ON DATABASE
    FOR EACH 1 MONTH DAY 28
```

0 表级触发器的触发事件包括某个基表上的 INSERT、DELETE 和 UPDATE 操作，无论对于哪种操作，都能够为其创建 BEFORE 触发器和 AFTER 触发器。如果触发器的动作代码不取决于受影响的数据，语句级触发器就非常有用。例如，可以在表上创建一个 BEFORE INSERT 语句触发器，以防止在某些特定期限以外的时间对一个表进行插入。

每张基表上可创建的触发器的个数没有限制，但是触发器的个数越多，处理 DML 语句所需的时间就越长，这是显而易见的。创建触发器的用户必须是基表的创建者，或者拥有 DBA 权限。注意，不存在触发器的执行权限，因为用户不能主动“调用”某个触发器，是否激发一个触发器是由系统来决定的。

对于语句级和元组级的触发器来说，都是在 DML 语句运行时激发的。在执行 DML 语句的过程中，基表上所创建的触发器按照下面的次序依次执行：

1. 如果有语句级前触发器的话，先运行该触发器；
2. 对于受语句影响每一行：
   1） 如果有行级前触发器的话，运行该触发器；
   2） 执行该语句本身；
   3） 如果有行级后触发器的话，运行该触发器；
3. 如果有语句级后触发器的话，运行该触发器。

需要注意的是，同类触发器的激发顺序没有明确的定义。如果顺序非常重要的话，应该把所有的操作组合在一个触发器中。触发器功能强大，但需要谨慎使用，过多的触发器或复杂触发器过程脚本会降低数据库的运行效率。

还需要注意的是，在 DM7 的数据守护环境下，备库上定义的触发器是不会被触发的。


```
    FROM TIME '09:00' TO TIME '18:00' FOR EACH 1 MINUTE
    DECLARE
    str VARCHAR;
    BEGIN
        PRINT 'HELLO WORLD';
    END;
    /
```

时间触发器实用性很强，如在凌晨（此时服务器的负荷比较轻）做一些数据的备份操作，对数据库中表的统计信息的更新操作等类似的事情。同时也可以作为定时器通知一些用户在未来的某些时间要做哪些事情。

13.5 触发器总结

表级触发器的触发事件包括某个基表上的 INSERT、DELETE 和 UPDATE 操作，无论对于哪种操作，都能够为其创建 BEFORE 触发器和 AFTER 触发器。如果触发器的动作代码不取决于受影响的数据，语句级触发器就非常有用。例如，可以在表上创建一个 BEFORE INSERT 语句触发器，以防止在某些特定期限以外的时间对一个表进行插入。

每张基表上可创建的触发器的个数没有限制，但是触发器的个数越多，处理 DML 语句所需的时间就越长，这是显而易见的。创建触发器的用户必须是基表的创建者，或者拥有 DBA 权限。注意，不存在触发器的执行权限，因为用户不能主动“调用”某个触发器，是否激发一个触发器是由系统来决定的。

对于语句级和元组级的触发器来说，都是在 DML 语句运行时激发的。在执行 DML 语句的过程中，基表上所创建的触发器按照下面的次序依次执行：

1. 如果有语句级前触发器的话，先运行该触发器；
2. 对于受语句影响每一行：
   1） 如果有行级前触发器的话，运行该触发器；
   2） 执行该语句本身；
   3） 如果有行级后触发器的话，运行该触发器；
3. 如果有语句级后触发器的话，运行该触发器。

0 表级触发器的触发事件包括某个基表上的 INSERT、DELETE 和 UPDATE 操作，无论对于哪种操作，都能够为其创建 BEFORE 触发器和 AFTER 触发器。如果触发器的动作代码不取决于受影响的数据，语句级触发器就非常有用。例如，可以在表上创建一个 BEFORE INSERT 语句触发器，以防止在某些特定期限以外的时间对一个表进行插入。

每张基表上可创建的触发器的个数没有限制，但是触发器的个数越多，处理 DML 语句所需的时间就越长，这是显而易见的。创建触发器的用户必须是基表的创建者，或者拥有 DBA 权限。注意，不存在触发器的执行权限，因为用户不能主动“调用”某个触发器，是否激发一个触发器是由系统来决定的。

对于语句级和元组级的触发器来说，都是在 DML 语句运行时激发的。在执行 DML 语句的过程中，基表上所创建的触发器按照下面的次序依次执行：

1. 如果有语句级前触发器的话，先运行该触发器；
2. 对于受语句影响每一行：
   1） 如果有行级前触发器的话，运行该触发器；
   2） 执行该语句本身；
   3） 如果有行级后触发器的话，运行该触发器；
3. 如果有语句级后触发器的话，运行该触发器。

需要注意的是，同类触发器的激发顺序没有明确的定义。如果顺序非常重要的话，应该把所有的操作组合在一个触发器中。触发器功能强大，但需要谨慎使用，过多的触发器或复杂触发器过程脚本会降低数据库的运行效率。

还需要注意的是，在 DM7 的数据守护环境下，备库上定义的触发器是不会被触发的。

需要注意的是，同类触发器的激发顺序没有明确的定义。如果顺序非常重要的话，应该把所有的操作组合在一个触发器中。触发器功能强大，但需要谨慎使用，过多的触发器或复杂触发器过程脚本会降低数据库的运行效率。

还需要注意的是，在 DM7 的数据守护环境下，备库上定义的触发器是不会被触发的。

0 表级触发器的触发事件包括某个基表上的 INSERT、DELETE 和 UPDATE 操作，无论对于哪种操作，都能够为其创建 BEFORE 触发器和 AFTER 触发器。如果触发器的动作代码不取决于受影响的数据，语句级触发器就非常有用。例如，可以在表上创建一个 BEFORE INSERT 语句触发器，以防止在某些特定期限以外的时间对一个表进行插入。

每张基表上可创建的触发器的个数没有限制，但是触发器的个数越多，处理 DML 语句所需的时间就越长，这是显而易见的。创建触发器的用户必须是基表的创建者，或者拥有 DBA 权限。注意，不存在触发器的执行权限，因为用户不能主动"调用"某个触发器，是否激发一个触发器是由系统来决定的。

对于语句级和元组级的触发器来说，都是在 DML 语句运行时激发的。在执行 DML 语句的过程中，基表上所创建的触发器按照下面的次序依次执行：

1. 如果有语句级前触发器的话，先运行该触发器；
2. 对于受语句影响每一行：
   1） 如果有行级前触发器的话，运行该触发器；
   2） 执行该语句本身；
   3） 如果有行级后触发器的话，运行该触发器；
3. 如果有语句级后触发器的话，运行该触发器。

需要注意的是，同类触发器的激发顺序没有明确的定义。如果顺序非常重要的话，应该把所有的操作组合在一个触发器中。触发器功能强大，但需要谨慎使用，过多的触发器或复杂触发器过程脚本会降低数据库的运行效率。

还需要注意的是，在 DM7 的数据守护环境下，备库上定义的触发器是不会被触发的。

第14章 管理视图、序列和同义词

视图、序列和同义词是数据库中比较重要的模式对象概念，达梦数据库支持这些模式对象的管理。

14.1 管理视图

视图是关系数据库系统提供给用户以多种角度观察数据库中数据的重要机制，它简化了用户数据模型，提供了逻辑数据独立性，实现了数据共享和数据的安全保密。视图是数据库技术中一个十分重要的功能。从系统实现的角度讲，视图是从一个或几个基表（或视图）导出的表，但它是一个虚表，即数据字典中只存放视图的定义（由视图名和查询语句组成），而不存放对应的数据，这些数据仍存放在原来的基表中。当对一个视图进行查询时，视图将查询其对应的基表，并且将所查询的结果以视图所规定的格式和次序进行返回。因此当基表中的数据发生变化时，从视图中查询出的数据也随之改变了。从用户的角度来讲，视图就像一个窗口，透过它可以看到数据库中用户感兴趣的数据和变化。当用户所需的数据是一张表的部分列、或部分行，或者数据是分散在多个表中，那么就可以创建视图来将这些满足条件的行和列组织到一个表，而不需要修改表的属性、甚至创建新的表。这样不仅简化了用户的操作，还可以提高数据的逻辑独立性，实现数据的共享和保密。

例如，为了防止非公司人员查询供应商的具体信息，包括 email、phone 等，这时可以建立视图，针对不用的用户建立查询供应商信息的视图。

```
CREATE VIEW normal_view AS SELECT name FROM person;
CREATE VIEW special_view AS SELECT name, sex, email, phone FROM person;
```

然后把视图 normal_view 的权限授予普通用户，只允许他们查询供应商的姓名信息，把 special_view 视图的权限授予高级用户，允许他们查看供应商的 email、电话等信息，这样就较好地实现了供应商信息的安全与保密。

关于视图的定义、删除、查询和修改可以参考《DM_SQL》。

用户也可以对那些经常进行的查询建立相应视图，如果经常使用的查询语句较庞大，这样可以节省繁琐语句的书写。

0 表级触发器的触发事件包括某个基表上的 INSERT、DELETE 和 UPDATE 操作，无论对于哪种操作，都能够为其创建 BEFORE 触发器和 AFTER 触发器。如果触发器的动作代码不取决于受影响的数据，语句级触发器就非常有用。例如，可以在表上创建一个 BEFORE INSERT 语句触发器，以防止在某些特定期限以外的时间对一个表进行插入。

每张基表上可创建的触发器的个数没有限制，但是触发器的个数越多，处理 DML 语句所需的时间就越长，这是显而易见的。创建触发器的用户必须是基表的创建者，或者拥有 DBA 权限。注意，不存在触发器的执行权限，因为用户不能主动“调用”某个触发器，是否激发一个触发器是由系统来决定的。

对于语句级和元组级的触发器来说，都是在 DML 语句运行时激发的。在执行 DML 语句的过程中，基表上所创建的触发器按照下面的次序依次执行：

1. 如果有语句级前触发器的话，先运行该触发器；
2. 对于受语句影响每一行：
   1） 如果有行级前触发器的话，运行该触发器；
   2） 执行该语句本身；
   3） 如果有行级后触发器的话，运行该触发器；
3. 如果有语句级后触发器的话，运行该触发器。

需要注意的是，同类触发器的激发顺序没有明确的定义。如果顺序非常重要的话，应该把所有的操作组合在一个触发器中。触发器功能强大，但需要谨慎使用，过多的触发器或复杂触发器过程脚本会降低数据库的运行效率。

还需要注意的是，在 DM7 的数据守护环境下，备库上定义的触发器是不会被触发的。

由于视图没有直接相关的物理数据，所以不能被索引。

DM 提供了一些以"V$"开头的视图来供用户了解当前服务器的使用情况。这些视图的数据在服务器运行期间一直变化，并且数据反映的主要是系统的性能信息，因此它们被称为动态性能视图。用户对动态性能视图只能进行查询操作，动态视图的具体内容可参考附录 2。

14.2 管理序列

序列（sequence）是 DM 数据库中的数据库实体之一。通过使用序列，多个用户可以产生和使用一组不重复的有序整数值。比如可以用序列来自动地生成主关键字值。序列通过提供唯一数值的顺序表来简化程序设计工作。

当一个序列第一次被查询调用时，它将返回一个预定值，该预定值就是在创建序列时所指定的初始值。默认情况下，对于升序序列，序列的缺省初始值为序列的最小值，对于降序序列，缺省初始值为序列的最大值。可以指定序列能生成的最大值，默认情况下，降序序列的最大值缺省为-1，升序序列的最大值为 2^{63}-1；也可以指定序列能生成的最小值，默认情况下，升序序列的最小值缺省为 1，降序序列的最小值为-2^{63}。序列的最大值和最小值可以指定为 longint（4 个字节）所能表示的最大和最小有符号整数。

在随后的每一次查询中，序列将产生一个按其指定的增量增长的值。增量可以是任意的正整数或负整数，但不能为 0。如果此值为负，序列是下降的，如果此值为正，序列是上升的。默认情况下，增加缺省为 1。

一旦序列生成，用户就可以在 SQL 语句中用以下伪列来存取序列的值：

1. CURRVAL 返回当前的序列值；
2. NEXTVAL 如果为升序序列，序列值增加并返回增加后的值；如果为降序序列，序列值减少并返回减少后的值。

序列可以是循环的，当序列的值达到最大值/最小值时，序列将从最小值/最大值计数。使用一个序列时，不保证将生成一串连续不断递增的值。例如，如果查询一个序列的下一个值供 insert 使用，则该查询是能使用这个序列值的唯一会话。如果未能提交事务处理，则序列值就不被插入表中，以后的 insert 将继续使用该序列随后的值。

序列在对编号的使用上具有很大用处，如果想对表建立一个列专门用来表示编号，如订单号，这样就可以使用序列，依次递增生成，用户不需进行特殊管理，这给用户带来了很大

0 表级触发器的触发事件包括某个基表上的 INSERT、DELETE 和 UPDATE 操作，无论对于哪种操作，都能够为其创建 BEFORE 触发器和 AFTER 触发器。如果触发器的动作代码不取决于受影响的数据，语句级触发器就非常有用。例如，可以在表上创建一个 BEFORE INSERT 语句触发器，以防止在某些特定期限以外的时间对一个表进行插入。

每张基表上可创建的触发器的个数没有限制，但是触发器的个数越多，处理 DML 语句所需的时间就越长，这是显而易见的。创建触发器的用户必须是基表的创建者，或者拥有 DBA 权限。注意，不存在触发器的执行权限，因为用户不能主动“调用”某个触发器，是否激发一个触发器是由系统来决定的。

对于语句级和元组级的触发器来说，都是在 DML 语句运行时激发的。在执行 DML 语句的过程中，基表上所创建的触发器按照下面的次序依次执行：

1. 如果有语句级前触发器的话，先运行该触发器；
2. 对于受语句影响每一行：
   1） 如果有行级前触发器的话，运行该触发器；
   2） 执行该语句本身；
   3） 如果有行级后触发器的话，运行该触发器；
3. 如果有语句级后触发器的话，运行该触发器。

需要注意的是，同类触发器的激发顺序没有明确的定义。如果顺序非常重要的话，应该把所有的操作组合在一个触发器中。触发器功能强大，但需要谨慎使用，过多的触发器或复杂触发器过程脚本会降低数据库的运行效率。

还需要注意的是，在 DM7 的数据守护环境下，备库上定义的触发器是不会被触发的。

方便。如果用户需要间隔的编号，创建序列时指定 INCREMENT，就可以生成用户需要的编号。关于序列的具体使用，请参考《DM_SQL》。

14.3 管理同义词

同义词相当于模式对象的别名，起着连接数据库模式对象和应用程序的作用。假如模式对象需要更换或者修改，则不用修改应用程序而直接修改同义词就可以了。

同义词是用来实现下列用途的数据库对象：

1. 为可以存在于本地或远程服务器上的其他数据库对象（称为基础对象）提供备用名称；
2. 提供抽象层以免客户端应用程序对基础对象的名称或位置进行更改。

同义词的好处在于用户可能需要某些对象在不同的场合采用不用的名字，使其适合不同人群的应用环境。例如，创建表 product，如果客户不认识这个英文词，这时可以增加同义词，命名“产品”，这样客户就有较直观的观念，一目了然。

关于同义词的具体使用，请参考《DM_SQL》。

14.4 查看视图、序列和同义词信息

视图、序列以及同义词的定义信息可以通过查看系统表 SYSOBJECTS 和 SYSTEXTS 得到，如查看普通视图 view_1 的信息，可以执行：

```
    SELECT b.* FROM  SYS.SYSOBJECTS a, SYS.SYSTEXTS b WHERE a.ID = b.ID and a.NAME
LIKE 'VIEW_1%';
```

可以得到：

```
    ID            SEQNO         TXT
    1       16778216         0          CREATE VIEW VIEW_1 AS SELECT ID FROM
SYSOBJECTS;
    2      16778216       1        SELECT ID FROM SYSOBJECTS
```

这样可以很明确知道 VIEW_1 视图的定义。

0 表级触发器的触发事件包括某个基表上的 INSERT、DELETE 和 UPDATE 操作，无论对于哪种操作，都能够为其创建 BEFORE 触发器和 AFTER 触发器。如果触发器的动作代码不取决于受影响的数据，语句级触发器就非常有用。例如，可以在表上创建一个 BEFORE INSERT 语句触发器，以防止在某些特定期限以外的时间对一个表进行插入。

每张基表上可创建的触发器的个数没有限制，但是触发器的个数越多，处理 DML 语句所需的时间就越长，这是显而易见的。创建触发器的用户必须是基表的创建者，或者拥有 DBA 权限。注意，不存在触发器的执行权限，因为用户不能主动“调用”某个触发器，是否激发一个触发器是由系统来决定的。

对于语句级和元组级的触发器来说，都是在 DML 语句运行时激发的。在执行 DML 语句的过程中，基表上所创建的触发器按照下面的次序依次执行：

1. 如果有语句级前触发器的话，先运行该触发器；
2. 对于受语句影响每一行：
   1） 如果有行级前触发器的话，运行该触发器；
   2） 执行该语句本身；
   3） 如果有行级后触发器的话，运行该触发器；
3. 如果有语句级后触发器的话，运行该触发器。

需要注意的是，同类触发器的激发顺序没有明确的定义。如果顺序非常重要的话，应该把所有的操作组合在一个触发器中。触发器功能强大，但需要谨慎使用，过多的触发器或复杂触发器过程脚本会降低数据库的运行效率。

还需要注意的是，在 DM7 的数据守护环境下，备库上定义的触发器是不会被触发的。

管理视图、序列和同义词

视图的定义信息也可以通过 SP_VIEWDEF 系统过程来查看。对于物化视图而言，其定义只有通过该系统函数才能完整地获取。

```
CALL  SP_VIEWDEF('SYSDBA', 'VIEW1');
```

第15章 模式对象的常规管理

用户的模式(SCHEMA）指的是用户账号拥有的对象集，在概念上可将其看作是包含表、视图、索引和权限定义的对象。在 DM 中，一个用户可以创建多个模式，一个模式中的对象（表、视图等）可以被多个用户使用。模式不是严格分离的，一个用户可以访问他所连接的数据库中有权限访问的任意模式中的对象。

系统为每一个用户自动建立了一个与用户名同名的模式作为其默认模式，用户还可以用模式定义语句建立其它模式。

采用模式的原因有几点：

1. 允许多个用户使用一个数据库而不会干扰其它用户；
2. 把数据库对象组织成逻辑组，让它们更便于管理；
3. 第三方的应用可以放在不同的模式中，这样可以避免和其它对象的名字冲突。模式类似于操作系统层次的目录，只不过模式不能嵌套。

DM7 模式可以通过 SQL 语句进行操作。DM7 模式主要包含以下的模式对象：

1. 表；
2. 视图；
3. 索引；
4. 触发器；
5. 存储过程/函数；
6. 序列；
7. 全文索引；
8. 包；
9. 同义词；
10. 类；
11. 外部链接。

在引用模式对象的时候，一般要在模式对象名前面加上模式名。具体格式如下：

[模式名].对象名

当然，在当前模式和要引用的模式对象所属的模式相同时，可以省略模式名。如果我们访问一个表时，没有指明该表属于哪一个模式，系统就会自动给我们在表前加上缺省的模式名。类似地，如果我们在创建对象时不指定该对象的模式，则该对象的模式为用户的缺省模式。

模式对象之外的其他对象统一称为非模式对象，非模式对象主要包括以下几种对象：

1. 用户；
2. 角色；
3. 权限；
4. 表空间。

15.1 在单个操作中创建多个模式对象

在 DM7 中，使用 CREATE SCHEMA 语句就可以创建一个空的模式对象，也可以在创建模式时，同时创建多个模式对象。CREATE SCHEMA 语句的语法说明参见《DM_SQL》。

下面的语句在创建模式 TEST 的同时，创建了属于模式 TEST 的一个序列和两张表。

```
CREATE SCHEMA TEST
    CREATE SEQUENCE ADDRESS_SEQ INCREMENT BY 1
```

```
CREATE TABLE ADDRESS(
     ADDRESSID INT,
         ADDRESS1 VARCHAR(60) NOT NULL,
         ADDRESS2 VARCHAR(60),
         CITY VARCHAR(30) NOT NULL,
         POSTALCODE VARCHAR(15) NOT NULL)
CREATE TABLE ADDRESS_TYPE(
     ADDRESS_TYPEID INT PRIMARY KEY,
         NAME VARCHAR(50) NOT NULL);
```

可以用下面的格式来引用 TEST 模式中的 ADDRESS 表：TEST.ADDRESS，如向 ADDRESS 表中插入一条数据。

```
INSERT INTO TEST.ADDRESS VALUES(1,'武汉市关山一路特 1 号光谷软件园 C6 栋 5 层','上海市闸北区江场三路 28 号 301 室','上海市','200436');
```

15.2 重命名模式对象

要重命名一个模式对象，那么这个模式对象必须在指定的模式里面。用户可以采用下面几种方式来重命名模式对象：

1. 删除原有的模式对象，重新创建；
2. 使用 ALTER … RENAME 语句（TABLE）。

如果采用的是删除并重建模式对象的方式来重命名模式对象，那么所有的基于这个模式对象的授权都将失效。在模式对象重建之后，基于原对象的权限和角色必须重新授予。

如果使用的是 ALTER … RENAME 语句来重命名模式对象，那么所有基于原模式对象的权限将转移到重命名后的模式对象上，不需要重新授权。如重命名 TEST 上的 ADDRESS 表：

```
ALTER TABLE TEST.ADDRESS RENAME TO ADDRESS1;
```

如果一个用户 USER1 在 ADDRESS 上拥有查询、插入的权限，那么重命名之后，USER1 在 ADDRESS1 上同样拥有查询、插入权限。

在重命名一个模式对象之前，需要考虑以下影响：

1. 所有基于重命名模式对象的视图和 PL/SQL 块都将失效，在使用之前，必须重新编译；
2. 所有基于重命名模式对象的同义词在使用的时候都会报错。

15.3 启用和停用触发器

触发器（TRIGGER）定义当某些与数据库有关的事件发生时，数据库应该采取的操作。触发器是在相关的事件发生时由服务器自动地隐式激发的。触发器是激发它们的语句的一个组成部分，即直到一个语句激发的所有触发器执行完成之后该语句才结束，而其中任何一个触发器执行的失败都将导致该语句的失败，触发器所做的任何工作都属于激发该触发器的语句。

一个触发器有两种不同的状态：

1. 启用（ENABLED）：处于开启状态的触发器在触发条件满足时，执行触发体。缺省状态下，新创建的触发器都处于开启状态；
2. 禁止（DISABLED）：处于禁止状态的触发器在触发条件满足时，也不会执行触发体。

要设置触发器的状态，可以通过 ALTER TRIGGER 语句来完成。用户必须满足以下条件：

1) 拥有该触发器；
2) 有 ALTER ANY TRIGGER 权限。

用户可以通过使用带有 ENABLED/DISABLED 选项的 ALTER TRIGGER 语句来启用/禁止触发器。如启用 addr_trig 触发器：

```
ALTER TRIGGER addr_trig ENABLED;
```

如禁用 addr_trig 触发器：

```
ALTER TRIGGER addr_trig DISABLED;
```

15.4 管理完整性约束

完整性约束规则，限制表中一个或者多个列的值。约束子句可以出现在 CREATE TABLE 或者 ALTER TABLE 语句中，确定约束条件并指定受到约束的列。

15.4.1 完整性约束状态

用户可以指定一个约束是启用（ENABLE）或禁止（DISABLED）。如果启用一个约束，那么在插入数据或者是更新数据时会对数据进行检查，不符合约束的数据被阻止进入。如果约束是禁止（DISABLED）的，不符合约束的数据被允许进入数据库。

1. 禁止约束

要执行完整性约束定义的规则，约束应当设置为开启状态。但是，在下面的情况下，从性能的角度考虑，可以暂时将完整性约束禁用。

1) 导入大量的数据到一张表中；
2) 做批处理操作并对一张表做大规模修改时；
3) 导入导出一张表。

在上面三种情形下，暂时禁用完整性约束能提高系统的性能，特别是在数据仓库的情况下。在禁用约束的情况下，可以将违反约束规则的数据插入到表中，因此，用户应当在上面列表中的操作结束之后启用约束。

2. 启用约束

在启用完整性约束的情况下，不满足约束规则的行是不能插入到表中的。

15.4.2 定义完整性约束

下面的 CREATE TABLE 和 ALTER TABLE 语句在定义的时候就启用完整性约束：

```
CREATE TABLE t_con (
id NUMBER(5) CONSTRAINT t_con_pk PRIMARY KEY);
ALTER TABLE t_con
ADD CONSTRAINT t_con_pk PRIMARY KEY (id);
```

使用 ALTER TABLE 语句来启用完整性约束，可能会失败，这是因为表中已存在的数据可能违反完整性约束条件。

当启用 UNIQUE 或者 PRIMARY KEY 约束时，系统就会创建一个对应的索引。

15.4.3 修改或删除现有的完整性约束

用户可以使用 ALTER TABLE 语句来启用、禁止、删除一个约束。当用户使用 UNIQUE 或者 PRIMARY KEY 约束时，系统会创建对应的索引；当这个约束被删除或者是被禁用，索

引就会被删除。

1. 禁用已被启用的约束

下面的 ALTER TABLE 语句可以禁止完整性约束：

```
ALTER TABLE t_con DISABLE CONSTRAINT t_con_pk;
```

当有外键引用 UNIQUE 或者 PRIMARY KEY 列时，用户不能禁用 UNIQUE 或者 PRIMARY KEY 约束。

2. 删除约束

用户可以通过 ALTER TABLE 语句和 DROP CONSTRAINT 参数来删除完整性约束。如：

```
ALTER TABLE t_con DROP CONSTRAINT t_con_pk;
```

如果有外键引用约束(UNIQUE 或者 PRIMARY KEY)时，在删除约束时必须加上 CASCADE 参数，否则不能删除。

15.4.4 查看约束信息

用户可以在系统表 SYSOBJECTS 和 SYSCONS 中查询约束的信息。例如，在 SYSOBJECTS 系统表中查找约束名为 t_con_pk 的信息。

```
SELECT * FROM SYSOBJECTS WHERE NAME='T_CON_PK';
```

查找所有约束的信息：

```
SELECT * FROM SYSOBJECTS WHERE TYPE$='CONS';
```

系统表 SYSOBJECTS 和 SYSCONS 的详细描述详见附录数据字典部分。

15.5 管理对象依赖性

在实际应用中，许多模式对象需要引用其他的对象。例如，一个视图中包含一个查询，其中引用了其他的表或视图。一个模式对象引用了别的对象被称为依赖对象，被引用的对象称为被依赖对象。

达梦数据库在对象被调用的时候自动重新编译，确保对象有效。

15.6 管理对象名称解析

SQL 语句中的对象名字是由两部分组成，之间用点号隔开。下面描述的是达梦数据库管理系统怎样解析对象名字。

1. 如果只有一个名字，而没有点号。
   在当前模式下寻找是否存在相同名字的对象，如果找到，则返回；否则报错；
2. 如果有点号，首先检测对象名的第一部分，如在 TEST.ADDRESS 中，TEST 就是第一部分。
   1) 寻找哪一个模式的名字和第一部分相同，如果找到，以此模式进行步骤 2)；否则，以当前模式进行步骤 2)；
   2) 在模式中需找是否有与对象名第二部分同名的对象，如果找到且待解析对象名只有两个部分，则返回；如果找到但待解析对象名多于两个部分，则转步骤 3)；否则报错；
   3) 在模式中查找包含在前一个对象中并且和待解析对象名当前解析部分相同名字的对象，如果找到，循环步骤 3)，直至所有的部分检测结束后再返回；如果没有找到，则直接报错。如 TEST.SCHOOL.CLASS.STUDENT，在 TEST 模

式下中的 SCHOOL 对象中寻找 CLASS 的对象，然后在 CLASS 对象中寻找 STUDENT 对象。

15.7 显示有关模式对象的信息

在 DM 数据库系统中，模式对象的信息主要记录在 SYSOBJECTS 系统表中。可以通过下面的语句来查询模式对象的相关信息。

```
SELECT * FROM SYSOBJECTS WHERE TYPE$ = 'SCHOBJ' OR TYPE$ = 'TABOBJ';
```

但是，SYSOBJECTS 系统表并不能将模式对象的所有信息存储起来。用户应该和其它系统表一起获得更加详细的信息。

如果用户想获得索引更详细的信息，可以查询 SYSINDEXES 表；如果用户想了解存储过程、函数等的定义信息，可以查询 SYSTEXTS 表；如果用户想了解更详细的约束的相关信息，可以查询 SYSCONS 表。（系统表的详细介绍参考附录 1。）

第三部分 高级数据库管理

第16章 数据库布局和存储管理

规划数据库结构时需要考虑如何管理数据库中的相关文件， 每个表空间存储什么数据，在表空间中创建几个多大的数据文件，以及数据文件存储的位置等。本章介绍了表空间的管理、数据文件的管理、重做日志文件的管理、回滚空间的管理和控制文件的管理。

16.1 管理表空间

表空间的管理操作需要 DM 服务器处于打开状态下。

16.1.1 创建表空间

创建表空间时需要指定表空间名和其拥有的数据文件列表。比如创建名为 bookshop 的表空间，并指定该空间上拥有 2 个数据文件，每个数据文件的大小为 128M。

```
    CREATE TABLESPACE bookshop DATAFILE 'd:\bookshop1.dbf' SIZE 128,
'd:\bookshop2.dbf' SIZE 128;
```

理论上最多允许有 65535 个表空间，但用户允许创建的表空间 ID 取值范围为 0~32767，超过 32767 的只允许系统使用，ID 由系统自动分配，ID 不能重复使用，即使删除掉已有表空间，也无法重复使用已用 ID 号，也就是说只要创建 32768 次表空间后，用户将无法再创建表空间。

16.1.2 扩展表空间

表空间通过数据文件来扩展，表空间的大小等于构成该表空间的所有数据文件的大小之和。所以要扩展表空间可以通过添加新的数据文件或者扩展表空间中已有的数据文件完成。数据文件的添加和扩展可见下一节。

16.1.3 删除表空间

只可以删除用户创建的表空间并且只能删除未使用过的表空间。删除表空间时会删除其拥有的所有数据文件。例如删除 bookshop 表空间。

```
    DROP TABLESPACE bookshop;
```

16.1.4 修改表空间名

可修改已存在的由用户创建的表空间的名称。比如可修改 bookshop 表空间名为

books。

```
    ALTER TABLESPACE bookshop RENAME TO books;
```

16.1.5 修改表空间状态

用户表空间有联机和脱机两种状态。系统表空间、回滚表空间、重做日志表空间和临时文件表空间不允许脱机。设置表空间状态为脱机状态时，如果该表空间有未提交的事务，则脱机失败报错。脱机后可对表空间的数据进行备份。例如修改 bookshop 表空间状态为脱机。

```
    ALTER TABLESPACE bookshop OFFLINE;
```

修改 bookshop 表空间状态为联机。

```
    ALTER TABLESPACE bookshop ONLINE;
```

注意：MPP 环境下，可能发现节点间的表空间不一致情况，如：EP01 为 ONLINE 状态，EP02 为 OFFLINE 状态，这个时候，如论执行 ONLINE 还是 OFFLINE 都是报错。需要用户介入，才可以解决问题。用户 LOCAL 方式登陆实例，并执行 SP_SET_SESSION_LOCAL_TYPE(1)，使得该会话可以执行 DDL 操作，再执行 ONLINE 或者 OFFLINE 即可。

16.1.6 修改表空间数据缓冲区

用户表空间可以切换使用的数据缓冲区，系统表空间、回滚表空间、重做日志表空间和临时文件表空间不允许修改数据缓冲区。可以使用的数据缓冲区有 NORMAL 和 KEEP。表空间修改成功后，并不会立即生效，而是需要服务器重启。缓冲池名 KEEP 是达梦的保留关键字，使用时必须加双引号。例如将 bookshop 表空间绑定到 KEEP 缓冲区。

```
    ALTER TABLESPACE bookshop CACHE= "KEEP";
```

16.1.7 查询表空间与数据文件对应关系

可以通过查询动态视图 V$TABLESPACE 得到系统中所有表空间的信息，通过查询动态视图 V$DATAFILE 得到系统中所有数据文件的信息。将两个动态视图以表空间 ID 为连接条件，通过以下查询可以得到表空间上对应的数据文件。

```
    SELECT ts.NAME, df.PATH FROM V$TABLESPACE AS ts, V$DATAFILE AS df WHERE ts.ID
= df.GROUP_ID;
```

16.1.8 表空间文件失效检查

LINUX 操作系统中，被进程打开的文件仍可以在 OS 系统中被删除，因此存在 DM7 数据文件可能被误删的风险。如果数据文件被删除，DM7 系统能够及时检测出来，并立刻停止对其继续使用并通知用户。

在 dm.ini 中参数 FIL_CHECK_INTERVAL 的值指定 DM7 系统检查数据文件是否仍存在的时间间隔。将其设为 0 表示不进行检查。

也可以通过系统过程 SP_FILE_SYS_CHECK()来手动的进行检查。

系统一旦检测出某个表空间内的数据文件被删除，则与该表空间所有的操作都将会失败，并报错该表空间内有数据文件被删除。

16.1.9 表空间失效文件恢复

LINUX 系统中被删除的文件，只要其句柄没有被关闭，可以在/proc/<pid>/fd 中找到其对应的文件副本。其中<pid>指打开该文件的进程 id。

利用该方法，结合 OS 命令，DM7 提供失效文件的恢复方案如下：

1、 调用系统过程 SP_TABLESPACE_PREPARE_RECOVER(tablespace_name)准备进行恢复；

2、 如果使用过程中 DM 报错表空间数据文件被删除，通过操作系统的 ps 命令找到当前 dmserver 的 PID：ps –ef|grepdmserver；

3、 使用操作系统 ls 命令查看被删除文件对应的副本：ls /proc/<PID>/fd–l，会发现被删除的文件后有(deleted)字样；

4、 使用操作系统的 cp 命令将文件复制到原位置 cpbak_fildata_file_path_dir；

5、 复制成功后，调用系统过程 SP_TABLESPACE_RECOVER(ts_name)完成表空间失效文件的恢复。

注意，要保证数据文件正确修复，需要保证在 SP_TABLESPACE_PREPARE_RECOVER 后进行数据文件的复制。

16.2 管理数据文件

管理数据文件的操作需要 DM 服务器处于打开状态下。

16.2.1 添加数据文件

可以在用户表空间中添加数据文件。添加的数据文件大小最小为 4096*页大小，如页大小为 8K，则可添加的文件最小值为 4096*8k=32M。比如在 bookshop 表空间中添加大小为 64M 的数据文件。

```
ALTER TABLESPACE bookshop ADD DATAFILE 'd:\book.dbf' SIZE 64;
```

一个表空间中，数据文件和镜像文件一起不能超过 256 个。例如，如果创建表空间的时候已经指定了 1 个数据文件，那么添加数据文件的时候，最多只能添加 255 个了。

16.2.2 扩展数据文件的大小

可以扩展用户表空间中已存在的数据文件的大小。比如扩展 bookshop 表空间中数据文件 book.dbf 大小至 128M。

```
ALTER TABLESPACE bookshop RESIZE DATAFILE 'd:\book.dbf' TO 128;
```

16.2.3 指定数据文件的扩展属性

可以指定数据文件是否可以扩展，每次扩展的空间大小以及数据文件可扩展到的最大空间大小，子句的语法为：

```
AUTOEXTEND OFF|ON [NEXT <文件扩展大小>] [MAXSIZE <文件限制大小>]
```

OFF 表示文件不可扩展，ON 表示文件可扩展。文件扩展大小表示当需要扩展文件时，文件一次增大的空间大小，取值范围是 0—2048，单位是 M。文件限制大小表示文件可扩展

的最大空间大小，为 0 或者 UNLIMITED 表示无限制，单位是 M。缺省情况下，文件扩展大小是 1M，文件的最大大小是无限制的。

创建表空间时可指定文件的扩展属性。如创建表空间时指定数据文件的扩展属性为可自动扩展，每次扩展大小为 10M，最大可扩展到 100M：

```
    CREATE TABLESPACE bookshop DATAFILE 'd:\book.dbf' SIZE 32 AUTOEXTEND ON NEXT
10 MAXSIZE 100;
```

在表空间中添加文件时可指定文件的扩展属性。如添加数据文件时指定扩展属性为不可自动扩展：

```
    ALTER TABLESPACE bookshop ADD DATAFILE 'd:\book.dbf' SIZE 1024 AUTOEXTEND OFF;
```

可修改表空间中已存在的数据文件的扩展属性。如修改数据文件的扩展属性为可自动扩展：

```
    ALTER TABLESPACE bookshop DATAFILE 'd:\book.dbf'  AUTOEXTEND ON;
```

16.2.4 修改数据文件的路径

可以修改用户表空间中已存在数据文件的路径，待修改的数据文件所在表空间必须处于脱机状态并且只可修改用户创建的表空间中文件的路径。如修改 bookshop 表空间中文件 book.dbf 的路径为 e:\ book.dbf。

```
    ALTER TABLESPACE bookshop RENAME DATAFILE 'd:\book.dbf' TO 'e:\book.dbf';
```

16.3 管理重做日志文件

16.3.1 添加重做日志文件

在服务器打开状态下，可以添加新的重做日志文件。添加的数据文件大小最小为 4096*页大小，如页大小为 8K，则可添加的文件最小值为 4096*8k=32M。如添加重做大小为 128M 的重做日志文件 DAMENG03.log。

```
    ALTER DATABASE ADD LOGFILE 'd:\DAMENG03.log' size 128;
```

16.3.2 扩展重做日志文件

在服务器打开状态下，可以扩展已有的重做日志文件的大小。如扩展重做日志文件 DAMENG03.log 到 256M。

```
    ALTER DATABASE RESIZE LOGFILE 'd:\DAMENG03.log' to 256;
```

16.4 管理回滚空间

回滚空间的管理和用户表空间的管理基本是一样的，区别是回滚空间的空间名固定为 ROLL，不可修改。可增加和扩展回滚空间中的回滚文件，设置回滚空间的扩展属性，相关操作可参考 16.2 管理数据文件中的说明。

回滚文件的路径记录在控制文件里面，可以使用 dmctlcvt 工具在 DM 服务器关闭的状态下对控制文件进行修改。使用 dmctlcvt 工具将控制文件转换为文本文件，编辑文本文件中要修改的文件的路径后再使用 dmctlcvt 工具将文本文件转换为控制文件即可。

首先转换控制文件到文本文件：

```
dmctlcvt  c2t D:\dm.ctl D:\ctl.txt
```

编辑 ctl.txt 文本文件中 fil_path=d:\roll.dbf 为 fil_path=e:\ roll.dbf，保存文本文件。复制 d:\roll.dbf 文件为 e:\ roll.dbf。

最后转换文本文件到控制文件：

```
dmctlcvt t2c D:\ctl.txt D:\dm.ctl
```

这种修改文件路径的方法也可用于重做日志文件，系统表空间文件等路径的修改。

16.5 管理控制文件

可以在 dm.ini 中通过设置 CTL_PATH 配置参数的值来指定控制文件的路径，缺省控制文件 dm.ctl 在数据目录下。例如可以把 dm.ctl 文件复制到 D 盘下，同时修改 dm.ini 中 CTL_PATH = D:\dm.ctl。

第17章 管理分区表和分区索引

在大型的企业应用或企业级的数据库应用中，要处理的数据量通常达到 TB 级，对于这样的大型表执行全表扫描或者 DML 操作时，效率是非常低的。

为了提高数据库在大数据量读写操作和查询时的效率，达梦数据库 DM7 提供了对表和索引进行分区的技术，把表和索引等数据库对象中的数据分割成小的单位，分别存放在一个个单独的段中，用户对表的访问转化为对较小段的访问，以改善大型应用系统的性能。

DM7 提供了水平和垂直分区两种分区方式。水平分区包括范围、哈希和列表三种方式，企业可以使用合适的分区方法，如日期（范围）、区域（列表），对大量数据进行分区。由于 DM7 划分的分区是相互独立且可以存储于不同的存储介质上的，完全可满足企业高可用性、均衡 IO、降低维护成本、提高查询性能的要求。

17.1 分区的概念

分区是指将表、索引等数据库对象划分为较小的可管理片段的技术，每一个片段称为分区子表或分区索引。一个表被分区后，对表的查询操作可以局限于某个分区进行，而不是整个表，这样可以大大提高查询速度。

DM7 支持对大表进行水平或垂直划分。例如，通讯公司将用户通话记录保存在一张表中，一年这个表产生 40GB 的数据。假设要对用户的通话信息按照季度进行统计，那么这样的统计需要在全表范围内进行。如果对表按季度进行水平分区，那么每个分区的大小平均为 10GB 左右，这样在进行统计时，只需在 10GB 的范围内进行即可。然而，假设现在只需统计用户通话的某些信息，通话信息表已经进行了垂直分区，并且需要统计的信息都在同一个垂直分区中，那么只需在那个分区中进行分区扫描即可，扫描范围也可大大减少。

DM7 采用子表方式创建分区表，分区表作为分区主表，而每一个分区以一个子表实体存在，即每一个分区都是一个完整的表，一般命名为主表名_分区名。对于水平分区，子表跟主表具有相同的逻辑结构，即分区子表与分区主表有相同的列定义和约束定义。对于垂直分区表，子表上的列是主表上列的子集。在 DM7 分区表中，主表本身不存储数据，所有数据只存储在子表中，从而实现不同分区的完全独立性。水平分区子表删除后，会将子表上的数据一起删除。

由于每一个分区都以一个子表作为实体，那么不同分区可以存储于相同表空间，也可以位于不同的表空间中。将这些分区放在不同的表空间中具有以下的好处：

1. 减少所有数据都损坏的可能性，一个表空间损坏不影响其他表空间，提高可用性；
2. 恢复时间大大减少；
3. 可以将同一个表中的数据分布在不同的磁盘上，从而均衡磁盘上的 I/O 操作；
4. 提高了表的可管理性、可利用性和访问效率。

分区操作对现存的应用和运行在分区表上的标准 DML 语句来说是透明的。但是，可以通过在 DML 中使用分区子表名字来对应用进行编程，使其充分利用分区的优点。

17.2 分区的方法

达梦数据库 DM7 支持对表进行水平分区和垂直分区。对于水平分区，提供以下分区方式：

1. 范围(range)水平分区：对表中的某些列上值的范围进行分区，根据某个值的范围，决定将该数据存储在哪个分区上；
2. 哈希（hash）水平分区：通过指定分区编号来均匀分布数据的一种分区类型，通过在 I/O 设备上进行散列分区，使得这些分区大小基本一致；
3. 列表（list）水平分区：通过指定表中的某个列的离散值集，来确定应当存储在一起的数据。例如，可以对表上的 status 列的值在('A'，'H'，'O')放在一个分区，值在('B'，'I'，'P')放在另一个分区，以此类推；
4. 组合分区表：按上述三种分区方区进行任意组合，将表进行多次分区，称为组合分区表。

而垂直分区就是将表按列分拆为多个分区，每个分区子表包含较少的列。如果查询表上的某些列，并且这些列都在一个分区中，那么只需扫描这个分区即可，可以减少 I/O 量。

17.3 创建水平分区表

在创建表的语法中，使用 partition 子句指定分区方式和分区列，以及分区的名字等信息，即可创建分区表。而分区子表可以指定 storage 子句，设置子表的存储属性，如所属表空间等；如果不指定，则继承分区主表的存储特性及表的其他属性。同时，支持组合分区表。

水平分区表的 ROWID 与其主表属性一致：LIST 表的水平分区表的 ROWID 是物理的；普通表的水平分区表的 ROWID 是逻辑的，且每个子表的 ROWID 都是从 1 开始增长，但是最终返回前，ROWID 的高字节会补充上子表序号。

17.3.1 创建范围分区表

范围分区是按照某个列或几个列的值的范围来创建分区，当用户向表中写入数据时，数据库服务器将按照这些列上的值进行判断，将数据写入相应的分区中。

在创建范围分区时，首先要指定分区列，即按照哪些列进行分区，然后为每个分区指定数据范围。范围分区支持 MAXVALUE 范围值的使用，MAXVALUE 相当于一个比任何值都大的值。范围分区非常适用于数据按时间范围组织的表，不同的时间段的数据属于不同的分区。

例如，以下语句创建一个范围分区表 callinfo，用来记录用户的 2010 年的电话通讯信息，包括主叫号码、被叫号码、通话时间和时长，并且根据季度进行分区。

```
CREATE  TABLE  callinfo(
caller  CHAR(15),
callee  CHAR(15),
time        DATETIME,
duration    INT
)
PARTITION BY RANGE(time)(
```

```
    PARTITION p1 VALUES LESS THAN ('2010-04-01'),
    PARTITION p2 VALUES LESS THAN ('2010-07-01'),
    PARTITION p3 VALUES LESS THAN ('2010-10-01'),
    PARTITION p4 VALUES EQU OR LESS THAN ('2010-12-31')  --'2010-12-31'也可替换为MAXVALUE
    );
```

值得注意的是，MAXVALUE 之间无法比较大小。如下所示：

```
    create table callinfo (
    caller  CHAR(15),
    callee  CHAR(15),
    time        DATETIME,
    duration    INT
    )
    partition by range(caller, callee)
            (
            partition p1 values less than ('a', 'b'),
            partition p2 values less than (maxvalue, 'd'),
            partition p3 values less than (maxvalue,maxvalue)
                );     ----报“范围分区值非递增”错误，建分区表失败
```

在创建分区表时，首先通过“PARTITION BY RANGE”子句指定分区的类型为范围分区，然后在这个子句之后指定一个或多个列作为分区列，如 callinfo 的 time 字段。

表中的每个分区都可以通过“PARTITION”子句指定一个名称。并且每一个分区都有一个范围，通过“VALUES LESS THAN”子句可以指定上界，而它的下界是前一个分区的上界。如分区 p2 的 time 字段取值范围是['2010-04-01', '2010-07-01')。如果通过“VALUES EQU OR LESS THAN”指定上界，即该分区包含上界值，如分区 p4 的 time 字段取值范围是['2010-10-01', '2010-12-31']。另外，可以对每一个分区指定 storage 子句，不同分区可存储在不同表空间中。

如果分区表包含多个分区列，采用多列比较方式定位匹配分区。首先比较第一个分区列值，如果无法确定目标分区，则继续比较后续分区列值，直到确定目标分区为止。匹配过程参看表 17.1。

表 17.1 多分区列匹配

插入记录	分区范围值		
	(10,10,10)	(20,20,20)	(30,30,30)
(5,100,200)	满足		
(10,10,11)		满足	
(31,1,1)	不满足	不满足	不满足

当在分区表中执行 DML 操作时，实际上是在各个分区子表上透明地修改数据。当执行 SELECT 命令时，可以指定查询某个分区上的数据。例如，以下语句查询 callinfo 表中分区 p1 的数据。

```
    SELECT * FROM callinfo PARTITION (p1);
```

17.3.2 创建 LIST 分区表

范围分区是按照某个列上的数据范围进行分区的，如果某个列上的数据无法通过划分范围的方法进行分区，并且该列上的数据是相对固定的一些值，可以考虑使用 LIST 分区。一般来说，对于数字型或者日期型的数据，适合采用范围分区的方法；而对于字符型数据，取值比较固定的，则适合于采用 LIST 分区的方法。

例如，创建一个产品销售记录表 sales，记录产品的销量情况。由于产品只在几个固定的城市销售，所以可以按照销售城市对该表进行分区。

```
CREATE  TABLE  sales(
sales_id    INT,
saleman CHAR(20),
saledate    DATETIME,
city        CHAR(10)
)
PARTITION BY LIST(city)(
PARTITION p1 VALUES ('北京', '天津'),
PARTITION p2 VALUES ('上海', '南京', '杭州'),
PARTITION p3 VALUES ('武汉', '长沙'),
PARTITION p4 VALUES ('广州', '深圳')
);
```

在创建 LIST 分区时，通过“PARTITION BY LIST”子句指定对表进行 LIST 分区，然后在每个分区中分区列的取值通过 VALUES 子句指定。当用户向表插入数据时，只要分区列的数据与 VALUES 子句指定的数据之一相等，该行数据便会写入相应的分区子表中。

注意的是，LIST 分区的分区键必须唯一。

17.3.3 创建哈希分区表

在很多情况下，用户无法预测某个列上的数据变化范围，因而无法实现创建固定数量的范围分区或 LIST 分区。

在这种情况下，DM7 哈希分区提供了一种在指定数量的分区中均等地划分数据的方法，基于分区键的散列值将行映射到分区中。当用户向表中写入数据时，数据库服务器将根据一个哈希函数对数据进行计算，把数据均匀地分布在各个分区中。在哈希分区中，用户无法预测数据将被写入哪个分区中。

现在重新考虑产品销售表的例子。如果销售城市不是相对固定的，而是遍布全国各地，这时很难对表进行 LIST 分区。如果为该表进行哈希分区，可以很好地解决这个问题。

```
CREATE  TABLE  sales01(
sales_id    INT,
saleman CHAR(20),
saledate    DATETIME,
city        CHAR(10)
)
PARTITION BY HASH(city)(
PARTITION p1,
```

```
PARTITION p2,
PARTITION p3,
PARTITION p4
);
```

如果不需指定分区表名，可以通过指定哈希分区个数来建立哈希分区表。

```
CREATE  TABLE  sales02(
sales_id    INT,
saleman CHAR(20),
saledate    DATETIME,
city        CHAR(10)
)
PARTITION BY HASH(city)
PARTITIONS 4 STORE IN (ts1, ts2, ts3, ts4);
```

PARTITIONS 后的数字表示哈希分区的分区数，STORE IN 子句中指定了哈希分区依次使用的表空间。使用这种方式建立的哈希分区表分区名是匿名的，DM7 统一使用 DMHASHPART+分区号（从 0 开始）作为分区名。例如，需要查询 sales 第一个分区的数据，可执行以下语句：

```
SELECT * FROM sales02 PARTITION (dmhashpart0);
```

17.3.4 创建组合分区表

在很多情况下，经过一次分区并不能精确地对数据进分类，这时需要组合分区表。

例如，创建一个产品销售记录表 sales，记录产品的销量情况。由于产品需要按地点和销售时间进行统计，则可以对该表进行 LIST-RANGE 分区。

```
DROP TABLE SALES;
CREATE  TABLE  SALES(
SALES_ID     INT,
SALEMAN CHAR(20),
SALEDATE    DATETIME,
CITY          CHAR(10)
)
PARTITION BY LIST(CITY)
 SUBPARTITION BY RANGE(SALEDATE) SUBPARTITION TEMPLATE(
    SUBPARTITION P11 VALUES LESS THAN ('2012-04-01'),
    SUBPARTITION P12 VALUES LESS THAN ('2012-07-01'),
    SUBPARTITION P13 VALUES LESS THAN ('2012-10-01'),
    SUBPARTITION P14 VALUES EQU OR LESS THAN (MAXVALUE))
(
    PARTITION P1 VALUES ('北京', '天津')
    (
      SUBPARTITION P11_1 VALUES LESS THAN ('2012-10-01'),
      SUBPARTITION P12_2 VALUES EQU OR LESS THAN (MAXVALUE)
    ),
    PARTITION P2 VALUES ('上海', '南京', '杭州'),
```

```
        PARTITION P3 VALUES (DEFAULT)
    );
```

在创建组合分区表时，指定了子分区模板，同时子分区 P1 自定义了子分区描述 P11_1 和 P11_2。P1 有两个子分区 P11_1 和 P11_2。而子分区 P2 和 P3 有四个子分区 P11、P12、P13 和 P14。

DM 支持最多八层组合分区。

下面给出一个三级分区的例子，更多级别的分区表的建表语句语法类推。

```
    CREATE TABLE STUDENT(NAME VARCHAR(20), AGE INT, SEX VARCHAR(10) CHECK (SEX
IN ('MAIL','FEMAIL')), GRADE INT CHECK (GRADE IN (7,8,9)))
    PARTITION BY LIST(GRADE)
      SUBPARTITION BY LIST(SEX) SUBPARTITION TEMPLATE
      (
        SUBPARTITION Q1 VALUES('MAIL'),
        SUBPARTITION Q2 VALUES('FEMAIL')
      ),
      SUBPARTITION BY RANGE(AGE) SUBPARTITION TEMPLATE
      (
        SUBPARTITION R1 VALUES LESS THAN (12),
        SUBPARTITION R2 VALUES LESS THAN (15),
        SUBPARTITION R3 VALUES LESS THAN (MAXVALUE)
       )
    (
      PARTITION P1 VALUES (7),
      PARTITION P2 VALUES (8),
      PARTITION P3 VALUES (9)
    );
```

17.4 在水平分区表建立索引

DM7 支持对水平分区表建立普通索引、唯一索引、聚集索引和函数索引。创建索引时若未指定 GLOBAL 关键字则建立的索引是局部索引，即每一个表分区都有一个索引分区，并且只索引该分区上的数据。如果指定了 GLOBAL 关键字则建立的索引是全局索引，即每个表分区的数据都被索引在同一个 B 树中。目前，仅堆表的水平分区表支持 GLOBAL 全局索引。堆表上的 primary key 会自动变为全局索引。

例如，在 sales 表上的 saledate 上建立索引。

```
    CREATE INDEX ind_sales_saldate ON sales(saledate);
    CREATE UNIQUE INDEX ind_sales_city ON sales(city);
```

对于非全局索引而言，建立分区索引后，每一个分区子表都会建立一个索引分区，负责索引分区子表的数据。由于每个索引分区只负责索引本分区上的数据，其他分区上的数据无法维护。因此，当对水平分区表建立非全局唯一索引时，只能建立分区键索引，即分区键必须都包含在索引键中。只有当分区键都包含在索引键中，才能对分区主表保证索引键唯一。全局唯一索引不受此约束。

另外，不能在水平分区表上建立局部唯一函数索引。

17.5 维护水平分区表

创建水平分区表后，DM7 提供了对分区表的修改，功能包括：

1. 增加分区：建立水平分区表后，可根据实际需要新增一个分区；
2. 删除分区：建立水平分区表后，可根据实际需要删除一个分区；
3. 合并分区：将相邻的两个范围分区合并为一个分区。合并分区通过指定两个分区名进行，将相邻的两个分区的数据进行合并，构建新的大分区。只能在范围分区上进行合并分区；
4. 拆分分区：将某一个范围分区拆分为相邻的两个分区。拆分分区时指定的常量表达式值必须是原范围分区的有效范围值。只能在范围分区上进行拆分分区；
5. 交换分区：将分区数据跟普通表数据交换功能，普通表必须跟分区表同构（拥有相同的列和索引）。不支持含有加密列的分区表交换分区。

在 DM7 中，由于局部索引反映基础表的结构，因此当对表的分区和子分区进行修改操作时，会自动地对局部索引进行相应的修改。

17.5.1 增加分区

DM7 支持用 ALTER TABLE ADD PARTITION 语句将新分区增加到最后一个现存分区的后面。例如，范围分区表 callinfo 现需要记录用户的 2011 年的第一季度的通讯信息，那么，需要为 2011 年第一季度增加一个分区，并将其存储在表空间 ts5 中。

```
ALTER TABLE callinfo
ADD PARTITION p5 VALUES LESS THAN ('2011-4-1') STORAGE (ON ts5);
```

对于范围分区，增加分区必须在最后一个分区范围值的后面添加，要想在表的开始范围或中间增加分区，应使用 SPLIT PARTITION 语句。

对于 LIST 分区，增加分区包含的离散值不能已存在于某个分区中。例如，为 LIST 分区表 sales 添加一个分区管理拉萨和呼和浩特的销售情况。

```
ALTER TABLE sales
ADD PARTITION p5 VALUES ('拉萨', '呼和浩特') STORAGE (ON ts5);
```

只能对范围分区和 LIST 分区增加分区，不能对哈希分区增加分区。并且增加分区不会影响分区索引，因为分区索引只是局部索引，新增分区仅是新增分区子表，并更新分区主表的分区信息，其他分区并不发生改变。

17.5.2 删除分区

DM7 支持用 ALTER TABLE DROP PARTITION 语句将分区删除。例如，范围分区表 callinfo 现需要删除记录用户的 2011 年的第一季度的通讯信息，那么，只需删除 callinfo 的分区 p1 即可。

```
ALTER TABLE callinfo DROP PARTITION p1;
```

只能对范围分区和 LIST 分区进行删除分区，哈希分区不支持删除分区。跟增加分区一样，删除分区不会影响分区索引，因为分区索引只是局部索引，删除分区仅是删除分区子表，并更新分区主表的分区信息，其他分区并不发生改变。

17.5.3 交换分区

假设上文提到的 callinfo 表是用于维护最近 12 个月的用户通话信息，超过 12 个月的订单需要迁移到该季度的通话信息历史表中，并且每一个季度都有一个相应的历史表。如果没有使用水平分区，需要较多的删除和插入操作，并产生大量的 redo 和 undo 日志。

如果使用分区表，如上文提到的 callinfo，只需使用交换分区即可完成以上功能。例如，2011 年第二季度已到了，需删除 2010 年第二季度的通话记录，因此，可通过以下脚本来实现：

```
CREATE  TABLE  callinfo_2011Q2(
caller  CHAR(15),
callee  CHAR(15),
time        DATETIME,
duration    INT
);
--交换分区
ALTER TABLE callinfo EXCHANGE PARTITION p2 WITH TABLE callinfo_2011Q2;
--删除原分区
ALTER TABLE callinfo DROP PARTITION p2;
--新增分区，记录 2011 年第二季度通话记录
ALTER TABLE callinfo
ADD PARTITION p6 VALUES LESS THAN ('2011-7-1') STORAGE (ON ts2);
```

通过交换分区实现分区 p2 和新建表 callinfo_2011Q2 的数据交换，表 callinfo_2011Q2 将得到 2010 年第二季度的通话记录，而分区 p2 数据将被清空。交换分区采用数据字典信息交换的技术，几乎不涉及 IO 操作，因此效率非常高。

仅范围分区和 LIST 分区支持交换分区，哈希分区表不支持。并且分区交换要求分区表跟交换表具有相同的结构（相同的列和索引，相同的索引定义顺序，不支持加密列），分区交换但并不会校验数据，如交换表的数据是否符合分区范围等，即不能保证分区交换后的分区上的数据符合分区范围。

17.5.4 合并分区

要想将两个范围分区的内容融合到一个分区，就要使用 ALTER TABLE MERGE PARTITION 语句。如果分区的数据很少，或相对其他分区某些分区的数据量较少，导致 I/O 不均衡，就可以考虑使用合并分区。

例如，可将 callinfo 的 2010 第 3 季度和第 4 季度合并成一个分区：

```
ALTER TABLE callinfo MERGE PARTITIONS p3, p4 into partition p3_4;
```

仅范围分区表支持合并分区，并且合并的分区必须是范围相邻的两分区。另外，合并的分区会导致数据的重组和分区索引的重建，因此，合并分区可能会比较耗时，所需时间取决于分区数据量的大小。

17.5.5 拆分分区

ALTER TABLE 语句的 SPLIT PARTITION 子句被用于将一分区中的内容重新划分成两个新的分区。当一个分区变得太大以至于要用很长时间才能完成备份、恢复或维护操作时，就应考虑做分割分区的工作，还可以用 SPLIT PARTITION 子句来重新划分 I/O 负载。

例如，将合并后的 p3_4 拆分为原两分区 p3 和 p4，分别记录 2010 年第三和第四季度的通话记录。

```
ALTER TABLE callinfo SPLIT PARTITION p3_4 AT ('2010-9-30') INTO (PARTITION p3, PARTITION p4);
```

需要注意的是，仅范围分区表支持拆分分区，并且拆分的分区值会落在低分区中。拆分分区另一个重要用途是作为新增分区的补充。通过拆分分区，可以对范围分区表的开始或中间范围添加分区。

另外，拆分分区会导致数据的重组和分区索引的重建，因此，拆分分区可能会比较耗时，所需时间取决于分区数据量的大小。

17.6 水平分区表的限制

DM7 水平分区表有如下限制条件：

1. 分区列类型必须是数值型、字符型或日期型，不支持 BLOB、CLOB、IMAGE、TEXT、LONGVARCHAR、BIT、BINARY、VARBINARY、LONGVARBINARY、时间间隔类型和用户自定义类型为分区列；
2. 范围分区和哈希分区的分区键可以多个，最多不超过 16 列；LIST 分区的分区键必须唯一；
3. 水平分区表指定主键和唯一约束时，分区键必须都包含在主键和唯一约束中；
4. 水平分区表不支持临时表；
5. 不能在水平分区表上建立自引用约束；
6. 普通环境中，水平分区表的各级分区数的总和上限是 65535；MPP 环境下，水平分区表的各级分区总数上限取决于 INI 参数 MAX_EP_SITES，上限为 2 ^(16 - $\log_2$MAX_EP_SITES)。比如：当 MAX_EP_SITES 为默认值 64 时，分区总数上限为 1024；
7. 不允许对分区子表执行任何 DDL 操作；
8. 哈希分区支持重命名、删除约束、设置触发器是否启用的修改操作；
9. 范围分区支持分区合并、拆分、增加、删除、交换、重命名、删除约束、设置触发器是否生效操作；
10. LIST 分区支持分区增加、删除、交换、重命名、删除约束、设置触发器是否生效操作；
11. 对范围分区增加分区值必须是递增的，即只能在最后一个分区后添加分区。LIST 分区增加分区值不能存在于其他已存在分区；
12. 当分区数仅剩一个时，不允许删除分区；
13. 仅能对相邻的范围分区进行合并，合并后的分区名可为高分区名或新分区名；
14. 拆分分区的分区值必须在原分区范围中，并且分区名不能跟已有分区名相同。拆分分区值落在低分区中；

15. 与分区进行分区交换的普通表，必须与分区表拥有相同的列及索引，但交换分区并不会对数据进行校验，即交换后的数据并不能保证数据完整性，如 CHECK 约束；分区表与普通表创建的索引顺序要求一致；
16. 水平分区表仅支持建立局部索引，不支持建立全局索引，即在分区表上建立索引，每一个表分区都有一个索引分区，并且只索引该分区上的数据，而分区主表上的索引并不索引数据；
17. 不能对水平分区表建立唯一函数索引和全文索引；
18. 不能对分区子表单独建立索引；
19. 在未指定 ENABLE ROW MOVEMENT 的分区表上执行更新分区键，不允许更新后数据发生跨分区的移动，即不能有行迁移；
20. 不能在分区语句的 STORAGE 子句中指定 BRANCH 选项；
21. 不允许引用水平分区子表作为外键约束；
22. 组合分区表最多支持八层；
23. 组合分区表仅支持下列修改表操作：新增单列、新增多列、删除列、删除约束、设置与删除列的默认值以及设置列 NULL 属性；
24. 水平分区表支持的列修改操作包括：新增单列、新增多列、删除列、修改列定义、修改列名称、设置列默认值、设置列 NULL 属性、撤销列默认值；
25. 水平分区表中包含大字段、自定义字段列，则定义时指定 ENABLE ROW MOVEMENT 参数无效，即不允许更新后数据发生跨分区的移动。
26. 间隔分区表的限制说明：
    1) 仅支持一级范围分区创建间隔分区。
    2) 只能有一个分区列，且分区列类型为日期或数值。
    3) 对间隔分区进行 SPLIT，只能在间隔范围内进行操作。
    4) 被 SPLIT\MERGE 的分区，其左侧分区不再进行自动创建。
    5) 不相邻的间隔的分区，不能 MERGE。
    6) 表定义不能包含 MAXVALUE 分区。
    7) 不允许新增分区。
    8) 不能删除起始间隔分区。
    9) 间隔分区表定义语句显示到起始间隔分区为止。
    10) 自动生成的间隔分区，均不包含边界值。
    11) 间隔表达式只能为常量或日期间隔函数。日期间隔函数为：NUMTOYMINTERVAL、NUMTODSINTERVAL；数值常量可以为整型、DEC 类型。

17.7 创建垂直分区表

DM7 垂直分区的主要目的是，使查询得以扫描较少的数据，减少 I/O 量，从而提高查询性能。例如，某个表包含七列，通常只引用该表的前三列，那么将该表的后四列拆分到一个单独的表中将有利于提高查询性能。

与水平分区相似，定义一个垂直分区表，系统将自动为其创建一个分区主表和若干个分区子表。分区主表不保存实际数据，只保存表定义、分区信息，实际数据保存在分区子表中。

每个分区子表包含定义的分区子表列和聚集索引列。并且，在所有的分区子表上建立了聚集索引。这样拆分后，每个子表只包含较少列数据，通过聚集索引列进行行匹配，各个分区子表中的每个逻辑行与其他分区子表的相同逻辑行匹配，可以方便的还原主表定义的完整

数据。

由于用户数据都存储在分区子表中，而子表对用户是透明的，因此，当用户需要获得分区主表所有列数据时，分区子表间会根据聚集索引键进行连接操作还原主表数据。而对主表的插入、更新和删除都将影响到相应的分区中。

如果查询只使用到垂直分区表的某些列，系统判断查询需要的数据是否只涉及某个分区子表，如果是，直接使用分区子表替代分区主表来进行查询优化。系统只需以较少的 I/O 代价，即可获得需要的数据。

以下是一个垂直分区表的例子：

```
CREATE TABLE orders(
    orderid     INT CLUSTER PRIMARY KEY,
    status          INT,
    price           DOUBLE,
    orderdate       DATETIME,
    customer        VARCHAR(20),
    comments        VARCHAR(256)
)
PARTITION BY COLUMN (
(orderid, status, price, orderdate) AS orders_p1 STORAGE (ON ts1),
(customer, comments) AS orders_p2 STORAGE (ON ts2)
);
```

订单表 orders 中经常访问的字段 orderid、status、price 和 orderdate 存放在垂直分区 orders_p1 中，其他字段存储在分区 orders_p2 中。orders 表中 orderid 是聚簇主键， orderid 虽然不是 orders_p2 的分区列，但也存储了聚簇主键 orderid 的副本，这样分区 orders_p1 和 orders_p2 就可通过 orderid 联系起来。

17.8 垂直分区表的限制

DM7 垂直分区表的限制如下：

1. 包含 IDENTITY 列的表不允许定义垂直分区表；
2. 垂直分区表不允许建立触发器；
3. 除 CLUSTER KEY 列外，其他任何列只能出现在一个分区中；
4. 垂直分区表上允许建立外键约束；
5. 垂直分区表上不允许建立除 UNIQUE 外的 CHECK 约束；
6. 不允许跨分区定义索引；
7. 一张垂直分区表最多允许有 32 个分区；
8. 禁止用户对垂直分区表子表的 INSERT/UPDATE/DELETE 直接操作，但允许直接 SELECT；
9. 禁止通过 ALTER TABLE 方式为垂直分区表增加 PK 约束；
10. ALTER TABLE 目前只支持为主表的列添加默认值、ALTER TABLE RENAME 表名（主表或子表）和 DROP CONSTRAINT，其余均报错返回；
11. 垂直分区表不支持建表的 BRANCH 选项；
12. 垂直分区表上视图不支持更新 CLUSTER KEY 列；
13. 垂直分区表中所有的大字段列必须位于同一个子表中；
14. 垂直分区表子表必须至少包含一个非 CLUSTER KEY 的列；

15. 垂直分区表在建表后不能更改聚集索引；如果建表时没有指定聚集索引，后续也不能再指定；
16. 垂直分区表上视图的 CHECK 约束不能跨分区，并且此时视图不能嵌套；
17. 垂直分区表不支持自引用约束；
18. 垂直分区表不支持 return into 功能。

第18章 管理列存储表

18.1 什么是列存储

达梦表数据存储方式分为行存储和列存储。行存储是以记录为单位进行存储的，数据页面中存储的是完整的若干条记录；列存储是以列为单位进行存储的，每一个列的所有行数据都存储在一起，一个段只存储一个列的数据，而且一个指定的页面中存储的都是某一个列的连续数据。

下面以基表（表 18.1）为例，对行、列存储进行对比。

表 18.1 基表

name 列	ID 列
SYSCLASSES	268435457
SYSCOLUMNS	268435458
SYSCONS	268435459
SYSDUAL	268435460
SYSGRANTS	268435461

如图 18.1 所示，列存储在具体存储实现方式上就是每一个列的数据存储在一个段中。

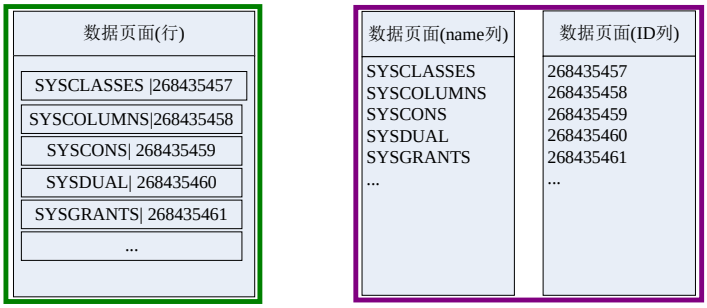


图 18.1 行存储（左）和列存储对比图

HFS（全称 Huge File System）是达梦数据库实现的，针对海量数据进行分析的一种高效、简单的列存储机制。HFS 表（也称为 HUGE 表）就是建立在 HFS 存储机制上的一种表。

18.2 什么是 HFS 表

HFS 表是建立在自己特有的表空间 HTS（HUGE TABLESPACE，即 HUGE 表空间）上的。最多可创建 32767 个 HUGE 表空间，其相关信息存储在动态视图 V$HUGE_TABLESPACE 中。

这个表空间与普通的表空间不同。普通的表空间，数据是通过段、簇、页来管理的，并且以固定大小（4K、8K、16K、32K）的页面为管理单位；而 HTS 相当于一个简单的文件系统，创建一个 HTS，其实就是创建一个空的目录（系统中有一个默认 HTS，目录名为 HMAIN）。在创建一个 HFS 表并插入数据时，数据库会在指定的 HTS 表空间目录下创建一系列的目录及文件，文件系统结构如下图所示：

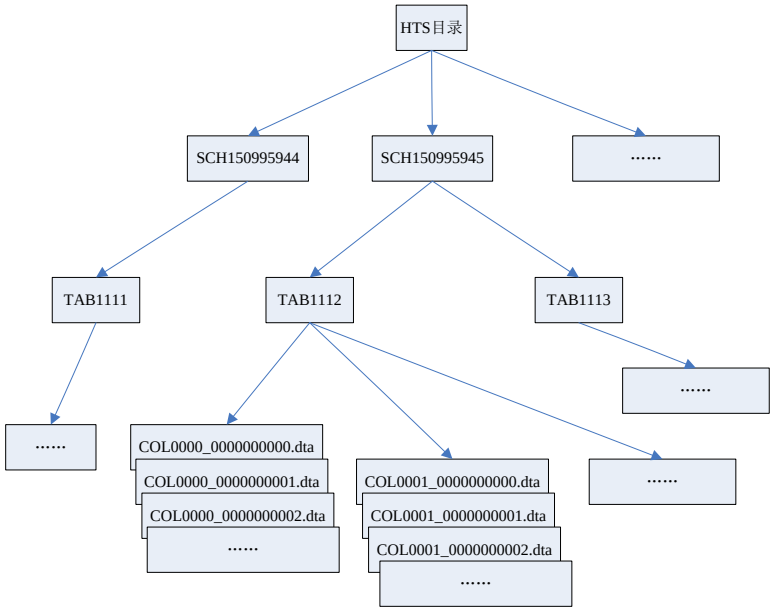


图 18.2 HFS 结构示意图

从上图可以看出，在 HTS 目录下成功创建 HFS 表，系统内部需要经过以下步骤：

首先，在 HTS 目录下创建这个表对应的模式目录。目录名为“SCH+长度为 9 的 ID 号”组成的字符串，比如：SCH150995944。创建时如果这个目录已经存在，则无需重新创建。

其次，在模式目录下创建对应的表目录。表目录也是同样的道理，表目录名为“TAB+长度为4的ID号”组成的字符串，比如：TAB1112。表目录中存放的是这个表中所有的文件。

再次，在新创建表后插入数据时，每一个列对应一个以dta为后缀的文件，文件大小可以在建表时指定，默认为64M，文件名为“COL+长度为4的列号+_+长度为10的文件号”。例如，在图20.2中，0000表示第1列，0001表示第2列……；0000000000表示第1个文件，0000000001表示第2个文件……。最初一个列只有一个文件即可，当随着数据量不断增长，一个文件已放不下之后，系统会自动创建新的文件来存储不断增长的数据。

对于一个文件，其内部存储是按照区来管理的，区是文件内部数据管理的最小单位，也是唯一的单位。一个区中，可以存储单列数据的行数是通过创建表时指定的，一经指定，在这个表的生命过程就不能再修改。所以，对于定长数据，一个区的大小是固定的；而对于变长数据，一般情况下区大小都是不相同的。一个区内，存储的仅仅是数据而已，对于 NULLABLE 属性的列，存储的还有相应的 NULL 标志。每一个区的开始位置及长度在文件内

都是 4K 对齐的。

对于一个 HFS 表，相应的还配备一个辅助表来管理其数据。因为在上面介绍的文件中只存储了数据，辅助表用来管理以及辅助系统用户操作这些数据，该辅助表是在创建 HFS 表时系统自动创建的，表名为“表名$AUX”，如果该 HFS 表为分区表，则辅助表名为“子表名$AUX”。辅助表的表名长度不能大于 128 个字节。辅助表中每一条记录对应文件中的一个数据区，辅助表包括下面 15 列：

1．COLID：表示当前这条记录对应的区所在的列的列 ID 号；

2．SEC_ID：表示当前这个记录对应的区的区 ID 号，每一个区都有一个 ID 号，并且唯一；

3．FILE_ID：表示这个区的数据所在的文件号；

4．OFFSET：表示这个区的数据在文件中的偏移位置，4K 对齐；

5．COUNT：表示这个区中存储的数据总数（有可能包括被删除的数据）；

6．ACOUNT：表示这个区中存储的实际数据行数；

7．N_LEN：表示这个区中存储的数据在文件中的长度，4K 对齐的；

8．N_NULL：表示这个区中的数据中包括的 NULL 值的行数；

9．N_DIST：表示这个区中所有数据互不相同的行数；

10．MAX_VAL：表示这个区中的最大值，精确值；

11．MIN_VAL：表示这个区中的最小值，精确值；

12．SUM_VAL：表示这个区中所有值的和，精确值；

13．CPR_FLAG：表示这个区是否压缩；

14．ENC_FLAG：表示这个区是否加密；

15．CHKSUM：用来较验的，该功能暂未启用。

前面 7 列是用来控制数据存取的，根据这些信息就可以知道这个区的具体存储位置、长度及基本信息。后面 8 列都是用来对这个区进行统计分析的。其中，COLID 和 SEC_ID 的组合键为辅助表的聚簇关键字。

HFS 表的存储方式有以下几个优点：

1． 同一个列的数据都是连续存储的，可以加快某一个列的数据查询速度；

2． 连续存储的列数据，具有更大的压缩单元和数据相似性，可以获得远优于行存储的压缩效率，压缩的单位是区；

3． 条件扫描借助数据区的统计信息进行精确过滤，可以进一步减少 IO，提高扫描效率；

4． 对 HFS 表进行修改时，不写日志，不通过 BUFFER 缓存，直接操纵文件，速度快；

5． 允许建立二级索引；

6． 支持以 ALTER TABLE 的方式添加或者删除 PK 和 UNIQUE 约束。

HFS 表拥有上述优点，当然它也有缺点：

1．HFS 表操作时封锁粒度较大，且不支持多版本并发控制；

2．HFS 表的插入、删除与更新操作处理都不能进行回滚；

3．建 HFS 表时仅支持定义 NULL、NOT NULL、UNIQUE 约束以及 PRIMARY KEY，后两种约束也可以通过 ALTER TABLE 的方式添加，但这两种约束不检查唯一性；

4．HFS 不允许建立一级索引，允许建立二级索引，不支持建位图索引，其中 UNIQUE 索引不检查唯一性；

5．HFS 表不支持事务，没有事务的特性；

6．不支持 SPACE LIMIT（空间限制）；

7．不支持建立全文索引；

8．不支持使用自定义类型；
9．不支持引用约束；
10．不支持 IDENTITY 自增列；
11．不支持大字段列；
12．不支持建触发器；
13．不允许垂直分区；
14．不支持游标的修改操作；
15．PK 和 UNIQUE 约束不检查唯一性，对应的索引都为虚索引；UNIQUE 索引也不检查唯一性，为实索引，索引标记中不包含唯一性标记，即和普通二级索引相同。

总之，HFS 表主要针对海量数据的扫描分析，并不适用于常规的联机事务处理。

18.3 创建 HFS 表

当用户确定自己要使用 HFS 表的时候，首先需要在模式中创建新表，创建一个 HFS 表需要有 CREATE TABLE 数据库权限，要想在其他用户的模式中创建新表需要有 CREATE ANY TABLE 数据库权限。

但是创建一个 HFS 表时，如果不使用默认的表空间，则必须要先创建一个 HUGE TABLESPACE（HTS），创建 HTS 语法如下：

```
CREATE HUGE TABLESPACE <表空间名> PATH <表空间路径>;
```

参数说明：

1．<表空间名> 表空间的名称，表空间名称最大长度 128 字节；

2．<表空间路径> 指明新生成的表空间在操作系统下的路径。

示例如下：

```
CREATE HUGE TABLESPACE HTS_NAME PATH 'e:\HTSSPACE';
```

在创建表时，可以指定表的存储属性，存储属性包括如下几个方面：

1．区大小（一个区的数据行数）

区大小可以通过设置表的存储属性来指定，区的大小必须是 2 的多少次方，如果不是则向上对齐。取值范围：1024 行~1024*1024 行。默认值为 65536 行。

例如，创建 HFS 表 test，指定区的大小为 2048，其它默认。

```
CREATE HUGE TABLE test(name VARCHAR, sno INT) STORAGE(SECTION (2048));
```

例如，创建 HFS 表 test，所有列都采用默认值。

```
CREATE HUGE TABLE test(name VARCHAR, sno INT);
```

2．是否记录区统计信息，即在插入时是否记下其最大值最小值

关于这一点有一个原则，如果这个列经常用作查询条件，并且数据不是很均匀，或者基本是有序的，那么做统计信息是非常有用的，反之则可以不做统计。缺省情况下，为记录区统计信息。如果想不记录，可通过设置 STAT NONE 实现。

例如，创建 HUGE 表 test，通过列存储属性指定统计信息属性（不记录区统计信息）。

```
CREATE HUGE TABLE test(name VARCHAR STORAGE (STAT NONE), sno INT);
```

又如，创建 HUGE 表 test，通过表存储属性指定统计信息属性（不记录区统计信息）。

```
CREATE HUGE TABLE test(name VARCHAR, sno INT) STORAGE (STAT NONE);
```

3．所属的表空间

创建 HFS 表，需要通过存储属性指定其所在的表空间，不指定则存储于默认表空间 HMAIN 中。HFS 表指定的表空间只能是 HTS 表空间，例如 HTS_NAME 为已指定 HTS 表空间。

```
CREATE HUGE TABLE test(name VARCHAR, sno INT) STORAGE (ON HTS_NAME)
```

4. 文件大小

创建 HFS 表时还可以指定单个文件的大小，通过表的存储属性来指定，取值范围为 16M~1024*1024M。不指定则默认为 64M。文件大小必须是 2 的多少次方，如果不是则向上对齐。

```
CREATE HUGE TABLE test(name VARCHAR, sno INT) STORAGE (filesize(64));
```

5. 指定压缩级别

为特定列指定压缩级别，在压缩速率和高压缩比之间设置级别值的范围 0~9。其中 2~9 值越小表示压缩速率越快，值越大表示压缩比越高。0，1 为快捷使用，默认值为 0，表示最大速率压缩，等价于 LEVEL2；设置成 1 时表示最高压缩比，即等价于 LEVEL 9；

例如，创建 HFS 表 test，指定 sno 列按照最大压缩比压缩。

```
CREATE HUGE TABLE test(name VARCHAR, sno INT) COMPRESS LEVEL 1 (sno);
```

6. 日志属性

HFS 表在操作过程中有可能会出现系统崩溃或者断电等问题，因为修改时采取的是直接写的策略，所以有可能会出现数据不一致的问题。为了保证数据的一致性，在操作时可以适当的做一些日志来保证数据的完整性，完整性保证策略主要是通过数据的镜像来实现的，镜像的不同程度可以实现不同程度的完整性恢复。镜像文件是放在表目录中的以.mir 为扩展名的文件。系统中主要提供三种方案：

1. LOG NONE：不做镜像。相当于不做数据一致性的保证，如果出错只能手动通过系统函数来修复表数据，当然速度是最快的，不需要额外的 IO，这种选项如果用户明确知道自己的环境不会出现问题可以采用，效率最高。
2. LOG LAST：做部分镜像。但是在任何时候都只对当前操作的区做镜像，如果当前区的操作完成了，那么这个镜像也就失效了，可能会被下一个被操作区覆盖，这样做的好处是镜像文件不会太大，同时也可以保证数据是完整的。但有可能遇到的问题是：一次操作很多的情况下，有可能一部分数据已经完成，另一部分数据还没有来得及做的问题。如果用户能接受这个问题的话这个选择不失为最佳选择，这也是系统默认的选择。
3. LOG ALL：全部做镜像。在操作过程中，所有被修改的区都会被记录下来，当一次操作修改的数据过多时，镜像文件有可能会很大，但好处是，能够保证操作完整性。比如，在操作过程中失败了，那么这个操作会完整的撤消，不存在上面一部分修改部分还没修改的问题。

   示例如下：

```
例：CREATE HUGE TABLE test(name VARCHAR, sno INT) LOG ALL;
```

下面是一个综合的创建 HFS 表的例子：

```
CREATE HUGE TABLE orders
(
    o_orderkey          INT,
    o_custkey           INT,
    o_orderstatus           CHAR(1),
    o_totalprice            FLOAT,
    o_orderdate         DATE,
    o_orderpriority         CHAR(15),
    o_clerk             CHAR(15),
    o_shippriority          INT,
```

```
        o_comment           VARCHAR(79) STORAGE(stat none)
    )STORAGE(section(65536) , filesize(64), on HTS_NAME) COMPRESS LEVEL 9 FOR
'QUERY HIGH'  (o_comment) log all;
```

在这个例子中，ORDERS 表的区大小为 65536 行，文件大小为 64M，指定所在的表空间为 HTS_NAME，做完整镜像，o_comment 列指定的区大小为不做统计信息，其它列（默认）都做统计信息，指定列 o_comment 列压缩类型为查询高压缩率，压缩级别为 9。

18.4 维护 HFS 表

HFS 表像行存储表一样，也可以做增删改查操作，操作方式是完全一样的，但删除及更新操作效率可能比行存储低一些，所以在 HFS 中不宜做频繁的删除及更新操作。因此，HFS 比较适合做分析型表的存储。

同时由于其没有事务性，在操作过程中如果崩溃或者断电时，其恢复的程度或者完整性是根据在创建表时指定的日志属性决定的。如果定义为 LOG NONE，有可能造成数据的不完整，需要手动修复；如果定义为 LOG LAST，有可能造成操作的不完整；如果定义为 LOG ALL，则不会有任何不完整的问题。

HUGE 表备份还原请参考《DM7_SQL 语言使用手册》附录 3。

18.5 查看有关 HFS 表的信息

1. 表定义

对一个 HFS 表，用户可以通过 CALL SP_TABLEDEF('SYSDBA', 'ORDERS');得到这个表的定义语句，可以具体了解表的各个列的数据类型信息、存储属性等，还可以查看在这个表上是否有压缩等等。

2. 数据存储情况

HFS 表有一个很好的特点就是有一个辅助表，辅助表中用户可以利用的信息很多，因为每一条记录对应一个区，所以可以查看每一个区的存储情况，每一个列的存储情况及每一个列中具有相同区 ID 的所有数据的情况等，还包括了很精确的统计信息，用户可以通过观察辅助表中的信息对表进行一些相应的操作。

第19章 管理堆表

19.1 什么是堆表

普通表都是以 B 树形式存放的，ROWID 都是逻辑的 ROWID，即从 1 一直增长下去。在并发情况下，每次插入过程中都需要逻辑生成 ROWID，这样影响了插入数据的效率；对于每一条数据都需要存储 ROWID 值，也会花费较大的存储空间。堆表就是基于上述两个理由而提出的。

简单地说，堆表是指采用了物理 ROWID 形式的表，即使用文件号、页号和页内偏移而得到 ROWID 值，这样就不需要存储 ROWID 值，可以节省空间。逻辑 ROWID 在插入或修改过程中，为了确保 ROWID 的唯一性，需要依次累加而得到值，这样就影响了效率，而堆表只需根据自己的文件号、页号和页内偏移就可以得到 ROWID，提高了效率。

普通表都是以 B 树形式而存储在物理磁盘上，而堆表则采用一种“扁平 B 树”方式存储，结构如图 19.1 所示。

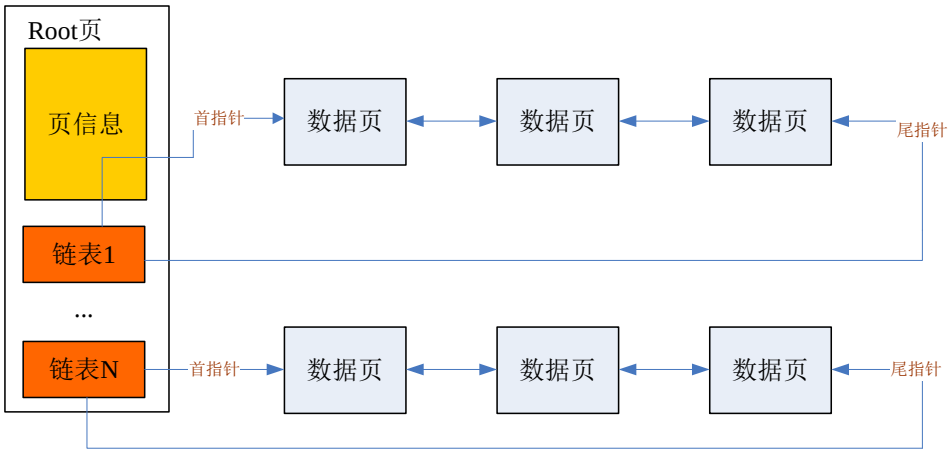


图 19.1 堆表存储方式

采用了物理 ROWID 形式的堆表，DM 服务器内部对聚集索引进行了调整，没有采用传统 B 树结构，取而代之的是“扁平 B 树”，数据页都是通过链表形式存储。为支持并发插入，扁平 B 树可以支持最多 128 个数据页链表（最多 64 个并发分支和最多 64 个非并发分支），在 B 树的控制页中记录了所有链表的首、尾页地址。对于非并发分支，如果分支数有多个，即存在多个链表，则不同的用户登录系统之后，会依据其事务 ID 号，随机选择一条链表来对堆表进行插入操作。对于并发分支，则不同用户会选择不同的分支来进行插入，如果存在多个用户选择了同一条分支的情况，才需要等待其他用户插入结束并释放锁之后才能进行插入。在并发情况下，不同用户可以在不同的链表上进行插入，效率得到较大提升。

19.2 创建堆表

堆表的创建有两种方式，一种是采用在配置文件 dm.ini 中设置参数，一种是在建表语句中显式指定堆表选项。

1. INI 参数方式

用户可以在配置文件中，添加 LIST_TABLE 参数：

1) 如果 LIST_TABLE = 1，则在未显式指定表是否为堆表或非堆表时，默认情况下创建的表为堆表；
2) 如果 LIST_TABLE = 0，则在未显式指定表是否为堆表或非堆表时，默认情况下创建的表为普通表形式。

2. SQL 语句显示指定

不管参数 LIST_TABLE 设置为何值，创建表时可以在 STORAGE 选项中指定需要创建的表形式， 与堆表创建形式相关的关键字有三个，分别是 NOBRANCH、BRANCH、CLUSTERBTR。详细语法形式参见《DM_SQL》。

1) NOBRANCH：如果指定为 NOBRANCH，则创建的表为堆表，并发分支个数为 0，非并发分支个数为 1；
2) BRANCH(n，m)：如果为该形式，则创建的表为堆表，并发分支个数为 n，非并发个数为 m；
3) BRANCH n：指定创建的表为堆表，并发分支个数为 n，非并发分支个数为 0；
4) CLUSTERBTR：创建的表为非堆表，即普通 B 树表。

如下例创建的 LIST_TABLE 表有并发分支 2 个，非并发分支 4 个。

```
CREATE TABLE LIST_TABLE(C1 INT) STORAGE(BRANCH (2,4));
```

19.3 堆表的限制

堆表由于其自身的特性，与普通表相比，也为自己增添了一些限制。堆表不支持以下功能：

1. 聚集索引

堆表采用了物理 ROWID，即通过文件号、页号和页内偏移直接生成该值。这样如果我们知道了 ROWID 值，也就知道文件号、页号和页内偏移这些变量，就可以直接定位到某条记录，所以没有必要再为堆表创建聚集索引了。在创建堆表时，系统会默认创建聚集索引，该索引只是一个根页信息。显式建立聚集索引是不允许的，如果用户需要借助聚集索引主键对数据进行排序则不推荐使用堆表。

2. 全文索引

DM 服务器暂时不支持此功能。

3. 列存储

由于列存储采用了不同方式对表进行物理存储， DM 服务器暂时不支持堆表的列存储。

19.4 维护堆表

堆表在进行数据扫描过程中，有着其先天的优势。如果知道了数据记录的 ROWID，则直接可以对 ROWID 进行解码，得到该记录的文件号、页号和页内偏移，也就得到了该记录。所以建议在经常查询的列上建立二级索引，这样在进行操作中，先通过二级索引找到记录 ROWID，就可以直接找到数据，效率有较大提高。

堆表虽然支持表的 ALTER 操作，但是建议轻易不要进行此类操作。对表进行 ALTER 操作，数据记录的 ROWID 有可能发生改变，这样每次进行 ALTER 操作，都可能进行索引的重建，需要花费较多的时间。

达梦服务器支持对堆表的备份与还原操作。还原数据时，B 树数据和二级索引可以同时被还原。

19.5 查看有关堆表的信息

可以通过系统过程SP_TABLEDEF(‘SCHEMA_NAME’,’TABLE_NAME’)查看堆表的定义信息，该函数的详细信息可参考 《DM_SQL》。

第20章 全文检索

现有的数据库系统，绝大多数是以结构化数据为检索的主要目标，因此实现相对简单。比如数值检索，可以建立一张排序好的索引表，这样速度可以得到提高。但对于非结构化数据，即全文数据，要想实现检索，一般都是采用模糊查询的方式实现的。这种方式不仅速度慢，而且容易将汉字错误切分，于是产生了全文检索技术。

全文检索技术是智能信息管理的关键技术之一，其主要目的就是实现对大容量的非结构化数据的快速查找。DM 实现了全文检索功能，并将其作为 DM 服务器的一个较独立的组件，提供更加准确的全文检索功能，较好地解决了模糊查询方式带来的问题。

20.1 全文检索概述

DM 全文检索根据已有词库建立全文索引，文本查询完全在索引上进行。全文索引为在字符串数据中进行复杂的词搜索提供了有效支持。

用户可以在指定表的文本列上创建和删除全文索引。创建全文索引后全文索引未插入任何索引信息。当用户填充全文索引时，系统才将定义了全文索引的文本列的内容进行分词，并根据分词结果填充索引。用户可以在进行全文索引填充的列上使用 CONTAINS 谓词进行全文检索。

DM7 全文索引改进了原有的分词算法，为全文检索提供了更好的基础。在创建全文索引成功后，假设索引名为 INDEX_NAME，则系统会自动产生如下相关的动态表：CTI$INDEX_NAME$I 和 CTI$INDEX_NAME$P，其表结构如 20.1 和 20.2 所示。第一个表主键为（WORD、FIRSTID、WID），用于保存分词结果，记录词的基本信息，通过该信息就可以快速地定位到该词的基表记录；第二个表主键为（PND_DOCID），用于保存基表发生的增量数据变化，用于修改全文索引时的增量填充。

表 20.1 CTI$INDEX_NAME$I 表结构

序号	字段名	类型	长度	精度	刻度	说明
1	WID	BIGINT	8	19	0	词 ID
2	WORD	VARCHAR	64	64	0	词文本，相同的词重复存储
3	TYPE	SMALLINT	2	5	0	词类型
4	FIRSTID	BIGINT	8	19	0	开始 ROWID，用于范围查找
5	LASTID	BIGINT	8	19	0	结束 ROWID
6	COUNT	INTEGER	4	10	0	词所在的文档数（即 ROWID 的个数）
7	ID_INFO	VARBINARY	4000	4000	0	保存所在文档所有的 ROWID，连续存放

表 20.2 CTI$INDEX_NAME$P 表结构

序号	字段名	类型	长度	精度	刻度	说明
1	PND_DOCID	BIGINT	8	19	0	文档 ROWID
2	PND_YPTE	SMALLINT	2	5	0	INS/UPD/DEL 类型

例如对示例库 bookshop 中的 address 表的 address1 列创建全文索引，创建的 SQL 语句如下。

```
    CREATE CONTEXT INDEX cti_address ON person.address (address1) LEXER
DEFAULT_LEXER;
```

执行成功之后，该索引信息会保存到 ctisys 模式下的 SYSCONTEXTINDEXES 系统表中，2 个自动生成的动态表 CTI$ CTI_ADDRESS$I 和 CTI$ CTI_ADDRESS$P 信息保存在 SYSOBJECTS 系统表中。这两个表和普通表一样，具有相关权限的用户可以进行查询、插入、更新和删除操作。

DM 全文索引创建的过程中，用户可以为分词器定义分词参数，即控制分词器分词的数量，包括 4 种分词参数：

1. CHINESE_LEXER，中文最少分词；
2. CHINESE_VGRAM_LEXER，中文最多分词；
3. ENGLISH_LEXER，英文分词；
4. DEFAULT_LEXER，默认分词，为中文最少分词。

指定中文分词参数可以切分英文，但是指定英文分词参数不可以切分中文。所以对存在中文的文本进行英文分词时会报分词参数指定错误。

中文最多分词可以将存在二义性的词划分出来，例如“和服装”会生成 “和”、“和服”、“服”、“服装”和“装”。那么在查询其中任何一个词的时候都可以检索到该文本。与之相对应的是最少分词，其消除了存在二义性的词，即前面举例的文本分词的结果为“和”、“服装”两个词，这样可以减少大量冗余词的存储，能进行更准确的匹配。英文分词即根据英文分隔符的分词，对超过 32 个字节的英文单词进行了拆分处理。

全文检索的中文分词依赖系统词库，该词库是只读的，不允许修改。中文分词时，对于非词库中的词，将作为未知词，相对已知词，其检索效率低。在特定背景的应用场景中，用户可以将专有词汇加入到自定义词库中，这有利于更准确地分词，从而提高全文检索性能。

为了提高词库的并发读性能，用户自定义词库的修改操作不会影响现有的分词，即词库的内存对象都是只读的，不允许修改，只有在词库重新加载（服务器重启或者修改前不使用全文检索）后才有效。

例如，创建一个中文新词库综合实例。

```
--创建一个新的词库文件
CALL SP_WORD_LIB_CREATE('user_word_lib.lib');
--向自定义词库添加一个新词
SELECT SF_WORD_LIB_ADD_WORD ('user_word_lib.lib','电监会');
--删除一个词
SELECT SF_WORD_LIB_DELETE_WORD ('user_word_lib.lib', '电监会');
--修改一个词
SELECT SF_WORD_LIB_MODIFY_WORD ('user_word_lib.lib', '电监会', '电会');
--删除自定义的词库（系统默认词库不允许删除）
CALL SP_WORD_LIB_DELETE('user_word_lib.lib');
--清空自定义的词库文件
SELECT SF_WORD_LIB_ CLEAR ('user_word_lib.lib');
--批量添加新词
SELECT SF_WORD_LIB_BATCH_ADD_WORDS ('user_word_lib.lib', 'new_words.txt');
--上述新词文件（new_words.txt）中，每个新词占一行，并且都为合法字符。如果存在重复的词，则只插入一次。具体格式如下：
新词 1
新词 2
…
--批量删除词。删除词的文件格式和批量插入词的文件格式相同。如果待删除的词不存在，则不会删除
SELECT SF_WORD_LIB_BATCH_DELETE_WORDS ('user_word_lib.lib', 'new_words.txt');
```

20.2 创建全文索引

在 DM 中，全文索引必须在基表上定义，而不能在系统表、视图、临时表、列存储表、垂直分区表、外部表上定义。同一列只能创建一个全文索引。DM 定义全文索引时，不需要在表上先建立主关键字索引。创建全文索引的列类型可为 CHAR、CHARACTER、VARCHAR、VARCHAR2、LONGVARCHAR、TEXT 或 CLOB，例如上节对 address 表创建的全文索引列为 VARCHAR 类型。DM 的全文索引现只支持简体中文、英文语言。较为详细的语法介绍可参考《DM_SQL》。

20.3 更新全文索引

全文索引的更新包括两种方式：完全更新和增量更新。

1. 完全更新

删除原有的全文索引，对基表进行全表扫描，逐一重构索引信息。在创建全文索引成功后，需完完全更新全文索引才可以执行有效的全文检索。完全更新全文索引没有次数限制，用户可以根据需要在增量更新或者是完全更新失败以及发生系统故障后都可以执行完全更新全文索引。例如执行如下 SQL 可以对 address 表的全文索引进行完全更新：

```
ALTER CONTEXT INDEX cti_address ON person.address REBUILD;
```

2. 增量更新

在上一次完全更新之后，表数据发生了变化，就可以使用增量更新来更新索引。增量更新只对新发生改变的数据进行重新分词生成索引信息，所以其代价是较小的。此时，也可以使用完全更新达到同样的更新索引的效果，但代价较大。

对全文索引的基表进行数据的增删改操作时，会将该数据的操作类型和 ROWID 信息保存到 CTI$INDEX_NAME$P 表中。

例如执行如下 SQL 可以对 address 表的全文索引进行增量更新：

```
ALTER CONTEXT INDEX cti_address ON person.address INCREMENT;
```

DM 在执行更新全文索引的过程中并没有产生相应的日志，即无法实现更新的回滚，如果在更新过程中发生任何错误，就必须执行完全更新获得正确的一致的全文索引信息。还需要注意的是当对已创建全文索引的基表进行批量数据修改（例如查询插入、批量导入等）时，需进行完全更新才能构建数据一致的新的全文索引，因为这些批量修改的数据信息未保存在全文索引动态信息表中，所以此时执行增量更新操作起不到更新全文索引的作用。

20.4 执行全文检索

全文检索根据查询关键字或者句子，从全文索引中查询满足条件的记录。DM7 支持对中英文关键字或者句子的布尔查询。

全文检索支持的检索类型有：

1. 支持英文单词的检索（单词不区分大小写）；
2. 支持全角英文的检索；
3. 支持简体中文词语的检索；
4. 支持常见单个汉字词的检索；

5. 支持简体中文长句子的检索；
6. 支持中英文混合的检索。

全文检索支持的检索方式有：

1. 在 CONTAINS 谓词内支持 AND、OR 和 AND NOT 的短语查询组合条件，例如查询 address 表中地址在洪山区太阳城的记录：

```
SELECT * FROM person.address WHERE CONTAINS(address1, '洪山区' AND '太阳城');
```

2. 全文检索支持单词或者句子的检索，例如查询 address 表中地址在江汉区发展大道的记录：

```
SELECT * FROM person.address WHERE CONTAINS(address1, '江汉区发展大道');
```

3. 不支持模糊方式的全文检索，例如查询“江汉区*”。
4. 检索条件子句可以和其他子句共同组成 WHERE 的检索条件，例如查询 address 表中地址在武汉市洪山区的记录。

```
SELECT * FROM person.address WHERE CONTAINS(address1, '洪山区') AND city LIKE '%武汉市%';
```

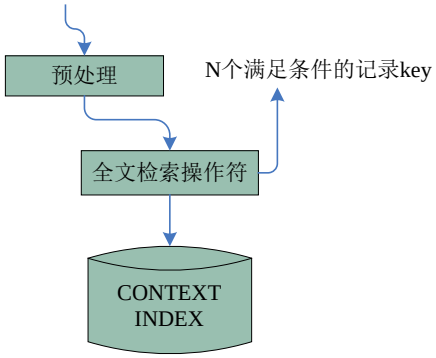


图 20.1 全文查询

如图 20.1 所示，DM 全文检索时对查询的文本做了预处理，首先根据词文本进行最少分词，并依据词频信息找出查询关键字，然后对这些关键字进行优先级排序，这样可以在第一次检索时找出所需文本的最小结果集，然后在该结果集的基础上再进行依次筛选，最终获得查询文本，这样能较大地提高查询性能。

20.5 删除全文索引

删除全文索引时，数据字典中相应的索引信息和全文索引内容都会被删除。删除索引有两种方式：使用语句删除和当模式对象发生改变时系统自动删除。第一种方式仅删除指定的基表的全文索引信息；第二种方式会删除模式对象（如数据库、基表等）上的所有全文索引。全文索引一旦删除就不能回滚，基于该表的全文检索就会失败，只能通过重新构建获得新的全文索引。例如删除 address 表，则会将表的全文索引 cti_address 也删除，包括与全文索引相关的动态表。

```
DROP CONTEXT INDEX cti_address ON person.address;
```

删除成功后，查询 ctisys 模式下的 SYSCONTEXTINDEXES 系统表会发现 cti_address 全文索引已删除。

第21章 管理事务

数据库是一个共享资源，可以被大量应用程序所共享。这些应用程序可以串行运行，但在绝大多数情况下，为了有效地利用数据库资源，多个应用程序会并发地访问数据库，这就是数据库的并发操作。此时，如果不对并发操作进行控制，则会存取不正确的数据，或破坏数据库数据的一致性。DM 数据库通过事务管理相关技术，有效解决了上述问题。本章阐述了事务的定义，并讲解如何使用事务来管理 DM 数据库。

21.1 事务简介

数据库事务是指作为单个逻辑工作单元的一系列操作的集合。一个典型的事务由应用程序中的一组操作序列组成，对于 DM 数据库来说，第一次执行 SQL 语句时，隐式地启动一个事务，以 COMMIT 或 ROLLBACK 语句/方法显式地结束事务。另外，在执行 DDL 前，DM 数据库会自动把前面的操作进行提交，DDL 前面的操作作为一个完整的事务结束，DDL 语句本身所属事务则根据“DDL_AUTO_COMMIT”配置参数决定是否隐式地提交（注意，无论 DDL_AUTO_COMMIT 参数如何设置，ALTER TABLESPACE 和 ALTER USER 操作总是自动提交的）。

COMMIT 操作会将该语句所对应事务对数据库的所有更新持久化（即写入磁盘），数据库此时进入一个新的一致性状态，同时该事务成功地结束。ROLLBACK 操作将该语句所对应事务对数据库的所有更新全部撤销，把数据库恢复到该事务初启动前的一致性状态。

我们以一个模拟的银行转账业务为例，假设一个银行客户 A 需要转 5000 元给 B，其具体业务步骤如下：

1. 从 A（ID 为 5236）的储蓄账户扣除 5000 元;

```
UPDATE account SET balance=balance-5000 WHERE id=5236;
```

2. 将 B（ID 为 5237）的储蓄账户增加 5000 元;

```
UPDATE account SET balance = balance +5000 WHERE id=5237;
```

3. 在业务日志中记录此次业务;

```
INSERT INTO trans_log VALUES(log_seq.NEXTVAL, 5236, 5237, 5000);
```

4. 提交事务。

```
COMMIT;
```

在上面的例子中，需要考虑两种情况：如果三条 SQL 语句全部正常执行，使账户间的平衡得以保证，那么此事务中对数据的修改就可以应用到数据库中；如果发生诸如资金不足、账号错误、硬件故障等问题，导致事务中一条或多条 SQL 语句不能执行，那么整个事务必须被回滚掉才能保证账户间的平衡。

DM 数据库提供了足够的事务管理机制来保证上面的事务要么成功执行，所有的更新都会写入磁盘，要么所有的更新都被回滚，数据恢复到执行该事务前的状态。无论是提交还是回滚，DM 保证数据库在每个事务开始前、结束后是一致的。

为了提高事务管理的灵活性，DM 数据库还提供了设置保存点（SAVEPOINT）和回滚到保存点的功能。保存点提供了一种灵活的回滚，事务在执行中可以回滚到某个保存点，在该保存点以前的操作有效，而以后的操作被回滚掉。可以使用 SAVEPOINT SAVEPOINT_NAME 命令创建保存点，使用 ROLLBACK TO SAVEPOINT SAVEPOINT_NAME

命令来回滚到保存点 SAVEPOINT_NAME。

21.2 事务特性

事务必须具备什么属性才是一个有效的事务呢？一个逻辑工作单元必须表现出四种属性，即原子性、一致性、隔离性和持久性，这样才能成为一个有效的事务。DM 数据库提供了各种机制以保证事务满足以上各种要求。

21.2.1 原子性

事务的原子性保证事务包含的一组更新操作是原子不可分的，也就是说这些更新操作是一个整体，对数据库而言全做或者全不做，不能部分地完成。这一性质即使在系统崩溃之后仍能得到保证，在系统崩溃之后将进行数据库恢复，用来恢复和撤销系统崩溃时处于活动状态的事务对数据库的影响，从而保证事务的原子性。系统对磁盘上的任何实际数据的修改之前都会将修改操作本身的信息记录到磁盘上。当发生崩溃时，系统能根据这些操作记录当时该事务处于何种状态，以此确定是撤销该事务所做出的所有修改操作，还是将修改的操作重新执行。

21.2.2 一致性

数据一致性是指表示客观世界同一事务状态的数据，不管出现在何时何处都是一致的、正确的、完整的。换句话说，数据一致性是任何点上保证数据以及内部数据结构的完整性，如 B 树索引的正确性。

一致性要求事务执行完成后，将数据库从一个一致状态转变到另一个一致状态。它是一种以一致性规则为基础的逻辑属性，例如在转账的操作中，各账户金额必须平衡，这一条规则对于程序员而言是一个强制的规定。事务的一致性属性要求事务在并发执行的情况下事务的一致性仍然满足。

21.2.3 隔离性

事务是隔离的，意味着每个事务的执行效果与系统中只有该事务的执行效果一样，也就是说，某个并发事务所做的修改必须与任何其他的并发事务所做的修改相互隔离。这样，只有当某个值被一个事务修改完并提交后才会影响到另一个事务。事务只会识别另一并发事务修改之前或者修改完成之后的数据，不会识别处于这中间状态的数据。事务的隔离行为依赖于指定的隔离级别。隔离级别的介绍请参考本章第 7 节。

21.2.4 持久性

持久性是指一个事务一旦被提交，它对数据库中数据的改变就是永久性的，接下来的其他操作和数据库故障不应该对其有任何影响。即一旦一个事务提交，DBMS 保证它对数据库中数据的改变应该是永久性的。如果 DM 数据库或者操作系统出现故障，那么在 DM 数据库

重启的时候，数据库会自动恢复。如果某个数据驱动器出现故障，并且数据丢失或者被损坏，可以通过备份和联机重做日志来恢复数据库。需要注意的是，如果备份驱动器也出现故障，且系统没有准备其他的可靠性解决措施，备份就会丢失，那么就无法恢复数据库了。

21.3 提交事务

提交事务就是提交事务对数据库所做的修改，将从事务开始的所有更新保存到数据库中，任何更改的记录都被写入日志文件并最终写入到数据文件，同时提交事务还会释放由事务占用的资源，如锁。如果提交时数据还没有写入到数据文件，DM 数据库后台线程会在适当时机（如检查点、缓冲区满）将它们写入。

具体说来，在一个修改了数据的事务被提交之前，DM 数据库进行了以下操作：

1. 生成回滚记录，回滚记录包含了事务中各 SQL 语句所修改的数据的原始值；
2. 在系统的重做日志缓冲区中生成重做日志记录，重做日志记录包含了对数据页和回滚页所进行的修改，这些记录可能在事务提交之前被写入磁盘；
3. 对数据的修改已经被写入数据缓冲区，这些修改也可能在事务提交之前被写入磁盘。

已提交事务中对数据的修改被存储在数据库的缓冲区中，它们不一定被立即写入数据文件内。DM 数据库自动选择适当的时机进行写操作以保证系统的效率。因此写操作既可能发生在事务提交之前，也可能在提交之后。

当事务被提交之后，DM 数据库进行以下操作：

1. 将事务任何更改的记录写入日志文件并最终写入到数据文件；
2. 释放事务上的所有锁，将事务标记为完成；
3. 返回提交成功消息给请求者。

在 DM 数据库中还存在有 3 种事务模式：自动提交模式、手动提交模式和隐式提交模式。

21.3.1 自动提交模式

除了命令行交互式工具 DISQL 外，DM 数据库缺省都采用自动提交模式。用户通过 DM 数据库的其他管理工具、编程接口访问 DM 数据库时，如果不手动/编程设置提交模式，所有的 SQL 语句都会在执行结束后提交，或者在执行失败时回滚，此时每个事务都只有一条 SQL 语句。

在 DISQL 中，用户也可以通过执行如下语句来设置当前会话为自动提交模式：

```
SET AUTOCOMMIT ON;
```

21.3.2 手动提交模式

在手动提交模式下，DM 数据库用户或者应用开发人员明确定义事务的开始和结束，这些事务也被称为显式事务。在 DISQL 中，没有设置自动提交时，就是处于手动提交模式，此时 DISQL 连接到服务器后第一条 SQL 语句或者事务结束后的第一条语句就标记着事务的开始，可以执行 COMMIT 或者 ROLLBACK 来提交或者回滚事务。

21.3.3 隐式提交

在手动提交模式下，当遇到 DDL 语句时，DM 数据库会自动提交前面的事务，然后开始一个新的事务执行 DDL 语句。这种事务提交被称为隐式提交。DM 数据库在遇到以下 SQL 语句时自动提交前面的事务：

1. CREATE；
2. ALTER；
3. TRUNCATE；
4. DROP；
5. GRANT；
6. REVOKE；
7. 审计设置语句。

21.4 回滚事务

回滚事务是撤消该事务所做的任何更改。回滚有两种形式：DM 数据库自动回滚，或者通过程序/ROLLBACK 命令手动回滚。除此之外，与回滚相关的还有回滚到保存点和语句级回滚，下面分别进行介绍。

21.4.1 自动回滚

若事务运行期间出现连接断开，DM 数据库都会自动回滚该连接所产生的事务。回滚会撤消事务执行的所有数据库更改，并释放此事务使用的所有数据库资源。DM 数据库在恢复时也会使用自动回滚。例如在运行事务时服务器突然断电，接着系统重新启动，DM 数据库就会在重启时执行自动恢复。自动恢复要从事务重做日志中读取信息以重新执行没有写入磁盘的已提交事务，或者回滚断电时还没有来得及提交的事务。

21.4.2 手动回滚

一般来说，在实际应用中，当某条 SQL 语句执行失败时，用户会主动使用 ROLLBACK 语句或者编程接口提供的回滚函数来回滚整个事务，避免不合逻辑的事务污染数据库，导致数据不一致。如果发生错误后确实只用回滚事务中的一部分，则需要用到回滚到保存点的功能。

21.4.3 回滚到保存点

除了回滚整个事务之外，DM 数据库的用户还可以部分回滚未提交事务，即从事务的最末端回滚到事务中任意一个被称为保存点的标记处。用户在事务内可以声明多个被称为保存点的标记，将一个大事务划分为几个较小的片断。之后用户在对事务进行回滚操作时，就可以选择从当前执行位置回滚到事务内的任意一个保存点。例如用户可以在一系列复杂的更新操作之间插入保存点，如果执行过程中一个语句出现错误，用户可以回滚到错误之前的某个

保存点，而不必重新提交所有的语句。当事务被回滚到某个保存点后，DM 数据库将释放被回滚语句中使用的锁。其他等待“被锁资源”的事务就可以继续执行，需要更新“被锁数据行”的事务也可以继续执行。

将事务回滚到某个保存点的过程如下：

1. 只回滚保存点之后的语句；
2. 保留该保存点，其后创建的保存点都被清除；
3. 释放此保存点之后获得的所有锁，保留该保存点之前的锁。

被部分回滚的事务依然处于活动状态，可以继续执行。DM 数据库用户可以使用 SAVEPOINT SAVEPOINT_NAME 命令创建保存点，使用 ROLLBACK TO SAVEPOINT SAVEPOINT_NAME 命令来回滚到保存点 SAVEPOINT_NAME。

21.4.4 语句级回滚

如果在一个 SQL 语句执行过程中发生了错误，此语句对数据库产生的影响将被回滚，回滚后就如同此语句从未执行过，这种操作被称为语句级回滚。语句级回滚只会使此语句所做的数据修改无效，不会影响此语句之前所做的数据修改。

当 INI 参数 ROLL_ON_ERR 为缺省值 0 时，在 SQL 语句执行过程中发生的错误，将会导致语句级回滚，例如违反唯一性、死锁（访问相同数据而产生的竞争）、运算溢出等。在 SQL 语句解析的过程中发生错误（例如语法错误），由于未对数据产生任何影响，因此不会产生语句级回滚。

21.5 事务锁定

DM 数据库支持多用户并发访问、修改数据，有可能出现多个事务同时访问、修改相同数据的情况。若对并发操作不加控制，就可能会访问到不正确的数据，破坏数据的一致性和正确性。DM 数据库采用封锁机制来解决并发问题，本节将详细介绍 DM 数据库中的事务锁定相关功能，包括锁的分类以及如何查看锁等。

21.5.1 锁模式

锁模式指定并发用户如何访问锁定资源。DM 数据库使用四种不同的锁模式：共享锁、排他锁、意向共享锁和意向排它锁。

1. 共享锁

共享锁（Share Lock，简称 S 锁）用于读操作，防止其他事务修改正在访问的对象。这种封锁模式允许多个事务同时并发读取相同的资源，但是不允许任何事务修改这个资源。

2. 排他锁

排它锁（Exclusive Lock，简称 X 锁）用于写操作，以独占的方式访问对象，不允许任何其他事务访问被封锁对象；防止多个事务同时修改相同的数据，避免引发数据错误；防止访问一个正在被修改的对象，避免引发数据不一致。一般在修改对象定义时使用。

3. 意向锁

意向锁（Intent Lock）用于读取或修改被访问对象数据时使用，多个事务可以同时

对相同对象上意向锁，DM 支持两种意向锁：

1) 意向共享锁（Intent Share Lock，简称 IS 锁）：一般在只读访问对象时使用；
2) 意向排它锁（Intent Exclusive Lock，简称 IX 锁）：一般在修改对象数据时使用。

四种锁模式的相容矩阵如下表所示，其中“Y”表示相容；“N”表示不相容。如表中第二行第二列为“Y”，表示如果已经加了 IS 锁时，其他用户还可以继续添加 IS 锁，第二行第五列为“N”，表示如果已经加了 IS 锁时，其他用户不能添加 X 锁。

表 21.1 锁相容矩阵

	IS	**IX**	**S**	**X**
IS	Y	Y	Y	N
IX	Y	Y	N	N
S	Y	N	Y	N
X	N	N	N	N

21.5.2 锁粒度

按照封锁对象的不同，锁可以分为 TID 锁和对象锁。

1. TID 锁

TID 锁以事务号为封锁对象，为每个活动事务生成一把 TID 锁，代替了其他数据库行锁的功能，防止多个事务同时修改同一行记录。DM 实现的是行级多版本，每一行记录隐含一个 TID 字段，用于事务可见性判断。

执行 INSERT、DELETE、UPDATE 操作时，设置事务号到 TID 字段。这相当于隐式地对记录上了一把 TID 锁，INSERT、DELETE、UPDATE 操作不再需要额外的行锁，避免了大量行锁对系统资源的消耗。只有多个事务同时修改同一行记录时，才会产生新的 TID 锁。例如，当事务 T1（事务号为 TID1）试图修改某行数据，而该行数据正在被另一个事务 T2（事务号为 TID2）修改，此时事务 T1 会生成一个新的 TID 锁，其锁对象为事务号 TID2，而非事务 T2。

同时多版本写不阻塞读的特性，SELECT 操作已经消除了行锁，因此 DM 中不再有行锁的概念。

2. 对象锁

对象锁是 DM 新引入的一种锁，通过统一的对象 ID 进行封锁，将对数据字典的封锁和表锁合并为对象锁，以达到减少封锁冲突、提升系统并发性能的目的。我们先看一下通常数据字典锁和表锁各自应承担的功能：

1) 数据字典锁：用来保护数据字典对象的并发访问，解决 DDL 并发和 DDL/DML 并发问题，防止多个事务同时修改同一个对象的字典定义，确保对同一个对象的 DDL 操作是串行执行的。并防止一个事务在修改字典定义的同时，另外一个事务修改对应表的数据。
2) 表锁：表锁用来保护表数据的完整性，防止多个事务同时采用批量方式插入、更新

一张表，防止向正在使用 FAST LOADER 工具装载数据的表中插入数据等，保证这些优化后数据操作的正确性。此外，表锁还有一个作用，避免对存在未提交修改的表执行 ALTER TABLE、TRUNCATE TABLE 操作。

为了实现与数据字典锁和表锁相同的封锁效果，从逻辑上将对象锁的封锁动作分为四类：

a) 独占访问（EXCLUSIVE ACCESS），不允许其他事务修改对象，不允许其他事务访问对象，使用 X 方式封锁

b) 独占修改（EXCLUSIVE MODIFY），不允许其他事务修改对象，允许其他事务共享访问对象，使用 S + IX 方式封锁

c) 共享修改（SHARE MODIFY），允许其他事务共享修改对象，允许其他事务共享访问对象，使用 IX 方式封锁

d) 共享访问（SHARE ACCESS），允许其他事务共享修改对象，允许其他事务共享访问对象，使用 IS 方式封锁

3. 显式锁定表

用户可以根据自己的需要显式的对表对象进行封锁。显式锁定表的语法如下：

```
LOCK TABLE <table_name> IN <lock_mode> MODE [NOWAIT];
```

lock_mode 是锁定的模式，可以选择的模式有 INTENT SHARE（意向共享）、INTENT EXCLUSIVE（意向排他）、SHARE（共享）和 EXCLUSIVE（排他），其含义分别如下：

1) 意向共享：不允许其他事务独占访问该表。意向共享锁定后，不同事务可以同时增、删、改、查该表的数据，也支持在该表上创建索引，但不支持修改该表的定义；

2) 意向排他：不允许其他事务独占访问和独占修改该表。被意向排他后，不同事务可以同时增、删、改、查该表的数据，不支持在该表上创建索引，也不支持修改该表定义；

3) 共享：只允许其他事务共享访问该表，仅允许其他事务查询表中的数据，但不允许增、删、改该表的数据；

4) 排他：以独占访问方式锁定整个表，不允许其他事务访问该表，是封锁力度最大的一种封锁方式。

当使用 NOWAIT 时，若不能立即上锁成功则立刻返回报错信息，不再等待。

21.5.3 查看锁

为了方便用户查看当前系统中锁的状态，DM 数据库专门提供了一个 V$LOCK 动态视图。通过该视图，用户可以查看到系统当前所有锁的详细信息，如锁的内存地址、所属事务 ID、上锁的对象、锁模式、对象 ID 以及行事务 ID。用户可以通过执行如下语句查看锁信息：

```
SELECT * FROM V$LOCK;
```

其结果看起来和下面类似：

```
ADDR       TRX_ID    LTYPE   LMODE   BLOCKED     TABLE_ID    ROW_IDX
---------- -------------------- -------------------- ------ -----
----------- ----------- --------------------
233113736    932       OBJECT   IX          0           1149           0
232810632    933       OBJECT   IX          0           1149        0
233113680    932       OBJECT   IS          0           -1          0
232810576    933       OBJECT   IS          0           -1          0
```

```
233113792    932       OBJECT   IS          0           4           0
232810688    933       OBJECT   IS          0           4           0
232810744    933       OBJECT   IS          0           33555437    0
232810520    933       TID      X       0          0           933
233113624    932       TID      X       0          0           932
232810800    933       TID      S       1          0           932
```

其中 ADDR 列表示锁的内存地址；TRX_ID 列表示锁所属的事务 ID；LTYPE 列表示被上锁对象的类型，可能是 OBJECT（对象锁）或者 TID（TID 锁）；LMODE 表示锁的模式，可能的取值有 S（共享锁）、X（排它锁）、IS（意向共享锁）、IX（意向排它锁）；BLOCKED 列表示锁是否处于上锁等待状态，0 表示已上锁成功，1 表示处于上锁等待状态；TABLE_ID 列表示表的 ID；ROW_IDX 列是 TID 锁的封锁对象事务 ID。

21.6 多版本

在多版本控制以前，数据库仅通过锁机制来实现并发控制。数据库对读操作上共享锁，写操作上排它锁，这种锁机制虽然解决了并发问题，但影响了并发性。例如，当对一个事务对表进行查询时，另一个对表更新的事务就必须等待。DM 数据库的多版本实现完全消除了行锁对系统资源的消耗，查询永远不会被阻塞也不需要上行锁，并通过 TID 锁机制消除了插入、删除、更新操作的行锁。数据库的读操作与写操作不会相互阻塞，并发度大幅度提高。

DM 数据库基于物理记录和回滚记录实现行级多版本支持，数据页中只保留物理记录的最新版本，通过回滚记录维护历史版本，所有事务针对特定的版本进行操作。

21.6.1 物理记录格式

为了适应多版本机制，高效地获取历史记录，每一条物理记录中包含了两个 字段：TID 和 RPTR。TID 保存修改记录的事务号，RPTR 保存回滚段中上一个版本回滚记录的物理地址 。插入、删除和更新物理记录时，RPTR 指向操作生成的回滚记录的物理地址。物理记录格式如下：

物理记录	TID	RPTR

21.6.2 回滚记录格式

回滚记录与物理记录一样，增加了两个字段：TID 和 RPTR。TID 保存回滚记录对应的事务号，RPTR 保存回滚段中上一个版本回滚记录的物理地址。

回滚记录	TID	RPTR

插入物理记录时，由于没有更老的版本数据，回滚记录的 RPTR 值为 NULL；更新和删除物理记录时，RPTR 指向原始物理记录的 RPTR，新物理记录的 RPTR 指向当前回滚记录的物理地址。

21.6.3 可见性原则

实现多版本控制的关键是可见性判断，找到对当前事务可见的特定版本数据。DM 通过

活动事务表，确定事务的可见性。根据事务隔离级的不同，在事务启动时（串行化），或者语句执行时（读提交），收集这一时刻所有活动事务，并记录系统中即将产生的事务号 NEXT_TID。DM 多版本可见性原则：

1. 物理记录的 TRXID 等于当前事务号，说明是本事务修改的物理记录，物理记录可见；
2. 物理记录的 TRXID 不在活动事务表中，并且 TRXID 小于 NEXT_TID，物理记录可见；
3. 物理记录的 TRXID 包含在活动事务表中，或者 TRXID 大于等于 NEXT_TID，物理记录不可见。

21.6.4 历史数据获取

当物理记录对当前事务不可见时，根据物理记录和回滚记录的 RPTR 指针，向前回溯一个历史版本记录，通过此历史版本记录的 TID 字段，依据事务可见性原则判断此版本的记录对当前事务是否可见。如可见即获取到了满足当前事务的历史版本数据；如不可见则根据 RPTR 指针继续向前回溯。如果一直不能找到对当前事务的可见版本（例如此记录是一个活动事务插入的新记录），则此记录将不会添加到查询结果集中。

下面以 UPDATE 为例描述多版本的实现；依次执行事务 T1 和 T2：

事务 T1

```
CREATE TABLE TEST_UPDATE (COL_1 INT PRIMARY KEY, COL_2 VARCHAR(10));
INSERT INTO TEST_UPDATE VALUES(1, 'ABCD');
```

事务 T2

```
UPDATE test_update set col_2='xyz' where col_1 = 1;
```

执行以上两个事务以后，表 TEST_UPDATE 的记录了两个版本

物理记录：

物理记录，字段值为（1，‘XYZ’）	TID（T2 的 ID）	RPTR（回滚记录地址）

回滚记录：

回滚记录，保存了老记录的值（1，‘ABCD’）	TID(T1 的 ID)	RPTR

21.6.5 回滚段自动清理

由于需要根据回滚记录回溯、还原物理记录的历史版本信息，因此不能在事务提交时立即清除当前事务产生的回滚记录。但是，如果不及时清理回滚段，可能造成回滚段空间的不断膨胀，占用大量磁盘空间。

DM 提供了自动清理、回收回滚段空间的机制。系统定时（缺省是每间隔 1 秒）扫描回滚段，根据回滚记录的 TID，判断是否需要保留回滚记录，清除那些对所有活动事务可见的回滚记录空间。

21.7 事务隔离级

在关系型数据库中，事务的隔离性分为四个隔离级别，在解读这四个级别前先介绍几个关于读数据的概念。

1. 脏读（DirtyRead）

所谓脏读就是对脏数据的读取，而脏数据所指的就是未提交的已修改数据。也就是说，一个事务正在对一条记录做修改，在这个事务完成并提交之前，这条数据是处于待定状态的（可能提交也可能回滚），这时，第二个事务来读取这条没有提交的数据，并据此做进一步的处理，就会产生未提交的数据依赖关系，这种现象被称为脏读。如果一个事务在提交操作结果之前，另一个事务可以看到该结果，就会发生脏读。

2. 不可重复读（Non-RepeatableRead）

一个事务先后读取同一条记录，但两次读取的数据不同，我们称之为不可重复读。如果一个事务在读取了一条记录后，另一个事务修改了这条记录并且提交了事务，再次读取记录时如果获取到的是修改后的数据，这就发生了不可重复读情况。

3. 幻像读（PhantomRead）

一个事务按相同的查询条件重新读取以前检索过的数据，却发现其他事务插入了满足其查询条件的新数据，这种现象就称为幻像读。

在 SQL-92 标准中，定义了四种隔离级别：读未提交、读提交、可重复读和串行化。每种隔离级别下对于读数据有不同的要求，表 21.2 中列出四种隔离级别下系统允许/禁止哪些类型的读数据现象。其中“Y”表示允许，“N”表示禁止。

表 21.2 隔离级别表

解决 / 级别	脏读	不可重复读	幻像读
读未提交	Y	Y	Y
读提交	N	Y	Y
可重复读	N	N	Y
串行化	N	N	N

在只有单一用户的数据库中，用户可以任意修改数据，而无需考虑同时有其他用户正在修改相同的数据。但在一个多用户数据库中，多个并发事务中包含的语句可能会修改相同的数据。数据库中并发执行的事务最终应产生有意义且具备一致性的结果。因此在多用户数据库中，对数据并发访问及数据一致性进行控制是两项极为重要的工作。

为了描述同时执行的多个事务如何实现数据一致性，数据库研究人员定义了被称为串行化处理的事务隔离模型。当所有事务都采取串行化模式执行时，我们可以认为同一时间只有一个事务在运行（串行的），而非并发的。

DM 数据库支持三种事务隔离级别：读未提交、读提交和串行化。其中，读提交是 DM 数据库默认使用的事务隔离级别。可重复读升级为更严格的串行化隔离级。

21.7.1 读提交隔离级

DM 数据库的读提交隔离可以确保只访问到已提交事务修改的数据，保证数据处于一致性状态，能够满足大多数应用的要求，并最大限度的保证系统并发性能，但可能会出现不可重复读取和幻像读。

用户可以在事务开始时使用以下语句设定事务为读提交隔离级：

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

21.7.2 串行化隔离级

在要求消除不可重复读取或幻像读的情况下，我们可以设置事务隔离级为串行化。跟读提交隔离级相比，串行化事务的查询本身不会增加任何代价，但修改数据可能引发“串行化事务被打断”错误。

具体来说，当一个串行化事务试图更新或删除数据时，而这些数据在此事务开始后被其他事务修改并提交时，DM 数据库将报“串行化事务被打断”错误。应用开发者应该充分考虑串行化事务带来的回滚及重做事务的开销，从应用逻辑上避免对相同数据行的激烈竞争导致产生大量事务回滚。并结合应用逻辑，捕获“串行化事务被打断”错误，进行事务重做等相应处理。如果系统中存在长时间运行的写事务，并且该长事务所操作的数据还会被其他短事务频繁更新的话，最好避免使用串行化事务。

用户可以在事务开始时使用以下语句设定事务为串行化隔离级：

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

21.7.3 读未提交隔离级

DM 数据库除了支持读提交、串行化两种隔离级之外，还支持读未提交这种隔离级。

读未提交隔离级别是最不严格的隔离级别。实际上，在使用这个隔离级别时，有可能发生脏读、不可重复读和幻像。一般来说，读未提交隔离级别通常只用于访问只读表和只读视图，以消除可见性判断带来的系统开销，提升查询性能。

用户可以在事务开始时使用以下语句，设定事务为读未提交隔离级：

```
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
```

21.7.4 只读事务

除了前面所述的各种标准特性外，DM 数据库还支持只读事务，只读事务只能访问数据，但不能修改数据。并且只读事务不会改变事务原有的隔离级。

用户可以在事务开始时使用以下语句，设定事务为只读事务：

```
SET TRANSACTION READ ONLY;
```

21.8 锁等待与死锁检测

阻塞和死锁是会与并发事务一起发生的两个事件，它们都与锁相关。当一个事务正在占用某个资源的锁，此时另一个事务正在请求这个资源上与第一个锁相冲突的锁类型时，就会发生阻塞。被阻塞的事务将一直挂起，直到持有锁的事务放弃锁定的资源为止。死锁与阻塞的不同之处在于死锁包括两个或者多个已阻塞事务，它们之间形成了等待环，每个都等待其他事务释放锁。例如事务 1 给表 T1 上了排他锁，第二个事务给表 T2 上了排他锁，此时事务 1 请求 T2 的排它锁，就会处于等待状态，被阻塞。若此时 T2 再请求表 T1 的排他锁，则 T2 也处于阻塞状态。此时这两个事务发生死锁，DM 数据库会选择牺牲掉其中一个事务。

在 DM 数据库中，INSERT、UPDATE、DELETE 是最常见的会产生阻塞和死锁的语句。

INSERT 发生阻塞的唯一情况是，当多个事务同时试图向有主键或 UNIQUE 约束的表中

插入相同的数据时，其中的一个事务将被阻塞，直到另外一个事务提交或回滚。一个事务提交时，另一个事务将收到唯一性冲突的错误；一个事务回滚时，被阻塞的事物可以继续执行。

当 UPDATE 和 DELETE 修改的记录，已经被另外的事务修改过，将会发生阻塞，直到另一个事务提交或回滚。

21.9 闪回

当用户操作不慎导致错误的删改数据时，非常希望有一种简单快捷的方式可以恢复数据。闪回技术，就是为了用户可以迅速处理这种数据逻辑损坏的情况而产生的。

闪回技术主要是通过回滚段存储的 UNDO 记录来完成历史记录的还原。设置 ENABLE_FLASHBACK 为 1 后，开启闪回功能。DM 会保留回滚段一段时间，回滚段保留的时间代表着可以闪回的时间长度。由 UNDO_RETENTION 参数指定。

开启闪回功能后，DM 会在内存中记录下每个事务的起始时间和提交时间。通过用户指定的时刻，查询到该时刻的事务号，结合当前记录和回滚段中的 UNDO 记录，就可以还原出特定事务号的记录。即指定时刻的记录状态。从而完成闪回查询。闪回查询功能完全依赖于回滚段管理，对于 DROP 等误操作不能恢复。闪回特性可应用在以下方面：

1. 自我维护过程中的修复：当一些重要的记录被意外删除，用户可以向后移动到一个时间点，查看丢失的行并把它们重新插入现在的表内恢复；
2. 用于分析数据变化：可以对同一张表的不同闪回时刻进行链接查询，以此查看变化的数据。

第四部分 故障排除和性能优化

第22章 问题跟踪和解决

问题跟踪和解决是最复杂的脑力劳动之一，其复杂主要体现在问题的无法预知性和问题根源的多样性。本章的目的是提供一种相对比较系统的方法来查找并解决使用 DM 数据库过程中可能遇到的各种难以跟踪、定位的问题，为 DBA 分析、解决问题提供帮助。

22.1 问题分析

当系统出现问题，无法及时响应用户/应用请求时，可能的原因是多方面的。一般来说，DBA 应该查看和分析的内容包括：

1. 网络是否正常

DBA 可以直接使用各种工具/软件来排除网络问题。如果远程操作有问题，但是本地操作没有问题，则有可能是网络出现故障或者网络带宽耗尽。但是如果本地也有问题，不能说明网络一定没有问题，此时还是需要用其他方式确定网络是否有故障，同时进一步分析本地问题产生的原因。

2. 内存使用量

用户可以通过操作系统提供的内存检测工具/命令来查看数据库占用内存情况，看是否数据库占用了过多内存，并且开始大量使用页面文件（Windows）/交换分区（Linux/UNIX），如果数据库占用内存过多，则需要进一步分析可能的原因：是否数据库的内存相关参数设置错误，是否客户端请求的资源过多并且一直没有释放（如不断打开连接/游标，并且一直不关闭）等等。对于参数设置错误，DBA 可以通过修改参数加以解决。如果是客户端请求资源过多，DBA 可以通过查询运行时动态视图（参考附录 2：动态性能视图）来检查资源使用情况，明确问题产生的原因并予以纠正。若排除其他原因后，发现数据库内存仍在不断增长，此时可以联系达梦公司技术服务人员协助解决。

3. CPU 使用率

当发现系统响应很慢甚至无法响应时，CPU 的使用率也是一个重要的观察指标。如果 CPU 使用率一直持续 90%以上，甚至 100%，则说明 CPU 使用率过高，此时需要分析导致 CPU 使用率过高的原因。可能的原因包括：写了错误的存储过程/函数死循环逻辑；某条 SQL 语句执行计划不好导致 CPU 使用率过高（如没有建立合适的索引等）；系统内部 SQL 语句都执行正常，只是实际应用负载过大等。针对上述原因，DBA 可以分别考虑改正存储过程/函数的死循环逻辑、建立合适的索引以及提供更高配置的软/硬件环境等措施分别予以解决。

4. I/O 是否正常

I/O 性能没有满足要求是导致很多系统性能低下的原因。通常情况下，主要是两个方面的原因导致 I/O 性能瓶颈：在系统规划时没有对 I/O 性能进行估算或者估算偏差太大，导

致存储的 I/O 性能无法满足要求；其次是没有利用好数据库特性，如没有建立合适的索引，导致经常要做全表扫描，消耗大量 I/O 带宽，这可以通过查看 SQL 语句执行计划来加以分析。

5. 系统日志

最后，DBA 还可以通过查看系统日志来辅助分析问题。在 DM 数据库运行过程中，会将一些关键信息记录到安装目录下一级 log 目录下的名称为 dm_YYYYMM.log 的日志文件中，其中 YYYY 表示年份，MM 表示月份，该文件会记录下数据库服务启动/关闭的时间、系统关键错误如打开文件失败等。另外，如果将 DM 数据库配置文件中的参数 SVR_LOG 设置为 1，则系统还会在刚才的 log 目录下生成名为 log_commitNN.log 的文件，在该文件中记录了启用 SVR_LOG 之后数据库接收到的所有 SQL 语句等信息，DBA 也可以通过分析该文件来帮助解决问题。

22.2 监控系统性能(V$)

在 DM 数据库中，定义了一系列以 V$为前缀的系统动态视图（参考附录 2：动态性能视图），这些表只有表结构信息，没有数据，有时也称之为虚视图。查询动态视图时，服务器动态加载数据。在 DM 数据库运行过程中，系统动态视图提供了大量系统内部信息，便于数据库管理员监视服务器的运行状况，并根据这些信息对数据库进行调优，达到提高数据库性能的目的。用户可以通过普通 SQL 查询语句来查询动态视图信息，也可以通过图形化客户端工具 Monitor 来进行查看。

某些动态性能视图如 V$SYSSTAT、V$WAIT_HISTORY 需要 ENABLE_MONITOR 参数设置为非 0 或者 2 后才会进行相关信息的收集。

22.3 数据库重演(REPLAY)

数据库重演（Database Replay）是 DM 中用来重现、定位和分析问题的一个重要手段，其基本原理是在数据库系统上捕获所有负载（记录外部客户端对服务器的请求），保存到二进制捕获文件，然后通过 DM 提供的数据库重演工具将捕获文件中的请求发送给捕获前由原始数据库备份恢复而来的重演测试系统上，从而帮助重现当时的场景。

用户可以调用系统过程 sp_start_capture 来启动捕获发往数据库的所有负载，并将该阶段收到的所有请求保存到二进制捕获文件中，然后使用 DM 提供的数据库重演客户端工具重放二进制捕获文件，再现当时真实环境的负载及运行情况，帮助进行问题跟踪和诊断。使用系统过程 sp_stop_capture 可以停止捕获。

Replay 的使用必须指定必要的执行参数，其调用格式为：

```
格式：DREPLAY  KEYWORD=value
例程：DREPLAY  SERVER=LOCALHOST:5236 FILE=.\test.cpt
必选参数：FILE
--如果需要获取帮助信息，可以调用 dreplay help，屏幕将显示如下信息：
关键字            说明（默认）
------------------------------------------------------------
SERVER           需要连接的服务器格式 SERVER:PORT(LOCALHOST:5236)
FILE             捕获文件及路径
HELP             打印帮助信息
```

22.4 检查数据物理一致性

DM 数据库提供了用于检查数据物理一致性的工具 dmdbchk。在数据库服务器正常关闭的情况下，可以使用 dmdbchk 对数据文件完整性进行校验，检验的内容主要包括：数据文件大小的校验；索引合法性校验；数据页面校验；系统对象 ID 校验等。在检验完毕后，dmdbchk 会在当前目录下生成一个名为 dbchk_err.txt 的检查报告，供用户查看。

dmdbchk 的使用必须指定必要的执行参数，其调用格式为：

```
格式:: dmdbchk [ini_file_path]
例程：
dmdbchk path=d:\dmdbms\bin\dm.ini
--如果需要获取帮助信息，可以调用 dmdbchk help，屏幕将显示如下信息：
关键字            说明（默认）
------------------------------------------------------------------
PATH              dm.ini 绝对路径或者当前目录的 dm.ini
HELP              打印帮助信息
```

22.5 调整配置参数

在 DM 数据库中，很多参数都是动态的，会自动响应各种负载，但是 DBA 仍然可以调用系统过程来改变 DM 实例的运行参数，从而获得更佳的性能体验。DBA 可以在 DM 数据库运行过程中执行 SF_GET_PARA_VALUE 、 SF_GET_PARA_DOUBLE_VALUE 和 SF_GET_PARA_STRING_VALUE 这三个函数来获取系统的当前配置参数，并且可以使用 SP_SET_PARA_VALUE 和 SP_SET_PARA_DOUBLE_VALUE 过程来修改静态/动态配置参数：

1. SF_GET_PARA_VALUE (scope int, paraname varchar(256))

配置参数的值类型为数值类型时使用该函数来获取当前值。SCOPE 参数为 1 表示获取 INI 文件中配置参数的值，为 2 表示获取内存中配置参数的值。

2. SF_GET_PARA_DOUBLE_VALUE(scope int, paraname varchar(8187))

配置参数的值类型为浮点型时使用该函数来获取当前值。SCOPE 参数为 1 表示获取 INI 文件中配置参数的值，为 2 表示获取内存中配置参数的值。

3. SF_GET_PARA_STRING_VALUE (scope int, paraname varchar(8187))

配置参数的值为字符串类型时用该系统函数来获取当前值。SCOPE 参数为 1 表示获取 INI 文件中配置参数的值，为 2 表示获取内存中配置参数的值。

4. SP_SET_PARA_VALUE (scope int, paraname varchar(256), value int64)

该过程用于修改整型静态配置参数和动态配置参数。SCOPE 参数为 1 表示在内存和 INI 文件中都修改参数值，此时只能修改动态的配置参数。参数为 2 表示只在 INI 文件中修改配置参数，此时可用来修改静态配置参数和动态配置参数。当 SCOPE 等于 1，试图修改静态配置参数时服务器会返回错误信息。只有具有 DBA 角色的用户才有权限调用 SP_SET_PARA_VALUE。

5. SP_SET_PARA_DOUBLE_VALUE (scope int, paraname varchar(8187),

alue double)

该过程用于修改浮点型静态配置参数和动态配置参数。SCOPE 参数为 1 表示在内存和 INI 文件中都修改参数值，此时只能修改动态的配置参数。参数为 2 表示只在 INI 文件中修改配置参数，此时可用来修改静态配置参数和动态配置参数。当 SCOPE 等于 1，试图修改静态配置参数时服务器会返回错误信息。只有具有 DBA 角色的用户才有权限调用 SP_SET_PARA_DOUBLE_VALUE。

6. SF_SET_SYSTEM_PARA_VALUE (paraname varchar(256), value int64\double\varchar（256）, deferred int, scope int64)

该过程用于修改系统整型、double、varchar 的静态配置参数或动态配置参数。DEFERRED 参数，为 0 表示当前 session 修改的参数立即生效，为 1 表示当前 session 不生效，后续再生效，默认为 0。SCOPE 参数为 1 表示在内存和 INI 文件中都修改参数值，此时只能修改动态的配置参数。参数为 2 表示只在 INI 文件中修改配置参数，此时可用来修改静态配置参数和动态配置参数。只有具有 DBA 角色的用户才有权限调用 SF_SET_SYSTEM_PARA_VALUE。

DM 的动态 INI 参数分为系统级和会话级两种级别。会话级参数在服务器运行过程中被修改时，之前创建的会话不受影响，只有新创建的会话使用新的参数值。

1. SF_SET_SESSION_PARA_VALUE (paraname varchar(8187), value bigint)

设置某个会话级 INI 参数的值，设置的参数值只对本会话有效。

2. SP_RESET_SESSION_PARA_VALUE (paraname varchar(8187))

重置某个会话级 INI 参数的值,使得这个 INI 参数的值和系统 INI 参数的值保持一致。

3. SF_GET_SESSION_PARA_VALUE (paraname varchar(8187))

获得当前会话的某个会话级 INI 参数的值。

需要注意的是，在对参数进行调整前，DBA 应该深刻理解配置参数中每个参数的含义和对系统的影响，避免由于错误的调整导致影响整个系统对外提供正常服务。对于一些关键业务，在实际调整前，建议在测试系统上先进行试验，验证通过后再在生产系统上进行调整。

22.6 优化数据库布局

数据库的布局直接影响整个系统的 I/O 性能。通常情况下，DBA 应该遵循下述原则：

1. 日志文件放在独立的物理磁盘上，保持与数据文件分开存储；
2. 预先估算并分配好磁盘空间，避免运行过程中频繁扩充数据文件；
3. 系统中不同表空间尽量分布在不同的磁盘上，这样当数据分布在多个表空间时，可以充分利用不同磁盘的并行 I/O 能力；
4. 对于分区表，无论是水平分区还是垂直分区，也尽可能将不同的分区放到不同的表空间；
5. 对于分析型应用，数据库的页大小和簇大小都可以考虑取最大值，并且在采用列存储的情况下，应该尽可能让每列存放在独立的表

第23章 动态管理/性能视图

23.1 理解动态管理视图

达梦数据库中的动态性能视图能自动收集数据库中的一些活动信息，系统管理员根据这些信息可以了解数据库运行的基本情况，为数据库的维护和优化提供依据。动态性能视图信息也是数据库中数据字典的一部分，与我们平常所说的数据字典不同的是，平常意义上的数据字典是指静态数据字典信息，也即用户访问数据字典信息时，内容不会发生改变，而动态视图信息是随着数据库的运行随时更改，具有一定的即时性。

系统管理员为了更好地了解数据库的一些运行时信息，可以查询动态视图表。首先系统管理员需要知道达梦数据库中提供了多少动态视图，有哪些类型动态视图，以及这些动态视图的用途是什么。关于这些内容可以参考附录 2。

动态视图表与静态字典信息表命名方式不同，静态字典表一般以 SYS 为前缀，如系统用户表 SYSUSERS，而动态视图则以 V$为前缀，如缓冲池动态信息表 V$BUFFER。

23.2 使用动态管理视图

在 DM7 中，动态视图提供的系统信息主要分为以下几个方面：

1. 系统信息

包括数据库版本、实例、统计信息、资源限制信息、进程信息、全局索引 IID 信息、事件信息；涉及的动态视图有 V$SESSIONS、V$INSTANCE、V$RESOURCE_LIMIT、V$PROCESS、V$IID、V$SYSSTAT 等。

例如查看数据库中实例信息。

```
SELECT * FROM V$INSTANCE;
```

结果：

```
 NAME    HOST_NAME      SVR_VERSION  DB_VERSION        START_TIME
STATUS$ MODE$  OGUID      RAC_SEQNO         RAC_ROLE
 1  DMSERVER PC-201103131435 DM DATABASE SERVER
V7.1.2.180-BUILD(2013.09.13-34445TRUNC)
   DB VERSION: 0X70008 2013-09-16 09:10:38       OPEN    NORMAL  0   0     SLAVE
```

2. 存储信息

包括数据库信息、表空间信息、数据文件信息、日志相关信息；涉及的动态视图有 V$DATAFILE、V$DATABASE、V$TABLESPACE、V$HUGE_TABLESPACE、V$RLOGFILE 等。

例如查询表空间信息。

```
SELECT * FROM V$TABLESPACE;
```

结果：

```
ID  NAME    CACHE   TYPE$   STATUS$ MAX_SIZE    TOTAL_SIZE  FILE_NUM
ENCRYPT_NAME    ENCRYPTED_KEY
0   SYSTEM      1   0   0   1408    1   <NULL>  <NULL>
1   ROLL        1   0   0   16384   1   <NULL>  <NULL>
```

```
4   MAIN        1   0   0   16384   1   <NULL>  <NULL>
5   SYSAUX      1   0   0   16384   1   <NULL>  <NULL>
3   TEMP        2   0   0   1280    1   <NULL>  <NULL>
```

3. 内存管理信息

包括内存池使用情况、BUFFER 缓冲区信息、虚拟机信息、虚拟机栈帧信息；涉及的动态视图有 V$VPOOL、V$VMS、V$STKFRM、V$BUFFER、V$BUFFERPOOL、V$EXP_BUFFERPOOL、V$BUFFER_LRU_FIRST、V$BUFFER_UPD_FIRST、V$BUFFER_LRU_LAST、V$BUFFER_UPD_LAST、V$RLOGBUF、V$COSTPARA 等。

例如查询内存池 BUFFERPOOL 的页数、读取页数和命中率信息。

```
SELECT NAME,N_PAGES,N_LOGIC_READS,RAT_HIT FROM V$BUFFERPOOL;
```

结果：

```
行号    NAME      N_PAGES       N_LOGIC_READS         RAT_HIT
1       KEEP      1024          0                  1.0000000000E+000
2       RECYCLE   8192          52                 9.8113207547E-001
3       NORMAL    1280          1772               9.2726321298E-001
4       NORMAL    8960          7                  7.0000000000E-001
```

4. 事务信息

包括所有事务信息、当前事务可见的事务信息、事务锁信息（TID 锁、对象锁）、回滚段信息、事务等待信息；涉及的动态视图有 V$TRX、V$TRXWAIT、V$TRX_VIEW、V$LOCK、V$PURGE 等。

例如查询系统中上锁的事务、锁类型，以及表 ID 信息。

```
SELECT TRX_ID,LTYPE,LMODE,TABLE_ID,ROW_IDX FROM V$LOCK;
```

结果：

```
行号  TRX_ID    LTYPE       LMODE     TABLE_ID      ROW_IDX
1       932     OBJECT        IX       1149         0
2       933     OBJECT        IX       1149         0
3       932     OBJECT        IS       -1           0
4       933     OBJECT        IS       -1           0
5       932     OBJECT        IS       4            0
6       933     OBJECT        IS       4            0
7       933     OBJECT        IS       33555437     0
8       933     TID       X       0            933
9       932     TID       X       0            932
10      933     TID       S        0           932
```

5. 线程信息

包括所有活动线程信息、线程作业信息、线程锁信息、线程的资源等待信息；涉及的动态视图有 V$THREADS、V$LATCHES、V$WAIT_CLASS 等。

例如查看系统中资源等待的情况。

```
SELECT CLASS_NAME,TOTAL_WAITS,TIME_WAITED FROM V$WAIT_CLASS;
```

结果：

```
行号   CLASS_NAME     TOTAL_WAITS     TIME_WAITED
1       IO READING              0       0
2       ROW LOCK WAITING        0       0
```

```
    3       TABLE LOCK WAITING      0       0
    4       DICT LOCK WAITING       0       0
    5       TASK WAITING    0       0
    6       NETWORK WAITING 0       0
    7       REDO LOG FLUSH WAITING  1       392
    8       PAGE CONFLICT   0       0
    9       BUFFER WAITING  0       0
```

6. 历史模块

包括 SQL 历史信息、SQL 执行节点历史信息、检查点历史信息、命令行历史信息、线程等待历史信息、死锁历史信息、回滚段历史信息、运行时错误历史信息、PL/SQL 中执行 DDL 语句的历史信息、返回大数据量结果集的历史信息、所有活动过线程的历史信息；涉及的东有视图有 V$CKPT_HISTORY、V$CMD_HISTORY、V$DEADLOCK_HISTORY、V$PLSQL_DDL_HISTORY、V$PRE_RETURN_HISTORY、V$RUNTIME_ERR_HISTORY、V$WAIT_HISTORY、V$WTHRD_HISTORY、V$SQL_HISTORY、V$SQL_NODE_HISTORY、V$SQL_NODE_NAME 等。

例如查询系统执行的 SQL 历史信息。

```
    SELECT SESS_ID,TOP_SQL_TEXT,TIME_USED FROM V$SQL_HISTORY;
```

结果：

```
    SESS_ID         TOP_SQL_TEXT                TIME_USED
1   187744368       INSERT INTO T1 VALUES(5,0);     21707
```

7. 缓存信息

包括 SQL 语句缓存、执行计划缓存、结果集缓存、字典缓存信息、字典缓存中的对象信息、代价信息；涉及的动态视图有 V$CACHEITEM、V$CACHEPLN、V$CACHERS、V$CACHESQL、V$DICT_CACHE_ITEM、V$DICT_CACHE 等。

例如查看字典缓存的信息。

```
    SELECT * FROM V$DICT_CACHE;
```

结果：

```
    ADDR        POOL_ID      TOTAL_SIZE        USED_SIZE         DICT_NUM
1   0X0B56F070      1          5242880           113530            36
```

8. 会话信息

包括连接信息、会话信息；涉及的动态视图有 V$CONNECT、V$STMTS、V$SESSIONS 等。

例如查看会话信息。

```
    SELECT SESS_ID,SQL_TEXT,STATE,CREATE_TIME,CLNT_HOST FROM V$SESSIONS;
```

结果：

```
    SESS_ID         SQL_TEXT        STATE          CREATE_TIME         CLNT_HOST
1    187744368       SELECT SESS_ID,SQL_TEXT,STATE,CREATE_TIME,CLNT_HOST FROM
V$SESSIONS;    ACTIVE    2011-09-19 19:20:38.000000     FREESKYC-FB8846
```

9. 捕获信息

捕获信息涉及的视图为 V$CAPTURE。

例如查看捕获信息。

```
    SELECT * FROM V$CAPTURE;
```

结果：

```
    STATE   VERSION    MSG_NUM   FILE_PATH     INIT_TIME   DURATION
FLUSH_BUF_NUM          FREE_BUF_NUM
    1       1        0       0      1970-01-01 08:00:00     0        0       0
```

查看动态视图可以不单单只查询一个动态视图表，还可利用动态视图表之间的联系得到更多想要的信息。

例如，系统管理员如果要对一条 SQL 语句进行调优，需要知道每个执行节点花费了多少时间，查询 V$SQL_NODE_NAME 可以知道执行节点的名字，查询 V$SQL_NODE_HISTORY 可以查询到每个执行节点的时间，通过两个动态视图表的执行节点类型 TYPE$字段做等值连接。

如执行一条 SQL 语句，然后查询其执行节点所花费时间，假设其执行 ID（EXEC_ID）为 4。

```
    SELECT * FROM t1 WHERE c1 = (SELECT d1 FROM t2 WHERE c2 = d2);
```

通过视图 V$SQL_NODE_NAME 与 V$SQL_NODE_HISTORY 视图查询结点执行时间：

```
    SELECT N.NAME, TIME_USED, N_ENTER FROM V$SQL_NODE_NAME N, V$SQL_NODE_HISTORY
H WHERE N.TYPE$ = H.TYPE$ AND EXEC_ID = 4;
```

结果为：

```
           NAME    TIME_USED     N_ENTER
    1      CSCN2   381     6
    2      CSCN2   250     3
    3      NLI2    52      14
    4      SLCT2   102     8
    5      HAGR2   11831   6
    6      PRJT2   32      6
    7      CSCN2   272     3
    8      HI3     19309   9
    9      PRJT2   29      6
    10     NSET2   120     4
    11     DLCK    23      2
```

根据结果可以看到执行计划中各执行节点花费的时间，进而对 SQL 语句进行分析和改写。

第24章 查询优化

数据库执行一条语句有多种方式，为了选择最优的执行方式，产生了查询优化器。查询优化器分析语句运行时的所有因素，选择最优的方式去执行，提高了查询效率。因此，查询优化是数据库执行 SQL 语句的重要过程，决定了数据库的查询性能。

24.1 优化目标

达梦数据库查询优化器的优化目标为最快响应时间。通过设置参数 FIRST_ROWS 来决定优先返回多少条记录给用户，而不需要等待全部结果确定后再输出，FIRST_ROWS 设置范围为 1~1000，单位为行。例如：FIRST_ROWS = 10，意思是查询出 10 条结果就立即返回给用户。可以根据实际情况，调整参数值。

24.2 查询优化器

查询优化器通过分析可用的执行方式和查询所涉及的对象统计信息来生成最优的执行计划。此外，如果存在 HINT 优化提示，优化器还需要考虑优化提示的因素。

查询优化器的处理过程包括：

1. 优化器生成所有可能的执行计划集合；
2. 优化器基于字典信息的数据分布统计值、执行语句涉及到的表、索引和分区的存储特点来估算每个执行计划的代价。代价是指 SQL 语句使用某种执行方式所消耗的系统资源的估算值。其中，系统资源消耗包括 I/O、CPU 使用情况、内存消耗等；
3. 优化器选择代价最小的执行方式作为该条语句的最终执行计划。

优化器所做的操作有：查询转换、估算代价、生成计划。

24.2.1 查询转换

查询转换是指把经过语法、语义分析的查询块之间的连接类型、嵌套关系进行调整，生成一个更好的查询计划。常用的查询转换技术包括过滤条件的下放、相关子查询的去相关性。

1. 过滤条件下放：在连接查询中，把部分表的过滤条件下移，在连接之前先过滤，可以减少连接操作的数据量，提升语句性能；
2. 相关子查询的去相关性：把与子查询相关的外表与内表采用半连接的方式执行，放弃默认采取的嵌套连接方式，对性能有较大提升。

24.2.2 估算代价

估算代价是指对执行计划的成本进行估算。执行节点之间的代价值相关性较强，一个执行节点的代价包括该节点包含的子节点代价。代价衡量指标包括选择率、基数、代价。

选择率是指满足条件的记录占总记录数的百分比。记录集可以是基表、视图、连接或分

组操作的结果集。选择率和查询谓词相关，如 name = ‘韩梅梅’；或者是谓词的连接，如 name =‘韩梅梅’ and no =‘0123’。一个谓词可以看作是一个过滤器，过滤掉结果集中不满足条件的记录。选择率的范围从 0 到 1。其中，0 表示没有记录被选中，1 表示行集中所有记录都被选中。

如果没有统计信息，则优化器依据过滤条件的类型来设置对应的选择率。例如，等值条件的选择率低于范围条件选择率。这些假定是根据经验值，认为等值条件返回的结果集最少。

如果有统计信息，则可以使用统计信息来估算选择率。例如，对于等值谓词（name =‘韩梅梅’），如果 name 列有 N 个不同值，那么，选择率是 N 分之一。

基数是指整个行集的行数，该行集可以是基表、视图、连接或分组操作的结果集。

代价表示资源的使用情况。查询优化器使用磁盘 I/O、CPU 占用和内存使用作为代价计算的依据，所以代价可以用 I/O 数、CPU 使用率和内存使用一组值来表示。所有操作都可以进行代价计算，例如扫描基表、索引扫描、连接操作或者对结果集排序等。

访问路径决定了从一个基表中获取数据所需要的代价。访问路径可以是基表扫描、索引扫描等。在进行基表扫描或索引扫描时，一次 I/O 读多个页，所以，基表扫描或索引全扫描的代价依赖于表的数据页数和多页读的参数值。二级索引扫描的代价依赖于 B 树的层次、需扫描的叶子块树以及根据 rowid 访问聚集索引的记录数。

连接代价是指访问两个连接的结果集代价与连接操作的代价之和。

24.2.3 生成计划

生成计划指计划生成器对给定的查询按照连接方式、连接顺序、访问路径生成不同的执行计划，选择代价最小的一个作为最终的执行计划。

连接顺序指不同连接项的处理顺序。连接项可以是基表、视图、或者是一个中间结果集。例如表 t1、t2、t3 的连接顺序是先访问 t1，再访问 t2，然后对 t1 与 t2 做连接生成结果集 r1，最后把 t3 与 r1 做连接。一个查询语句可能的计划数量是与 FROM 语句中连接项的数量成正比的，随着连接项的数量增加而增加。

24.3 数据访问路径

访问路径指从数据库中检索数据的方法。一般情况下，索引访问用于检索表的小部分数据，全表扫描用于访问表的大部分数据。OLTP 应用中，一般使用索引访问路径，因为 OLTP 中包含了许多高选择率的 SQL 语句。而决策支持系统则倾向于执行全表扫描来获取数据。从数据库中定位和检索数据的方法有：全表扫描、聚集索引扫描、二级索引扫描等。

全表扫描是指从基表中检索数据时，扫描该表中所有的数据。全表扫描方式适合检索表中大部分数据，这时比索引扫描更加有效率。

索引扫描是指通过指定语句中的索引列进行遍历来检索表中的数据。索引扫描是从基于一列或多列的索引中检索数据。索引不仅包含索引值，还包含对应表中数据的 ROWID。如果需要访问的不是索引列，这时需要通过 ROWID 或聚集索引来找到表中的数据行。

索引扫描包含聚集索引扫描和二级索引扫描。由于在聚集索引中，包含了表中所有的列值，所以检索数据时只需要扫描这一个索引就可以得到所有需要的数据。如果是二级索引，由于只包含索引列以及对应的 ROWID，如果查询列不在二级索引中则还需要扫描聚集索引来得到所需要的数据。

查询优化器选择访问路径基于以下几个因素：

1．执行语句中可能的访问路径；

2．估算每条执行路径的代价。为了选择一个访问路径，优化器首先会通过检查语句中的 FROM 子句和 WHERE 子句中的条件表达式来决定哪一个访问路径可以使用。优化器会根据可用的访问路径生成可能的执行计划集合，然后使用索引、列和表的统计信息来估算每个计划的代价。最后，优化器选择最小代价的那个执行计划。

影响优化器选择访问路径的因素有语句中的提示（HINT）和统计信息。用户可以在执行的语句中使用 HINT 来指定访问路径。而统计信息会根据表中数据的分布情况决定采用哪个访问路径会产生最小的代价。

24.4 连接

查询语句中 FROM 子句包含多个表时,我们称为连接查询。如 SELECT * FROM t1,t2 就是连接查询。

生成连接查询的执行计划，需要考虑三方面因素：

1. 访问路径

对于每张表采用何种方式来获取数据。例如：全表扫描、索引扫描等。

查询优化器会估算每种扫描方式的代价，选择代价较小的访问路径。

2. 连接方式

确定两张表之间采用哪种连接方式。例如：哈希连接、嵌套连接、归并连接、外连接。

等值连接条件一般会选择哈希连接；非等值连接条件会采用嵌套连接；连接列均为索引列时，会采用归并连接。

1) 嵌套连接：两张表进行非等值连接时会选择嵌套连接。相当于两张表进行笛卡尔集操作。此时，优化器会选择一张代价较小的表作为外表(驱动表)，另一张表作为内表，外表的每条记录与内表进行一次连接操作。

2) 哈希连接：两张表进行等值连接时会选择哈希连接。以一张表的连接列为哈希键，构造哈希表，另张表的连接列进行哈希探测，找到满足条件的记录。由于哈希命中率高，因此，在大数据量情况下，哈希连接的效率较高。哈希连接的代价是建立哈希表和哈希探测的代价。

3) 归并连接：两张表的连接列均为索引列，则可以按照索引顺序进行归并，一趟归并就可以找出满足条件的记录。如果查询列也属于索引列的子集，则归并连接只需扫描索引，会有更好的性能表现。在两表连接条件不是等值（如<，<=，>，>=）情况下时，归并排序连接很有用。

4) 外连接：外连接分为左外连接、右外连接、全外连接。作为外表的数据会全部返回，如果没有与外表匹配的记录，则填充 NULL 值。右外连接与左外连接的处理过程类似，只是外表不同，一个是左表，一个是右表。全外连接是进行左外连接和右外连接，返回两次外连接的 union 结果集。

例 1：左外连接：

```
SELECT * FROM t1 LEFT OUTER JOIN t2 ON t1.c1=t2.d1;
```

例 1 中 t1 表为外表(左表)，如果 t2 表中不存在与 t1.c1 相等的记录，则 t2 表的该行记录用 NULL 填充。右外连接与左外类似。

例 2：全外连接

```
SELECT * FROM t1 FULL OUTER JOIN t2 ON t1.c1=t2.d1;
```

例 2 中分别以 t1 为左表进行左外和右外连接，两次结果进行 union，返回最终

结果。

子查询会转换成半连接。共有四种半连接方式：哈希半连接、索引半连接、嵌套半连接、归并半连接。等值连接条件会选择哈希\索引\归并半连接，非等值连接条件会选择嵌套半连接。

1) 哈希半连接：以外表的连接列为 KEY 构造哈希表，内表的连接列进行探测来查找满足连接条件的记录；

2) 索引半连接：如果子查询的连接列为索引前导列，可采用索引半连接。处理过程为外表的数据对子查询使用索引查找，返回满足条件的记录；

3) 归并半连接：如果相关子查询的连接条件列均为索引列，可采用归并半连接。按照索引顺序，对外表、内表进行同步扫描，返回满足条件的记录；

4) 嵌套半连接：如果连接条件为非等值，可转换为嵌套半连接。处理过程为外表的每条记录去遍历内表，返回满足条件的记录。

3. 连接顺序

当超过 2 张表进行连接时，就需要考虑表之间的连接顺序。不合适的连接顺序对执行效率有较大影响。一般原则是，经过连接可以产生较小结果集的表优先处理。

一个连接查询通常会对应多个执行计划，查询优化器会根据优化规则、代价估算挑选最优的执行计划。

24.5 统计信息

对象统计信息描述数据是如何在数据库中存储的。统计信息是优化器的代价计算的依据，可以帮助优化器较精确地估算成本，对执行计划的选择起着至关重要的作用。

达梦数据库的统计信息分三种类型：表统计信息、列统计信息、索引统计信息。通过直方图来表示。统计信息生成过程分以下三个步骤：

1. 确定采样的数据：根据数据对象，确定需要分析哪些数据。
   1） 表：计算表的行数、所占的页数目、平均记录长度
   2） 列：统计列数据的分布情况
   3） 索引：统计索引列的数据分布情况
2. 确定采样率
   根据数据对象的大小，通过内部算法，确定数据的采样率。采样率与数据量成反比。
3. 生成直方图
   有两种类型的直方图：频率直方图和等高直方图。根据算法分析表的数据分布特征，确定直方图的类型。频率直方图的每个桶(保存统计信息的对象)的高度不同，等高直方图每个桶的高度相同。例如，对列生成统计信息，当列值分布比较均匀时，会采用等高直方图，否则，采用频率直方图。

在执行查询时，如果数据对象存在统计信息，代价算法可以根据统计信息中的数据，比较精确地计算出操作所需花费的成本，以此来确定连接方式、对象访问路径、连接顺序，选择最优的执行计划。

用户也可以通过修改 OPTIMIZER_DYNAMIC_SAMPLING 参数值在缺乏统计信息时进行动态统计信息收集。

24.6 执行计划

执行计划是 SQL 语句的执行方式，由查询优化器为语句设计的执行方式，交给执行器去执行。在 SQL 命令行使用 EXPLAIN 可以打印出语句的执行计划。

例如：

建表和建索引语句：

```
CREATE TABLE T1(C1 INT,C2 CHAR);
CREATE TABLE T2(D1 INT,D2 CHAR);
CREATE  INDEX  IDX_T1_C1  ON  T1(C1);
INSERT INTO T1 VALUES(1,'A');
INSERT INTO T1 VALUES(2,'B');
INSERT INTO T1 VALUES(3,'C');
INSERT INTO T1 VALUES(4,'D');
INSERT INTO T2 VALUES(1,'A');
INSERT INTO T2 VALUES(2,'B');
INSERT INTO T2 VALUES(5,'C');
INSERT INTO T2 VALUES(6,'D');
```

打印执行计划：

```
EXPLAIN SELECT A.C1+1,B.D2 FROM T1 A, T2 B WHERE A.C1 = B.D1;
```

执行计划如下：

```
1  #NSET2: [0, 16, 9]
2    #PRJT2: [0, 16, 9]; EXP_NUM(2), IS_ATOM(FALSE)
3      #NEST LOOP INDEX JOIN2: [0, 16, 9]
4        #CSCN2: [0, 4, 5]; INDEX33555535(B)
5        #SSEK2: [0, 4, 0]; SCAN_TYPE(ASC), IDX_T1_C1 (A),
SCAN_RANGE[T2.D1,T2.D1]
```

这个执行计划看起来就像一棵树，执行过程为：控制流从上向下传递，数据流从下向上传递。其中，类似[0, 16, 9]这样的三个数字，分别表示估算的操作符代价、处理的记录行数和每行记录的字节数。同一层次中的操作符，如本例中的 CSCN 和 SSEK2，由父节点 NEST LOOP INDEX JOIN 控制它们的执行顺序。

该计划的大致执行流程如下：

1) CSCN2：扫描 T2 表的聚集索引，数据传递给父节点索引连接；

2) NEST LOOP INDEX JOIN2：当左孩子有数据返回时取右侧数据；

3) SSEK2：利用 T2 表当前的 D1 值作为二级索引 IDX_T1_C1 定位查找的 KEY，返回结果给父节点；

4) NEST LOOP INDEX JOIN2：如果右孩子有数据则将结果传递给父节点 PRJT2，否则继续取左孩子的下一条记录；

5) PRJT2：进行表达式计算 C1+1, D2；

6) NSET2：输出最后结果；

7) 重复过程 1) ~ 4)直至左侧 CSCN2 数据全部取完。

用户如果想了解更多关于操作符的知识，请查看动态视图 V$SQL_NODE_NAME，手册的附录 4 给出了常用操作符的说明。

24.7 使用索引

为了提高查询效率，用户一般会在表中创建索引。查询中的条件列为索引列时，如果索引扫描代价最小，优化器就会采用索引扫描。索引扫描有多种方式，例如，索引等值查询、索引范围查询。如果查询列属于索引列的子集，则通过索引扫描就可以获得数据，否则，还需要根据 ROWID 或者 PK 在聚集索引中定位记录。

常用的索引类型有唯一索引、组合索引、函数索引。各自有不同的使用场景。

1. 条件列具有 UNIQUE 约束，则可以创建唯一索引，减少索引扫描次数；
2. 条件列是多个列，而且可以过滤掉大部分数据，可以在多个列上创建组合索引，把等值条件列作为组合索引的首列；
3. 条件列使用确定性函数(同样环境下多次执行得到相同的结果)，可以创建函数索引，会把函数值进行存储，使用方式与普通索引一样；
4. 在空间数据应用中，可以创建空间索引提高空间查询的效率。

24.8 并行查询

24.8.1 并行查询概念

倘若没有并行查询技术，一个串行执行的查询语句只能利用 CPU 或者磁盘设备中的一个，而不能利用整个计算机的处理能力。并行查询技术的出现，使得单个 SQL 语句能利用多个 CPU 和磁盘设备的处理能力。其优势在于可以通过多个线程来处理查询任务，从而提高查询的效率。

达梦数据库为具有多个 CPU 的数据库服务器提供并行查询的功能，以优化查询任务的性能。数据库服务器只有具有多个 CPU，才能使用并行执行查询操作，来提高查询任务的速度。

达梦数据库通过三个步骤来完成并行查询：首先，确定并行任务数；其次，确定并行工作线程数；最后，执行查询。并行查询相关参数见下表：

表 24.1 并行查询相关参数

参数名	缺省值	属性	说明
MAX_PARALLEL_DEGREE	1	动态，会话级	用来设置默认并行任务个数。取值范围：1~128。缺省值 1，表示无并行任务。当 PARALLEL_POLICY 值为 1 时该参数值才有效。
PARALLEL_POLICY	0	静态	用来设置并行策略。取值范围：0、1 和 2，缺省为 0。其中，0 表示不支持并行；1 表示自动并行模式；2 表示手动并行模式。
PARALLEL_THRD_NUM	10	静态	用来设置并行工作线程个数。取值范围：1~1024。

24.8.2 确定并行任务个数

当开启自动并行（PARALLEL_POLICY=1）时，参数 MAX_PARALLEL_DEGREE 生效，控制并行查询最多使用的线程数。MAX_PARALLEL_DEGREE 缺省值为 1，表示不并行。此时若指定参数对应的 HINT “PARALLEL”，则使用 HINT 值；

当开启手动并行（PARALLEL_POLICY=2）时，参数 MAX_PARALLEL_DEGREE 失效，用户需要在语句中使用此参数对应的 HINT “PARALLEL”指定语句的并行度，否则不并行。

1. 在 INI 参数中设置默认值

INI 参数 MAX_PARALLEL_DEGREE 设置最大并行任务个数。取值范围：1~128。缺省值 1，表示无并行任务，此参数仅在 PARALLEL_POLICY 值为 1 时才有效。

例如，在 INI 参数中将 MAX_PARALLEL_DEGREE 设置为 3 的格式如下：

```
MAX_PARALLEL_DEGREE      3
```

然后，使用一般的 SQL 语句查询即可执行并行查询，不需要使用 HINT。如：

```
SELECT * FROM SYSOBJECTS;
```

2. 在 SQL 语句中使用“PARALLEL”关键字特别指定

当 PARALLEL_POLICY=2 时，需要在 SQL 语句中通过“PARALLEL”HINT 指定并行度，否则不并行。若 PARALLEL_POLICY=1，则 SQL 语句中使用的“PARALLEL”HINT 总是优先于 MAX_PARALLEL_DEGREE 参数设置。

“PARALLEL”关键字的用法是在数据查询语句的 SELECT 关键字后，增加 HINT 子句来实现。

HINT 语法格式如下：

```
/*+ PARALLEL([<表名>] <并行任务个数>) */
```

例如，下面的例子中，即使已经设置了 MAX_PARALLEL_DEGREE 默认值 3，但实际使用的为 PARALLEL 指定的任务个数 4：

```
SELECT /*+ PARALLEL(4) */ * FROM SYSOBJECTS;
```

另外，每个语句中仅能设置一次并行任务个数，如果设置了多次，则以最后一次设置为准，而且任务个数在全语句中生效。

例如，下面的例子中，使用的并行任务个数为 2。

```
SELECT /*+ PARALLEL(1) *//*+ PARALLEL(2) */ * FROM SYSOBJECTS;
```

这种方式能够为单条查询语句设置额外的并行任务个数，以此来提高某些特殊查询任务的性能。

24.8.3 确定并行工作线程数

在执行并行查询任务之前，您需要指定完成该任务的并行工作线程数。值得注意的是，实际使用的线程数并非总是等于并行工作线程数。并行工作线程数是在 INI 参数中设定的，实际使用并行工作线程数是根据系统的实际状况确定的。

1. 并行工作线程数，在 INI 参数中设定

首先，使用 PARALLEL_POLICY 参数来设置并行策略。取值范围：0、1 和 2，默认值 0。其中，0 表示不支持并行；1 表示自动并行模式；2 表示手动并行模式。

当开启本地并行（PARALLEL_POLICY>0）时，使用 PARALLEL_THRD_NUM 指定本地并行查询使用的线程数，取值范围为 1~1024，缺省值为 10。需要注意的是，若 PARALLEL_POLICY=1，如果 PARALLEL_THRD_NUM=1，则按照 CPU 个数创建并行线程。

例如，设置并行策略 PARALLEL_POLICY 为 2，即手动设置并行工作线程数；同时，设置并行工作线程数 PARALLEL_THRD_NUM 为 4 个。

```
PARALLEL_POLICY                    2
PARALLEL_THRD_NUM                  4
```

当然，并非所有的查询都适合使用并行查询。大量占用 CPU 周期的查询最适合采用并行查询的功能。例如，大型表的连接查询、大量数据的聚合和大型结果集的排序等都很适合采用并行查询。对于简单查询（常用于事务处理应用程序）而言，执行并行查询所需的额外协调工作会大于潜在的性能提升。所以，数据库管理员在确定是否需要使用并行策略的时候，

需要慎重。

2. 实际使用的线程数，达梦数据库会根据每个并行查询操作自动检测

实际使用线程数是数据库在查询计划执行时初始化的时候确定的。也就是说，这不需要用户去干预，而是系统根据并行任务数和实际空闲的并行工作线程数来确定的。此操作所依据的条件如下：首先，检测达梦数据库是否运行在具有多个CPU的计算机上。只有具有多个CPU 的计算机才能使用并行查询。这是一个硬性的限制条件。其次，检测可用的空闲工作线程是否足够。并行查询到底采用多少线程数，除了跟操作的复杂程度相关外，还跟当时的服务器状态相关，如是否有足够的可用的空闲工作线程数量等。每个并行查询操作都要求一定的工作线程数量才能够执行；而且执行并行计划比执行串行计划需要更多的线程，所需要的线程数量也会随着任务个数的提高而增加。当无法满足特定并行查询执行的线程要求时，数据库引擎就会自动减少任务个数，甚至会放弃并行查询而改为串行计划。所以，即使同一个操作在不同时候可能会采用不同的线程数。

例如，即使设置并行工作线程数为4。而实际使用的线程数可能只有3个，或者更少。

24.8.4 执行查询

当以上内容确定好之后，数据库就会执行具体的查询任务。

24.8.5 使用场景

使用手动并行模式时，只需要在 INI 参数中设置好如下 2 个参数，然后执行并行 SQL 查询语句时，需手动指定当前并行任务个数。若不指定，将不使用并行。设置的 2 个参数如下：

```
PARALLEL_POLICY     2
PARALLEL_THRD_NUM    4
```

使用自动并行模式时，一般指定三个如下三个参数：

```
MAX_PARALLEL_DEGREE     3
PARALLEL_POLICY    1
PARALLEL_THRD_NUM    10
```

另外，当 PARALLEL_POLICY 为 0 时，即使有并行任务，也不支持并行。

然后，执行语法格式类似“SELECT * FROM SYSOBJECTS;”的并行 SQL 语句即可，本条语句使用默认并行任务数 3。

当然，如果单条查询语句不想使用默认并行任务数，可以通过在 SQL 语句中增加 HINT，通过“PARALLEL”关键字来特别指定。此时，执行的并行 SQL 语句格式为“SELECT /*+ PARALLEL(SYSOBJECTS 4) */ * FROM SYSOBJECTS;”，本条语句使用的并行任务数为 4。

24.9 查询计划重用

如果同一条语句执行频率较高，或者每次执行的语句仅仅是常量值不同，则可以考虑使用计划重用机制。避免每次执行都需要优化器进行分析处理，可以直接从计划缓存中获取已有的执行计划，减少了分析优化过程，提高执行率。

对于计划重用，达梦数据库提供了 INI 参数 USE_PLN_POOL 来控制，当置为 1 时，会启用计划重用。

24.10 结果集重用

执行计划的生成与优化是一个非常依赖 CPU 的操作，而执行一个查询获得结果集也是一个非常消耗资源的操作。当系统连续执行两个完全相同的 SQL 语句，其执行计划和结果集很有可能是相同的，如果重新生成和执行计划，会大大浪费系统资源。这时如果使用计划重用和结果集重用，系统的响应速度可以大大提升。

结果集重用是基于计划重用的，如果查询的计划不能缓存，则其查询结果集必然不能缓存。此外，当语句的游标属性为 FORWARD ONLY 时，默认查询不会生成结果集。而参数 BUILD_FORWARD_RS 可以强制在此类查询中生成结果集，以便进行结果集重用。

可通过设置 INI 参数 RS_CAN_CACHE 来控制结果集重用。当置为 0 时表示手动模式（MANUAL），在此模式下默认不缓存查询结果集，但是 DBA 可以通过语句提示等方法指示系统对必要的查询结果集进行缓存；置为 1 时表示强制模式（FORCE），在此模式下默认缓存所有可缓存结果集，但是 DBA 也可以通过新增的配置参数以及语句提示等方法取消某些不合适的结果集缓存。

当 RS_CAN_CACHE 为 1 时，还可以通过设置 INI 参数 RS_CACHE_TABLES 和 RS_CACHE_MIN_TIME 对缓存的结果集进行限制和过滤。RS_CACHE_TABLES 指定可以缓存结果集的基表清单，只有查询涉及的所有基表全部在参数指定范围内，此查询才会缓存结果集。RS_CACHE_MIN_TIME 则指定了缓存结果集的查询语句执行时间的下限，只有实际执行时间不少于指定值的查询结果集才会缓存。

DBA 可以通过在 SQL 语句中设置 “RESULT_CACHE”或“NO_RESULT_CACHE” HINT 手动指示查询的结果集是否缓存。如：

```
select /*+ RESULT_CACHE */ id, name from sysobjects;
```

或者

```
select /*+ NO_RESULT_CACHE */ id, name from sysobjects;
```

在语句中使用 HINT 指定结果集缓存的优先级要高于 INI 中相关参数的设置。

还可以使用系统过程 SP_SET_PLN_RS_CACHE 来强制设置指定计划结果集缓存的生效及失效。这个系统过程对结果集缓存的指定高于其它所有结果集缓存的设置。

在以下情况下，DM 不支持结果缓存：

1. 必须是单纯的查询语句计划，PL 脚本中包含查询语句也不能缓存结果集。
2. 查询语句的计划本身必须是缓存的。
3. 守护环境中的备库不支持结果集缓存。
4. MPP 等集群环境下不支持结果集缓存（3、4 两点限制都是因为无法精确控制基表的数据更新时戳）。
5. 查询语句中包含以下任意一项，其结果集都不能缓存：
   1) 包含临时表；
   2) 包含序列的 CURVAL 或 NEXTVAL；
   3) 包含非确定的 SQL 函数或包方法(现有逻辑是不支持所有 SQL 函数或包方法)；
   4) 包含 RAND、SYSDATE 等返回值实时变化的系统函数；
   5) 包含其它的一些实时要素。

第25章 SQL 调优

25.1 简介

SQL 调优作为数据库性能调优中的最后一个环节，对查询性能产生着直接的影响。在进行正式的 SQL 调优前，用户首先要关注下列几点：

1．达梦数据库安装时的配置参数是否符合应用场景需求；

2．达梦数据库的 INI 配置文件中各项参数是否已经处于最优配置；

3．应用系统中数据库设计是否合理。

本章将介绍定位高负载的 SQL 语句的方法，利用自动 SQL 调整功能进行优化，以及如何开发有效的 SQL 语句和使用优化器提示来影响执行计划。

25.2 调优目标

SQL 调优的整体目标是使用最优的执行计划，这意味着 IO 以及 CPU 代价最小。具体而言调优主要关注下列方面：

- **表扫描**

如果计划中对某大表使用了全索引扫描，那么用户需要关注是否存在着该表的某个查询条件使得过滤后可以淘汰至少一半的数据量。通过添加相应的索引，全索引扫描可能被转换为范围扫描或等值查找。添加的二级索引可以包含该表上所有被选择项以避免 BLKUP 操作符的查找操作带来的第二次 IO 开销，但无疑这会增加二级索引的大小。用户需权衡二者的利弊以选择正确的处理方式。

- **连接操作的顺序和类型**

多表连接时，不同的连接顺序会影响中间结果集数量的大小，这时调优的目标就是要找到一种能使中间结果保持最小的连接顺序。

对于给定的一个连接或半连接，DM7 可以用 HASH 连接、嵌套循环连接、索引连接或者是归并连接实现。通过分析表的数据量大小和索引信息，SQL 调优目标是选择最适宜的操作符。

对半连接而言，HASH 连接还可细分为左半 HASH 和右半 HASH。用户可以通过始终对数据量小的一侧建立 HASH 来进行调优。

- **分组操作**

分组操作往往要求缓存所有数据以找到属于同一组的所有数据，在大数据量情况下这会带来大量的 IO。用户应该检查 SQL 查询和表上索引信息，如果可以利用包含分组列的索引，那么执行计划就会使用排序分组从而不用缓存中间结果。

25.3 确定高负载的 SQL

在设置性能监视级别（ENABLE_MONITOR）为大于 1 后，可以通过查询动态视图 V$LONG_EXEC_SQLS 或 V$SYSTEM_LONG_EXEC_SQLS 来确定高负载的 SQL 语句。前者显示最近 1000 条执行时间较长的 SQL 语句，后者显示服务器启动以来执行时间最长的 20 条 SQL 语句。例如：

```
SELECT * FROM V$LONG_EXEC_SQLS;
```

或者

```
SELECT * FROM V$SYSTEM_LONG_EXEC_SQLS;
```

25.4 自动 SQL 调整

使用查询优化向导工具，输入需要进行调整的 SQL 语句，向导工具将在分析完执行计划后给出推荐索引的提示。用户只需按提示建立相应索引即可。

25.5 开发有效的 SQL 语句

SQL 语言是一种相当灵活的结构化查询语言。用户可以利用多种不同形式的查询语句完成相同的查询功能。为了使执行效率达到最优，用户需要参考以下原则以开发出有效的 SQL 语句：

1．避免使用 OR 子句

OR 子句在实际执行中会被转换为类似于 UNION 的查询。如果某一个 OR 子句不能利用上索引则会使用全表扫描造成效率低下，应避免使用。

如果 OR 子句都是对同一列进行过滤，用户可以考虑使用 IN VALUE LIST 的过滤形式。如：

```
SELECT ... WHERE CITY = 'SHANGHAI' OR CITY = 'WUHAN' OR CITY = 'BEIJING';
```

调整为

```
SELECT ... WHERE CITY IN( 'SHANGHAI','WUHAN','BEIJING');
```

2．避免使用困难的正则表达式

在 SQL 语言中，LIKE 关键字支配通配符匹配，含通配符的表达式被称为正则表达式。有的正则表达式可以自动优化为非匹配的。例如：a LIKE 'L%'可以优化为 a>='L' AND a <'M',这样就可以用到 a 上的索引。即使没有索引，转换后的比较也更快。再如：a LIKE 'LM_'可以转化为 a>='LM' AND a<'LN' AND a LIKE 'LM_'。虽然仍然包含着通配符匹配，但大大缩小了匹配的范围。

所谓困难的正则表达式是指开头和结尾都为通配符的正则表达式，如'_L%'、'%L_'，优化器没办法缩小它们的匹配范围，也不可能用到索引而必须使用全表扫描。因此要尽可能避免这样的正则表达式。

如果仅仅是开头为通配符，用户可以在列 a 上建立 REVERSE(a)这样一个函数索引，利用函数索引反转待匹配项从而使用函数索引进行范围扫描。

3．灵活使用伪表(SYSDUAL)

首先可以利用伪表进行科学计算，执行语句 SELECT 3*4 FROM SYSDUAL，则可以得到结果 12；

其次，在某些方面使用 SYSDUAL 可提高效率。例如：查询过程中要判断表 t1 中是否有满足 condition1 条件的记录存在，可执行以下语句：

```
SELECT COUNT(*) INTO x FROM t1 WHERE condition1;
```

然后根据变量 x 的取值来判断。但是当 t1 非常大时该语句执行速度很慢，而且由于不知道 SELECT 返回的个数，不能用 SELECT *代替。事实上这个查询可以利用伪表来完成：

```
SELECT 'A' INTO y FROM SYSDUAL
WHERE EXISTS (SELECT 1 FROM t1 WHERE condition1);
```

判断 x 值，如等于'A'则 T1 中有记录。调整后的语句执行速度明显比上一句高。

另外，在 DM7 的语法里是可以省略 FROM 子句的，这时系统会自动加上 FROM SYSDUAL。

因此前面的科学计算例子可以简化为 SELECT 3*4;

4．SELECT 项避免‘*’

除非用户确实要选择表中所有列，否则 SELECT *这种写法将让执行器背上沉重的负荷。因为每一列的数据不得不自下往上层层向上传递。不仅仅如此，如果用户查询的是垂直分区表，那么更大的麻烦在于垂直分区表的所有子表都要参与连接操作；如果用户查询的是列存储表，那么列存储所带来的 IO 优势将损耗殆尽。

任何时候，用户都要了解表结构和业务需求，小心地选择需要的列并一一给出名称，避免直接用 SELECT *。

5．避免功能相似的重复索引

索引并未越多越好。抛开优化器面对众多索引逐一试探所耗费的时间不谈，如果表上增删改操作频繁，那么索引的维护将会成为大麻烦，尤其是函数索引的计算开销更不能忽略。

6．使用 COUNT(*)统计结果行数

如果对单表查询 COUNT(*)且没有过滤条件，那么 DM7 优化器会直接读取相关索引中存储的行数信息，加以回滚段中其他事务插入或删除元组的行数修正，迅速地给出最终结果而避免对实际数据的读取。相比之下，COUNT(列名)会对数据进行读操作，执行效率远低于 COUNT(*)。

即使查询中含有过滤条件，由于 DM7 特有的批处理方式，COUNT(*)依旧快于其他写法。这是因为 COUNT(*)无需取得行的具体值而仅仅需要行数这一信息。

需要额外说明的是，COUNT(*)会将 NULL 值计算在内而 COUNT(列名)是不包含 NULL 值的，因此用户要结合应用场景决定是否可以使用 COUNT(*)。

7．使用 EXPLAIN 来查看执行计划

在查询语句或者插入、删除、更新语句前增加 EXPLAIN 关键字，DM7 将显示其执行计划而无需实际执行它。查阅 V$SQL_NODE_NAME 表中每个操作符的含义，用户可以很方便且直观地了解数据如何被处理及传递。如果启用了统计信息收集，那么对照执行计划和对动态视图 V$SQL_NODE_HISTORY，V$SQL_NODE_NAME 的查询结果，用户就可以知道在实际执行中每一个操作符执行的时间，进而找出性能瓶颈。

8．UNION 和 UNION ALL 的选择

UNION 和 UNION ALL 的区别是前者会过滤掉值完全相同的元组，为此 UNION 操作符需要建立 HASH 表缓存所有数据并去除重复，当 HASH 表大小超过了 INI 参数指定的限制时还会做刷盘。

因此如果应用场景并不关心重复元组或者不可能出现重复，那么 UNION ALL 无疑优于 UNION。

9．优化 GROUP BY ... HAVING

GROUP BY 最常见的实现有 HASH 分组（HAGR）和排序分组（SAGR）。前者需要缓存中间结果；如果用户在 GROUP BY 的列上建立索引，那么优化器就会判断并可能使用上该索引，这时的 GROUP BY 就会变为 SAGR。

HAVING 是分组后对结果集进行的过滤，如果过滤条件无关集函数操作，用户可以考虑将过滤条件放在 WHERE 而不是 HAVING 中。DM7 优化器会判断并自动转换部分等效于 WHERE 的 HAVING 子句，但显式地给出最佳 SQL 语句会让优化器工作得更好。

10．使用优化器提示（HINT）

利用经验对优化器的计划选择进行调整，HINT 是 SQL 调整不可或缺的一步。我们将在下一节给出详细说明。

25.6 使用优化器提示

DM 查询优化器采用基于代价的方法。在估计代价时，主要以统计信息或者普遍的数据分布为依据。在大多数情况下，估计的代价都是准确的。但在一些比较特殊的场合，例如缺少统计信息，或统计信息陈旧，或抽样数据不能很好地反映数据分布时，优化器选择的执行计划不是“最优”的，甚至可能是很差的执行计划。

开发人员和用户对于数据分布是很清楚的，他们往往能知道 SQL 语句按照哪种方法执行会最快。在这种情况下，用户可以提供一种方法，指示优化器按照固定的方法去选择 SQL 的执行计划。

DM 把这种人工干预优化器的方法称为 HINT，它使优化器根据用户的需要来生成指定的执行计划。如果优化器无法生成相应的执行计划，该 HINT 将会被忽略。

HINT 的常见格式如下所示：

```
SELECT /*+ HINT1 [HINT2]*/  列名 FROM 表名 WHERE_CLAUSE ;
UPDATE 表名 /*+ HINT1 [HINT2]*/ SET 列名 =变量 WHERE_CLAUSE ;
DELETE FROM 表名  /*+ HINT1 [HINT2]*/ WHERE_CLAUSE ;
```

需要注意的是：如果 HINT 的语法没有写对或指定的值不正确，DM 并不会报错，而是直接忽略 HINT 继续执行。

DM 支持的 HINT 功能见下表。

表 25.1 HINT 功能

HINT 类型	名称	备注
路径 HINT	INDEX	表使用索引
	NO_INDEX	表不使用索引
连接操作 HINT	USE_HASH	两表使用哈希连接
	NO_USE_HASH	两表不使用哈希连接
	USE_MERGE	两表使用归并连接
	NO_USE_MERGE	两表不使用归并连接
	USE_NL	两表使用嵌套连接
	NO_USE_NL	两表不使用嵌套连接
	USE_NL_WITH_INDEX	两表使用索引连接
	NO_USE_NL_WITH_INDEX	两表不使用索引连接
	SEMI_GEN_CROSS	优先采用半连接转换为等价内连接
	NO_SEMI_GEN_CROSS	不采用半连接转换为内连接
	USE_CVT_VAR	优先采用变量改写方式实现连接
	NO_USE_CVT_VAR	不采用变量改写方式实现连接
连接顺序 HINT	ORDER	指定表的连接顺序
统计信息 HINT	STAT	指定表的行数估算
语句级 INI 的 HINT	LIKE_OPT_FLAG	详见 2.1.1.1 的 INI 参数说明
	CASE_WHEN_CVT_IFUN	
	CNNTB_OPT_FLAG	
	COMPLEX_VIEW_MERGING	
	DEL_HP_OPT_FLAG	
	ENABLE_DIST_IN_SUBQUERY_OPT	
	ENABLE_DIST_VIEW_UPDATE	
	ENABLE_HASH_JOIN	
	ENABLE_IN_VALUE_LIST_OPT	
	ENABLE_INDEX_FILTER	

	ENABLE_INDEX_JOIN	
	ENABLE_JOIN_FACTORIZATION	
	ENABLE_MERGE_JOIN	
	ENABLE_RQ_TO_INV	
	ENABLE_RQ_TO_SPL	
	ENHANCED_BEXP_TRANS_GEN	
	HAGR_DISTINCT_OPT_FLAG	
	HAGR_PARALLEL_OPT_FLAG	
	HASH_CMP_OPT_FLAG	
	HLSM_FLAG	
	IN_LIST_AS_JOIN_KEY	
	INDEX_SKIP_SCAN_RATE	
	LPQ_HASH_LR_RATE	
	MAX_OPT_N_OR_BEXPS	
	MAX_OPT_N_TABLES	
	MAX_PARALLEL_DEGREE	
	MAX_PHC_BE_NUM	
	MPP_HASH_LR_RATE	
	MPP_INDEX_JOIN_OPT_FLAG	
	MPP_NLI_OPT_FLAG	
	MULTI_IN_CVT_EXISTS	
	MULTI_UPD_OPT_FLAG	
	NONREFED_SUBQUERY_AS_CONST	
	OLAP_FLAG	
	OP_SUBQ_CVT_IN_FLAG	
	OPT_OR_FOR_HUGE_TABLE_FLAG	
	OPTIMIZER_AGGR_GROUPBY_ELIM	
	OPTIMIZER_DYNAMIC_SAMPLING	
	OPTIMIZER_MAX_PERM	
	OPTIMIZER_MODE	
	OPTIMIZER_OR_NBEXP	
	OR_CVT_HTAB_FLAG	
	OR_NBEXP_CVT_CASE_WHEN_FLAG	
	ORDER_BY_NULLS_FLAG	
	OUTER_CVT_INNER_PULL_UP_COND_FLAG	
	OUTER_CVT_INNER_PULL_UP_COND_FLAG	
	OUTER_JOIN_FLATING_FLAG	
	OUTER_JOIN_INDEX_OPT_FLAG	
	OUTER_OPT_NLO_FLAG	
	PARALLEL_MODE_COMMON_DEGREE	
	PARTIAL_JOIN_EVALUATION_FLAG	
	PHF_NTTS_OPT	
	PRJT_REPLACE_NPAR	
	PUSH_SUBQ	
	REFED_EXISTS_OPT_FLAG	
	REFED_SUBQ_CROSS_FLAG	

	RS_CAN_CACHE	
	SEL_ITEM_HTAB_FLAG	
	STAR_TRANSFORMATION_ENABLED	
	SUBQ_CVT_SPL_FLAG	
	TOP_DIS_HASH_FLAG	
	TOP_ORDER_OPT_FLAG	
	UPD_DEL_OPT	
	USE_FK_REMOVE_TABLES_FLAG	
	USE_FTTS	
	USE_HAGR_FLAG	
	USE_INDEX_SKIP_SCAN	
	USE_REFER_TAB_ONLY	
	VIEW_FILTER_MERGING	
	VIEW_PULLUP_FLAG	
	VIEW_PULLUP_MAX_TAB	

25.6.1 索引提示

1. 使用索引

目前 DM7 提供的 HINT 为表索引的选择 HINT，它指示使用指定索引进行数据检索。

语法：表名 + INDEX + 索引名 或 /*+ INDEX (表名[,] 索引名) {INDEX (表名[,] 索引名)} */

一个语句中最多指定 8 个索引。在后一种语法格式中，如果查询中给出了表的别名那么必须使用别名。

假设表 t1 上 id 和 name 列上都存在着单列索引。

例 1

```
SELECT * FROM t1 WHERE id > 2011 AND name < 'xxx';
```

如果 id 列上能过滤更多数据，可以指示用索引 idx_t1_id。

```
SELECT * FROM t1 index idx_t1_id WHERE id > 2011 AND name < 'xxx';
```

或

```
SELECT /*+INDEX(t1, idx_t1_id) */ * FROM t1 WHERE id > 2011 AND name < 'xxx';
```

例 2

```
SELECT * FROM t1 WHERE id > 2011 AND name < 'xxx' ORDER BY name;
```

如上面例子所示，虽然 id 能过滤更多数据，但是考虑到后面的排序操作，可以指示使用索引 idx_t1_name，这样可以省略掉 ORDER BY。

```
SELECT * FROM t1 INDEX idx_t1_name WHERE id > 2011 AND name < 'xxx' ORDER BY name;
```

或

```
SELECT /*+ INDEX(a idx_t1_name)*/ * FROM t1 a WHERE id > 2011 AND name < 'xxx' ORDER BY name;
```

2. 不使用索引

语法：/*+ NO_INDEX (表名[,] 索引名) { NO_INDEX (表名[,] 索引名)} */

可以指定多个索引，则这些索引都不能被使用。一个语句中最多指定 8 个索引。

25.6.2 连接方法提示

DBA 可以通过指定两个表间的连接方法来检测不同连接方式的查询效率，指定的连接可能由于无法实现或代价过高而被忽略。如果连接方法提示中的表名(别名)或索引名无效也会被自动忽略。

1. USE_HASH

强制两个表间使用指定顺序的哈希连接，例如：

```
EXPLAIN SELECT /*+ USE_HASH(t1, t2) */ * FROM t1, t2 WHERE t1.c1 = t2.d1;
1  #NSET2: [0, 1, 24]
2    #PRJT2: [0, 1, 24]; exp_num(6), is_atom(FALSE)
3      #HASH2 INNER JOIN: [0, 1, 24];  KEY_NUM(1);
4        #CSCN2: [0, 1, 12]; INDEX33555442(T1)
5        #CSCN2: [0, 1, 12]; INDEX33555443(T2)
```

2. NO_USE_HASH

强制两个表间不能使用指定顺序的哈希连接，例如：

```
EXPLAIN SELECT /*+ NO_USE_HASH(t1, t2) */ * FROM t1, t2 WHERE t1.c1 = t2.d1;
1  #NSET2: [0, 1, 24]
2    #PRJT2: [0, 1, 24]; exp_num(6), is_atom(FALSE)
3      #HASH2 INNER JOIN: [0, 1, 24];  KEY_NUM(1);
4        #CSCN2: [0, 1, 12]; INDEX33555443(T2)
5        #CSCN2: [0, 1, 12]; INDEX33555442(T1)
```

NO_USE_HASH(t1, t2)表示不允许 t1 作为左表，t2 作为右表的哈希连接，但 t1 作为右表的哈希连接还是允许的。

3. USE_NL

强制两个表间使用嵌套循环连接，例如：

```
EXPLAIN SELECT /*+ USE_NL(a, b) */ * FROM t1 a, t2 b WHERE a.c1 = b.d1;
1  #NSET2: [49, 1, 24]
2    #PRJT2: [49, 1, 24]; exp_num(6), is_atom(FALSE)
3      #SLCT2: [49, 1, 24]; A.C1 = B.D1
4        #NEST LOOP INNER JOIN2: [49, 1, 24];
5          #CSCN2: [0, 1, 12]; INDEX33555442(T1 as A)
6          #CSCN2: [0, 1, 12]; INDEX33555443(T2 as B)
```

4. NO_USE_NL

强制两个表间不能使用嵌套循环连接，例如：

```
EXPLAIN SELECT /*+ NO_USE_NL(a, b) */ * FROM t1 a, t2 b WHERE a.c1 = b.d1;
1  #NSET2: [0, 1, 24]
2    #PRJT2: [0, 1, 24]; exp_num(6), is_atom(FALSE)
3      #HASH2 INNER JOIN: [0, 1, 24];  KEY_NUM(1);
4        #CSCN2: [0, 1, 12]; INDEX33555442(T1 as A)
5        #CSCN2: [0, 1, 12]; INDEX33555443(T2 as B)
```

5. USE_NL_WITH_INDEX

当连接情况为左表+右表索引时，强制两个表间使用索引连接，例如：

```
CREATE INDEX idx_t2_d1 ON t2(d1);
SELECT /*+ USE_NL_WITH_INDEX(t1, idx_t2_d1) */ * FROM t1, t2 WHERE t1.c1 =
t2.d1;
1  #NSET2: [0, 1, 24]
2    #PRJT2: [0, 1, 24]; exp_num(6), is_atom(FALSE)
```

```
    3      #NEST LOOP INDEX JOIN2: [0, 1, 24]
    4        #CSCN2: [0, 1, 12]; INDEX33555442(T1)
    5        #BLKUP2: [0, 1, 0]; IDX_T2_D1(T2)
    6          #SSEK2: [0, 1, 0]; scan_type(ASC), IDX_T2_D1(T2),
scan_range[T1.C1,T1.C1]
```

6. NO_USE_NL_WITH_INDEX

当连接情况为左表+右表索引时，强制两个表间不能使用索引连接，例如：

```
    CREATE INDEX idx_t2_d1 on t2(d1);
    SELECT /*+ NO_USE_NL_WITH_INDEX(t1, idx_t2_d1) */ * FROM t1, t2 WHERE t1.c1
= t2.d1;
    1   #NSET2: [0, 1, 24]
    2     #PRJT2: [0, 1, 24]; exp_num(6), is_atom(FALSE)
    3       #HASH2 INNER JOIN: [0, 1, 24];  KEY_NUM(1);
    4         #CSCN2: [0, 1, 12]; INDEX33555442(T1)
    5         #CSCN2: [0, 1, 12]; INDEX33555443(T2)
```

7. USE_MERGE

强制两个表间使用归并连接，例如：

```
    CREATE INDEX idx_t1_c1 on t1(c1);
    CREATE INDEX idx_t2_d1 on t2(d1);
    SELECT /*+ USE_MERGE(t1 t2) */ * FROM t1, t2 WHERE t1.c1 = t2.d1 AND t1.c1 >
1 AND t2.d1 > 1;
    1   #NSET2: [0, 1, 24]
    2     #PRJT2: [0, 1, 24]; exp_num(6), is_atom(FALSE)
    3       #MERGE INNER JOIN3: [0, 1, 24];
    4         #BLKUP2: [0, 1, 12]; IDX_T1_C1(T1)
    5           #SSEK2: [0, 1, 12]; scan_type(ASC), IDX_T1_C1(T1),
scan_range(1,max]
    6         #BLKUP2: [0, 1, 12]; IDX_T2_D1(T2)
    7           #SSEK2: [0, 1, 12]; scan_type(ASC), IDX_T2_D1(T2),
scan_range(1,max]
```

当连接类型为外连接时，无法使用归并连接，此时即使指定 USE_MERGE，也不起作用。

8. NO_USE_MERGE

强制两个表间不能使用归并连接，例如：

```
    CREATE INDEX idx_t1_c1 on t1(c1);
    CREATE INDEX idx_t2_d1 on t2(d1);
    SELECT /*+ NO_USE_MERGE(t1 t2) */ * FROM t1, t2 WHERE t1.c1 = t2.d1 AND t1.c1 >
1 AND t2.d1 > 1;
    1   #NSET2: [0, 1, 24]
    2     #PRJT2: [0, 1, 24]; exp_num(6), is_atom(FALSE)
    3       #SLCT2: [0, 1, 24]; T2.D1 > 1
    4         #NEST LOOP INDEX JOIN2: [0, 1, 24]
    5           #BLKUP2: [0, 1, 12]; IDX_T1_C1(T1)
    6             #SSEK2: [0, 1, 12]; scan_type(ASC), IDX_T1_C1(T1),
scan_range(1,max]
    7           #BLKUP2: [0, 1, 0]; IDX_T2_D1(T2)
    8             #SSEK2: [0, 1, 0]; scan_type(ASC), IDX_T2_D1(T2),
scan_range[T1.C1,T1.C1]
```

9. SEMI_GEN_CROSS

优先采用半连接转换为等价的内连接，仅 OPTIMIZER_MODE=1 有效。例如：

```
    EXPLAIN SELECT /*+ SEMI_GEN_CROSS  OPTIMIZER_MODE(1) */ COUNT(*) FROM
SYSOBJECTS A WHERE A.ID IN (SELECT B.ID FROM SYSOBJECTS B);
    1   #NSET2: [1, 1, 8]
    2     #PRJT2: [1, 1, 8]; exp_num(1), is_atom(FALSE)
    3       #AAGR2: [1, 1, 8]; grp_num(0), sfun_num(1)
    4         #HASH2 INNER JOIN: [1, 966, 8];  KEY_NUM(1);
    5           #PRJT2: [0, 960, 4]; exp_num(1), is_atom(FALSE)
    6             #DISTINCT: [0, 960, 4]
    7               #CSCN2: [0, 966, 4]; SYSINDEXSYSOBJECTS(SYSOBJECTS as B)
    8           #CSCN2: [0, 966, 4]; SYSINDEXSYSOBJECTS(SYSOBJECTS as A)
```

10. NO_SEMI_GEN_CROSS

不采用半连接转换为等价的内连接，仅 OPTIMIZER_MODE=1 有效。例如：

```
    EXPLAIN SELECT /*+ NO_SEMI_GEN_CROSS OPTIMIZER_MODE(1) */ COUNT(*) FROM
SYSOBJECTS A WHERE A.ID IN (SELECT B.ID FROM SYSOBJECTS B);
    1   #NSET2: [1, 1, 4]
    2     #PRJT2: [1, 1, 4]; exp_num(1), is_atom(FALSE)
    3       #AAGR2: [1, 1, 4]; grp_num(0), sfun_num(1)
    4         #HASH LEFT SEMI JOIN2: [1, 966, 4]; KEY_NUM(1);
    5           #CSCN2: [0, 966, 4]; SYSINDEXSYSOBJECTS(SYSOBJECTS as A)
    6           #CSCN2: [0, 966, 4]; SYSINDEXSYSOBJECTS(SYSOBJECTS as B)
```

11. USE_CVT_VAR

优先采用变量改写方式实现连接，适合驱动表据量少而另一侧计划较复杂的场景，仅 OPTIMIZER_MODE=1 有效。例如：

```
    EXPLAIN SELECT /*+ USE_CVT_VAR OPTIMIZER_MODE(1) */ COUNT(*)
    FROM SYSOBJECTS A
    WHERE A.ID = 1000 AND EXISTS (SELECT 1 FROM SYSOBJECTS B, SYSOBJECTS C WHERE
B.ID = C.ID AND A.NAME = B.NAME);
    1   #NSET2: [0, 1, 60]
    2     #PRJT2: [0, 1, 60]; exp_num(1), is_atom(FALSE)
    3       #AAGR2: [0, 1, 60]; grp_num(0), sfun_num(1)
    4         #NEST LOOP SEMI JOIN2: [0, 1, 60];  join condition(A.NAME =
B.NAME)[with var](0)
    5           #SLCT2: [0, 1, 60]; A.ID = 1000
    6             #CSCN2: [0, 1, 60]; SYSINDEXSYSOBJECTS(SYSOBJECTS as A)
    7           #HASH2 INNER JOIN: [1, 1, 56];  KEY_NUM(1);
    8             #SLCT2: [0, 1, 52]; B.NAME = var1
    9               #CSCN2: [0, 1, 52]; SYSINDEXSYSOBJECTS(SYSOBJECTS as B)
    10            #CSCN2: [0, 966, 4]; SYSINDEXSYSOBJECTS(SYSOBJECTS as C)
```

12. NO_USE_CVT_VAR

不考虑变量改写方式实现连接，仅 OPTIMIZER_MODE=1 有效。例如：

```
    EXPLAIN SELECT /*+ NO_USE_CVT_VAR OPTIMIZER_MODE(1) */ COUNT(*)
    FROM SYSOBJECTS A
    WHERE A.ID = 1000 AND EXISTS (SELECT 1 FROM SYSOBJECTS B, SYSOBJECTS C WHERE
B.ID = C.ID AND A.NAME = B.NAME);
    1   #NSET2: [2, 1, 60]
    2     #PRJT2: [2, 1, 60]; exp_num(1), is_atom(FALSE)
    3       #AAGR2: [2, 1, 60]; grp_num(0), sfun_num(1)
```

```
4        #HASH LEFT SEMI JOIN2: [2, 1, 60]; KEY_NUM(1);
5          #SLCT2: [0, 1, 60]; A.ID = 1000
6            #CSCN2: [0, 1, 60]; SYSINDEXSYSOBJECTS(SYSOBJECTS as A)
7          #HASH2 INNER JOIN: [1, 1049, 56];  KEY_NUM(1);
8            #CSCN2: [0, 966, 4]; SYSINDEXSYSOBJECTS(SYSOBJECTS as C)
9            #CSCN2: [0, 966, 52]; SYSINDEXSYSOBJECTS(SYSOBJECTS as B)
```

25.6.3 连接顺序提示

多表连接时优化器会考虑各种可能的排列组合顺序。 使用 ORDER HINT 指定连接顺序提示可以缩小优化器试探的排列空间，进而得到接近 DBA 所期望的查询计划。如果连接顺序和连接方法提示同时指定且二者间存在自相矛盾，优化器会以连接顺序提示为准。

ORDER HINT

语法：/*+ ORDER (t1, t2 , t3, … tn) */

例 1：

```
SELECT * FROM t1, t2 , t3, t4 WHERE …
```

如果期望表的连接顺序是 t1, t2, t3，那么可以加入这样的提示：

```
SELECT /*+ ORDER(t1, t2, t3 )*/* FROM t1, t2 , t3, t4 WHERE …
```

在指定上述连接顺序后，t4,t1,t2,t3 或 t1,t2,t4,t3 会被考虑；t3,t1,t2 或 t1,t3,t2 不被考虑。

连接顺序也可以和连接方法同时指定用于得到更特定的执行计划。例如：

```
SELECT /*+ OPTIMIZER_MODE(1), ORDER(t1,t2,t3,t4) ,USE_HASH(t1,t2),
USE_HASH(t2,t3), USE_HASH(t3,t4)*/* FROM t1,t2,t3,t4 WHERE t1.c1=t2.d1 AND t2.d2
= t3.e2 AND t3.e1 = t4.f1;
```

此时的执行计划如下：

```
#NSET2: [2, 1, 0]
  #PRJT2: [2, 1, 32]; exp_num(8), is_atom(FALSE)
    #HASH2 INNER JOIN: [2, 1, 32];  KEY_NUM(1);
      #HASH2 INNER JOIN: [1, 1, 24];  KEY_NUM(1);
        #HASH2 INNER JOIN: [0, 1, 16];  KEY_NUM(1);
          #CSCN2: [0, 1, 8]; INDEX33555486(T1)
          #CSCN2: [0, 1, 8]; INDEX33555487(T2)
        #CSCN2: [0, 1, 8]; INDEX33555490(T3)
      #CSCN2: [0, 1, 8]; INDEX33555491(T4)
```

25.6.4 统计信息提示

优化器在计划优化阶段会自动获取基表的行数。但是一些特殊类型的表行数估算并不准确，或者 DBA 希望了解表大小对计划影响的时候，需要手动设置表的行数。

语法：/*+ STAT (表名, 行数) */

统计信息提示只能针对基表设置，视图和派生表等对象设置无效。如果表对象存在别名则必须使用别名。行数只能使用整数，或者整数+K（千），整数+M（百万），整数+G（十亿）。行数提示设置后，统计信息的其它内容也会做相应的调整。

例 1：

```
CREATE TABLE T_S(C1 INT);
```

```
INSERT INTO T_S SELECT LEVEL FROM DUAL CONNECT BY LEVEL<= 100;
COMMIT;
STAT 100 ON T_S(C1);
EXPLAIN SELECT /*+ STAT(T_S,1M) */ * FROM T_S WHERE C1 <= 10;
```

计划如下：

```
1  #NSET2: [107, 100000, 12]
2    #PRJT2: [107, 100000, 12]; exp_num(2), is_atom(FALSE)
3      #SLCT2: [107, 100000, 12]; T_S.C1 <= 10
4        #CSCN2: [107, 100000, 12]; INDEX33555897(T_S)
```

不使用 HINT 时计划：

```
1  #NSET2: [0, 10, 12]
2    #PRJT2: [0, 10, 12]; exp_num(2), is_atom(FALSE)
3      #SLCT2: [0, 10, 12]; T_S.C1 <= 10
4        #CSCN2: [0, 100, 12]; INDEX33555897(T_S)
```

第五部分 数据高可用性

第26章 故障恢复

26.1 概述

数据库系统在运行过程中可能会发生一些故障。造成故障的原因多种多样，包括磁盘崩溃、电源故障、软件错误，甚至人为破坏。这些情况一旦发生，就可能会丢失数据，数据库系统将无法正常运行。因此，故障恢复是数据库系统必不可少的组成部分，即数据库系统必须保证即使发生故障，也可以保障数据的完整性和一致性。

支持故障恢复的技术主要是日志，日志以一种安全的方式记录数据库系统变更的历史信息，一旦系统出现故障，数据库系统可以根据日志将系统恢复至故障发生前的某个时刻。数据库系统的日志分为两种类型：一是 REDO 日志，在数据被修改后记录它的新值；另一种是 UNDO 日志，在数据被修改前记录它的旧值。

另外，当服务器处于归档模式时，如果数据库发生故障，通过备份文件和归档日志可以恢复到指定时间点。

26.2 REDO 日志

在恢复操作中最重要的结构是联机 REDO 日志。REDO 日志存放在日志表空间文件中，这些文件存储数据库所做的所有物理更改信息。达梦数据库每一个实例都有一个相关联的联机重做日志，通过重做日志可以保证数据库的完整性和一致性。

DM SERVER 的 REDO 日志用于存储被修改数据的新值，包括事务对数据文件和回滚段的修改。REDO 日志每次被修改以后，都会自动生成一个新的日志序列值 LSN（LOG SEQUENCE NUMBER）。LSN 取值范围 0~正无穷大，新建的库 LSN 为 0，以后日志每被写入一次，LSN 值增加 1。

REDO 日志里包含有一种特殊的记录，叫 PWR（Page Written Record）日志。PWR 包括（ts_id, fil_id, page_no, page_lsn）信息。每个数据页刷盘时，都会生成一条对应的 PWR 日志。PWR 日志的 lsn 与上一个日志的 lsn 相同。在以下情况下可以利用 PWR 日志：

1. RAC 故障恢复时：利用 PWR 日志减少加载磁盘数据页的次数；
2. 故障恢复时：利用 PWR 日志提升 redo 速度；
3. 增量备份时：利用 PWR 日志提升备份速度。

在第 2 章中提到过，系统采用了单独的日志文件来存储 REDO 日志，即联机日志文件。DM 至少有两个日志文件，日志文件是循环利用的。日志文件由日志表空间管理。DM 支持增加日志文件和扩展日志文件大小。

例 1 增加大小为 80M 的日志文件 dameng_003.log

```
ALTER DATABASE ADD LOGFILE 'dameng_003.log' size 80;
```

例 2 将日志文件 dameng_003.log 大小更改为 100M

```
ALTER DATABASE RESIZE LOGFILE 'dameng_003.log' to 100;
```

在 MOUNT 状态下，支持对日志文件的重命名操作。

例 3 将日志文件 dameng_003.log 重命名为 dameng_004.log

```
ALTER DATABASE RENAME LOGFILE 'dameng_003.log' to 'dameng_004.log';
```

26.3 重做日志归档

DM 服务器可以运行在两种模式下，即归档模式和非归档模式，这两种模式可以由用户进行设置，系统在归档模式下运行时，会产生归档日志文件，此时系统管理员应该事先预留出足够的磁盘空间以便存储归档日志文件。通过 dm.ini 和 dmarch.ini 可以配置归档，参见第 2 章。归档日志包括以下五种类型：

26.3.1 本地归档

写入 REDO 日志到本地归档文件，在 REDO 日志写入日志文件后触发，由归档线程完成本地归档动作，最多可以设置 8 个本地归档。若磁盘空间不足，所有本地归档一旦失效，系统会被强制挂起，直到磁盘空间释放，本地归档成功后，再继续执行。

26.3.2 实时归档

在写入 REDO 日志到日志文件之前，通过 MAL 系统发送 REDO 日志到远程服务器，远程服务器收到 REDO 日志后，返回确认消息。收到确认消息后，执行后续操作，发送 REDO 日志失败，或从备库返回的数据库模式不是 STANDBY，将数据库切换为 SUSPEND 状态，阻塞所有 REDO 日志的写入操作。只能配置 1 个实时归档。

26.3.3 即时归档

即时归档在主库将 Redo 日志写入联机 Redo 日志文件后，再通过 MAL 系统将 Redo 日志发送到备库。即时归档是读写分离集群的实现基础，与实时归档的主要区别是发送 Redo 日志的时机不同。一个主库可以配置 1~8 个即时备库。

26.3.4 异步归档

在设定的时间点或者每隔设定时间，启动归档 REDO 日志发送。设置定时归档，必须确保至少有一个本地归档。系统调度线程根据设定，触发归档 REDO 日志发送事件。通过 MAL 系统，获取远程服务器的当前 LSN，生成发送归档 REDO 日志任务，加入任务队列。归档任务线程获取任务，通过 MAL 系统，发送到远程服务器。最多可以设置 8 个异步归档。

26.3.5 MPP 归档

在 MPP 环境下，当主 EP 写入 REDO 日志到日志文件之前，通过 MAL 系统发送 REDO 日志到远程服务器（备库），远程服务器收到 REDO 日志后，返回确认消息。收到确认消息后，执行后续操作。如果发送 REDO 日志失败，备库不做任何操作，主库也不会 SUSPEND，后续通过 MONITOR 进行恢复。最多可以配置 8 个 MPP 归档。

26.4 检查点

创建数据库时，联机日志文件通常被扩展至一定长度，其内容则被初始化为空，当系统运行时，该文件逐渐被产生的日志所填充。为了达到循环利用日志系统空间的目的，必须在所有日志文件空间将被占满时，系统能够自动清空一部分日志，以便重用日志文件的空间，为了保证被清空的日志所“保护”的数据在磁盘上是安全的，需要引入一个关键的数据库概念—检查点。当系统产生检查点时，将系统缓冲区中被修改过的数据页写入磁盘，以保证当前日志所“保护”的数据页都已安全写入磁盘，这样日志文件即可被安全重用。

当服务器启动和关闭时，系统都会产生检查点。服务器运行过程中，系统会自动判断是否需要执行检查点；当空闲日志空间不足时，系统自动产生一个检查点；系统调度线程也会定时产生检查点；还可以通过调用系统函数 CHECKPOINT 主动生成检查点。

系统调度线程根据 dm.ini 的参数配置，产生检查点，下面给出具体的例子和说明。

```
CKPT_INTERVAL       = 1800      #每间隔 1800 秒，产生检查点间隔
CKPT_FLUSH_RATE     = 5         #检查点的刷盘比例为 5%，将系统中所有脏页的 5%写入磁盘
CKPT_FLUSH_PAGES    = 1000      #一个检查点最少写入 1000 个脏页
```

检查点系统函数 CHECKPOINT（FLUSH_RATE），其参数 FLUSH_RATE 为 INTEGER 类型或 DOUBLE 类型，指示刷盘比例，替代 dm.ini 中的 CKPT_FLUSH_RATE，同时 CKPT_FLUSH_PAGES 参数同时生效。例如，执行 SELECT CHECKPOINT(10);会将系统中 10%脏页写入磁盘。

26.5 回滚段与回滚记录

DM SERVER 采用回滚段机制来处理 UNDO 日志。回滚段由一定数量的回滚页组成，回滚页存放的是一批回滚记录。回滚记录存放被修改数据的旧值，并有专属的格式，与物理记录格式不同。回滚段的管理同一般数据文件一样，其回滚数据页的分配、淘汰和释放也交由数据缓冲区完成。回滚文件属于回滚表空间。回滚段的数据并不会永久保留，事务结束后，由系统的 PURGE 模块释放回滚页。

REDO 日志记录了所有对数据库数据进行修改的数据，当然也包括回滚段的修改数据。因此，REDO 日志同样也会保护回滚数据。当使用重做日志恢复数据库时，系统将重做回滚段中修改了的数据，并将这些修改写回到回滚段中，从而形成回滚记录。回滚记录与重做记录类似，只不过回滚记录记录被修改数据的旧值，以达到数据回滚到原来状态的目的。

另外，回滚表空间不允许修改和删除操作。

26.6 系统故障恢复

系统故障也就是我们通常所说的系统崩溃。系统崩溃的原因比较复杂，有硬件故障，或者是数据库软件或操作系统的漏洞等。它导致整个系统停止运行，内存中的数据全部丢失，但磁盘上存储的数据仍然有效。

26.6.1 恢复流程

当系统故障发生时，系统管理员可以通过查看系统日志文件以了解系统故障发生的原因或故障发生之前系统的运行状况。多数情况下，系统在故障发生之前会自动记录产生故障的原因，但也不全是这样，如系统掉电时，发生故障的原因可能来不及记录。总之，在处理系

统故障之前，系统管理员需要完全了解系统故障发生的原因，并采取相应的措施，如更换硬件，升级操作系统或数据库软件等，必要时还可以联系达梦公司的技术支持工程师，以获得必要的支持。

系统故障恢复的最后一步则是重新启动 DM 服务器。此时，系统将从最近一个检查点处开始扫描联机日志，并重做日志记录的内容，这样系统即恢复到了故障发生前的某个很近的时刻，然后将系统中仍然活动的事务再依次利用回滚段进行回滚。由此可以看出，系统故障恢复可以保证事务的原子性和持久性，即系统故障发生时，仍然活动的事务可以被安全回滚，而已经提交的事务则保证其所修改数据的持久性。

26.6.2 并行恢复

为了提高系统恢复的效率，达梦数据库采用了并行恢复技术，同时重做 REDO 日志，提高恢复性能。为保证恢复的正确执行，理论上应严格按日志生成顺序重做 REDO 日志。事实上，按日志生成顺序执行重做的根本原因是为了保证 REDO 日志重做完成后，数据页中保存的是最后一次更新操作生成的数据。达梦数据库在恢复时，将不相关数据页的 REDO 日志收集出来，生成多个独立的子任务，并行执行重做，充分利用多个 CPU 的运算能力，提升恢复效率。通过 INI 参数 REDO_THREAD_NUM 可配置重做线程数，默认是 4 个线程。

26.7 介质故障恢复

介质故障指的是由于各种原因导致数据库系统存储在磁盘上的数据被损坏，如磁盘损坏等。介质故障是数据库系统最为严重的故障，此时系统已经无法重新启动，磁盘上的数据也无法复原。

出现介质故障后，系统管理员首先需要分析介质故障发生的原因，并采取措施，如磁盘损坏则更换磁盘等。由于系统的数据已经全部或部分丢失，此时只能依赖以前建立的备份和系统产生的归档日志文件进行恢复。由于涉及到备份，具体的介质恢复策略将在备份与还原相关章节详细介绍。

需要说明的是，为了避免系统在出现故障时丢失数据，应关闭磁盘缓存（若存在的话），具体事项请咨询操作系统管理员。

第27章 备份与还原

27.1 概述

数据库的备份与还原是系统容灾的重要方法。在一个生产系统中，数据库往往处于核心的地位。为了保证数据的安全，人们想出了各种各样的方法，比如创建 RAC 环境，配置异地容灾，使用磁盘阵列，备份与还原等。备份意味着把重要的数据复制到安全的存储介质上，还原则意味着在必要的时候再把以前备份的数据复制到最初的位置，以保证用户可以访问这样的数据。

DM 数据库管理系统支持的备份方式包括物理备份、逻辑备份和 B 树备份，其中 B 树备份是介于物理备份和逻辑备份之间的一种形态。物理备份分为数据库级的备份和用户表空间级的备份；B 树备份仅有用户表级的备份。为了提高备份恢复的安全性，减少备份文件占用磁盘空间的大小，系统支持对备份数据加密和压缩。

27.2 备份

27.2.1 基本概念

27.2.1.1 完全备份与增量备份

达梦数据库管理系统支持数据库和表空间的完全备份和增量备份。完全备份是指一个备份包含指定数据库或表空间的所有数据。当用户进行完全备份时，系统需要做的事情就是将一个数据库或表空间所包含的文件内容都拷贝到用户指定的备份文件中。完全备份包含了一个数据库或表空间的所有数据，这样的一个备份通常很大，需要占用很多的磁盘空间，尤其是系统在运行过程中，定期产生的完全备份所占用磁盘上的空间将非常大，同时，其备份过程也会很长。为了解决这个问题，DM 提供了增量备份的功能，即不用备份当前数据库或表空间的所有数据，而只是以该数据库或表空间最近的一个备份为基础，仅仅拷贝自该备份以来所有被修改过的数据，形成一个增量备份。相对于完全备份而言，增量备份小得多，同时其备份的时间也会更短。

当用户选择在数据库或表空间上进行增量备份时，系统会自动将该数据库或表空间最近的一个备份作为其基础备份，基础备份可以是完全备份也可以是一个增量备份，但如果该数据库或表空间自创建以来还没有进行过备份，则增量备份无法进行。还要特别注意的是，如果该数据库自最近一次还原以来还没有进行过备份，则增量备份也无法进行，这是因为考虑到系统的最近那一次的还原会使以前的备份对于当前系统都失效，鉴于这一点，系统管理员在每次系统还原后就立即作一次完全备份是一个很好的习惯。

由于增量备份是基于某一个备份而生成的，所以对于增量备份的还原也有一个限制条件，即增量备份将要还原的系统，必须是刚刚利用其基础备份还原过并且还没有被启动过服务器的系统，否则增量备份无法进行还原。这个限制必须非常严格，是因为用户的数据需要受到严格的保护。如果利用基础备份还原后系统被启动过，则无法保证是否有新的事务修改过数据库，而此时利用增量备份还原可能会导致那部分被修改过的数据丢失。为了让用户能很轻松地完成这个严格的还原过程，DM 提供了在还原增量备份时自动还原基础备份的功能。

需要说明的是，当该增量备份的基础备份也是增量备份时，可能会引起级联的基础备份还原。

完全备份与增量备份各有缺点和优点。完全备份的优点是，只需要一次备份的数据，就能完全将数据库还原到备份时的状态，缺点是由于对整个数据库的数据备份，因此备份文件需要比较大的磁盘空间。由于增量备份是基于数据库的某一次备份(增量备份或是完全备份)做的操作，因此在还原时若增量备份所依赖的基础备份丢失，则该备份文件就变得不可用，相对于完全备份而言，不够可靠；另一方面，由于只是对某一段时间内修改数据的备份，因此备份文件比较小，备份速度比较快。需要说明的是，增量备份所依赖的基础备份有且仅有一个完全备份。

27.2.1.2 联机备份与脱机备份

DM 提供了两种方式执行备份操作，包括脱机方式和联机方式。所谓脱机方式，即数据库服务器关闭以后所进行的备份，联机方式则是指数据库服务器正常运行过程中进行的备份。由于脱机方式执行备份时需要关闭数据库，所以会对用户的访问产生影响，而且很多重要的数据库都需要进行 7 * 24 小时访问。脱机备份的优势在于无需备份日志保证数据的一致性，因为在备份过程中用户无法修改数据库中的数据，另外无论是否处于归档模式，都能进行脱机备份。联机方式支持数据库备份、用户表空间备份和用户表备份，而脱机方式只支持数据库备份。利用这两种方式进行操作时，需要注意的问题有两点：首先，在进行脱机备份时，必须保证系统是正常关闭的，如果处于不正常关闭的状态，脱机备份无法进行；其次，在进行联机库级备份和表空间备份时，必须保证系统处于归档模式，否则联机备份不能进行。

27.2.2 物理备份

27.2.2.1 备份数据库

为防止重要数据丢失，对整个数据库进行备份是最保险的方式，利用备份文件可以将数据库还原到备份时的状态。实现备份数据库的基本原理是将数据库对应的物理数据文件，包括数据页文件和回滚页文件的内容拷贝到备份文件中。DM 支持多用户并发操作数据库，在用户联机备份过程中，可能有其它用户在操作数据库，因此对于联机备份，除了备份物理数据文件还需要备份物理日志文件。备份日志文件的目的是保证数据库的完整性，这里提到的物理日志文件是归档日志文件。归档日志文件记录了备份期间所有用户对数据库进行的操作，还原时，利用数据文件和归档日志文件的内容即能将数据库恢复到备份时的状态。也可以指定不备份日志，通过记录备份开始和备份结束的日志节点，在数据库恢复的时候，利用库的归档，重做备份开始到备份结束之间的日志，保证数据库的完整性。备份时间与备份数据量成正比，保证数据不丢失的情况下，少备份数据即能提高备份效率，因此拷贝文件不是简单地将数据文件和日志文件的所有内容拷贝到备份文件，而是仅仅拷贝使用过的数据页，这样能大大减少磁盘 IO，提高备份性能。

DM 提供联机和脱机两种方式执行数据库备份。联机方式通过 SQL 命令提供，脱机方式则是通过备份工具提供。

27.2.2.1.1 联机备份

```
    BACKUP DATABASE [FULL |INCREMENT WITH BACKUPDIR ‘<备份目录>’{,‘<备份目录>’} [USE
PWR] ] TO <备份名> [BAKFILE ‘<备份路径>’] [BACKUPINFO ‘<备份描述>’] [MAXSIZE <限制
```

```
大小>] [IDENTIFIED BY <密钥>[WITH ENCRYPTION<TYPE>][ENCRYPT WITH <加密算法>]][COMPRESSED [LEVEL <压缩级别>]][WITHOUT LOG][PARALLEL '<并行映射文件路径>'];
```

参数说明：

(1) FULL|INCREMENT 备份类型，FULL 表示完全备份，INCREMENT 表示增量备份；

(2) <备份目录> 基础备份所在的目录，最大长度为 256 个字节；

(3) USE PWR 增量备份时有效，是否使用 PWR 日志优化增量备份过程；

(4) <备份名> 备份的名称，在 DMDBMS 中以此标识不同的备份；

(5) <备份路径> 备份文件存放的完整路径；

(6) <备份描述> 备份的描述信息；

(7) <限制大小> 最大备份文件大小，以 M 为单位，取值范围 33M~2G；

(8) <密钥> 备份加密通过使用 IDENTIFIED BY 来指定密码；

(9) WITH ENCRYPTION <TYPE> 用来指定加密类型，0 表示不加密，1 表示简单加密，2 表示复杂加密；

(10) COMPRESSED 用来指定是否压缩级别，取值范围 1~9。如果使用 0，表示不压缩，1 表示压缩快、但是压缩比低，9 表示压缩慢、但是压缩比高；

(11) WITHOUT LOG 联机数据库备份是否备份日志。如果使用，则表示不备份，否则表示备份；

(12) PARALLEL 用来指定并行映射文件的完整路径；

(13) <并行映射文件路径> 并行映射文件的完整路径，最大长度为 256 个字节，如：E:\bak.parallel；

(14) <加密算法> 加密算法具体见《DM_SQL》相关章节，但是不支持 xxx_cbc 和 xxx_ecb 算法加密。缺省情况下，算法为 AES256_CFB。

备份成功后会在备份默认目录下生成备份文件，若没有指定备份文件名，则系统自动生成以“数据库名_日期.bak”格式的备份文件，通过该备份文件可以进行数据库还原。

例如，执行数据库备份，备份类型为完全备份，备份文件名为 C:\database_full.bak。

```
BACKUP DATABASE FULL TO full_bak BAKFILE 'C:\database_full.bak';
```

27.2.2.1.2 脱机备份

```
DMBACKUP KEYWORD=value
```

确认 DM 数据库服务器已停止后，在 Windows“命令提示符”窗口中输入带参数的 DMBACKUP 命令启动该工具，参数说明见下表。

表 27.1 DMBACKUP 工具参数说明

参数	含义	备注
INI_PATH	配置文件路径，最大长度为 256 字节	若未设置 SHOW_BAK_INFO 参数，则为必选参数
NAME	备份名，最大长度为 128 字节	若未设置 SHOW_BAK_INFO 参数，则为必选参数
BAKFILE_PATH	备份文件路径	可选参数
TYPE	备份类型，默认为完全备份，FULL 表示完全备份，INCREMENT 表示增量备份	可选参数
BASE_BAK_DIR	增量备份时的指定基备份目录	可选参数，仅对增量备份有效
DESCRIBE	备份文件描述，即备注信息，注意：描述如果含有空格，需要用双引号括起来，否则会报错	可选参数
MAXSIZE	备份文件最大大小，取值范围为：33-2048，单位为 M（兆），默认为 2048	可选参数

ENCRYPT	加密类型，0 表示不加密，1 表示简单加密，2 表示复杂加密	可选参数
PASSWORD	加密密码，长度最大为 128 字节，如果选择不加密，此处不必指定	可选参数
ENCRYPT_NAME	加密算法，可通过查看动态视图 v$CIPHERS 获取加密算法信息。不支持 xxx_cbc 和 xxx_ecb 算法。	可选参数
COMPRESS	指定备份为压缩，如果不选择，则表示不压缩	可选参数
PARALLEL	指定并行备份映射文件完整路径，如：E:\bak.parallel	可选参数
SHOW_BAK_INFO	指定备份存放的目录，可以指定 1~16 个	可选参数
HELP	打印帮助信息	

例如，对名为 test 数据库执行一个完全备份：

```
DMBACKUP  TYPE=FULL INI_PATH=c:\dmdata\dameng\dm.ini NAME=test
```

27.2.2.1.3 并行映射文件

为了提高数据库备份性能，针对物理备份，系统提供了并行备份的功能。并行备份的核心思想是将对数据库的备份拆分为对多个表空间的备份。系统在执行备份前，将物理存储位置临近的表空间作为基本备份单位，随后同时备份不同的备份单位，从而使得系统在执行备份时，充分利用了 CPU 资源和磁盘组资源，达到提升备份性能的目的。利用并行备份功能，需要系统管理员配置并行映射文件，映射文件格式如下所示。映射文件配置内容与系统并行备份的性能直接相关，因此，系统管理员应在充分了解数据库中各个表空间所在磁盘的物理位置，合理设置配置文件。此外，每个备份单位设置的备份目录应尽量在不同的磁盘组，这样能保证读写文件都能充分利用磁盘 IO，使备份性能最优化。

```
[表空间名，表空间名，……]=备份目录 1
[表空间名，表空间名，……]=备份目录 2
[表空间名，表空间名，……]=备份目录 3
……
[default]=备份目录 N
```

说明：

1. 备份目录必需是本地物理磁盘上已经存在的目录；
2. 设置表空间名必需存在，若不存在则报错；
3. 每一行只能有一个“[表空间名，表空间名……]=备份目录”项；
4. 自动忽略同一备份目录下对应的多个同名表空间名，只取其中一个；
5. 一个表空间名不能对应多个备份目录，否则报错；
6. 对于相同的备份目录，DM7 自动将它们作为一个备份单位；
7. default：没有显示设置的表空间，备份数据都存放在 default 对应的目录下；
8. [default]行必需存在，DM7 认为[default]为最后一行，不统计[default]后续行的设置；
9. 映射文件的扩展名为 parallel；
10. 不允许显示设置系统表空间的磁盘目录；
11. 对于含有特殊字符的表空间名，比如逗号、空格、中括号、等号，系统不保证解析映射文件的正确性。

举例说明：

(1)假设数据库数据分布在三个磁盘组 DISK1，DISK2 和 DISK3 中。SYSTEM，MAIN，RLOG，ROLL 表空间在磁盘组 DISK1；TS1，TS2 表空间在磁盘组 DISK2；TS3 表空间在

DISK3，配置映射文件 bak.parallel 如下。

```
[TS1，TS2]=D:\DAMENG_1
[TS3]= C:\DAMENG_2
[default]=E:\DAMENG
```

从上述配置内容可知，TS1，TS2 作为一个备份单位，备份数据存储在 D:\DAMENG_1 中；TS3 作为一个备份单位，备份数据存储在 C:\DAMENG_2 中；系统表空间作为一个备份单位，备份数据存储在 E:\DAMENG 目录中。

(2) 利用配置映射文件 E:\bak.parallel 执行联机并行备份操作，主备份文件为 C:\parallel.bak，各个子备份分别在 C:\DAMENG_1，D:\DAMENG_2 和 E:\DAMENG 目录中。

```
BACKUP DATABASE FULL TO PARA_BAK BAKFILE 'C:\parallel.bak'  PARALLEL 'E:\bak.parallel'
```

(3) 利用配置映射文件 E:\bak.parallel 执行脱机并行全量备份操作，主备份文件为 C:\parallel02.bak，各个子备份分别在 C:\DAMENG_1，D:\DAMENG_2 和 E:\DAMENG 目录中。

```
DMBACKUP      TYPE=FULL      INI_PATH=c:\dmdata\dameng\dm.ini   NAME=TEST PARALLEL=E:\bak.parallel
```

27.2.2.2 备份用户表空间

DM 除了支持对整个数据库备份外，还支持备份单个用户表空间。与备份数据库一样，备份表空间也支持完全备份和增量备份。不过，备份表空间仅仅支持联机备份。备份用户表空间提供了一种更灵活的方式备份用户数据。通常情况下，不同的用户表空间可以部署在不同的物理磁盘上，当某个或某几个磁盘出现介质故障时，只需要利用特定表空间备份还原出现故障的表空间，在保证数据完整性的前提下，提高了备份还原的效率。与备份数据库一样，备份用户表空间的基本原理是拷贝数据文件。此外，执行备份用户表空间时，需要保证系统处于归档模式，因为表空间的还原依赖于归档日志文件。

系统提供联机 SQL 命令执行表空间备份，不支持对系统表空间、日志表空间、回滚表空间、临时表空间执行备份操作。

```
BACKUP TABLESPACE <表空间名>  [FULL| INCREMENT WITH BACKUPDIR '<备份目录>'{,'<备份目录>'}] TO <备份名> [BAKFILE '<备份路径>'] [BACKUPINFO '<备份描述>'] [MAXSIZE <限制大小>] [IDENTIFIED BY <密钥>[WITH ENCRYPTION <TYPE>]] [COMPRESSED [LEVEL <压缩级别>]];
```

例如备份名为 t1 的表空间，备份类型为 FULL，备份文件名为 'C:\tablespace_bak.bak'。

```
BACKUP TABLESPACE t1  FULL TO TS_BAK BAKFILE 'C:\tablespace_bak.bak' ;
```

27.2.3 B 树备份

DM 除了提供常规的备份方式以外，还提供了一种新的备份方式——B 树备份。B 树备份是一种介于物理备份与逻辑备份的实现方式，支持单表备份。B 树备份、还原实现了数据的无损恢复，效率高于逻辑备份、还原。

27.2.3.1 备份用户表

B 树备份可单独备份一个表，也可单独备份某一个分区子表。

假如，某个用户因为误操作导致某个表被破坏，而数据库中其它对象是完好的。若需要恢复该表，可以借助还原表所属的表空间实现，如果表跨表空间，则涉及的表空间都需要还原，或是还原整个数据库才能达到恢复表的目的。B 树备份提供表级备份的功能，使得只需还原特定的表即可，大大提高了备份还原的灵活性。

DM 中所有表都以索引方式组织，包括三种类型索引结构：B 树索引，LIST 索引和列存储索引。一个表有且仅有一个聚簇索引，若表有唯一性约束，或是表上显式建有索引，那么该表还包含一个或多个二级索引，聚簇索引和二级索引包含了表中的所有数据。表的底层存储方式为 B 树备份用户表提供了可行性，因为只需要备份表所包含的索引数据，即完成了对表的备份。备份单个表功能实现使得 B 树备份比普通备份更灵活，为用户提供了更多层次的备份方式。另外，相比于逻辑备份，B 树备份是介于物理备份与逻辑备份的实现方式，因此效率相对更高。

系统提供联机 SQL 命令执行 B 树备份用户表。对于表级备份，系统不支持临时表和垂直分区子表的备份。如果直接对水平分区表子表进行备份，那么在还原时服务器并不会检查分区表版本信息前后是否一致，这一点由用户保证。

对于含有附加列的表不支持备份，可以通过重组表数据语法：alter table 表名 rebuild columns，消除附加列以后后再备份。

```
    BACKUP TABLE <表名>  TO <备份名> [BAKFILE ‘<备份路径>’] [BACKUPINFO ‘<备份描述>’] [MAXSIZE <限制大小>] [IDENTIFIED BY <密钥>[WITH ENCRYPTION <TYPE>]] [COMPRESSED [LEVEL <压缩级别>]];
```

命令参数的含义可以参见 27.2.2 节。

例如备份名字为 test 的表，备份文件名为'C:\table_bak.bak'。

```
    BACKUP TABLE test TO table_bak BAKFILE 'C:\table_bak.bak';
```

27.2.4 备份加密与压缩

备份文件中包含了数据库中的原始数据，为提高备份数据的安全性，无论是普通备份还是 B 树备份，DM 都提供了对备份数据进行加密的功能。系统中支持简单加密和复杂加密。简单加密和复杂加密实现原理都是在备份过程中利用经典的加密算法对数据页面加密，然后将密文写入备份文件，区别在于复杂加密方式对写入备份文件的每一页数据都进行加密，而简单加密方式只是选择性地对某些页面进行加密。显然，简单加密的效率相对于复杂加密效率要高，但同时，简单加密的备份数据不如复杂加密安全。

此外，DM 还支持备份数据压缩功能。压缩功能的实现能有效缩短备份文件的大小，减少所占的磁盘空间。

不论是加密还是压缩，都是在正常执行备份数据时的额外操作，与此同时，压缩和加密的备份文件用于还原时，还需要执行解压缩和解密操作，因此会对备份、还原性能造成一定的影响。系统管理员应根据实际情况，在磁盘空间利用率、备份、还原性能，以及备份数据安全性之间做权衡，确定在执行备份操作时是否需要设置压缩，加密参数。

举例说明：

(1) 联机备份数据库，并对备份数据压缩，备份文件存放路径为 C:\database_cpr.bak。

```
    BACKUP DATABASE FULL TO cpr_bak BAKFILE 'C:\database_cpr.bak' COMPRESSED;
```

(2) 联机备份名为 t1 的表空间，备份类型为 FULL，对备份数据进行简单加密，加密密码为 ABCDEF，备份文件存放路径为 C:\tablespace_bak.bak。

```
    BACKUP TABLESPACE t1 FULL TO ts_ bak BAKFILE 'C:\tablespace_bak.bak'
```

```
IDENTIFIED BY ABCDEF  WITH  ENCRYPTION 1;
```

(3)联机备份 SYSDBA 模式下的 test 表，对备份数据进行复杂加密，加密密码为 ABCDEF，并且对备份数据压缩。备份文件存放路径为 C:\table_bak.bak。

```
BACKUP TABLE SYSDBA.test TO table_bak BAKFILE 'C:\table_bak.bak'
IDENTIFIED BY ABCDEF  WITH  ENCRYPTION 2 COMPRESSED;
```

27.3 还原

27.3.1 基本概念

27.3.1.1 联机还原与脱机还原

当系统由于种种原因出现介质故障时，需要利用系统在早些时候建立的备份和归档日志进行还原。DM 支持多层次的备份与还原，包括表级、表空间级以及数据库级。对于数据库级的还原只支持脱机还原，不提供联机还原的功能；对于表空间级和表级还原，只支持联机还原，不支持脱机还原的功能。用户可以根据具体受损数据内容选择合适的备份进行还原，若只有一个或几个表数据被破坏，则通过还原用户表功能即能还原表数据；若只有一个或几个表空间被破坏，则可以通过还原指定的几个表空间实现数据恢复；若整个数据库数据受损，则只能通过还原数据库实现数据恢复。表级、表空间级和数据库级的备份粒度逐步加大，备份文件大小以及还原消耗时间都逐步递增，因此建议用户根据实际应用情况适时对表、表空间和数据库做备份，以备恢复数据时使用。以下三节将分别介绍还原数据库，还原表空间和还原用户表功能。

需要说明的是，备份文件和待还原数据库的建库参数 USE_NEW_HASH 应保持一致，若取值不一致，则还原时会报错。

27.3.2 还原数据库

DM 支持利用还原工具对数据库进行还原。根据不同备份类型，包括联机备份、脱机备份、完全备份以及增量备份，还原工具选择不同的还原策略实现恢复用户数据的目的。首先讨论联机备份与脱机备份，脱机备份仅包含数据页文件的内容，而联机备份文件除了包含数据页文件内容以外，可能还包括备份过程中产生的日志文件内容。因此在利用联机备份还原时，除了拷贝数据页至数据库文件以外，可能还需要重做备份的日志文件内容，达到还原数据库的目的。在还原联机库备份且不带日志的备份时，需要用户指定归档目录，由用户保证备份期间的归档日志存在且完好，在还原库时，需要重做该日志。如果日志不存在，则备份文件就无效，报错归档日志错误。

接下来讨论完全备份与增量备份，对于增量备份，还原工具会根据用户提供备份的信息搜集它的基础备份，若基础备份仍然为增量备份，则工具会继续搜集，直至搜集到一个类型为完全备份的基础备份为止。因此在这个过程中，工具会构造一个基础备份链表，备份链表基于备份间的依赖关系组织。比如，备份 A 是备份 B 的增量备份，则可以说 A 依赖于 B。若用户首先进行了一次完全备份 A，然后在 A 的基础上做了增量备份 B，随后基于增量备份 B 又做了增量备份 C，最后基于增量备份 C 做了增量备份 D。那么此时利用增量备份 D 还原数据库时，系统搜集到得备份链表如 30.1 所示，执行还原时，首先利用完全备份恢复 A 进行还原，然后依次利用增量备份 B，C 和 D 还原，最终完成整个还原过程。需要说明的是，若备份类型为 B 树备份的数据库备份是不存在增量备份的。

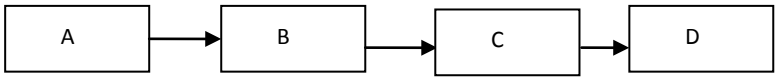


图 27.1 备份链表

脱机库还原支持无备份文件的还原，原理就是利用归档文件，在目标库上执行 REDO 操作，把库还原到指定的时间点或者 LSN。限制条件是，1）目标库和归档必须是源自同一个库；2）必须保证源库的归档是完整，否则会出现不可预知的错误，即要保证备份之后产生的归档是完整的。要利用归档还原，需要利用备份文件还原一个目标库，修改目标库 db_magic 和源库中归档文件的 db_magic 一致。然后就可以执行还原操作了。

库 db_magic 修改工具的用法如下：

```
DMMDF       PATH    TYPE
```

参数说明：

path ：文件全路径。文件为 SYSTEM.DBF 或 DAMENG01.LOG。

type ：1 或 2。1 表示 0 号数据文件，与 path 中 SYSTEM.DBF 对应；2 表示 0 号日志文件，与 path 中 DAMENG01.LOG 对应。

使用说明：

0 号数据文件和 0 号日志文件都必须修改。

还原工具的用法如下：

```
DMRESTORE  KEYWORD=value 或 KEYWORD=(value1,value2,...,valueN)
```

确认 DM 数据库服务器已停止后，在 Windows“命令提示符”窗口中输入带参数的 DMRESTORE 命令启动该工具，参数说明见表 27.2。

表 27.2 DMRESTORE 工具参数说明

参数	含义	备注
INI_PATH	配置文件路径，最大长度为 256 字节	可选参数
FILE	用于还原的备份文件完整路径	必选参数
MAPPED_FIL	当指定 res_type 为 4 时，输出备份文件的映射信息到该文件；如果指定 res_type 为 0 或 3 时，则还原时，会采用 MAPPED_FIL 文件中的 data_path 和 mirror_path 路径为准（如果用户希望使用相对路径策略还原，请删除相关的组，同时修改要修改的 data_path 和 mirror_path）。 此外，如果指定 MAPPED_FIL 参数，则本工具中 datafil_path 和 mirror_path 参数会失效；反之，则生效	可选参数
TIME	指定还原的时间点和指定还原的 LSN，需要配合 ARCHIVE_DIR 使用，若指定了 ARCHIVE_DIR，既没有指定 TIME 和 END_LSN 则会重做所有的归档；如果指定了某一个则会还原到指定的 TIME 或者 LSN；如果都指定了，则会还原到早的一个时间点（或者 LSN）；如果指定 END_LSN 为 1，则表示还原到备份的时候，不会 redo 归档	可选参数
END_LSN		
BACKUP_DIR	备份文件存放路径，用于搜索备份文件（默认会搜索备份文件所在的目录，以及库的默认备份目录）	可选参数
ARCHIVE_DIR	归档日志存放的目录，可以指定 1~8 个。在还原联机库备份且不带日志（WITHOUT LOG）的备份文件时，ARCHIVE_DIR 为必选参数	可选参数
DATAFIL_PATH	指定还原后的新文件路径([编号，路径]，……，[编号，路径])。编号与路径成对出现，最多可以指定 16 个。编号为大于 0 的整数。 MAPPED_FIL 优先	可选参数
PARALLEL_DIR	并行映射文件存放路径，用于搜索映射文件	可选参数
SHOW_BAK_INFO	可选值（0,1,2）0：表示利用备份还原；1：仅显示备份信息，不进行还原；2：表示使用归档日志进行还原；默认值为 0。 该参数已被 RES_TYPE 取代，为兼容之前版本，暂时保留。如果同时指定 RES_TYPE 和 SHOW_BAK_INFO，则 RES_TYPE 有效，SHOW_BAK_INFO 无效	可选参数
RES_TYPE	可选值（0,1,2,3,4）0：表示利用备份还原；1：仅显示备份信息，不	可选参数

	进行还原；2：表示使用归档日志进行还原；3：仅还原单个完全备份或者增量备份；4：仅仅输出备份文件映射信息到文件。默认值为 0	
MIRROR_PATH	指定备份库中的镜像文件路径在还原时的映射路径。([编号，路径]，……，[编号，路径])。编号与路径成对出现，最多可以指定 16 个。编号为大于 0 的整数，映射路径绝对路径。MAPPED_FIL 优先	可选参数
LOG	指定一个文件，还原时日志会写入该文件	可选参数
DUMMY	静默模式，可以指定{(Y)es,(N)o}。默认 No。 交互信息处理：打印(P)。Y：打印所有交互信息。N：不打印交互信息。	可选参数
PASSWORD	还原时，指定备份时的加密密码	
ENCRYPT_NAME	还原时用来解密的算法名。缺省算法为 AES256_CFB	
REDO_RARCH_OPT	是否优化重做归档(1, 2)。1：不优化， 2：优化， 默认为 1 不优化	可选参数
HELP	打印帮助信息	可选参数

备注：若设置 SHOW_BAK_INFO 参数为 1，则不需要利用备份进行还原操作，可以不指定 INI_PATH；指定了 BACKUP_DIR 参数时，则会收集备份目录下的所有的备份，并分组输出；如果指定了 FILE，且该备份文件是增量备份，则会收集增量备份相关的所有备份输出。

27.3.2.1 举例说明

27.3.2.1.1 使用备份文件还原

例如，利用备份文件 c:\dmdata\backup\test.bak 还原数据库

```
DMRESTORE  INI_PATH=c:\dmdata\dameng\dm.ini FILE=c:\dmdata\backup\test.bak
```

例如，利用归档文件 c:\dmdata\arch 还原数据库

```
第一步，修改目标库中的 db_magic。
--原备份库为 D:\tmp1\DAMENG，新还原的库为 D:\tmp2\DAMENG
1.  使用 DMMDF 工具查看源备份库中 db_magic。
dmmdf D:\tmp1\DAMENG\system.dbf 1
查看到 db_magic 为 1375075864。记下。
2. 使用 DMMDF 工具查看目标库中 system.dbf 的 db_magic。
dmmdf D:\tmp2\DAMENG\system.dbf 1
--查看到 db_magic 为 1375251734。
输入 1 后，出现 Please input new value:
输入：1375075864。
3. 使用 DMMDF 工具查看目标库中 DAMENG01.log 的 db_magic。
dmmdf D:\tmp2\DAMENG\DAMENG01.log 2
--查看到 db_magic 为 1375257160。
输入 7 后，出现 Input the new value:
输入：1375075864。
至此，修改完毕。
第二步，还原数据库
DMRESTORE   INI_PATH=c:\dmdata\dameng\dm.ini  ARCHIVE_DIR=(c:\dmdata\arch)
SHOW_BAK_INFO=2
```

27.3.2.1.2 使用归档日志还原

利用备份文件，DM 可以将数据库、表空间和表还原到备份时的状态。但用户可能更希望

将数据还原到最新的状态。这点不用担心，事实上系统足够智能，在利用备份完成还原后，会去归档目录搜索归档日志，并分析归档日志是否与还原的数据库匹配，若存在匹配的归档日志，则系统会重做这部分日志，将数据库还原到最新的状态；若没有可用的归档日志，则只能还原到备份时的状态。

利用指定归档目录下归档日志还原数据库 test

```
DMRESTORE  INI_PATH=c:\dmdata\dameng\dm.ini FILE=c:\dmdata\backup\test.bak
ARCHIVE_DIR = c:\dmdata\dameng\arch
```

27.3.2.1.3 还原到指定时间点或 LSN

上节叙述了达梦系统使用归档日志将数据库还原到最新的状态，这一节讨论与归档日志相关的另一个功能：还原到指定的时间点。根据用户需求，可以将数据库还原到指定的时间点。还原到时间点的功能依赖于归档日志，在备份完成时，系统会记录一个备份时间，因此要还原的时间点一定在备份完成时间之后，否则系统会提示报错信息。借助备份文件完成还原后，开始重做归档日志，区别于完全还原将所有可用的归档日志全部重做，若指定还原到时间点，则只重做早于时间点的日志，从而达到将数据库还原到指定时间的状态的目的。需要注意的是，由于表空间只是数据库的一部分，为保证还原后，数据库中的所有数据处于最新状态，还原表空间会重做该表空间所有可用的归档日志，因此还原表空间不支持还原到时间点功能。比如用户在 2011-9-8 17:20:20 时对数据库 test 做了一个备份，在 2011-10-8 13:34:20 想将数据库 test 还原到时间点 2011-9-25 15:20:20，若用户保证从备份完成的时间点到指定时间点这段时间的归档日志都完好，则通过如下还原命令就能达到预期效果。

还原数据库 test 到指定时间 2011-9-25 15:20:20

```
DMRESTORE  INI_PATH=c:\dmdata\dameng\dm.ini FILE=c:\dmdata\backup\test.bak
TIME="2011-9-25 15:20:20"  ARCHIVE_DIR = c:\dmdata\dameng\arch
```

达梦系统使用归档还原，还可以指定还原特定的 END_LSN，备份文件中会记录一个备份结束的 LSN，如果指定 END_LSN，则必须保证该 END_LSN 大于备份文件中的记录的最后一个 LSN，否则会无视该 END_LSN，而还原到最新状态，如果同时指定了 TIME 则会以最早的为标准。

还原数据库 test 到指定 LSN：20000

```
DMRESTORE  INI_PATH=c:\dmdata\dameng\dm.ini FILE=c:\dmdata\backup\test.bak
END_LSN=20000  ARCHIVE_DIR = c:\dmdata\dameng\arch
```

27.3.2.1.4 使用并行映射文件还原

对于并行备份产生的备份文件，系统支持利用还原工具对数据库进行并行还原。与并行备份一样，并行还原需要用户指定并行映射文件的存储位置。由于并行备份可能是增量备份，而备份的基础备份也有可能是并行备份，因此执行并行还原需要指定一个映射文件存放的目录，用于还原的并行备份的映射文件都存放在该目录下。并行还原能提高还原性能，使得数据服务迅速还原。需要说明的是，备份时的映射文件内容应与还原时映射文件内容保持一致，但这不是必需的，系统在还原前，会对用户设置的映射文件内容进行校验，判断映射文件内容是否与备份时映射文件内容兼容，保证系统能正确还原数据库。

利用并行备份文件 c:\dmdata\backup\test.bak 还原数据库，并行映射文件为 E:\bak.parallel。

```
DMRESTORE  INI_PATH=c:\dmdata\dameng\dm.ini FILE=c:\dmdata\backup\test.bak
PARALLEL_DIR=E:\
```

27.3.2.1.5 使用不同库归档恢复数据库

使用不同库的归档恢复数据库，应用场景如下：

1．创建一个源库 D；

2．做一个备份 B；

3．在 D 上做一些操作， 目的是生成一些归档 A；

4． 创建一个新库 D2；

5．用 B+A 还原恢复；

6．在 D2 上做一些操作， 生成一些归档 A2；

7．现在有了 B，A，A2，理论上应该可以恢复出一个最新状态的 D2。

对应上述场景，操作步骤如下：

1.初始化源库和目标库，待备份和还原。

```
--D
dminit path=e:\tmp\del1
--D2
dminit path=e:\tmp\del2
```

2.源库配置归档并启动,归档路径为 e:\tmp\del1\dameng\arch。

```
dmserver e:\tmp\del1\dameng\dm.ini
```

3.备份源库，可以联机或脱机备份，这里以联机为例，备份文件为 B。

```
backup database bakfile'e:\tmp\db_del1_bak_01.bak';
```

4.操作源数据库，生成一些归档 A。

```
drop table test;
create table test(c1 int);
insert into test values(1);
commit;
select * from test;
```

5.利用源库的备份及归档还原数据库到 D2。

```
dmrestore ini_path=e:\tmp\del2\dameng\dm.ini
file=e:\tmp\db_del1_bak_01.bak ARCHIVE_DIR=e:\tmp\del1\dameng\arch
```

6.配置归档，启动数据库 D2，并执行一些操作生成归档 A2，归档路径为 e:\tmp\del2\dameng\arch。

```
dmserver e:\tmp\del2\dameng\dm.ini
insert into test values(2);
insert into test values(3);
commit;
select * from test;
```

查询结果：

```
行号      C1
---------- -----------
1         1
2         2
3         3
```

7.关闭数据库 D2，利用备份文件 B、归档 A、A2 还原数据库到 D2。

■ 利用 B+A 还原目标库

```
dmrestore ini_path=e:\tmp\del2\dameng\dm.ini
file=e:\tmp\db_del1_bak_01.bak ARCHIVE_DIR=e:\tmp\del1\dameng\arch
```

■ 查看归档 A2 的 db_magic。

```
c:\dmdbms\bin>dmrachk.exe
arch_fil=E:\TMP\del2\DAMENG\arch\ARCHIVE_LOCAL1_20151123102621688.log
rachk V7.1.5.22-Build(2015.11.17-62910trunc)
/*****************************************************************/
归档文件 E:\TMP\del2\DAMENG\arch\ARCHIVE_LOCAL1_20151123102621688.log 明细.
arch_status    : INACTIVE
arch n_rpags   : 377
arch_db_magic  : 1448245568
arch_pemnt_magic : 1448244670
arch_next_seq  : 9023
arch_lsn       : 26611
arch_seq       : 8647
clsn          : 29330
rarch file len : 197120
rarch file free: 197120
rarch create time  : 2015-11-23 10:26:21
rarch close time  : 2015-11-23 10:26:21
/*****************************************************************/
```

■ 修改目标库 D2 的 db_magic 与 A2 的一致，A2 的 db_magic 为 1448245568。

```
dmmdf E:\TMP\del2\DAMENG\SYSTEM.DBF 1
dmmdf E:\TMP\del2\DAMENG\DAMENG01.log 2
```

■ 利用归档 A2 恢复。

```
dmrestore ini_path=e:\tmp\del2\dameng\dm.ini
ARCHIVE_DIR=e:\tmp\del2\dameng\arch res_type=2
```

8.启动数据 D2 查看结果。

```
select * from test;
行号      C1
---------- -----------
1         1
2         2
3         3
```

27.3.3 还原表空间

DM 提供 SQL 命令对表空间进行还原。与数据库还原类似，表空间备份分为完全备份和增量备份，但表空间备份没有联机备份和脱机备份之分，表空间备份只支持联机备份。表空间还原根据不同的表空间备份类型选择不同的还原策略对表空间进行还原。与库级还原和表级还原不同，表空间还原依赖于归档日志，利用备份文件还原表空间只能还原到备份时所在数据库中，并且还原后，表空间处于最新状态。由于待还原表空间的文件名和文件数目可能与备份时的不一致，因此还原后需要更新文件系统。另外，由于从备份表空间到还原表空间的时间间隔中，系统可能累积了很多归档日志，为了保证跨表空间的对象数据同步，需要重做该表空间的所有归档日志。

系统提供联机 SQL 命令执行表空间还原，还原前需要将表空间置为脱机状态，不支持对系统表空间、日志表空间、回滚表空间、临时表空间执行还原操作。

```
RESTORE TABLESPACE <表空间名> FROM '<备份路径>' | BACKUP <备份文件名>
[ INDENTIFIED BY <密码>] [ WITH BACKUPDIR '<备份目录>'{,'<备份目录>'}][WITH
```

```
ARCHIVEDIR ‘归档目录’{,’归档目录’}][DBFILE <文件 ID> TO ‘<文件路径>’ {,DBFILE <文件 ID> TO ‘<文件路径>’}] [MIRROR <文件 ID> TO ‘<文件路径>’{,MIRROR <文件 ID> TO ‘<文件路径>’}];
```

参数说明：

(1) <表空间名> 需要还原的表空间名称；

(2) <备份路径> 备份文件存放的完整路径；

(3) <备份文件名> 备份的名称，在 DMDBMS 中以此标识不同的备份；

(4) <密码> 加密备份表空间时，用户设置的密码；

(5) <备份目录> 搜集备份文件的目录；

(6) <归档目录> 还原时，搜集归档文件的目录；

(7) <文件 ID> 需要重新设置文件路径文件对应的文件号；

(8) <文件路径> 设置还原后，数据文件的路径；

(9) MIRROR 要使用数据文件镜像，必须在建库时开启页校验的参数 page_check。

例如，利用备份文件 C:\ts.bak 还原表空间 t1。

```
ALTER TABLESPACE t1 OFFLINE;
RESTORE TABLESPACE t1 FROM 'C:\tablespace_bak.bak' IDENTIFIED BY ABCDEF;
ALTER TABLESPACE t1 ONLINE;
```

对于表空间联机还原，还原前需要将表空间置为脱机状态；还原后，为了使用表空间，需要重新将表空间置为联机状态。

在表空间还原的时候，如果出现异常（如掉电），还原表空间的操作异常终止，那么表空间可能已经被损坏。这种情况下，只有再次还原才能够修复。

27.3.4 还原用户表

与还原数据库和还原表空间不同，表备份文件没有完全备份和增量备份之分，这是由它的底层实现决定的。数据库备份和表空间备份都属于物理备份范畴，而表备份属于 B 树备份范畴。还原用户表不是简单拷贝备份文件的数据，而是根据备份文件的数据重建表。重建表的本质就是重建索引的过程。因此，还原用户表，会首先还原该表的聚簇索引，然后依次还原二级索引。若待还原的表不存在，则系统会自动重建该表，并将数据还原到备份时的状态。系统提供联机 SQL 命令执行还原用户表，不支持临时表和垂直分区子表的还原。

```
RESTORE TABLE  FROM  '<文件路径>' | BACKUP <备份名> [IDENTIFIED BY <密钥>]
```

参数含义参见 27.3.3 还原表空间这一节。

例如，利用备份文件 C:\table.bak 还原表。

```
RESTORE TABLE FROM 'C:\table_bak.bak' IDENTIFIED BY ABCDEF;
```

第28章 数据复制

28.1 概述

达梦数据复制（DATA REPLICATION）是一个分担系统访问压力、加快异地访问响应速度、提高数据可靠性的解决方案。将一个服务器实例上的数据变更复制到另外的服务器实例。可以用于解决大、中型应用中出现的因来自不同地域、不同部门、不同类型的数据访问请求导致数据库服务器超负荷运行、网络阻塞、远程用户的数据响应迟缓的问题。

28.2 重要概念

1. 数据库状态

服务器的状态，在不同的状态下，对能够进行的操作有不同的限制。详见数据守护部分。

2. 主服务器

发起复制操作的服务器，称为主服务器。

3. 从服务器

接收主服务器发送的数据并进行复制的服务器，称为从服务器。

4. 复制节点

涉及到复制的服务器，主服务和从服务的统称。一个节点既可以是主服务器也可以是从服务器。

5. 复制服务器（RPS）

在数据复制环境中，负责配置复制环境，定义复制关系的服务器。RPS 有且仅有一台，它只负责配置和监控，并不参与到复制过程中。

6. 复制关系

复制关系指明主服务器和从服务器以何种方式进行复制。按照复制的方式，复制关系分为同步复制和异步复制。

7. 同步复制关系

主服务器数据更新立即复制到从服务器。

8. 异步复制关系

主服务器和从服务器在某段时间内数据可能是不同的，主服务器数据更新不会立刻同步到从服务器，而是在经过一段时间后才进行复制。异步复制的同步时机由指定的定时器确定。

9. 逻辑日志

记录产生数据变化的逻辑操作的日志。记录的逻辑操作包括 INSERT、UPDATE、DELETE、TRUNCATE、ROLLBACK 和 COMMIT。

10. 复制源对象

主服务器上作为复制数据源的对象，可以是库、模式或表。在该对象上的操作都会被记录成逻辑日志，发送给从服务器进行复制操作。

11. 复制目标对象

从服务器上作为复制数据目标的对象。从服务器接收到逻辑日志后，将复制源对象的变

化复制到复制目标对象中。

12. 复制对象映射

一对复制源对象和复制目标对象构成一个复制对象映射。构成映射的源对象和目标对象必须是同一类型的对象。根据对象的类型，复制映射分为库级、模式级和表级三个级别。其中表级要求源表和目标表结构完全一致，库级和模式级没有要求。库级和模式级复制映射会将其 DDL 也进行复制。复制映射包括只读模式和非只读模式。对于只读模式的映射，映射的目的表禁止用户更新。

13. 复制组

一组逻辑相关的复制关系可以构造成为复制组。通过复制组，可以构造出一对多复制、多对一复制、级联复制、对称复制、循环复制等复杂的逻辑复制环境。

28.3 体系构架

数据复制系统由以下部件构成：

1. 复制服务器；
2. 复制节点；
3. 各实例站点间通讯的 MAL 系统。

除了系统管理员通过复制服务器定义复制和处理异常外，其他部分的处理及主从服务器之间复制操作对于用户是透明的。

整个复制环境的配置境况如下图所示。

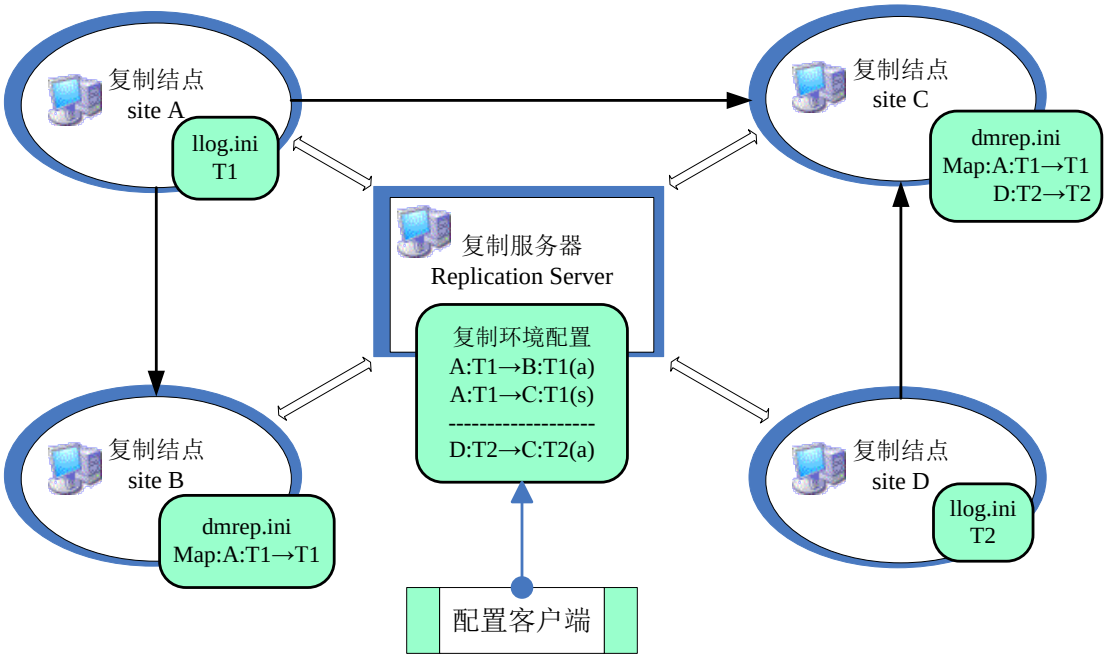


图 28.1 复制环境结构图

在整个环境中有且仅有一台复制服务器（RPS），用户通过 RPS 定义复制及复制环境，但 RPS 并不参与到复制过程中。

DM7 中，将复制逻辑日志按照配置归档到本站点指定目录称为本地归档，将日志的发送称为日志的远程归档。

复制节点上，与复制相关的配置文件有 dmtimer.ini、dmllog.ini、dmrep.ini。各个配置文件在复制中的功能如下表所示。

表 28.1 复制中各配置文件的作用

名称	简介	有效节点	功能

dmtimer.ini	定时器配置文件	主服务器	记录异步复制的定时器信息
dmllog.ini	逻辑日志配置文件	主服务器	记录复制源对象及其逻辑日志相关的信息(包括逻辑本地归档信息和远程归档信息)
dmrep.ini	复制信息配置文件	主服务器/从服务器	主服务器上记录复制相关逻辑日志的编号; 从服务器上记录复制关系的主服务器信息以及复制对象映射的信息

这些配置文件内的信息意义见第 2 章。这些文件均是 RPS 依据用户配置通过 MAL 系统自动生成或修改的，管理员不需要手动进行管理。

在数据复制过程中，除了配置文件，其他的重要文件如下表。

表 28.2 复制中的重要文件

名称	有效节点	功能
逻辑日志文件	主服务器	在数据文件夹内的 llog_01.log 和 llog_02.log 两个文件，记录在执行过程中所有的逻辑日志。
逻辑日志归档文件	主服务器	根据配置，生成在指定的逻辑日志归档路径中，名称为 llog 生成时间.log，是特定逻辑日志下所有记录的本地归档文件，也是发送给复制从服务器的数据来源
复制数据文件	从服务器	在数据文件夹下的 repx（x 为复制关系的 id）下，以 repx_y.rep（x，y 为该文件内记录的编号）为文件名，是从服务器接收到复制数据之后的临时缓存文件，也是复制执行的依据。

这些文件都是流式文件，主服务器在执行过程中，逻辑日志先记录到逻辑日志文件中，并根据配置的信息，将逻辑日志分别分发归档到不同的逻辑日志归档文件中，并在恰当的时机将逻辑日志归档文件的内容发送给从服务器。

从服务器接收到一批数据就产生一个复制数据文件将数据存储其中，防止从服务器复制速度低导致复制数据的丢失。

逻辑日志文件大小限制为 32M，两个文件交替使用。

归档路径内每个逻辑日志归档文件大小限制为 32M，文件写满后增加新文件来存放新的记录。在 dmllog.ini 中有配置归档路径内所有文件总的空间限制，若达到限制的空间，则会删除之前的文件；若配置为无空间限制，则管理员可根据复制情况进行删除。

复制数据文件是从服务器一次接收的复制数据的临时文件，其大小不会超过 32K；在其数据复制结束后会被自动删除。

28.4 配置数据复制

配置数据复制在 RPS 上进行，所有的配置接口见《DM_SQL》附录 3 的“5）数据复制管理”章节。在配置数据复制之前，需要保证复制服务器和所有待配置节点的实例名各不相同，配置好其 MAL 系统并保证网络环境正常。按照复制组、复制关系、复制表映射的顺序配置复制环境。

下面举一个简单的例子来说明数据复制的配置。

1. 准备工作

参与复制的复制实例的信息如下表所示。

表 28.3 参与复制的实例信息

服务器	实例名	IP 地址	服务器端口号	MAL 端口号	文件目录
复制服务器	A	192.168.0.11	5236	5241	C:\data\DAMENG
主服务器	B	192.168.0.12	5236	5242	C:\data\DAMENG

从服务器	C	192.168.0.13	5236	5243	C:\data\DAMENG

假设 B 上存在 USER1.T1 表,C 上存在与 B 上 USER1.T1 表结构完全相同的 USER2.T2 表，现需要创建一个 B 上 USER1.T1 表到 C 上 USER2.T2 表的同步复制关系，其名称为 REPB2C。

2. 参数设置

dm.ini 配置，分别修改 dm.ini 中对应项如下表所示。

表 28.4 复制中 dm.ini 的配置

服务器	dm.ini 设置
复制服务器	INSTANCE_NAME = A PORT_NUM = 5236 MAL_INI = 1
主服务器	INSTANCE_NAME = B PORT_NUM = 5236 MAL_INI = 1
从服务器	INSTANCE_NAME = C PORT_NUM = 5236 MAL_INI = 1

dmmal.ini 配置，用户实际配置时需根据情况更改对应项。

```
MAL_CHECK_INTERVAL  = 5
MAL_CONN_FAIL_INTERVAL = 5
[MAL_INST1]
  MAL_INST_NAME     = A
  MAL_HOST          = 192.168.0.11
  MAL_PORT          = 5241
  MAL_INST_PORT     = 5236
  MAL_INST_HOST     = 192.168.0.11

[MAL_INST2]
  MAL_INST_NAME     = B
  MAL_HOST          = 192.168.0.12
  MAL_PORT          = 5242
  MAL_INST_PORT     = 5236
  MAL_INST_HOST     = 192.168.0.12

[MAL_INST3]
  MAL_INST_NAME     = C
  MAL_HOST          = 192.168.0.13
  MAL_PORT          = 5243
  MAL_INST_PORT     = 5236
  MAL_INST_HOST     = 192.168.0.13
```

每个站点的 dmmal.ini 配置必须一致，一个站点配置好后可直接拷贝到另外两个站点。

3. 复制服务器初始化

如果是第一次使用复制服务器，需要对复制服务器执行初始化操作。通过执行系统函数 SP_INIT_REP_SYS(create_flag)来初始化复制服务器。其主要作用是创建复制用户（SYSREP/SYSREP）和创建复制服务器上需要的系统表。SP_INIT_REP_SYS 的参数 create_flag 为 1 时表示创建用户和系统表，为 0 时表示删除用户和系统表。

4. 环境的配置

以上工作完成后，即可进行复制环境的配置了。

1) 启动 3 台服务器，启动的顺序不分先后。

2) 登录 RPS A，保证服务器状态为 OPEN，开始复制配置。

3) 创建复制组 REP_GRP_B2C

```
SP_RPS_ADD_GROUP('REP_GRP_B2C', '主从同步复制');
```

4) 开始设置

```
SP_RPS_SET_BEGIN('REP_GRP_B2C');
```

5) 添加复制关系

```
SP_RPS_ADD_REPLICATION ('REP_GRP_B2C', 'REPB2C', 'B 到 C 的同步复制', 'B', 'C',
NULL, 'D:\REPB2C');
```

6) 添加复制映射

```
SP_RPS_ADD_TAB_MAP('REPB2C', 'USER1', 'T1', 'USER2', 'T2', 0);
```

7) 提交设置

```
SP_RPS_SET_APPLY();
```

至此，复制环境配置完成。以上的例子只是一个最简单的复制环境。复制的配置灵活，在同一个复制组内，一个主服务器可以有多个从服务器，一个复制节点可以既是主服务器又是从服务器。管理员可以根据实际需要，配置出对称、一对多、多对一、级联、循环的复制环境。

在配置过程中或配置完成后，可以对复制的配置进行修改。修改包括复制组、复制关系、复制对象的删除和复制关系属性的修改。这些修改操作都必须在开始复制 SP_RPS_SET_BEGIN 和提交复制 SP_RPS_SET_APPLY 之间进行。若需要删除复制组，则该复制组不能处于配置阶段，即该组的配置已经提交或取消。

删除复制映射

```
SP_RPS_DROP_TAB_MAP('REPB2C', 'USER1', 'T1', 'USER2', 'T2');
```

添加一个定时器，将同步复制修改为异步复制

```
SP_RPS_ADD_TIMER('B2C_TIMER','',1,0,0,0,'19:50:33',NULL,'2011-08-24
19:50:33',NULL,1);
SP_RPS_REP_RESET_TIMER('REPB2C','B2C_TIMER');
```

修改复制的错误超时时间，超时的时间单位是秒

```
SP_RPS_SET_ROUTE_FAULT_TIMEOUT('REPB2C', 60);
```

删除复制关系

```
SP_RPS_DROP_REPLICATION('REPB2C');
```

删除整个复制组

```
SP_RPS_DROP_GROUP('REP_GRP_B2C');
```

另外，在配置或修改配置时想要取消操作，可以使用如下系统过程结束配置。

```
SP_RPS_SET_CANCEL();
```

28.5 监控数据复制

28.5.1 复制故障监控

配置完成后复制服务器 RPS 虽然不参与复制的具体执行，但是在复制过程中，还是建议保持 RPS 的运行来对复制进行监控。

复制系统内，状态分为复制节点的状态和复制关系的状态。具体内容见下表。

表 28.5 节点状态和复制关系状态

类型	子类型	意义	值
节点实例	系统状态	正常	0
		恢复	1
		系统异常	2
	网络状态	正常	0
		连接异常	4
复制关系	复制关系状态	正常	0
		连接异常	1
		待删除	512
		已在主服务器删除	528
		已在从服务器删除	513

上表复制节点的系统状态在 sysrep.rps_instances 中的 VALID_FLAG 字段表示；网络状态在 sysrep.rps_instances 表中 NET_VALID_FLAG 的字段表示。

复制关系的状态在 sysrep.rps_replications 中的 VALID_FLAG 字段表示。

复制系统的配置信息可以通过在复制 RPS 上查询系统表来实现。在 RPS 上，复制相关的信息都在 sysrep 模式中，相关的表及其各字段的意义见附录 5。

复制的监视按复制组为单位进行，RPS 每隔 1 分钟轮询探查所有的复制组，在每个复制组内，依次探测各个复制节点，要求其返回复制节点本身的状态和其所涉及的复制的状态。

这里需要特别说明的是节点的系统异常状态是指该节点在复制环境中的状态，是一个推测值。当 RPS 无法得到节点的返回消息，其他节点的返回消息中所有涉及到该节点的复制关系都是异常时，RPS 就认为该节点系统异常。

若复制结点或复制关系发生异常，这些异常都会记录在 sysrep.rps_fault_history 表中。若其 END_TIME 字段为 NULL，表示该故障还没有结束，管理员需尽快检查复制节点的状态及其网络连接。

DM7 并没有特意设置对错误历史的管理接口，管理员可以通过对 sysrep.rps_fault_history 表的删除和查询来实现错误记录的管理。

28.5.2 复制故障处理

1. 故障超时设置和处理

在配置阶段，管理员可以配置复制关系和复制节点的故障超时时间。复制运行时当复制关系出现异常或节点出现系统异常时，RPS 会计算故障持续时间，在设置的超时时间以内，RPS 都会尝试进行故障恢复；若发现故障时间超过了设置的超时时间，则会删除该复制关系或节点，当然，删除节点时，该节点涉及的所有复制关系都会被删除。

删除复制关系时，若复制关系的主服务器或从服务器是异常状态，该复制关系不会立刻被清除，它会处于删除状态，在仍正常运行的服务器上删除；等待异常的服务器恢复后，将其从该服务器的复制配置中删除后，RPS 才会彻底的清理该复制关系；这也是复制关系状态中“待删除”、“已从主服务器上删除”和“已在从服务器上删除”状态的原因。

2. 复制关系异常修复

当 RPS 发现某复制关系异常时，RPS 会通知该复制关系的主服务器，要求其重新建立彼此间的网络连接，修复复制关系。

如果修复成功，重置 RPS 上复制关系的状态，并将复制关系的异常计时设为无效；若修复失败，则判断异常时间是否超时，若超时，则进入删除流程，否则等待到下一个监控修复时间进行恢复。

修复成功后，如果是同步复制，则修复后立刻将异常期间的逻辑日志发送给从服务器，

若是异步复制，则会等待定时器触发复制。

3. 复制节点网络故障恢复

RPS 在每次探查阶段中探查节点之前都会检查与节点间的网络连接；发现某结点无法通讯，则会跳过本轮的故障恢复阶段，直到 RPS 能够与之建立连接。

在网络故障期间，管理员可以通过查询 sysrep.rps_fault_history 表中实例网络异常的记录，尽快修复网络异常。

4. 复制节点系统异常恢复

当 RPS 在探查阶段中认定发现某节点系统异常，则会将所有该节点涉及到的复制关系置为异常状态，记录在 sysrep.rps_fault_history 中，等待管理员恢复节点系统。

复制节点系统异常恢复的过程是执行故障超时处理和修复剩余的复制关系两者统一执行的过程。节点系统恢复后，RPS 会根据故障期间故障处理的结果，删除因超时而被删除的复制关系；修复节点内剩余的所有复制关系。

5. 复制错误监控

在复制过程中，特别是在多对一或多对多的复制环境中，可能出现数据冲突。数据冲突发生后，从服务器会将其记录在相应的日志中，RPS 每隔固定时间会轮询所有结点，收集其间产生的数据冲突记录并将其插入到 sysrep.rps_conflicts 表中，管理员可以通过查询该表来查看。

DM7 并没有特意设置对复制冲突记录的管理接口，管理员可以通过对 sysrep.rps_conflicts 表的删除和查询来实现错误记录的管理。

28.6 复制用户和系统表

执行系统函数 SP_INIT_REP_SYS(1)创建的复制用户和系统表如下：

一 复制用户，被授予 RESOURCE 角色

```
用户名：SYSREP
密码：SYSREP
```

二 复制系统表

```
--1.复制组表
TABLE SYSREP.RPS_GROUPS(
    NAME        VARCHAR(128),               --组名
    ID          INT,                              --组 ID
    DESC$       VARCHAR(1024),            --组描述
    CLUSTER PRIMARY KEY (NAME)
);

--2.复制实例表
TABLE SYSREP.RPS_INSTANCES(
    INST_NAME        VARCHAR(128),     --复制节点实例名
    GRP_ID           INT,                    --复制组 ID
    INST_ID          INT,                    --实例在复制组中编号
    FAULT_TIMEOUT     INT,                --故障超时处理值，以秒为单位，0 为立即超时
    VALID_FLAG      INT,            --节点有效标记，1 或 0
    FAULT_TIME      DATETIME,            --节点故障开始时间
    NET_VALID_FLAG  INT,             --网络有效标记，1 或 0
    NET_FAULT_TIME  DATETIME,         --网络故障开始时间
```

```
    CLUSTER PRIMARY KEY(GRP_ID, INST_NAME)
);

--3.复制关系表
TABLE SYSREP.RPS_REPLICATIONS (
    REP_NAME       VARCHAR(128),            --复制名
    GRP_ID         INT,             --复制组 ID
    REP_ID         INT,             --复制 ID，全局唯一
    MINST_ID         INT,               --主节点实例编号
    SINST_ID         INT,               --从节点实例编号
    ARCH_DIR       VARCHAR(256),             --主节点归档日志目录
    FAULT_TIMEOUT   INT,                   --故障超时处理值，以秒为单位，0 为立即超时
    VALID_FLAG    INT,                --复制关系状态值
    FAULT_TIME    DATETIME,           --故障开始时间
    SYNC_FLAG           INT,           --同步、异步标记
    TIMER_NAME          VARCHAR(128),  --异步定时器名称
    DESC$          VARCHAR(1024),         --复制描述
    CLUSTER PRIMARY KEY(GRP_ID, REP_NAME)
);

--4.复制映射表
TABLE SYSREP.RPS_TABMAPS(
    REP_ID         INT,                    --复制 ID
    MSCH_NAME      VARCHAR(128),    --主表模式名
    MTAB_NAME      VARCHAR(128),    --主表名
    MSCH_ID       INT,              --主表模式 ID
    MTAB_ID        INT,                    --主表 ID
    SSCH_NAME      VARCHAR(128),    --从表模式名
    STAB_NAME      VARCHAR(128),    --从表名
    SSCH_ID         INT,             --从表模式 ID
    STAB_ID        INT,                    --从表 ID
    READONLY_MODE INT,            --映射模式 1:只读模式，0：非只读模式
    CLUSTER PRIMARY KEY(REP_ID, MTAB_ID, STAB_ID)
);

--5.故障历史表
TABLE SYSREP.RPS_FAULT_HISTORY(
    GRP_NAME    VARCHAR(128),                    --复制组 ID
    OBJ_NAME    VARCHAR(128),                    --故障节点或关系编号
    FAULT_TYPE  VARCHAR(128),       --故障类型，'INSTANCE'或'ROUTE'
    START_TIME  DATETIME,             --故障开始时间
    END_TIME      DATETIME              --故障结束时间
);

--6.冲突历史表
TABLE SYSREP.RPS_CONFLICTS
(
    SEQ_NO          BIGINT,                     --序列号
```

```
    REP_ID          INT,                                --对应复制号
    INST_ID         INT,                                --产生冲突的实例编号
    TABLE_ID        INT,                                --冲突表 ID
    TYPE                BYTE,                               --冲突类型(更新冲突，插入冲
突)
    OCC_TIME        DATETIME,                   --冲突产生时间
    KEY_DATA        VARCHAR(8000),      --冲突数据的 PK 值，如包含多个 KEY 值，则以逗号
分隔。如键值超长则截断
    CLUSTER    PRIMARY KEY(SEQ_NO)
);

--7.复制定时器表
TABLE SYSREP.RPS_TIMERS(
    NAME            VARCHAR(128),     --定时器名称
    TYPE$                   INT,   --定时类型，取值可参见 SP_ADD_TIMER
    FREQ_INTERVAL            INT,      --间隔天数
    FREQ_SUB_INTERVAL      INT,    --间隔的月/周（调度类型决定）数
    FREQ_MINUTE_INTERVAL    INT,               --间隔的分钟数
    START_TIME           TIME,             --开始时间
    END_TIME               TIME,           --结束时间
    DURING_START_DATE      DATETIME,      --开始时间点
    DURING_END_DATE      DATETIME,    --结束时间点
    NO_END_DATA_FLAG    INT,
  DESC$              VARCHAR(1000),     --定时器描述
    CLUSTER PRIMARY KEY(NAME)
);
```

第29章 大规模并行处理 MPP

29.1 概述

为了支持海量数据存储和处理、高并发处理、高性价比、高可用性等需求，提供高端数据仓库解决方案，达梦数据库提供了大规模并行处理 MPP 架构，以极低的成本代价，为客户提供业界领先的计算性能。达梦采用完全对等无共享（share-nothing）的 MPP 架构，支持 SQL 并行处理，可自动化分区数据和并行查询，各实例无 I/O 冲突。

MPP 系统工作起来就像是一台单独的计算机，由于采用自动化的并行处理，在分析型数据仓库等 OLAP 应用中，查询性能比传统的单节点数据库大大提高。

MPP 系统为新一代数据仓库所需的大规模数据和复杂查询提供了先进的软件级解决方案，具有业界先进的架构和高度的可靠性，能帮助企业管理好数据，使之更好的服务于企业，推动数据依赖型企业的发展。

29.2 基本概念

1. MPP

Massive Parallel Processing，大规模并行处理。MPP 架构最多可以同时部署 1024 个节点，采用统一的并行操作数据库引擎，将数据分散在不同的数据库节点上，在高速的内部网络环境下，对于海量数据的并发查询可极大地减少 I/O，提高查询效率。

2. 执行节点（EP）

MPP 环境内每个实际运行的实例节点称为一个执行节点（基于主备的 MPP 环境内备库除外）。

3. 主、从 EP

MPP 是对等无共享架构，对整个系统来说，每个节点作用都是一样的，用户可以连接到其中的任何一个进行操作。对每个用户会话来说，执行节点具有主从之分。用户会话实际连接的那个节点称为主 EP，其余的节点都称为从 EP。

29.3 体系架构

达梦的 MPP 架构提供数据存储和查询的自动化并行，插入的数据会根据表定义的类型被自动分区保存到相应的节点，查询时在各节点的数据库引擎上并行执行查询，并将所有结果数据自动汇集到主查询节点上，数据无共享、无 I/O 冲突。

可靠性方面，提供了数据守护方案，满足用户对数据可靠性、安全性的需求，解决由于硬件故障、自然灾害等原因导致的数据库服务长时间中断问题。MPP 系统中的每个节点都可配置数据守护系统，增加多个节点数据的完整冗余镜像。节点和镜像节点之间可以互相转换成 MPP 系统中的执行节点。

系统体系架构图如下图所示：

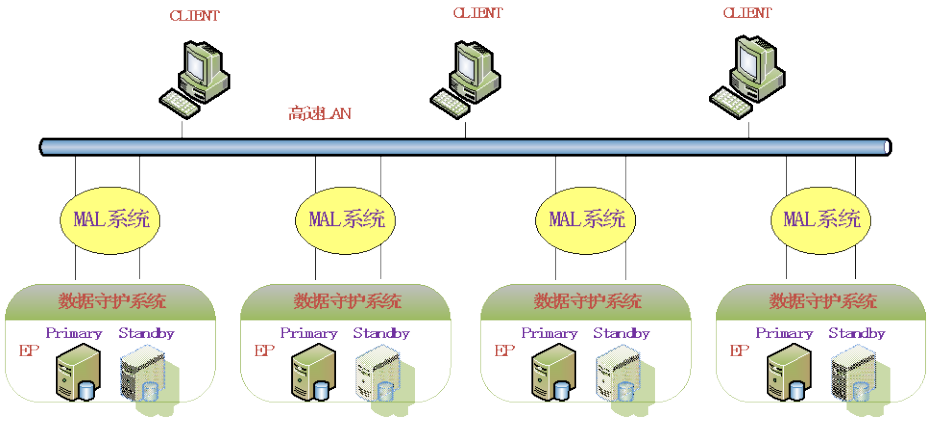


图 29.1 MPP 体系架构

客户端可连接任意一个 EP 站点进行操作，所有 EP 对客户来说都是对等的。图中互为镜像的节点是配置了守护进程的主备库，在主库发生故障时可以通过备库接管将镜像备库切换为主库，继续提供对外服务，大大提高了系统的可用性和可靠性，为了更加严格的保证系统可靠性，目前 MPP 主备系统不再支持自动切换模式。在确保节点安全的情况下，用户也可以不用配置数据守护系统。

MPP 系统内每个节点只负责自身部分数据的读写，执行计划是完全并行执行，完全无 I/O 冲突，充分利用硬件资源，只是额外增加了数据在节点间相互传递的通信时间，在高速局域网内，通信时间占整体时间的比例更小，更能体现大规模并行处理的优势，随着系统规模的扩大，并行支路越多，优势越明显。

通用数据库主要是是针对 OLTP 处理功能设计的，在运行大量小规模交易查询数据时效果最好。从结构上来说，采用了完全共享或磁盘共享架构。完全共享体系局限于单节点服务器，通常价格比较昂贵，其扩展性和性能受到相应的限制。磁盘共享体系允许系统带有多个服务器实例，这些实例与 SAN 或其它共享存储设备相连。这种体系需要通过一个狭窄的数据管道将所有 I/O 信息过滤到昂贵的共享磁盘子系统。

与此相比，基于硬件的数据仓库平台也采用完全无共享体系。在这种体系下，通讯功能部署在一个高宽带网络互连体系上，用户通过一个主控制节点执行并行查询。该体系的一个重要优势就是每个节点都有一个通往本地磁盘的独立高速通道，不但简化了体系，还提供良好的扩展性。但这些平台所面临的困难是需要价格昂贵的专用硬件，显著增加了系统成本。

达梦 MPP 的完全对等无共享体系架构结合了完全无共享体系的优点，在此基础上又前进了一步，通过软件提升处理能力，它不需要价格昂贵的专用硬件，也不采用增加主控制节点来协调所有并行处理的主从式方法，而是各个节点完全对等，更进一步简化了体系的实现，也消除了系统可能存在的主节点瓶颈问题。

综上所述，将主流架构特点比较列于下表中

表 29.1 主流架构特点比较

架构名称	特点
完全共享	局限于单节点服务器，价格昂贵，扩展性、性能受限。
共享磁盘	允许多个服务器实例共享存储设备，可有效解决单实例负载问题，具有一定的扩展性，但当节点增加到一定程度以后，由于对 I/O 资源、锁资源等的激烈竞争，反而导致性能的下降，扩展性和性能在系统规模变大时受限。同时共享磁盘等硬件成本也十分昂贵。
完全不共享	部署在高速网络，各节点相对独立，无共享 I/O，扩展性和性能良好，缺点是某些此类架构的系统基于硬件，价格昂贵，成本高，系统中有一个主控节点，系统规模扩充时可能成为瓶颈，主控节点无备份，容易形成单点故障。

完全对等不共享	DM7 采用的是 MPP 架构，继承了完全不共享架构的优点，且各节点完全对等，不需要专用硬件，不存在主控节点，消除了潜在瓶颈以及单节点故障问题。

29.4 支持的功能

29.4.1 支持多种分布

支持随机分布、复制分布（fully）、哈希分布、范围分布、LIST 列表分布等多种分布表的类型，用户可根据实际情况选择合适的分布类型。详见《DM_SQL》中建表语法部分。

29.4.2 函数和过程

支持系统内部函数、用户定义的函数、过程等。支持函数、过程中的查询语句，支持过程中动态执行语句。

29.4.3 支持触发器

支持系统所有的触发器类型，包括事件触发器、时间触发器、DML 触发器、视图触发器（Instead of 触发器）等。

29.4.4 支持分区表

支持水平分区表、垂直分区表，即实现“数据分布后再分区”。在数据分布到各节点的基础上后，再在单个节点上将数据再次分区，可进一步提高查询性能。分布的类型和分区的类型可以混合搭配，比如建立范围分布的水平分区堆表。

29.4.5 自增列和序列

所有节点维护一个可以同步的自增列和序列值。mpp 下暂时不支持创建带自增列的表的聚簇索引。

29.4.6 支持大字段

MPP 下支持带大字段的表。

29.4.7 支持列存储

1. 支持 HFS 表

借助巨型表空间优化列存储数据的读写效率，从而实现对列存储表数据装载以及查询扫描的性能提升。MPP 下对巨型列存储表数据进行分布，进一步提高了列存储的读写性能。

29.4.8 支持事务的 ACID 特性

MPP 系统中，实现了事务的两阶段提交，包括预提交阶段和提交阶段。借助 monitor 协调作用，在单个或多个节点故障重启时，依然能保证事务的 ACID 特性，保证了 MPP 系统的可靠性。

29.5 数据加载

支持数据快速加载，MPP 下分布表的数据在主 EP 上实现重分发，用法和普通表的加载一样。

29.6 环境配置

达梦的 MPP 环境包含两种配置模式。一种是普通配置模式，一种是基于主备的配置模式。当用户对系统的可靠性要求不高、系统可中断、能保证节点安全或进行简单的 MPP 测试时，只需使用普通配置模式，否则可使用基于主备的的配置模式。

基于主备的配置模式下，如果主库出现故障，可以等待主库恢复或者使用命令手动接管故障主库，以继续对外提供服务。

29.6.1 普通配置

普通配置模式下，每个执行节点 EP 不配置数据守护系统，系统中只包含所有 EP 节点。下面举例包含 2 个实例节点的 MPP 普通配置，步骤如下：

1. 规划

表 29.2 普通配置节点规划

实例名	机器 IP	服务器端口	MAL 端口	MPP_SEQNO
EP01	192.168.0.1	5236	5269	0
EP02	192.168.0.2	5237	5270	1

2. 配置 dm.ini

首先，初始化 EP01，EP02 两个库，两个库的 dm.ini 参数要求完全一致。然后分别配置两个库中的 dm.ini。

EP01 的 dm.ini 配置如下：

```
INSTANCE_NAME  = EP01
PORT_NUM      =  5236

MAL_INI       =  1
MPP_INI        =  1
```

EP02 的 dm.ini 配置如下：

```
INSTANCE_NAME = EP02
PORT_NUM      =  5237

MAL_INI       =  1
```

```
MPP_INI       =  1
```

3. 配置 dmmal.ini

两个实例配置一样，直接拷贝即可。

```
[mal_inst1]
  mal_inst_name = EP01
  mal_host      = 192.168.0.1
  mal_port      = 5269
  mal_inst_host  = 192.168.0.1
  mal_inst_port  = 5236

[mal_inst2]
  mal_inst_name = EP02
  mal_host      = 192.168.0.2
  mal_port      = 5270
  mal_inst_host  = 192.168.0.2
  mal_inst_port  = 5237
```

4. 配置 dmmpp.ctl

dmmpp.ctl 是一个二进制文件，用户不能直接配置。用户需要首先配置 dmmpp.ini，然后利用 dmctlcvt 工具进行转换得到生成 dmmpp.ctl 文件。两个实例使用相同的 ctl 文件，只需拷贝一份。

表 29.3 dmmpp.ini 配置项

配置项	配置含义
MPP_2PC_FLAG	可选项，是否采用两阶段方式提交事务。 取值 0，1。0：不采用，1：采用。默认 0，一般使用默认即可。
[SERVICE_NAME]	标识每个实例的选项名
MPP_SEQ_NO	实例在 mpp 系统内的序号，取值范围为 0~1023
MPP_INST_NAME	节点实例名

dmmpp.ini 配置如下：

```
[service_name1]
  mpp_seq_no   = 0
  mpp_inst_name = EP01

[service_name2]
  mpp_seq_no   = 1
  mpp_inst_name = EP02
```

利用工具转换成 dmmpp.ctl。示例如下：

```
dmctlcvt t2c d:\dameng\dmmpp.ini d:\dameng\dmmpp.ctl
```

以上步骤完成之后，只需分别启动两个实例节点，如果能够登录成功，说明配置成功了。

29.6.2 基于主备的配置

基于主备的 MPP 环境配置比普通配置步骤稍复杂一些。MPP 主备系统由 MPP 监控器(MPP_MONITOR)、多个 MPP 节点 (EP)和 EP 的备库共同组成。配置 MPP 系统，涉及到的文件如下：

- 对于 MPP_MONITOR 和每个 EP 站点，需要配置两个文件：dmmal.ini 和

dmmpp.ctl；

- 对于每个 EP，需要启用 dm.ini 中的 MAL_INI、ARCH_INI 和 MPP_INI 参数；
- 对于每个 EP 备库，需要启用 dm.ini 中的 MAL_INI 和 ARCH_INI。

下面以一个具体的实例，说明基于主备的 MPP 系统的配置过程。配置节点数为 2。主库实例名为 EP01，EP02，备库实例名分别为 EP11、 EP22。配置步骤如下：

1． 规划

主库：

表 29.4 基于主备的配置节点规划—主库

实例名	机器 IP	端口	MAL 端口	MPP_SEQNO
EP01	192.168.0.28	5236	5269	0
EP02	192.168.0.29	5237	5270	1

备库：

表 29.5 基于主备的配置节点规划—备库

实例名	对应主库	机器 IP	端口	MAL 端口
EP11	EP01	192.168.0.30	5238	5271
EP22	EP02	192.168.0.31	5239	5272

监控器：

表 29.6 基于主备的配置节点规划—监控器

监控器	机器 IP	MAL 端口
MONITOR	192.168.0.32	5280

2． 初始化库

基于主备的 MPP 配置，要求主备库数据严格一致，因此用户配置主备时，需要保证这一点。所以只需初始化主库的库即可。Windows 下初始化库脚本如下：

```
--192.168.0.28 上
dminit path=D:\MPP_ARCH_TEST\ep01
--192.168.0.29 上
dminit path=D:\MPP_ARCH_TEST\ep02
```

3． 检查主库的数据库状态是否一致

要求主库的状态严格一致，即 lsn 值严格一致。

```
-- 启动 ep01 服务器，以 mount 方式
dmserver D:\MPP_ARCH_TEST\ep01\DAMENG\dm.ini mount
--启动命令行工具 disql，登录主数据库查询 LSN 值
SQL>select file_lsn, cur_lsn from v$rlog;

-- 启动 ep02 服务器，以 mount 方式
dmserver D:\MPP_ARCH_TEST\ep02\DAMENG\dm.ini mount
--启动命令行工具 disql，登录主数据库查询 LSN 值
SQL>select file_lsn, cur_lsn from v$rlog;
```

如果 ep01 和 ep02 的 file_lsn、cur_lsn 值完全一致，说明主库的数据库状态完全一致，那么退出达梦服务，执行步骤 4，否则返回步骤 2。

4． 备库数据准备

各 EP 的备库的数据准备，理论指导详见数据守护相关文档。

本例中使用拷贝的方式，将主库数据拷贝到备库目录下，保证绝对路径完全一致。

```
将 192.168.0.28 的 D:\MPP_ARCH_TEST\ep01 拷贝到 192.168.0.30 的 D:\MPP_ARCH_TEST\ep01
```

将 192.168.0.29 的 D:\MPP_ARCH_TEST\ep02 拷贝到 192.168.0.31 的 D:\MPP_ARCH_TEST\ep02

5. 配置 dm.ini

192.168.0.28 ep01 目录下，实例 EP01 配置如下：

```
INSTANCE_NAME = EP01
PORT_NUM    = 5236
MAL_INI      =  1
MPP_INI       =  1
ARCH_INI          =  1
```

192.168.0.31 ep02 目录下，实例 EP22 配置如下：

```
INSTANCE_NAME = EP22
PORT_NUM    = 5239
MAL_INI      =  1
ARCH_INI         =  1
```

192.168.0.29 ep02 目录下，实例 EP02 配置如下：

```
INSTANCE_NAME = EP02
PORT_NUM    = 5237
MAL_INI       =  1
MPP_INI       =  1
ARCH_INI          =  1
```

192.168.0.30 ep01 目录下，实例 EP11 配置如下：

```
INSTANCE_NAME = EP11
PORT_NUM    = 5238
MAL_INI      =  1
ARCH_INI         =  1
```

6. 配置 dmmal.ini

monitor 和所有 EP 主备站点共用同一套 dmmal.ini。

所以请确保如下目录下都拷贝了 dmmal.ini 文件：

192.168.0.32 的 D:\MPP_ARCH_TEST\monitor；--新创建一个 monitor 文件夹，用来存放 dmmal.ini 和下面介绍的 dmmpp.ctl 文件。

192.168.0.28 的 D:\MPP_ARCH_TEST\ep01\DAMENG；

192.168.0.30 的 D:\MPP_ARCH_TEST\ep01\DAMENG。

192.168.0.29 的 D:\MPP_ARCH_TEST\ep02\DAMENG；

192.168.0.31 的 D:\MPP_ARCH_TEST\ep02\DAMENG；

dmmal.ini 的配置如下：

```
MAL_CHECK_INTERVAL  = 3
MAL_CONN_FAIL_INTERVAL = 3

[mal_inst1]
  mal_inst_name = EP01
  mal_host      = 192.168.0.28
  mal_port      = 5269
  mal_inst_host  = 192.168.0.28
  mal_inst_port  = 5236

[mal_inst2]
  mal_inst_name = EP02
  mal_host      = 192.168.0.29
  mal_port      = 5270
  mal_inst_host  = 192.168.0.29
  mal_inst_port  = 5237
```

```
[mal_inst3]
  mal_inst_name = EP11
  mal_host      = 192.168.0.30
  mal_port      = 5271
  mal_inst_host  = 192.168.0.30
  mal_inst_port  = 5238


[mal_inst5]
  mal_inst_name = EP22
  mal_host      = 192.168.0.31
  mal_port      = 5272
  mal_inst_host  = 192.168.0.31
  mal_inst_port  = 5239


[monitor]
  mal_inst_name = monitor
  mal_host      = 192.168.0.32
  mal_port      = 5280
```

7. 配置 dmmpp.ctl

monitor 和所有 EP 站点共用同一套 dmmpp.ctl 文件。

所以请确保如下目录下都拷贝了 dmmpp.ctl 文件：

192.168.0.32 的 D:\MPP_ARCH_TEST\monitor；

192.168.0.28 的 D:\MPP_ARCH_TEST\ep01\DAMENG；

192.168.0.30 的 D:\MPP_ARCH_TEST\ep01\DAMENG。

192.168.0.29 的 D:\MPP_ARCH_TEST\ep02\DAMENG；

192.168.0.31 的 D:\MPP_ARCH_TEST\ep02\DAMENG；

dmmpp.ctl 是一个二进制文件，用户不能直接配置。需要首先配置 dmmpp.ini，然后利用 ctlcvt 工具进行转换生成 dmmpp.ctl 文件。

dmmpp.ini 配置如下：

```
[service_name1]
mpp_seq_no    = 0
mpp_inst_name   = EP01

[service_name2]
mpp_seq_no    = 1
mpp_inst_name   = EP02
```

利用工具转换成 dmmpp.ctl。示例如下：

```
dmctlcvt                t2c                D:\MPP_ARCH_TEST\ep01\dmmpp.ini
D:\MPP_ARCH_TEST\ep01\dmmpp.ctl
```

8. 配置 dmarch.ini

192.168.0.28 目录 ep01 中，实例 EP01 配置如下：

```
[ARCHIVE_LOCAL1]
ARCH_TYPE       = LOCAL
ARCH_DEST       = D:\MPP_ARCH_TEST\ep01\DAMENG\arch
ARCH_FILE_SIZE  = 128
ARCH_SPACE_LIMIT    = 0 #归档文件限制，单位 MB，取值范围（1024~4294967294），0 表示无空间限制
[ARCHIVE_MPP1]
ARCH_TYPE       = MARCH
ARCH_DEST       = EP11
```

192.168.0.30 目录 ep01 中，实例 EP11 配置如下：

```
[ARCHIVE_LOCAL1]
ARCH_TYPE       = LOCAL
ARCH_DEST       = D:\MPP_ARCH_TEST\ep01\DAMENG\arch
ARCH_FILE_SIZE      = 128
ARCH_SPACE_LIMIT    = 0     #归档文件限制，单位 MB，取值范围（1024~4294967294），0 表示无空间限制
[ARCHIVE_MPP1]
ARCH_TYPE       = MARCH
ARCH_DEST       = EP01
```

192.168.0.29 目录 ep02 中，实例 EP02 配置如下：

```
[ARCHIVE_LOCAL1]
ARCH_TYPE       = LOCAL
ARCH_DEST       = D:\MPP_ARCH_TEST\ep02\DAMENG\arch
ARCH_FILE_SIZE      = 128
ARCH_SPACE_LIMIT    = 0 #归档文件限制，单位 MB，取值范围（1024~4294967294），0 表示无空间限制

[ARCHIVE_MPP1]
ARCH_TYPE       = MARCH
ARCH_DEST       = EP22
```

192.168.0.31 目录 ep02 中，实例 EP22 配置如下：

```
[ARCHIVE_LOCAL1]
ARCH_TYPE       = LOCAL
ARCH_DEST       = D:\MPP_ARCH_TEST\ep02\DAMENG\arch
ARCH_FILE_SIZE  = 128
ARCH_SPACE_LIMIT    = 0     #归档文件限制，单位 MB，取值范围（1024~4294967294），0 表示无空间限制
[ARCHIVE_MPP1]
ARCH_TYPE       = MARCH
ARCH_DEST       = EP02
```

每个 EP 站点和备库将必须配置本地归档，主库归档配置中包含所有备库，每个备库归档配置包含主库和其它备库。

9. 启动主库和备库

为设置服务器状态和模式，服务器需要以 mount 方式启动。

```
--192.168.0.28
dmserver D:\MPP_ARCH_TEST\ep01\DAMENG\dm.ini mount
--192.168.0.30
dmserver D:\MPP_ARCH_TEST\ep02\DAMENG\dm.ini mount
--192.168.0.29
dmserver D:\MPP_ARCH_TEST\ep02\DAMENG\dm.ini mount
--192.168.0.31
dmserver D:\MPP_ARCH_TEST\ep01\DAMENG\dm.ini mount
```

10.设置实例的状态和模式

通过如下 DISQL 命令实现将各站点分别设置为主库或备库模式。分别将 EP01、EP02 设置为主库，将 EP11、EP22 设置为备库。

```
通过 DISQL 命令登录 EP01(192.168.0.28：5236),执行
```

```
SQL>alter database primary;
通过 DISQL 命令登录 EP02(192.168.0.29：5237),执行
SQL>alter database primary;
通过 DISQL 命令登录 EP11(192.168.0.30：5238),执行
SQL>alter database standby;
通过 DISQL 命令登录 EP22(192.168.0.31：5239),执行
SQL>alter database standby;
```

11. 启动 mpp_monitor

windows 下启动脚本：

```
dmmpp_monitor path=D:\MPP_ARCH_TEST\monitor
```

参数 path 为 dmmal.ini 和 dmmpp.ctl 等配置文件以及 montior 运行过程中产生日志文件的存放目录。

基于主备配置的 MPP 系统，monitor 必需按主备模式启动，即 MODE=0。mpp_monitor 工具参数如下表所示。

表 29.7 MPP_MONITOR 工具参数说明

参数	含义	备注
PATH	dmmpp.ctl、dmmal.ini 的绝对目录或者当前目录，最大长度为 256 字节	必选参数
THREAD_INTERVAL	检查 EP 站点的间隔时间，取值范围 1~10 秒，默认值 1	可选参数
SITE_INTERVAL	检查站点间网络超时的间隔时间，取值范围 0 或 3~1800 秒，默认值，不检测超时（此值一般在 dmmal.ini 中的 mal_check_interval 为 0 时才设置）	可选参数
MODE	monitor 启动模式，取值范围 0~1，默认值为 0。0 表示 monior 以主备模式启动；1 表示 monitor 以非主备方式启动，仅处理事务的故障恢复。	可选参数
-NOCONSOLE	以服务方式启动	可选参数
HELP	打印帮助信息	可选参数

12. MPP 系统的退出

对于普通配置的 MPP，用户正常退出各个 EP 站点即可，EP 站点之间的关闭顺序没有先后之分；对于基于主备的配置，用户需首先退出监控器，然后再退出各个 EP 站点，最后退出 EP 备用站点。

29.6.3 监控器使用说明

设置好主备库的状态和模式后，按照先启动所有实例，最后启动 dmmpp_monitor 的次序启动 MPP 主备系统，主备库都以 MOUNT 配置方式启动，次序不分先后。监控器成功启动后，其运行时相关信息记录在 monitor 的配置文件目录的名为 dmmpp_monitor.log 的文件中。

MPP_monitor 监控器的命令说明如下：

帮助 help：

```
h
+----------------------+------------------------------+
|help/h                |   帮助信息                   |
+----------------------+------------------------------+
|show                  |   MPP 主备信息               |
+----------------------+------------------------------+
```

```
|switchover 备库实例名  |   切换至指定备库            |
+----------------------+-----------------------------+
|takeover   主库实例名  |   接管指定的主库            |
+----------------------+-----------------------------+
|exit                 |   退出                      |
+----------------------+-----------------------------+
```

详细信息 show：

```
show
------------------------------MPP 系统信息汇总------------------------------
EP 站点数:  2
系统状态:  正常
切换模式:  NON AUTO FAILOVER

站点号 实例名  实例模式  实例状态    FLSN   CLSN   SLSN   SSLSN
0     EP01    PRIMARY   OK      21198   21198   21198   21198
      EP11    STANDBY   OK      21198   21198   21198   21198

1     EP02    PRIMARY   OK      21198   21198   21198   21198
      EP22        STANDBY      OK              21198      21198      21198
21198--------------------------------------------------------------------------
```

站点号用来标识各个 EP 站点，每组站点下可以查看主节点和对应的备库节点信息。

1. 主库故障处理

使用场景：EP 主站点故障。把 EP01 关闭模拟 EP01 故障，监视器检测到 EP01 故障后自动打印的信息及查看的系统信息如下：

```
2013-09-30 15:02:10  设置主库实例 EP01 为 ERROR 状态
show
------------------------------MPP 系统信息汇总------------------------------
EP 站点数:  2
系统状态:  存在 EP 故障
切换模式:  NON AUTO FAILOVER

站点号 实例名  实例模式  实例状态    FLSN   CLSN   SLSN   SSLSN
0     EP01    PRIMARY   ERROR   21198   21198   21198   21198
      EP11    STANDBY   OK      21198   21198   21198   21198

1     EP02    PRIMARY   OK      21198   21198   21198   21198
      EP22    STANDBY   OK      21198   21198   21198   21198
--------------------------------------------------------------------------
```

重新启动主库，主库会自动恢复，查看信息如下：

```
2013-09-30 15:06:30  设置主库实例 EP01 为 OK 状态
show
------------------------------MPP 系统信息汇总------------------------------
EP 站点数:  2
系统状态:  正常
切换模式:  NON AUTO FAILOVER

站点号 实例名  实例模式  实例状态     FLSN    CLSN    SLSN    SSLSN
```

```
0     EP01    PRIMARY   OK       22704    22704    22704    22704
      EP11    STANDBY   OK       22704    22704    22704    22704

1     EP02    PRIMARY   OK       21198    21198    21198    21198
      EP22    STANDBY   OK       21198    21198    21198    21198
-------------------------------------------------------------------------
```

2. 备库故障恢复

备库故障，将 EP22 退出后，mpp_monitor 自动检测并查看信息显示如下：

```
2013-09-30 15:38:15  备库实例 EP22 故障
show
------------------------------MPP 系统信息汇总------------------------------
EP 站点数:  2
系统状态:  正常
切换模式:  NON AUTO FAILOVER

站点号 实例名  实例模式  实例状态     FLSN    CLSN    SLSN    SSLSN
0     EP01    PRIMARY   OK       22704    22704     22704    22704
      EP11    STANDBY   OK       22704    22704     22704    22704

1     EP02    PRIMARY   OK       21198    21198     21198    21198
      EP22    STANDBY   ERROR    21198   21198     21198    21198
-------------------------------------------------------------------------
```

重新启动备库，备库可以自动恢复到系统中：

```
2013-09-30 15:47:42  设置主库实例 EP02 为 RECOVERY 状态
2013-09-30 15:47:44  主库 EP02 的备库实例 EP22 恢复成功
2013-09-30 15:47:44  设置主库实例 EP02 为 OK 状态
show
------------------------------MPP 系统信息汇总------------------------------
EP 站点数:  2
系统状态:  正常

站点号 实例名  实例模式  实例状态     FLSN    CLSN    SLSN    SSLSN
0     EP01    PRIMARY   OK       22704    22704    22704    22704
      EP11    STANDBY   OK       22704   22704   22704   22704

1     EP02    PRIMARY   OK       21198   21198   21198   21198
      EP22    STANDBY   OK       21198   21198   21198   21198
-------------------------------------------------------------------------
```

查看 monitor 目录下的日志文件：

```
2013-09-30 15:38:15  备库实例 EP22 故障
2013-09-30 15:47:42  试图为主库实例 EP02 添加备库实例 EP22
2013-09-30 15:47:42  设置备库实例 EP22 的参数 mpp_ini 为 0 成功
2013-09-30 15:47:42  MPP 监视器设置备库实例 EP22 为 OPEN 状态成功
2013-09-30 15:47:42  实例 EP22 信息：系统状态(OPEN), 系统模式(STANDBY),
slsn(21198)
2013-09-30 15:47:42  设置主库实例 EP02 为 RECOVERY 状态
2013-09-30 15:47:43  EP02 进入 SUSPEND 状态
```

```
2013-09-30 15:47:44  主站点 EP02 与备库实例 EP22 完成同步
2013-09-30 15:47:44  实例 EP02 OPEN 成功
2013-09-30 15:47:44  主库 EP02 的备库实例 EP22 恢复成功
2013-09-30 15:47:44  设置主库实例 EP02 为 OK 状态
```

3. 手动主备切换

使用场景：MPP 系统由故障状态恢复正常后，原来的备库变为 MPP 站点主库，原来的 MPP 站点主库变为备库。若原 MPP 站点主库性能比原备库性能要高出很多，那么就会将原主库恢复为 MPP 站点主库的需求。使用手动主备切换，可以完成此项功能。

手动切换前系统状态如下：

```
show
------------------------------MPP 系统信息汇总------------------------------
EP 站点数:  2
系统状态:  正常
切换模式:  NON AUTO FAILOVER

站点号 实例名  实例模式  实例状态     FLSN    CLSN    SLSN    SSLSN
0      EP11    PRIMARY   OK       24210   24210    24210   24210
       EP01    STANDBY   OK       24210   24210    24210   24210

1      EP02    PRIMARY   OK       21198   21198    21198   21198
       EP22    STANDBY   OK       21198   21198    21198   21198
----------------------------------------------------------------------------
```

输入 switchover EP01，将 EP01 切换为主库：

```
switchover ep01
2013-09-30 16:05:31  MPP 监视器设置主库实例 EP11 为 MOUNT 状态成功
2013-09-30 16:05:31  MPP 监视器设置主库实例 EP11 为 STANDBY 模式成功
2013-09-30 16:05:31  设置站点 EP11 的 mpp_ini 为 0 参数成功
2013-09-30 16:05:31  实例 EP01 apply keep buf 成功
2013-09-30 16:05:31  MPP 监视器设置备库实例 EP01 为 MOUNT 状态成功
2013-09-30 16:05:31  MPP 监视器设置备库实例 EP01 为 PRIMARY 模式成功
2013-09-30 16:05:31  设置备库实例 EP01 的参数 mpp_ini 为 1 成功
2013-09-30 16:05:31  MPP 主节点 EP11 切换至备用节点 EP01
2013-09-30 16:05:31  更新归档成功
2013-09-30 16:05:31  MPP 系统配置信息广播至控制文件成功
2013-09-30 16:05:31  MPP 监视器设置备库实例 EP11 为 OPEN 状态成功
2013-09-30 16:05:32  MPP 监视器设置主库实例 EP01 为 OPEN 状态成功
2013-09-30 16:05:32  手动切换成功
```

输入 show 命令，再次查看当前 MPP 配置情况：

```
show
------------------------------MPP 系统信息汇总------------------------------
EP 站点数:  2
系统状态:  正常
切换模式:  NON AUTO FAILOVER

站点号 实例名  实例模式  实例状态    FLSN    CLSN    SLSN    SSLSN
0      EP01    PRIMARY   OK       25716    25716    25716    25716
       EP11    STANDBY   OK       25716    25716    25716    25716
```

```
1     EP02    PRIMARY   OK       21198    21198    21198    21198
      EP22    STANDBY   OK       21198    21198    21198    21198
--------------------------------------------------------------------------
```

switchover 能成功执行的条件：

1．主库处于 OK 状态

2．备库处于正常备库

3．MPP 系统处于正常状态

手动切换过程中，若遇故障，则处理办法如下：

a) 主库正常，备库正常，则备库成功成为主库，主库成为备库，MPP 系统进入正常状态

b) 主库正常，备库故障，或者两者都故障，则不进行切换，MPP 系统恢复正常状态，故障备库等待系统故障恢复

c) 主库故障，备库正常，则备库成为主库，MPP 系统进入正常状态，故障主库等待系统故障恢复

4．备库接管主库

备库接管的条件：

a） 主库故障

b） 存在正常的备库

c） MPP 系统状态 ERROR

场景如下：主库实例 EP01 故障，备库实例 EP11 正常，可以使用命令 takeover 手动进行接管，将备库变成主库。接管前的状态为：

```
show
------------------------------MPP 系统信息汇总------------------------------
EP 站点数:  2
系统状态:  存在 EP 故障
切换模式:  NON AUTO FAILOVER

站点号 实例名  实例模式   实例状态    FLSN    CLSN    SLSN    SSLSN
0     EP01    PRIMARY   ERROR    25716    25716    25716    25716
      EP11    STANDBY   OK       25716    25716    25716    25716

1     EP02    PRIMARY   OK       21198    21198    21198    21198
      EP22    STANDBY   OK       21198    21198    21198    21198


--------------------------------------------------------------------------
```

接管后：

```
takeover ep01
2013-10-11 10:33:47  主库实例 EP01 出现故障, 系统试图进行主备切换
2013-10-11 10:33:47  实例 EP11 apply keep buf 成功
2013-10-11 10:33:47  选择 EP11 作为新主库
2013-10-11 10:33:47  EP11 进入 MOUNT 状态
2013-10-11 10:33:47  EP11 变为主库模式
2013-10-11 10:33:59  设置主库实例 EP11 为 OK 状态
2013-10-11 10:33:59  设置站点 EP11 的 mpp_ini 为 1 参数成功!
2013-10-11 10:33:59  实例 EP11 OPEN 成功
2013-10-11 10:33:59  MPP 系统主备切换完毕
```

```
2013-10-11 10:34:00  接管实例 EP01 成功
show
------------------------------MPP 系统信息汇总-------------------------------
EP 站点数:  2
系统状态:  正常
切换模式:  NON AUTO FAILOVER

站点号 实例名 实例模式   实例状态 FLSN       CLSN       SLSN       SSLSN
0     EP11   PRIMARY  OK     27222       27222       27222       27222
      EP01   STANDBY  ERROR  GET FAILED  GET FAILED  GET FAILED GET FAILED

1     EP02   PRIMARY  OK     21198   21198   21198   21198
      EP22   STANDBY  OK     21198   21198   21198   21198
------------------------------------------------------------------------------
```

实例 EP01 故障，无法获取 LSN 信息，因此显示均为 GET FAILED。

注：TAKEOVER 命令要慎用!!!! 需要确保备库在主库故障前是正常状态，即备库接收了主库故障前的所有日志。如果备库是主库故障后才启动的，或者还未恢复完成，可能导致主备数据不一致的风险。

5. MPP 系统故障恢复

无论是基于主备的 MPP 系统，还是普通配置的 MPP，若事务采用两阶段方式提交，即配置文件 dmmpp.ini 设置了 mpp_2pc_flag = 1，当 MPP 系统在运行过程中，出现网络故障或节点故障时，都需要借助 monitor 完成 MPP 系统的故障恢复。

对于主备配置的 MPP，当系统出现故障时，无需特殊处理，monitor 会自动完成故障修复；对于普通配置的 MPP，需要用户手动以 MODE=1 的方式启动 monitor 完成故障修复，启动参数参考表 30.7 MPP_MONITOR 工具参数说明。

6. MPP 系统退出

对于普通配置的 MPP，用户正常退出各个 EP 站点即可，EP 站点之间的关闭顺序没有先后之分；对于基于主备的配置，用户需首先退出监控器，然后再退出各个主库实例，最后退出备库实例。

29.7 并行查询计划

在 MPP 模式下，查询是通过并行查询计划来进行。所谓的 MPP 并行查询计划，是在单节点的查询计划中合适的位置插入了通讯操作符，负责主从 EP 相互之间的数据移动和同步。目前包含 4 个通讯操作符，详细信息见下表：

表 29.8 通讯操作符

名称	功能
MPP GATHER	主 EP 收集所有数据，从 EP 将数据发送到主 EP
MPP COLLECT	用于替换顶层 MPP GATHER，除了收集数据到主节点，还增加主从节点间的同步执行功能，防止从节点不断发送数据到主节点造成邮件堆积
MPP DISTRIBUTE	各 EP 间相互分发数据
MPP BRODCAST	类似 MPP GATHER，收集数据，还包含聚集函数运算功能
MPP SCATTER	主 EP 发散数据到所有从 EP，一般和 MPP GATHER 配合使用

29.8 MPP 常见用法

MPP 常见用法汇总如下：

1.获取当前会话连接的 EP 的 MPP 序号

```
select sf_get_self_ep_seqno();
```

2.根据 rowid 获取本行数据来自哪个 EP

```
select sf_get_ep_seqno(rowid);
```

3.获取 EP 节点配置信息

```
select * from v$mpp_cfg_item where sf_get_ep_seqno(rowid) = sf_get_self_ep_seqno();
```

4.获取当前会话连接的实例名

```
select name from v$instance where sf_get_ep_seqno(rowid) = sf_get_self_ep_seqno();
```

5.获取 MPP 系统内所有 EP 的所有会话

```
select * from v$sessions;
```

6.获取当前连接的实例上的所有会话

```
select * from v$sessions where sf_get_ep_seqno(rowid) = sf_get_self_ep_seqno();
```

7.获取实例 EP01 上的所有会话

```
select * from v$sessions where sf_get_ep_seqno(rowid) = (select distinct ep_seqno from v$mpp_cfg_item where inst_name ='EP01');
```

8.获取连接实例上的表 test 使用空间

```
select table_used_pages('SYSDBA','TEST');
```

9.获取每个实例上的表 test 使用空间

```
select table_used_pages('SYSDBA','TEST'), name from v$instance;
```

10.获取所有实例上的表 test 总共使用的空间

```
select sum(total_size) from (select table_used_pages('SYSDBA','TEST') total_size, name from v$instance);
```

第六部分 附录

附录 1 数据字典

1. SYSOBJECTS

记录系统中所有对象的信息。

序号	列	数据类型	说明
1	NAME	VARCHAR(128)	对象名称
2	ID	INTEGER	对象 ID
3	SCHID	INTEGER	TYPE$=SCHOBJ 或者 TYPE$=TABOBJ 时表示对象所属的模式 ID，否则为 0。
4	TYPE$	VARCHAR(10)	对象的主类型。 1）库级:UR，SCH，POLICY,GDBLINK，GSYNOM， DSYNOM，DIR，OPV，SPV，RULE，DMNOBJ； 2）模式级：SCHOBJ； 3）表级：TABOBJ
5	SUBTYPE$	VARCHAR(10)	对象的子类型。分为三种： 1）用户对象：USER， ROLE； 2）模式对象：UTAB，STAB， VIEW，PROC， SEQ， PKG，TRIG， DBLINK，SYNOM， CLASS，TYPE，JCLASS，DOMAIN，CHARSET，CLLT，CONTEXT； 3）表对象：INDEX，CNTIND，CONS
6	PID	INTEGER	对象的父对象 ID，为-1 表示当前行 PID 列无意义
7	VERSION	INTEGER	对象的版本
8	CRTDATE	DATETIME	对象的创建时间
9	INFO1	INTEGER	表对象： 表数据所在的缓冲区 ID(0xFF000000)，数据页填充因子(0x00F00000)，BRANCH(0x000FF000) ，NOBARNCH(0x00000FF0) ，BRANCHTYPE(0x0000000F) 用户对象：BYTE(4)用户类型 视图对象：BIT(0) CHECK, BIT(1) CHECK CASCADE, BIT(2) 是否加密,BIT(4)SYSTEM 触发器对象： BIT(1) TV|EVENT FLAG , BIT(2,3) 执行类型(前或后), BIT(4)是否加密, BIT(5)是否系统级

			对于 TV 触发器：BIT(6) RSFLAG, BIT(7) NEW REFED FLAG, BIT(8) OLD REFED FLAG, BIT(9) ALL NEW MDF FLAG 对于事件触发器：BIT(6,7) SCOPE, BIT(8,11) SCHEDUAL TYPE 约束对象：列数 存储过程：BIT(0)是否存储过程，BIT(1)是否加密，BIT(2)是否系统级 角色：角色类型 序列：BYTE(1)是否循环，BYTE(2)是否排序，BYTE(3)是否有缓存 同义词：是否带系统标识 包：BIT(1)文本是否加密，BIT(2)是否带系统标识
10	INFO2	INTEGER	表/用户/数据库/表空间：BYTE(4)空间限制值 视图：基表 ID
11	INFO3	BIGINT	序列：起始值 触发器：BYTE(0-3)EVENTS TV 触发器，BYTE(4)更新操作可触发的字段， BYTE(5)行前触发器中可被触发器修改值的新行字段，BYTE(6)元组级触发器中引用的字段， 事件触发器，BYTE(4)间隔，BYTE(5)子间隔，BYTE(6,7)分间隔 表：BYTE(0)表类型或临时表类型，BYTE(1)日志类型或错误响应或不可用标识，BYTE(2)是否临时表会话级，BYTE(3-4)区大小，BYTE(5)标记分布表 用户：BYTE(2)默认表空间 ID
12	INFO4	BIGINT	序列：增量
13	INFO5	VARBINARY(128)	表：BYTE(10)BLOB 数据段头 序列：BYTE(8)序列最大值，BYTE(8)序列最小值，BYTE(2)文件 ID，BYTE(4)页号，BYTE(2)序列当前位置
14	INFO6	VARBINARY(2048)	视图：BYTE(4)表或视图 ID 触发器： TV 触发器，BYTE(2)更新操作可触发字段， BYTE(2)元组级触发器前可能被触发器修改值的字段，BYTE(2)元组级触发器中引用的字段， 事件触发器，BYTE(8)开始/ BYTE(8)结束日期、BYTE(5)开始/ BYTE(5)结束时间 约束对象：(BYTE(4)ID)表列链表

			同义词：BYTE(2)模式名和 BYTE(2)对象名 表： IDENTITY(BYTE(8) FOR SEED，BYTE(8) FOR INCREMENT)或 BYTE(4)列 ID
15	INFO7	BIGINT	保留
16	INFO8	VARBINARY(1024)	表： 外部表的控制文件路径 或者 BYTE(2)水平分区表记录总的子表数目，BYTE(4)垂直分区记录子表 ID
17	VALID	CHAR(1)	对象是否有效

2. SYSINDEXES

记录系统中所有索引定义信息。

序号	列	数据类型	说明
1	ID	INTEGER	索引 ID
2	ISUNIQUE	CHAR(1)	是否为唯一索引
3	GROUPID	SMALLINT	所在表空间的 ID
4	ROOTFILE	SMALLINT	存放根的文件号
5	ROOTPAGE	INTEGER	存放根的页号
6	TYPE$	CHAR(2)	类型
7	XTYPE	INTEGER	索引标记，联合其他字段标记索引类型
8	FLAG	INTEGER	标记是否是系统索引，虚索引
9	KEYNUM	SMALLINT	索引包含的键值数目
10	KEYINFO	VARBINARY(816)	索引的键值信息
11	INIT_EXTENTS	SMALLINT	初始簇数目
12	BATCH_ALLOC	SMALLINT	下次分配簇数目
13	MIN_EXTENTS	SMALLINT	最小簇数

3. SYSCOLUMNS

记录系统中所有列定义的信息。

序号	列	数据类型	说明
1	NAME	VARCHAR(128)	列名
2	ID	INTEGER	父对象 ID
3	COLID	SMALLINT	列 ID
4	TYPE$	VARCHAR(128)	列数据类型
5	LENGTH$	INTEGER	列定义长度
6	SCALE	SMALLINT	列定义刻度
7	NULLABLE$	CHAR(1)	是否允许为空
8	DEFVAL	VARCHAR(2048)	缺省值
9	INFO1	SMALLINT	水平分区表：分区列的序号 其他表：BIT(0)压缩标记 列存储表： BIT(0)压缩标记；BIT(1-12)区大小； BIT(13)列存储的区上是否做最大最小值统计；BIT(14)是否加密列 视图：BYTE(2)多层视图中的最原始表的列 ID

			存储过程：BYTE(2)参数类型
10	INFO2	SMALLINT	非垂直分区表：BIT(0)是否自增列； BIT(14)是否加密列 垂直分区表：BIT(0~9)列存储选项(列存储的表空间)； BIT(14)是否加密列 视图：BYTE(2)多层视图中直接上层或 BYTE(2)垂直分区对应基表的列 ID 列存储表：group_id

4. SYSCONS

记录系统中所有约束的信息。

序号	列	数据类型	说明
1	ID	INTEGER	约束 ID
2	TABLEID	INTEGER	所属表 ID
3	COLID	SMALLINT	列 ID。暂时不支持，无意义。全部为-1
4	TYPE$	CHAR(1)	约束类型
5	VALID	CHAR(1)	约束是否有效
6	INDEXID	INTEGER	索引 ID
7	CHECKINFO	VARCHAR(2048)	check 约束的文本
8	FINDEXID	INTEGER	外键所引用的索引 ID
9	FACTION	CHAR(2)	前一字符对应外键的更新动作，后一字符对应外键的删除动作
10	TRIGID	INTEGER	动作触发器 ID

5. SYSSTATS

记录系统中的统计信息。

序号	列	数据类型	说明
1	ID	INTEGER	统计信息 ID
2	COLID	SMALLINT	列 ID，表级统计为-1
3	T_FLAG	CHAR(1)	对象的标记
4	T_TOTAL	BIGINT	总行数
5	N_SMAPLE	BIGINT	采样个数
6	N_DISTINCT	BIGINT	不同值的个数
7	N_NULL	BIGINT	空值个数
8	V_MIN	VARBINARY(255)	列的最小值
9	V_MAX	VARBINARY(255)	列的最大值
10	BLEVEL	TINYINT	B 树层次
11	N_LEAF_PAGES	BIGINT	叶子段总页数
12	N_LEAF_USED_PAGES	BIGINT	叶子占用的页数
13	CLUSTER_FACTOR	INTEGER	索引的 cluster_factor
14	N_BUCKETS	SMALLINT	直方图桶数目
15	DATA	BLOB	直方图数据

6. SYSDUAL

为不带表名的查询而设，用户一般不需查看。

序号	列	数据类型	说明
1	ID	INTEGER	始终为 1

7. SYSTEXTS

存放字典对象的文本信息。

序号	列	数据类型	说明
1	ID	INTEGER	所属对象的 ID
2	SEQNO	INTEGER	视图对象文本的信息的含义，0 表示视图定义，1 表示视图的查询子句
3	TXT	TEXT	文本信息

8. SYSGRANTS

记录系统中权限信息。

序号	列	数据类型	说明
1	URID	INTEGER	被授权用户/角色 ID
2	OBJID	INTEGER	授权对象 ID，对于数据库权限为-1
3	COLID	INTEGER	表/视图列 ID，非列权限为-1
4	PRIVID	INTEGER	权限 ID
5	GRANTOR	INTEGER	授权者 ID
6	GRANTABLE	CHAR(1)	权限是否可转授，Y 可转授，N 不可转授

9. SYSAUDIT

记录系统中的审计设置。

序号	列	数据类型	说明
1	LEVEL	SMALLINT	审计级别
2	UID	INTEGER	用户 ID
3	TVPID	INTEGER	表/视图/触发器/存储过程函数 ID
4	COLID	SMALLINT	列 ID
5	TYPE	SMALLINT	审计类型
6	WHENEVER	SMALLINT	审计情况

10. SYSAUDITRULES

记录系统中审计规则的信息。

序号	列	数据类型	说明
1	ID	INTEGER	规则 ID
2	RULENAME	VARCHAR(128)	规则名
3	USERID	INTEGER	用户 ID
4	SCHID	INTEGER	模式 ID
5	OBJID	INTEGER	操作对象 ID
6	COLID	SMALLINT	列 ID
7	OPTYPE	SMALLINT	操作类型
8	WHENEVER$	SMALLINT	审计情况
9	ALLOW_IP	VARCHAR(1024)	允许的 IP
10	ALLOW_DT	VARCHAR(1024)	时间段
11	INTERVAL$	INTEGER	时间间隔
12	TIMES	INTEGER	当前操作时间

11. SYSHPARTTABLEINFO

记录系统中分区表的信息。

序号	列	数据类型	说明
1	BASE_TABLE_ID	INTEGER	基表 ID
2	PART_TABLE_ID	INTEGER	分区表 ID

3	PARTITION_TYPE	VARCHAR(10)	分区类型
4	PARTITION_NAME	VARCHAR(128)	分区名
5	HIGH_VALUE	VARBINARY(8188)	LIST 分区的临界值； 范围分区的分区值； 哈希分区此值为 NULL
6	INCLUDE_HIGH_VALUE	CHAR(1)	对 LIST 分区表示分区是否包含临界值； 对范围分区始终为 1；对哈希分区此值为 0
7	RESVD1	INTEGER	保留
8	RESVD2	INTEGER	保留
9	RESVD3	INTEGER	保留
10	RESVD4	VARCHAR(128)	保留
11	RESVD5	VARCHAR(2000)	保留

12. SYSMACPLYS

记录策略定义。

序号	列	数据类型	说明
1	ID	INTEGER	策略 ID
2	NAME	VARCHAR(128)	策略名

13. SYSMACLVLS

记录策略的等级。

序号	列	数据类型	说明
1	PID	INTEGER	策略 ID
2	ID	SMALLINT	等级 ID
3	NAME	VARCHAR(128)	等级名

14. SYSMACCOMPS

记录策略的范围。

序号	列	数据类型	说明
1	PID	INTEGER	策略 ID
2	ID	SMALLINT	范围 ID
3	NAME	VARCHAR(128)	范围名

15. SYSMACGRPS

记录策略所在组的信息。

序号	列	数据类型	说明
1	PID	INTEGER	策略 ID
2	ID	SMALLINT	组 ID
3	PARENTID	SMALLINT	父节点 ID
4	NAME	VARCHAR(128)	组名

16. SYSMACLABELS

记录策略的标记信息。

序号	列	数据类型	说明
1	PID	INTEGER	策略 ID
2	ID	INTEGER	标记 ID
3	LABEL	VARCHAR(4000)	标记信息

17. SYSMACTABPLY

记录表策略信息。

序号	列	数据类型	说明
1	TID	INTEGER	表 ID
2	PID	INTEGER	策略 ID
3	COLID	SMALLINT	列 ID
4	OPTIONS	BYTE	可见性

18. SYSMACUSRPLY

记录用户的策略信息。

序号	列	数据类型	说明
1	UID	INTEGER	用户 ID
2	PID	INTEGER	策略 ID
3	MAXREAD	INTEGER	最大读标记 ID
4	MINWRITE	INTEGER	最小写标记 ID
5	DEFTAG	INTEGER	默认标记 ID
6	ROWTAG	INTEGER	行级标记 ID
7	PRIVS	BYTE	特权

19. SYSMACOBJ

记录扩展客体标记信息。

序号	列	数据类型	说明
1	OBJID	INTEGER	对象 ID
2	COLID	SMALLINT	列 ID
3	PID	INTEGER	策略 ID
4	TAG	INTEGER	标记 ID

20. SYSCOLCYT

记录列的加密信息。

序号	列	数据类型	说明
1	TID	INTEGER	表 ID
2	CID	SMALLINT	列 ID
3	ENC_ID	INTEGER	加密类型 ID
4	ENC_TYPE	CHAR(1)	加密类型
5	HASH_ID	INTEGER	哈希算法 ID
6	HASH_TYPE	CHAR(1)	是否加盐
7	CIPHER	VARCHAR(1024)	密钥

21. SYSACCHISTORIES

记录登录失败的历史信息。

序号	列	数据类型	说明
1	LOGINID	INTEGER	登录 ID
2	LOGINNAME	VARCHAR(128)	登录名
3	TYPE$	INTEGER	登录类型
4	ACCPIP	VARCHAR(128)	访问 IP
5	ACCDT	DATETIME	访问时间

22. SYSPWDCHGS

记录密码的修改信息。

序号	列	数据类型	说明

1	LOGINID	INTEGER	登录 ID
2	OLD_PWD	VARCHAR(48)	旧密码
3	NEW_PWD	VARCHAR(48)	新密码
4	MODIFIED_TIME	TIMESTAMP	修改日期

23. SYSCONTEXTLIBS

记录词库的信息。

序号	列	数据类型	说明
1	LIB_ID	INTEGER	词库 ID
2	LIB_NAME	VARCHAR(32)	词库名
3	LIB_PATH	VARCHAR(256)	词库路径
4	DEF_LIB	CHAR(1)	缺省词库标记
5	RESV1	VARCHAR(10)	保留
6	RESV2	INTEGER	保留
7	RESV3	INTEGER	保留
8	RESV4	INTEGER	保留

24. SYSCONTEXTINDEXES

记录全文索引的信息。

序号	列	数据类型	说明
1	NAME	VARCHAR(128)	索引名
2	ID	INTEGER	索引号
3	TABLEID	INTEGER	基表号
4	COLID	SMALLINT	列号
5	UPD_TIMESTAMP	TIMESTAMP	索引更新时间
6	TIID	INTEGER	CTI$INDEX_NAME$I 表 ID
7	TRID	INTEGER	CTI$INDEX_NAME$R 表 ID
8	TPID	INTEGER	CTI$INDEX_NAME$P 表 ID
9	WSEG_TYPE	SMALLINT	分词参数类型
10	RESVD1	SMALLINT	保留
11	RESVD2	INTEGER	保留
12	RESVD3	INTEGER	保留
13	RESVD4	VARCHAR(1000)	保留

25. SYSTABLECOMMENTS

记录表或视图的注释信息。

序号	列	数据类型	说明
1	SCHNAME	VARCHAR(128)	模式名
2	TVNAME	VARCHAR(128)	表/视图名
3	TABLE_TYPE	VARCHAR(10)	对象类型
4	COMMENT$	VARCHAR(1024)	注释信息

26. SYSCOLUMNCOMMENTS

记录列的注释信息。

序号	列	数据类型	说明
1	SCHNAME	VARCHAR(128)	模式名
2	TVNAME	VARCHAR(128)	表/视图名
3	COLNAME	VARCHAR(128)	列名

4	TABLE_TYPE	VARCHAR(10)	对象类型
5	COMMENT$	VARCHAR(1024)	注释信息

27. SYSUSERS

记录系统中用户信息。

序号	列	数据类型	说明
1	ID	INTEGER	用户 ID
2	PASSWORD	VARCHAR(128)	用户口令
3	AUTHENT_TYPE	INTEGER	用户认证方式：NDCT_DB_AUTHENT/NDCT_OS_AUTHENT/NDCT_NET_AUTHENT/NDCT_UNKOWN_AUTHENT
4	SESS_PER_USER	INTEGER	在一个实例中，一个用户可以同时拥有的会话数量
5	CONN_IDLE_TIME	INTEGER	用户会话的最大空闲时间
6	FAILED_NUM	INTEGER	用户登录失败次数限制
7	LIFE_TIME	INTEGER	一个口令在终止使用前可以使用的天数
8	REUSE_TIME	INTEGER	一个口令在可以重新使用之前必须经过的天数
9	REUSE_MAX	INTEGER	一个口令在可以重新使用前必须改变的次数
10	LOCK_TIME	INTEGER	用户口令锁定时间
11	GRACE_TIME	INTEGER	用户口令过期后的宽限时间
12	LOCKED_STATUS	SMALLINT	用户登录是否锁定：LOGIN_STATE_UNLOCKED/LOGIN_STATE_LOCKED
13	LATEST_LOCKED	TIMESTAMP(19)	用户最后一次的锁定时间
14	PWD_POLICY	INTEGER	用户口令策略：NDCT_PWD_POLICY_NULL/NDCT_PWD_POLICY_1/NDCT_PWD_POLICY_2/NDCT_PWD_POLICY_3/NDCT_PWD_POLICY_4/NDCT_PWD_POLICY_5
15	RN_FLAG	INTEGER	是否只读
16	ALLOW_ADDR	VARCHAR(500)	允许的 IP 地址
17	NOT_ALLOW_ADDR	VARCHAR(500)	不允许的 IP 地址
18	ALLOW_DT	VARCHAR(500)	允许登录的时间段
19	NOT_ALLOW_DT	VARCHAR(500)	不允许登录的时间段
20	LAST_LOGIN_DTID	VARCHAR(128)	上次登录时间
21	LAST_LOGIN_IP	VARCHAR(128)	上次登录 IP 地址
22	FAILED_ATTEMPS	INTEGER	将引起一个账户被锁定的连续注册失败的次数
23	ENCRYPT_KEY	VARCHAR(256)	用户登录的存储加密密钥

28. SYSOBJINFOS

记录对象的依赖信息。

序号	列	数据类型	说明
1	ID	INTEGER	被依赖类的 ID

2	TYPE$	VARCHAR(100)	对象依赖类型
3	INT_VALUE	INTEGER	对象类型对应的值
4	STR_VALUE	VARCHAR(2048)	如果是域对象，表示 DOMAIN+域 ID。其他对象暂未利用
5	BIN_VALUE	VARBINARY(2048)	暂未利用

29. SYSRESOURCES

记录用户使用系统资源的限制信息。

序号	列	数据类型	说明
1	ID	INTEGER	用户 ID
2	CPU_PER_CALL	INTEGER	用户的一个请求能够使用的 CPU 时间上限（单位：秒）
3	CPU_PER_SESSION	INTEGER	一个会话允许使用的 CPU 时间上限（单位：秒）
4	MEM_SPACE	INTEGER	会话占有的私有内存空间上限（单位：MB）
5	READ_PER_CALL	INTEGER	每个请求能够读取的数据页数
6	READ_PER_SESSION	INTEGER	一个会话能够读取的总数据页数上限
7	INFO1	VARCHAR(256)	一个会话连接、访问和操作数据库服务器的时间上限（单位：10 分钟）

30. SYSCOLINFOS

记录列的附加信息，例如是否虚拟列。

序号	列	数据类型	说明
1	ID	INTEGER	表 ID
2	COLID	SMALLINT	列 ID
3	INFO1	INTEGER	第一位表示是否虚拟列
4	INFO2	INTEGER	备用
5	INFO3	INTEGER	备用

附录 2 动态性能视图

1. V$AUDITRECORDS

显示审计记录，用来查询当前系统默认路径下的审计文件信息。此动态性能视图只有在审计开关打开时才有内容，且只有审计用户可以查询。

序号	列	数据类型	说明
1	USERID	INTEGER	用户 ID
2	USERNAME	VARCHAR(128)	用户名
3	ROLEID	INTEGER	角色 ID。 没有具体角色的用户和 SQL 序列审计，没用角色信息。
4	ROLENAME	VARCHAR(128)	角色名。 没有具体角色的用户和 SQL 序列审计，没用角色信息。
5	IP	VARCHAR(25)	IP 地址
6	SCHID	INTEGER	模式 ID
7	SCHNAME	VARCHAR(128)	模式名
8	OBJID	INTEGER	对象 ID
9	OBJNAME	VARCHAR(128)	对象名
10	OPERATION	VARCHAR(128)	操作类型名
11	SUCC_FLAG	CHAR(1)	成功标记
12	SQL_TEXT	VARCHAR(8188)	SQL 文本
13	DESCRIPTION	VARCHAR(8188)	描述信息
14	OPTIME	DATETIME	操作时间

2. V$BUFFER

缓冲池动态性能表，用来记录除循环使用缓冲池外的缓冲池控制页结构的信息。

序号	列	数据类型	说明
1	POOL_ID	INTEGER	缓冲池 ID
2	SPACE_ID	INTEGER	表空间 ID
3	FILE_ID	INTEGER	文件 ID
4	PAGE_NO	INTEGER	数据在文件中的页号
5	RWLOCK	BIGINT	线程锁地址
6	PAGE	BIGINT	数据页对应的内存块地址
7	HASH	BIGINT	HASH 地址
8	LSN	BIGINT	页的当前 LSN
9	INDEXID	INTEGER	页内索引 ID
10	PAGETYPE	INTEGER	页类型
11	UPD_NEXT	BIGINT	下一个脏页地址
12	UPD_PREV	BIGINT	上一个脏页地址
13	N_FIXED	INTEGER	数据页被引用的次数
14	STATUS	INTEGER	页状态：空闲，使用，正在读，正在写
15	ACCESS_CNT	INTEGER	正在访问的页号
16	FIRST_LSN	BIGINT	第一次被修改时对应的日志文件号
17	FIRST_FIL	INTEGER	第一次被修改时对应的已经刷盘的最新文件号
18	FIRST_OFF	BIGINT	第一次被修改时对应的已经刷盘的最新文件偏移

3. V$BUFFER_RECYCLE

缓冲池动态性能表，用来记录循环使用缓冲池控制页结构的信息。

序号	列	数据类型	说明
1	POOL_ID	INTEGER	缓冲池 ID
2	SPACE_ID	INTEGER	表空间 ID
3	FILE_ID	INTEGER	文件 ID
4	PAGE_NO	INTEGER	数据在文件中的页号
5	RWLOCK	BIGINT	线程锁地址
6	PAGE	BIGINT	数据页对应的内存块地址
7	HASH	BIGINT	HASH 地址
8	LSN	BIGINT	页的当前 LSN
9	INDEXID	INTEGER	页内索引 ID
10	PAGETYPE	INTEGER	页类型
11	UPD_NEXT	BIGINT	下一个脏页地址
12	UPD_PREV	BIGINT	上一个脏页地址
13	N_FIXED	INTEGER	数据页被引用的次数
14	STATUS	INTEGER	页状态：空闲，使用，正在读，正在写
15	ACCESS_CNT	INTEGER	正在访问的页号
16	FIRST_LSN	BIGINT	第一次被修改时对应的日志文件号
17	FIRST_FIL	INTEGER	第一次被修改时对应的已经刷盘的最新文件号
18	FIRST_OFF	BIGINT	第一次被修改时对应的已经刷盘的最新文件偏移

4. V$BUFFERPOOL

页面缓冲池动态性能表，用来记录页面缓冲池结构的信息。

序号	列	数据类型	说明
1	ID	INTEGER	缓冲池 ID
2	NAME	VARCHAR(20)	缓冲池名称 NORMAL/KEEP/RECYCLE/FAST
3	PAGE_SIZE	INTEGER	基缓冲池页大小，不包括扩展池页
4	N_PAGES	INTEGER	页数
5	N_FIXED	INTEGER	数据页被引用的次数
6	FREE	INTEGER	空闲页数目
7	N_DIRTY	INTEGER	脏页数目
8	N_CLEAR	INTEGER	非空闲页数目
9	N_TOTAL_PAGES	INTEGER	页大小，包括扩展池页
10	N_MAX_PAGES	INTEGER	最多的页数
11	N_LOGIC_READS	INTEGER	READ 命中的次数
12	N_DISCARD	INTEGER	淘汰的页数
13	N_PHY_READS	INTEGER	READ 未命中的次数
14	N_PHY_M_READS	INTEGER	READ 为命中，批量读的次数
15	RAT_HIT	FLOAT	命中率
16	N_EXP_BUFFERPOOL	INTEGER	扩展缓冲池个数
17	N_UPD_REMOVE	INTEGER	从 update 链表删除页总数
18	N_PHY_WRITE	INTEGER	物理写入磁盘总数
19	N_UPD_PUT	INTEGER	RAC 远程读取数据后，加入 update 链表总数
20	N_UPD_SEARCH	INTEGER	RAC 远程读取数据后，查找 update 链表插入位置扫描总

			数

5. V$BUFFER_LRU_FIRST

显示所有缓冲池 LRU 链首页信息。

序号	列	数据类型	说明
1	POOL_ID	INTEGER	缓冲池 ID
2	SPACE_ID	INTEGER	表空间 ID
3	FILE_ID	INTEGER	文件 ID
4	PAGE_NO	INTEGER	数据在文件中的页号
5	RWLOCK	BIGINT	线程锁地址
6	PAGE	BIGINT	数据页对应的内存块地址
7	HASH	BIGINT	HASH 地址
8	LSN	BIGINT	页的当前 LSN
9	LRU_NEXT	BIGINT	下一个非空闲页地址
10	LRU_PREV	BIGINT	上一个非空闲页地址
11	UPD_NEXT	BIGINT	下一个脏页地址
12	UPD_PREV	BIGINT	上一个脏页地址
13	N_FIXED	INTEGER	数据页被引用的次数
14	STATUS	INTEGER	页状态：空闲，使用，正在读，正在写
15	ACCESS_CNT	INTEGER	正在访问的页号
16	FIRST_LSN	BIGINT	第一次被修改时对应的日志文件号
17	FIRST_FIL	INTEGER	第一次被修改时对应的已经刷盘的最新文件号
18	FIRST_OFF	BIGINT	第一次被修改时对应的已经刷盘的最新文件偏移

6. V$BUFFER_UPD_FIRST

显示所有缓冲池 UPDATE 链首页信息。

序号	列	数据类型	说明
1	POOL_ID	INTEGER	缓冲池 ID
2	SPACE_ID	INTEGER	表空间 ID
3	FILE_ID	INTEGER	文件 ID
4	PAGE_NO	INTEGER	数据在文件中的页号
5	RWLOCK	BIGINT	线程锁地址
6	PAGE	BIGINT	数据页对应的内存块地址
7	HASH	BIGINT	HASH 地址
8	LSN	BIGINT	页的当前 LSN
9	LRU_NEXT	BIGINT	下一个非空闲页地址
10	LRU_PREV	BIGINT	上一个非空闲页地址
11	UPD_NEXT	BIGINT	下一个脏页地址
12	UPD_PREV	BIGINT	上一个脏页地址
13	N_FIXED	INTEGER	数据页被引用的次数
14	STATUS	INTEGER	页状态：空闲，使用，正在读，正在写
15	ACCESS_CNT	INTEGER	正在访问的页号
16	FIRST_LSN	BIGINT	第一次被修改时对应的日志文件号
17	FIRST_FIL	INTEGER	第一次被修改时对应的已经刷盘的最新文件号
18	FIRST_OFF	BIGINT	第一次被修改时对应的已经刷盘的最新文件偏移

7. V$BUFFER_LRU_LAST

显示所有缓冲池 LRU 链末页信息。

序号	列	数据类型	说明
1	POOL_ID	INTEGER	缓冲池 ID
2	SPACE_ID	INTEGER	表空间 ID
3	FILE_ID	INTEGER	文件 ID
4	PAGE_NO	INTEGER	数据在文件中的页号
5	RWLOCK	BIGINT	线程锁地址
6	PAGE	BIGINT	数据页对应的内存块地址
7	HASH	BIGINT	HASH 地址
8	LSN	BIGINT	页的当前 LSN
9	LRU_NEXT	BIGINT	下一个非空闲页地址
10	LRU_PREV	BIGINT	上一个非空闲页地址
11	UPD_NEXT	BIGINT	下一个脏页地址
12	UPD_PREV	BIGINT	上一个脏页地址
13	N_FIXED	INTEGER	数据页被引用的次数
14	STATUS	INTEGER	页状态：空闲，使用，正在读，正在写
15	ACCESS_CNT	INTEGER	正在访问的页号
16	FIRST_LSN	BIGINT	第一次被修改时对应的日志文件号
17	FIRST_FIL	INTEGER	第一次被修改时对应的已经刷盘的最新文件号
18	FIRST_OFF	BIGINT	第一次被修改时对应的已经刷盘的最新文件偏移

8． V$BUFFER_UPD_LAST

显示所有缓冲池 UPDATE 链末页信息。

序号	列	数据类型	说明
1	POOL_ID	INTEGER	缓冲池 ID
2	SPACE_ID	INTEGER	表空间 ID
3	FILE_ID	INTEGER	文件 ID
4	PAGE_NO	INTEGER	数据在文件中的页号
5	RWLOCK	BIGINT	线程锁地址
6	PAGE	BIGINT	数据页对应的内存块地址
7	HASH	BIGINT	hash 地址
8	LSN	BIGINT	页的当前 LSN
9	LRU_NEXT	BIGINT	下一个非空闲页地址
10	LRU_PREV	BIGINT	上一个非空闲页地址
11	UPD_NEXT	BIGINT	下一个脏页地址
12	UPD_PREV	BIGINT	上一个脏页地址
13	N_FIXED	INTEGER	数据页被引用的次数
14	STATUS	INTEGER	页状态：空闲，使用，正在读，正在写
15	ACCESS_CNT	INTEGER	正在访问的页号
16	FIRST_LSN	BIGINT	第一次被修改时对应的日志文件号
17	FIRST_FIL	INTEGER	第一次被修改时对应的已经刷盘的最新文件号
18	FIRST_OFF	BIGINT	第一次被修改时对应的已经刷盘的最新文件偏移

9． V$CACHEITEM

显示缓冲区中缓冲项的相关信息。在 ini 参数 USE_PLN_POOL !=0 时才统计。

序号	列	数据类型	说明
1	ADDRESS	BIGINT	CACHE 项的地址
2	TYPE$	VARCHAR	CACHE 项的类型：SQL，计划（PLN），结果集（RS）

3	OVERFLOW	CHAR	是否溢出
4	IN_POOL	CHAR	是否在内存池中
5	DISABLED	CHAR	是否可用
6	N_FIXED	INTEGER	此缓冲项被引用的次数
7	TS_VALUE	INTEGER	时间戳
8	ITEM_SIZE	BIGINT	缓存节点大小（字节数）
9	N_HIT	INTEGER	节点命中次数

10. V$CACHEPLN

显示执行计划缓冲区中的相关信息。在 ini 参数 USE_PLN_POOL !=0 时才统计。并且动态视图信息不加入计划缓冲区。

序号	列	数据类型	说明
1	CACHE_ITEM	BIGINT	计划在缓冲区中的地址
2	TYPE$	VARCHAR	计划的类型：SQL 语句，存储过程，触发器
3	HASH_VALUE	INTEGER	HASH 值
4	SCHID	INTEGER	SCHEMA 的 ID 值
5	USER_ID	INTEGER	用户 ID
6	OBJ_ID	INTEGER	对象 ID（过程/触发器 ID）
7	RS_CAN_CACHED	CHAR	结果集是否可以缓存在缓冲区中
8	N_RS_CACHED	INTEGER	结果集在缓冲区的数目
9	N_TABLE	INTEGER	计划涉及的表的数目，仅在 INI 参数 RS_CAN_CACHE 为 1 时有效
10	TABLEID	VARCHAR(256)	计划涉及的表的 ID，用空格隔开
11	SQLSTR	VARCHAR(4096)	计划对应的 SQL 语句
12	SQLCACHE	BIGINT	对应 V$CACHESQL 中的 CACHE_ITEM
13	DDLSCRIPT	VARCHAR(4096)	DDL 执行脚本
14	RET_CMD	SMALLINT	返回响应命令字
15	STMT_TYPE	INTEGER	SQL 语句类型
16	N_LIT_PARA	INTEGER	常量参数数目
17	N_CLNT_PARA	INTEGER	用户定义的参数数目
18	N_COLS	INTEGER	要返回的列的数目
19	SEL_UPDATABLE	CHAR	计划是否可更新
20	MEM_SIZE	BIGINT	缓存节点大小（字节数）
21	RS_CAN_CACHED_IN_RULE	CHAR	依据基本规则是否可以缓存结果集('Y'或'N')，实际是否缓存还受结果集缓存相关参数影响，或者被手工强制设置不缓存

11. V$CACHERS

显示结果集缓冲区的相关信息。在 ini 参数 USE_PLN_POOL !=0 时才统计。

序号	列	数据类型	说明
1	CACHE_ITEM	BIGINT	结果集在缓冲区中的地址，对应 V$CACHEITEM 中的 ADDRESS
2	PLN	BIGINT	结果集对应查询计划,对应 V$CACHEPLN 里的 CACHE_ITEM
3	N_TABLES	INTEGER	结果集涉及的表的数目
4	TABLEID	VARCHAR(256)	结果集涉及的表的 ID，用空格隔开
5	MEM_SIZE	BIGINT	缓存节点大小（字节数）

6	EXEC_TIME	INTEGER	结果集生成所消耗的时间，单位：ms

12．V$CACHESQL

显示 SQL 缓冲区中 SQL 语句的信息。在 ini 参数 USE_PLN_POOL !=0 时才统计。

序号	列	数据类型	说明
1	CACHE_ITEM	BIGINT(19)	在内存中的位置，对应 V$CACHEITEM 中的 ADDRESS
2	SQL	VARCHAR(4096)	SQL 文本
3	LEN	INTEGER	SQL 文本长度
4	HASH_VALUE	INTEGER	HASH 值

13．V$CAPTURE

显示捕获信息。

序号	列	数据类型	说明
1	STATE	INTEGER	捕获状态
2	VERSION	SMALLINT	捕获文件的版本
3	MSG_NUM	BIGINT	捕获消息的条数
4	FILE_PATH	VARCHAR(256)	捕获文件路径
5	INIT_TIME	TIMESTAMP(0)	捕获开始时间
6	DURATION	INTEGER	捕获持续时间
7	FLUSH_BUF_NUM	INTEGER	待刷盘缓冲数
8	FREE_BUF_NUM	INTEGER	空闲缓冲数

14．V$CIPHERS

显示系统加密算法信息。

序号	列	数据类型	说明
1	CYT_ID	INTEGER	算法 ID
2	CYT_NAME	VARCHAR(256)	算法名
3	CYT_TYPE	INTEGER	算法类型 1： 分组对称加密算法 2：流式对称加密算法 3：非对称加密算法，保留 4：散列算法
4	BLOCK_SIZE	INTEGER	块大小
5	KH_SIZE	INTEGER	KEY 或 HASH 大小

15．V$CKPT_HISTORY

显示检查点历史信息。

序号	列	数据类型	说明
1	SEQNO	INTEGER	序列号
2	CKPT_CMD	VARCHAR(24)	检查点
3	THREAD_ID	BIGINT	线程 ID
4	START_TIME	DATETIME	开始时间
5	TIME_USED	BIGINT	从开始到结束使用时间
6	PAGE_FLUSHED	INTEGER	刷盘数
7	FIL	INTEGER	文件号
8	OFF	BIGINT	文件偏移量
9	CKPT_LSN	BIGINT	检查点 LSN

16．V$CMD_HISTORY

通过本视图可以观察系统的一些命令的历史信息。其中 cmd 指的是 SESS_ALLOC，SESS_FREE，CKPT，TIMER_TRIG，SERERR_TRIG，LOG_REP，MAL_LETTER， CMD_LOGIN 等。

序号	列	数据类型	说明
1	CMD	VARCHAR(24)	命令
2	THREAD_ID	BIGINT	线程 ID
3	SESS_ID	BIGINT	SESSION 的 ID
4	SESS_SEQ	INTEGER	会话序列号，用来唯一标识会话
5	TRX_ID	BIGINT	事务 ID
6	STMT_ID	INTEGER	语句 ID
7	START_TIME	DATETIME	命令开始时间
8	TIME_USED	BIGINT	命令从开始执行到执行结束花费的时间

17．V$CONNECT

显示活动连接的所有信息。

序号	列	数据类型	说明
1	NAME	VARCHAR (128)	连接名称
2	SADDR	BIGINT	会话地址
3	CREATE_TIME	TIMESTAMP(0)	会话创建时间
4	STATUS$	VARCHAR(128)	连接状态
5	TYPE$	VARCHAR(128)	连接类型
6	PROTOCOL_TYPE	TINYINT	协议类型
7	IP_ADDR	VARCHAR(23)	IP 地址

18．V$COSTPARA

显示 SQL 计划的代价信息。

序号	列	数据类型	说明
1	RAN_IO_R_COST	DEC(5,3)	随机读代价
2	RAN_IO_W_COST	DEC(5,3)	随机写代价
3	SEQ_IO_R_COST	DEC(5,3)	顺序读代价
4	SEQ_IO_W_COST	DEC(5,3)	顺序写代价
5	SEL_RATE_EQU	DEC(5,3)	等于谓词的选择率
6	SEL_RATE_SINGLE	DEC(5,3)	非等于谓词的选择率
7	INDEX_JOIN_HIT_RATIO DEC	DEC(5,3)	索引连接命中率

19．V$DATAFILE

显示数据文件信息。

序号	列	数据类型	说明
1	GROUP_ID	INTEGER	所属的表空间 ID
2	ID	INTEGER	数据库文件 ID
3	PATH	VARCHAR (256)	数据库文件路径
4	CLIENT_PATH	VARCHAR (256)	数据库文件路径，专门提供给客户端
5	CREATE_TIME	TIMESTAMP(0)	数据库文件创建时间
6	STATUS$	TINYINT	状态
7	RW_STATUS	TINYINT	读写状态：1 读，2 写
8	LAST_CKPT_TIME	TIMESTAMP(0)	最后一次检查点时间。只对 online 的 DB 做统计，否则就是空
9	MODIFY_TIME	TIMESTAMP(0)	文件修改时间

10	MODIFY_TRX	BIGINT	修改事务
11	TOTAL_SIZE	BIGINT	总大小。单位：页数
12	FREE_SIZE	BIGINT	空闲大小。单位：页数
13	FREE_PAGE_NO	BIGINT	数据文件中连续空白页的起始页号
14	PAGES_READ	BIGINT	读页
15	PAGES_WRITE	BIGINT	写页
16	PAGE_SIZE	INTEGER	页大小，以 BYTE 为单位
17	READ_REQUEST	INTEGER	读请求
18	WRITE_REQUEST	INTEGER	写请求
19	MAX_SIZE	INTEGER	文件最大大小，以 M 为单位
20	AUTO_EXTEND	INTEGER	是否支持自动扩展：1 支持，0 不支持
21	NEXT_SIZE	INTEGER	文件每次扩展大小，以 M 为单位
22	MIRROR_PATH	VARCHAR(256)	镜像文件路径

20. V$DATABASE

显示数据库信息。

序号	列	数据类型	说明
1	NAME	VARCHAR (128)	数据库名称。
2	CREATE_TIME	TIMESTAMP	数据库创建时间
3	ARCH_MODE	CHAR(1)	归档模式：归档或不归档。默认为不归档
4	LAST_CKPT_TIME	TIMESTAMP(0)	最后一次检查点时间。只对 ONLINE 的 DB 做统计，否则就是空
5	STATUS$	TINYINT	状态。1：启动；2：启动，redo 完成；3：MOUNT；4：打开；5：挂起；6：关闭
6	ROLE$	TINYINT	角色。0：普通；1：主库；2：备库
7	MAX_SIZE	BIGINT	最大大小。0 代表只受操作系统限制
8	TOTAL_SIZE	BIGINT	总大小
9	RAC_NODES	INTEGER	RAC集群系统中的实例总数

21. V$DEADLOCK_HISTORY

记录死锁的历史信息。

序号	列	数据类型	说明
1	SEQNO	INTEGER	编号
2	TRX_ID	BIGINT	事务 ID
3	SESS_ID	BIGINT	会话 ID
4	SESS_SEQ	INTEGER	会话序列号，用来唯一标识会话
5	SQL_TEXT	VARCHAR(1000)	产生死锁的 SQL 语句
6	HAPPEN_TIME	DATETIME	死锁发生的时间

22. V$DICT_CACHE_ITEM

显示字典缓存中的字典对象信息。

序号	列	数据类型	说明
1	TYPE	VARCHAR(128)	字典对象的类型
2	ID	INTEGER	字典对象 ID
3	NAME	VARCHAR(128)	字典对象的名称
4	SCHID	INTEGER	字典对象所属模式
5	PID	INTEGER	父 ID

注：TYPE 列对应类型如下：

DB、TABLE、VIEW、INDEX、USER、ROLE、PROC、TRIGGER、CONSTRAINT、SECHEMA、SEQUENCE、DBLINK、SYSROLE、PACKAGE、OBJECT、SYNOM、CRYPT、CIPHER、IDENTITY

23. V$DICT_CACHE

显示字典缓存信息。

序号	列	数据类型	说明
1	ADDR	VARCHAR(20)	地址
2	POOL_ID	INTEGER	缓冲池 ID
3	TOTAL_SIZE	INTEGER	总大小
4	USED_SIZE	INTEGER	实际使用大小
5	DICT_NUM	INTEGER	字典对象总数

24. V$EXP_BUFFERPOOL

显示扩展缓冲区信息，目前只有 DEFAULT BUFFERPOOL 才有扩展缓冲区。

序号	列	数据类型	说明
1	POOL_ID	INTEGER	扩展缓冲区所在缓冲区 ID
2	POOL_NAME	VARCHAR(20)	扩展缓冲区所在缓冲区 NAME
3	PAGE_SIZE	INTEGER	缓冲区页大小
4	N_PAGES	INTEGER	扩展缓冲区页数
5	BUFFER_START	BIGINT	扩展缓冲区起始地址
6	BUFFER_TAIL	BIGINT	扩展缓冲区尾地址

25. V$FLASHBACK_TRX_INFO

显示闪回信息。

序号	列	数据类型	说明
1	START_TRXID	BIGINT	闪回开始事务 ID
2	START_TIMESTAMP	TIMESTAMP	开始时间
3	COMMIT_TRXID	BIGINT	闪回完成事务 ID
4	COMMIT_TIMESTAMP	TIMESTAMP	完成时间
5	LOGIN_USER	VARCHAR(256)	闪回用户名
6	UNDO_CHANGE#	INTEGER	回滚记录序号
7	OPERATION	CHAR(1)	DML 操作类型。 D：删除；U：修改；I：插入；N：更新插入（专门针对 CLUSTER PRIMARY KEY 的插入）； C：事务提交；P：预提交记录；O：default
8	TABLE_NAME	VARCHAR(256)	表名
9	TABLE_OWNER	VARCHAR(256)	表所属用户名
10	ROW_ID	BIGINT	ROWID
11	UNDO_SQL	VARCHAR(3900)	回滚 SQL 语句

26. V$IID

显示下一个创建的数据库对象的 ID。该视图提供用户可以查询下一个创建对象的 ID 的值，可以方便用户查询预知自己所要建立对象的信息。

序号	列	数据类型	说明
1	NAME	VARCHAR(100)	对象名称
2	VALUE	BIGINT	对象值

27. V$INSTANCE

显示实例信息。

序号	列	数据类型	说明
1	NAME	VARCHAR (128)	实例名称
2	HOST_NAME	VARCHAR(128)	主库名称
3	SVR_VERSION	VARCHAR (128)	服务器版本
4	DB_VERSION	VARCHAR (40)	数据库版本
5	START_TIME	TIMESTAMP	服务器启动时间
6	STATUS$	VARCHAR(128)	系统状态
7	MODE$	VARCHAR(128)	模式
8	OGUID	INTEGER	控制文件的 OGUID
9	RAC_SEQNO	INTEGER	RAC 序号
10	RAC_ROLE	VARCHAR(32)	RAC 系统角色（MASTER/SLAVE）

28. V$LATCHES

显示正在等待的线程信息。

序号	列	数据类型	说明
1	OBJECT	BIGINT	等待的对象
2	REQUEST_TYPE	CHAR	等待的锁类型：S 锁，X 锁。
3	THREAD_ID	BIGINT	等待线程 ID
4	N_READERS	INTEGER	读线程个数
5	WRITER	BIGINT	写线程 ID
6	N_WRITERS	INTEGER	写线程拥有该锁的次数
7	WRITE_WAITING	CHAR	是否有写线程在等待，如果有，则不让读线程进入
8	N_READERS_WAIT	INTEGER	读等待个数
9	N_WRITERS_WAIT	INTEGER	写等待个数
10	N_IO_WAIT	INTEGER	IO 等待个数
11	SPACE_ID	INTEGER	页面缓冲控制信息的表空间 ID
12	FILE_ID	INTEGER	页面缓冲控制信息的文件 ID
13	PAGE_NO	INTEGER	数据在文件中的页号

29. V$LOCK

显示活动的事务锁信息。

序号	列	数据类型	说明
1	ADDR	BIGINT	锁地址
2	TRX_ID	BIGINT	所属事务 ID
3	LTYPE	VARCHAR(10)	锁类型：TID 锁、对象锁
4	LMODE	CHAR(2)	锁模式：S 锁、X 锁、IX 锁、IS 锁
5	BLOCKED	INTEGER	是否阻塞
6	TABLE_ID	INTEGER	对应表锁、字典对象 ID
7	ROW_IDX	BIGINT	TID 锁对象事务 ID

30. V$PLSQL_DDL_HISTORY

记录 PLSQL 中执行的 DDL 语句，主要监控 truncate table 和 Execute immediate DDL 语句的情况。

序号	列	数据类型	说明
1	SEQNO	INTEGER	该类语句的执行编号
2	SESS_ID	BIGINT	执行会话 ID
3	SESS_SEQ	INTEGER	会话序列号，用来唯一标识会话

4	TRX_ID	BIGINT	执行事务 ID
5	EXEC_ID	INTEGER	虚拟机的执行序号
6	VM_ID	INTEGER	虚拟机 ID
7	STKFRM_LEVEL	INTEGER	当前栈帧在虚拟机的层次
8	DDL_FROM	VARCHAR(10)	若为 TRUNCATE TABLE，则为'TRUNC'，否则为'EXEC'
9	SQL_TEXT	VARCHAR(1000)	获得该层栈帧执行的 SQL 语句，若字符长度超过 1000，则取前 1000 个字符。
10	DDL_TIME	DATETIME	语句执行时间
11	MAIL_ID	BIGINT	邮件标识号

31. V$PRE_RETURN_HISTORY

记录大量数据返回结果集的历史信息（查询大量数据产生）。

序号	列	数据类型	说明
1	SEQNO	INTEGER	编号
2	TRX_ID	BIGINT	事务 ID
3	VM_ID	INTEGER	虚拟机 ID
4	EXEC_ID	INTEGER	执行 ID，记录执行的次数
5	SESS_ID	BIGINT	会话 ID
6	SESS_SEQ	INTEGER	会话序列号，用来唯一标识会话
7	SQL_TEXT	VARCHAR(1000)	返回的 SQL 语句
8	N_ROWS	BIGINT	返回的行数
9	IS_FIRST	CHAR	是否第一次返回
10	RETURN_TIME	DATETIME	记录返回的时间
11	MAL_ID	BIGINT	邮件标识号

32. V$PROCESS

显示当前进程信息。

序号	列	数据类型	说明
1	PID	INTEGER	进程 ID
2	PNAME	VARCHAR(256)	进程名
3	TRACE_NAME	VARCHAR(256)	日志路径
4	TYPE$	TINYINT(3)	类型

33. V$PSEG_SYS

显示当前回滚段信息。

序号	列	数据类型	说明
1	N_ITEM	INTEGER	系统中存在回滚项目的个数
2	EXTENT_SIZE	INTEGER	每次新申请的回滚页数目

34. V$PSEG_ITEMS

显示回滚系统中当前回滚项信息。

序号	列	数据类型	说明
1	NTH	INTEGER	ITEM 序号
2	N_PAGES	INTEGER	回滚页数
3	N_EXTEND	INTEGER	扩展回滚段次数
4	N_PURGED_PAGES	INTEGER	已 PURGE 的页数
5	N_USED_PAGES	INTEGER	正在使用的页数

6	N_UREC_BYTES	BIGINT	生成回滚记录的总字节数
7	N_COMMIT_TRX	INTEGER	已经提交未 PURGE 的事务数
8	RESERVE_TIME	INTEGER	事务提交后，最长的保留时间，单位秒
9	STACK_SIZE	INTEGER	空闲回滚页堆栈中总页数
10	STACK_SP	INTEGER	空闲回滚页堆栈中空闲页数

35. V$PSEG_COMMIT_TRX

显示回滚项中已提交但未 PURGE 的事务信息。

序号	列	数据类型	说明
1	ITEM_NTH	INTEGER	隶属回滚项的序号
2	TRX_ID	BIGINT	事务 ID
3	CMT_TIME	TIMESTAMP	事物提交时间
4	FPA_FILE_ID	INTEGER	起始页地址文件号。-1 表示该事务没有修改数据，所以回滚文件号为空
5	FPA_PAGE_NO	INTEGER	起始页地址页 ID。-1 表示该事务没有修改数据，所以回滚页为空

36. V$PSEG_PAGE_INFO

显示当前回滚页信息。

序号	列	数据类型	说明
1	ITEM_NTH	INTEGER	隶属回滚项的序号
2	FILE_ID	INTEGER	文件号
3	PAGE_NO	INTEGER	页号
4	TRX_ID	BIGINT	对应事务 ID
5	STATUS	INTEGER	0 ACTIVE，1 COMMIT，2 PURGE，3 PRECOMMIT
6	N_REC	INTEGER	回滚记录个数
7	N_USED_BYTES	INTEGER	页内已使用字节数

37. V$PURGE

显示当前 PURGE 回滚段信息。

序号	列	数据类型	说明
1	OBJ_NUM	INTEGER	待 PURGE 事务个数
2	IS_RUNNING	CHAR	是否正在运行 PURGE(Y/N)
3	PURG_FOR_TS	CHAR	是否正在 PURGE 表空间（Y/N）

38. V$PURGE_PSEG_OBJ

显示 PURGE 系统中，待 PURGE 的所有 PSEG 对象信息。

序号	列	数据类型	说明
1	TRX_ID	BIGINT	事务 ID
2	NEED_PURGE	CHAR	是否需要 PURGE（Y/N）
3	TAB_NUM	INTEGER	涉及表数

39. V$PURGE_PSEG_TAB

显示待 PURGE 表信息。

序号	列	数据类型	说明
1	TRX_ID	BIGINT	事务 ID
2	TAB_ID	INTEGER	表 ID

3	GROUP_ID	INTEGER	控制页所在表空间 ID
4	FILE_ID	INTEGER	控制页文件 ID
5	PAGE_NO	INTEGER	控制页编号
6	TAB_TYPE	INTEGER	表类型：0 普通表、1 全局临时表、2 本地临时表、3 列存储表、4 垂直分区表主表、5 垂直分区表子表、6 范围分区表主表、7 范围分区表子表、8HASH 分区表主表、9HASH 分区表子表、10 位图连接索引表、11LIST 分区表主表、12LIST 分区表子表、13 外部表、14 记录类型数组所用的临时表、15REMOTE TABLE、19HUGE TABLE、24 范围分区表主表、25 范围分区表子表、26HASH 分区表主表、27HASH 分区表子表、28LIST 分区表主表、29LIST 分区表子表、32 位图索引表
7	ROW_COUNT	BIGINT	insert/delete 操作影响表行数。每插入一行加 1，每删除一行减 1

40. V$RESOURCE_LIMIT

显示表、用户的空间限制信息。

序号	列	数据类型	说明
1	NAME	VARCHAR(128)	表/用户名称
2	ID	INTEGER	表/用户 ID
3	TYPE	VARCHAR(10)	表/用户类型
4	SPACE_LIMIT	INTEGER	空间限制大小，以页为单位
5	SPACE_USED	INTEGER	空间实际使用大小，以页为单位

41. V$RLOG

显示日志的总体信息。通过该视图可以了解系统当前日志事务号 LSN 的情况、归档日志情况、检查点的执行情况等。

序号	列	数据类型	说明
1	CKPT_LSN	BIGINT	最近一次检查点 LSN
2	FILE_LSN	BIGINT	已经到盘上的 LSN
3	FLUSH_LSN	BIGINT	当前准备刷盘的 LSN
4	CUR_LSN	BIGINT	当前的 LSN
5	NEXT_SEQ	INTEGER	下一页页号
6	N_MAGIC	INTEGER	本次运行，日志的 magic
7	DB_MAGIC	INTEGER	数据库的 magic
8	FLUSH_PAGES	INTEGER	Flush 链表中的总页数
9	FLUSHING_PAGES	INTEGER	正在刷盘的总页数
10	CUR_FILE	INTEGER	记录刷文件前当前文件的 ID
11	CUR_OFFSET	BIGINT	记录刷文件前 cur_file 的 free
12	CKPT_FILE	INTEGER	最近一次检查点对应的当时的文件号
13	CKPT_OFFSET	BIGINT	最近一次检查点对应的当时的文件偏移
14	FREE_SPACE	BIGINT	目前可用的日志空间
15	TOTAL_SPACE	BIGINT	日志总空间

42. V$RLOGBUF

显示日志 BUFFER 信息。通过该视图可以查询日志 BUFFER 的使用情况，如 BUFFER 状

态、总大小、已使用大小，这样可以带来帮助，如 BUFFER 剩余空间过小产生的失败等。

序号	列	数据类型	说明
1	BEGIN_LSN	BIGINT	log buf 起始 LSN
2	END_LSN	BIGINT	log buf 结束 LSN
3	TOTAL_PAGES	INTEGER	日志 buffer 的总大小页数
4	N_PAGES	INTEGER	Buffer 中剩下可用的页数

43. V$CKPT

显示日志文件刷新的 SLOT 对象信息。

序号	列	数据类型	说明
1	CKPT_RLOG_SIZE	INTEGER	检查点日志大小
2	CKPT_DIRTY_PAGES	INTEGER	脏页的数量。产生多少脏页后，才强制产生检查点
3	CKPT_INTERVAL	INTEGER	系统强制产生检查点的时间间隔(s)
4	CKPT_FLUSH_RATE	DEC(5,2)	每次检查点脏页的刷盘比例
5	CKPT_FLUSH_PAGES	INTEGER	每次检查点至少刷盘的脏页数
6	LAST_BEGIN_TIME	TIMESTAMP	最近一次执行的开始时间
7	LAST_END_TIME	TIMESTAMP	最近一次执行的结束时间
8	CKPT_LSN	BIGINT	最近一次检查点 LSN
9	CKPT_FILE	BIGINT	最近一次检查点对应的当时的文件号
10	CKPT_OFFSET	BIGINT	最近一次检查点对应的当时的文件偏移
11	STATE	VARCHAR(128)	检查点状态，只有2种。0 NONE，其他 PROCESSING

44. V$RLOGFILE

显示日志文件的具体信息。包括文件号、完整路径、文件的状态、文件大小等等。

序号	列	数据类型	说明
1	GROUP_ID	INTEGER	表空间 ID
2	FILE_ID	INTEGER	文件 ID
3	PATH	VARCHAR(256)	文件路径
4	CLIENT_PATH	VARCHAR(256)	文件路径，专门提供给客户端
5	CREATE_TIME	TIMESTAMP	创建时间
6	RLOG_SIZE	BIGINT	文件大小

45. V$RUNTIME_ERR_HISTORY

监控运行时错误历史。异常分为三种：一种是系统异常，用户没有捕获，由 vm_raise_runtime_error 产生；第二种是用户异常，用户捕获错误，并抛出自定义异常，由 nthrow_exec 产生；第三种是语法异常，语法未通过，由 nsvr_build_npar_cop_out 产生。

V$RUNTIME_ERR_HISTORY 视图中各个列的含义如下：

序号	列	数据类型	说明
1	SEQNO	INTEGER	该类语句的执行编号
2	SESS_ID	BIGINT	执行会话 ID
3	SESS_SEQ	INTEGER	会话序列号，用来唯一标识会话
4	TRX_ID	BIGINT	执行事务 ID
5	EXEC_ID	INTEGER	虚拟机的执行序号
6	VM_ID	INTEGER	虚拟机 ID
7	STKFRM_LEVEL	INTEGER	当前栈帧在虚拟机的层次
8	SQL_TEXT	VARCHAR(1000)	获得该层栈帧执行的 SQL 语句，若字符长度超过 1000，则取前 1000 个字符

9	SU_FLAG	CHAR	U =用户异常，S =系统异常，P =语法异常
10	ECPT_CODE	INTEGER	异常错误号
11	ECPT_DESC	VARCHAR(256)	异常描述
12	MAL_ID	BIGINT	邮件标识号
13	ERR_TIME	DATETIME	错误产生的时间

46. V$SESSIONS

显示会话的具体信息，如执行的 sql 语句、主库名、当前会话状态、用户名等等。

序号	列	数据类型	说明
1	SESS_ID	BIGINT	会话 ID
2	SESS_SEQ	INTEGER	会话序列号，用来唯一标识会话
3	SQL_TEXT	VARCHAR(1000)	取 sql 的头 1000 个字符
4	STATE	VARCHAR(8)	会话状态。共 6 种状态：CREATE 创建、STARTUP 启动、IDLE 空闲、ACTIVE 活动、WAIT 等待、UNKNOWN 未知
5	N_STMT	INTEGER	STMT 的容量
6	N_USED_STMT	INTEGER	已使用的 STMT 数量
7	SEQ_NO	INTEGER	序列号
8	CURR_SCH	VARCHAR(128)	当前模式
9	USER_NAME	VARCHAR(128)	当前用户
10	TRX_ID	BIGINT	事务 id
11	CREATE_TIME	DATETIME	会话创建时间
12	CLNT_TYPE	VARCHAR(128)	客户端类型（4 种）
13	TIME_ZONE	VARCHAR(6)	时区
14	CHK_CONS	CHAR	是否忽略约束检查
15	CHK_IDENT	CHAR	是否忽略自增列指定列表检查
16	RDONLY	CHAR	是否是只读会话
17	INS_NULL	CHAR	列不存在 default 选项，是否插入空值
18	COMPILE_FLAG	CHAR	是否忽略编译选项，用于 view，procedure，trigger……的编译
19	AUTO_CMT	CHAR	是否自动提交
20	DDL_AUTOCMT	CHAR	Ddl 语句是否自动提交
21	RS_FOR_QRY	CHAR	非查询语句生成结果集标记
22	CHK_NET	CHAR	是否检查网络
23	ISO_LEVEL	INTEGER	隔离级
24	CLNT_HOST	VARCHAR(128)	客户端主机名
25	APPNAME	VARCHAR(128)	应用程序名
26	CLNT_IP	VARCHAR(128)	客户端 IP
27	OSNAME	VARCHAR(128)	客户端操作系统名
28	CONN_TYPE	VARCHAR(20)	连接类型
29	VPOOLADDR	VARCHAR(128)	内存池的首地址
30	RUN_STATUS	VARCHAR(20)	记录运行状态，可能值为“IDLE”与“RUNNING”
31	MSG_STATUS	VARCHAR(20)	记录消息处理状态，可能值为“RECIEVE”（已接受）与“SEND”（已发送）
32	LAST_RECV_TIME	DATETIME	记录最近接收的消息时间
33	LAST_SEND_TIME	DATETIME	记录最近发送的消息时间

34	DCP_FLAG	CHAR(1)	1) 是否是通过 DCP_PORT 登录 DCP 服务器的会话，Y 表示是，N 表示否 2) 是否是通过 DCP 代理连接到 MPP 的会话，Y 表示是，N 表示否
35	THRD_ID	INTEGER	线程 ID

47. V$SESSION_HISTORY

显示会话历史的记录信息，如主库名、用户名等，与 V$SESSIONS 的区别在于会话历史记录只记录了会话一部分信息，对于一些动态改变的信息没有记录，如执行的 SQL 语句等。

序号	列	数据类型	说明
1	SESS_ID	BIGINT	会话 ID
2	SESS_SEQ	INTEGER	会话序列号，用来唯一标识会话
3	CURR_SCH	VARCHAR(128)	当前模式
4	USER_NAME	VARCHAR(128)	当前用户
5	CREATE_TIME	DATETIME	会话创建时间
6	CLNT_TYPE	VARCHAR(128)	客户类型（4 种）
7	TIME_ZONE	VARCHAR(6)	时区
8	RDONLY	CHAR	是否是只读会话
9	DDL_AUTOCMT	CHAR	DDL 语句是否自动提交
10	RS_FOR_QRY	CHAR	非查询语句生成结果集标记
11	CHK_NET	CHAR	是否检查网络
12	CLNT_HOST	VARCHAR(128)	客户主库名
13	APPNAME	VARCHAR(128)	应用程序名
14	CLNT_IP	VARCHAR(128)	客户端 IP
15	OSNAME	VARCHAR(128)	客户端操作系统名
16	CONN_TYPE	VARCHAR(20)	连接类型

48. V$SQL_HISTORY

显示执行 SQL 的历史记录信息；可以方便用户经常使用的记录进行保存。

序号	列	数据类型	说明
1	SEQ_NO	INTEGER	序列号
2	SQL_ID	INTEGER	当前语句的 SQL ID
3	SESS_ID	BIGINT	会话 ID
4	SESS_SEQ	INTEGER	会话序列号，用来唯一标识会话
5	TRX_ID	BIGINT	事务 ID
6	THREAD_ID	BIGINT	线程 ID
7	TOP_SQL_TEXT	VARCHAR(1000)	栈帧中第一个 SQL
8	SEC_SQL_TEXT	VARCHAR(1000)	栈帧中第二个 SQL
9	THRD_SQL_TEXT	VARCHAR(1000)	栈帧中第三个 SQL
10	START_TIME	DATE_TIME	SQL 执行的起始时间
11	TIME_USED	BIGINT	SQL 执行所使用时间（usec）
12	IS_OVER	CHAR	是否结束
13	EXEC_ID	INTEGER	SQL 执行 ID
14	VM	BIGINT	执行 SQL 的虚拟机
15	STKFRM	BIGINT	当前的栈帧
16	STK_LEVEL	INTEGER	当前栈帧的级别
17	BYTES_DYNAMIC_ALLOCED	BIGINT	动态分配字节数

18	BYTES_DYNAMIC_FREED	BIGINT	动态释放字节数
19	CUR_SQL_NODE	BIGINT	当前的 SQL 节点
20	MAL_ID	BIGINT	邮件标识号
21	N_LOGIC_READ	INTEGER	语句逻辑读的次数
22	N_PHY_READ	INTEGER	语句物理读的次数
23	AFFECTED_ROWS	INTEGER	语句影响的行数
24	HARD_PARSE_FLAG	INTEGER	语句硬解析标记，0：软解析；1：语义解析；2：硬解析

49. V$SQL_NODE_HISTORY

通过该视图既可以查询 SQL 执行节点信息，包括 SQL 节点的类型、进入次数和使用时间等等；又可以查询所有执行的 SQL 节点执行情况，如哪些使用最频繁、耗时多少等。

当 INI 参数 ENABLE_MONITOR 等于 3 时，才会记录 SQL 执行节点信息。

序号	列	数据类型	说明
1	SEQ_NO	INTEGER	序列号
2	EXEC_ID	INTEGER	执行 ID
3	NODE	BIGINT	节点 ID
4	TYPE$	INTEGER	节点类型
5	BYTES_DYNAMIC_ALLOCED	BIGINT	动态分配字节数
6	BYTES_DYNAMIC_FREED	BIGINT	动态释放字节数
7	N_ENTER	INTEGER	节点进入次数
8	TIME_USED	INTEGER	节点执行所耗时
9	PLN_OP_ID	INTEGER	MPP 模式下，节点所属通讯操作符中的序号
10	BYTES_SEND	INTEGER	发送的字节数
11	BYTES_RECV	INTEGER	接收的字节数
12	ROWS_SEND	INTEGER	发送的行数
13	ROWS_RECV	INTEGER	接收的行数
14	BDTA_SEND	INTEGER	发送 BDTA 的次数
15	BDTA_RECV	INTEGER	接收 BDTA 的次数
16	MAL_ID	BIGINT	邮件标识号

50. V$SQL_NODE_NAME

显示所有的 sql 节点描述信息，包括 sql 节点类型、名字和详细描述。

序号	列	数据类型	说明
1	TYPE$	INTEGER	节点类型
2	NAME	VARCHAR(24)	节点的名字
3	DESC_CONTENT	VARCHAR(128)	节点的详细描述

51. V$STKFRM

显示虚拟机栈桢信息。

序号	列	数据类型	说明
1	VM_ID	INTEGER	虚拟机 ID
2	FRAME_LEVEL	INTEGER	栈的层次
3	ADDR	BIGINT	地址
4	RET_IP	BIGINT	返回的指令地址
5	CURR_METHOD	BIGINT	当前方法
6	LOCAL_SPACE	INTEGER	临时变量空间大小
7	ARG_SPACE	INTEGER	参数空间大小

8	OFF_LOCALS	BIGINT	每个变量基于原栈顶的偏移
9	OFF_ARGS	BIGINT	每个参数基于原栈顶的偏移
10	CURR_BP	BIGINT	新的栈顶
11	IP_BASE	BIGINT	指令基址
12	SQL_VM_NODE	BIGINT	VM 地址
13	SQL_TRX_IDS	BIGINT	事务 ID 地址
14	SYSTEM_FLAG	CHAR	系统标记
15	FRAME_SIZE	INTEGER	栈的大小

52. V$STMTS

显示当前活动会话的最近的语句的相关信息。

序号	列	数据类型	说明
1	STMT_ID	INTEGER	语句 ID，从 iid 页取得
2	TYPE$	INTEGER	语句类型，如 select 或 update 等
3	EXEC_DIRECT	CHAR	是否立即执行标记
4	SESS_ID	BIGINT	对应的 Session 的 ID
5	SESS_SEQ	INTEGER	会话序列号，用来唯一标识会话
6	PLN	BIGINT	查询计划
7	DML_RCNT	BIGINT	DML 操作所影响的行数
8	REQ_RCNT	BIGINT	prepare 或 execute 查询要求返回的行数
9	BATCH_FLAG	CHAR	是否批量绑定参数
10	N_PARA_ROWS	INTEGER	多行参数时的行数目
11	NTH_PARA_ROW	INTEGER	多行参数时的当前行序号
12	CUR_PARA_OFF	INTEGER	参数的当前偏移
13	PARA_BUF	BIGINT	参数的内存地址
14	SQL_TEXT	VARCHAR(1000)	SQL 语句
15	CURSOR_FORWARD_ONLY	CHAR	是否是只能向前移动的游标
16	CURSOR_NAME	VARCHAR(128)	游标名字
17	CURSOR_KEEP_NAME	CHAR	是否保存游标名字
18	CURSOR_REF_ID	INTEGER	引用的游标语句句柄
19	CURSOR_REF_SEQNO	INTEGER	引用语句的执行序号

53. V$SYSSTAT

显示系统统计信息。

序号	列	数据类型	说明
1	ID	INTEGER	统计对象 ID
2	CLASSID	INTEGER	统计对象所属类别 ID。1：字典信息 2：SQL 3：事务 4：检查点 5：RLOG 6：UNDO 7：IO 8：B 树 9：网络 10：文件 11：内存 12：CPU 13：OS 15：限流控制 20：其它
3	NAME	VARCHAR(128)	统计对象名
4	STAT_VAL	BIGINT	统计值

54. V$TABLESPACE

显示表空间信息，不包括回滚表空间信息。

序号	列	数据类型	说明

1	ID	INTEGER	表空间 ID
2	NAME	VARCHAR (128)	表空间名称
3	CACHE	VARCHAR (20)	CACHE 名
4	TYPE$	TINYINT	表空间类型：1 DB 类型，2 临时表空间
5	STATUS$	TINYINT	状态。0 ONLINE，1 OFFLINE
6	MAX_SIZE	BIGINT	最大大小。0 代表只受操作系统限制（暂无实际意义）
7	TOTAL_SIZE	BIGINT	总大小(以页为单位)
8	FILE_NUM	INTEGER	包含的文件数
9	ENCRYPT_NAME	VARCHAR(128)	加密算法名
10	ENCRYPTED_KEY	VARCHAR(500)	加密密钥，16 进制

55. V$HUGE_TABLESPACE

显示 HUGE 表空间信息，不包括回滚表空间信息。

序号	列	数据类型	说明
1	ID	INTEGER	表空间 ID
2	NAME	VARCHAR (128)	表空间名称
3	PATHNAME	VARCHAR (256)	表空间路径

56. V$THREADS

显示系统中所有活动线程的信息。

序号	列	数据类型	说明
1	ID	BIGINT	线程 ID
2	NAME	VARCHAR(128)	线程名
3	START_TIME	DATETIME	线程开始时间

57. V$TRX

显示所有活动事务的信息。通过该视图可以查看所有系统中所有的事务以及相关信息，如锁信息等。

序号	列	数据类型	说明
1	ID	BIGINT	当前活动事务的 ID 号
2	NEXTID	BIGINT	下一个事务 ID 号
3	MIN_ACTIVE_ID	BIGINT	所有活动事务 ID 号最小者
4	STATUS	VARCHAR(20)	当前事务的状态
5	ISOLATION	INTEGER	隔离级
6	READ_ONLY	CHAR	是否是一个只读事务
7	SESS_ID	BIGINT	当前事务的所在会话
8	SESS_SEQ	INTEGER	会话序列号，用来唯一标识会话
9	INS_CNT	INTEGER	插入回滚记录个数
10	DEL_CNT	INTEGER	删除回滚记录个数
11	UPD_CNT	INTEGER	更新回滚记录个数
12	UPD_INS_CNT	INTEGER	更新插入回滚记录个数
13	UREC_SEQNO	INTEGER	当前 UNDO 记录的递增序列号
14	WAITING	BIGINT	事务等待的锁

58. V$TRXWAIT

显示事务等待信息。

序号	列	数据类型	说明
1	ID	BIGINT	事务 ID

2	WAIT_FOR_ID	BIGINT	所等待的事务 ID
3	WAIT_TIME	INTEGER	当前等待时间

59. V$TRX_VIEW

显示当前事务可见的所有活动事务视图信息。根据达梦多版本规则，通过该视图可以查询系统中自己所见的事务信息；可以通过与 v$trx 表的连接查询它所见事务的具体信息。

序号	列	数据类型	说明
1	SELF_ID	BIGINT	活动事务 ID
2	ACTIVE_ID	BIGINT	所见的事务活动事务 ID

60. V$VERSION

显示版本信息，包容服务器版本号与 DB 版本号。

序号	列	数据类型	说明
1	BANNER	VARCHAR(80)	版本标识

61. V$VMS

显示虚拟机信息。

序号	列	数据类型	说明
1	ID	INTEGER	虚拟机 ID
2	TRX_ID	BIGINT	事务 ID
3	STMT_ID	INTEGER	语句 ID
4	EXP_FLAG	CHAR	表达式标识
5	VSTACK_SIZE	INTEGER	栈大小
6	VSTACK	BIGINT	栈基址
7	VTOP	INTEGER	栈顶离基址的距离
8	VUSED	INTEGER	栈已用空间的大小
9	MEMOBJ	BIGINT	内存地址
10	STKFRM_DEPTH	INTEGER	栈深度
11	FREE_STKFRMS	INTEGER	已释放的栈
12	CURR_FRM	BIGINT	当前栈
13	IP	BIGINT	IP 地址
14	RT_HEAP	BIGINT	PL/SQL 所使用的堆
15	SQL_NO	INTEGER	SQL 语句树的序号
16	RS_SEQ_NO	INTEGER	结果集序号
17	ECPT_CODE	INTEGER	返回的 CODE
18	ERR_DESC	VARCHAR(256)	错误描述
19	ROW_AFFECHED	BIGINT	结果集的行数
20	SQL_TYPE	INTEGER	SQL 语句类型
21	N_FUNS	INTEGER	已使用函数的数目

62. V$VPOOL

显示当前系统中各种内存池的信息。如数据字典缓冲池，session 池等。

序号	列	数据类型	说明
1	VPOOL_ID	INTEGER	内存池的 ID
2	BUFFER	BIGINT	内存池空间的地址
3	V_SIZE	BIGINT	内存池的大小
4	MAX_FREE	INTEGER	free_list 最大可能幂数
5	USED	BIGINT	当前已分配大小

6	IS_PUBLIC	CHAR	内存池是否是共用的
7	IS_DIRTY	CHAR	内存池是否使用了池外空间
8	V_DESC	VARCHAR(256)	内存池描述

63. V$WAIT_CLASS

通过该动态视图可以按等待的类别查询等待的时间和次数。等待的类别有：

1. IO READING
2. ROW LOCK WAITING
3. TABLE LOCK WAITING
4. DICT LOCK WAITING
5. TASK WAITING
6. NETWORK WAITING
7. REDO LOG FLUSH WAITING
8. PAGE CONFLICT
9. BUFFER WAITING

通过查询该视图，可以得到系统中哪些资源等待的时间较长，哪些资源等待比较频繁，这样可以较好的提出相应措施，改善系统性能。

序号	列	数据类型	说明
1	CLASS_NO	SMALLINT	等待类别号
2	CLASS_NAME	VARCHAR(32)	等待类别的名字
3	TOTAL_WAITS	BIGINT	等待的次数
4	TIME_WAITED	BIGINT	等待的时间

64. V$WAIT_HISTORY

通过该视图可以查询等待事件的具体信息，如等待的线程 id，会话 id 等。可以查看具体等待事件的信息，如果某个事务等待时间过长，则可以查询到具体事务信息以及所在的线程和所牵涉的对象，分析原因进行优化等操作。

序号	列	数据类型	说明
1	THREAD_ID	BIGINT	线程 ID
2	TRX_ID	BIGINT	事务 ID
3	WAIT_CLASS	INTEGER	等待类型号
4	WAIT_OBJECT	BIGINT	等待对象
5	WAIT_START	DATETIME	等待事件的开始时间
6	WAIT_TIME	BIGINT	等待时间（单位：usec）
7	SPACE_ID	INTEGER	表空间号
8	FILE_ID	INTEGER	文件号
9	PAGE_NO	INTEGER	页号

65. V$WTHRD_HISTORY

通过本视图可以观察系统从启动以来,所有活动过线程的相关历史信息。其中 CHG_TYPE 有 REUSE_OK（本 SESSION 重用成功）, REUSE_FAIL (重用失败),TO_IDLE(不重用，直接变 IDLE)等几种类型。

序号	列	数据类型	说明
1	SEQNO	INTEGER	序号
2	THREAD_ID	BIGINT	线程号
3	CHG_TYPE	VARCHAR(24)	线程类型
4	CHG_TIME	DATETIME	更改时间

66. V$HASH_MERGE_USED_HISTORY

HASH MERGE 连接操作符使用的缓存信息。

序号	列	数据类型	说明
1	TYPE$	INTEGER	节点类型
2	SEQNO	INTEGER	序号
3	EXEC_ID	INTEGER	执行 ID
4	SQL_TEXT	VARCHAR(1000)	执行 SQL 文本
5	MERGE_USED	INTEGER	HASH MERGE 使用的缓存空间，单位 MB
6	MAL_ID	BIGINT	邮件标识号

67. V$DM_INI

实例配置信息。所有 ini 参数和 dminit 建库参数信息（系统值、最小值和最大值）。

序号	列	数据类型	说明
1	PARA_NAME	VARCHAR (128)	参数名称
2	PARA_VALUE	VARCHAR (256)	系统参数值
3	MIN_VALUE	VARCHAR (256)	最小值
4	MAX_VALUE	VARCHAR (256)	最大值

68. V$DM_ARCH_INI

归档参数信息。

序号	列	数据类型	说明
1	ARCH_NAME	VARCHAR (128)	归档名称
2	ARCH_TYPE	VARCHAR (128)	归档类型
3	ARCH_DEST	VARCHAR (256)	归档实例名
4	ARCH_FILE_SIZE	INTEGER	归档文件大小
5	ARCH_SPACE_LIMIT	INTEGER	归档文件的磁盘空间限制，单位是 MB
6	ARCH_TIMER_NAME	VARCHAR (128)	定时器名称
7	ARCH_IS_VALID	CHAR(1)	归档状态，是否有效

69. V$DM_MAL_INI

MAL 参数信息。

序号	列	数据类型	说明
1	MAL_TYPE	VARCHAR (128)	MAL 名
2	INST_NAME	VARCHAR (256)	站点名
3	IP	VARCHAR (30)	IP 地址
4	MAL_PORT	INTEGER	端口号
5	INST_IP	VARCHAR (256)	对应实例 IP 地址
6	INST_PORT	INTEGER	对应实例的端口号

70. V$DM_REP_RPS_INST_NAME_INI

数据复制服务器参数信息。

序号	列	数据类型	说明
1	INST_RPS	VARCHAR (256)	RPS 服务器的实例名

71. V$DM_REP_MASTER_INFO_INI

数据复制主库参数信息。

序号	列	数据类型	说明
1	REP_ID	INTEGER	复制 ID

72. V$DM_REP_SLAVE_INFO_INI

数据复制从机参数信息。

序号	列	数据类型	说明
1	REP_ID	INTEGER	复制 ID
2	MASTER_INSTNAME	VARCHAR (256)	主站点实例名

73. V$DM_REP_SLAVE_TAB_MAP_INI

数据复制从机表对应关系参数信息。

序号	列	数据类型	说明
1	REP_ID	INTEGER	复制 ID
2	SRC_TAB_ID	INTEGER	源表 ID
3	DST_TAB_ID	INTEGER	目的表 ID

74. V$DM_REP_SLAVE_SRC_COL_INFO_INI

数据复制从机列对应关系参数信息。

序号	列	数据类型	说明
1	REP_ID	INTEGER	参数值
2	SRC_TAB_ID	INTEGER	源表 ID
3	DST_ TAB_ID	INTEGER	目的表 ID
4	COL_ID	INTEGER	列 ID
5	SQL_PL_TYPE	INTEGER	SQL 类型
6	LEN	INTEGER	SQL 长度
7	PREC	INTEGER	游标类型

75. V$DM_LLOG_INFO_INI

逻辑日志信息参数信息。

序号	列	数据类型	说明
1	LLOG_NAME	VARCHAR (128)	逻辑日志类型名
2	ID	INTEGER	ID
3	LLOG_PATH	VARCHAR (256)	路径
4	REP_FLAG	BIT	是否存在复制关系
5	REP_INSTNAME	VARCHAR (256)	复制关系从站点实例名
6	REP_TYPE	VARCHAR (128)	表示同步或异步的复制关系
7	REP_TIMER	VARCHAR (128)	表示当异步复制关系时，本地定时器的名字

76. V$DM_LLOG_TAB_MAP_INI

逻辑日志与表对应的参数信息。

序号	列	数据类型	说明
1	LLOG_ID	INTEGER	日志 ID
2	TAB_ID	INTEGER	表 ID

77. V$DM_TIMER_INI

定时器参数信息。

序号	列	数据类型	说明
1	TIMER_NAME	VARCHAR(128)	定时器名称
2	TYPE	TINYINT	类型
3	FREQ_MONTH_WEEK_INTERVAL	INTEGER	月或周的间隔
4	FREQ_SUB_INTERVAL	INTEGER	间隔天数
5	FREQ_MINUTE_INTERVAL	INTEGER	分钟间隔

6	START_TIME	TIME	开始时间
7	END_TIME	TIME	结束时间
8	DURING_START_DATE	DATETIME	有效时间段的开始日期时间
9	DURING_END_DATE	DATETIME	有效时间段的结束日期时间
10	NO_END_DATE_FLAG	CHAR(1)	无结束日期标识
11	DESCRIBE	VARCHAR(256)	描述
12	IS_VALID	CHAR(1)	是否有效

78. V$UTSK_INFO

查询守护进程向服务器发送请求的执行情况。

注意在 RUNNING 字段值为 TRUE 时，DSEQ、CODE 和 SEND_LSN 的值才有意义，另外 SEND_LSN 和 Recover_BREAK 在主库上查询才有意义，并且需要主库的守护进程处于 Recovery 状态。

序号	列	数据类型	说明
1	RUNNING	VARCHAR(64)	当前系统是否正在处理守护进程请求，TRUE 表示正在处理，FALSE 表示没有。
2	DSEQ	INTEGER	守护进程发送的报文序号
3	CODE	INTEGER	DSEQ 对应的执行结果
4	SEND_LSN	BIGINT	主库发送给备库的最大归档 LSN
5	RECOVER_BREAK	VARCHAR(64)	用于显示主库的 Recover 流程是否被中断，TRUE 表示被中断，FALSE 表示没有
6	AUTO_SWITCH	VARCHAR(64)	守护系统配置的切换模式，TRUE 表示故障自动切换，FALSE 表示故障手动切换。

79. V$SYSTEMINFO

系统信息视图。

序号	列	数据类型	说明
1	N_CPU	INTEGER	CPU 个数
2	TOTAL_PHY_SIZE	BIGINT	物理内存总大小
3	FREE_PHY_SIZE	BIGINT	剩余物理内存大小
4	TOTAL_VIR_SIZE	BIGINT	虚拟内存总大小
5	FREE_VIR_SIZE	BIGINT	剩余虚拟内存大小
6	TOTAL_DISK_SIZE	BIGINT	磁盘总大小
7	FREE_DISK_SIZE	BIGINT	剩余磁盘大小
8	DRIVER_NAME	VARCHAR(5)	驱动器名称
9	DRIVER_TOTAL_SIZE	BIGINT	驱动器总空间大小
10	DRIVER_FREE_SIZE	BIGINT	驱动器剩余空间大小
11	LOAD_ONE_AVERAGE	FLOAT	每分钟平均负载
12	LOAD_FIVE_AVERAGE	FLOAT	每五分钟平均负载
13	LOAD_FIFTEEN_AVERAGE	FLOAT	每十五分钟平均负载
14	CPU_USER_RATE	FLOAT	用户级的 CPU 使用率
15	CPU_SYSTEM_RATE	FLOAT	用户级的 CPU 使用率
16	CPU_IDLE_RATE	FLOAT	用户级的 CPU 使用率
17	SEND_BYTES_TOTAL	BIGINT	发送的总字节数
18	RECEIVE_BYTES_TOTAL	BIGINT	接收的总字节数

19	SEND_BYTES_PER_SECOND	BIGINT	当前每秒发送字节数
20	RECEIVE_BYTES_PER_SECOND	BIGINT	当前每秒接收字节数
21	SEND_PACKAGES_PER_SECOND	BIGINT	当前每秒发送数据包数
22	RECEIVE_PACKAGES_PER_SECOND	BIGINT	当前每秒接收数据包数

80. V$NETINFO

网络信息视图。

序号	列	数据类型	说明
1	RECV_BYTES	BIGINT	发送的字节数
2	SEND_BYTES	BIGINT	接收的字节数

81. V$SEGMENTINFO

索引叶子段信息视图。查询该视图时，一定要带 WHERE 条件，并且必须是等值条件。

序号	列	数据类型	说明
1	INDEX_ID	INTEGER	索引 ID
2	SEG_ID	INTEGER	段 ID
3	LOGIC_PAGE_NO	INTEGER	逻辑页号
4	PHY_PAGE_NO	INTEGER	物理页号
5	PHY_FIRST_PAGE_IN_EXTENT	INTEGER	物理页所属的 extent 的第一个物理页
6	FIL_ID	INTEGER	文件 ID
7	USE_BYTES	BIGINT	已使用的字节数
8	FREE_BYTES	BIGINT	空闲的字节数

82. V$PARAMETER

显示 ini 参数和 dminit 建库参数的类型及参数值信息(当前会话值、系统值及 dm.ini 文件中的值)。

序号	字段	类型	说明
1	ID	INTEGER	ID 号
2	NAME	VARCHAR(80)	参数名字
3	TYPE	VARCHAR(200)	参数类型。 READ ONLY：手动参数，表示服务器运行过程中不可修改； IN FILE：静态参数，只可修改 ini 文件； SYS 和 SESSION：动态参数，ini 文件和内存同时可修改，其中，SYS 系统级参数；SESSION 会话级参数
4	VALUE	VARCHAR(4000)	参数的值（当前会话）
5	SYS_VALUE	VARCHAR(4000)	参数的值（系统）
6	FILE_VALUE	VARCHAR(4000)	参数的值（ini 文件）
7	DESCRIPTION	VARCHAR2(255)	参数描述

83. V$MAL_SYS

MAL 系统信息视图。

序号	列	数据类型	说明
1	SYS_STATUS	INTEGER	MAL 系统状态：0:OPEN,1:PRE_SHUTDOWN,2:SHUTDOWN
2	STMT_ID	INTEGER	MAL 系统当前 stmtid

3	NEXT_MAL_ID	BIGINT	下一个 MAL_ID
4	MAL_PORT	INTEGER	MAL 监听端口
5	N_SITE	INTEGER	MAL 配置的站点数目
6	MAL_NUM	INTEGER	MAL 系统邮箱数目
7	MAL_SEQ_NO	INTEGER	站点本身的 MAL 序号
8	EMPTY_LET_NUM	INTEGER	空邮件数目
9	MAL_CHECK_INTERVAL	INTEGER	链路检测间隔
10	MAL_CONN_FAIL_INTERVAL	INTEGER	认定链路断开的时间间隔
11	MAL_COMPRESS_LEVEL	INTEGER	邮件压缩级别

84. V$MAL_INFO

MAL 邮箱信息视图。

序号	列	数据类型	说明
1	MAL_ID	BIGINT	MAL 标示号
2	ORG_SITE_NO	INTEGER	创建 MAL 的原始站点号
3	DEST_SITE_NO	INTEGER	目标站点号
4	MAX_BUF_SIZE	BIGINT	保存邮件的最大缓存大小
5	USED_BUF_SIZE	BIGINT	已经使用的缓存大小
6	RECEIVE_NUM	INTEGER	收到的邮件数
7	DISCARD_NUM	INTEGER	废弃的邮件数
8	LETTER_NUM	INTEGER	普通邮件数目
9	PLN_LET_NUM	INTEGER	执行计划中的邮件总数目
10	WAIT_FLAG	INTEGER	等待状态。1：等待 0：非等待
11	MCPR_FLAG	INTEGER	MAL 任务处理情况。0：RESUME； 1：MPLN；2：CACEL；4：PAUSE
12	SESS_ID	BIGINT	MAL 对应的会话 ID，和 V$SESSIONS 中的 SESS_ID 对应

85. V$MAL_LETTER_INFO

MAL 上的信件信息视图。

序号	列	数据类型	说明
1	MAL_ID	BIGINT	MAL 标示号
2	IS_IN_MEM	INTEGER	是否在缓存中
3	FILE_ID	INTEGER	所在临时表空间的文件号
4	PAGE_NO	INTEGER	所在临时表空间的页号
5	STMT_ID	INTEGER	句柄 ID
6	PLN_OP_ID	INTEGER	通讯操作符的计划序号
7	ORG_SITE	INTEGER	原始站点号
8	MPP_EXEC_ID	INTEGER	操作符的执行序号
9	BUILD_TIME	BIGINT	邮件发送序号
10	P_SRC_SITE	INTEGER	邮件的物理站点号
11	L_SRC_SITE	INTEGER	邮件的逻辑站点号
12	MSG_LEN	INTEGER	信件的消息长度

86. V$MAL_USING_LETTERS

服务器中正在使用或者使用过但是没有释放的邮件信息，用于检查 MAL 系统潜在的内存泄露。

序号	列	数据类型	说明

1	MAL_ID	BIGINT	MAL 标示号
2	STMT_ID	INTEGER	句柄 ID
3	PLN_OP_ID	INTEGER	通讯操作符的计划序号
4	ORG_SITE	INTEGER	原始站点号
5	SRC_SITE	INTEGER	源站点号
6	DEST_SITE	INTEGER	目标站点号
7	MPP_EXEC_ID	INTEGER	操作符的执行序号
8	BUILD_TIME	BIGINT	邮件发送序号
9	P_SRC_SITE	INTEGER	邮件的物理站点号
10	L_SRC_SITE	INTEGER	邮件的逻辑站点号
11	CMD_TYPE	VARCHAR(128)	邮件的消息类型
12	MSG_LEN	INTEGER	信件的消息长度

87. V$MPP_CFG_SYS

MPP 系统配置信息视图。

序号	列	数据类型	说明
1	SYS_STATE	VARCHAR(128)	系统状态
2	N_SITE	INTEGER	总站点数
3	N_ERR_SITE	INTEGER	故障站点数
4	SELF_EP_SEQNO	INTEGER	当前站点序号

88. V$MPP_CFG_ITEM

MPP站点配置信息视图。

序号	列	数据类型	说明
1	SERVICE_NAME	VARCHAR(128)	服务名
2	INST_NAME	VARCHAR(128)	实例名
3	EP_SEQNO	INTEGER	站点序号
4	STATE	VARCHAR (128)	站点状态

89. V$MAL_SITE_INFO

MAL站点信息视图，MPP模式下，自动收集MPP各个站点的信息。

序号	列	数据类型	说明
1	SRC_SITE_SEQ	INTEGER	发送邮件的源站点
2	DEST_SITE_SEQ	INTEGER	目标站点序号
3	MAL_PORT_NUM	INTEGER	目标站点到本站点的 MAL 链路数
4	BUILD_TIME	BIGINT	下一个邮件发送序号
5	LBTAPFDS	BIGINT	收到的邮件中已处理的最后一个邮件的序号
6	CUR_LETTER_NUM	INTEGER	当前保存不连续邮件的个数
7	MAX_LETTER_NUM	INTEGER	目前为止不连续邮件的最大个数
8	TOTAL_LINK_NUM	INTEGER	已创建到目标站点的 MAL_LINK 数
9	FREE_LINK_NUM	INTEGER	当前空闲的 MAL_LINK 数
10	SEND_LETTER_NUM	INTEGER	当前发送的邮件数

90. V$DBLINK

动态使用到的数据库链接信息视图。

序号	列	数据类型	说明
1	LINK_CONN	BIGINT	DBLINK 连接句柄
2	LINK_ID	INTEGER	DBLINK 的 ID

3	LINK_NAME	VARCHAR(128)	DBLINK 的名称
4	SCH_ID	INTEGER	DBLINK 模式 ID
5	OWNER_ID	INTEGER	DBLINK 用户 ID
6	IS_PUBLIC	VARCHAR(3)	是否为 PUBLIC
7	LOGIN_NAME	VARCHAR(128)	登录名
8	HOST_NAME	VARCHAR(128)	主库名
9	PORT_NUM	INTEGER	端口号
10	LOGGED_ON	VARCHAR(3)	DBLINK 当前是否已链接
11	HETEROGENEOUS	VARCHAR(3)	DBLINK 同步链接为 YES，异步链接为 NO
12	PROTOCOL	VARCHAR(6)	DBLINK 通信协议
13	IN_USE	VARCHAR(3)	当前 DBLINK 句柄是否正在被使用

91. V$DANGER_EVENT

数据库重要事件和行为信息视图。

序号	列	数据类型	说明
1	OPTIME	DATETIME	重要事件或行为发生的时间
2	OPERATION	VARCHAR(1024)	重要事件或行为执行的语句
3	OPUSER	VARCHAR(128)	执行重要事件或行为的用户

92. V$RAC_EP_INFO

显示实例信息。

序号	列	数据类型	说明
1	EP_NAME	VARCHAR (128)	实例名称
2	EP_SEQNO	INTEGER	RAC 序号
3	EP_GUID	BIGINT	EP 唯一标识码
4	EP_TIMESTAMP	BIGINT	EP 时间戳
5	EP_MODE	VARCHAR(32)	EP 模式
6	EP_STATUS	VARCHAR(32)	EP 状态

93. V$RAC_GBS_POOL

显示 GBS 控制结构的信息。

序号	列	数据类型	说明
1	N_CTL	INTEGER	GBS 控制块总数
2	N_FREE_CTL	INTEGER	空闲的 GBS 控制块数目
3	N_SUB_POOL	INTEGER	GBS_POOL 个数

94. V$RAC_GBS_POOLS_DETAIL

显示分片的 GBS_POOL 详细信息。

序号	列	数据类型	说明
1	POOL_ID	INTEGER	GBS_POOL 编号
2	N_USED_CTL	INTEGER	正在使用的 GBS 控制块数目
3	N_REQUEST	INTEGER	正在等待 GBS 控制块的请求数目
4	N_FREE_REQUEST	INTEGER	空闲的 GBS 请求控制块数目

95. V$RAC_GBS_CTL

显示 GBS 控制块信息。多个 pool，依次扫描。

序号	列	数据类型	说明
1	POOL_ID	INTEGER	GBS_POOL 编号

2	TS_ID	INTEGER	表空间号
3	FILE_ID	INTEGER	文件号
4	PAGE_NO	INTEGER	页号
5	ACCESS_MAP	INTEGER	曾经访问此数据页的 EP
6	FRESH_EP	INTEGER	最新数据所在 EP
7	FRESH_LSN	BIGINT	最新修改对应的 LSN 值
8	N_REPLACED	INTEGER	控制块被替换次数
9	N_REVOKED	INTEGER	权限被回收次数
10	N_OWNER	INTEGER	拥有权限的 EP 数
11	N_REQUEST	INTEGER	请求授权的 EP 数
12	N_REVOKING	INTEGER	正在回收权限的 EP 数

96. V$RAC_GBS_CTL_DETAIL

显示 GBS 控制块详细信息。多个 pool，依次扫描。

序号	列	数据类型	说明
1	POOL_ID	INTEGER	GBS_POOL 编号
2	TS_ID	INTEGER	表空间号
3	FILE_ID	INTEGER	文件号
4	PAGE_NO	INTEGER	页号
5	ACCESS_MAP	INTEGER	曾经访问此数据页的 EP
6	FRESH_EP	INTEGER	最新数据所在 EP
7	FRESH_LSN	BIGINT	最新修改对应的 LSN 值
8	N_REPLACED	INTEGER	控制块被替换次数
9	N_REVOKED	INTEGER	权限被回收次数
10	N_OWNER	INTEGER	拥有权限的 EP 数
11	N_REQUEST	INTEGER	请求授权的 EP 数
12	N_REVOKING	INTEGER	正在回收权限的 EP 数
13	TYPE	VARCHAR(32)	详细信息类型（OWNER/REQUEST/REVOKING）
14	MODE	INTEGER	封锁模式，0/1/2/4：N_LATCH/X_LATCH/S_LATCH/F_LATCH
15	EP_SEQNO	INTEGER	拥有、请求、或者回收封锁的 EP
16	REAL_FLUSH	CHAR	是否真正执行刷盘请求（'Y'/'N'）

97. V$RAC_GBS_CTL_LRU_FIRST

显示 GBS 控制块 LRU 链表首页信息。多个 pool，依次扫描。

序号	列	数据类型	说明
1	POOL_ID	INT	GBS_POOL 编号
2	TS_ID	INT	表空间号
3	FILE_ID	INT	文件号
4	PAGE_NO	INTEGER	页号
5	ACCESS_MAP	INT	曾经访问此数据页的 EP
6	FRESH_EP	INTEGER	最新数据所在 EP
7	FRESH_LSN	BIGINT	最新修改对应的 LSN 值
8	N_REPLACED	INTEGER	控制块被替换次数
9	N_REVOKED	INTEGER	权限被回收次数
10	N_OWNER	INTEGER	拥有权限的 EP 数
11	N_REQUEST	INTEGER	请求授权的 EP 数

12	N_REVOKING	INTEGER	正在回收权限的 EP 数

98. V$RAC_GBS_CTL_LRU_FIRST_DETAIL

显示 GBS 控制块 LRU 链表首页详细信息。多个 pool，依次扫描。

序号	列	数据类型	说明
1	POOL_ID	INTEGER	GBS_POOL 编号
2	TS_ID	INTEGER	表空间号
3	FILE_ID	INTEGER	文件号
4	PAGE_NO	INTEGER	页号
5	ACCESS_MAP	INTEGER	曾经访问此数据页的 EP
6	FRESH_EP	INTEGER	最新数据所在 EP
7	FRESH_LSN	BIGINT	最新修改对应的 LSN 值
8	N_REPLACED	INTEGER	控制块被替换次数
9	N_REVOKED	INTEGER	权限被回收次数
10	N_OWNER	INTEGER	拥有权限的 EP 数
11	N_REQUEST	INTEGER	请求授权的 EP 数
12	N_REVOKING	INTEGER	正在回收权限的 EP 数
13	TYPE	VARCHAR(32)	详细信息类型（OWNER/REQUEST/REVOKING）
14	MODE	INTEGER	封锁模式，0/1/2/4: N_LATCH/X_LATCH/S_LATCH/F_LATCH
15	EP_SEQNO	INTEGER	拥有、请求、或者回收封锁的 EP
16	REAL_FLUSH	CHAR	是否真正执行刷盘请求（'Y'/'N'）

99. V$RAC_GBS_CTL_LRU_LAST

显示 GBS 控制块 LRU 链表尾页信息。多个 pool，依次扫描。

序号	列	数据类型	说明
1	POOL_ID	INTEGER	GBS_POOL 编号
2	TS_ID	INTEGER	表空间号
3	FILE_ID	INTEGER	文件号
4	PAGE_NO	INTEGER	页号
5	ACCESS_MAP	INTEGER	曾经访问此数据页的 EP
6	FRESH_EP	INTEGER	最新数据所在 EP
7	FRESH_LSN	BIGINT	最新修改对应的 LSN 值
8	N_REPLACED	INTEGER	控制块被替换次数
9	N_REVOKED	INTEGER	权限被回收次数
10	N_OWNER	INTEGER	拥有权限的 EP 数
11	N_REQUEST	INTEGER	请求授权的 EP 数
12	N_REVOKING	INTEGER	正在回收权限的 EP 数

100. V$RAC_GBS_CTL_LRU_LAST_DETAIL

显示 GBS 控制块 LRU 链表尾页详细信息。多个 pool，依次扫描。

序号	列	数据类型	说明
1	POOL_ID	INTEGER	GBS_POOL 编号
2	TS_ID	INTEGER	表空间号
3	FILE_ID	INTEGER	文件号
4	PAGE_NO	INTEGER	页号
5	ACCESS_MAP	INTEGER	曾经访问此数据页的 EP
6	FRESH_EP	INTEGER	最新数据所在 EP

7	FRESH_LSN	BIGINT	最新修改对应的 LSN 值
8	N_REPLACED	INTEGER	控制块被替换次数
9	N_REVOKED	INTEGER	权限被回收次数
10	N_OWNER	INTEGER	拥有权限的 EP 数
11	N_REQUEST	INTEGER	请求授权的 EP 数
12	N_REVOKING	INTEGER	正在回收权限的 EP 数
13	TYPE	VARCHAR(32)	详细信息类型（OWNER/REQUEST/REVOKING）
14	MODE	INTEGER	封锁模式，0/1/2/4： N_LATCH/X_LATCH/S_LATCH/F_LATCH
15	EP_SEQNO	INTEGER	拥有、请求、或者回收封锁的 EP
16	REAL_FLUSH	CHAR	是否真正执行刷盘请求（'Y'/'N'）

101. V$RAC_GBS_REQUEST_CTL

显示等待 GBS 控制块的请求信息。多个 pool，依次扫描。

序号	列	数据类型	说明
1	POOL_ID	INTEGER	GBS_POOL 编号
2	TS_ID	INTEGER	表空间号
3	FILE_ID	INTEGER	文件号
4	PAGE_NO	INTEGER	页号
5	MODE	INTEGER	封锁模式，0/1/2/4： N_LATCH/X_LATCH/S_LATCH/F_LATCH
6	EP_SEQNO	INTEGER	请求封锁的 EP

102. V$RAC_LBS_POOL

显示 LBS 控制结构的信息。

序号	列	数据类型	说明
1	N_CTL	INTEGER	LBS 控制块总数
2	N_FREE_CTL	INTEGER	空闲的 LBS 控制块数目
3	N_SUB_POOL	INTEGER	LBS_POOL 个数

103. V$RAC_LBS_POOLS_DETAIL

显示分片的 LBS_POOL 详细信息。多个 pool，依次扫描。

序号	列	数据类型	说明
1	POOL_ID	INTEGER	LBS_POOL 编号
2	N_USED_CTL	INTEGER	正在使用的 LBS 控制块数目
3	N_REQUEST	INTEGER	正在等待 LBS 控制块的请求数目
4	N_FREE_REQUEST	INTEGER	空闲的 LBS 请求控制块数目

104. V$RAC_NO_VIO_PAGE

显示不需要 VIO 的数据页信息。

序号	列	数据类型	说明
1	TS_ID	INTEGER	表空间号
2	FILE_ID	INTEGER	文件号
3	PAGE_NO	INTEGER	页号

105. V$RAC_LBS_CTL

显示 LBS 控制块信息。多个 pool，依次扫描。

序号	列	数据类型	说明
1	POOL_ID	INTEGER	LBS_POOL 编号

2	TS_ID	INTEGER	表空间号
3	FILE_ID	INTEGER	文件号
4	PAGE_NO	INTEGER	页号
5	ACCESS_MAP	INTEGER	曾经访问此数据页的 EP
6	FRESH_EP	INTEGER	最新数据所在 EP
7	FRESH_LSN	BIGINT	最新修改对应的 LSN 值
8	N_REPLACED	INTEGER	控制块被替换次数
9	N_REVOKED	INTEGER	权限被回收次数
10	N_FIXED	INTEGER	引用记数
11	MODE	INTEGER	获得 GBS 授权的封锁模式
12	PHY_LSN	BIGINT	数据页上的最新 LSN 值
13	N_REQUEST	INTEGER	请求获得授权的工作线程数

106. V$RAC_LBS_CTL_LRU_FIRST

显示 LBS 的 LRU_FIRST 控制块信息。多个 pool，依次扫描。

序号	列	数据类型	说明
1	POOL_ID	INTEGER	LBS_POOL 编号
2	TS_ID	INTEGER	表空间号
3	FILE_ID	INTEGER	文件号
4	PAGE_NO	INTEGER	页号
5	ACCESS_MAP	INTEGER	曾经访问此数据页的 EP
6	FRESH_EP	INTEGER	最新数据所在 EP
7	FRESH_LSN	BIGINT	最新修改对应的 LSN 值
8	N_REPLACED	INTEGER	控制块被替换次数
9	N_REVOKED	INTEGER	权限被回收次数
10	N_FIXED	INTEGER	引用记数
11	MODE	INTEGER	获得 GBS 授权的封锁模式
12	PHY_LSN	BIGINT	数据页上的最新 LSN 值
13	N_REQUEST	INTEGER	请求获得授权的工作线程数

107. V$RAC_LBS_CTL_LRU_LAST

显示 LBS 的 LRU_LAST 控制块信息。多个 pool，依次扫描。

序号	列	数据类型	说明
1	POOL_ID	INTEGER	LBS_POOL 编号
2	TS_ID	INTEGER	表空间号
3	FILE_ID	INTEGER	文件号
4	PAGE_NO	INTEGER	页号
5	ACCESS_MAP	INTEGER	曾经访问此数据页的 EP
6	FRESH_EP	INTEGER	最新数据所在 EP
7	FRESH_LSN	BIGINT	最新修改对应的 LSN 值
8	N_REPLACED	INTEGER	控制块被替换次数
9	N_REVOKED	INTEGER	权限被回收次数
10	N_FIXED	INTEGER	引用记数
11	MODE	INTEGER	获得 GBS 授权的封锁模式
12	PHY_LSN	BIGINT	数据页上的最新 LSN 值
13	N_REQUEST	INTEGER	请求获得授权的工作线程数

108. V$RAC_LBS_CTL_DETAIL

显示 LBS 控制块详细信息。多个 pool，依次扫描。

序号	列	数据类型	说明
1	POOL_ID	INTEGER	LBS_POOL 编号
2	TS_ID	INTEGER	表空间号
3	FILE_ID	INTEGER	文件号
4	PAGE_NO	INTEGER	页号
5	ACCESS_MAP	INTEGER	曾经访问此数据页的 EP
6	FRESH_EP	INTEGER	最新数据所在 EP
7	FRESH_LSN	BIGINT	最新修改对应的 LSN 值
8	N_REPLACED	INTEGER	控制块被替换次数
9	N_REVOKED	INTEGER	权限被回收次数
10	N_FIXED	INTEGER	引用记数
11	MODE	INTEGER	获得 GBS 授权的封锁模式
12	PHY_LSN	BIGINT	数据页上的最新 LSN 值
13	N_REQUEST	INTEGER	请求获得授权的工作线程数
14	REQUEST_MODE	INTEGER	请求的封锁模式
15	REVOKE_LSN	BIGINT	回收权限时，GBS 上的最新 LSN 值

109. V$RAC_LBS_CTL_LRU_FIRST_DETAIL

显示 LBS 的 LRU_FIRST 控制块详细信息。多个 pool，依次扫描。

序号	列	数据类型	说明
1	POOL_ID	INTEGER	LBS_POOL 编号
2	TS_ID	INTEGER	表空间号
3	FILE_ID	INTEGER	文件号
4	PAGE_NO	INTEGER	页号
5	ACCESS_MAP	INTEGER	曾经访问此数据页的 EP
6	FRESH_EP	INTEGER	最新数据所在 EP
7	FRESH_LSN	BIGINT	最新修改对应的 LSN 值
8	N_REPLACED	INTEGER	控制块被替换次数
9	N_REVOKED	INTEGER	权限被回收次数
10	N_FIXED	INTEGER	引用记数
11	MODE	INTEGER	获得 GBS 授权的封锁模式
12	PHY_LSN	BIGINT	数据页上的最新 LSN 值
13	N_REQUEST	INTEGER	请求获得授权的工作线程数
14	REQUEST_MODE	INTEGER	请求的封锁模式
15	REVOKE_LSN	BIGINT	回收权限时，GBS 上的最新 LSN 值

110. V$RAC_LBS_CTL_LRU_LAST_DETAIL

显示 LBS 的 LRU_LAST 控制块详细信息。多个 pool，依次扫描。

序号	列	数据类型	说明
1	POOL_ID	INTEGER	LBS_POOL 编号
2	TS_ID	INTEGER	表空间号
3	FILE_ID	INTEGER	文件号
4	PAGE_NO	INTEGER	页号
5	ACCESS_MAP	INTEGER	曾经访问此数据页的 EP
6	FRESH_EP	INTEGER	最新数据所在 EP
7	FRESH_LSN	BIGINT	最新修改对应的 LSN 值

8	N_REPLACED	INTEGER	控制块被替换次数
9	N_REVOKED	INTEGER	权限被回收次数
10	N_FIXED	INTEGER	引用记数
11	MODE	INTEGER	获得 GBS 授权的封锁模式
12	PHY_LSN	BIGINT	数据页上的最新 LSN 值
13	N_REQUEST	INTEGER	请求获得授权的工作线程数
14	REQUEST_MODE	INTEGER	请求的封锁模式
15	REVOKE_LSN	BIGINT	回收权限时，GBS 上的最新 LSN 值

111. V$RAC_GTV_SYS

显示 GTV 控制结构的信息。

序号	列	数据类型	说明
1	T_INFO_NUM	INTEGER	系统已提交、未 PURGE 事务所修改的表对象个数
2	NEXT_TRXID	BIGINT	下一个事务 ID
3	MAX_PURGABLE_TRXID	BIGINT	最大可 PURGE 的事务 ID
4	UNDO_TRXID	BIGINT	回滚段中，正在被访问的最小事务 ID
5	UNDO_CNT	INTEGER	UNDO_TRXID 被设置的次数

112. V$RAC_GTV_TINFO

显示 TINFO 控制结构的信息。

序号	列	数据类型	说明
1	TABLE_ID	INTEGER	系统已提交、未 PURGE 事务所修改的表 ID
2	TRX_ID	BIGINT	修改操作的事务 ID
3	ROWCNT	BIGINT	修改操作影响的行数

113. V$RAC_GTV_ACTIVE_TRX

显示全局活动事务信息。

序号	列	数据类型	说明
1	EP_SEQNO	INTEGER	事务所在的站点号
2	TRX_ID	BIGINT	事务 ID
3	NEXT_TRXID	BIGINT	下一个事务 ID
4	MIN_ACTIVE_TRXID	BIGINT	活动事务链表中的最小事务 ID
5	REFED_TRXID	BIGINT	活动事务链表中的事务 ID

114. V$RAC_LOCK

显示全局活动的事务锁信息。

序号	列	数据类型	说明
1	EP_SEQNO	INTEGER	拥有该锁的站点号
2	ADDR	BIGINT	锁地址
3	TRX_ID	BIGINT	所属事务 ID
4	LTYPE	VARCHAR(10)	锁类型：TID 锁、对象锁
5	LMODE	CHAR(2)	锁模式：S 锁、X 锁、IX 锁、IS 锁
6	BLOCKED	INTEGER	是否阻塞
7	TABLE_ID	INTEGER	对应表锁、字典对象 ID
8	ROW_IDX	BIGINT	被封锁事务 ID

115. V$RAC_TRX

显示所有活动事务的信息。通过该视图可以查看所有系统中所有的事务以及相关信息，如锁信息等。

序号	列	数据类型	说明
1	EP_SEQNO	INTEGER	事务所在站点号
2	ID	BIGINT	当前活动事务的 ID 号
3	NEXTID	BIGINT	下一个事务 ID 号
4	MIN_ACTIVE_ID	BIGINT	所有活动事务 ID 号最小者
5	STATUS	VARCHAR(20)	当前事务的状态
6	ISOLATION	INTEGER	事务隔离级，0：读未提交；1：读提交；2：可重复读；3：串行化
7	READ_ONLY	CHAR	是否是一个只读事务
8	SESS_ID	BIGINT	当前事务的所在会话
9	SESS_SEQ	INTEGER	会话序列号，用来唯一标识会话
10	INS_CNT	INTEGER	插入回滚记录个数
11	DEL_CNT	INTEGER	删除回滚记录个数
12	UPD_CNT	INTEGER	更新回滚记录个数
13	UPD_INS_CNT	INTEGER	更新插入回滚记录个数
14	UREC_SEQNO	INTEGER	当前 Undo 记录的递增序列号
15	WAITING	BIGINT	事务等待的锁

116. V$RAC_TRXWAIT

显示事务等待信息。

序号	列	数据类型	说明
1	EP_SEQNO	INTEGER	事务所在站点号
2	ID	BIGINT	事务 ID
3	WAIT_FOR_ID	BIGINT	所等待的事务 ID
4	WAIT_SEQNO	INTEGER	等待的事务所在站点号
5	WAIT_TIME	INTEGER	当前已等待时间（s）

117. V$RAC_TRX_VIEW

显示当前事务可见的所有活动事务视图信息。根据达梦多版本规则，通过该视图可以查询系统中自己所见的事务信息；可以通过与 V$RAC_TRX 表的连接查询它所见事务的具体信息。

序号	列	数据类型	说明
1	SELF_ID	BIGINT	活动事务 ID
2	EP_SEQNO	INTEGER	可见事务所在站点号
3	ACTIVE_ID	BIGINT	所见的事务活动事务 ID

118. V$DB_CACHE

数据字典缓存表，用于记录数据字典的实时信息。

序号	列	数据类型	说明
1	DB_ADDR	VARBINARY(8)	数据字典地址
2	POOL_ID	INTEGER	缓存池 ID
3	TOTAL_SIZE	INTEGER	缓存池总空间（字节）
4	USED_SIZE	INTEGER	实际使用的空间（字节）
5	DICT_NUM	INTEGER	缓存池中字典对象的总数
6	SIZE_LRU_DISCARD	BIGINT	所有被淘汰字典对象空间的总和（字节）
7	LRU_DISCARD	INTEGER	字典对象被淘汰的次数
8	DDL_DISCARD	INTEGER	DDL 操作导致字典对象被淘汰的次数

119. V$DB_OBJECT_CACHE

数据字典对象缓存表，用于记录数据字典中每个对象的信息。

序号	列	数据类型	说明
1	TYPE	VARCHAR(32)	字典对象类型
2	ID	INTEGER	字典对象的 ID
3	NAME	VARCHAR(128)	字典对象的名称
4	SCHID	INTEGER	所在模式的 ID
5	PID	INTEGER	所属对象的 ID
6	STATUS	VARCHAR(16)	状态
7	T_SIZE	INTEGER	对象结构所占空间
8	R_SIZE	INTEGER	对象在缓存中实际的空间
9	VERSION	INTEGER	对象的版本号
10	ID_HASH	VARBINARY(8)	对象 ID 的 HASH 值
11	NAME_HASH	VARBINARY(8)	对象 NAME 的 HASH 值
12	LOAD_TIME	DATETIME	对象被加载的时间
13	N_DECODE	INTEGER	对象 DECODE 的次数
14	N_SEARCH	INTEGER	对象 SEARCH 的次数

120. V$OPEN_STMT

连接语句句柄表，用于记录 SESSION 上语句句柄的信息。

序号	列	数据类型	说明
1	SESS_ADDR	VARBINARY(8)	SESSION 地址
2	SESSION#	INTEGER	SESSION 的 ID
3	STMT_ID	INTEGER	STMT 的 ID
4	STMT_SEQNO	INTEGER	STMT 的执行序号
5	TYPE	VARCHAR(16)	语句类型
6	REP_TYPE	VARCHAR(16)	复制语句类型
7	EXEC_DIRECT	CHAR(1)	是否直接执行
8	PLN_ADDR	VARBINARY(8)	执行计划的地址
9	ROWCOUNT	BIGINT	影响记录的行数
10	ROWCOUNT_FOR_DML	BIGINT	DML 影响的行数
11	ROWCOUNT_REQUEST	BIGINT	请求的行数
12	BATCH_FLAG	CHAR(1)	批量处理标识
13	N_PARA_ROWS	INTEGER	多行参数时的行数
14	NTH_PARA_ROW	INTEGER	多行参数时的当前行序号
15	CUR_PARA_OFF	INTEGER	参数的当前偏移
16	PARA_BUF	VARBINARY(8)	参数的内存地址
17	SQL_STR	VARCHAR(1000)	执行语句的前 1000 个字符
18	BPARAM_CAN_OPT	CHAR(1)	绑定批量参数时是否可优化
19	CURSOR_FORWARD_ONLY	CHAR(1)	对应游标是否 FORWARD_ONLY
20	CURSOR_NAME	VARCHAR(128)	对应游标名
21	CURSOR_KEEP_NAME	CHAR(1)	对应游标是否保存名字
22	CURSOR_REF_ID	INTEGER	引用游标语句句柄 ID
23	CURSOR_REF_SEQNO	INTEGER	引用语句的执行序号
24	MPP_EXEC_ID	INTEGER	MPP 下的执行 ID
25	VM_ADDR	VARBINARY(8)	VM 地址

26	DBG_FLAG	CHAR(1)	是否 DEBUG
27	STMT_DBG	VARBINARY(8)	DEBUG 的语句句柄地址
28	MPP_DDL_FLAG	CHAR(1)	MPP 下 DDL 标识
29	REMOTE_OPERATION	CHAR(1)	是否是 DBLINK 的远程操作
30	RS_BDTA_FLAG	CHAR(1)	返回的结果集是否直接将 BDTA 打包
31	RS_BDTA_SIZE	INTEGER	返回的打包结果集时最大的行数
32	STMT_LINK	VARBINARY(8)	用于串连语句的链表地址
33	LPQ	VARBINARY(8)	并行查询地址
34	RS_INFO	VARBINARY(8)	结果集地址
35	MPLN_ADDR	VARBINARY(8)	从 EP 上保留 MPLN
36	BIND_PARAM	VARBINARY(8)	绑定参数地址
37	BAK_SYS	VARBINARY(8)	备份链表信息
38	RT_STACK	VARBINARY(8)	MPP 从 EP 的运行时信息堆栈

121. V$SQLTEXT

显示缓冲区中的 SQL 语句信息。

序号	列	数据类型	说明
1	SQL_ADDR	VARBINARY(8)	语句在缓存中的地址
2	SQL_ID	INTEGER	语句编号（唯一标识）
3	N_EXEC	INTEGER	语句执行次数
4	HASH_VALUE	INTEGER	语句 HASH 值
5	CMD_TYPE	VARCHAR(16)	语句类型（来自语句类型，例如 DRO_STMT_CTAB）
6	SQL_TEXT	VARCHAR(1000)	SQL 语句内容（如果超过 1 千字符，则分段存储）
7	SQL_NTH	INTEGER	SQL 语句段号（从 0 开始）
8	HASH	VARBINARY(8)	SQL 语句的哈希节点地址
9	LINK_ADDR	VARBINARY(8)	下一个 SQL 语句地址

122. V$SQL_PLAN

显示缓冲区中的执行计划信息。

序号	列	数据类型	说明
1	PLN_ADDR	VARBINARY(8)	计划在缓存中的地址
2	HASH_VALUE	INTEGER	计划 HASH 值
3	SQL_ID	INTEGER	语句编号（唯一标识）
4	PLN_TYPE	VARCHAR(16)	计划类型
5	SQLSTR	VARCHAR(1000)	语句内容
6	RT_METHOD	VARBINARY(8)	计划的运行时方法
7	SVPT	VARCHAR(128)	计划的保存点名称
8	N_LIT_PARAS	INTEGER	常量参数个数
9	N_CLNT_PARAS	INTEGER	客户端绑定参数个数
10	N_COLS	INTEGER	涉及到的列个数
11	SEL_UPDATABLE	CHAR(1)	是否为查询更新
12	N_NDCTS	INTEGER	涉及到的字典对象个数
13	N_SUBPLNS	INTEGER	子计划个数
14	N_SUBPGS	INTEGER	子过程个数
15	PRE_COMMIT	CHAR(1)	执行 DDL 之前先提交事物

16	IS_RECURSIVE	CHAR(1)	是否为递归调用
17	BPARAM_CAN_OPT	CHAR(1)	参数是否可以优化
18	NDCT_VERSION	INTEGER	字典对象版本号
19	CAN_REUSE	CHAR(1)	计划是否可重用
20	HAS_SQL	CHAR(1)	是否有对应的 SQL 语句
21	HASH	VARBINARY(8)	计划哈希节点地址
22	SCHID	INTEGER	模式 ID
23	USER_ID	INTEGER	用户 ID
24	OBJ_ID	INTEGER	对象 ID
25	RS_CAN_CACHE	CHAR(1)	结果集是否缓存
26	RS_CAN_CLT_CACHE	CHAR(1)	客户端是否缓存结果集
27	RS_MUTEX	VARBINARY(8)	互斥量地址
28	N_TABLES	INTEGER	涉及的表个数
29	LINK_ADDR	VARBINARY(8)	下一个计划地址
30	PHD_TIME	DATETIME	生成计划时间
31	OPTIMIZER	VARCHAR(128)	优化方式

123. V$SQL_PLAN_NODE

当 INI 参数 ENABLE_MONITOR 为 3 时，显示执行计划的节点信息。

序号	列	数据类型	说明
1	PLN_ADDR	VARBINARY(8)	计划地址
2	SQL_ID	INTEGER	语句编号(唯一标识)
3	NODE_ADDR	VARBINARY(8)	节点地址
4	PARENT	VARBINARY(8)	父节点地址
5	LEFT_CHILD	VARBINARY(8)	左孩子地址
6	RIGHT_CHILD	VARBINARY(8)	右孩子地址
7	HAS_INVOKE	CHAR(1)	是否包含函数
8	OPERATION	VARCHAR(30)	操作符名称
9	OPTIONS	VARCHAR(30)	扫描类型
10	TABLE#	INTEGER	表 ID
11	TABLE_NAME	VARCHAR(128)	表名
12	TABLE_ALIAS	VARCHAR(128)	表别名
13	INDEX_NAME	VARCHAR(128)	索引名称
14	SCHID	INTEGER	模式 ID
15	DEPTH	INTEGER	节点层次
16	CARD	BIGINT	行数
17	BYTES	BIGINT	字节数
18	COST	BIGINT	代价
19	CPU_COST	BIGINT	CPU 代价
20	IO_COST	BIGINT	IO 代价
21	SCAN_RANGE	VARCHAR(128)	扫描范围
22	FILTER	VARCHAR(1000)	过滤表达式
23	JOIN_COND	VARCHAR(1000)	连接表达式

124. V$SQL_SUBPLAN

显示子计划信息。

序号	列	数据类型	说明

1	PLN_ADDR	VARBINARY(8)	计划地址
2	SQL_ID	INTEGER	语句编号（唯一标识）
3	SUBPLN	VARBINARY(8)	子计划地址（SUBPLN 在 V$SQL_PLAN_DCTREF 中即为 PLN_ADDR）

125. V$SQL_PLAN_DCTREF

显示所有执行计划相关的详细字典对象信息。

序号	列	数据类型	说明
1	PLN_ADDR	VARBINARY(8)	计划地址
2	SQL_ID	INTEGER	语句编号(唯一标识)
3	TYPE	VARCHAR(16)	字典对象类型（DATABASE、TABLE、VIEW、INDEX、USER、ROLE、PROCEDURE、TRIGGER、CONSTRAINT、SCHEMA、SEQUENCE、LOGIN、DBLINK、SYSROLE、PACKAGE、OBJECT、SYNOM、CRYPTION、CIPHER、IDENTITY、SYS PRIVILEGE、OBJ PRIVILEGE、POLICY、RULE、COLUMN、DOMAIN、CHARSET、COLLATION、CONTEXT INDEX、USER LOGIN、REGEXP REWRITE、NORMAL REWRITE、UNKNOWN）
4	STATUS	VARCHAR(16)	字典对象状态。共四种状态（NORMAL、CREATING、ALTERING、DROPPING），本视图中的执行计划只涉及 DML 语句的 NORMAL 状态
5	R_SIZE	INTEGER	字典对象大小（字节）
6	VERSION	INTEGER	字典对象版本号
7	NAME	VARCHAR(128)	字典对象名称
8	SCHID	INTEGER	模式 ID
9	ID	INTEGER	字典对象 ID
10	UID	INTEGER	用户 ID
11	TS_ID	INTEGER	表空间 ID

126. V$EVENT_NAME

显示当前系统所支持的等待事件的类型汇总信息。

序号	列	数据类型	说明
1	EVENT#	SMALLINT	事件编号
2	EVENT	VARCHAR(32)	事件名称。dbfile read：读用户数据文件；dbfile multi read：批量读用户数据文件；dbfile read wait：写用户数据文件；dbfile remote read：rac 引起的读磁盘；instance recovery read：recorvery 引起的读磁盘；dbfile remote write：rac 引起的写磁盘；dbfile discard write：buf 不够引起的写磁盘；dbfile ckpt write：ckpt 引起的写磁盘；dbfile extend：扩库文件引起的写磁盘；shared memory pool busy：共享内存并发使用冲突；buffer busy wait：buffer 并发使用冲突；dict cache busy：字典 cache 并发使用冲突；plan cache busy：计划 cache 并发使用冲突；redo log system

			busy：redo log 系统并发冲突；redo log buffer busy：redo log 缓冲区并发冲突；kernel busy：kernel 并发冲突；table lock busy：表锁系统并发冲突；tid lock busy：行锁系统并发冲突；parallel bdta pool busy：并行 bdta pool 并发冲突；iid system busy：iid 系统并发冲突；session system busy：session 系统并发冲突；public vpool busy：公共 vpool 并发冲突；pseg queue busy：pseg 队列并发冲突；pseg stack busy：pseg 堆栈并发冲突；page busy wait：数据页并发冲突；redo log flush wait：事务建 redo log flush 等待；table lock wait：事务间表锁等待发生；trxid lock wait：事务间行锁等待发生；dead lock：事务间产生死锁；transaction system busy：事务系统并发冲突；transaction view busy：事务可见性视图并发冲突；purge system busy：purge 系统并发冲突；file system busy：文件系统并发冲突；out of share mem pool：共享内存池不足；out of share coldata pool：共享 coldata 池不足
3	WAIT_CLASS#	INTEGER	事件类别编号：0、1、2、3、4。分别和 WAIT_CLASS 中的等待事件类别 User I/O、System I/O 、Concurrency、Transaction、Memory 一一对应
4	WAIT_CLASS	VARCHAR(32)	事件类别名称：User I/O、System I/O 、Concurrency、Transaction、Memory

127. V$SYSTEM_EVENT

显示自系统启动以来所有等待事件的详细信息。

序号	列	数据类型	说明
1	EVENT#	INTEGER	事件编号
2	EVENT	VARCHAR(32)	事件名称。
3	TOTAL_WAITS	INTEGER	等待次数
4	TIME_WAITED	INTEGER	等待时间（单位：1/100 秒）
5	TIME_WAITED _MICRO	BIGINT	等待时间（单位：微秒）
6	AVERAGE_WAIT _MICRO	INTEGER	平均等待时间（单位：微秒）
7	SMAX_TIME	INTEGER	最长等待时间（单位：微秒）
8	SMIN_TIME	INTEGER	最短等待时间（单位：微秒）
9	WAIT_CLASS#	INTEGER	等待事件类别编号
10	WAIT_CLASS	VARCHAR(32)	等待事件类别名称

128. V$SESSION_EVENT

显示当前会话等待事件的所有信息。

序号	列	数据类型	说明
1	SESSADDR	VARBINARY(8)	会话地址
2	SESSION#	INTEGER	会话 ID
3	EVENT#	INTEGER	事件编号

4	EVENT	VARCHAR(32)	事件名称
5	TOTAL_WAITS	INTEGER	等待次数
6	TIME_WAITED	INTEGER	等待时间（单位：1/100 秒）
7	TIME_WAITED_MICRO	BIGINT	等待时间（单位：微秒）
8	AVERAGE_WAIT_MICRO	INTEGER	平均等待时间（单位：微秒）
9	SMAX_TIME	INTEGER	最长等待时间（单位：微秒）
10	SMIN_TIME	INTEGER	最短等待时间（单位：微秒）
11	WAIT_CLASS#	INTEGER	等待事件类别编号
12	WAIT_CLASS	VARCHAR(32)	等待事件类别名称

129. V$SESSION_WAIT_HISTORY

显示会话等待事件的历史信息。

序号	列	数据类型	说明
1	SESSADDR	VARBINARY(8)	会话地址
2	SESSION#	INTEGER	会话 ID
3	SQL_ID	INTEGER	语句的 SQL ID
4	EVENT#	INTEGER	事件编号
5	EVENT	VARCHAR(32)	事件名称
6	P1TEXT	VARCHAR(32)	等待事件对应的参数 1 说明(如果没有足够参数，则为 NULL，以下参数一样)
7	P1	VARCHAR(256)	等待事件对应的参数 1 值
8	P2TEXT	VARCHAR(32)	等待事件对应的参数 2 说明
9	P2	INTEGER	等待事件对应的参数 2 值
10	P3TEXT	VARCHAR(32)	等待事件对应的参数 3 说明
11	P3	INTEGER	等待事件对应的参数 3 值
12	P4TEXT	VARCHAR(32)	等待事件对应的参数 4 说明
13	P4	INTEGER	等待事件对应的参数 4 值
14	TIME_WAITED	INTEGER	等待时间（单位：1/100 秒）
15	TIME_WAITED_MICRO	INTEGER	等待时间（单位：微秒）
16	WAIT_CLASS#	INTEGER	等待事件类别编号
17	WAIT_CLASS	VARCHAR(32)	等待事件类别名称

130. V$LONG_EXEC_SQLS

当 INI 参数 ENABLE_MONITOR 大于 1 时，显示系统最近 1000 条执行时间超过预定值的 SQL 语句。默认预订值为 1000 毫秒。可通过 SP_SET_LONG_TIME 系统函数修改，通过 SF_GET_LONG_TIME 系统函数查看当前值。

序号	列	数据类型	说明
1	SESS_ID	BIGINT	SESSION 的 ID
2	SQL_ID	INTEGER	语句的 SQL ID
3	SQL_TEXT	VARCHAR(1024)	SQL 文本
4	EXEC_TIME	BIGINT	执行时间，单位毫秒
5	FINISH_TIME	TIMESTAMP(0)	执行结束时间
6	N_RUNS	INTEGER	执行次数
7	SEQNO	INTEGER	编号
8	TRX_ID	BIGINT	事务号
9	SESS_SEQ	INTEGER	会话序列号，用来唯一标识会话

131. V$SYSTEM_LONG_EXEC_SQLS

当 INI 参数 ENABLE_MONITOR 大于 1 时，显示系统自启动以来执行时间最长的 20 条 SQL 语句，不包括执行时间低于预定值的语句。

序号	列	数据类型	说明
1	SESS_ID	BIGINT	SESSION 的 ID
2	SQL_ID	INTEGER	语句的 SQL ID
3	SQL_TEXT	VARCHAR(1024)	SQL 文本
4	EXEC_TIME	BIGINT	执行时间，单位毫秒
5	FINISH_TIME	DATETIME(0)	执行结束时间
6	N_RUNS	INTEGER	执行次数
7	SEQNO	INTEGER	编号
8	TRX_ID	BIGINT	事务号
9	SESS_SEQ	INTEGER	会话序列号，用来唯一标识会话

132. V$MTAB_USED_HISTORY

当 INI 参数 ENABLE_MONITOR 大于 1 时，显示系统自启动以来使用 MTAB 空间最多的 50 个操作符信息。

序号	列	数据类型	说明
1	EXEC_ID	INTEGER	语句的执行 ID
2	SQL_TEXT	VARCHAR(1024)	SQL 文本
3	STK_LEVEL	INTEGER	当前栈帧的级别
4	CUR_SQL_NODE	BIGINT	当前的 SQL 节点
5	OP_SEQ_NO	INTEGER	操作符在执行计划中序号
6	OP_TYPE$	INTEGER	操作符节点类型
7	MTAB_USED_BY_M	INTEGER	操作符使用的 MTAB 空间，以 M 为单位

133. V$SORT_HISTORY

当 INI 参数 ENABLE_MONITOR 大于 1 时，显示系统自启动以来使用排序页数最多的 50 个操作符信息。

序号	列	数据类型	说明
1	EXEC_ID	INTEGER	语句的执行 ID
2	SQL_TEXT	VARCHAR(1024)	SQL 文本
3	STK_LEVEL	INTEGER	当前栈帧的级别
4	CUR_SQL_NODE	BIGINT	当前的 SQL 节点
5	OP_SEQ_NO	INTEGER	操作符在执行计划中序号
6	OP_TYPE$	INTEGER	操作符节点类型
7	N_PAGES	INTEGER	排序使用的页数
8	N_MERGE_SORT	INTEGER	归并排序的趟数
9	SORT_BYTES	INTEGER	参与排序的每行字节数（估算值）
10	SORT_ROWS	BIGINT	参与排序的行数

134. V$SEQCACHE

显式当前系统中缓存的序列的信息。

序号	列	数据类型	说明
1	ID	INTEGER	序列 ID
2	NAME	VARCHAR(128)	序列名
3	NEXTVAL	BIGINT	序列的值

4	CACHE_NUM	INTEGER	创建序列时指定的缓存值的个数
5	N_USED	INTEGER	已经使用的缓存值的个数
6	OVERFLOW	INTEGER	当前序列是否已溢出的标识

135. V$EXTERNAL_CIPHERS

显式系统中所有的第三方加密算法信息。

序号	列	数据类型	说明
1	ID	INTEGER	算法 ID
2	NAME	VARCHAR(128)	算法名
3	LIB	VARCHAR(300)	算法所在的 lib 库文件名
4	VALID	CHAR	算法是否有效。‘Y’：是；‘N’：否

136. V$IFUN

显示数据库提供的所有函数。

序号	列	数据类型	说明
1	NAME	VARCHAR(128)	函数名
2	ID	INTEGER	函数 ID
3	ARG_NUM	INTEGER	参数个数
4	HAS_RT_FACT	CHAR(1)	是否存在运行时因素，‘Y’：是；‘N’：否
5	IS_READONLY	CHAR(1)	是否只读
6	IS_MPP_BRO	CHAR(1)	MPP 下是否需要广播
7	IS_MPP_FORBIDEN	CHAR(1)	MPP 下是否禁用该函数
8	IS_MOUNT_ONLY	CHAR(1)	系统 MOUNT 状态下是否可以使用该函数
9	IS_INCLUDE_SQL	CHAR(1)	函数中是否会执行 SQL 语句
10	EXTRA_ATTR	VARCHAR(1024)	额外的属性信息
11	CLASS$	VARCHAR(128)	函数所属的类别
12	COMMENT$	VARCHAR(1024)	函数说明

137. V$IFUN_ARG

显示数据库提供的所有函数的参数。

序号	列	数据类型	说明
1	ID	INTEGER	函数 ID
2	NAME	VARCHAR(128)	函数参数名
3	SEQ	INTEGER	参数序号
4	DATA_TYPE	VARCHAR(64)	参数数据类型
5	LEN	INTEGER	参数数据长度
6	PREC	INTEGER	数据数据精度
7	IO_TYPE	VARCHAR(8)	参数输入输出类型
8	COMMENT$	VARCHAR(1024)	参数说明

138. V$CACHEPKG

显示当前系统中的包的使用信息。

序号	列	数据类型	说明
1	NAME	VARCHAR(128)	包名
2	N_FIXED	INTEGER	包被引用的次数
3	DISABLED	CHAR	包是否可用
4	MEM_SIZE	INTEGER	包的内存大小
5	ADDRESS	VARBINARY(8)	包的地址

139. V$DBMS_LOCKS

显示当前系统中的申请的 DBMS_LOCK 包封锁情况。

序号	列	数据类型	说明
1	HANDLE	VARCHAR(128)	封锁的锁对象句柄
2	ID	INTEGER	封锁的锁对象 ID
3	MODE	VARCHAR(10)	封锁的模式
4	OWN_SESS	BIGINT	申请封锁的 session
5	WAIT_SESS	BIGINT	封锁等待的 session，无等待为 NULL

140. V$ARCHIVED_LOG

显式当前实例的所有归档日志文件信息。此动态视图与 Oracle 兼容，下表中未列出的列 DM 暂不支持，查询时均显示 NULL。

序号	列	数据类型	说明
1	NAME	VARCHAR(513)	归档日志文件名
2	THEAD#	BIGINT	默认为 0
3	SEQUENCE#	INTEGER	日志文件的序号
4	FIRST_CHANGE#	BIGINT	日志文件所记录日志的最小 LSN 值
5	NEXT_CHANGE#	BIGINT	日志文件所记录日志的最大 LSN 值
6	FIRST_TIME	DATETIME	日志文件所记录的日志的起始时间
7	NEXT_TIME	DATETIME	日志文件所记录的日志的截止时间
8	ARCHIVED	VARCHAR(3)	默认为 ARCHIVED
9	DELETED	VARCHAR(3)	默认为 NO
10	STATUS	VARCHAR(1)	默认为 A
11	IS_RECOVERY_DEST_FILE	VARCHAR(3)	默认为 NO

141. V$LOGMNR_LOGS

显式当前会话添加的需要分析的归档日志文件。此动态视图与 Oracle 兼容，下表中未列出的列 DM 暂不支持，查询时均显示 NULL。

序号	列	数据类型	说明
1	LOG_ID	INTEGER	日志文件 ID 号
2	FILENAME	VARCHAR(512)	日志文件 ID 名
3	LOW_TIME	DATETIME	日志文件创建时间
4	HIGH_TIME	DATETIME	日志文件最后修改时间
5	DB_ID	INTEGER	日志文件的实例 ID 号
6	DB_NAME	VARCHAR(8)	日志文件的实例名
7	THREAD_ID	BIGINT	默认为 0
8	THREAD_SQN	INTEGER	日志文件的序号
9	LOW_SCN	BIGINT	日志文件最小 LSN
10	NEXT_SCN	BIGINT	日志文件最大 LSN
11	DICTIONARY_BEGIN	VARCHAR(3)	默认为 NO
12	DICTIONARY_END	VARCHAR(3)	默认为 NO
13	TYPE	VARCHAR(8)	默认为 ARCHIVED
14	BLOCKSIZE	INTEGER	默认为 512
15	FILESIZE	BIGINT	日志文件的大小
16	STATUS	INTEGER	默认为 0

142. V$LOGMNR_PARAMETERS

显式当前会话 START_LOGMNR 启动日志文件分析的参数。此动态视图与 Oracle 兼容，下表中未列出的列 DM 暂不支持，查询时均显示 NULL。

序号	列	数据类型	说明
1	START_DATE	DATETIME	过滤分析归档日志的起始时间
2	END_DATE	DATETIME	过滤分析归档日志的截止时间
3	START_SCN	BIGINT	过滤分析归档日志的起始 LSN
4	END_SCN	BIGINT	过滤分析归档日志的截止 LSN
5	OPTIONS	INTEGER	分析归档日志的选项

143. V$LOGMNR_CONTENTS

显式当前会话日志分析的内容。此动态视图与 Oracle 兼容，下表中未列出的列 DM 暂不支持，查询时均显示 NULL。

序号	列	数据类型	说明
1	SCN	BIGINT	当前记录的 LSN
2	START_SCN	BIGINT	当前事务的起始 LSN
3	COMMIT_SCN	BIGINT	当前事务的截止 LSN
4	TIMESTAMP	DATETIME	当前记录的创建时间
5	START_TIMESTAMP	DATETIME	当前事务的起始时间
6	COMMIT_TIMESTAMP	DATETIME	当前事务的截止时间
7	XID	BINARY(8)	当前记录的事务 ID 号
8	OPERATION	VARCHAR(32)	操作类型包括 start、insert、update、delete、commit、rollback 等语句
9	OPERATION_CODE	INTEGER	操作类型。1 插入操作，2 删除操作，3 更新操作，4 批量更新，6 事务起始语句，7 提交操作，36 回滚操作
10	ROLL_BACK	INTEGER	当前记录是否被回滚，1:是，0：否
11	TABLE_NAME	VARCHAR(128)	操作的表名
12	ROW_ID	VARCHAR(20)	对应记录的行号
13	USERNAME	VARCHAR(128)	执行这条语句的用户名
14	RBASQN	INTEGER	对应的归档日志文件号
15	RBABLK	INTEGER	RBASQN 所指日志文件的块号，从 0 开始
16	RBABYTE	INTEGER	RBABLK 所指块号的块内偏移
17	DATA_OBJ#	INTEGER	对象 ID 号
18	DATA_OBJV#	INTEGER	对象版本号
19	SQL_REDO	VARCHAR(4000)	客户端发送给数据库的 SQL 语句。
20	SQL_UNDO	VARCHAR(4000)	暂不支持。
21	RS_ID	VARCHAR(32)	记录集 ID
22	SSN	INTEGER	连续 SQL 标志。如果 SQL 长度超过单个 sql_redo 字段能存储的长度，则 SQL 会被截断成多个 SQL 片段在结果集中“连续”返回。
23	CSF	INTEGER	与 SSN 配合，最后一个片段的 csf 值为 0，其余片段的值均为 1。没因超长发生截断的 SQL 该字段值均为 0。

24	REDO_VALUE	BIGINT	用于数据挖掘新值
25	UNDO_VALUE	BIGINT	用于数据挖掘旧值
26	CSCN	BIGINT	与 COMMITSCN 一样，已过时

144. V$RESERVED_WORDS

保留字统计表，记录保留字的分类信息。

序号	列	数据类型	说明
1	KEYWORD	VARCHAR(30)	关键字名字
2	LENGTH	INTEGER	关键字长度
3	RESERVED	VARCHAR(1)	是否是保留字
4	RES_SQL	VARCHAR(1)	是否是 SQL 保留字，不能作 SQL 中的标识符
5	RES_PL	VARCHAR(1)	是否是 PLSQL 保留字，不能作 PLSQL 语句块中的标识符
6	RES_SCHEMA	VARCHAR(1)	是否是模式保留字，不能作模式标识符
7	RES_VARIABLE	VARCHAR(1)	是否是变量保留字，不能作变量标识符
8	RES_ALIAS	VARCHAR(1)	是否是别名保留字，不能作别名标识符

145. V$CONTEXT

显示当前会话所有上下文的名字空间、属性和值。

序号	列	数据类型	说明
1	NAMESPACE	VARCHAR(30)	上下文名字空间
2	ATTRIBUTE	VARCHAR(30)	名字空间属性
3	VALUE	VARCHAR(4000)	属性值

146. V$PKGPROCS

显示包中的方法信息。

序号	列	数据类型	说明
1	PKGID	INTEGER	包 ID
2	PKGNAME	VARCHAR(128)	包名
3	MTDID	INTEGER	方法 ID，每个包中方法从 1 开始编号
4	MTDNAME	VARCHAR(128)	方法名
5	IS_PROC	CHAR(1)	是否是过程，'Y'表示是过程，'N'表示是函数

147. V$PKGPROCPARAMS

显示包中方法的参数信息。

序号	列	数据类型	说明
1	NAME	VARCHAR(128)	参数名
2	PKGID	INTEGER	包 ID
3	MTDID	INTEGER	方法 ID
4	PARAMID	SMALLINT	参数 ID，每个方法的参数从 1 开始编号，0 表示返回值
5	TYPE$	VARCHAR(128)	参数类型
6	LENGTH$	INTEGER	参数长度
7	SCALE	SMALLINT	参数精度
8	NULLABLE	CHAR(1)	参数是否可为空
9	IOFLAG	CHAR(2)	参数输入输出标记，'I'表示输入，'O'表示输出，'IO'表示输入输出，'R'表示返回值
10	DEFVAL	VARCHAR(2000)	参数缺省值

148. V$RECV_ROLLBACK_TRX

显示数据库启动时回滚的所有事务信息。

序号	列	数据类型	说明
1	TRX_ID	BIGINT	事务号
2	BEGIN_LSN	BIGINT	事务启动逻辑日志的 LSN 值，只在 INI 参数开启逻辑日志情况下有效
3	BEGIN_TIME	TIMESTAMP	事务启动时间，只在 INI 参数开启逻辑日志情况下有效
4	N_UPAGES	INTEGER	回滚页总数
5	N_URECS	INTEGER	回滚记录总数

149. V$SEGMENT_INFOS

显示所有的段信息。

序号	列	数据类型	说明
1	TS_ID	INTEGER	表空间 ID
2	SEG_ID	INTEGER	段 ID
3	TYPE	VARCHAR(256)	段的类型（INNER 表示内节点段，LEAF 表示叶子节点段，LIST 表示堆表段和 BLOB 表示 BLOB 段）
4	OBJ_ID	INTEGER	若段为内节点段或叶子节点段，则为索引 ID；若为 BLOB 段，则为 BLOB 所在的表 ID
5	INODE_FILE_ID	INTEGER	段的 inode 所在的页的文件号
6	INODE_PAGE_NO	INTEGER	段的 inode 所在的页的页号
7	INODE_OFFSET	INTEGER	段的 inode 所在的页的页内的偏移
8	N_FULL_EXTENT	INTEGER	段中的满簇数
9	N_FREE_EXTENT	INTEGER	段中的空闲簇数
10	N_FRAG_EXTENT	INTEGER	段中的半满簇数
11	N_FRAG_PAGE	INTEGER	段中的半满簇中总页数

150. V$BTREE_INNER_PAGES/V$BTREE_LEAF_PAGES

索引的叶子段/内节点段的页信息视图。查询该视图时，一定要带 WHERE 条件，并且必须是等值条件。如：`select * from v$btree_leaf_pages where index_id = id;`

序号	列	数据类型	说明
1	INDEX_ID	INTEGER	索引 ID
2	LEVEL	INTEGER	页所在的层数
3	GROUP_ID	INTEGER	页的表空间号
4	FILE_ID	INTEGER	页的文件号
5	PAGE_NO	INTEGER	页的页号
6	PAGE_LSN	BIGINT	页的版本号
7	N_REC	INTEGER	页的记录数

注意：在 V$BTREE_INNER_PAGES 视图中，索引 BTREE 的控制页的 level 指定为-1。

151. V$BTREE_LIST_PAGES

LIST 索引的叶子段的页信息视图。查询该视图时，一定要带 WHERE 条件，并且必须是等值条件。如：`select * from v$btree_list_pages where index_id = id;`

序号	列	数据类型	说明
1	INDEX_ID	INTEGER	索引 ID

2	LIST_NO	INTEGER	页所在的链表编号
3	GROUP_ID	INTEGER	页的表空间号
4	FILE_ID	INTEGER	页的文件号
5	PAGE_NO	INTEGER	页的页号
6	PAGE_LSN	BIGINT	页的版本号
7	N_REC	INTEGER	页的记录数

152. V$TABLE_LOB_PAGES

表中的大字段的页信息视图。查询该视图时，一定要带 WHERE 条件，并且必须是等值条件。如：select * from v$table_lob_pages where table_id = *id*;

序号	列	数据类型	说明
1	TABLE_ID	INTEGER	表 ID
2	LOB_ID	BIGINT	大字段编号
3	TYPE	INTEGER	LOB 段页的类型
4	GROUP_ID	INTEGER	页的表空间号
5	FILE_ID	INTEGER	页的文件号
6	PAGE_NO	INTEGER	页的页号
7	PAGE_LSN	BIGINT	页的版本号

注意：在 TABLE_LOB_PAGES 视图中，LOB 段的首页输出时，LOB_ID 为-1。

153. V$SEGMENT_PAGES

段中数据页的信息视图。查询该视图时，一定要带 WHERE 条件，并且必须是等值条件。例如：select * from v$segment_pages where group_id=*tid* and seg_id = s*id*;

序号	列	数据类型	说明
1	GROUP_ID	INTEGER	表空间 ID
2	SEG_ID	INTEGER	段编号
3	FILE_ID	INTEGER	页的文件号
4	PAGE_NO	INTEGER	页的页号

154. V$DCPINSTS

仅当 INI 参数 ENABLE_DCP_MODE 为 1 时才能查询此动态视图，显示 DCP 对应 MPP 集群的所有节点信息。

序号	列	数据类型	说明
1	NAME	VARCHAR(128)	节点实例名称
2	EPNO	INTEGER	节点编号
3	IP	VARCHAR(100)	IP 地址
4	PORT	INTEGER	端口号

155. V$DCP_CONNPOOL

仅当 INI 参数 ENABLE_DCP_MODE 为 1 时才能查询此动态视图，显示 DCP 缓冲池的信息。

序号	列	数据类型	说明
1	ID	BIGINT	会话 ID
2	USERNAME	VARCHAR(128)	会话用户名
3	CUR_SCH	VARCHAR(128)	当前模式名
4	EPNO	INTEGER	当前连接的 MPP 的 EPNO
5	EXEC_MODE	INTEGER	当前事务执行模式，1：探测；2：强制

6	FREE_FLAG	CHAR	是否空闲，Y：是；N：否
7	RELEASE_TS	INTEGER	上一次放回缓冲池的时间戳，仅当FREE_FALG=Y时有效

156. V$LICENSE

显示LICENSE信息，用来查询当前系统的LICENSE信息。

序号	列	数据类型	说明
1	LIC_VERSION	VARCHAR(256)	许可证版本号
2	SERIES_NO	VARCHAR(256)	序列号
3	SERVER_SERIES	VARCHAR(256)	服务器颁布类型
4	SERVER_TYPE	VARCHAR(256)	服务器发布类型
5	SERVER_VER	VARCHAR(256)	服务器版本号
6	EXPIRY_DATE	DATE	有效日期
7	AUTHORIZED_CUSTOMER	VARCHAR(256)	用户名称
8	AUTHORIZED_USER_NUMBER	BIGINT	授权用户数
9	CONCURRENCY_USER_NUMBER	BIGINT	并发连接数
10	MAX_CPU_NUM	BIGINT	最大CPU数目
11	NOACTIVE_DEADLINE	DATE	未激活状态截止日期
12	HARDWARE_ID	VARCHAR(256)	硬件编码
13	CHECK_CODE	VARCHAR(16)	校验码
14	PRODUCT_TYPE	VARCHAR(8)	产品类型：内容为：DM7/DM6/DMETLV4/DMETLV3/DMHSV3，以后根据需要可增加
15	PROJECT_NAME	VARCHAR(128)	项目名称
16	CPU_TYPE	VARCHAR(24)	授权运行的CPU类型
17	OS_TYPE	VARCHAR(24)	授权运行的操作系统
18	MAX_CORE_NUM	INTEGER	授权最大CPU核个数
19	HARDWARE_TYPE	VARCHAR(24)	硬件绑定类型
20	CLUSTER_TYPE	VARCHAR(24)	授权使用的集群类型
21	DATE_GEN	DATE	KEY的生成日期

157. V$INSTANCE_LOG_HISTORY

用于查询服务器实例运行期间生成的最近1万条事件日志。

序号	参数	说明
1	SEQNO	日志序号，从0开始递增。每次服务器重启后清零
2	LOG_TIME	日志生成时间，精确到秒
3	PID	生成日志的进程ID
4	THREAD_NAME	生成日志的线程名
5	LEVEL$	日志级别描述。可取值为：INFO、WARN、ERROR、FATAL
6	TXT	日志详细内容

158. V$OBJECT_USAGE

记录索引监控信息。

序号	列	数据类型	说明
1	INDEX_NAME	VARCHAR(128)	索引名
2	SCH_NAME	VARCHAR(128)	模式名

3	TABLE_NAME	VARCHAR(128)	表名
4	MONITORING	VARCHAR(3)	是否被监控(YES\NO)
5	USED	VARCHAR(3)	是否被使用(YES\NO)
6	START_MONITORING	VARCHAR(19)	开始监控时间
7	END_MONITORING	VARCHAR(19)	停止监控时间

159. V$RECOVER_STATUS

该视图需要在主库上查询，用于查询备库的恢复进度，如果已恢复完成，查询结果为空。

注意这里显示的是主库向备库发送日志的进度，由于备库重做日志也需要时间，在最后一批日志发送完成后，KBYTES_TO_Recover 为 0，Recover_PERCENT 为 100%，表示主库已经完成所有日志发送，需要等待备库将最后一批日志重做完成，此时主库的守护进程可能仍然处于 Recovery 状态，待备库重做完成后，主库的守护进程会自动切换 Open 状态。

RECOVER_PERCENT = (KBYTES_TOTAL - KBYTES_TO_Recover)/ KBYTES_TOTAL

序号	列	数据类型	说明
1	PRIMARY_NAME	VARCHAR(256)	主库实例名
2	STANDBY_NAME	VARCHAR(256)	当前正在恢复的备库实例名
3	PRIM_LSN	BIGINT	主库当前的 FLSN
4	SEND_LSN	BIGINT	主库已发给备库的最大 LSN
5	KBYTES_TOTAL	BIGINT	主库本地的日志总量，单位为 KB
6	KBYTES_TO_RECOVER	BIGINT	主库需要发送给备库的日志量，单位为 KB
7	FILES_TO_RECOVER	INTEGER	主库需要发给备库重做的日志文件个数
8	RECOVER_PERCENT	VARCHAR(64)	主库向备库已发送日志的比例

160. V$KEEP_BUF

该视图需要在备库上查询，用于查询备库上的 KEEP_BUF 信息。

读写分离集群下备库没有 KEEP_BUF 机制，该视图查询结果为空。

序号	列	数据类型	说明
1	TYPE	VARCHAR(64)	KEEP_BUF 中保存的日志类型。
2	MIN_LSN	BIGINT	KEEP_BUF 中保存的最小 LSN
3	MAX_LSN	BIGINT	KEEP_BUF 中保存的最大 LSN
4	PTX_OFF	SMALLINT	KEEP_BUF 中的 PTX 偏移
5	LOG_SIZE	INTEGER	KEEP_BUF 中的日志长度，单位为字节

161. V$RAPPLY_SYS

该视图需要在备库上查询，用于查询备库重做日志时的一些系统信息。

序号	列	数据类型	说明
1	APPLYING	VARCHAR(64)	备库的重做线程是否正在处理任务，查询结果为 TRUE 或 FALSE
2	SEC_MAX_LSN	BIGINT	备库收到的第二大 LSN 值
3	MAX_LSN	BIGINT	备库收到的最大 LSN 值
4	TASK_NUM	INTEGER	备库上的重做任务数
5	Primary_NAME	VARCHAR(128)	备库对应的主库实例名
6	HAS_KEEP_BUF	VARCHAR(64)	备库上是否有 KEEP_BUF，查询结果为 TRUE 或 FALSE

162. V$RAPPLY_LOG_TASK

该视图需要在备库上查询，用于查询备库所有重做任务的日志信息。

序号	列	数据类型	说明
1	TYPE	VARCHAR(64)	日志类型
2	MIN_LSN	BIGINT	日志的最小 LSN 值
3	MAX_LSN	BIGINT	日志的最大 LSN 值
4	PTX_OFF	SMALLINT	重做日志的 PTX 偏移
5	LOG_SIZE	INTEGER	日志长度，单位为字节

163. V$ARCH_FILE

查询本地归档日志信息。

序号	列	数据类型	说明
1	DB_MAGIC	INTEGER	数据库的 MAGIC 值
2	STATUS	VARCHAR(64)	归档日志状态
3	LEN	BIGINT	日志长度，单位为字节
4	FREE	BIGINT	写日志偏移位置
5	ARCH_LSN	BIGINT	归档文件起始 LSN，仅对归档文件有意义
6	CLSN	BIGINT	检查点的 LSN
7	ARCH_SEQ	INTEGER	归档文件起始 SEQ，仅对归档文件有意义
8	NEXT_SEQ	INTEGER	下一个 SEQ 值
9	CREAT_TIME	TIMESTAMP	归档文件创建时间
10	CLOSE_TIME	TIMESTAMP	归档文件关闭时间
11	PATH	VARCHAR(256)	归档文件所在路径，包括归档文件名

164. V$ARCH_STATUS

查询归档状态信息。

序号	列	数据类型	说明
1	ARCH_TYPE	VARCHAR(256)	归档类型
2	ARCH_DEST	VARCHAR(256)	归档目标，本地归档为归档路径，其他类型为归档目标实例名。
3	ARCH_STATUS	VARCHAR(256)	归档状态，Valid 为有效状态，Invalid 为无效状态。

165. V$MAL_LINK_STATUS

查询本地实例到远程实例的 MAL 链路连接状态。

序号	列	数据类型	说明
1	SRC_SITE	VARCHAR(256)	源站点实例名
2	DEST_SITE	VARCHAR(256)	目的站点实例名。
3	CTL_LINK_STATUS	VARCHAR(256)	控制链路连接状态，CONNECTED 表示连接已建立，DISCONNECT 表示连接断开。
4	DATA_LINK_STATUS	VARCHAR(256)	数据链路连接状态，CONNECTED 表示连接已建立，DISCONNECT 表示连接断开。

166. V$DMWATCHER

守护本实例的守护进程信息。

序号	列	数据类型	说明
1	N_GROUP	INTEGER	守护进程守护的实例个数
2	GROUP_NAME	VARCHAR(128)	本实例所在的组名
3	DW_TYPE	VARCHAR(32)	本组守护类型
4	DW_MODE	VARCHAR(32)	本组守护模式

5	AUTO_RESTART	INTEGER	本组是否自动重启
6	DW_STATUS	VARCHAR(64)	本组守护进程状态
7	DW_SUB_STATUS	VARCHAR(64)	本组守护进程子状态
8	LAST_MSG_TIME	DATETIME	最近收到守护进程消息的时间

167. V$DB_PIPES

记录使用 DBMS_PIPE 包创建的管道的相关信息。

列名	类型	说明
NAME	VARCHAR(30) NOT NULL	管道的名字
OWNERID	BIGINT NOT NULL	管道拥有者 ID
TYPE	VARCHAR(7)	管道类型:PUBLIC/PRIVATE
PIPE_SIZE	BIGINT	管道的长度

附录 3 ORACLE 兼容视图

为了提高 DM 与 ORACLE 的兼容性，DM 提供了如下一批视图。

1. DBA_ROLES

显示系统中所有的角色。

序号	列	数据类型	说明
1	ROLE	VARCHAR(128)	角色名
2	PASSWORD_REQUIRED	VARCHAR(1)	是否需要 ROLE 密码。 DM 不支持，值为 NULL
3	AUTHENTICATION_TYPE	VARCHAR(1)	角色的验证机制。 DM7 只支持 CREATE ROLE ROLE1 的方式，取值为 NULL。

2. DBA_TAB_PRIVS

显示系统中所有表的权限。

序号	列	数据类型	说明
1	GRANTEE	VARCHAR(128)	被授权用户名
2	OWNER	VARCHAR(128)	表的拥有者
3	TABLE_NAME	VARCHAR(128)	表名
4	GRANTOR	VARCHAR(128)	授权者
5	PRIVILEGE	VARCHAR(32767)	权限名称
6	GRANTABLE	VARCHAR(3)	权限是否可转授。Y 是，N 否
7	HIERARCHY	VARCHAR(2)	表示权限是否以有 HIERARCHY OPTION 的方式授予。DM 机制不支持的功能项，取值为 NO

3. USER_TAB_PRIVS

显示当前用户作为对象拥有者、授权者或被授权者的数据库表权限。结构同 DBA_TAB_PRIVS。

4. ALL_TAB_PRIVS

显示当前用户可见的，数据库表的权限。结构同 DBA_TAB_PRIVS。

5. DBA_SYS_PRIVS

显示系统中所有传授给用户和角色的权限。

序号	列	数据类型	说明
1	GRANTEE	VARCHAR(128)	被授权用户名
2	PRIVILEGE	VARCHAR(32767)	权限名称
3	ADMIN_OPTION	VARCHAR(3)	是否可以转授。YES 可转授，NO 不可转授

6. USER_SYS_PRIVS

传授给当前用户的系统权限。

序号	列	数据类型	说明
1	USERNAME	VARCHAR(128)	用户名
2	PRIVILEGE	VARCHAR(32767)	权限名称
3	ADMIN_OPTION	VARCHAR(3)	是否可转授。YES 可转授，NO 不可转授

7. DBA_USERS

显示系统中所有的用户。

序号	列	数据类型	说明
1	USERNAME	VARCHAR(128)	用户名
2	USER_ID	INT	用户 ID
3	PASSWORD	VARCHAR(128)	密码
4	ACCOUNT_STATUS	VARCHAR(24)	账号状态，有如下状态 OPEN EXRIRYED(GRACE) & LOCKED EXRIRYED & LOCKED LOCKED LOCKED EXPIRED EXPIRED(GRACE)
5	LOCK_DATE	DATETIME(0)	如果用户被锁定，锁定开始的时间
6	EXPIRY_DATE	DATETIME(6)	密码有效期至
7	DEFAULT_TABLESPACE	VARCHAR(128)	默认表空间
8	TEMPORARY_TABLESPACE	VARCHAR(4)	默认临时表空间，DM7 为“TEMP”
9	CREATED	DATETIME(6)	创建时间
10	PROFILE	VARCHAR(32767)	表空间所在路径
11	INITIAL_RSRC_CONSUMER_GROUP	VARCHAR(1)	初始资源所在组
12	EXTERNAL_NAME	VARCHAR(1)	用户外部名称，目前为 NULL
13	PASSWORD_VERSIONS	VARCHAR(20)	密码版本
14	EDITIONS_ENABLED	VARCHAR(1)	是否只读。Y 是，N 否。
15	AUTHENTICATION_TYPE	VARCHAR(19)	用户登陆验证类型
16	NOWDATE	DATETIME(6)	当前日期时刻

8. ALL_USERS

当前用户可见的所用用户。ALL_USERS 的信息均来自 DBA_USERS。

序号	列	数据类型	说明
1	USERNAME	VARCHAR(128)	用户名
2	USER_ID	INT	用户 ID
3	CREATED	DATETIME(6)	创建时间

9. USER_USERS

当前用户。

序号	列	数据类型	说明
1	USERNAME	VARCHAR(128)	用户名
2	USER_ID	INT	用户 ID
3	ACCOUNT_STATUS	VARCHAR(24)	账号状态，有如下状态 OPEN EXRIRYED(GRACE) & LOCKED EXRIRYED & LOCKED LOCKED LOCKED EXPIRED EXPIRED(GRACE)
4	LOCK_DATE	DATETIME(0)	如果为 LOCK 状态，LOCK 时间段内

			该值为 NULL
5	EXPIRY_DATE	DATETIME(6)	密码失效时间。目前为 NULL
6	DEFAULT_TABLESPACE	VARCHAR(1)	默认表空间
7	TEMPORARY_TABLESPACE	VARCHAR(1)	默认临时表空间，DM7 为“TEMP”
8	CREATED	VARCHAR(1)	创建时间
9	INITIAL_RSRC_CONSUMER_GROUP	VARCHAR(1)	初始资源所在组。该视图中为 NULL
10	EXTERNAL_NAME	VARCHAR(1)	用户外部名称。该视图中为 NULL

10. DBA_ROLE_PRIVS

系统中的所有角色权限。

序号	列	数据类型	说明
1	GRANTEE	VARCHAR(128)	被授权用户名
2	GRANTED_ROLE	VARCHAR(128)	被授予的角色
3	ADMIN_OPTION	VARCHAR(3)	是否可以转授
4	DEFAULT_ROLE	VARCHAR(1)	是否为默认角色，Y 是，N 否。目前都为 NULL

11. USER_ROLE_PRIVS

传授给当前用户的角色。

序号	列	数据类型	说明
1	USERNAME	VARCHAR(128)	被授权用户名
2	GRANTED_ROLE	VARCHAR(128)	被授予的角色
3	ADMIN_OPTION	VARCHAR(3)	是否可以转授，Y 是，N 否
4	DEFAULT_ROLE	VARCHAR(1)	是否为默认角色。DM 均为 NULL
5	OS_GRANTED	VARCHAR(1)	是否由操作系统授权，DM 均为 NULL

12. ALL_CONSTRAINTS

当前用户拥有的所有约束信息。

序号	列	数据类型	说明
1	OWNER	VARCHAR(128)	约束拥有者
2	CONSTRAINT_NAME	VARCHAR(128)	约束名
3	CONSTRAINT_TYPE	VARCHAR(1)	约束类型： C – 表上的检查约束；P – 主键约束；U – 唯一约束；R – 引用列；V – 视图上的检查约束
4	TABLE_NAME	VARCHAR(128)	约束所在的表名或视图名
5	SEARCH_CONDITION	VARCHAR(2048)	CHECK 约束的条件
6	R_OWNER	VARCHAR(128)	引用约束所引用的表的拥有者
7	R_CONSTRAINT_NAME	VARCHAR(128)	引用约束所引用的表上的唯一约束的名称
8	DELETE_RULE	VARCHAR(9)	引用约束的删除规则，DM 支持如下规则 CASCADE SET NULL SET DEFAULT NO ACTION
9	STATUS	VARCHAR(8)	约束的状态：ENABLED 可用，DISABLED 不可用
10	DEFERRABLE	VARCHAR(14)	约束是否延迟生效。DEFERRABLE 延迟生效， NOT DEFERRABLE 不延迟生效

11	DEFERRED	VARCHAR(9)	约束是否初始为延迟生效。 DEFERRED 生效，IMMEDIATE 不生效
12	VALIDATED	VARCHAR(13)	是否所有数据都符合约束。 VALIDATED 符合，NOT VALIDATED 不符合
13	GENERATED	VARCHAR(14)	约束名是用户指定的(USER NAME) 还是系统指定的(GENERATED NAME)。DM 均为 NULL
14	BAD	VARCHAR(3)	约束所指定的时间格式（TO_DATE）是否是可以引起歧义的。取值如下 BAD --会引起歧义 NULL --不会引起歧义 DM 均为 NULL
15	RELY	VARCHAR(4)	是否约束是强制性的。RELY 强制，NULL 非强制 DM 均为 NULL
16	LAST_CHANGE	DATETIME(6)	约束的最后修改时间
17	INDEX_OWNER	VARCHAR(128)	索引的所有者
18	INDEX_NAME	VARCHAR(128)	索引名，只对唯一约束和主键约束有效
19	INVALID	VARCHAR(7)	约束是否是无效的，取值如下： INVALID --无效的 NULL --有效的
20	VIEW_RELATED	VARCHAR(14)	约束依赖视图，取值如下： DEPEND ON VIEW--依赖于视图 NULL--不依赖视图 DM 均为 NULL

13. DBA_CONSTRAINTS

系统中所有的约束信息，结构同 ALL_CONSTRAINTS。

14. USER_CONSTRAINTS

当前用户所拥有的表上定义的约束，结构同 ALL_CONSTRAINTS。

15. DBA_TABLES

用户能够看到的所有表。

序号	列	数据类型	说明
1	OWNER	VARCHAR(128)	表拥有者
2	TABLE_NAME	VARCHAR(128)	表名
3	TABLESPACE_NAME	VARCHAR(128)	表所在表空间名。 对于分区表，临时表和索引组织表为 NULL
4	CLUSTER_NAME	VARCHAR(1)	聚簇名
5	IOT_NAME	VARCHAR(128)	索引组织表名
6	STATUS	VARCHAR(8)	表的状态。UNUSABLE无效，VALID 有效
7	PCT_FREE	NUMBER	块的最小空闲百分比，分区表为 NULL
8	PCT_USED	NUMBER	块的最小使用百分比，分区表为 NULL
9	INI_TRANS	NUMBER	初始事务数，分区表为 NULL
10	MAX_TRANS	NUMBER	最大事务数，分区表为 NULL
11	INITIAL_EXTENT	NUMBER	初始簇大小，单位：字节。分区表为 NULL
12	NEXT_EXTENT	NUMBER	下一个簇的大小，单位：字节。分区表为 NULL
13	MIN_EXTENTS	NUMBER	最小簇大小，分区表为 NULL

14	MAX_EXTENTS	NUMBER	最大簇大小，分区表为 NULL
15	PCT_INCREASE	NUMBER	簇的增长百分比，分区表为 NULL
16	FREELISTS	NUMBER	空闲链的分配个数，分区表为 NULL
17	FREELIST_GROUPS	NUMBER	空闲连组的分配个数，分区表为 NULL。DM 不支持，返回 NULL
18	LOGGING	VARCHAR(1)	表的变化是否需要记录日志，Y 是，N 否。 分区表为 NULL。DM 不支持，返回 NULL
19	BACKED_UP	VARCHAR(1)	最后一次修改后表是否已备份，Y 是，N 否
20	NUM_ROWS	NUMBER	表里的记录数
21	BLOCKS	NUMBER	表中已使用块的数目
22	EMPTY_BLOCKS	NUMBER	表中未使用的块的数目
23	AVG_SPACE	NUMBER	每个数据块的平均空闲空间，单位：字节
24	CHAIN_CNT	NUMBER	表中已被链接数据的行数
25	AVG_ROW_LEN	NUMBER	表中每行数据的平均长度。DM 不支持，返回 NULL
26	AVG_SPACE_FREELIST_BLOCKS	NUMBER	一条链上所有块的平均空闲空间，兼容用，DM 不支持，返回 NULL
27	NUM_FREELIST_BLOCKS	NUMBER	链上所有块的数量，兼容用，DM 不支持，返回 NULL
28	DEGREE	VARCHAR(1)	每个实例扫描表的线程数，兼容用，DM 不支持，返回 NULL
29	INSTANCES	VARCHAR(1)	扫描全表需要访问的实例数，DM 不支持
30	CACHE	VARCHAR(1)	表是否被 BUFFER CACHE 缓存。Y 是，N 否
31	TABLE_LOCK	VARCHAR(8)	表锁是否可用。ENABLED 可用，DISABLED 不可用
32	SAMPLE_SIZE	NUMBER	分析该表时的取样大小
33	LAST_ANALYZED	DATE	最后分析表的时间
34	PARTITIONED	VARCHAR(3)	是否为分区表，YES 是，NO 否
35	IOT_TYPE	VARCHAR(8)	表是否为索引组织表，如果是则为 IOT，IOT_OVERFLOW，或 IOT_MAPPING。如果不是则为 NULL
36	TEMPORARY	VARCHAR(1)	是否为临时表，Y 是，N 否
37	SECONDARY	VARCHAR(1)	DM 无意义，返回 NULL
38	NESTED	VARCHAR(1)	是否为嵌套表，YES 是，NO 否
39	BUFFER_POOL	VARCHAR(1)	表的缓冲池：DEFAULT、KEEP、RECYCLE、NULL。分区表为 NULL。
40	FLASH_CACHE	VARCHAR(7)	FLASH_CACHE 信息，DM 无意义，返回 NULL
41	CELL_FLASH_CACHE	VARCHAR(7)	CELL_FLASH_CACHE 信息，DM 无意义，返回 NULL
42	ROW_MOVEMENT	VARCHAR(7)	分区记录的移动是否允许。ENABLED允许，DISABLED 不允许
43	GLOBAL_STATS	VARCHAR(3)	分区的统计是来源于分区表（YES），还是通过子分区估计所得（NO）。
44	USER_STATS	VARCHAR(2)	分区的统计信息用户是否可以直接访问，YES 是，NO 否
45	DURATION	VARCHAR(15)	临时表的生命期： • SYS$SESSION –整个会话期间 • SYS$TRANSACTION – COMMIT 操作后立即删除

46	SKIP_CORRUPT	VARCHAR(1)	兼容列用。DM 无意义，返回 NULL
47	MONITORING	VARCHAR(1)	表是否配置了监视属性，YES 是，NO 否
48	CLUSTER_OWNER	VARCHAR(1)	聚簇的所有者
49	DEPENDENCIES	VARCHAR(1)	创建表时是否指定 ROWDEPENDENCIES 属性，DM 无意义返回 NULL
50	COMPRESSION	VARCHAR(8)	表是否压缩 (ENABLED) OR (DISABLED); 分区表为 NULL
51	COMPRESS_FOR	VARCHAR(1)	压缩选项：BASIC、OLTP、QUERY LOW、QUERY HIGH、ARCHIVE LOW、ARCHIVE HIGH、NULL
52	DROPPED	VARCHAR(1)	表是否已被删除并且还可以回滚，YES 是，NO 否。 分区表为 NULL
53	READ_ONLY	VARCHAR(1)	表是否是只读的，YES 是，NO 否
54	SEGMENT_CREATED	VARCHAR(3)	表的段是否被创建，YES 是，NO 否
55	RESULT_CACHE	VARCHAR(7)	结果集缓存模式的注释：DEFAULT–表还没有被注释、FORCE、MANUAL

16. ALL_TABLES

当前用户能够访问的表，结构同 DBA_TABLES。

17. USER_TABLES

当前用户所拥有的表，结构同 DBA_TABLES，没有 OWNER 列。

18. USER_ALL_TABLES

当前用户所拥有的表。

序号	列	数据类型	说明
1	TABLE_NAME	VARCHAR(128)	表名
2	TABLESPACE_NAME	VARCHAR(128)	表所在表空间名。 对于分区表，临时表和索引组织表为 NULL
3	CLUSTER_NAME	VARCHAR(1)	聚簇名
4	IOT_NAME	VARCHAR(128)	索引组织表名
5	STATUS	VARCHAR(8)	表的状态。UNUSABLE无效，VALID 有效
6	PCT_FREE	NUMBER	块的最小空闲百分比，分区表为 NULL
7	PCT_USED	NUMBER	块的最小使用百分比，分区表为 NULL
8	INI_TRANS	NUMBER	初始事务数，分区表为 NULL
9	MAX_TRANS	NUMBER	最大事务数，分区表为 NULL
10	INITIAL_EXTENT	NUMBER	初始簇大小，单位：字节。分区表为 NULL
11	NEXT_EXTENT	NUMBER	下一个簇的大小，单位：字节。分区表为 NULL
12	MIN_EXTENTS	NUMBER	最小簇大小，分区表为 NULL
13	MAX_EXTENTS	NUMBER	最大簇大小，分区表为 NULL
14	PCT_INCREASE	NUMBER	簇的增长百分比，分区表为 NULL
15	FREELISTS	NUMBER	空闲链的分配个数，分区表为 NULL
16	FREELIST_GROUPS	NUMBER	空闲连组的分配个数，分区表为 NULL。DM 不支持，返回 NULL
17	LOGGING	VARCHAR(1)	表的变化是否需要记录日志，Y 是，N 否。 分区表为 NULL。DM 不支持，返回 NULL
18	BACKED_UP	VARCHAR(1)	最后一次修改后表是否已备份，Y 是，N 否

19	NUM_ROWS	NUMBER	表里的记录数
20	BLOCKS	NUMBER	表中已使用块的数目
21	EMPTY_BLOCKS	NUMBER	表中未使用的块的数目
22	AVG_SPACE	NUMBER	每个数据块的平均空闲空间，单位：字节
23	CHAIN_CNT	NUMBER	表中已被链接数据的行数
24	AVG_ROW_LEN	NUMBER	表中每行数据的平均长度，DM 不支持，返回 NULL
25	AVG_SPACE_FREELIST_BLOCKS	NUMBER	一条链上所有块的平均空闲空间，兼容用，DM 不支持，返回 NULL
26	NUM_FREELIST_BLOCKS	NUMBER	链上所有块的数量，兼容用，DM 不支持，返回 NULL
27	DEGREE	VARCHAR(1)	每个实例扫描表的线程数，兼容用，DM 不支持，返回 NULL
28	INSTANCES	VARCHAR(1)	扫描全表需要访问的实例数，DM 不支持
29	CACHE	VARCHAR(1)	表是否被 BUFFER CACHE 缓存。Y 是，N 否
30	TABLE_LOCK	VARCHAR(8)	表锁是否可用。ENABLED 可用，DISABLED 不可用
31	SAMPLE_SIZE	NUMBER	分析该表时的取样大小
32	LAST_ANALYZED	DATE	最后分析表的时间
33	PARTITIONED	VARCHAR(3)	是否为分区表，YES 是，NO 否
34	IOT_TYPE	VARCHAR(8)	表是否为索引组织表，如果是则为 IOT，IOT_OVERFLOW，或 IOT_MAPPING。如果不是则为 NULL
35	OBJECT_ID_TYPE	VARCHAR(1)	是用户定义的还是系统定义的，DM 不支持
36	TABLE_TYPE_OWNER	VRACHAR(1)	对象表的所有者，DM 不支持
37	TABLE_TYPE	VARCHAR(1)	对象表类型，DM 不支持
38	TEMPORARY	VARCHAR(1)	是否为临时表，Y 是，N 否
39	SECONDARY	VARCHAR(1)	DM 无意义，返回 NULL
40	NESTED	VARCHAR(1)	是否为嵌套表，YES 是，NO 否
41	BUFFER_POOL	VARCHAR(1)	表的缓冲池：DEFAULT、KEEP、RECYCLE、NULL。分区表为 NULL。
42	FLASH_CACHE	VARCHAR(7)	FLASH_CACHE 信息，DM 无意义，返回 NULL
43	CELL_FLASH_CACHE	VARCHAR(7)	CELL_FLASH_CACHE 信息，DM 无意义，返回 NULL
44	ROW_MOVEMENT	VARCHAR(7)	分区记录的移动是否允许。ENABLED允许，DISABLED不允许
45	GLOBAL_STATS	VARCHAR(3)	分区的统计是来源于分区表（YES），还是通过子分区估计所得（NO）。
46	USER_STATS	VARCHAR(2)	分区的统计信息用户是否可以直接访问，YES 是，NO 否
47	DURATION	VARCHAR(15)	临时表的生命期： • SYS$SESSION –整个会话期间 • SYS$TRANSACTION – COMMIT 操作后立即删除
48	SKIP_CORRUPT	VARCHAR(1)	兼容列用。DM 无意义，返回 NULL
49	MONITORING	VARCHAR(1)	表是否配置了监视属性，YES 是，NO 否
50	CLUSTER_OWNER	VARCHAR(1)	聚簇的所有者

51	DEPENDENCIES	VARCHAR(1)	创建表时是否指定 ROWDEPENDENCIES 属性，DM 无意义返回 NULL
52	COMPRESSION	VARCHAR(8)	表是否压缩 (ENABLED) OR (DISABLED); 分区表为 NULL
53	COMPRESS_FOR	VARCHAR(1)	压缩选项：BASIC、OLTP、QUERY LOW、QUERY HIGH、ARCHIVE LOW、ARCHIVE HIGH、NULL
54	DROPPED	VARCHAR(1)	表是否已被删除并且还可以回滚，YES 是，NO 否。分区表为 NULL
55	SEGMENT_CREATED	VARCHAR(3)	表的段是否被创建，YES 是，NO 否

19. USER_TAB_COLS

当前用户所拥有的表、视图或聚簇的列。

序号	列	数据类型	说明
1	TABLE_NAME	VARCHAR(128)	表、视图或聚簇的名字
2	COLUMN_NAME	VARCHAR(128)	列名
3	DATA_TYPE	VARCHAR(128)	列类型名
4	DATA_TYPE_MOD	VARCHAR(3)	DM 不支持的属性，取值为 NULL
5	DATA_TYPE_OWNER	VARCHAR(128)	列数据类型的所有者
6	DATA_LENGTH	NUMBER	列长度。单位：字节
7	DATA_PRECISION	NUMBER	精度
8	DATA_SCALE	NUMBER	刻度
9	NULLABLE	VARCHAR(1)	是否允许为 NULL，Y 是，N 否
10	COLUMN_ID	NUMBER	列序号（1 开始）
11	DEFAULT_LENGTH	NUMBER	默认值长度
12	DATA_DEFAULT	TEXT	默认值
13	NUM_DISTINCT	NUMBER	不同值的个数
14	LOW_VALUE	VARBINARY(32)	列最小值
15	HIGH_VALUE	VARBINARY(32)	列最大值
16	DENSITY	NUMBER	列的密度
17	NUM_NULLS	NUMBER	NULL 值的个数
18	NUM_BUCKETS	NUMBER	直方图槽数
19	LAST_ANALYZED	DATETIME(6)	最后一次分析的时间
20	SAMPLE_SIZE	NUMBER	取样大小
21	CHARACTER_SET_NAME	VARCHAR(44)	字符集名字：CHAR_CS、NCHAR_CS
22	CHAR_COL_DECL_LENGTH	NUMBER	列数据字符类型长度
23	GLOBAL_STATS	VARCHAR(3)	对于分区表，统计信息是作为一个整体搜集(YES)，还是通过分区和子分区来估算(NO)
24	USER_STATS	VARCHAR(3)	统计信息生成是否通过手动，YES 是，NO 否
25	AVG_COL_LEN	NUMBER	列数据的平均长度，单位：字节
26	CHAR_LENGTH	NUMBER	显示时的字符长度，对下述类型有效：CHAR、VARCHAR、NCHAR、NVARCHAR
27	CHAR_USED	VARCHAR(1)	列是字节长度还是字符长度 (B) OR (C)，OR (NULL)： • CHAR

			• VARCHAR • NCHAR • NVARCHAR
28	V80_FMT_IMAGE	VARCHAR(3)	列数据是否为 8.0 版本的 IMAGE 格式，YES 是，NO 否
29	DATA_UPGRADED	VARCHAR(3)	列数据是否已经更新为最新的类型版本格式，YES 是，NO 否
30	HIDDEN_COLUMN	VARCHAR(3)	是否是一个隐藏列，YES 是，NO 否
31	VIRTUAL_COLUMN	VARCHAR(3)	是否是一个伪列，YES 是，NO 否
32	SEGMENT_COLUMN_ID	NUMBER	列在段中的序号
33	INTERNAL_COLUMN_ID	NUMBER	列的内部序号
34	HISTOGRAM	VARCHAR(15)	直方图类型，DM 不支持，返回 NULL
35	QUALIFIED_COL_NAME	VARCHAR(4000)	合格的列名

20. DBA_TAB_COLUMNS

显示数据库中所有表、视图或聚簇的列。

序号	列	数据类型	说明
1	OWNER	VARCHAR(128)	对象的拥有者
2	TABLE_NAME	VARCHAR(128)	表、视图或聚簇的名字
3	COLUMN_NAME	VARCHAR(128)	列名
4	DATA_TYPE	VARCHAR(128)	列类型名
5	DATA_TYPE_MOD	VARCHAR(1)	DM 不支持的属性，取值为 NULL
6	DATA_TYPE_OWNER	VARCHAR(1)	列数据类型的所有者
7	DATA_LENGTH	NUMBER	列长度。单位：字节
8	DATA_PRECISION	NUMBER	精度
9	DATA_SCALE	NUMBER	刻度
10	NULLABLE	VARCHAR(1)	是否允许为 NULL，Y 是，N 否
11	COLUMN_ID	NUMBER	列序号（1 开始）
12	DEFAULT_LENGTH	NUMBER	默认值长度
13	DATA_DEFAULT	VARCHAR(2048)	默认值
14	NUM_DISTINCT	NUMBER	不同值的个数
15	LOW_VALUE	VARCHAR(1)	列最小值
16	HIGH_VALUE	VARCHAR(1)	列最大值
17	DENSITY	NUMBER	列的密度
18	NUM_NULLS	NUMBER	NULL 值的个数
19	NUM_BUCKETS	NUMBER	直方图槽数
20	LAST_ANALYZED	DATE	最后一次分析的时间
21	SAMPLE_SIZE	NUMBER	取样大小
22	CHARACTER_SET_NAME	VARCHAR(8)	字符集名字：CHAR_CS、NCHAR_CS
23	CHAR_COL_DECL_LENGTH	NUMBER	列数据字符类型长度
24	GLOBAL_STATS	VARCHAR(3)	对于分区表，统计信息是作为一个整体搜集(YES)，还是通过分区和子分区来估算(NO)
25	USER_STATS	VARCHAR(2)	统计信息生成是否通过手动，YES 是，NO 否
26	AVG_COL_LEN	NUMBER	列数据的平均长度，单位：字节
27	CHAR_LENGTH	NUMBER	显示时的字符长度，对下述类型有效：CHAR、

			VARCHAR、NCHAR、NVARCHAR
28	CHAR_USED	VARCHAR(1)	列是字节长度还是字符长度 (B) OR (C)，OR (NULL)： • CHAR • VARCHAR • NCHAR • NVARCHAR
29	V80_FMT_IMAGE	VARCHAR(1)	列数据是否为 8.0 版本的 IMAGE 格式，YES 是，NO 否
30	DATA_UPGRADED	VARCHAR(1)	列数据是否已经更新为最新的类型版本格式，YES 是，NO 否
31	HISTOGRAM	VARCHAR(1)	直方图类型，DM 不支持，返回 NULL

21．USER_TAB_COLUMNS

显示当前用户所拥有的表、视图或聚簇的列。

序号	列	数据类型	说明
1	TABLE_NAME	VARCHAR(128)	表、视图或聚簇的名字
2	COLUMN_NAME	VARCHAR(128)	列名
3	DATA_TYPE	VARCHAR(128)	列类型名
4	DATA_TYPE_MOD	VARCHAR(1)	DM 不支持的属性，取值为 NULL
5	DATA_TYPE_OWNER	VARCHAR(1)	列数据类型的所有者
6	DATA_LENGTH	NUMBER	列长度。单位：字节
7	DATA_PRECISION	NUMBER	精度
8	DATA_SCALE	NUMBER	刻度
9	NULLABLE	VARCHAR(1)	是否允许为 NULL，Y 是，N 否
10	COLUMN_ID	NUMBER	列序号（1 开始）
11	DEFAULT_LENGTH	NUMBER	默认值长度
12	DATA_DEFAULT	VARCHAR(2048)	默认值
13	NUM_DISTINCT	NUMBER	不同值的个数
14	LOW_VALUE	VARCHAR(1)	列最小值
15	HIGH_VALUE	VARCHAR(1)	列最大值
16	DENSITY	NUMBER	列的密度
17	NUM_NULLS	NUMBER	NULL 值的个数
18	NUM_BUCKETS	NUMBER	直方图槽数
19	LAST_ANALYZED	DATE	最后一次分析的时间
20	SAMPLE_SIZE	NUMBER	取样大小
21	CHARACTER_SET_NAME	VARCHAR(8)	字符集名字：CHAR_CS、NCHAR_CS
22	CHAR_COL_DECL_LENGTH	NUMBER	列数据字符类型长度
23	GLOBAL_STATS	VARCHAR(3)	对于分区表，统计信息是作为一个整体搜集(YES)，还是通过分区和子分区来估算(NO)
24	USER_STATS	VARCHAR(2)	统计信息生成是否通过手动，YES 是，NO 否
25	AVG_COL_LEN	NUMBER	列数据的平均长度，单位：字节
26	CHAR_LENGTH	NUMBER	显示时的字符长度，对下述类型有效：CHAR、VARCHAR、NCHAR、NVARCHAR
27	CHAR_USED	VARCHAR(1)	列是字节长度还是字符长度 (B) or (C)，or (NULL)： CHAR

			VARCHAR NCHAR NVARCHAR
28	V80_FMT_IMAGE	VARCHAR(1)	列数据是否为 8.0 版本的 image 格式，YES 是，NO 否
29	DATA_UPGRADED	VARCHAR(1)	列数据是否已经更新为最新的类型版本格式，YES 是，NO 否
30	HISTOGRAM	VARCHAR(1)	直方图类型，DM 不支持，返回 NULL

22. DBA_CONS_COLUMNS

显示当前数据库中所有的约束涉及列。

序号	列	数据类型	说明
1	OWNER	VARCHAR(128)	约束的模式名
2	CONSTRAINT_NAME	VARCHAR(128)	约束名
3	TABLE_NAME	VARCHAR(128)	约束的表名
4	COLUMN_NAME	VARCHAR(128)	约束列名
5	POSITION	DEC(10)	当前 COLUMN 所在的 KEY 序号

23. ALL_CONS_COLUMNS

显示当前用户有权限访问的约束涉及列。

序号	列	数据类型	说明
1	OWNER	VARCHAR(128)	约束的模式名
2	CONSTRAINT_NAME	VARCHAR(128)	约束名
3	TABLE_NAME	VARCHAR(128)	约束的表名
4	COLUMN_NAME	VARCHAR(128)	约束列名
5	POSITION	DEC(10)	当前 COLUMN 所在的 KEY 序号

24. USER_CONS_COLUMNS

显示当前用户所拥有的约束涉及列。结构同 ALL_CONS_COLUMNS。

25. DBA_INDEXES

显示数据库中所有的索引。

序号	列	数据类型	说明
1	OWNER	VARCHAR(128)	所有者
2	INDEX_NAME	VARCHAR(128)	索引名
3	INDEX_TYPE	VARCHAR(21)	索引类型，DM 支持如下取值： • DOMAIN • FUNCTION-BASED NORMAL • BITMAP • FLAT • NORMAL • VIRTUAL • CLUSTER
4	TABLE_OWNER	VARCHAR(128)	索引对象的所有者
5	TABLE_NAME	VARCHAR(128)	索引对象名
6	TABLE_TYPE	VARCHAR(5)	索引对象类型，DM 支持在表上建索引，取值 TABLE

7	UNIQUENESS	VARCHAR(9)	是否为唯一索引(UNIQUE) 或 (NONUNIQUE)
8	COMPRESSION	VARCHAR(8)	索引压缩是否可用(ENABLED) 或(DISABLED)
9	PREFIX_LENGTH	NUMBER	压缩 KEY 前的列数，DM 不支持，返回 NULL
10	TABLESPACE_NAME	VARCHAR(128)	索引所在的表空间名
11	INI_TRANS	NUMBER	初始事务数
12	MAX_TRANS	NUMBER	最大事务数
13	INITIAL_EXTENT	NUMBER	初始簇大小
14	NEXT_EXTENT	NUMBER	下一个簇大小
15	MIN_EXTENTS	NUMBER	最小簇数目
16	MAX_EXTENTS	NUMBER	最大簇数目
17	PCT_INCREASE	NUMBER	簇扩展百分比
18	PCT_THRESHOLD	NUMBER	每个索引占用的最大空间百分比
19	INCLUDE_COLUMN	NUMBER	包含在索引组织表的主键索引中的最后一个列 ID
20	FREELISTS	NUMBER	空闲链表的数目
21	FREELIST_GROUPS	NUMBER	空闲链组的个数
22	PCT_FREE	NUMBER	块中最小空闲的百分比
23	LOGGING	VARCHAR(3)	索引修改是否记录日志: • YES • NO
24	BLEVEL	NUMBER	B 树的层次
25	LEAF_BLOCKS	NUMBER	索引上叶子块的个数
26	DISTINCT_KEYS	NUMBER	索引上不同值的个数
27	AVG_LEAF_BLOCKS_PER_KEY	NUMBER	每个索引不同值里的平均叶子块个数
28	AVG_DATA_BLOCKS_PER_KEY	NUMBER	用来表示索引数据分布的列，DM 无意义，返回 NULL
29	CLUSTERING_FACTOR	NUMBER	聚集因子，DM 无意义，返回 NULL
30	STATUS	VARCHAR(8)	非分区索引的状态 VALID 或 UNUSABLE
31	NUM_ROWS	NUMBER	索引中的行数
32	SAMPLE_SIZE	NUMBER	索引分析的取样大小
33	LAST_ANALYZED	DATE	最后一次索引分析的时间
34	DEGREE	VARCHAR(1)	每个实例扫描索引的线程个数或 DEFAULT
35	INSTANCES	VARCHAR(1)	扫描索引的实例个数或 DEFAULT
36	PARTITIONED	VARCHAR(3)	索引是否被分区，YES 是，NO 否
37	TEMPORARY	VARCHAR(1)	是否是临时表上的索引，Y 是，N 否
38	GENERATED	VARCHAR(1)	索引名是否是系统生成，Y 是，N 否
39	SECONDARY	VARCHAR(1)	DM 不支持，该列无意义
40	BUFFER_POOL	VARCHAR(20)	索引块使用缓冲池的方式: • DEFAULT • KEEP • RECYCLE • NULL
41	FLASH_CACHE	VARCHAR(1)	索引块使用 FLASH CACHE 信息。DM 无意义，返回 NULL

42	CELL_FLASH_CACHE	VARCHAR(1)	索引块 CELL_FLASH_CACHE 信息。DM 无意义，返回 NULL
43	USER_STATS	VARCHAR(2)	统计信息是否是用户生成，YES 是，NO 否
44	DURATION	VARCHAR(15)	临时表的生命周期： • SYS$SESSION –整个会话期间 • SYS$TRANSACTION – COMMIT 操作后立即删除
45	PCT_DIRECT_ACCESS	NUMBER	索引组织表的二级索引中有效行的百分比。DM 无意义，返回 NULL
46	ITYP_OWNER	VARCHAR(1)	域索引的索引类型的所有者
47	ITYP_NAME	VARCHAR(1)	域索引的索引类型名
48	PARAMETERS	VARCHAR(1)	域索引的参数串
49	GLOBAL_STATS	VARCHAR(3)	对于分区表，统计信息是作为一个整体搜集(YES)，还是通过分区和子分区来估算(NO)
50	DOMIDX_STATUS	VARCHAR(1)	域索引状态，DM 无意义，返回 NULL
51	DOMIDX_OPSTATUS	VARCHAR(1)	域索引的操作状态，DM 无意义，返回 NULL
52	FUNCIDX_STATUS	VARCHAR(1)	函数索引状态，DM 无意义，返回 NULL
53	JOIN_INDEX	VARCHAR(1)	是否为组合索引，YES 是，NO 否
54	IOT_REDUNDANT_PKEY_ELIM	VARCHAR(1)	DM 无意义，返回 NULL
55	DROPPED	VARCHAR(1)	索引是否已被删除并在回滚段中，YES 是，NO 否 。分区表为 NULL
56	VISIBILITY	VARCHAR(7)	索引是否可见。 VISIBLE 可见，INVISIBLE 不可见
57	DOMIDX_MANAGEMENT	VARCHAR(1)	域索引的管理方式(SYSTEM_MANAGED) 或 (USER_MANAGED)
58	SEGMENT_CREATED	VARCHAR(1)	索引段是否已建立，YES 是，NO 否

26. ALL_INDEXES

当前用户有权访问的索引，结构同 DBA_INDEXES。

27. USER_INDEXES

当前用户所拥有的索引，结构同 DBA_INDEXES，除了 OWNER 列。

28. DBA_VIEWS

显示数据库中所有的视图。

序号	列	数据类型	说明
1	OWNER	VARCHAR(128)	视图所有者
2	VIEW_NAME	VARCHAR(128)	视图名
3	TEXT_LENGTH	INTEGER	视图文本的长度
4	TEXT	TEXT	视图文本
5	TYPE_TEXT_LENGTH	NUMBER	对象视图定义文本长度， DM 不支持，返回 NUL
6	TYPE_TEXT	VARCHAR(4000)	对象视图定义文本， DM 不支持，返回 NULL
7	OID_TEXT_LENGTH	NUMBER	对象视图 WITH OBJECT IDENTIFIER 子句文本长度 DM 不支持，返回 NULL
8	OID_TEXT	VARCHAR(4000)	对象视图 WITH OBJECT IDENTIFIER 子句文本

			DM 不支持，返回 NULL
9	VIEW_TYPE_OWNER	VARCHAR(30)	对象视图对象的所有者 DM 不支持，返回 NULL
10	VIEW_TYPE	VARCHAR(30)	对象视图的源对象 DM 不支持，返回 NULL
11	SUPERVIEW_NAME	VARCHAR(30)	上级视图名 DM 不支持，返回 NULL
12	EDITIONING_VIEW	VARCHAR(1)	保留
13	READ_ONLY	VARCHAR(1)	视图是否只读，Y 是，N 否

29. ALL_VIEWS

当前用户能够访问的所有视图，结构同 DBA_VIEWS。

30. USER_VIEWS

当前用户所拥有的所有视图，结构同 DBA_VIEWS，没有 OWNER 列。

31. DBA_TRIGGERS

显示当前数据库的全部触发器。

序号	列	数据类型	说明
1	OWNER	VARCHAR(128)	触发器模式名
2	TRIGGER_NAME	VARCHAR(128)	触发器名
3	TRIGGERING_TYPE	VARCHAR(16)	触发器类型： • BEFORE STATEMENT • BEFORE EACH ROW • AFTER STATEMENT • AFTER EACH ROW • INSTEAD OF • COMPOUND 表触发器 1) INSERT、DELETE 和 UPDATE 触发器 2) 元组级触发器和语句级触发器 3)BEFORE 和 AFTER 触发器 事件触发器
4	TRIGGERING_EVENT	VARCHAR(128)	表触发器、事件触发器、DML，DDL 或数据库事件
5	TABLE_OWNER	VARCHAR(128)	触发器的表的模式
6	TABLE_NAME	VARCHAR(128)	如果是 TABLE 或 VIEW，表名或视图名
7	COLUMN_NAME	VARCHAR(1)	触发器是嵌套表，则嵌套表的列名
8	STATUS	CHAR(1)	禁用或启用
9	TRIGGER_BODY	TEXT	触发器体

32. ALL_TRIGGERS

显示当前用户有权限访问的触发器。如果有 CREATE ANY TRIGGER 权限，则等同 DBA_TRIGGERS。

序号	列	数据类型	说明
1	OWNER	VARCHAR(128)	触发器模式名
2	TRIGGER_NAME	VARCHAR(128)	触发器名
3	TRIGGERING_TYPE	VARCHAR(16)	触发器类型： • BEFORE STATEMENT • BEFORE EACH ROW

			• AFTER STATEMENT • AFTER EACH ROW • INSTEAD OF • COMPOUND 表触发器 1) INSERT、DELETE 和 UPDATE 触发器 2) 元组级触发器和语句级触发器 3)BEFORE 和 AFTER 触发器 事件触发器
4	TRIGGERING_EVENT	VARCHAR(128)	表触发器、事件触发器 DML，DDL，或数据库事件
5	TABLE_OWNER	VARCHAR(128)	触发器的表的模式
6	TABLE_NAME	VARCHAR(128)	表名或视图名
7	COLUMN_NAME	VARCHAR(1)	触发器是嵌套表，则嵌套表的列名
8	STATUS	CHAR(1)	禁用或启用
9	TRIGGER_BODY	TEXT	触发器体

33. USER_TRIGGERS

显示当前用户所拥有的触发器。

序号	列	数据类型	说明
1	OWNER	VARCHAR(128)	触发器模式名
2	TRIGGERING_TYPE	VARCHAR(16)	触发器类型： • BEFORE STATEMENT • BEFORE EACH ROW • AFTER STATEMENT • AFTER EACH ROW • INSTEAD OF • COMPOUND 表触发器 1) INSERT、DELETE 和 UPDATE 触发器 2) 元组级触发器和语句级触发器 3)BEFORE 和 AFTER 触发器 事件触发器
3	TRIGGERING_EVENT	VARCHAR(128)	表触发器、事件触发器。DML，DDL，或数据库事件
4	TABLE_OWNER	VARCHAR(128)	触发器的表的模式
5	TABLE_NAME	VARCHAR(128)	表名或视图名
6	COLUMN_NAME	VARCHAR(1)	触发器是嵌套表，则嵌套表的列名
7	STATUS	CHAR(1)	禁用或启用
8	TRIGGER_BODY	TEXT	触发器体

34. DBA_OBJECTS

显示数据库中所有的对象。

序号	列	数据类型	说明
1	OWNER	VARCHAR(128)	对象所属用户
2	OBJECT_NAME	VARCHAR(128)	对象名
3	SUBOBJECT_NAME	VARCHAR(1)	子对象名，例如分区。DM 为 NULL
4	OBJECT_ID	NUMBER	对象的 ID

5	DATA_OBJECT_ID	NUMBER	对象所在段的 ID，DM 不支持，返回 NULL
6	OBJECT_TYPE	VARCHAR(15)	对象类型（取值如 TABLE 或 INDEX）
7	CREATED	DATETIME(6)	对象创建时间
8	LAST_DDL_TIME	VARCHAR(2048)	DDL 语句（包括授权）最后修改时间
9	TIMESTAMP	DATETIME(6)	对象的时间戳
10	STATUS	VARCHAR(7)	对象状态。DM为NULL
11	TEMPORARY	VARCHAR(1)	表明是否是临时表
12	GENERATED	VARCHAR(1)	Y 表示系统创建的对象，N 表示用户创建的对象。 只支持表、视图、触发器、存储过程、同义词、包
13	SECONDARY	VARCHAR(1)	DM为NULL
14	NAMESPACE	NUMBER	对象命名空间。DM为NULL
15	EDITION_NAME	VARCHAR(1)	版本名。DM为NULL

35. USER_OBJECTS

当前用户所拥有的对象。结构同 DBA_OBJECTS，少了 OWNER 列。

36. ALL_OBJECTS

当前用户可访问的对象。结构同 DBA_OBJECTS。

37. USER_COL_COMMENTS

当前用户所拥有的所有表和视图上的列注释。

序号	列	数据类型	说明
1	OWNER	VARCHAR(128)	对象所属用户
2	TABLE_NAME	VARCHAR(128)	表视图名
3	COLUMN_NAME	VARCHAR(128)	列名
4	COMMENTS	VARCHAR(1024)	列注释

38. DBA_COL_COMMENTS

当前数据库中的所有表和视图上的列注释。

序号	列	数据类型	说明
1	OWNER	VARCHAR(128)	对象所属用户
2	TABLE_NAME	VARCHAR(128)	表视图名
3	COLUMN_NAME	VARCHAR(128)	列名
4	COMMENTS	VARCHAR(1024)	列注释

39. ALL_COL_COMMENTS

当前用户可访问的所有表和视图上的列注释。

序号	列	数据类型	说明
1	OWNER	VARCHAR(128)	对象所属用户
2	TABLE_NAME	VARCHAR(128)	表视图名
3	COLUMN_NAME	VARCHAR(128)	列名
4	COMMENTS	VARCHAR(1024)	列注释

40. USER_TAB_COMMENTS

当前用户所拥有的所有表和视图上的注释。

序号	列	数据类型	说明

1	TABLE_NAME	VARCHAR(128)	表名
2	TABLE_TYPE	VARCHAR(5)	表类型
3	COMMENTS	VARCHAR(1024)	表或视图注释

41. DBA_TAB_COMMENTS

当前数据库中的所有表和视图上的注释。

序号	列	数据类型	说明
1	OWNER	VARCHAR(128)	对象所属用户
2	TABLE_NAME	VARCHAR(128)	表名
3	TABLE_TYPE	VARCHAR(5)	表类型
4	COMMENTS	VARCHAR(1024)	表或视图注释

42. ALL_TAB_COMMENTS

当前用户可访问的所有表和视图上的注释。

序号	列	数据类型	说明
1	OWNER	VARCHAR(128)	对象所属用户
2	TABLE_NAME	VARCHAR(128)	表名
4	TABLE_TYPE	VARCHAR(5)	表类型
5	COMMENTS	VARCHAR(1024)	表或视图注释

43. USER_PART_TABLES

显示当前用户所有的分区表信息，视图结构与 ALL_PART_TABLES 一样，区别是少了 OWNER 字段。

44. DBA_PART_TABLES

显示整个数据库所有的分区表信息，视图结构与 ALL_PART_TABLES 一样。

45. ALL_PART_TABLES

显示当前用户有权访问的所有的分区表信息。

序号	列	数据类型	说明
1	OWNER	VARCHAR(128)	分区表的拥有者
2	TABLE_NAME	VARCHAR(128)	分区表名
3	PARTITIONING_TYPE	VARCHAR(8)	分区表类型：UNKNOWN、RANGE、HASH、LIST、VERTICAL（垂直分区）
4	SUBPARTITIONING_TYPE	VARCHAR(8)	组合分区类型：NONE、RANGE、HASH、LIST
5	PARTITION_COUNT	BIGINT	分区个数
6	DEF_SUBPARTITION_COUNT	BIGINT	组合分区个数
7	PARTITIONING_KEY_COUNT	INTEGER	分区中 KEY 包含列的个数
8	SUBPARTITIONING_KEY_COUNT	INTEGER	组合分区中 KEY 包含列的个数
9	STATUS	VARCHAR(5)	分区表状态(UNUSABLE) 或 (VALID)
10	DEF_TABLESPACE_NAME	VARCHAR(128)	表空间名
11	DEF_PCT_FREE	NUMBER	增加一个分区表时 PCTFREE的默认值。DM 默认为 NULL。
12	DEF_PCT_USED	NUMBER	增加一个分区表时 PCTUSED的默认值。DM 默认为 NULL。
13	DEF_INI_TRANS	NUMBER	增加一个分区表时 INITRANS的默认值。DM 默认

			为 NULL。
14	DEF_MAX_TRANS	NUMBER	增加一个分区表时 MAXTRANS的默认值。DM 默认为 NULL。
15	DEF_INITIAL_EXTENT	VARCHAR(7)	增加一个分区表时 INITIAL的默认值。DM 默认为 DEFAULT。
16	DEF_NEXT_EXTENT	VARCHAR(7)	增加一个分区表时 NEXT 的默认值。DM 默认为 DEFAULT。
17	DEF_MIN_EXTENTS	VARCHAR(7)	增加一个分区表时 MINEXTENTS 的默认值。DM 默认为 DEFAULT。
18	DEF_MAX_EXTENTS	VARCHAR(7)	增加一个分区表时 MAXEXTENTS 的默认值。DM 默认为 DEFAULT。
19	DEF_MAX_SIZE	VARCHAR(7)	增加一个分区表时 MAXSIZE的默认值。DM 默认为 DEFAULT。
20	DEF_PCT_INCREASE	VARCHAR(7)	增加一个分区表时 PCTINCREASE的默认值。DM 默认为 DEFAULT。
21	DEF_FREELISTS	INTEGER	增加一个分区表时 FREELISTS的默认值。DM 默认为 0。
22	DEF_FREELIST_GROUPS	INTEGER	增加一个分区表时 FREELISTGROUPS的默认值。DM 默认为 0。
23	DEF_LOGGING	VARCHAR(4)	增加一个分区时是否记录额外日志。 DM 不支持，返回 NULL
24	DEF_COMPRESSION	VARCHAR(7)	增加一个分区表时 默认是否压缩： ENABLED DISABLED
25	DEF_COMPRESS_FOR	VARCHAR(12)	增加一个分区表时 哪些操作进行压缩： BASIC、OLTP、QUERY LOW、QUERY HIGH、ARCHIVE LOW 、 ARCHIVE HIGH 、UNKNOWN、NULL。 DM 不支持，返回 NULL.
26	DEF_BUFFER_POOL	VARCHAR(7)	增加一个分区表时使用 STORAGE 子句指定的 BUFFER_POOL的值。 DM 不支持，返回 DEFAULT
27	DEF_FLASH_CACHE	VARCHAR(7)	增加一个分区表时使用 STORAGE 子句指定的 FLASH_CACHE的值。 DM 不支持，返回 DEFAULT
28	DEF_CELL_FLASH_CACHE	VARCHAR(7)	增加一个分区表时 CELL FLASH CACHE的默认值：DEFAULT、KEEP、NONE。
29	REF_PTN_CONSTRAINT_NAME	VARCHAR(128)	引用分区表的引用约束名。 DM 不支持，返回 NULL
30	INTERVAL	VARCHAR(1000)	分区字符串， DM 不支持，返回 NULL
31	IS_NESTED	VARCHAR(2)	分区表是否是嵌套表，YES 是，NO 否 。

46. USER_TAB_PARTITIONS

显示当前用户所拥有的分区表的详细分区信息，视图结构与 ALL_TAB_PARTITIONS 一样，区别是少了 TABLE_OWNER 字段。

47. DBA_TAB_PARTITIONS

显示整个数据库所有的分区表的详细分区信息，视图结构与 ALL_TAB_PARTITIONS 一样。

48. ALL_TAB_PARTITIONS

显示当前用户可以访问的的所有分区表的详细分区信息。

序号	列	数据类型	说明
1	TABLE_OWNER	VARCHAR(128)	表的拥有者
2	TABLE_NAME	VARCHAR(128)	表名
3	COMPOSITE	VARCHAR(3)	是否是组合分区：YES 是，NO 否
4	PARTITION_NAME	VARCHAR(128)	分区名
5	SUBPARTITION_COUNT	BIGINT	组合分区个数
6	HIGH_VALUE	VARCHAR(8187)	分区边界最大值
7	HIGH_VALUE_LENGTH	INTEGER	分区边界最大值的长度
8	PARTITION_POSITION	INTEGER	表的第几个分区
9	TABLESPACE_NAME	VARCHAR(128)	分区所在的表空间名
10	PCT_FREE	NUMBER	空闲空间百分比 DM 不支持，返回 NULL
11	PCT_USED	NUMBER	使用空间百分比 DM 不支持，返回 NULL
12	INI_TRANS	NUMBER	初始化事务数 DM 不支持，返回 NULL
13	MAX_TRANS	NUMBER	最大事务数 DM 不支持，返回 NULL
14	INITIAL_EXTENT	NUMBER	INITIAL值。 DM 不支持，返回 NULL
15	NEXT_EXTENT	NUMBER	NEXT值。 DM 不支持，返回 NULL
16	MIN_EXTENT	NUMBER	段中最小的簇数 DM 不支持，返回 NULL
17	MAX_EXTENT	NUMBER	段中最大的簇数 DM 不支持，返回 NULL
18	MAX_SIZE	NUMBER	MAXSIZE值。 DM 不支持，返回 NULL
19	PCT_INCREASE	NUMBER	PCTINCREASE值。 DM 不支持，返回 NULL
20	FREELISTS	NUMBER	FREELISTS值。 DM 不支持，返回 NULL
21	FREELIST_GROUPS	NUMBER	FREELISTGROUPS值。 DM 不支持，返回 NULL
22	LOGGING	VARCHAR(4)	分区表是否有额外日志记录。 DM 不支持，返回 NULL
23	COMPRESSION	VARCHAR(7)	分区是否压缩
24	COMPRESS_FOR	VARCHAR(12)	分区表时 哪些操作进行压缩： • BASIC • OLTP • QUERY LOW • QUERY HIGH • ARCHIVE LOW • ARCHIVE HIGH • NULL

			DM 不支持，返回 NULL
25	NUM_ROWS	NUMBER	分区的行数。
26	BLOCKS	NUMBER	分区用的块数。DM 全部为 NULL。
27	EMPTY_BLOCKS	NUMBER	分区空块数。DM 全部为 NULL。
28	AVG_SPACE	NUMBER	分区的平均空闲数。DM 全部为 NULL。
29	CHAIN_CNT	NUMBER	分区 CHAINED ROWS 数。DM 全部为 NULL。
30	AVG_ROW_LEN	NUMBER	行的平均数据长度。DM 全部为 NULL。
31	SAMPLE_SIZE	NUMBER	分区表收集统计信息的样本数。 DM 全部为 NULL。
32	LAST_ANALYZED	DATE	分区最近一次统计分析的时间。DM 全部为 NULL。
33	BUFFER_POOL	VARCHAR(7)	分区的 BUFFER POOL 值： • DEFAULT • KEEP • RECYCLE • NULL DM 全部为 DEFAULT。
34	FLASH_CACHE	VARCHAR(7)	分区的 FLASH CACHE 值 • DEFAULT • KEEP • NONE DM 全部为 DEFAULT。
35	CELL_FLASH_CACHE	VARCHAR(7)	分区的 CELL FLASH CACHE 值： • DEFAULT • KEEP • NONE DM 全部为 DEFAULT。
36	GLOBAL_STATS	VARCHAR(2)	分区的统计是来源于分区表（YES），还是通过子分区估计所得（NO）。DM 默认为 NO。
37	USER_STATS	VARCHAR(3)	分区的统计信息用户是否可以直接访问，YES 是，NO 否 。DM 全部为 YES。
38	IS_NESTED	VARCHAR(2)	分区表是否是嵌套表，YES 是，NO 否 。DM 全部为 NO。
39	PARENT_TABLE_PARTITION	VARCHAR(128)	父分区名
40	INTERVAL	VARCHAR(2)	分区表的区间分区 (YES) 或范围分区 (NO)
41	SEGMENT_CREATED	VARCHAR(3)	分区的段是否创建，YES 是，NO 否 。DM 全部为 YES。

49. DBA_SOURCE

数据库中所有的存储对象和源码。

序号	列	数据类型	说明
1	OWNER	VARCHAR(128)	对象的所有者
2	NAME	VARCHAR(128)	对象的名称
3	TYPE	VARCHAR(10)	对象的类型
4	LINE	NUMBER	行数，DM 不支持，为 NULL
5	TEXT	TEXT	存储对象的源代码

50. ALL_SOURCE

当前用户可访问的存储对象和源码，结构同 DBA_SOURCE。

51. USER_SOURCE

当前用户拥有的存储对象和源码，结构同 DBA_SOURCE，除了 OWNER 列。

52. DBA_TRIGGER_COLS

触发器中使用的列，DM 暂不支持，查询为空集。

序号	列	数据类型	说明
1	TRIGGER_OWNER	VARCHAR2(30)	触发器所有者
2	TRIGGER_NAME	VARCHAR2(30)	触发器名称
3	TABLE_OWNER	VARCHAR2(30)	触发器所属的表的所有者
4	TABLE_NAME	VARCHAR2(30)	触发器所属的表名
5	COLUMN_NAME	VARCHAR2(4000)	触发器中使用的列名
6	COLUMN_LIST	VARCHAR2(3)	列是否定义在 UPDATE 语句中，YES 是，NO 否
7	COLUMN_USAGE	VARCHAR2(17)	触发器中列的使用方式 • NEW IN • OLD IN • NEW IN OLD IN • NEW OUT • NEW IN OUT • NEW OUT OLD IN • NEW IN OUT OLD IN • PARENT IN

53. ALL_TRIGGER_COLS

当前用户有权访问的触发器的列。结构同 DBA_TRIGGER_COLS。

54. USER_TRIGGER_COLS

当前用户所拥有的触发器或表上所定义的触发器中使用的列，结构同 DBA_TRIGGER_COLS。

55. DBA_DB_LINKS

数据库中所有的 DB_LINK。

序号	列	数据类型	说明
1	OWNER	VARCHAR(32767)	LINK 的所有者
2	DB_LINK	VARCHAR(128)	LINK 名称
3	USERNAME	VARCHAR(128)	LINK 登录名
4	HOST	VARCHAR(1024)	LINK 连接主库
5	CREATED	DATETIME(6)	LINK 的创建时间

56. ALL_DB_LINKS

当前用户可访问的 DB_LINK，结构同 DBA_DB_LINKS。

57. USER_DB_LINKS

当前用户所拥有的 DBLINK，结构同 DBA_DB_LINKS，没有 OWNER 列。

58. DBA_SEQUENCES

数据库中所有的序列。

序号	列	数据类型	说明
1	SEQUENCE_OWNER	VARCHAR2(128)	序列的所有者
2	SEQUENCE_NAME	VARCHAR2(128)	序列名称
3	MIN_VALUE	BIGINT	最小值

序号	列	数据类型	说明
4	MAX_VALUE	BIGINT	最大值
5	INCREMENT_BY	BIGINT	增加步长
6	CYCLE_FLAG	VARCHAR(1)	循环标记
7	ORDER_FLAG	VARCHAR(1)	顺序标记
8	CACHE_SIZE	VARCHAR(1)	缓存大小
9	LAST_NUMBER	VARCHAR(1)	最后的序列值

59. ALL_SEQUENCES

当前用户可访问的序列。结构同 DBA_SEQUENCES

60. USER_SEQUENCES

当前用户所拥有的序列，结构同 DBA_SEQUENCES，没有 SEQUENCE_OWNER 列。

61. DBA_SYNONYMS

数据库中所有的同义词。

序号	列	数据类型	说明
1	OWNER	VARCHAR(128)	所有者
2	SYNONYM_NAME	VARCHAR(128)	同义词名
3	TABLE_OWNER	VARCHAR(128)	同义词所指向的对象的所有者
4	TABLE_NAME	VARCHAR(128)	同义词所指的对象，如表/视图、过程、同义词、序列等
5	DB_LINK	VARCHAR(1)	LINK 名称

62. ALL_SYNONYMS

当前用户可访问的同义词，结构同 DBA_SYNONYMS。

63. USER_SYNONYMS

当前用户所拥有的同义词，结构同 DBA_SYNONYM，没有 OWNER 列。

64. DBA_TABLESPACES

数据库中所有的表空间。

序号	列	数据类型	说明
1	TABLESPACE_NAME	VARCHAR(128)	表空间名
2	BLOCK_SIZE	BIGINT	块大小
3	INITIAL_EXTENT	VARCHAR(1)	初始分配大小
4	NEXT_EXTENT	VARCHAR(1)	下次分配大小
5	MIN_EXTENTS	VARCHAR(1)	最小分配次数
6	MAX_EXTENTS	VARCHAR(1)	最大分配次数
7	MAX_SIZE	BIGINT	段最大大小
8	PCT_INCREASE	VARCHAR(1)	相对于上次增长百分比
9	MIN_EXTLEN	VARCHAR(1)	最小扩展长度
10	STATUS	INTEGER	状态：ONLINE 或OFFLINE 或READ ONLY
11	CONTENTS	VARCHAR(9)	内容。PERMANENT 或TEMPORARY
12	LOGGING	VARCHAR(1)	日志属性（缺省为进行日志）
13	FORCE_LOGGING	VARCHAR(1)	强制日志记录模式下
14	EXTENT_MANAGEMENT	VARCHAR(1)	扩展管理跟踪：DICTIONARY

序号	列	数据类型	说明
			或LOCAL
15	ALLOCATION_TYPE	VARCHAR(1)	扩展分配类型
16	PLUGGED_IN	VARCHAR(1)	表示表空间为追加
17	SEGMENT_SPACE_MANAGEMENT	VARCHAR(1)	创建表空间时SEGMENT_MANAGEMENT子句指定表明段的管理方式。DM无意义，返回NULL
18	DEF_TAB_COMPRESSION	VARCHAR(8)	默认表压缩是否可用。DM无意义，返回NULL
19	RETENTION	VARCHAR(1)	创建表空间时指定的RETENTION项。DM无意义，返回NULL
20	BIGFILE	VARCHAR(1)	创建表空间是否为BIGFILE表空间，DM无意义，返回NULL
21	PREDICATE_EVALUATION	VARCHAR(1)	DM无意义，返回NULL
22	ENCRYPTED	VARCHAR(1)	是否是加密表空间
23	COMPRESS_FOR	VARCHAR(1)	默认为何种操作压缩，DM无意义，返回NULL

65. USER_TABLESPACES

当前用户可访问的表空间，结构同DBA_TABLESPACES。

66. DBA_COL_PRIVS

数据库中的所有列对象。

序号	列	数据类型	说明
1	GRANTEE	VARCHAR(128)	被授权者
2	OWNER	VARCHAR(128)	所有者
3	TABLE_NAME	VARCHAR(128)	对象名
4	COLUMN_NAME	VARCHAR(128)	列名
5	GRANTOR	VARCHAR(128)	授权者
6	PRIVILEGE	VARCHAR(32767)	权限
7	GRANTABLE	VARCHAR(3)	是否可转授

67. ALL_COL_PRIVS

当前用户作为所有者、授权者或被授权者以及被授权给PUBLIC的所有列对象。

序号	列	数据类型	说明
1	GRANTOR	VARCHAR(128)	授权者
2	GRANTEE	VARCHAR(128)	被授权者
3	TABLE_SCHEMA	VARCHAR(128)	模式名
4	TABLE_NAME	VARCHAR(128)	表名
5	COLUMN_NAME	VARCHAR(128)	列名
6	PRIVILEGE	VARCHAR(32767)	权限
7	GRANTABLE	VARCHAR(3)	是否可转授

68. USER_COL_PRIVS

当前用户作为所有者、授权者或被授权者所有列对象，结构同DBA_COL_PRIVS。

69. DBA_IND_COLUMNS

系统中所有的索引列。

序号	列	数据类型	说明
1	INDEX_OWNER	VARCHAR(128)	索引所有者
2	INDEX_NAME	VARCHAR(128)	索引名称
3	TABLE_OWNER	VARCHAR(128)	表的所有者
4	TABLE_NAME	VARCHAR(128)	表名
5	COLUMN_NAME	VARCHAR(128)	列名
6	COLUMN_POSITION	NUMBER	索引中列的位置
7	COLUMN_LENGTH	NUMBER	列的数据长度
8	CHAR_LENGTH	NUMBER	字符类型定义的最大长度
9	DESCEND	VARCHAR2(4)	正序或逆序排序

70. ALL_IND_COLUMNS

当前用户可访问的索引列，结构同 DBA_IND_COLUMNS。

71. USER_IND_COLUMNS

当前用户所拥有的索引列。表及索引均属于当前用户。

序号	列	数据类型	说明
1	INDEX_NAME	VARCHAR(128)	索引名称
2	TABLE_NAME	VARCHAR(128)	表名
3	COLUMN_NAME	VARCHAR(128)	列名
4	COLUMN_POSITION	DEC(10)	索引中列的位置
5	COLUMN_LENGTH	DEC(10)	列的数据长度
6	CHAR_LENGTH	DEC(10)	字符类型定义的最大长度
7	DESCEND	VARCHAR2(4)	正序或逆序排序

72. SYS.SESSION_PRIVS

显示用户当前可访问的权限

序号	列	数据类型	说明
1	PRIVILEGE	VARCHAR(32767)	权限名

73. SYS.SESSION_ROLES

显示当前可授权给用户的角色

序号	列	数据类型	说明
1	ROLE	VARCHAR(32767)	角色名

74. USER_MVIEWS

当前用户所拥有的物化视图

序号	列	数据类型	说明
1	SCHID	INTEGER	模式 ID
2	MVIEW_NAME	VARCHAR(128)	物化视图名称
3	QUERY	TEXT	文本信息
4	QUERY_LEN	INTEGER	文本信息长度，单位：字节
5	REWRITE_ENABLED	VARCHAR(128)	是否可以被重写。Y 是，N 否
6	REFRESH_MODE	VARCHAR(128)	刷新模式：DEMAND,COMMIT
7	REFRESH_METHOD	VARCHAR(128)	刷新方法：COMPLETE,FORCE,FAST,NEVER

8	MVIEW_TYPE	VARCHAR(128)	快速刷新类型： SIMPLE：简单； AGGREGATE：聚合； JOIN：连接； Sub-Query：子查询； COMPLEX：不可快速刷新。
9	LAST_REFRESH_TYPE	VARCHAR(128)	最后一次刷新的类型
10	STALENESS	VARCHAR(128)	物化视图状态： UNUSEABLE：物化视图不可用，即从未刷新过 FRESH：物化视图数据是最新的； NEEDS_COMPILE：物化视图数据陈旧； COMPILE_ERROR：化视图解析出错，如基表不存在； NEEDS_FULL_REFRESH：物化视图数据陈旧，需要一次全刷新； UNDEFINED：物化视图包含远程表状态不可知 。
11	LAST_REFRESH_DATE	DATETIME(6)	最后刷新的日期

75. DBA_DATA_FILES

显示数据文件信息。

序号	列	数据类型	说明
1	FILE_NAME	VARCHAR(256)	数据文件名称
2	FILE_ID	INTEGER	数据文件所属文件 id
3	TABLESPACE_NAME	VARCHAR(128)	数据文件所属表空间名
4	BYTES	BIGINT	数据文件大小，单位：字节
5	BLOCKS	BIGINT	数据文件块大小
6	STATUS	VARCHAR(9)	数据文件状态。AVAILABLE 可用，INVALID 不可用
7	RELATIVE_FNO	INTEGER	所在表空间的数据文件个数
8	AUTOEXTENSIBLE	VARCHAR(9)	是否可自动扩展
9	MAXBYTES	NUMBER	最大数据文件大小，单位：字节
10	MAXBLOCKS	INTEGER	最大数据文件大小，单位：块
11	INCREMENT_BY	INTEGER	可用于自动扩展的数据块数量
12	USER_BYTES	BIGINT	用户数据可使用的文件大小
13	USER_BLOCKS	BIGINT	数据可使用的块数量
14	ONLINE_STATUS	VARCHAR(9)	文件状态。SYSOFF、SYSTEM、OFFLINE、ONLINE、RECOVER

76. SYS.SYSAUTH$

显示所有对象被授于的权限。

序号	列	数据类型	说明
1	GRANTEE#	INTEGER	被授权者id
2	PRIVILEGE#	VARCHAR(32767)	被授权的权限
3	SEQUENCE#	BIGINT	系统序号
4	OPTION$	INTEGER	是否可以转授

77. DBA_SEGMENTS

显示数据库中所有段的存储信息。

序号	列	数据类型	说明
1	OWNER	VARCHAR2(128)	段的拥有者
2	SEGMENT_NAME	VARCHAR2(128)	段名
3	PARTITION_NAME	VARCHAR2(128)	分区名
4	SEGMENT_TYPE	VARCHAR2(15)	段分三种类型："INDEX","TABLE PARTION","TABLE"
5	SEGMENT_SUBTYPE	VARCHAR(1)	暂时为 NULL
6	TABLESPACE_NAME	VARCHAR(128)	表空间名
7	HEADER_FILE	SMALLINT	段头所在文件 ID
8	HEADER_BLOCK	INTEGER	段头所在块 ID
9	BYTES	BIGINT	段的大小，单位为字节
10	BLOCKS	INTEGER	段的大小，单位为块
11	EXTENTS	INTEGER	段的簇数
12	INITIAL_EXTENT	BIGINT	段的初始簇大小
13	NEXT_EXTENT	BIGINT	段的下一个簇的大小
14	MIN_EXTENTS	SMALLINT	簇的最小值
15	MAX_EXTENTS	BIGINT	簇的最大值
16	MAX_SIZE	BIGINT	同 MAX_EXTENTS
17	RETENTION	VARCHAR(1)	暂时为 NULL
18	MINRETENTION	VARCHAR(1)	暂时为 NULL
19	PCT_INCREASE	VARCHAR(1)	暂时为 NULL
20	FREELISTS	VARCHAR(1)	暂时为 NULL
21	FREELIST_GROUPS	VARCHAR(1)	暂时为 NULL
22	RELATIVE_FNO	SMALLINT	与段相关的文件个数
23	BUFFER_POOL	VARCHAR(7)	段中块使用缓冲池的方式，有如下几种方式："KEEP"，"RECYCLE"，"FAST"，"DEFAULT"
24	FLASH_CACHE	VARCHAR(1)	暂时为 NULL
25	CELL_FLASH_CACHE	VARCHAR(1)	暂时为 NULL

78. USER_SEGMENTS

显示数据库中当前用户的段的存储信息。结构与 DBA_SEGMENTS 相同。

79. DBA_DEPENDENCIES

只有具备 DBA 角色的用户才可以查询的对象的依赖关系。

序号	列	数据类型	说明
1	OWNER	VARCHAR(128)	对象的模式名
2	NAME	VARCHAR(128)	对象名
3	TYPE	VARCHAR(17)	对象类型（被引用对象类型）： TABLE（CLASS,TYPE,DOMAIN,SEQUENCE） VIEW（TABLE,VIEW,PACKAGE,PROCEDURE,SEQUENCE） PROCEDURE （TABLE,VIEW,PROCEDURE,PACKAGE,CLASS,TYPE） PACKAGE （TABLE,VIEW,PROCEDURE,PACKAGE,CLASS,TYPE） TRIGGER （TABLE,VIEW,PROCEDURE,PACKAGE,CLASS,TYPE） SYNONYM（TABLE,VIEW,PROCEDURE PACKAGE,CLASS,TYPE,SEQUENCE） CLASS（CLASS,TYPE） TYPE（CLASS,TYPE） INDEX（FUNCTION(确定性函数)）

			FUNCTION（FUNCTION,TABLE,VIEW,PROCEDURE, PACKAGE,CLASS,TYPE）
4	REFERENCED_OWNER	VARCHAR(128)	被引用对象的模式名
5	REFERENCED_NAME	VARCHAR(32767)	被引用对象名
6	REFERENCED_TYPE	VARCHAR(17)	被引用对象类型 TABLE VIEW PROCEDURE SYNONYM CLASS PACKAGE TYPE DOMAIN SEQUENCE FUNCTION
7	REFERENCED_LINK _NAME	VARCHAR(1)	默认为 NULL
8	DEPENDENCY_TYPE	VARCHAR(4)	默认为 HARD

80. ALL_DEPENDENCIES

当前用户有权限访问的所有对象的依赖关系。

序号	列	数据类型	说明
1	OWNER	VARCHAR(128)	对象的模式名
2	NAME	VARCHAR(128)	对象名
3	TYPE	VARCHAR(17)	对象类型（被引用对象类型）： TABLE（CLASS,TYPE,DOMAIN,SEQUENCE） VIEW （TABLE,VIEW,PACKAGE,PROCEDURE,SEQUENCE ） PROCEDURE （TABLE,VIEW,PROCEDURE,PACKAGE,CLAS S,TYPE） PACKAGE （TABLE,VIEW,PROCEDURE,PACKAGE,CLAS S,TYPE） TRIGGER （TABLE,VIEW,PROCEDURE,PACKAGE,CLAS S,TYPE） SYNONYM（TABLE,VIEW,PROCEDURE PACKAGE,CLASS,TYPE,SEQUENCE） CLASS（CLASS,TYPE） TYPE（CLASS,TYPE） INDEX（FUNCTION (确定性函数)） FUNCTIO（FUNCTION,TABLE,VIEW,PROCEDURE, PACKAGE,CLASS,TYPE）
4	REFERENCED_OWNER	VARCHAR(128)	被引用对象的模式名
5	REFERENCED_NAME	VARCHAR(128)	被引用对象名
6	REFERENCED_TYPE	VARCHAR(17)	被引用对象类型 TABLE VIEW PROCEDURE SYNONYM CLASS PACKAGE

			TYPE DOMAIN SEQUENCE FUNCTION
7	REFERENCED_LINK_NAME	VARCHAR(1)	默认为 NULL
8	DEPENDENCY_TYPE	VARCHAR(4)	默认为 HARD

81. USER_DEPENDENCIES

查看当前用户所属模式下的对象依赖关系。

序号	列	数据类型	说明
1	OWNER	VARHCAR(128)	对象的模式名
1	NAME	VARCHAR(128)	对象名
2	TYPE	VARCHAR(17)	对象类型（被引用对象类型）: TABLE（CLASS,TYPE,DOMAIN,SEQUENCE） VIEW （TABLE,VIEW,PACKAGE,PROCEDURE,SEQUENCE） PROCEDURE （TABLE,VIEW,PROCEDURE,PACKAGE,CLASS,TYPE） PACKAGE （TABLE,VIEW,PROCEDURE,PACKAGE,CLASS,TYPE） TRIGGER （TABLE,VIEW,PROCEDURE,PACKAGE,CLASS,TYPE） SYNONYM（TABLE,VIEW,PROCEDURE PACKAGE,CLASS,TYPE,SEQUENCE） CLASS（CLASS,TYPE） TYPE（CLASS,TYPE） INDEX（FUNCTION (确定性函数)） FUNCTION(FUNCTION,TABLE,VIEW,PROCEDURE,PACKAGE,CLASS,TYPE）
3	REFERENCED_OWNER	VARCHAR(128)	被引用对象的模式名
4	REFERENCED_NAME	VARCHAR(32767)	被引用对象名
5	REFERENCED_TYPE	VARCHAR（17）	被引用对象类型 TABLE VIEW PROCEDURE SYNONYM CLASS PACKAGE TYPE DOMAIN SEQUENCE FUNCTION
6	REFERENCED_LINK_NAME	VARCHAR（1）	默认为 NULL
7	SCHEMAID	INTEGER	当前模式的 ID 号
8	DEPENDENCY_TYPE	VARCHAR(4)	默认为 HARD

82. USER_IND_PARTITIONS

查看当前用户所属模式的一级分区表索引信息。

序号	列	数据类型	说明
1	INDEX_NAME	VARCHAR(128)	索引名
2	SCH_NAME	VARCHAR(128)	模式名
3	STATUS	VARCHAR(8)	索引状态(USABLE:可用； UNUSABLE:不可用)
4	PARTITION_NAME	VARCHAR(128)	分区名
5	SUBPARTITION_COUNT	INTEGER	二级分区个数
6	HIGH_VALUE	VARBINARY（8188）	分区信息
7	INITIAL_EXTENT	INTEGER	创建索引初始簇数目
8	MIN_EXTENT	INTEGER	索引最小蔟数目
9	LOGGING	VARCHAR(3)	索引变化是否需要记录日志(YES 是，NO 否)

83. USER_IND_SUBPARTITIONS

查看当前用户所属模式的多级分区表索引信息。

序号	列	数据类型	说明
1	INDEX_NAME	VARCHAR(128)	索引名
2	SCH_NAME	VARCHAR(128)	模式名
3	STATUS	VARCHAR(8)	索引状态(USABLE:可用； UNUSABLE:不可用)
4	PARTITION_NAME	VARCHAR(128)	分区名
5	SUBPARTITION_NAME	VARCHAR(128)	多级分区名称
6	SUBPARTITION_POSITION	INTEGER	多级分区序号
6	HIGH_VALUE	VARBINARY（8188）	分区信息
7	INITIAL_EXTENT	INTEGER	创建索引初始簇数目
8	MIN_EXTENT	INTEGER	索引最小蔟数目
9	LOGGING	VARCHAR(3)	索引变化是否需要记录日志(YES 是，NO 否)

84. ALL_PART_KEY_COLUMNS

查看数据库中一级水平分区表的分区列信息。

序号	列	数据类型	说明
1	OWNER	VARCHAR(128)	所属模式名
2	NAME	VARCHAR(128)	所属分区表名
3	OBJECT_TYPE	VARCHAR(5)	对象类型，DM 只支持 TABLE（表）类型
4	COLUMN_NAME	VARCHAR(128)	分区列名
5	COLUMN_POSITION	INTEGER	分区列序号

85. USER_PART_KEY_COLUMNS

查看当前用户所属模式的一级水平分区表的分区列信息。

序号	列	数据类型	说明

1	NAME	VARCHAR(128)	所属分区表名
2	OBJECT_TYPE	VARCHAR(5)	对象类型，DM 只支持 TABLE（表）类型
3	COLUMN_NAME	VARCHAR(128)	分区列名
4	COLUMN_POSITION	INTEGER	分区列序号

86. ALL_SUBPART_KEY_COLUMNS

查看数据库中多级分区表的分区列信息。

序号	列	数据类型	说明
1	OWNER	VARCHAR(128)	所属模式名
2	NAME	VARCHAR(128)	所属分区表名
3	OBJECT_TYPE	VARCHAR(5)	对象类型，DM 只支持 TABLE（表）类型
4	COLUMN_NAME	VARCHAR(128)	分区列名
5	COLUMN_POSITION	INTEGER	分区列序号

87. USER_SUBPART_KEY_COLUMNS

查看当前用户所属模式的多级水平分区表的分区列信息。

序号	列	数据类型	说明
1	NAME	VARCHAR(128)	所属分区表名
2	OBJECT_TYPE	VARCHAR(5)	对象类型，DM 只支持 TABLE（表）类型
3	COLUMN_NAME	VARCHAR(128)	分区列名
4	COLUMN_POSITION	INTEGER	分区列序号

88. ALL_TAB_COLS

查看当前用户可见的表、视图的列信息。

序号	列	数据类型	说明
1	OWNER	VARCHAR(128)	用户名
2	TABLE_NAME	VARCHAR(128)	表、视图或聚簇的名字
3	COLUMN_NAME	VARCHAR(128)	列名
4	DATA_TYPE	VARCHAR(128)	列类型名
5	DATA_TYPE_MOD	VARCHAR(3)	DM 不支持的属性，取值为 NULL
6	DATA_TYPE_OWNER	VARCHAR(128)	列数据类型的所有者
7	DATA_LENGTH	NUMBER	列长度。单位：字节
8	DATA_PRECISION	NUMBER	精度
9	DATA_SCALE	NUMBER	刻度
10	NULLABLE	VARCHAR(1)	是否允许为 NULL，Y 是，N 否
11	COLUMN_ID	NUMBER	列序号（1 开始）
12	DEFAULT_LENGTH	NUMBER	默认值长度
13	DATA_DEFAULT	TEXT	默认值
14	NUM_DISTINCT	NUMBER	不同值的个数
15	LOW_VALUE	VARBINARY(32)	列最小值
16	HIGH_VALUE	VARBINARY(32)	列最大值
17	DENSITY	NUMBER	列的密度
18	NUM_NULLS	NUMBER	NULL 值的个数

19	NUM_BUCKETS	NUMBER	直方图槽数
20	LAST_ANALYZED	DATETIME(6)	最后一次分析的时间
21	SAMPLE_SIZE	NUMBER	取样大小
22	CHARACTER_SET_NAME	VARCHAR(44)	字符集名字：CHAR_CS、NCHAR_CS
23	CHAR_COL_DECL_LENGTH	NUMBER	列数据字符类型长度
24	GLOBAL_STATS	VARCHAR(3)	对于分区表，统计信息是作为一个整体搜集(YES)，还是通过分区和子分区来估算(NO)
25	USER_STATS	VARCHAR(3)	统计信息生成是否通过手动，YES 是，NO 否
26	AVG_COL_LEN	NUMBER	列数据的平均长度，单位：字节
27	CHAR_LENGTH	NUMBER	显示时的字符长度，对下述类型有效：CHAR、VARCHAR、NCHAR、NVARCHAR
28	CHAR_USED	VARCHAR(1)	列是字节长度还是字符长度 (B) OR (C)，OR (NULL)： • CHAR • VARCHAR • NCHAR • NVARCHAR
29	V80_FMT_IMAGE	VARCHAR(3)	列数据是否为 8.0 版本的 IMAGE 格式，YES 是，NO 否
30	DATA_UPGRADED	VARCHAR(3)	列数据是否已经更新为最新的类型版本格式，YES 是，NO 否
31	HIDDEN_COLUMN	VARCHAR(3)	是否是一个隐藏列，YES 是，NO 否
32	VIRTUAL_COLUMN	VARCHAR(3)	是否是一个伪列，YES 是，NO 否
33	SEGMENT_COLUMN_ID	NUMBER	列在段中的序号
34	INTERNAL_COLUMN_ID	NUMBER	列的内部序号
35	HISTOGRAM	VARCHAR(15)	直方图类型，DM 不支持，返回 NULL
36	QUALIFIED_COL_NAME	VARCHAR(4000)	合格的列名

89. DBA_TAB_COLS

查看数据库中所有表、视图的列信息。与 ALL_TAB_COLS 的列相同

90. DBA_PART_KEY_COLUMNS

查看数据库中所有一级水平分区表的分区列信息。

序号	列	数据类型	说明
1	OWNER	VARCHAR(128)	所属模式名
2	NAME	VARCHAR(128)	所属分区表名
3	OBJECT_TYPE	VARCHAR(5)	对象类型，DM 只支持 TABLE（表）类型
4	COLUMN_NAME	VARCHAR(128)	分区列名
5	COLUMN_POSITION	INTEGER	分区列序号

91. ALL_SOURCE_AE

当前用户可见的存储对象的定义信息

序号	列	数据类型	说明
1	OWNER	VARCHAR(128)	所属模式名

2	NAME	VARCHAR(128)	所属分区表名
3	TYPE	VARCHAR(12)	对象类型
4	LINE	INTEGER	保留字段，置 0
5	TEXT	VARCHAR(128)	对象定义
6	EDITION_NAME	VARCHAR(12)	保留字段，置空

92. DBA_SOURCE_AE

数据库中所有存储对象的定义信息

序号	列	数据类型	说明
1	OWNER	VARCHAR(128)	所属模式名
2	NAME	VARCHAR(128)	所属分区表名
3	TYPE	VARCHAR(12)	对象类型
4	LINE	INTEGER	保留字段，置 0
5	TEXT	VARCHAR(128)	对象定义
6	EDITION_NAME	VARCHAR(12)	保留字段，置空

93. USER_SOURCE_AE

当前用户下的存储对象的定义信息

序号	列	数据类型	说明
1	NAME	VARCHAR(128)	所属分区表名
2	TYPE	VARCHAR(12)	对象类型
3	LINE	INTEGER	保留字段，置 0
4	TEXT	VARCHAR(128)	对象定义
5	EDITION_NAME	VARCHAR(12)	保留字段，置空

94. ALL_DIRECTORIES

当前用户可以访问的所有目录信息

序号	列	数据类型	说明
1	OWNER	VARCHAR(128)	目录所有者
2	DIRECTORY_NAME	VARCHAR(128	目录名
3	DIRECTORY_PATH	VARCHAR(512)	目录路径

95. DBA_DIRECTORIES

当前系统中所有目录的信息，只有拥有 DBA 权限的用户可以查看，结构同 ALL_DIRECTORIES。

96. USER_TYPES

当前用户下的类类型与自定义类型信息。

序号	列	数据类型	说明
1	TYPE_NAME	VARCHAR(128)	类型名
2	TYPE_OID	INTEGER	类型 ID
3	TYPECODE	VARCHAR(10)	类型标识
4	ATTRIBUTES	VARCHAR(1)	类型的属性个数，目前都为 0
5	METHODS	VARCHAR(1)	类型的方法个数，目前都为 0
6	PREDEFINED	VARCHAR(2)	类是否是 PREDEFINED，目前都为 NO
7	INCOMPLETE	VARCHAR(2)	类是否有编译警告，目前都为 NO

8	FINAL	VARCHAR(3)	类是否为 FINAL
9	INSTANTIABLE	VARCHAR(3)	类是否为 INSTANTIABLE
10	SUPERTYPE_OWNER	VARCHAR(128)	父类的模式名，目前都为 NULL
11	SUPERTYPE_NAME	VARCHAR(128)	父类名，目前都为 NULL
12	LOCAL_ATTRIBUTES	VARCHAR(1)	子类本地变量，目前都为 NULL
13	LOCAL_METHODS	VARCHAR(1)	子类本地方法，目前都为 NULL
14	TYPEID	INTEGER	类 ID

附录 4 执行计划操作符

操作符名称	说明
AAGR2	简单聚集；如果没有分组（group by）， 则总的就一个组，直接计算聚集函数
AFUN	分析函数计算
ASCN	数组当作表来扫描
ASSERT	约束检查
BLKUP2	定位查找
BMAND	位图索引的与运算
BMCNT	位图索引的行数计算
BMCVT	位图索引的 ROWID 转换
BMMG	位图索引归并
BMOR	位图索引的或运算
BMSEK	位图索引的范围查找
CONST VALUE LIST	常量列表
CONSTC	用于索引跳跃扫描
CSCN2	聚集索引扫描
CSEK2	聚集索引数据定位
CTNS	用于实现全文索引的 CONTAINS
DELETE	删除数据
DELETE_REMOTE	DBLINK 删除操作
DISTINCT	去重
DSCN	动态视图表扫描
ESCN	外部表扫描
EXCEPT	集合的差运算，且取差集后删除重复项
EXCEPT ALL	集合的差运算，且取差集后不删除重复项
FAGR2	快速聚集，如果没有 where 条件，且取 count(*)，或者基于索引的 MAX/MIN 值，则可以快速取得集函数的值
FILL BTR	填充 B 树
FTTS	MPP\LPQ 下，对临时表的优化
GSEK	空间索引查询
HAGR2	HASH 分组，并计算聚集函数
HASH FULL JOIN2	HASH 全外连接
HASH LEFT JOIN2	HASH 左外连接
HASH LEFT SEMI JOIN2	HASH 左半连接
HASH LEFT SEMI MULTIPLE JOIN	多列 NOT IN
HASH RIGHT JOIN2	HASH 右外连接
HASH RIGHT SEMI JOIN2	HASH 右半连接
HASH RIGHT SEMI JOIN32	用于 OP SOME/ANY/ALL 的 HASH 右半连接
HASH2 INNER JOIN	HASH 内连接

HEAP TABLE	临时结果表
HEAP TABLE SCAN	临时结果表扫描
HFDEL	删除 HUGE 表数据
HFINS2	HUGE 表插入记录
HFINS3	MPP 下优化的 HUGE 表插入记录
HFINS4	非 MPP 下，针对 HUGE 水平分区主表的插入优化，需要参数 HFINS_PARALLEL_FLAG=2
HFLKUP	根据 ROWID 检索 HUGE 表数据
HFS UPDATE	更新 HUGE 表数据
HFSCN	HUGE 表的逐行扫描
HFSEK	根据 KEY 检索 HUGE 表数据
HIERARCHICAL QUERY	层次查询
INDEX JOIN LEFT JOIN2	索引左连接
INDEX JOIN SEMI JOIN2	索引半连接
INSERT	插入记录
INSERT3	MPP 下，查询插入优化处理
INSERT_LIST	堆表插入
INSERT_REMOTE	DBLINK 插入操作
INTERSECT	集合的交运算，且取交集后删除重复项
INTERSECT ALL	集合的交运算，且取交集后不删除重复项
LOCAL BROADCAST	本地并行模式下，消息广播到各线程，包含必要的聚集函数合并计算
LOCAL DISTRIBUTE	本地并行模式下，消息各线程的相互重分发
LOCAL GATHER	本地并行模式下，消息收集到主线程
LOCAL SCATTER	本地并行模式下，主线程向各从线程广播消息
LOCK TID	上锁
LPQ COLLECT	并行下数据收集处理，代替 LPQ GATHE
LSET	DBLINK 查询结果集
MERGE INNER JOIN3	归并内连接
MERGE SEMI JOIN3	归并半连接
MPP BROADCAST	MPP 模式下，消息广播到各站点，包含必要的聚集函数合并计算
MPP COLLECT	用于替换顶层 MPP GATHER，除了收集数据到主节点，还增加主从节点间的同步执行功能，防止从节点不断发送数据到主节点造成邮件堆积
MPP DISTRIBUTE	MPP 模式下，消息各站点的相互重分发
MPP GATHER	MPP 模式下，消息收集到主站点
MPP SCATTER	MPP 模式下，主站点向各从站点广播消息
MSYNC	MPP 下数据同步处理
MVCC CHECK	多版本检查
NCUR2	游标操作
NEST LOOP FULL JOIN2	嵌套循环全外连接
NEST LOOP INDEX JOIN2	索引内连接
NEST LOOP INNER JOIN2	嵌套循环内连接
NEST LOOP LEFT JOIN2	嵌套循环左外连接

NEST LOOP SEMI JOIN2	嵌套循环半连接
NTTS2	临时表，临时存放数据
NSET2	结果集(result set)收集，一般是查询计划的顶层节点
PARALLEL	控制水平分区子表的扫描
PIPE2	管道；先做一遍右儿子，然后执行左儿子，并把左儿子的数据向上送，直到左儿子不再有数据
PRJT2	关系的“投影”(project)运算，用于选择表达式项的计算
PSCN	批量参数当作表来扫描
REMOTE SCAN	远程表扫描
RN	实现 ROWNUM
RNSK	实现 ROWNUM
SAGR2	如果输入流是有序的，则使用流分组，并计算聚集函数
SELECT INTO2	查询插入
SET TRANSACTION	事务操作(START 除外)
SLCT2	关系的“选择”（select）运算，用于查询条件的过滤
SORT2	排序
SORT3	排序
SPOOL2	临时表；和 TEMP TABLE SPOOL(NTTS)不同的是，它的数据集不向父亲节点传送，而是被编号，用编号和 KEY 来定位访问；而 TEMP TABLE SPOOL 的数据，主动传递给父亲节点
SSCN	直接使用二级索引进行扫描
SSEK2	二级索引数据定位
START TRANSACTION	启动会话
STAT	统计信息计算
TOPN2	取前 N 条记录
UFLT	处理 UPDATE FROM 子句
UNION	UNION 计算
UNION ALL	UNION ALL 运算
UNION ALL(MERGE)	UNION ALL 运算（使用归并）
UNION FOR OR2	OR 过滤的 UNION 计算
UPDATE	更新数据
UPDATE_REMOTE	DBLINK 更新操作

附录 5 数据复制的系统表

1) 复制组表

```
CREATE TABLE SYSREP.GROUPS (
    NAME        VARCHAR(128),       // 复制组名
    ID          INT,                    // 复制组 ID
    DESC$       VARCHAR(1000),      // 描述
    CLUSTER PRIMARY (NAME)
);
```

2) 复制节点实例表

```
CREATE TABLE SYSREP.INSTANCES(
    INST_NAME       VARCHAR(128),   // 复制节点实例名
    GRP_ID          INT,                // 复制组 ID
    INST_ID         INT,                // 实例在复制组中编号
    FAULT_TIMEOUT       INT,            // 故障超时处理值，以秒为单位，0 为立即超
时
    VALID_FLAG      CHAR(1),            // 节点系统状态
    FAULT_TIME      DATETIME,       // 节点故障开始时间
    NET_VALID_FLAG  CHAR(1),            // 网络状态
    NET_FAULT_TIME  DATETIME,       // 网络故障开始时间
    CLUSTER PRIMAY KEY(GRP_ID, INST_NAME)
);
```

3) 复制关系表

```
CREATE TABLE SYSREP.REPLICATIONS (
    REP_NAME        VARCHAR(128),   // 复制名
    GRP_ID          INT,                // 复制组 ID
    REP_ID          INT,                // 复制 ID，全局唯一
    MINST_ID            INT,                // 主节点实例编号
    SINST_ID            INT,                // 从节点实例编号
    ARCH_PATH       VARCHAR(256),   // 主节点归档日志路径
    FAULT_TIMEOUT   INT,                // 故障超时处理值，以秒为单位，0 为立即超
时
    VALID_FLAG      CHAR(1),            // 复制关系状态
    FAULT_TIME      DATETIME,       // 故障开始时间
    DESC$           VARCHAR(1000),  // 复制描述
    CLUSTER PRIMARY KEY(GRP_ID, REP_NAME)
);
```

4) 复制映射表

```
CREATE TABLE SYSREP.TABMAPS(
    REP_ID         INT,             //复制 ID
    MSCH_NAME      VARCHAR(128),    //主表模式名
    MTAB_NAME      VARCHAR(128),    //主表名
    MSCH_ID       INT,             //主表模式 ID
    MTAB_ID        INT,             //主表 ID
    SSCH_NAME      VARCHAR(128),    //从表模式名
    STAB_NAME      VARCHAR(128),    //从表名
    SSCH_ID        INT,                //从表模式 ID
    STAB_ID        INT,             //从表 ID
    READONLY_MODE INT,              //映射模式 1:只读模式，0：非只读模式
    CLUSTER PRIMARY KEY(REP_ID, MTAB_ID, STAB_ID)
);
```

5) 复制故障历史表

```
CREATE TABLE SYSREP.REP_FAULT_HISTORY(
    GRP_NAME        VARCHAR(128),   // 复制组
```

```
        OBJ_NAME        VARCHAR(128),   // 故障节点或关系
        FAULT_TYPE      VARCHAR(128),   // 故障类型描述
        START_TIME      DATETIME,       // 故障开始时间
        END_TIME        DATETIME        // 故障结束时间
    );
```

6) 复制冲突历史表

```
    CREATE TABLE SYSREP.RPS_CONFLICTS
    (
        REP_ID          INT,            // 对应复制号
        INST_ID         INT,            // 产生冲突的节点编号
        TABLE_ID            INT,            // 冲突表 ID
        OCC_TIME        DATETIME,   // 冲突产生时间
        KEY_DATA        VARCHAR(8000), //冲突数据的 PK 值，如包含多个 KEY 值，则以逗号分隔。如键值超长则截断
    );
```

7) 复制定时器表

```
    CREATE TABLE SYSREP.TIMERS (
        REP_NAME            VARCHAR(128),       // 复制名
        TYPE$               INT,            //  定 时 类 型 ， 取 值 可 参 见 SP_RPS_CREATE_TIMER
        FERQ_INTERVAL       INT,            // 间隔天数
        FREQ_SUB_INTERVAL   INT,            // 间隔的月/周（调度类型决定）数
        FREQ_MINUTE_INTERVAL    INT,        // 间隔的分钟数
        START_TIME          TIME,       // 开始时间
        END_TIME            TIME,       // 结束时间
        DURING_START_DATE   DATETIME    // 开始时间点
        DURING_END_DATE DATETIME,   // 结束时间点
        DESC$               VARCHAR(1000),      // 定时器描述
        CLUSTER PRIMARY KEY(REP_NAME)
    );
```

附录 6 DM 技术支持

如果您在安装或使用 DM 及其相应产品时出现了问题，请首先访问我们的 Web 站点 http://www.dameng.com/。在此站点我们收集整理了安装使用过程中一些常见问题的解决办法，相信会对您有所帮助。

您也可以通过以下途径与我们联系，我们的技术支持工程师会为您提供服务。

达梦数据库（武汉）有限公司

地址：武汉市关山一路特 1 号光谷软件园 C6 栋 5 层

邮编：430073

电话：(+86)027-87588000

传真：(+86)027-87588000-8039

达梦数据库（北京）有限公司

地址：北京市海淀区北三环西路 48 号数码大厦 B 座 905

邮编：100086

电话：(+86)010-51727900

传真：(+86)010-51727983

达梦数据库（上海）有限公司

地址：上海市闸北区江场三路 28 号 301 室

邮编：200436

电话：(+86)021-33932716

传真：(+86)021-33932718

地址：上海市浦东张江高科技园区博霞路 50 号 403 室

邮编：201203

电话：（+86）021-33932717

传真：（+86）021-33932717-801

达梦数据库（广州）有限公司

地址：广州市荔湾区中山七路 330 号荔湾留学生科技园 703 房

邮编：510145

电话：(+86)020-38371832

传真：(+86)020-38371832

达梦数据库（海南）有限公司

地址：海南省海口市玉沙路富豪花园 B 座 1602 室

邮编：570125

电话：(+86)0898-68533029

传真：(+86)0898-68531910

达梦数据库（南宁）办事处

地址：广西省南宁市科园东五路四号南宁软件园五楼

邮编：530003

电话：（+86）0771-2184078

传真：（+86）0771-2184080

达梦数据库（合肥）办事处

地址：合肥市包河区马鞍山路金帝国际城 7 栋 3 单元 706 室

邮编：230022

电话：（+86）0551-3711086

达梦数据库（深圳）办事处

地址：深圳市福田区皇岗路高科利大厦 A 栋 24E 邮编：518033

电话：0755-83658909

传真：0755-83658909

技术服务：

电话：400-991-6599

邮箱：dmtech@dameng.com